



pvx

User Guide

Colby Leider

Time stretch, pitch shift, automation, augmentation, and reference workflows

Command-line practice, quality tuning, multistage rendering, and machine-learning pipelines

pvx repository handbook

Compiled from project documentation and repository source

July 29, 2026

About This Guide

This book is a curated, book-length guide to **pvx** built from the repository’s user-facing documentation, examples, and command reference. It is meant to be read in layers: a practical first pass for musicians and engineers who want useful results quickly, a deeper pass for readers who want to understand the phase-vocoder model, and a reference pass for people building repeatable workflows, augmentation pipelines, or release processes around the tool.

The supported public surface for the current alpha remains intentionally narrow: **pvx**, **pvxvoc**, **pvxfreeze**, **pvxwarp**, **pvxformant**, **pvxfilter**, **pvxretune**, and **pvxanalysis**. Where the appendices document broader flags and exploratory surfaces, they should be read as inventory rather than a stronger compatibility promise.

Preface

[TBA]

This preface marks the beginning of a longer editorial layer for the pvx project. The technical reference can tell a reader what a flag accepts; it cannot by itself answer the question that arises in a session: what should I listen for, what should I change first, and when is an artifact useful rather than merely accidental? This book exists to put those questions beside the commands.

Phase-vocoder work rewards both curiosity and restraint. A small time change can be nearly invisible. An extreme stretch can make a cymbal bloom into weather. A voice can be moved through an impossible register while still carrying a recognisable vowel. The same transform that yields a clean utility render can become an instrument when it is pushed past its transparent operating range. pvx is built for that continuum, from sensible production repair to deliberate spectral invention.

The guide therefore has two jobs. One is pragmatic: it should help someone reach a dependable result without treating digital signal processing as a private language. The other is historical and musical: it should make clear that these tools come from a long line of experiments in speech, tape, synthesis, and computer music. The algorithms are not neutral buttons. They embody choices about continuity, frequency resolution, timing, and what a listener is willing to hear as a coherent sound.

The present volume is intentionally a living document. Its public contract follows the alpha surface described in the release material. Some pages record exploratory capabilities because understanding the surrounding landscape is useful, but a printed description is not a promise that every experiment will remain stable. The supported commands are the ones to carry into a session, a batch pipeline, or a release workflow.

The chapters can be read in order, but they do not demand it. A producer can start with the guided workflows and return to the mathematics only when a decision needs explanation. A researcher can begin with the first chapter and then move directly to architecture, windows, and the API material. A person building a dataset pipeline can treat the cookbook and flag reference as a working bench. The same book should be useful at three speeds: a quick answer during a render, a careful study at a desk, and a deeper rereading after one has heard the artifacts for oneself.

The remaining preface will eventually include a fuller account of the project’s origin, contributors, and the listening practices that informed its defaults. Until then, [TBA] is kept here deliberately: it is an invitation to distinguish the finished technical apparatus from the still-forming human story.

Introduction

[TBA]

pvx is a command-line environment for changing how recorded sound occupies time, pitch, spectrum, and space. Its core is the phase vocoder, but the experience of using it is not reducible to one algorithm. The command line holds choices about analysis windows, phase propagation, transient preservation, automation curves, channels, format, loudness, and output discipline. This introduction offers a map of those choices before the later chapters name every option.

0.1 The central promise

The ordinary recording presents pitch and duration as linked facts. Play a tape faster and the event becomes shorter and higher. Play it slower and it becomes longer and lower. Digital analysis makes it possible to loosen that link. Time can be stretched while pitch remains broadly stable; pitch can move while duration remains broadly stable. The word *broadly* matters. Audio is not a set of independent knobs. A transform must decide what continuity to preserve, and every decision leaves a trace.

pvx gives that trace a place in the workflow. Some material wants transparency: dialogue, a solo instrument, a finished mix, or a restoration pass. Other material wants character: a frozen chord, a voice that turns granular at the edges, or an attack that opens into a spectral cloud. Both are valid uses. The practical skill is to name the goal before selecting the settings.

0.2 A workflow begins with listening

Before choosing an FFT size or a phase-locking mode, listen to the source as if it were a score. Ask where the important attacks are, whether the sound is dominated by pitched partials or noise, whether the stereo image carries musical information, and how much change the listener is expected to accept. A drum loop and a sustained organ chord may both be called audio, but they make almost opposite demands on a transform. The loop asks for sharp onset identity; the chord asks for stable harmonic motion.

One useful first pass is to render only a short region. Keep the source untouched. Make a conservative version and an ambitious version. Compare them at the level where the finished work will be heard. Headphones reveal phase movement and high-frequency fizz. Speakers reveal whether a transient still has weight. A mono fold-down reveals whether a stereo treatment has undermined the image. This is not extra ceremony. It is how a parameter becomes audible rather than theoretical.

0.3 The four levers

Most pvx work can be understood through four levers:

1. **Time.** The stretch ratio changes the output duration. It also changes the distance between reconstructed analysis frames, which is why large ratios make phase and transient decisions more audible.
2. **Pitch.** Semitone and cent controls map musical intervals to frequency ratios. Pitch moves are often paired with formant treatment when a voice needs to retain its apparent vocal-tract identity.
3. **Continuity.** Window length, hop size, phase locking, and transient modes determine how the analysis observes motion and how the resynthesis joins it together.
4. **Change over time.** Automation lets a parameter follow a curve. Instead of treating a file as a single static setting, a render can grow, bend, open, narrow, or shift across its duration.

The later reference material separates these controls because the command line needs precise names. While learning, it is better to remember their relationship. Time and pitch describe the intended transformation. Continuity describes the cost of making it sound whole. Automation describes how the transformation moves through the piece.

0.4 Transparent, expressive, and extreme rendering

There are three useful attitudes toward phase-vocoder output. Transparent rendering aims to leave the listener unsure that anything happened except the requested change. It favours moderate ratios, appropriate windows, protected transients, and careful comparison against the source. Expressive rendering lets a little process become part of the timbre. It can work beautifully on pads, sampled voices, guitars, and ambient material. Extreme rendering makes the artifacts themselves the subject: the granular haze, the stretched breath, the unstable pitch edge, and the strange persistence of a single moment.

None of these attitudes is inherently more advanced than another. The mistake is to treat an extreme result as failed transparency, or a clean result as timid creativity. The command is only half the gesture. The context in which it is heard completes it.

0.5 The stable surface and the experimental edge

For the current alpha, this guide treats `pvx`, `pvxvoc`, `pvxfreeze`, `pvxwarp`, `pvxformant`, `pvxfilter`, `pvxre-tune`, and `pvxanalysis` as the supported command surface. They are the commands to learn first and the commands around which fixtures and release checks are organised. Other names may appear in source, examples, or larger inventories. Those are useful for orientation, but they do not widen the compatibility promise.

This distinction is a kindness to future work. It lets the project explore without pretending every experiment is ready to be depended upon. It also gives readers a clear place to stand. Learn the supported tools deeply. Use the exploratory material with curiosity, a disposable output directory, and a willingness to verify the result.

0.6 Reading and working with this book

Each major chapter has a different role. The first chapter establishes the vocabulary and history. Getting Started gives a dependable first render. The Quality Guide teaches diagnostic listening and parameter tuning. Mathematical Foundations explains why windows, hops, and phase updates behave as they do. The cookbook chapters turn recurring musical and production tasks into named recipes. The appendix is for the moment when a descriptive explanation is no longer enough and an exact flag is needed.

The document is also designed to be used with a terminal open. Read a small section, run a small command, and listen. Preserve the command line that produced a result you like. Keep input and output names unambiguous. For longer work, use checkpoints, manifests, or explicit notes about the settings. The best render is hard to value if it cannot be reproduced.

The introduction is deliberately fuller than a command synopsis, and it will grow with the project. The [TBA] marker remains because this is still an alpha handbook. The technical route is present now; the final account of its audience, practice, and evolution is still being written.

Reading Paths

If you are new to `pvx`, read this book in this order:

1. *Getting Started with pvx*
2. *pvx Quality Guide*
3. *pvx Example Cookbook*
4. The appendices when you need exact flags or deployment details

If you are using `pvx` as a production or research tool, add:

1. *Mathematical Foundations*
2. *pvx Application Programming Interface (API) Overview*
3. *ML Integration*
4. *CLI Flags Reference*

List of Symbols

N	FFT size, in samples
H_a	Analysis hop, in samples
H_s	Synthesis hop, in samples
t	Frame index
k	Frequency-bin index
$X_t[k]$	Complex spectrum for frame t , bin k
$\phi_t[k]$	Observed phase for frame t , bin k
$\hat{\phi}_t[k]$	Reconstructed output phase

Contents

About This Guide	iii
Preface	v
Introduction	vii
0.1 The central promise	vii
0.2 A workflow begins with listening	vii
0.3 The four levers	vii
0.4 Transparent, expressive, and extreme rendering	viii
0.5 The stable surface and the experimental edge	viii
0.6 Reading and working with this book	viii
Reading Paths	ix
List of Symbols	xi
 I Foundations	 1
1 What Is a Phase Vocoder?	3
1.1 A quick mental model	3
1.2 The historical thread	4
1.2.1 Speech analysis before digital audio	4
1.2.2 Tape, television, and the wish to uncouple time from pitch	4
1.2.3 Flanagan’s digital formulation	5
1.2.4 From research technique to musical tool	7
1.2.5 A chronology of ideas, not merely machines	7
1.2.6 Flanagan and Golden: phase as transmitted information	7
1.2.7 Portnoff: the FFT makes the method practical	8
1.2.8 Computer music and the widening of purpose	8
1.2.9 Mark Dolson and the tutorial synthesis	9
1.2.10 Horizontal coherence and vertical coherence	9
1.2.11 Jean Laroche and Mark Dolson: explaining phasiness	10
1.2.12 Pitch shifting, harmonising, and spectral translation	11
1.2.13 Transients expose the limits of stationarity	12
1.2.14 Stereo and multichannel coherence	12
1.3 A deeper history of spectral time	13
1.3.1 Harmonic analysis before electronic sound	13
1.3.2 Telegraphy, telephony, and the filter-bank imagination	14
1.3.3 Homer Dudley, the vocoder, and the Voder	14
1.3.4 Secure speech and large-scale vocoder engineering	15
1.3.5 Tape music and independent control by segmentation	15
1.3.6 Sampling, digital audio, and the FFT threshold	16
1.3.7 Flanagan and Golden in the Bell Labs setting	16

1.3.8	Portnoff and the practical digital formulation	17
1.3.9	James A. Moorer and the computer-music turn	17
1.3.10	UC San Diego, CARL, and a software culture	17
1.3.11	Mark Dolson: research, software, and pedagogy	18
1.3.12	Sinusoidal modeling and a neighboring lineage	18
1.3.13	IRCAM and the institutionalization of spectral transformation	19
1.3.14	Jonathan Harvey and spectral ambiguity	19
1.3.15	Roger Reynolds and the genealogy of Transfigured Wind	20
1.3.16	Trevor Wishart and sound morphing	20
1.3.17	JoAnn Kuchera-Morin and expanded duration	20
1.3.18	Puckette, phase locking, and real-time environments	21
1.3.19	Laroche and Dolson: from artifact to explanatory model	21
1.3.20	Transients become a separate research object	21
1.3.21	WSOLA, PSOLA, granular methods, and productive rivalry	22
1.3.22	Spectral envelopes, formants, and the source-filter return	22
1.3.23	Noise, residuals, and the limits of sinusoidal phase	23
1.3.24	Stereo, spatial hearing, and multichannel history	23
1.3.25	Open tools, standards, and software migration	23
1.3.26	From offline render to interactive instrument	24
1.3.27	Adaptive and nonstationary time-frequency analysis	24
1.3.28	Machine learning enters the surrounding workflow	24
1.3.29	A historiography of claims and cautions	25
1.3.30	Reading the foundational papers as historical documents	25
1.3.31	The hidden history of computation time	26
1.3.32	A fuller genealogy of phase-vocoder software	27
1.3.33	The analysis file as a new kind of score	29
1.3.34	Compositional histories beyond the canonical examples	29
1.3.35	Artifact history as listening history	30
1.3.36	Teaching, diagrams, and the public life of the algorithm	32
1.3.37	Bell Laboratories as a long technical precondition	32
1.3.38	Stanford, CCRMA, and model-based sound	33
1.3.39	UCSD, CARL, and the command-line studio in detail	33
1.3.40	IRCAM as a meeting of research and commission	34
1.3.41	Speech as the first difficult material	35
1.3.42	Sustained instrumental sound and the discovery of coherence	35
1.3.43	Percussion and the historical challenge of the onset	36
1.3.44	Noise, ambience, and environmental recordings	36
1.3.45	Extreme duration and the change of listening scale	36
1.3.46	Preservation, reconstruction, and obsolete systems	37
1.3.47	A decade-by-decade change in the imagined user	37
1.3.48	Annotated chronology	39
1.3.49	Reading the history through software design	41
1.3.50	Source notes for the expanded history	41
1.3.51	What the historical work leaves us	42
1.4	What the transform sees	42
1.5	A small comparison	43
1.6	How pvx puts the idea to work	43

II Orientation and Core Workflow 45

2 Getting Started with pvx 47

2.1	Quick Setup (Install + PATH)	47
2.2	Launch-Ready Helper Commands	47
2.3	Running Any Command with uv	48
2.4	First 3 Commands to Run	48
2.5	What Problem pvx Solves	48
2.6	Analog Tape Methods for Pitch/Time Shifting	49
2.7	Basic DSP Terms (Plain Language)	49
2.8	Supported File Types	49
2.9	Stretch vs Pitch Shift	49
2.10	Denoising with Phase-Consistent STFT Processing	50
2.11	Time-Varying Parameter Control (CSV/JSON)	50
2.12	Visual Mental Model of STFT	52
2.13	Example Audio Scenarios	52
2.13.1	Use-Case Matrix (Where to start quickly)	52
2.14	Step-by-Step Walkthrough (One Small WAV)	53
2.14.1	Step A: Baseline stretch	53
2.14.2	Step B: Add transient protection	54
2.14.3	Step C: Pitch shift with formant preservation	54
2.14.4	Step D: Auto profile planning	54
2.15	What Outputs Should Sound Like	54
2.16	New Quality Modes (Beginner Version)	54
2.16.1	Hybrid transient mode (recommended on speech/drums)	54
2.16.2	Stereo coherence lock (recommended for wide stereo sources)	54
2.17	Intent Presets	55
2.18	Simpler One-Line Pipelines	55
2.19	Estimate Stretch Budget Before Extreme Jobs	55
2.20	Feature Tracking for Sidechain Control	55
2.21	Output Policy Controls	56
2.22	Next Steps	56
2.23	Extra Beginner Recipes (Quick Wins)	56
3	pvx Quality Guide	59
3.1	Fast Starting Presets	59
3.2	Artifact -> Fix Table	59
3.3	Practical Recipes	60
3.3.1	Speech stretch with transient protection	60
3.3.2	Drum-safe stretch	60
3.3.3	Stereo coherence lock	60
3.3.4	Denoise without musical-noise artifacts	60
3.4	Parameter Ranges That Usually Work	60
3.5	Debug Workflow	61
4	Supported Surface	63
4.1	Stable in 0.1.x	63
4.2	Beta in 0.1.x	63
4.3	Experimental in 0.1.x	64
4.4	Compatibility Shims	64
4.5	Docs Guide	64
5	Supported File Types	65
5.1	I/O Behavior by Category	65

5.2	Full Audio Container List (Current Build)	65
5.3	Verify on Your Machine	66
III Theory and Internal Model		67
6	pvx Mathematical Foundations	69
6.1	Notation	69
6.2	STFT Analysis and Synthesis	69
6.3	Transform Backend Families and Tradeoffs	69
6.3.1	Sample Transform Use Cases	71
6.4	Phase-Vocoder Frequency and Phase Update	71
6.5	Time Stretch and Pitch Ratio	71
6.6	Dynamic Control Interpolation (CSV/JSON)	72
6.7	Microtonal Retune Mapping	72
6.8	Loudness and Dynamics	73
6.9	Spatial Coherence and Channel Alignment	73
6.10	Function Graph Gallery	73
7	pvx Architecture	75
7.1	Runtime and CLI Flow	75
7.2	Algorithm Registry and Dispatch	75
7.3	Documentation Build Graph	75
7.4	CI + Pages	75
8	pvx Window Reference	77
8.1	Notation	77
8.2	Formula Families	77
8.2.1	W0	77
8.2.2	W1	77
8.2.3	W2	77
8.2.4	W3	77
8.2.5	W4	78
8.2.6	W5	78
8.2.7	W6	78
8.2.8	W7	78
8.2.9	W8	78
8.2.10	W9	78
8.2.11	W10	78
8.2.12	W11	78
8.2.13	W12	79
8.2.14	W13	79
8.2.15	W14	79
8.2.16	W15	79
8.2.17	W16	79
8.2.18	W17	79
8.2.19	W18	79
8.3	Quantitative Metrics	79
8.4	Comparative Window Summary	80
8.5	Complete Window Atlas	84
8.5.1	hann	85
8.5.2	hamming	89

8.5.3	blackman	93
8.5.4	blackmanharris	97
8.5.5	nuttall	101
8.5.6	flattop	105
8.5.7	blackman nuttall	109
8.5.8	exact blackman	113
8.5.9	sine	117
8.5.10	bartlett	121
8.5.11	boxcar	125
8.5.12	triangular	129
8.5.13	bartlett hamm	133
8.5.14	tukey	137
8.5.15	tukey 0p1	141
8.5.16	tukey 0p25	145
8.5.17	tukey 0p75	149
8.5.18	tukey 0p9	153
8.5.19	parzen	157
8.5.20	lanczos	161
8.5.21	welch	165
8.5.22	gaussian 0p25	169
8.5.23	gaussian 0p35	173
8.5.24	gaussian 0p45	177
8.5.25	gaussian 0p55	181
8.5.26	gaussian 0p65	185
8.5.27	general gaussian 1p5 0p35	189
8.5.28	general gaussian 2p0 0p35	193
8.5.29	general gaussian 3p0 0p35	197
8.5.30	general gaussian 4p0 0p35	201
8.5.31	exponential 0p25	205
8.5.32	exponential 0p5	209
8.5.33	exponential 1p0	213
8.5.34	cauchy 0p5	217
8.5.35	cauchy 1p0	221
8.5.36	cauchy 2p0	225
8.5.37	cosine power 2	229
8.5.38	cosine power 3	233
8.5.39	cosine power 4	237
8.5.40	hann poisson 0p5	241
8.5.41	hann poisson 1p0	245
8.5.42	hann poisson 2p0	249
8.5.43	general hamming 0p50	253
8.5.44	general hamming 0p60	257
8.5.45	general hamming 0p70	261
8.5.46	general hamming 0p80	265
8.5.47	bohman	269
8.5.48	cosine	273
8.5.49	kaiser	277
8.5.50	rect	281
9	Lessons from Paul Koonce's PVC for pvx	285
9.1	What PVC Got Right	285
9.2	Composable command-line tools	285

9.3	Dynamic control from external files	285
9.4	Terminal-first discoverability	285
9.5	Explicit defaults and transparent controls	285
9.6	Script-based workflow reuse	286
9.7	Where pvx Can Improve Further (PVC-Inspired)	286
9.8	Suggested Ongoing Checklist	286
10	PVC-to-pvx Parity Matrix	287
10.1	Scope	287
10.2	Capability Matrix	287
10.3	Implemented in Phase 1 and Phase 2	288
10.4	Schema Notes	288
10.4.1	PVXAN	288
10.4.2	PVXRF	288
10.5	Implemented in Phase 3, Phase 4, and Phase 5	288
10.6	Implemented in Phase 6 and Phase 7	289
10.7	Immediate Next Steps	289
10.7.1	Sorted Top-Level Command Roadmap	289
IV	Workflow Cookbook	291
11	pvx Example Cookbook	293
11.1	Starter Path (Run These First)	293
11.2	Use-Case Index (By Theme)	293
11.3	Slow Down Speech	294
11.4	Time-Compress Speech (Faster Playback)	294
11.5	Raise Pitch Without Changing Speed	294
11.6	Lower Pitch With Formant Preservation	295
11.7	Stretch + Preserve Formants	295
11.8	Extreme Time Stretch Ambient Pad	295
11.9	Freeze a Harmonic Moment	295
11.10	Retune Vocal to A440	296
11.11	Morph Two Sounds	296
11.12	Create Chorus/Unison Thickness	297
11.13	De-Noise Field Recording	297
11.14	De-Reverb a Room Recording	298
11.15	Segment-Based Dynamic Stretch via CSV	298
11.16	Sidechain Pitch Trajectory (A Controls B)	298
11.17	GPU Acceleration	299
11.18	Batch Processing Folder	299
11.19	Pipe Multiple Tools in One Line	299
11.20	Auto Profile + Auto Transform (Planning)	300
11.21	Multi-Resolution Fusion Render	300
11.22	Checkpoint + Resume Long Render	300
11.23	Harmonize Triad Stack	301
11.24	Timeline Warp with <code>pvxwarp</code>	301
11.25	Layer Harmonic and Percussive Paths	301
11.26	JSON Manifest + Automation-Friendly Logging	302
11.27	Speech Natural Stretch (Hybrid Transients)	302

11.28	Drums Safe Stretch	302
11.29	Stereo Coherent Stretch	302
11.30	Extreme Ambient Stretch (10-minute target)	303
11.31	Pitch Shift with Formant Preservation	303
11.32	Hybrid Transient Mode (Manual Tuning)	303
11.33	Benchmark Command (pvx vs Rubber Band vs librosa)	304
11.34	Transient Reset Mode for Cleaner Attacks	304
11.35	WSOLA-Only Transient Handling	304
11.36	Mid/Side Stereo Coherence Stretch	304
11.37	Reference-Channel Lock for Multichannel Content	305
11.38	Irrational Pitch Ratio Input	305
11.39	Just-Ratio Pitch Input	305
11.40	N-TET CSV Map (19-TET) with <code>pvxconform</code>	306
11.41	Just-Intonation CSV Conform Map	306
11.42	Transform A/B: FFT vs DCT	306
11.43	CZT for Non-Power-of-Two Frame Setup	306
11.44	Full Mastering Chain in One Render	307
11.45	Safety Limiter + Hard Clip Guard	307
11.46	Plan-Only Diagnostic (No Render)	307
11.47	Checkpointed Long Render Workflow	307
11.48	Multi-Tool Pipe with Streaming I/O	308
11.49	Batch Tree Processing with <code>find</code> + <code>xargs</code>	308
11.50	Vocal Retune by Scale/Root (<code>pvxretune</code>)	308
11.51	Harmonize Then Widen in One Pipeline	309
11.52	Ambient Macro-Chain (Stretch + Cleanup + Loudness)	309
11.53	Broadcast-Style Speech Finishing (Stretch + Loudness + Safety)	309
11.54	Extreme Stretch with Checkpointed Recovery	310
11.55	Room Recording Rehab (De-Reverb Then Stretch)	310
11.56	Scale-Quantized Retune Followed by Unison Widening	310
11.57	Morph Then Freeze for Hybrid Pads	311
11.58	CUDA Attempt with Deterministic CPU Fallback Workflow	311
11.59	5.1/Multichannel Reference-Lock Stretch	311
11.60	Mid/Side Lock with Formant-Preserving Pitch Shift	312
11.61	Confidence-Gated External Pitch Control via Pipe	312
11.62	Microtonal Just-Ratio Map Playback	312
11.63	Speech Clarity Pass (Identity Lock + Conservative Window/Hop)	313
11.64	Transform Research A/B (FFT vs Hartley)	313
11.65	Multi-Resolution Fusion With Explicit Weights	313
11.66	Conservative Field-Restoration Chain	314
11.67	Reproducible Ambient Session With Manifest Logging	314
11.68	Managed One-Line Chain (No Manual Pipe Wiring)	314
11.69	Chunked Stream Wrapper + Output Policy Controls	314
11.70	Dynamic Stretch via Direct CSV Flag Input	315
11.71	Dynamic Pitch Ratio via JSON + Polynomial Interpolation	315
11.72	Stairstep (Sample-and-Hold) Control Mode	316
11.73	Dynamic Analysis Resolution (N-FFT + Hop Size Maps)	317
11.74	Stateful Stream Mode with Dynamic Stretch CSV	317
11.75	Generate a Stretch Envelope (Function Stream)	318

11.76	Reshape and Densify a Control Map	318
11.77	Run PVC Parity Benchmark Gate	319
11.78	Persist STFT Analysis Artifact (PVXAN)	319
11.79	Derive and Inspect a Reusable Response Artifact (PVXRF)	320
11.80	Response-Profile Denoising with <code>pvx noisefilter</code>	320
11.81	Time-Varying Spectral Filtering with <code>pvx tvfilter</code>	320
11.82	Spectral Peak Emphasis with <code>pvx bandamp</code>	321
11.83	Response-Shaped Spectral Dynamics with <code>pvx spec-compander</code>	321
11.84	Ring Modulation Fundamentals with <code>pvx ring</code>	322
11.85	Resonant Ring Filtering with <code>pvx ringfilter</code>	322
11.86	Harmonic Chord Mapping with <code>pvx chordmapper</code>	322
11.87	Inharmonic Spectral Warping with <code>pvx inharmonator</code>	323
11.88	Diagnose Environment and Install Issues (<code>pvx doctor</code>)	323
11.89	Minimal Public-Demo Sequence (<code>pvx quickstart</code>)	323
11.90	Quality-First First Pass (<code>pvx safe</code>)	323
11.91	Transform Availability and Selection (<code>pvx transforms</code>)	324
11.92	Fast End-to-End Health Check (<code>pvx smoke</code>)	324
11.93	Deterministic AI Dataset Augmentation (<code>pvx augment</code>)	324
11.94	Speaker-Balanced Split Assignment for Augmentation	325
11.95	Contrastive Paired Views for Self-Supervised Learning	325
11.96	Resume + Append for Long Augmentation Runs	326
11.97	Manifest Validation and Merge	326
11.98	Augmentation Benchmark Gate	326
12	Feature-Driven Sidechain Examples	327
12.1	Quick Start	327
12.2	Inspect Feature Columns Before Routing	327
12.3	Single-Feature Recipes	328
12.3.1	<code>f0_hz</code> -> pitch ratio	328
12.3.2	<code>pitch_ratio</code> -> stretch (inverse motion)	328
12.3.3	<code>confidence</code> -> subtle pitch bend depth	328
12.3.4	<code>voicing_prob</code> -> stretch	328
12.3.5	<code>rms_norm</code> -> stretch	328
12.3.6	<code>spectral_flux</code> -> stretch	328
12.3.7	<code>onset_strength</code> -> attack-reactive stretch	329
12.3.8	<code>transientness</code> -> transient-safe stretch variation	329
12.3.9	<code>spectral_centroid_hz</code> -> brightness-linked pitch	329
12.3.10	<code>spectral_flatness</code> -> noisy/tonal pitch response	329
12.3.11	<code>harmonic_ratio</code> -> harmonicity-linked pitch	329
12.3.12	<code>inharmonicity</code> -> anti-harmonic pitch detune	329
12.3.13	<code>rolloff_norm</code> -> pitch	329
12.3.14	<code>pitch_stability</code> -> micro-variation amount	330
12.4	Multi-Feature Recipes (Different Features -> Different Targets)	330
12.4.1	MFCC + MPEG-7 spectral flux	330
12.4.2	Formant + onset	330
12.4.3	Beat phase + downbeat phase	330
12.4.4	Tempo + centroid	330
12.4.5	Stereo cues: interaural level difference + interaural time difference	330
12.4.6	Noise-aware: hiss + hum	331
12.4.7	Speech vs music probabilities	331

12.4.8	Formant 2 + spectral spread	331
12.4.9	Loudness proxy + transient mask	331
12.4.10	MPEG-7 centroid + MPEG-7 spread	331
12.4.11	MPEG-7 attack-time + MFCC driver	331
12.5	Route Operator Patterns (<code>const</code> , <code>inv</code> , <code>pow</code> , <code>mul</code> , <code>add</code> , <code>affine</code> , <code>clip</code>)	332
12.5.1	Start from a feature, then scale and bias	332
12.5.2	Exaggerate movement with power law	332
12.5.3	Invert and clamp	332
12.6	Feature-Vector Recipes (MFCC + MPEG-7)	332
12.6.1	MFCC second coefficient driver	332
12.6.2	MFCC + MPEG-7 audio spectrum envelope band	332
12.6.3	MPEG-7 spectral crest and decrease	333
12.6.4	Increase MFCC dimensionality to 20	333
12.6.5	Build custom vector features, then route them	333
12.7	Multiple Guide Files (A and B both influence target)	333
12.7.1	Blend pitch behavior from two guides	333
12.7.2	Guide A controls pitch, guide B controls stretch	334
12.8	Manual Pipe Variants	335
12.9	Reuse a Saved Feature Map for Fast Iteration	335
12.10	Compact Feature Set (Smaller CSV)	335
12.11	Safe Starting Ranges	335
13	Follow Workflow Migration	337
13.1	Why This Change	337
13.2	Old vs New	337
13.3	When to Keep Manual Pipes	337
13.4	Rollout Guidance	337
14	100 Crazy Things To Try With pvx	339
14.1	Extreme Time-Scale Madness	339
14.2	Freeze + Morph Experiments	340
14.3	Pitch, Harmony, and Tuning Extremes	341
14.4	Transients, Stereo, and Layer Split	341
14.5	Sidechain Follow and Feature-Driven Control	342
14.6	Control-Map Authoring With Envelope + Reshape	343
14.7	Analysis/Response Artifact and Spectral Operator Ideas	344
14.8	Ring, Resonator, Chordmapper, Inharmonator	345
14.9	Multi-Stage Pipelines, Streaming, and Batch Tricks	346
14.10	QA, Stress, and Reproducibility Challenges	347
V	Python, ML, and Dataset Work	349
15	pvx Application Programming Interface (API) Overview	351
15.1	Import Paths	351
15.2	Minimal Time-Stretch Example	351
15.3	Minimal Pitch-Shift (Duration-Preserving)	351
15.4	Jupyter Notebook Snippets	352
15.4.1	Spectrogram visualization snippet	352
15.4.2	Compare phase locking modes	352
15.5	Batch Processing Example (Python)	353

15.6	Segment Processing Example (Python)	353
15.7	Related CLI Equivalents	354
16	ML/DL Framework Integration Guide	355
16.1	Contents	355
16.2	Installation	355
16.3	Quick Start , Pure Python	356
16.4	Intent Preset Pipelines	356
16.5	PyTorch Integration	357
16.5.1	PvxAugmentDataset	357
16.5.2	PvxAugmentTransform (torchvision-style)	357
16.5.3	AudioCollator for variable-length batches	358
16.6	HuggingFace Datasets Integration	358
16.6.1	Basic map() usage	358
16.6.2	Contrastive view generation (SSL)	358
16.6.3	One-liner convenience	359
16.6.4	Saving augmented datasets	359
16.7	TensorFlow / tf.data Integration	359
16.7.1	Tensor map function	359
16.7.2	Dict-based map for keyed datasets	360
16.8	Determinism and Reproducibility	360
16.9	Performance Tips	360
16.9.1	Offline pre-computation (recommended for large datasets)	360
16.9.2	Online augmentation with worker processes	360
16.9.3	Caching expensive transforms	361
16.9.4	TimeStretch and PitchShift , Engine Selection	361
16.9.5	Streaming Augmentation for Long-Form Audio	362
16.9.6	Impulse Response Database	362
16.9.7	Mix online and offline augmentation	363
16.10	GPU-Accelerated Transforms (PyTorch)	363
16.10.1	Mixing GPU and NumPy transforms	364
16.10.2	Available GPU transforms	364
16.11	Available Transforms	364
17	AI Research and Data Augmentation with pvx	367
17.1	Core Command	367
17.2	Intent Profiles	367
17.3	Reproducibility Controls	367
17.4	Manifest Fields	368
17.5	Example Workflows	369
17.5.1	Speech Robustness Set	369
17.5.2	Music Information Retrieval Set	369
17.5.3	Contrastive Planning-Only Pass	369
17.5.4	With explicit manifest paths	369
17.5.5	Resume and append	369
17.5.6	Manifest merge + stats	370
17.6	Policy File Template	370
17.7	Benchmarking Augmentation Pipelines	370
17.8	Running with uv	371
17.9	Research Notes	371
18	Audio Augmentation Cookbook	373

18.1	Contents	373
18.2	ASR , Automatic Speech Recognition	373
18.3	Keyword Spotting / Wake-Word Detection	374
18.4	Speaker Verification / Identification	375
18.5	Speech Enhancement and Denoising	375
18.6	Music Classification and Tagging	376
18.7	Beat Tracking and Tempo Estimation	376
18.8	Pitch Estimation and Melody Extraction	377
18.9	Audio Event Detection / Sound Classification	378
18.10	Self-Supervised / Contrastive Learning	378
18.11	Music Source Separation	379
18.12	Augmentation Intensity Guide	379
19	pvx Pipeline Cookbook	381
19.1	Analysis/QA	381
19.1.1	Quality metrics on processed speech	381
19.2	Automation	381
19.2.1	A/B report generation	381
19.2.2	Benchmark matrix sweep	381
19.2.3	Quality regression check	381
19.3	Batch	381
19.3.1	Batch stretch over folder	381
19.3.2	Dry-run output validation	382
19.4	Mastering	382
19.4.1	Integrated loudness targeting with limiter	382
19.4.2	Soft clip and hard safety ceiling	382
19.5	Microtonal	382
19.5.1	Custom cents map retune	382
19.5.2	Conform CSV with per-segment ratios	382
19.6	Phase-vocoder core	382
19.6.1	Moderate vocal stretch with formant preservation	382
19.6.2	Independent cents retune	383
19.6.3	Extreme stretch with multistage strategy	383
19.6.4	Ultra-smooth speech stretch (600x)	383
19.6.5	Auto-profile plan preview	383
19.6.6	Multi-resolution fusion stretch	383
19.6.7	Checkpointed long render with manifest	383
19.7	Pipelines	383
19.7.1	Time-stretch -> denoise -> dereverb in one pipe	383
19.7.2	Morph -> formant -> unison	384
19.7.3	Pitch-follow sidechain map (A controls B)	384
19.8	Spatial	384
19.8.1	VBAP adaptive panning via algorithm dispatcher	384
19.9	Transform selection	384
19.9.1	Default production backend (FFT + transient protection)	384
19.9.2	Reference Fourier baseline using explicit DFT mode	384
19.9.3	Prime-size frame experiment with CZT backend	384
19.9.4	DCT timbral compaction for smooth harmonic material	385
19.9.5	DST odd-symmetry color pass	385
19.9.6	Hartley real-basis exploratory render	385

19.9.7	A/B sweep of transform backends from shell loop	385
19.10	Notes	385
VI	Operations and Delivery	387
20	pvx Benchmarks	389
20.1	Reproduce	389
20.2	Benchmark Spec	389
20.3	Host	389
20.4	Backend Summary	390
20.5	Per-Signal Results	390
21	Installing and Managing pvx with Homebrew	393
21.1	Stable Installation	393
21.2	Development Installation	393
21.3	Upgrading	394
21.4	Uninstalling	394
21.5	What the Formula Installs	394
21.6	Shell Path	395
21.7	Troubleshooting	395
21.8	Maintainer Release Flow	396
21.9	Formula Version Policy	396
22	Alpha Release Guide	397
22.1	Supported Surface	397
22.2	Release Checklist	397
22.3	Alpha Principles	397
A	pvx Command-Line Interface (CLI) Flags Reference	399
A.1	Unique Long Flags	399
A.2	hps-pitch-track	400
A.3	pvx	402
A.4	pvxanalysis	413
A.5	pvxcodec	414
A.6	pvxconform	415
A.7	pvxdenoise	416
A.8	pvxdeverb	416
A.9	pvxenvelope	417
A.10	pvxfilter	420
A.11	pvxformant	421
A.12	pvxfreeze	422
A.13	pvxgain	423
A.14	pvxharmmap	424
A.15	pvxharmonize	425
A.16	pvxlayer	426
A.17	pvxmorph	427
A.18	pvxnoise	430
A.19	pvxreshape	431
A.20	pvxresponse	433
A.21	pvxretune	434

A.22	pvxring	435
A.23	pvxrir	436
A.24	pvxspecaugment	438
A.25	pvxtrajectoryreverb	439
A.26	pvxtransient	441
A.27	pvxunison	442
A.28	pvxvoc	443
A.29	pvxwarp	461
B	pvx Algorithm Limitations and Applicability	463
B.1	Group-Level Summary	463
B.2	analysis_qa_and_automation	464
B.3	creative_spectral_effects	465
B.4	denoise_and_restoration	466
B.5	dereverb_and_room_correction	467
B.6	dynamics_and_loudness	468
B.7	granular_and_modulation	469
B.8	pitch_detection_and_tracking	470
B.9	retune_and_intonation	470
B.10	separation_and_decomposition	472
B.11	spatial_and_multichannel	472
B.12	spectral_time_frequency_transforms	475
B.13	time_scale_and_pitch_core	476
C	pvx Diagram Atlas	477
C.1	End-to-End pvxvoc Flow	477
C.2	STFT Analysis / Synthesis Timeline	477
C.3	Phase Propagation and Locking	477
C.4	Hybrid Transient Engine	477
C.5	Transient Feature Stack	478
C.6	WSOLA Similarity Search	478
C.7	Stereo / Multichannel Coherence Modes	478
C.8	Microtonal Map Processing	478
C.9	Control-Rate Interpolation Graph Examples	478
C.10	Function Graph Gallery	479
C.11	Extreme Stretch: Multistage Strategy	479
C.12	Transform Backend Decision Path	479
C.13	Mastering Chain Block Diagram	479
C.14	Metrics Printing Path	479
C.15	Benchmark Runner + CI Gate	479
C.16	Pipe Chaining Pattern	479
C.17	Checkpoint / Resume State Machine	480
C.18	Troubleshooting Decision Tree	480
C.19	Data Product Map	480
C.20	Window/Overlap Tradeoff Map	480
C.21	Phase Reset vs Hybrid Boundary Behavior	480
C.22	Multi-Resolution Fusion Fan-In	480
C.23	Pitch-Map Confidence Gate	480
C.24	Microtonal CSV Interpolation Flow	481

C.25	Audio Metrics Table Generation	481
C.26	Quality-First Optimization Loop	481
C.27	Benchmark Metric Taxonomy	481
C.28	Documentation Build Pipeline	481
D	pvx Phasiness Implementation Plan	483
D.1	Why this paper matters	483
D.2	Benefits we can extract for pvx	483
D.3	Phased implementation	483
D.3.1	Phase 1: Coherence-aware phase propagation in core	483
D.3.2	Phase 2: Adaptive coherence policy	483
D.3.3	Phase 3: Stereo/multichannel coherence extension	484
D.3.4	Phase 4: Benchmark and regression gates	484
D.3.5	Phase 5: UX and docs	484
D.4	Validation targets	484
D.5	Source paper and cited references to track	484
E	pvx Citation Quality Report	485
E.1	Link-Type Summary	485
E.2	Entries Still Using Scholar Links	485
F	PVC Phase 3-5 Architecture	493
F.1	Module Layout	493
F.2	Processing Contracts	493
F.3	Phase 3 Design	493
F.4	Phase 4 Design	494
F.5	Phase 5 Design	494
F.6	Backward Compatibility and UX	494
G	A Phase-Vocoder Listening Library: 100 Musical Works	495
G.1	How to use the library	495
G.2	Documented phase-vocoder practice	496
G.2.1	1. Trevor Wishart: Vox 5	496
G.2.2	2. Roger Reynolds: Transfigured Wind I	496
G.2.3	3. JoAnn Kuchera-Morin: Dreampaths	496
G.2.4	4. Jonathan Harvey: Mortuos Plango, Vivos Voco	496
G.2.5	5. Trevor Wishart: Supernova	497
G.2.6	6. Trevor Wishart: Vox Cycle	497
G.3	Spectral-composition context	498
G.3.1	7. Trevor Wishart: Red Bird	498
G.3.2	8. Trevor Wishart: Tongues of Fire	498
G.3.3	9. Tristan Murail: Desintegrations	498
G.3.4	10. Wendy Carlos: Digital Moonscapes	498
G.3.5	11. Jean-Claude Risset: Inharmonique	499
G.3.6	12. Jean-Claude Risset: Songes	499
G.3.7	13. Kaija Saariaho: Lichtbogen	499
G.3.8	14. Kaija Saariaho: NoaNoa	499
G.3.9	15. Paul Lansky: Six Fantasies on a Poem by Thomas Campion	499
G.4	Voice, text, and formants	500
G.4.1	16. Hildegard von Bingen: O vis aeternitatis	500
G.4.2	17. Anonymous: Dies irae	500
G.4.3	18. Josquin des Prez: Ave Maria virgo serena	500

G.4.4	19. Thomas Tallis: Spem in alium	500
G.4.5	20. Claudio Monteverdi: Lamento d'Arianna	501
G.4.6	21. Henry Purcell: Dido's Lament	501
G.4.7	22. Johann Sebastian Bach: Erbarme dich	501
G.4.8	23. George Frideric Handel: Lascia ch'io pianga	501
G.4.9	24. Wolfgang Amadeus Mozart: Der Holle Rache	501
G.4.10	25. Franz Schubert: Erlkonig	502
G.4.11	26. Richard Wagner: Mild und leise	502
G.4.12	27. Gustav Mahler: Ich bin der Welt abhanden gekommen	502
G.4.13	28. Arnold Schoenberg: Mondestrunken	502
G.4.14	29. Luciano Berio: Sequenza III	502
G.4.15	30. Steve Reich: Different Trains	503
G.5	Sustained tone and resonance	504
G.5.1	31. Antonio Vivaldi: Winter, Largo	504
G.5.2	32. Johann Sebastian Bach: Air on the G String	504
G.5.3	33. Ludwig van Beethoven: Symphony No. 7, Allegretto	504
G.5.4	34. Franz Schubert: String Quintet in C, Adagio	504
G.5.5	35. Frederic Chopin: Nocturne in E-flat major, Op. 9 No. 2	505
G.5.6	36. Richard Wagner: Prelude to Das Rheingold	505
G.5.7	37. Anton Bruckner: Symphony No. 7, Adagio	505
G.5.8	38. Claude Debussy: Prelude a l'apres-midi d'un faune	505
G.5.9	39. Maurice Ravel: Daphnis et Chloe, Lever du jour	505
G.5.10	40. Ralph Vaughan Williams: Fantasia on a Theme by Thomas Tallis	506
G.5.11	41. Gustav Holst: Neptune, the Mystic	506
G.5.12	42. Olivier Messiaen: Louange a l'Eternite de Jesus	506
G.5.13	43. Gyorgy Ligeti: Lux Aeterna	506
G.5.14	44. Arvo Part: Spiegel im Spiegel	506
G.5.15	45. John Tavener: The Protecting Veil	507
G.6	Transient and rhythmic stress tests	508
G.6.1	46. Johann Sebastian Bach: Prelude in C major, BWV 846	508
G.6.2	47. Ludwig van Beethoven: Piano Sonata No. 14, third movement	508
G.6.3	48. Niccolo Paganini: Caprice No. 24	508
G.6.4	49. Frederic Chopin: Etude Op. 10 No. 4	508
G.6.5	50. Nikolai Rimsky-Korsakov: Flight of the Bumblebee	509
G.6.6	51. Igor Stravinsky: The Rite of Spring, Augurs of Spring	509
G.6.7	52. Edgard Varese: Ionisation	509
G.6.8	53. John Cage: First Construction in Metal	509
G.6.9	54. Dave Brubeck Quartet: Take Five	509
G.6.10	55. James Brown: Funky Drummer	510
G.6.11	56. Kraftwerk: Numbers	510
G.6.12	57. Public Enemy: Bring the Noise	510
G.6.13	58. Aphex Twin: Vordhosbn	510
G.6.14	59. Squarepusher: My Red Hot Car	510
G.6.15	60. Autechre: Gantz Graf	511
G.7	Polyphony and orchestral density	512
G.7.1	61. Giovanni Pierluigi da Palestrina: Missa Papae Marcelli, Kyrie	512
G.7.2	62. Johann Sebastian Bach: The Art of Fugue, Contrapunctus I	512
G.7.3	63. Wolfgang Amadeus Mozart: Symphony No. 40, first movement	512
G.7.4	64. Ludwig van Beethoven: Symphony No. 9, finale	512
G.7.5	65. Hector Berlioz: Symphonie fantastique, Marche au supplice	513
G.7.6	66. Johannes Brahms: Symphony No. 4, finale	513

G.7.7	67. Gustav Mahler: Symphony No. 2, finale	513
G.7.8	68. Claude Debussy: La mer, Dialogue du vent et de la mer	513
G.7.9	69. Charles Ives: The Unanswered Question	513
G.7.10	70. Igor Stravinsky: The Firebird, Infernal Dance	514
G.7.11	71. Bela Bartok: Music for Strings, Percussion and Celesta, third movement	514
G.7.12	72. Benjamin Britten: The Young Person's Guide to the Orchestra	514
G.7.13	73. Iannis Xenakis: Metastaseis	514
G.7.14	74. Gyorgy Ligeti: Atmospheres	514
G.7.15	75. John Adams: Short Ride in a Fast Machine	515
G.8	Electronic, studio, and ambient listening	516
G.8.1	76. Karlheinz Stockhausen: Gesang der Junglinge	516
G.8.2	77. Karlheinz Stockhausen: Kontakte	516
G.8.3	78. Delia Derbyshire: Doctor Who Theme	516
G.8.4	79. The Beatles: Tomorrow Never Knows	516
G.8.5	80. Pink Floyd: On the Run	517
G.8.6	81. Brian Eno: An Ending (Ascent)	517
G.8.7	82. Laurie Anderson: O Superman	517
G.8.8	83. Kate Bush: Running Up That Hill	517
G.8.9	84. Cocteau Twins: Lorelei	517
G.8.10	85. My Bloody Valentine: Only Shallow	518
G.8.11	86. The Orb: Little Fluffy Clouds	518
G.8.12	87. Boards of Canada: Roygbiv	518
G.8.13	88. Bjork: All Is Full of Love	518
G.8.14	89. Radiohead: Everything in Its Right Place	518
G.8.15	90. Oneohtrix Point Never: Replica	519
G.9	Extreme duration and temporal form	520
G.9.1	91. Johann Pachelbel: Canon in D	520
G.9.2	92. Ludwig van Beethoven: Symphony No. 5, opening	520
G.9.3	93. Richard Wagner: Prelude to Tristan und Isolde	520
G.9.4	94. Maurice Ravel: Bolero	520
G.9.5	95. Samuel Barber: Adagio for Strings	521
G.9.6	96. John Cage: Organ2/ASLSP	521
G.9.7	97. Steve Reich: Piano Phase	521
G.9.8	98. Alvin Lucier: I Am Sitting in a Room	521
G.9.9	99. William Basinski: The Disintegration Loops 1.1	521
G.9.10	100. Max Richter: Sleep	522
G.10	Comparative listening worksheet	523
H	Transfer-Function and Automation Graph Atlas	525
H.1	Transfer functions	525
H.1.1	pitch ratio from semitones	526
H.1.2	pitch ratio from cents	528
H.1.3	dynamics transfer curves	530
H.1.4	soft-clip transfer functions	532
H.1.5	morph blend magnitude curves	534
H.1.6	mask exponent response	536
H.1.7	phase-mix angle response	538
H.2	Automation and interpolation	540
H.2.1	sample and hold	540
H.2.2	nearest neighbour	541
H.2.3	linear	541
H.2.4	cubic	542

H.2.5	exponential	542
H.2.6	smoothstep S-curve	543
H.2.7	smootherstep	543
H.2.8	polynomial order 1	544
H.2.9	polynomial order 2	544
H.2.10	polynomial order 3	545
H.2.11	polynomial order 5	545
Index Registration Notes		547
Glossary		549
Bibliography		557
Open Media Credits		563

List of Figures

1.1	The phase-vocoder loop. Each output frame is rebuilt from a spectrum whose phase has been advanced on a new time grid.	3
1.2	Time stretching changes the distance between reconstructed frames while preserving their spectral content.	3
1.3	A Voder demonstration at the 1939 New York World's Fair, reproduced from Homer Dudley's 1940 paper. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.	4
1.4	Dudley's 1940 block diagram of the voice mechanism. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.	5
1.5	A historical vocoder schematic from Dudley's 1940 paper. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.	6
1.6	Dudley's companion Voder schematic, showing the related synthesis-oriented system. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.	6
1.7	A selected chronology of representations and implementation ideas leading to modern phase-vocoder practice.	7
1.8	The FFT-oriented analysis and synthesis loop associated with the practical digital phase vocoder.	8
1.9	A conceptual parameter trajectory, the representation that makes time remapping and spectral editing intelligible.	9
1.10	Horizontal and vertical phase relationships in the STFT lattice.	10
1.11	Identity phase locking assigns neighbouring bins to spectral peaks and preserves their observed relative phases.	11
1.12	A conceptual spectral translation used for pitch shifting and harmonisation.	12
1.13	A transient can occupy only a small part of a frame, making onset preservation a separate design problem.	12
1.14	The phase vocoder grew from a loop of mathematical, acoustical, communications, and musical practices rather than a single linear ancestry.	14
1.15	Phase-vocoder, sinusoidal-modeling, and residual-processing lineages increasingly converged in hybrid systems.	19
1.16	Successive research eras enlarged the set of sound properties represented explicitly. Earlier questions survived inside later systems rather than being discarded.	28
1.17	The shrinking audition loop altered compositional method as profoundly as increases in nominal signal quality.	28
1.18	A conservative software genealogy. Solid lines indicate broad institutional or interface continuities; dashed lines indicate contemporary exchange rather than direct descent.	31
1.19	Musical technology develops through feedback among models, implementations, artistic practice, and learned perception.	31
1.20	Institutions contributed different working emphases. The diagram records exchange and convergence, not exclusive ownership of ideas.	38
1.21	Extreme stretching changes which temporal layer behaves as musical form. The boundaries are material-dependent rather than fixed ratio thresholds.	38

1.22	Automation treats a parameter as a continuous curve across the render, not as a sequence of unrelated settings.	42
1.23	A conceptual window response. Window choice balances temporal detail against frequency selectivity.	42
8.1	Time-domain shape of the hann window.	86
8.2	Magnitude spectrum of the hann window.	87
8.3	Time-domain shape of the hamming window.	90
8.4	Magnitude spectrum of the hamming window.	91
8.5	Time-domain shape of the blackman window.	94
8.6	Magnitude spectrum of the blackman window.	95
8.7	Time-domain shape of the blackmanharris window.	98
8.8	Magnitude spectrum of the blackmanharris window.	99
8.9	Time-domain shape of the nuttall window.	102
8.10	Magnitude spectrum of the nuttall window.	103
8.11	Time-domain shape of the flattop window.	106
8.12	Magnitude spectrum of the flattop window.	107
8.13	Time-domain shape of the blackman nuttall window.	110
8.14	Magnitude spectrum of the blackman nuttall window.	111
8.15	Time-domain shape of the exact blackman window.	114
8.16	Magnitude spectrum of the exact blackman window.	115
8.17	Time-domain shape of the sine window.	118
8.18	Magnitude spectrum of the sine window.	119
8.19	Time-domain shape of the bartlett window.	122
8.20	Magnitude spectrum of the bartlett window.	123
8.21	Time-domain shape of the boxcar window.	126
8.22	Magnitude spectrum of the boxcar window.	127
8.23	Time-domain shape of the triangular window.	130
8.24	Magnitude spectrum of the triangular window.	131
8.25	Time-domain shape of the bartlett hann window.	134
8.26	Magnitude spectrum of the bartlett hann window.	135
8.27	Time-domain shape of the tukey window.	138
8.28	Magnitude spectrum of the tukey window.	139
8.29	Time-domain shape of the tukey 0p1 window.	142
8.30	Magnitude spectrum of the tukey 0p1 window.	143
8.31	Time-domain shape of the tukey 0p25 window.	146
8.32	Magnitude spectrum of the tukey 0p25 window.	147
8.33	Time-domain shape of the tukey 0p75 window.	150
8.34	Magnitude spectrum of the tukey 0p75 window.	151
8.35	Time-domain shape of the tukey 0p9 window.	154
8.36	Magnitude spectrum of the tukey 0p9 window.	155
8.37	Time-domain shape of the parzen window.	158
8.38	Magnitude spectrum of the parzen window.	159
8.39	Time-domain shape of the lanczos window.	162
8.40	Magnitude spectrum of the lanczos window.	163
8.41	Time-domain shape of the welch window.	166
8.42	Magnitude spectrum of the welch window.	167
8.43	Time-domain shape of the gaussian 0p25 window.	170
8.44	Magnitude spectrum of the gaussian 0p25 window.	171
8.45	Time-domain shape of the gaussian 0p35 window.	174
8.46	Magnitude spectrum of the gaussian 0p35 window.	175
8.47	Time-domain shape of the gaussian 0p45 window.	178

8.48	Magnitude spectrum of the gaussian 0p45 window.	179
8.49	Time-domain shape of the gaussian 0p55 window.	182
8.50	Magnitude spectrum of the gaussian 0p55 window.	183
8.51	Time-domain shape of the gaussian 0p65 window.	186
8.52	Magnitude spectrum of the gaussian 0p65 window.	187
8.53	Time-domain shape of the general gaussian 1p5 0p35 window.	190
8.54	Magnitude spectrum of the general gaussian 1p5 0p35 window.	191
8.55	Time-domain shape of the general gaussian 2p0 0p35 window.	194
8.56	Magnitude spectrum of the general gaussian 2p0 0p35 window.	195
8.57	Time-domain shape of the general gaussian 3p0 0p35 window.	198
8.58	Magnitude spectrum of the general gaussian 3p0 0p35 window.	199
8.59	Time-domain shape of the general gaussian 4p0 0p35 window.	202
8.60	Magnitude spectrum of the general gaussian 4p0 0p35 window.	203
8.61	Time-domain shape of the exponential 0p25 window.	206
8.62	Magnitude spectrum of the exponential 0p25 window.	207
8.63	Time-domain shape of the exponential 0p5 window.	210
8.64	Magnitude spectrum of the exponential 0p5 window.	211
8.65	Time-domain shape of the exponential 1p0 window.	214
8.66	Magnitude spectrum of the exponential 1p0 window.	215
8.67	Time-domain shape of the cauchy 0p5 window.	218
8.68	Magnitude spectrum of the cauchy 0p5 window.	219
8.69	Time-domain shape of the cauchy 1p0 window.	222
8.70	Magnitude spectrum of the cauchy 1p0 window.	223
8.71	Time-domain shape of the cauchy 2p0 window.	226
8.72	Magnitude spectrum of the cauchy 2p0 window.	227
8.73	Time-domain shape of the cosine power 2 window.	230
8.74	Magnitude spectrum of the cosine power 2 window.	231
8.75	Time-domain shape of the cosine power 3 window.	234
8.76	Magnitude spectrum of the cosine power 3 window.	235
8.77	Time-domain shape of the cosine power 4 window.	238
8.78	Magnitude spectrum of the cosine power 4 window.	239
8.79	Time-domain shape of the hann poisson 0p5 window.	242
8.80	Magnitude spectrum of the hann poisson 0p5 window.	243
8.81	Time-domain shape of the hann poisson 1p0 window.	246
8.82	Magnitude spectrum of the hann poisson 1p0 window.	247
8.83	Time-domain shape of the hann poisson 2p0 window.	250
8.84	Magnitude spectrum of the hann poisson 2p0 window.	251
8.85	Time-domain shape of the general hamming 0p50 window.	254
8.86	Magnitude spectrum of the general hamming 0p50 window.	255
8.87	Time-domain shape of the general hamming 0p60 window.	258
8.88	Magnitude spectrum of the general hamming 0p60 window.	259
8.89	Time-domain shape of the general hamming 0p70 window.	262
8.90	Magnitude spectrum of the general hamming 0p70 window.	263
8.91	Time-domain shape of the general hamming 0p80 window.	266
8.92	Magnitude spectrum of the general hamming 0p80 window.	267
8.93	Time-domain shape of the bohman window.	270
8.94	Magnitude spectrum of the bohman window.	271
8.95	Time-domain shape of the cosine window.	274
8.96	Magnitude spectrum of the cosine window.	275
8.97	Time-domain shape of the kaiser window.	278
8.98	Magnitude spectrum of the kaiser window.	279

8.99 Time-domain shape of the rect window.	282
8.100 Magnitude spectrum of the rect window.	283
H.1 Pitch ratio from semitones.	527
H.2 Pitch ratio from cents.	529
H.3 Dynamics transfer curves.	531
H.4 Soft-clip transfer functions.	533
H.5 Morph blend magnitude curves.	535
H.6 Mask exponent response.	537
H.7 Phase-mix angle response.	539
H.8 Sample and hold interpolation.	540
H.9 Nearest neighbour interpolation.	541
H.10 Linear interpolation.	541
H.11 Cubic interpolation.	542
H.12 Exponential interpolation.	542
H.13 Smoothstep s-curve interpolation.	543
H.14 Smootherstep interpolation.	543
H.15 Polynomial order 1 interpolation.	544
H.16 Polynomial order 2 interpolation.	544
H.17 Polynomial order 3 interpolation.	545
H.18 Polynomial order 5 interpolation.	545

List of Tables

1.1	Representational thresholds in the long history of the phase vocoder.	16
1.2	Artifacts as historical research questions.	22
1.3	Selected chronology of phase-vocoder history and its neighboring lineages.	39
1.4	Three ways to think about changing recorded sound.	43
8.1	Comparison of all pvx analysis windows.	81
8.2	Measured properties of the hann window.	85
8.3	Measured properties of the hamming window.	89
8.4	Measured properties of the blackman window.	93
8.5	Measured properties of the blackmanharris window.	97
8.6	Measured properties of the nuttall window.	101
8.7	Measured properties of the flattop window.	105
8.8	Measured properties of the blackman nuttall window.	109
8.9	Measured properties of the exact blackman window.	113
8.10	Measured properties of the sine window.	117
8.11	Measured properties of the bartlett window.	121
8.12	Measured properties of the boxcar window.	125
8.13	Measured properties of the triangular window.	129
8.14	Measured properties of the bartlett hann window.	133
8.15	Measured properties of the tukey window.	137
8.16	Measured properties of the tukey 0p1 window.	141
8.17	Measured properties of the tukey 0p25 window.	145
8.18	Measured properties of the tukey 0p75 window.	149
8.19	Measured properties of the tukey 0p9 window.	153
8.20	Measured properties of the parzen window.	157
8.21	Measured properties of the lanczos window.	161
8.22	Measured properties of the welch window.	165
8.23	Measured properties of the gaussian 0p25 window.	169
8.24	Measured properties of the gaussian 0p35 window.	173
8.25	Measured properties of the gaussian 0p45 window.	177
8.26	Measured properties of the gaussian 0p55 window.	181
8.27	Measured properties of the gaussian 0p65 window.	185
8.28	Measured properties of the general gaussian 1p5 0p35 window.	189
8.29	Measured properties of the general gaussian 2p0 0p35 window.	193
8.30	Measured properties of the general gaussian 3p0 0p35 window.	197
8.31	Measured properties of the general gaussian 4p0 0p35 window.	201
8.32	Measured properties of the exponential 0p25 window.	205
8.33	Measured properties of the exponential 0p5 window.	209
8.34	Measured properties of the exponential 1p0 window.	213
8.35	Measured properties of the cauchy 0p5 window.	217
8.36	Measured properties of the cauchy 1p0 window.	221
8.37	Measured properties of the cauchy 2p0 window.	225

8.38	Measured properties of the cosine power 2 window.	229
8.39	Measured properties of the cosine power 3 window.	233
8.40	Measured properties of the cosine power 4 window.	237
8.41	Measured properties of the hann poisson 0p5 window.	241
8.42	Measured properties of the hann poisson 1p0 window.	245
8.43	Measured properties of the hann poisson 2p0 window.	249
8.44	Measured properties of the general hamming 0p50 window.	253
8.45	Measured properties of the general hamming 0p60 window.	257
8.46	Measured properties of the general hamming 0p70 window.	261
8.47	Measured properties of the general hamming 0p80 window.	265
8.48	Measured properties of the bohman window.	269
8.49	Measured properties of the cosine window.	273
8.50	Measured properties of the kaiser window.	277
8.51	Measured properties of the rect window.	281

Part I

Foundations

CHAPTER 1

What Is a Phase Vocoder?

A phase vocoder is an analysis-and-resynthesis instrument for recorded sound. It listens to a signal through a sequence of short, overlapping windows, estimates how each frequency component is moving, and rebuilds the signal on a new time grid. That one change of grid is what makes duration and pitch independently controllable. The result may be transparent, deliberately unreal, or somewhere wonderfully in between.

The name can be misleading. A conventional vocoder divides speech into broad frequency bands and transfers their envelopes to a carrier. A phase vocoder also measures frequency content, but its central concern is the phase advance of thousands of narrow spectral bins from one frame to the next. This lets it infer local frequency and re-schedule time without throwing away continuity.

1.1 A quick mental model

Imagine a recording as a roll of film. Each frame is not an image but a tiny spectrum: how much bass, midrange, brightness, noise, and harmonic detail is present at one moment. A phase vocoder overlaps those frames so the film has no visible joins. It can space the reconstructed frames farther apart to lengthen time, closer together to compress time, or use an additional resampling step to move pitch.



Figure 1.1: The phase-vocoder loop. Each output frame is rebuilt from a spectrum whose phase has been advanced on a new time grid.

The practical cycle is simple:

1. Window a short slice of audio.
2. Transform it into magnitude and phase.
3. Compare phase with the preceding slice to estimate true frequency.
4. Advance a new output phase at the desired synthesis hop.
5. Invert the spectrum and overlap-add the result.

The word *phase* matters because magnitude alone is not enough to make a stable waveform. A sine wave can have the same magnitude at many moments while occupying different positions in its cycle. When thousands of partials are rebuilt without a coherent phase story, the ear hears blur, shimmer, or the familiar metallic wash called phasiness.

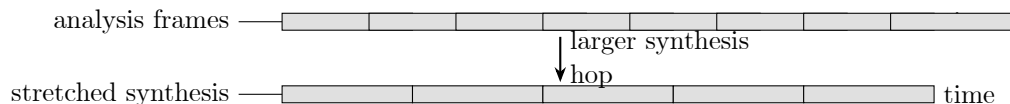


Figure 1.2: Time stretching changes the distance between reconstructed frames while preserving their spectral content.

1.2 The historical thread

The phase vocoder belongs to a longer attempt to separate the identity of sound from the speed at which it unfolds. Mechanical and tape methods came first. If tape runs faster, a recording becomes shorter and its pitch rises. If it runs slower, it becomes longer and its pitch falls. The basic artistic wish, especially in speech, film, and music production, was to change one of those dimensions while keeping the other stable.

1.2.1 Speech analysis before digital audio

In the 1930s and 1940s, Bell Telephone Laboratories developed systems that analyzed speech into spectral control information and reconstructed it with a separate source. Homer Dudley's vocoder and Voder were built for speech research and demonstration, not for the later practice of digital time scaling. Still, they established the crucial conceptual split: a sound can be described by time-varying spectral structure and then recreated by another mechanism.

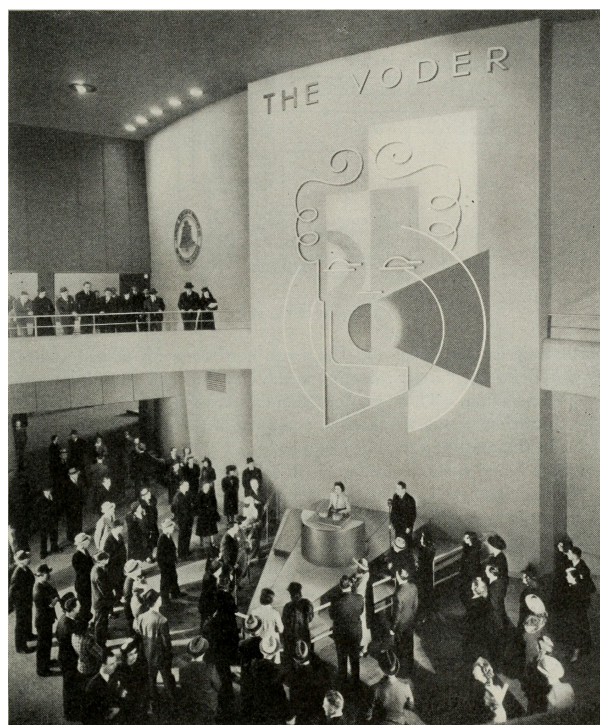


Figure 1.3: A Voder demonstration at the 1939 New York World's Fair, reproduced from Homer Dudley's 1940 paper. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.

The historical vocoder used filter banks and envelope followers. It was not yet a short-time Fourier transform, but it trained a generation of engineers to think of speech as a continuously changing spectrum rather than a single indivisible waveform.

1.2.2 Tape, television, and the wish to uncouple time from pitch

During the tape era, machines such as Anton Springer's rotating-head systems and the later Eltro information rate changer attacked the same problem by slicing and reassembling material mechanically. The techniques made duration and pitch more independent than ordinary varispeed, but editing points, tape-head geometry, and repeated segments imposed their own audible fingerprint. Film and studio work adopted these systems precisely because the problem was already urgent: dialogue had to fit picture, voices had to become strange, and musical phrases had to land where an edit demanded.

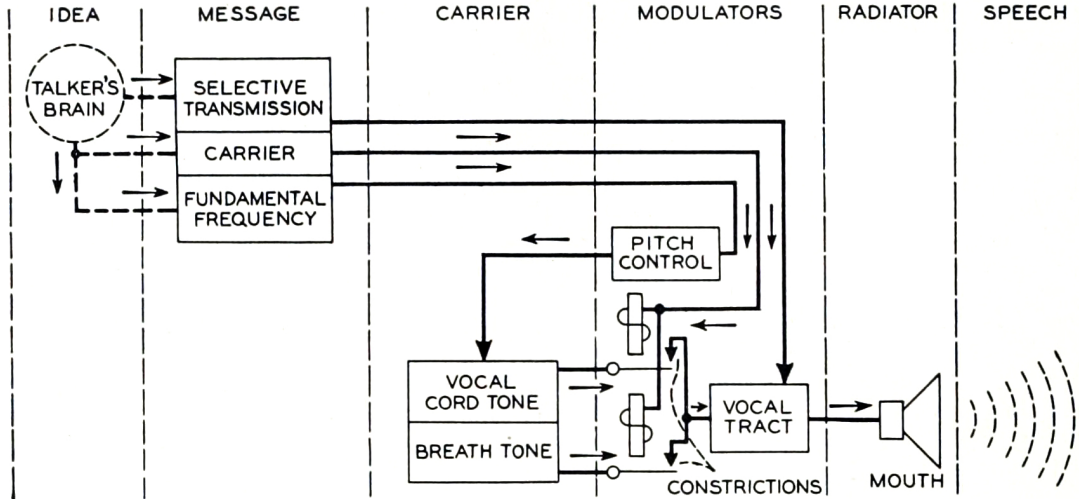


Fig. 6—Block diagram of the voice mechanism.

Figure 1.4: Dudley’s 1940 block diagram of the voice mechanism. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.

The digital phase vocoder inherits that ambition but changes the object being stitched. Instead of small tape segments, it joins carefully phase-propagated spectra. The audible problem remains recognizable: continuity is everything.

1.2.3 Flanagan’s digital formulation

James L. Flanagan introduced the phase-vocoder idea in the 1960s as part of speech analysis and synthesis research. His formulation described a signal in terms of slowly varying amplitudes and instantaneous phases for bands of frequency. By the mid-1970s, Mark Portnoff showed how the short-time Fourier transform and the fast Fourier transform made the method practical on digital hardware.

The key move is to compare observed phase advance with the advance expected at the center of each FFT bin. Their difference estimates the signal’s local frequency. That estimate is then accumulated on a new synthesis timeline.

$$\Delta\phi_t[k] = \text{princarg}\left(\phi_t[k] - \phi_{t-1}[k] - \frac{2\pi k H_a}{N}\right)$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

$$\hat{\omega}_t[k] = \frac{2\pi k}{N} + \frac{\Delta\phi_t[k]}{H_a}$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

$$\hat{\phi}_t[k] = \hat{\phi}_{t-1}[k] + \hat{\omega}_t[k]H_s$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Here H_a is the analysis hop, H_s is the synthesis hop, N is the FFT size, and k identifies a spectral bin. Increasing H_s relative to H_a spreads reconstructed frames farther apart, lengthening the signal.

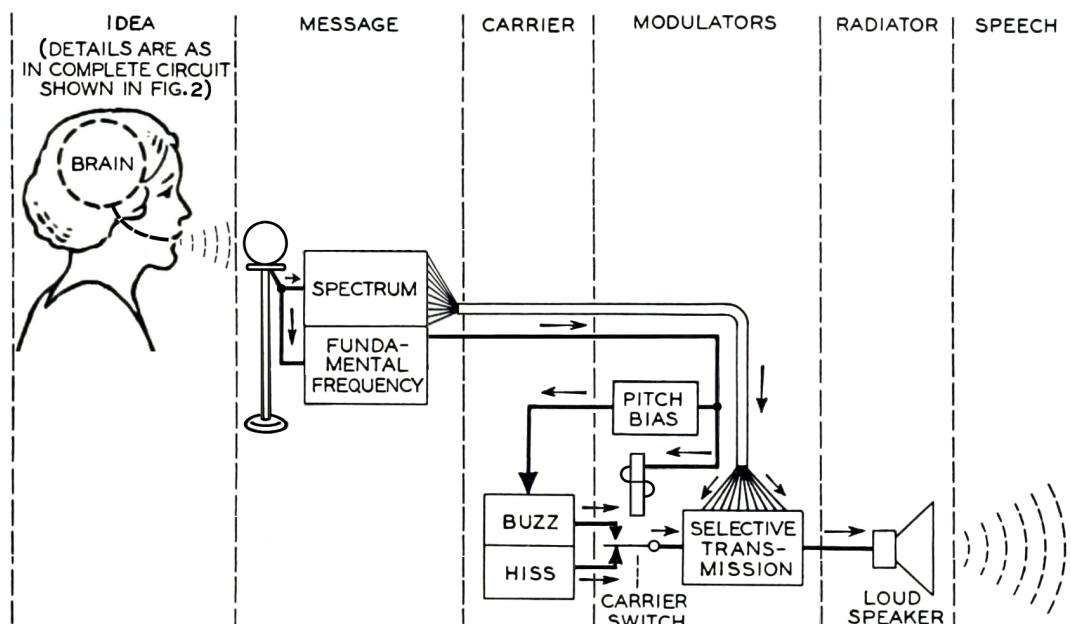


Fig. 7—Schematic circuit of the vocoder.

Figure 1.5: A historical vocoder schematic from Dudley's 1940 paper. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.

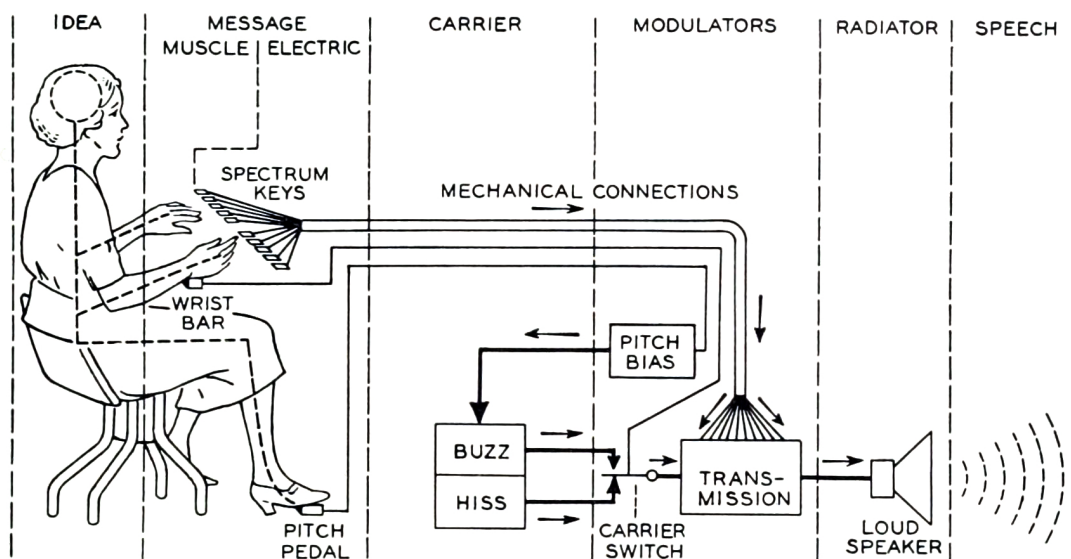


Fig. 8—Schematic circuit of the voder.

Figure 1.6: Dudley's companion Voder schematic, showing the related synthesis-oriented system. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.

1.2.4 From research technique to musical tool

In the 1980s and 1990s, the phase vocoder became an expressive music-processing technique as personal computers and dedicated DSP systems became capable of enough overlapping FFTs. The method made radical time stretching, pitch transposition, spectral freezing, and formant experiments possible in a way tape systems could not. The tradeoff was clear: a phase vocoder can preserve stable tones remarkably well, but transients and rapidly changing spectra require special care.

Modern systems therefore add phase locking, transient detection, multi-resolution analysis, identity phase handling, and hybrid resynthesis. `pvx` follows this tradition with controls that make the quality decisions explicit rather than mysterious.

1.2.5 A chronology of ideas, not merely machines

The history of the phase vocoder is easiest to understand as a sequence of changing representations. Dudley's channel vocoder represented speech by slowly varying band energies and source decisions. Flanagan and Golden represented a signal by short-time amplitude and phase information. Portnoff reorganised the same ideas around the FFT so they could be computed efficiently. Dolson made the digital method legible to musicians and computer-music practitioners. Laroche and Dolson then revisited the phase assumptions that caused loss of presence and gave the field a practical account of vertical coherence.

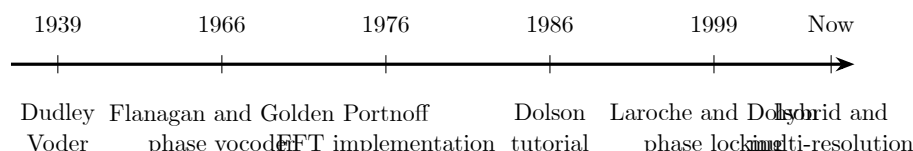


Figure 1.7: A selected chronology of representations and implementation ideas leading to modern phase-vocoder practice.

This chronology should not be read as a single straight line of replacement. Channel vocoders, sinusoidal models, overlap-add systems, time-domain methods, and phase-vocoder techniques continued to influence one another. The important shift is that each generation made a different quantity convenient to manipulate. Once short-time phase derivatives became available, duration could be changed by editing the resynthesis schedule rather than by mechanically changing playback speed.

1.2.6 Flanagan and Golden: phase as transmitted information

James L. Flanagan and Robert M. Golden's 1966 paper, *Phase Vocoder*, described speech in terms of short-time amplitude and phase spectra and simulated a complete analysis-transmission-synthesis system on a digital computer. Their title joined an older word, vocoder, to the quantity that distinguished the new representation. The method did not merely transmit the output of a bank of envelope followers. It retained information about local phase change, which could be interpreted as instantaneous frequency within each analysis channel.

For a complex channel output $X_k(t) = A_k(t)e^{j\phi_k(t)}$, the local angular frequency can be expressed as:

$$\omega_k(t) = \frac{d\phi_k(t)}{dt}$$

where $\omega_k(t)$ is instantaneous angular frequency for channel k , $\phi_k(t)$ is its unwrapped phase, and t is continuous time.

The equation looks modest, but it changes the meaning of phase from a static angle into a trajectory. If that trajectory is sampled consistently, a sinusoid that falls between channel centres can be represented by the difference between expected and observed phase advance. That principle survives in modern STFT implementations.

Flanagan and Golden also discussed time-scale expansion and compression. The historical significance is not that every later implementation copied their system detail for detail. It is that the paper established a phase-aware analysis-resynthesis vocabulary in which timing could become an independent operation. The paper appeared in the *Bell System Technical Journal*, volume 45, number 9, pages 1493 through 1509, with DOI 10.1002/j.1538-7305.1966.tb01706.x.

1.2.7 Portnoff: the FFT makes the method practical

Mark R. Portnoff's 1976 paper, *Implementation of the Digital Phase Vocoder Using the Fast Fourier Transform*, moved the method toward the computational form now familiar in software. The FFT performed the bulk of both analysis and synthesis. Windows, frame hops, phase estimates, and overlap-add reconstruction could be described as a repeatable block algorithm rather than as a large bank of individually implemented filters.

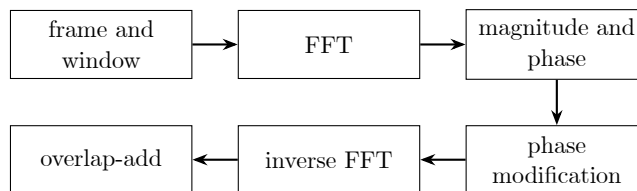


Figure 1.8: The FFT-oriented analysis and synthesis loop associated with the practical digital phase vocoder.

The computational saving matters because a phase vocoder requires many transforms. For a signal of length L , FFT size N , and analysis hop H_a , an approximate transform count is:

$$F \approx 2 \left\lceil \frac{L - N}{H_a} \right\rceil + 2$$

where F is the combined number of forward and inverse transforms, L is the signal length in samples, N is the FFT size, and H_a is the analysis hop in samples.

The exact count depends on padding and boundary policy, but the relationship explains why hop size is both a quality parameter and a cost parameter. Smaller hops increase temporal sampling and overlap while increasing the number of transforms. Portnoff's FFT formulation made that tradeoff manageable on the minicomputers of the period and natural on later workstations.

Portnoff also placed window design close to implementation. A phase estimate is only as useful as the frame that produced it. Leakage, main-lobe width, and overlap behaviour determine how clearly a sinusoidal component appears and how smoothly frames recombine. The complete window atlas later in this book develops that part of the inheritance.

1.2.8 Computer music and the widening of purpose

By the late 1970s and 1980s, the phase vocoder had escaped the narrow category of speech transmission. Computer-music systems treated analysis data as compositional material. Magnitudes could be frozen, interpolated, filtered, or transferred. Phase trajectories could be advanced at a new rate. A sound could be stretched until its internal partials became a landscape.

This change of audience mattered. A telecommunications engineer might measure intelligibility and bandwidth. A composer might value a transformation precisely because it revealed hidden modulation or made a transient dissolve into texture. The same artifact could be an error in one setting and an aesthetic resource in another. Phase-vocoder history is therefore also a history of listening criteria.

The separation of analysis from resynthesis encouraged archives of spectral frames and offline transformations. A program could analyze once and render many times. This pattern remains visible in pvx

analysis artifacts, response profiles, checkpointing, and batch workflows. What changed is the speed and scale: operations that once required a research workstation can now be scripted across a corpus.

1.2.9 Mark Dolson and the tutorial synthesis

Mark Dolson’s 1986 article, *The Phase Vocoder: A Tutorial*, became a major bridge between specialist signal-processing literature and computer-music practice. Its importance lies partly in synthesis. The phase vocoder had accumulated several equivalent descriptions, implementation conventions, and practical uses. Dolson presented the method as a coherent system and related analysis parameters to transformations that musicians could recognize.

The tutorial emphasized that the analysis produces time-varying amplitude and frequency information. Time scaling can then be understood as resampling those trajectories or changing the relationship between analysis and synthesis time. Pitch modification can be built by combining time scaling with sample-rate conversion, or by altering spectral placement more directly.

Let the time-scale factor be α . A simple hop relationship is:

$$H_s = \alpha H_a$$

where H_s is the synthesis hop, H_a is the analysis hop, and α is the requested output-duration ratio.

This compact equation is not a complete algorithm because phase must follow the new schedule. It is nevertheless the control relationship users encounter most directly. If $\alpha > 1$, output frames are spaced farther apart and the sound becomes longer. If $0 < \alpha < 1$, they are spaced closer together and the sound becomes shorter.

Dolson also helped establish a vocabulary for practical applications: time scaling, pitch transposition, cross-synthesis, spectral modification, and the analysis of musical sounds. The tutorial appeared in *Computer Music Journal*, volume 10, number 4, pages 14 through 27. Its enduring role is pedagogical. It taught readers to see the phase vocoder as a general musical instrument whose operations emerge from one analysis-resynthesis model.

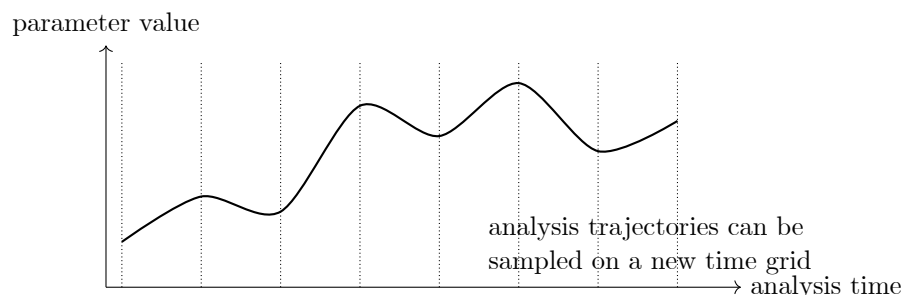


Figure 1.9: A conceptual parameter trajectory, the representation that makes time remapping and spectral editing intelligible.

1.2.10 Horizontal coherence and vertical coherence

For many years the classic phase vocoder’s central weakness was described with perceptual words: phasiness, reverberation, diffuseness, or loss of presence. These terms overlap but are not identical. They point to a sound whose partials no longer seem to belong to one compact source.

Two kinds of relationship help explain the problem. Horizontal coherence concerns a bin across successive frames. Its phase should evolve according to the component’s instantaneous frequency. Vertical coherence concerns neighbouring bins within one frame. Bins belonging to the same spectral peak should retain a meaningful phase relationship.

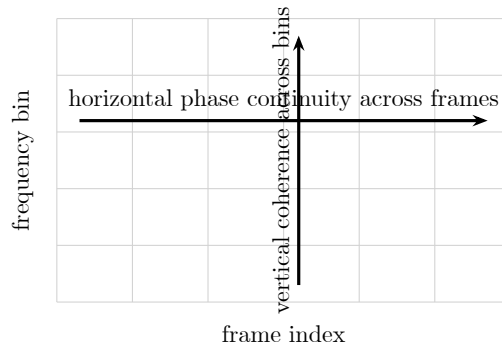


Figure 1.10: Horizontal and vertical phase relationships in the STFT lattice.

Classic propagation can preserve horizontal continuity while allowing bins within a peak to drift independently. The resynthesised peak then behaves less like one sinusoidal component observed through a window and more like a collection of unrelated oscillators. That decorrelation is one source of the diffuse quality associated with phase-vocoder stretching.

1.2.11 Jean Laroche and Mark Dolson: explaining phasiness

Jean Laroche and Mark Dolson’s 1999 paper, *Improved Phase Vocoder Time-Scale Modification of Audio*, directly examined phasiness and proposed new phase-calculation techniques. The paper’s historical importance is that it connected a familiar perceptual complaint to the loss of phase coherence across bins and then offered practical algorithms that improved sound quality.

The authors distinguished phase propagation at spectral peaks from the treatment of bins around those peaks. A peak can be regarded as the centre of a region of influence. Rather than allowing every bin to accumulate phase independently, surrounding bins can inherit a phase relationship from the peak.

For peak bin p and neighbouring bin k , identity phase locking can be written conceptually as:

$$\hat{\phi}_t[k] = \hat{\phi}_t[p] + \phi_t[k] - \phi_t[p]$$

where $\hat{\phi}_t[k]$ is the synthesis phase of neighbour bin k , $\hat{\phi}_t[p]$ is the propagated synthesis phase of peak bin p , $\phi_t[k]$ is the observed analysis phase of bin k , and $\phi_t[p]$ is the observed analysis phase of the peak.

The subtraction retains the analysis-frame phase offset between the neighbour and its peak. The addition places that offset around the peak’s coherent synthesis phase. The bins therefore preserve more of the structure imposed by one underlying partial and its window transform.

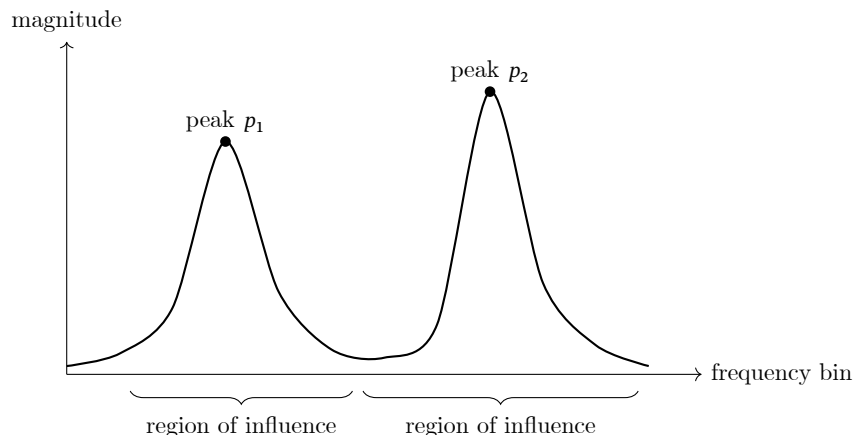


Figure 1.11: Identity phase locking assigns neighbouring bins to spectral peaks and preserves their observed relative phases.

Laroche and Dolson described two extensions and reported significantly improved results. Their work also showed that improved coherence could reduce computational cost in some formulations. The paper appeared in *IEEE Transactions on Speech and Audio Processing*, volume 7, number 3, pages 323 through 332, DOI 10.1109/89.759041.

The contribution should not be reduced to one option labelled phase locking. It changed the field’s account of why the classic sound occurred. Once phasiness was described as a coherence problem, later systems could compare strategies for peak selection, region assignment, transient handling, and channel consistency.

1.2.12 Pitch shifting, harmonising, and spectral translation

Laroche and Dolson also described frequency-domain techniques for pitch shifting, harmonising, chorusing, and non-standard frequency modification. Their 1999 workshop paper presented integer-bin and fractional-bin spectral translation approaches with phase adjustment across successive frames. This line of work is important for pvx because pitch processing is not merely a time stretch followed by anonymous resampling. It can be understood as controlled movement of spectral regions.

For a semitone displacement s , the ideal frequency ratio is:

$$r = 2^{s/12}$$

where r is the multiplicative frequency ratio and s is the displacement in equal-tempered semitones.

For a cent displacement c , the corresponding relation is:

$$r = 2^{c/1200}$$

where r is the multiplicative frequency ratio and c is the displacement in cents.

These equations state the musical target, not the full spectral operation. A practical algorithm must decide how magnitudes move between bins, how phases remain continuous, how components crossing one another are treated, and whether formant structure should follow the pitch shift.

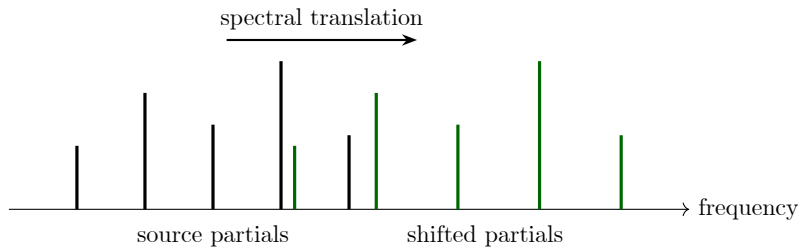


Figure 1.12: A conceptual spectral translation used for pitch shifting and harmonisation.

1.2.13 Transients expose the limits of stationarity

The phase vocoder assumes that a short frame can be treated as a useful local spectral description. A transient challenges that assumption. Its energy changes rapidly, and the onset may occupy only a small fraction of the window. Stretching the surrounding spectral frames can spread that event through time and reduce its impact.

Transient-aware systems detect sudden changes, preserve or reset selected phases, shorten the effective window, or route the event through a time-domain method. Each strategy makes a different compromise. A reset restores alignment at an onset but can interrupt a stable sinusoidal trajectory. A longer window improves low-frequency discrimination but includes more samples from both sides of the attack.

One common spectral-flux measure is:

$$SF_t = \sum_k \max(|X_t[k]| - |X_{t-1}[k]|, 0)$$

where SF_t is positive spectral flux at frame t , $X_t[k]$ is the complex STFT coefficient at bin k , and the maximum retains only increases in magnitude.

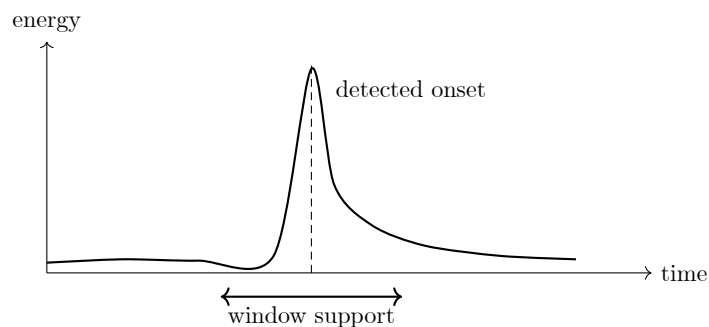


Figure 1.13: A transient can occupy only a small part of a frame, making onset preservation a separate design problem.

This problem led to transient detection, phase resets, hybrid methods, and multi-resolution systems. It also explains why no single window is best for every source. A percussive signal asks the analysis to localise time; a sustained bass note asks it to discriminate nearby low frequencies.

1.2.14 Stereo and multichannel coherence

The historical phase vocoder was often described for one channel. Modern production material is commonly stereo or multichannel, and independent processing can disturb interchannel time and level differences. A stable image requires the algorithm to consider relationships between channels as well as relationships between bins.

For channels i and j , interchannel phase difference is:

$$\Delta\phi_{ij}(k, t) = \phi_i(k, t) - \phi_j(k, t)$$

where $\Delta\phi_{ij}(k, t)$ is the phase difference at bin k and frame t , while ϕ_i and ϕ_j are the channel phases.

Preserving that difference exactly is not always the only goal. Low-frequency image stability, diffuse ambience, decorrelated noise, and transient direction can require different treatment. pvx therefore exposes coherence choices rather than assuming that copying one channel’s phase is universally correct.

1.3 A deeper history of spectral time

The phase vocoder did not appear from one isolated invention. It emerged when several older lines of work became computationally compatible: harmonic analysis, telephone speech research, filter-bank engineering, tape manipulation, digital sampling, fast Fourier transforms, and computer music. Each line contributed a different idea about what sound was and what part of it could be edited. The resulting instrument is therefore best understood as a meeting point rather than a single device with one uninterrupted genealogy.

This longer account follows both engineering and musical history. Engineering history explains why amplitude, frequency, and phase became measurable trajectories. Musical history explains why anyone wished to stretch those trajectories, freeze them, exchange them between sources, or make them cease to resemble their origin. The two histories repeatedly altered one another. A representation designed for economical transmission became a means of composition; an artifact identified by musicians became a research problem; a laboratory process became an interactive effect.

1.3.1 Harmonic analysis before electronic sound

The distant mathematical prehistory begins with the idea that a complex periodic motion can be represented as a sum of simpler oscillations. Joseph Fourier’s work on heat conduction in the early nineteenth century supplied the formal language later used to describe sound spectra. Fourier was not building an audio processor, and it would be misleading to make him the inventor of every spectral technique. His importance is more specific: Fourier series and transforms made it possible to treat a waveform and its frequency components as two descriptions of related information.

Nineteenth-century acoustics made that equivalence tangible. Resonators, tuning forks, sirens, flame apparatus, and mechanical waveform devices allowed investigators to isolate or display periodic behavior. Hermann von Helmholtz used tuned resonators to study partials and vowel quality. His account of tone sensation connected physical spectra with auditory experience and helped establish the distinction between fundamental pitch and spectral coloration. Later phase-vocoder practice would revisit the same distinction through pitch shifting and formant preservation.

Early acoustical instruments were selective but not instantaneous. A resonator might reveal energy around one frequency, yet it did not produce the dense, uniformly sampled time-frequency grid familiar from a modern spectrogram. The historical achievement was conceptual. Sound could be analyzed into components, components could have unequal strengths, and the pattern of those strengths mattered to timbre. Once those claims became ordinary, the later question was how rapidly and precisely the pattern could be measured as it changed.

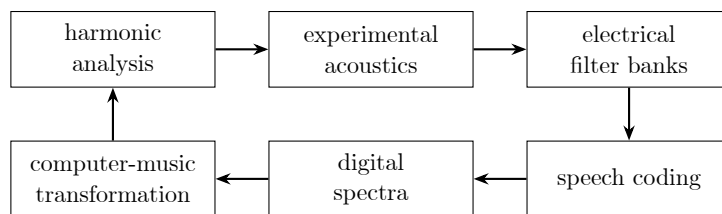


Figure 1.14: The phase vocoder grew from a loop of mathematical, acoustical, communications, and musical practices rather than a single linear ancestry.

The role of phase was less obvious to early listeners than the role of amplitude. A steady sinusoid shifted in absolute phase can sound unchanged when heard alone. This encouraged the belief that phase was perceptually unimportant. The belief is only conditionally useful. Relative phase affects waveform shape, transients, localization, interference, and the ability to join adjacent frames. The phase vocoder’s history can be read as the gradual discovery that phase may be unobtrusive in one static test and indispensable in a changing analysis-synthesis system.

1.3.2 Telegraphy, telephony, and the filter-bank imagination

Electrical communication transformed acoustical analysis into an engineering problem. Telegraph and telephone systems had to transmit useful information through channels with limited bandwidth and noise. Engineers learned to describe circuits by frequency response, to divide spectra into bands, and to measure how speech intelligibility survived filtering. The telephone network created both the institutional scale and the practical urgency for speech analysis.

A filter bank turns one broadband signal into several narrower signals. Each output describes what occurs in one frequency region over time. This architecture encouraged an important abstraction: speech might be transmitted through control functions rather than by reproducing every detail of the waveform. If the slowly varying energy in each band could be measured, perhaps a receiver could reconstruct an intelligible approximation from those measurements and a suitable excitation.

The abstraction was economical and scientific at once. It reduced transmission requirements, but it also offered a model of speech production. A source, periodic for voiced sounds or noisy for unvoiced sounds, passed through a resonant vocal tract. The resonances shaped vowels and speaker identity. Later source-filter models, linear predictive coding, true-envelope methods, and formant-preserving pitch shifts would refine this separation. The historical vocoder was one early operational version of it.

Filter-bank thinking also introduced a tension that remains in spectral software. Narrow bands provide fine frequency selectivity but respond slowly to change. Wide bands respond more quickly but merge nearby components. In digital terminology, this becomes the time-frequency tradeoff of the analysis window. The underlying compromise predates the FFT by decades.

1.3.3 Homer Dudley, the vocoder, and the Voder

At Bell Telephone Laboratories, Homer Dudley developed the channel vocoder during the 1920s and 1930s. The system analyzed speech with bandpass filters, extracted control envelopes, and used those envelopes to shape synthesized excitation at the receiver. A voiced-unvoiced decision selected periodic or noise-like excitation. Dudley’s public Voder demonstration at the 1939 New York World’s Fair reversed the emphasis: a trained operator controlled a keyboard and pedal system to synthesize speech directly.

These systems are ancestors in concept, not phase vocoders in the modern algorithmic sense. They did not use overlapping FFT frames or propagate short-time spectral phase. Their historical contribution was the separation of a sound into excitation and time-varying spectral control. They demonstrated that recognizability could survive a radical change of representation and that intelligible speech could be reconstructed from parameters.

The Voder also revealed the labor hidden by a representation. The machine did not automatically understand language. Skilled operators learned coordinated gestures that produced vowels, consonants, pitch inflection, and timing. That fact is relevant to current software. A sophisticated transform still requires a user to learn which controls correspond to audible outcomes. Better automation changes the interface to the labor; it does not eliminate the need for judgment.

Dudley’s diagrams remain striking because they place physiology, analysis, and synthesis beside one another. The human vocal mechanism is shown as a source filtered by resonances. The electrical vocoder replaces that mechanism with detectors and controlled oscillators or noise. Modern formant tools still operate inside this conceptual space, even when their analysis uses cepstra, sinusoidal models, or high-resolution envelopes.

1.3.4 Secure speech and large-scale vocoder engineering

During the Second World War, vocoder principles contributed to secure speech systems, most famously SIGSALY. Its engineering history involved quantization, synchronization, encryption, and transmission as well as spectral coding. SIGSALY was not a phase vocoder, but it showed that analysis-synthesis speech systems could operate as large technical infrastructures rather than isolated laboratory demonstrations.

The scale of such systems mattered. They required stable oscillators, coordinated timing, calibrated channels, and disciplined operation. In miniature, the same concerns recur in modern offline rendering. Analysis and synthesis must agree on sample rate, hop, window, phase convention, and channel order. A transform can be mathematically plausible yet fail because its states are not synchronized.

Wartime and postwar communications research also accelerated digital and statistical treatments of signals. Quantization noise, prediction, coding, and transmission became mature fields. The phase vocoder would eventually benefit from this environment even though its distinctive formulation arrived later. It inherited a culture in which speech could be decomposed, represented numerically, evaluated, and reconstructed.

The ethical context should not disappear from the technical story. Communications research was funded by military, governmental, and commercial priorities, while musical applications often arrived as secondary appropriations. Computer music repeatedly transformed tools built for control, transmission, or calculation into means of ambiguity and expression. That redirection is part of the phase vocoder’s cultural history.

1.3.5 Tape music and independent control by segmentation

The tape studio approached time and pitch through physical media. Ordinary varispeed couples them: faster playback raises pitch and shortens duration, while slower playback lowers pitch and lengthens duration. Composers embraced that coupling as an effect, but film, broadcasting, and speech work often demanded more independent control.

Rotating-head and segmented-tape devices changed duration by rearranging short pieces of continuously moving tape. The Eltro information rate changer became a well-known example. Such machines could compress or expand speech while approximately retaining pitch. Their quality depended on segment length, head motion, overlap, and the periodicity of the source. Joins could flutter, repeat, or interrupt transients.

This family of methods established several practical ideas later rediscovered in software. First, time modification could be accomplished by changing the schedule of local units. Second, units should overlap or be chosen at compatible waveform positions to hide joins. Third, a unit length appropriate for voiced speech might be poor for noise or percussion. Fourth, artifacts could become stylistic marks rather than mere failures.

Tape studios added another crucial practice: repeated audition. A transformation was judged in relation to material, not only by a generic specification. Engineers marked tape, rehearsed edits, and compared versions. The command-line render inherits this iterative studio method. A phase-vocoder parameter is historically continuous with a physical splice decision in the sense that both become meaningful only in a particular sound.

1.3.6 Sampling, digital audio, and the FFT threshold

Digital signal processing supplied the numerical conditions for a general phase-aware spectral instrument. Sampling represented a waveform as a sequence of numbers. The discrete Fourier transform represented a finite block as complex frequency coefficients. Overlap-add and filter-bank theory explained how blocks could reconstruct a continuous signal. The remaining obstacle was computational cost.

The publication of the Cooley-Tukey fast Fourier transform algorithm in 1965 made efficient Fourier calculation widely available, though related fast methods had earlier precedents. The historical point is not that the FFT instantly created digital audio. Rather, it moved repeated spectral analysis from an extravagant operation toward a practical building block. This mattered enormously for a system that required one transform after another across an entire recording.

Mainframe and minicomputer audio remained expensive. Samples occupied scarce storage, converters were specialized, and a render could take far longer than its program duration. Offline work encouraged a particular software architecture: analyze a sound into an intermediate file, perform transformations on that representation, and resynthesize later. Spectral data became an artifact that could be archived and edited.

This architecture shaped musical thought. A spectrum was no longer merely an explanatory graph. It became material with persistence. A composer could return to analysis data, alter selected bands, interpolate frames, or exchange features between sounds. The file-oriented ancestry remains visible in modern analysis caches, response artifacts, and resumable render plans.

Table 1.1: Representational thresholds in the long history of the phase vocoder.

Period	Convenient representation	New practical possibility
Nineteenth century	harmonic components and resonances	relate waveform complexity to pitch and timbre
Early telephony	filter-bank channel energies	transmit spectral control rather than a complete waveform
Tape era	short physical segments	alter duration through local rearrangement
Early digital era	sampled waveforms and DFT blocks	calculate and store complex spectra
FFT era	repeated short-time transforms	analyze and resynthesize complete sounds efficiently
Workstation era	editable spectral files	compose with trajectories, peaks, envelopes, and frames
Contemporary systems	adaptive and multichannel models	vary resolution, coherence, and processing policy over time

1.3.7 Flanagan and Golden in the Bell Labs setting

James L. Flanagan and Robert M. Golden’s 1966 *Phase Vocoder* belongs to Bell Labs speech research but introduced a representation with consequences beyond transmission. The system analyzed short-time amplitude and phase, represented phase change as frequency information, and resynthesized speech after manipulation. Its concern with phase distinguished it from a channel-envelope vocoder.

The word *phase* in the title should be understood dynamically. Absolute phase at one instant is less useful than phase change over time. A channel’s phase derivative describes local frequency. This allows a component between nominal channel centers to be represented more accurately than a fixed filter label alone would permit. It also provides the information needed to continue that component on a modified time grid.

The paper included time expansion and compression among its applications. That placed independent temporal control inside an analysis-synthesis framework before digital music studios could use the method

routinely. The publication date, one year after the influential Cooley-Tukey FFT paper, marks a threshold at which phase-aware spectral representation and fast computation were becoming mutually relevant.

Flanagan’s broader work on speech analysis and synthesis provided context. The phase vocoder was one element in a sustained effort to understand speech production, perception, and coding. Later musical accounts sometimes detach the algorithm from speech history, but doing so hides why formants, voiced-unvoiced distinctions, intelligibility, and channel behavior entered the field so early.

1.3.8 Portnoff and the practical digital formulation

Mark Portnoff’s work in the 1970s gave the digital phase vocoder a form recognizable to present-day implementers. His 1976 paper described an FFT implementation, and his 1980 work addressed time-scale modification of speech through short-time Fourier analysis. Windows, overlapping frames, phase differences, and reconstruction were organized into a block-processing method.

Portnoff’s contribution was not merely speed. An FFT-oriented description standardized the objects a programmer manipulated. One had arrays of complex bins rather than a diagram of idealized continuous filters. Expected phase advance could be computed from bin index and hop size. Residual phase could be wrapped to a principal interval. Synthesis phase could be accumulated.

That concreteness exposed implementation questions that continue to matter: how frames are centered, how boundaries are padded, whether the window is periodic or symmetric, how overlap is normalized, and what happens when a bin has negligible magnitude. Different programs could implement the same paper and still disagree audibly at transients or file edges.

Portnoff’s speech examples also kept evaluation tied to intelligibility and naturalness. Musical uses would add other criteria, but they did not replace these. Dialogue stretching, language learning, accessibility, and synchronization still ask for the transparent operation that motivated early work.

1.3.9 James A. Moorer and the computer-music turn

James A. Moorer’s 1978 article on the use of the phase vocoder in computer music helped move the technique into a compositional frame. Computer music had already developed synthesis languages, digital recording systems, and analysis tools. The phase vocoder joined these as a way to derive a manipulable description from recorded sound.

The computer-music turn changed the questions asked of the algorithm. How far could time be extended before a source became texture? Could amplitude trajectories from one sound control another? Could individual spectral regions be shifted independently? Could analysis reveal structures that notation did not capture? Such questions treated the intermediate representation as an instrument.

Moorer’s work also belongs to a wider period of laboratory software development in which algorithms traveled through reports, code listings, shared systems, and personal contact. A method was not disseminated only by a journal paper. It moved when a researcher visited another studio, when code was ported to a new machine, or when a composer learned to interpret an analysis file.

This social mode of transmission explains why software genealogies are often difficult to reconstruct. Program names changed, local modifications went unpublished, and an algorithm described in one institution might be rewritten elsewhere under different conventions. A responsible history distinguishes documented lineage from plausible influence.

1.3.10 UC San Diego, CARL, and a software culture

The Computer Audio Research Laboratory at the University of California, San Diego developed an influential environment for computer music and audio processing. CARL software included phase-vocoder tools and encouraged the Unix model of composable programs. Analysis, transformation, and synthesis could be separated into commands connected by files and scripts.

This tool culture matters to pvx directly. A command-line program makes an algorithm repeatable and inspectable. Parameters can be recorded in shell history, placed under version control, swept across a corpus, or reused in a composition. The result is not automatically artistically better, but the procedure becomes easier to reconstruct.

CARL also illustrates the importance of intermediate formats. Spectral analysis data could be passed among operations rather than hidden inside one monolithic application. Such modularity encourages experimentation because a user can insert an unusual step between established ones. It also creates compatibility obligations: frame metadata, bin conventions, and phase interpretation must remain consistent.

The laboratory context brought researchers, composers, and software developers into contact. Phase-vocoder history at UCSD therefore includes both algorithmic refinement and a way of working. Tools were not merely products delivered to passive users. They were objects of study that musicians could modify.

1.3.11 Mark Dolson: research, software, and pedagogy

Mark Dolson's early 1980s research examined a tracking phase vocoder and the analysis of ensemble sounds. Tracking addressed a difficulty hidden by fixed-bin language: a physical partial can move from one bin to another. Treating every bin as a permanent oscillator confuses representation with source. Peak and trajectory models attempt to follow the component instead.

Dolson's work connected several roles. He investigated algorithms, contributed to software practice, explored musical applications, and wrote the 1986 tutorial that became the most widely cited introduction for computer musicians. The tutorial did more than simplify mathematics. It organized a scattered practice into a teachable family of transformations.

Pedagogical texts shape technology because they determine what a generation considers normal. Dolson's account made amplitude and frequency trajectories central. It showed time scaling and pitch shifting as related operations and described cross-synthesis and spectral modification as natural extensions. Readers could imagine the phase vocoder as a general-purpose instrument rather than a specialized speech coder.

The tutorial also preserved the offline, file-oriented imagination of its period. That can seem remote in an age of real-time plug-ins, but it encouraged careful transformations impossible to perform interactively on available hardware. Offline time was not only a limitation. It allowed ambitious calculations, long sources, and compositional planning.

1.3.12 Sinusoidal modeling and a neighboring lineage

The phase vocoder developed beside sinusoidal analysis-synthesis. McAulay and Quatieri modeled speech as tracked sinusoids. Xavier Serra and Julius O. Smith's spectral modeling synthesis separated deterministic sinusoidal components from a stochastic residual. PARSHL and related systems gave musicians explicit control over peaks and tracks.

These methods share short-time spectra with the phase vocoder but interpret them differently. A classic phase vocoder propagates dense bin phases. A sinusoidal model identifies peaks, estimates their parameters, connects them into tracks, and resynthesizes oscillators. The residual accounts for energy not well described by stable sinusoids.

The neighboring lineage influenced phase locking, peak tracking, formant processing, and hybrid transformation. It also clarified that one analysis model need not explain all parts of a sound equally. A violin tone, bow noise, room response, and attack may demand different representations.

Modern systems frequently combine these ideas without announcing a strict category. They may use an STFT frame, identify sinusoidal peaks, classify transients and noise, propagate phases for one region, and resynthesize another region stochastically. The history is therefore braided rather than competitive.

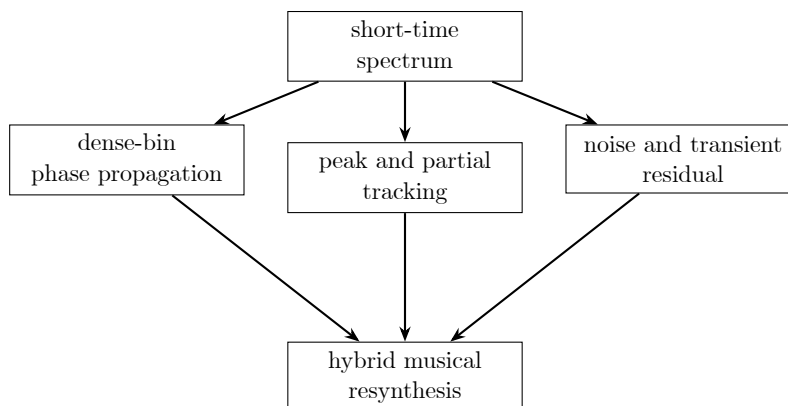


Figure 1.15: Phase-vocoder, sinusoidal-modeling, and residual-processing lineages increasingly converged in hybrid systems.

1.3.13 IRCAM and the institutionalization of spectral transformation

IRCAM provided another major setting in which spectral analysis became a compositional resource. Research, software development, and commissioned music were unusually close. Tools could be tested through demanding artistic projects, while compositional questions could motivate new analysis and transformation methods.

Accounts of the Super Phase Vocoder lineage commonly connect Moorer’s early **p-voc**, CARL-related work, Dolson’s collaboration at IRCAM, developments by Philippe Depalle, and later work by Axel Roebel. Exact software ancestry should be stated cautiously because systems were rewritten and extended. What is well documented is that SuperVP became an advanced engine for analysis, time scaling, transposition, filtering, cross-synthesis, and source-filter transformations.

AudioSculpt placed such processing behind visual analysis and editing. Spectrograms, markers, and treatment parameters made the intermediate representation available to users who did not write signal-processing code. SuperVP also entered real-time Max environments through modules for playback, scrubbing, transformation, and cross-synthesis.

The IRCAM trajectory shows how a research algorithm becomes infrastructure. A mature engine includes file handling, parameter conventions, real-time state, quality modes, envelope estimation, transient classification, and documentation. The core phase update remains important, but reliability emerges from the surrounding system.

1.3.14 Jonathan Harvey and spectral ambiguity

Jonathan Harvey’s *Mortuos Plango, Vivos Voco* (1980) is a landmark of computer music made at IRCAM. The work draws on recordings of the great tenor bell of Winchester Cathedral and the voice of Harvey’s son, a chorister there. Analysis of the bell supplied partial frequencies that shaped the piece’s pitch organization, while transformations brought voice and bell into ambiguous relation.

The piece should not be described as one continuous phase-vocoder demonstration. Its realization involved FFT analysis, Music V, synthesis, transposition, looping, and other processes. The phase-vocoder context is nevertheless important because spectral analysis and time-pitch transformation made the recorded sources structurally available.

Its historical force lies in the relation between technology and form. The bell is not merely an effect sample. Its inharmonic spectrum becomes harmonic material. The voice is not simply speech laid over electronics. Its vowels and partials become a route between human and metallic identities. Computer analysis mediates a spiritual and acoustical idea.

For a phase-vocoder listener, the work offers a discipline: attend to when source recognition remains stable, when it becomes ambiguous, and when a hybrid identity emerges. Those transitions are more historically revealing than trying to label every sound by one algorithm.

1.3.15 Roger Reynolds and the genealogy of Transfigured Wind

Roger Reynolds's *Transfigured Wind* series developed through several versions in the 1980s. The Library of Congress preserves an unusually rich genealogy of the work, including plans, commentary, audio, and discussion of computer components. Reynolds used recorded flute material and processes including phase-vocoder analysis and resynthesis, along with editorial algorithms such as SPLITZ and SPIRLZ.

The project makes transformation audible as formal thought. A flute gesture can be decomposed, prolonged, separated into layers, and placed in relation to live performance. Reynolds emphasized that the computer sounds derive from recorded flute. Transformation enlarges the instrument's world without replacing its identity with an unrelated synthesizer.

The archival record is historically valuable because it shows decisions, revisions, and versions rather than only a finished master. It reveals the labor of choosing source gestures, categorizing them, editing outputs, and coordinating fixed media with performers. This corrects the fantasy that an algorithm produces a composition automatically.

The series also demonstrates how phase-vocoder time operates at multiple scales. A single attack can become a spectral object. A phrase can be expanded. Layers can move at different speeds. Large form can be planned around the return and transformation of recorded identity.

1.3.16 Trevor Wishart and sound morphing

Trevor Wishart's work gives one of the clearest artistic accounts of phase-vocoder data as material. In writings on *Vox 5* and later retrospectives, Wishart described exploring the contents of analysis files and developing tools for sound morphing. The aim was not simply to stretch a voice cleanly. It was to move continuously among vocal, animal, mechanical, and environmental identities.

Morphing depends on selecting what changes and what remains continuous. Spectral envelopes, partial frequencies, noise, duration, and gesture can move at different rates. A convincing transformation may preserve the rhythm of one source while gradually adopting the spectrum of another. The phase-vocoder representation makes several of these dimensions separately addressable.

Vox 5 became a canonical example because the transformation participates in a larger vocal imagination. Extended performance, speech, and computer processing occupy the same world. The technology does not function as an external accompaniment. It expands the possible anatomy of the voice.

Wishart's later account, extending from early experiments to *Supernova*, also reminds historians that techniques mature through sustained personal toolmaking. A published algorithm is only a starting vocabulary. Composers construct families of operations, test perceptual thresholds, and develop conventions specific to their work.

1.3.17 JoAnn Kuchera-Morin and expanded duration

JoAnn Kuchera-Morin's *Dreampaths* (1989) is frequently cited for extensive early phase-vocoder transformation. Accounts describe dramatic expansion of recorded duration, spectral glissandi, and the alteration of instrumental and vocal sounds. Her broader work joins composition, computer systems, and immersive media.

Extreme stretching changes the listener's scale of attention. A thirty-second source expanded to several minutes no longer functions as an ordinary phrase. Vibrato becomes a slow trajectory, breath becomes texture, and room noise becomes part of the orchestration. The phase vocoder acts less like a correction tool than a microscope for temporal structure.

Such work also reveals the limits of simple quality language. Smearing may be unacceptable in dialogue and productive in a dreamlike electroacoustic texture. Historical interpretation must ask what the composition values. Fidelity can mean fidelity to source identity, to gesture, to spectral detail, or to a planned transformation.

Kuchera-Morin’s practice broadens the institutional map beyond Bell Labs, Stanford, UCSD, and IRCAM. Phase-vocoder history was made by composers and researchers across universities and studios, including figures whose contributions were not always centered in standard textbook narratives.

1.3.18 Puckette, phase locking, and real-time environments

Miller Puckette’s phase-locked vocoder work in the mid-1990s addressed the loss of coherence that made stretched tonal sounds diffuse. It belongs to a broader period in which real-time FFT processing became available in flexible environments such as Max. Spectral transformation could increasingly respond during performance rather than only after an offline render.

Phase locking groups bins around spectral peaks so they do not evolve as unrelated oscillators. The idea drew phase-vocoder practice closer to sinusoidal modeling while retaining an STFT resynthesis framework. It made explicit that neighboring bins often describe one windowed partial.

Real-time operation changed design priorities. Latency became audible to performers. Peak detection and phase updates had to complete before the next block. Parameters needed smoothing because an abrupt control gesture could produce a discontinuity. User interfaces had to expose enough control without requiring interpretation of every bin.

The history of Max, Pure Data, Csound, and related systems is therefore part of the phase vocoder’s history. They made spectral operations available as reusable objects or opcodes. A technique formerly encountered through specialist code could enter an improvisation, installation, or classroom patch.

1.3.19 Laroche and Dolson: from artifact to explanatory model

Jean Laroche and Mark Dolson’s work in the late 1990s gave phasiness a more precise explanation. The classic algorithm could maintain phase continuity for each bin across time while allowing bins within a spectral peak to lose their relative organization. The output retained horizontal coherence but weakened vertical coherence.

This distinction turned a musician’s complaint into an engineering model. The diffuse quality was not merely the inevitable sound of Fourier processing. It followed from a particular independence assumption. Identity and scaled phase-locking methods offered alternative relationships around spectral peaks.

The 1999 work on improved time-scale modification became a reference point for later algorithms and evaluations. It also influenced real-time systems such as PhaVoRIT, which combined scaled phase locking with interactive concerns and perceptual testing.

The historical lesson is methodological. Artifacts can be evidence. A repeated perceptual description may indicate that the representation has broken a relationship listeners rely on. Naming that relationship can lead to a better algorithm and a better listening vocabulary.

1.3.20 Transients become a separate research object

As phase locking improved tonal coherence, transients remained conspicuous. An onset is localized in time and spread in frequency, almost the opposite of a stationary sinusoid. A long analysis window mixes samples before and after the event. When synthesis frames are moved apart, the onset’s energy can be distributed across a longer interval.

Research by Chris Duxbury, Mike Davies, Mark Sandler, Axel Roebel, and others treated transient detection and preservation as explicit problems. Proposed strategies included detecting attacks, resetting phase, moving transient frames without stretching them, separating transient and steady components, and using multiple resolutions.

Roebel’s work emphasized transient processing within the phase-vocoder framework. Later adaptive systems varied analysis according to transience or corrected magnitudes with multiresolution information while retaining a coherent phase path. The field increasingly accepted that one global resolution was a convenience rather than an acoustical truth.

This research also changed software interfaces. A user might now choose sensitivity, protection duration, reset mode, or hybrid behavior. Such parameters are historical sediment: each exists because a class of sounds exposed a weakness in an earlier default.

Table 1.2: Artifacts as historical research questions.

Perceptual report	Representation problem	Research response
diffuse or reverberant tone	neighboring bins lose relative phase organization	peak-based identity and scaled phase locking
smeared attack	one long frame treats an onset as stationary content	transient detection, phase reset, and hybrid routing
metallic noise	noise is forced into deterministic bin trajectories	stochastic or noise-aware resynthesis
shifted vowel identity	harmonics move with the broad spectral envelope	source-filter separation and envelope preservation
unstable stereo center	channels are transformed independently	shared timing, reference phase, and interchannel constraints

1.3.21 WSOLA, PSOLA, granular methods, and productive rivalry

The phase vocoder has never been the only method for time or pitch modification. Time-domain overlap-add methods align waveform segments. WSOLA selects segments by waveform similarity. PSOLA works near pitch-synchronous positions and became influential in speech synthesis and modification. Granular methods reorganize short grains with flexible timing and envelopes.

Each method makes different assumptions. PSOLA can preserve voiced speech efficiently when pitch marks are reliable, but irregular or polyphonic material complicates the model. WSOLA can retain attacks and local waveform character, but large transformations or sustained complex tones may reveal repetition and alignment choices. Granular methods embrace a textural unit whose size becomes part of the sound.

Rivalry improved phase-vocoder research by clarifying evaluation. A method might excel on sustained harmony and lose on drums. Another might preserve speech attacks and struggle with polyphony. Hybrid systems emerged because source categories did not respect algorithm names.

Commercial time-stretching increasingly concealed these choices behind quality modes or source presets. The user selected solo voice, polyphonic music, rhythmic material, or extreme stretch, and the engine changed its internal strategy. pvx exposes more of the policy because reproducibility and experimentation benefit from explicit controls.

1.3.22 Spectral envelopes, formants, and the source-filter return

Pitch shifting made an old speech problem newly audible. Moving every spectral feature upward can make a voice seem smaller; moving everything downward can make it seem larger or muffled. The harmonic series and the broad vocal-tract envelope do not play the same perceptual role.

Cepstral methods and later true-envelope estimation allowed systems to separate fine harmonic structure from broad resonant shape. Axel Roebel and Xavier Rodet developed efficient spectral-envelope estimation and applied it to pitch shifting with envelope preservation. Shape-invariant voice transformation extended the idea.

This line reconnects the phase vocoder to Dudley’s source-filter model. The technology has changed from filter-bank envelopes to high-resolution short-time spectra, but the conceptual split remains: excitation can move in pitch while resonant identity follows another rule.

Formant preservation is not always transparency. A composer may deliberately move formants against pitch, exchange envelopes between sources, or animate vocal size. Once the envelope is explicit, identity becomes another trajectory available for design.

1.3.23 Noise, residuals, and the limits of sinusoidal phase

Noise exposes another limit. A stable sinusoid benefits from a coherent phase trajectory. Broadband noise does not consist of partials that should all remain phase-locked in the same way. Stretching random phase as though it were deterministic can produce metallic or frozen coloration.

Researchers explored stochastic representations, noise-specific phase treatment, and hybrid decomposition. Gaussian-noise stretching became a specialized topic because ordinary phase propagation changes statistical character. Sinusoidal plus residual models offered one route: track coherent peaks and synthesize the remaining energy as shaped noise.

The problem is musically important. Breath, bow scrape, cymbal wash, reverberation, and environmental sound contain mixtures of stable and stochastic behavior. A processor that preserves only the pitched center may remove the material’s scale or intimacy.

Modern quality often depends on classification across time and frequency rather than one decision for an entire frame. A region can be treated as sinusoidal, transient, or noisy, then crossfaded with neighboring decisions. The phase vocoder becomes a framework for local policies.

1.3.24 Stereo, spatial hearing, and multichannel history

Early formulations commonly assumed a single channel. Stereo and multichannel production added phase relationships that encode direction, width, and ambience. Processing channels independently can move the center image, change apparent width, or weaken mono compatibility.

Spatial coherence research draws on time-delay estimation, array processing, and spatial audio as well as phase-vocoder work. Strategies include using one reference channel for peak and phase decisions, preserving interchannel phase differences, or treating diffuse components separately from localized ones.

There is no universal spatial rule. A centered vocal may require strong locking. Diffuse reverberation may become unnaturally narrow if forced to follow one channel. Low-frequency coherence may matter differently from high-frequency decorrelation. Multichannel transformation therefore made policy and source analysis even more important.

The historical shift from mono speech to immersive media is substantial. The same short-time representation now participates in binaural production, surround installations, microphone arrays, and spatial machine-learning data. Channel count multiplies both creative possibility and failure modes.

1.3.25 Open tools, standards, and software migration

Phase-vocoder techniques spread through Csound, CARL, the Composers’ Desktop Project, IRCAM Forum software, Max, Pure Data, SoundHack, GRM tools, research toolboxes, and code examples. Each environment emphasized different workflows: score-driven synthesis, Unix commands, graphical patching, offline transformation, or plug-in use.

Open and academic tools preserved techniques that might otherwise have disappeared with obsolete hardware. They also exposed implementation differences. One program stored amplitude and frequency, another magnitude and phase. One used oscillator-bank resynthesis, another inverse FFT. File formats and window conventions did not always travel cleanly.

Porting became a form of scholarship. To move a phase vocoder to a new machine, a developer had to interpret old code, documentation, and numerical assumptions. Bugs could become characteristic sounds; fixes could break compatibility with archived analysis files.

Standards such as SDIF attempted to make sound descriptions portable. The broader lesson for pvx is that an analysis artifact needs schema, units, version, and provenance. Spectral data without interpretation is not a durable historical object.

1.3.26 From offline render to interactive instrument

By the 1990s and 2000s, faster processors and dedicated DSP hardware made real-time phase-vocoder use increasingly practical. Interactive time stretching appeared in performance systems, DJ tools, games, installations, and musical interfaces. PhaVoRIT explicitly investigated real-time interactive stretching and user evaluation.

Real time is not simply offline processing performed faster. It changes what can be known. Future transients may not yet be available for lookahead. A performer expects bounded latency. Control values can move abruptly. A dropped block is more damaging than a slow offline job.

Sliding-transform methods explored different latency and update structures. GPU implementations pursued throughput. Real-time SuperVP modules brought advanced transformations to Max. Such systems turned phase-vocoder state into part of an instrument that had to respond continuously.

Interactive control also shifted musical agency. A stretch factor could follow a runner’s pace, a gesture sensor, or another performer. Time transformation became relational rather than predetermined. The output clock could negotiate with an external clock.

1.3.27 Adaptive and nonstationary time-frequency analysis

One fixed window embodies one compromise. Adaptive and nonstationary approaches attempt to vary that compromise according to frequency or signal behavior. Constant-Q methods provide frequency-dependent resolution. Nonstationary Gabor frames offer invertible representations with changing windows. Multiresolution phase vocoders combine analyses to protect transients and resolve sustained partials.

These approaches are historically significant because they challenge the idea that the STFT grid is the sound. The grid is a measurement choice. A bass partial, a cymbal attack, and a consonant may each become clearer under a different observation scale.

Adaptive methods also create new coherence problems. Results from different resolutions must be combined without phase contradiction. Some systems retain one phase path and use other resolutions to correct magnitude. Others build mathematically consistent nonstationary frames.

The increasing mathematical sophistication does not remove listening. Adaptation criteria still encode assumptions about transience, harmonicity, and relevance. A detector trained on drums may misread a bowed attack. Historical progress remains a dialogue between formal guarantees and material exceptions.

1.3.28 Machine learning enters the surrounding workflow

Contemporary audio systems often use machine learning around, beside, or in place of classical transforms. Neural source separation can provide stems before time modification. Pitch models can supply more robust trajectories. Learned transient or quality models can guide parameters. Neural vocoders synthesize waveforms from acoustic features, though their use of the word *vocoder* belongs to a different lineage.

The shared name can cause confusion. A neural speech vocoder such as WaveNet, HiFi-GAN, or related systems reconstructs audio from learned acoustic representations. A phase vocoder propagates phase in a short-time spectral transformation. Both are analysis-synthesis ideas, but their models, training requirements, and artifacts differ.

Hybrid workflows are more historically interesting than replacement stories. A learned separator can isolate voice and drums; a phase vocoder can then use different policies for each. A learned pitch tracker

can drive formant-aware transformation. Classical transforms provide determinism and inspectable controls where a model may provide classification or estimation.

The phase vocoder remains useful partly because its failures are legible. Window, hop, phase, and transient decisions can be inspected. That transparency matters in research, education, dataset generation, and reproducible production.

1.3.29 A historiography of claims and cautions

Technical histories often become too tidy. They assign one invention to one person, treat later papers as complete replacements, and describe musical works as demonstrations of a named algorithm. The documentary record is messier. Ideas were developed in parallel, software was rewritten locally, and compositions combined many processes.

Several cautions improve accuracy. Dudley’s vocoder is a conceptual ancestor, not an FFT phase vocoder. Cooley and Tukey popularized a fast algorithm but did not originate every FFT factorization. IRCAM’s SuperVP has a lineage of many contributors rather than one author. A work made in a phase-vocoder studio need not use that process in every passage.

The distinction between invention and dissemination also matters. Flanagan and Golden formulated the phase vocoder. Portnoff made an FFT implementation practical and explicit. Moorer and Dolson helped make the technique meaningful to computer musicians. Puckette, Laroche, Dolson, Roebel, and many others improved coherence and transient behavior. Tool builders brought it to wider communities.

Finally, user practice is part of history. Defaults, presets, forum explanations, code examples, and listening habits determine which algorithms survive. A technically superior method can disappear if it is difficult to integrate. A simple implementation can become canonical because generations learn from it.

1.3.30 Reading the foundational papers as historical documents

A list of publication dates cannot show how the phase vocoder changed. The major papers differ not only in method but also in audience, vocabulary, diagrams, examples, and assumptions about available machines. Reading them in sequence reveals a change from communications engineering to an increasingly musical and perceptual discipline. It also reveals continuity. Questions about intelligibility, parameter economy, and reconstruction never vanished when composers entered the story.

Flanagan and Golden’s 1966 paper belongs to an environment in which speech communication was a central technical object. Its diagrams describe analysis and synthesis channels, phase derivatives, and signal transmission. Time expansion appears as an application of a representation designed to preserve more information than a magnitude-only channel vocoder. The paper’s historical originality lies in treating phase change as useful transmitted information and in showing how that information could support a modified reconstruction.

Portnoff’s publications move the account closer to code. The FFT makes a bank of narrow channels computationally regular, and the short-time transform places analysis in a repeated frame structure. A reader can recognize the modern ingredients: windowed blocks, overlaps, complex spectra, expected phase advance, and synthesis. The papers also preserve the intellectual connection between a modulated filter bank and a discrete transform. That connection matters because the phase vocoder is not simply an image editor for spectrograms. It is an analysis-synthesis system whose channels must agree over time.

Allen and Rabiner’s account of short-time Fourier analysis and synthesis provided a broader frame in which these operations could be understood. Their work helped make reconstruction conditions, time-varying spectra, and overlap processing part of a shared signal-processing vocabulary. It supplied intellectual infrastructure even when a later implementation did not cite every detail directly.

Moorer’s computer-music article changes the implied reader. The important question is no longer only whether speech can be transmitted or expanded intelligibly. It is what a composer can do with an analysis. Spectral modification, time scaling, and resynthesis become parts of a studio process. The article appears

during a period when computer music systems were becoming extensive enough to support libraries of reusable transformations, yet scarce enough that access to a laboratory still determined who could experiment.

Dolson's tutorial is a document of consolidation. It translates a technically scattered field into a coherent explanation for musicians and programmers. The tutorial's enduring value comes from the way it joins pictures, equations, implementation ideas, and audible consequences. It presents the phase vocoder as a family of operations on trajectories rather than as one fixed effect. This pedagogical act helped establish the conceptual interface later programs inherited.

Griffin and Lim's work on estimating a signal from a modified short-time magnitude addresses a related but distinct problem. If an operation changes magnitude without retaining a compatible phase, inversion becomes an estimation task. The iterative algorithm alternates between time-domain consistency and a desired magnitude. Its place in history is important because it demonstrates that an arbitrary spectrogram is not necessarily the transform of any ordinary waveform. Spectral editing therefore has geometrical constraints, not merely aesthetic ones.

Puckette's phase-locked approach and Laroche and Dolson's later analysis bring the listening complaint called phasiness into the center of technical explanation. These writings reinterpret a spectral peak as a structured neighborhood rather than an accidental collection of bins. Their diagrams and derivations make coherence visible as a relation both across frames and within a frame. The resulting methods belong to a moment when computer-music practice had generated enough shared listening experience for an artifact to become a stable research object.

Roebel's transient research extends that perceptual turn. The attack is no longer treated as a difficult exception to an otherwise uniform signal. It becomes a component with its own detection, preservation, and reconstruction problem. Later multiresolution research continues this shift by asking whether analysis scale itself should follow the material. The documents thus trace a widening ontology: first channels, then bins, then peaks, then transient and noise classes, and finally adaptive regions with different models.

1.3.31 The hidden history of computation time

Algorithm histories can make ideas seem available as soon as they were published. In practice, the cost of conversion, storage, transformation, and audition governed what artists could attempt. A short modern render may conceal a chain that once required a scheduled laboratory session, dedicated converters, a mainframe job, an intermediate tape, and a return visit after computation finished.

Early digital audio consumed memory at a scale that was disproportionate to ordinary computing resources. A few seconds of sound could be a substantial dataset. Analysis enlarged it because each frame carried many spectral values and because some systems stored amplitude-frequency pairs rather than a compact waveform. Long stretches multiplied output duration again. A musically interesting request could therefore become a storage problem before it became a signal-processing problem.

Processing time affected form. When each trial was expensive, composers had reason to plan transformations in advance, work from short source excerpts, and preserve intermediate results. Parameter sweeps were possible, but they were deliberate batches rather than instant gestures. Listening occurred after a delay. That delay encouraged written plans, catalogs of source sounds, and naming systems for renders.

The workstation changed the rhythm of iteration. A composer could inspect a spectrogram, select a region, launch an operation, and compare alternatives in one working session. Graphical systems made time-frequency selection spatially direct. The mouse did not simplify the mathematics, but it shortened the path between noticing a feature and acting on it.

Real-time processing changed the epistemology again. A performer could learn a transformation through bodily feedback. Controls could be explored continuously rather than as a sequence of files. Yet interactivity introduced a strict deadline. An offline renderer could use future samples, revise an estimate, or spend several seconds on one frame. A live system had to return the next block on time.

Contemporary machines support much larger transforms and more elaborate classifiers, but abundance introduces its own historical condition. A modern user may audition hundreds of settings without recording

why one worked. Reproducible command lines, manifests, and checkpoints restore some of the documentary discipline imposed by scarce computing. `pvx` belongs to this later culture: computation is relatively cheap, while preserving intention remains difficult.

1.3.32 A fuller genealogy of phase-vocoder software

Software lineage is harder to narrate than publication history because code is mutable. A program may be copied, translated, optimized, renamed, or partly rewritten. Documentation may survive while source code disappears, or source may survive without a reliable account of local use. The following genealogy therefore emphasizes working cultures and documented relationships rather than claiming one unbroken family tree.

Early laboratory implementations were inseparable from institutional computing. Bell Labs systems supported speech research. Stanford, UCSD, MIT, and other centers developed computer-music programs around the machines and converters they possessed. A phase vocoder was often not one portable application. It was a pipeline of analysis, transformation, and synthesis programs embedded in local formats and job-control conventions.

At UCSD, CARL software helped establish a Unix-oriented audio culture. Small programs could be composed into larger processes, and files allowed analysis to persist between stages. This architecture encouraged users to treat spectral information as a durable medium. It also made errors inspectable. One could examine headers, compare stages, or rerun only the transformation rather than repeat the analysis.

Csound offered another path. Its score-and-orchestra model placed analysis-based processing beside synthesis, sampling, and control-rate composition. Phase-vocoder opcodes and analysis formats allowed a sound file to participate in a notated computational instrument. Csound's portability helped techniques travel beyond the institutions where they were first developed.

The Composers' Desktop Project emphasized affordable offline tools for composers using personal computers. Its extensive catalog of transformations made spectral operations part of a broader craft of sound editing. CDP's command-oriented workflow preserved the idea that unusual results often arise by chaining modest processes rather than selecting one comprehensive effect.

SoundHack, developed by Tom Erbe, gave artists direct access to classic and experimental sound transformations on personal computers. Its phase-vocoder and convolution tools became important in studios and classrooms. The program illustrates a recurrent form of dissemination: a researcher-programmer interprets specialist literature, builds an approachable implementation, and thereby changes which ideas become part of ordinary musical practice.

At IRCAM, the SuperVP and AudioSculpt lineage integrated research on high-quality stretching, transposition, source-filter transformation, transients, and spectral selection. AudioSculpt made analysis visually navigable, while SuperVP supplied a deep processing engine. Max integrations then brought parts of that engine into interactive performance. The lineage demonstrates how one institution can sustain an algorithm across offline, graphical, and real-time forms.

Miller Puckette's Max and Pure Data environments made FFT processing patchable. Educational phase-vocoder examples exposed windowing, real and imaginary channels, phase accumulation, and overlap as signal networks. A patch could be altered while it ran. That visibility made the algorithm teachable not only through prose and equations but also through live signal flow.

Commercial workstations and plug-ins translated research into source categories and quality settings. Names such as monophonic, polyphonic, rhythmic, speech, transient, or formant-preserving hid combinations of algorithms behind production language. This was useful for users, but it made exact genealogy less transparent. Two products could both advertise high-quality stretching while using substantially different mixtures of phase propagation, waveform alignment, transient slicing, and resynthesis.

Open-source libraries later supplied reusable primitives for STFT processing, resampling, onset detection, and pitch analysis. Rubber Band, SoundTouch, librosa, SuperCollider extensions, and many research repositories widened access, though not all are phase vocoders and not all pursue the same quality goals. Their

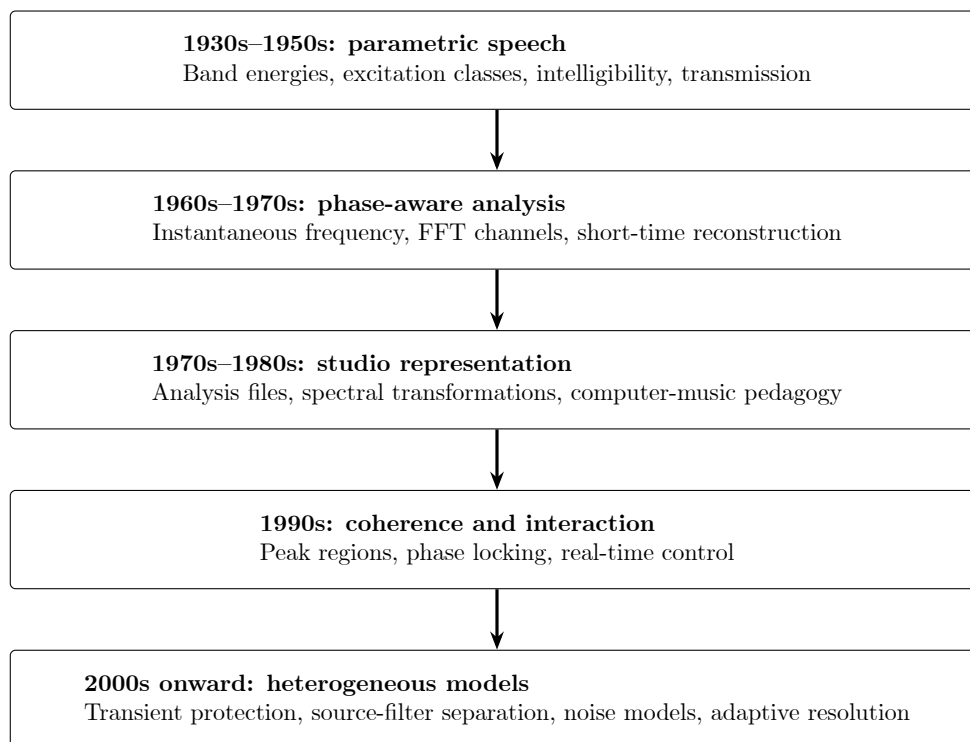
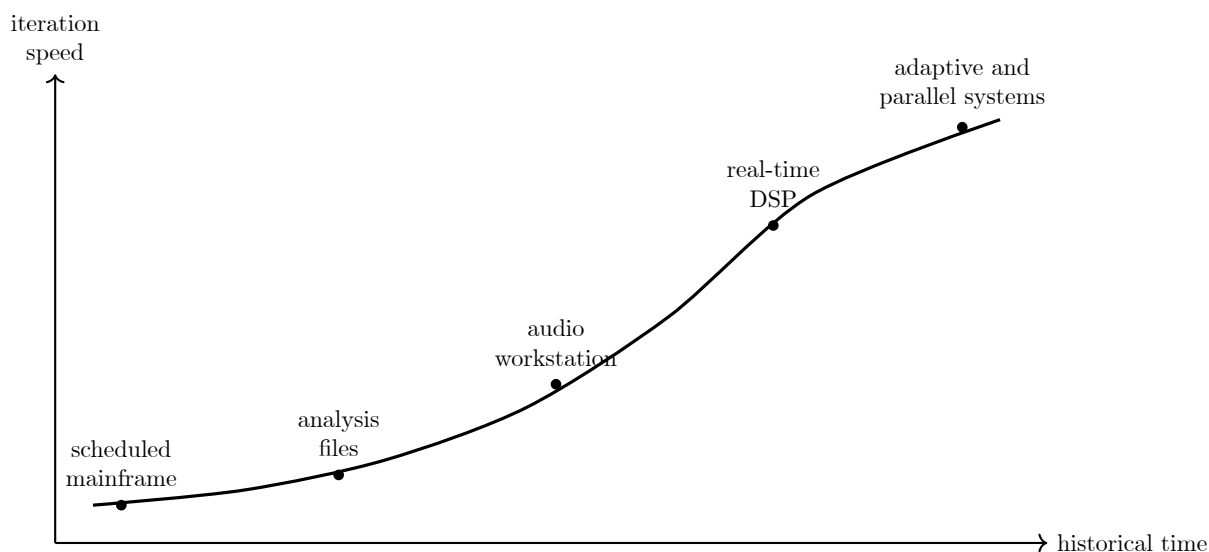


Figure 1.16: Successive research eras enlarged the set of sound properties represented explicitly. Earlier questions survived inside later systems rather than being discarded.



The curve is conceptual, not a benchmark. It records a change in the practical interval between proposing a transformation and hearing it.

Figure 1.17: The shrinking audition loop altered compositional method as profoundly as increases in nominal signal quality.

importance lies partly in testability. Implementations can be compared, modified, and incorporated into reproducible systems.

pvx inherits several branches of this history. Its command-line form recalls CARL and CDP. Its explicit analysis and transform stages recall laboratory pipelines. Its concern with stable manifests and resumable work responds to long offline renders. Its phase, transient, formant, and multichannel policies reflect later perceptual research. The result is not a museum reconstruction. It is a contemporary arrangement of ideas whose origins remain legible.

1.3.33 The analysis file as a new kind of score

The intermediate analysis file deserves a history of its own. It changed a recording from a fixed waveform into a collection of trajectories that could be revised without repeating the original performance. For composers, this was comparable to gaining a score for properties that conventional notation rarely describes: partial amplitude, local frequency, spectral envelope, noise distribution, and the evolution of a transient.

The analogy must not be taken too literally. An analysis is not neutral transcription. Window length determines what counts as local. Hop size determines the density of observations. Peak thresholds decide which components appear stable. A file format determines whether phase, frequency, or only magnitude survives. Every analysis is already an interpretation.

Still, persistence changes creative work. A user can inspect one frame, draw an envelope, transpose only a region, or use the amplitude pattern of one source to reshape another. Operations can be chained while keeping the original analysis intact. Several resyntheses can be understood as readings of the same document.

Analysis files also create archival problems. A waveform is comparatively self-describing when its sample rate, channel count, and encoding are known. Spectral data requires more context: transform size, window definition, hop, frame centering, normalization, phase convention, units, channel policy, and software version. Without these, a file may remain numerically readable while losing its musical meaning.

The history of formats such as phase-vocoder analysis files and SDIF shows attempts to make representations portable. Portability is partly social. A format succeeds when tools agree on semantics, documentation is maintained, and users trust that old work can be reopened. A formally elegant container without active readers may preserve less history than a plain but widely understood file.

The score analogy also clarifies authorship. An analysis may derive from a performer's recording, be transformed by a composer, rendered by a program, and revised by later software. The output embodies decisions at every stage. Good provenance does not settle all artistic questions, but it prevents the machine from appearing as an anonymous source of sound.

1.3.34 Compositional histories beyond the canonical examples

The most responsible musical history distinguishes direct documentation from stylistic proximity. Many late twentieth-century works engage spectral analysis, prolonged sound, resynthesis, or electronic transformation. Not all use a phase vocoder, and similarity of sound is not proof. The wider repertory is nevertheless important because it established the listening habits through which phase-vocoder transformations became musically intelligible.

Jean-Claude Risset's computer-music research demonstrated that acoustic analysis could inform synthesis and composition. His studies of trumpet tones and his paradoxical pitch and rhythm effects made perceptual organization a compositional parameter. Risset's work is broader than phase-vocoder history, but it helped create the intellectual world in which an analyzed sound could become a model rather than merely a recording.

Paul Lansky's tape and computer works explored speech, language, and the boundary between recognition and abstraction. Processes in this repertory include linear predictive coding, granular procedures, and other computer transformations, depending on the work. The relevance is methodological. Speech is heard both as a carrier of meaning and as structured sound, a double attention central to the earlier communications history of the vocoder.

Denis Smalley’s writing on spectromorphology supplied a vocabulary for how spectral content and temporal shape behave in electroacoustic music. It is not an implementation manual and does not prescribe a phase vocoder. Its historical importance lies in giving listeners and composers terms for onset, continuation, termination, motion, texture, and source bonding. These categories help describe why two mathematically similar stretches can function differently.

Horacio Vaggione’s work foregrounded composition across multiple time scales. Microsonic detail, event structure, and larger form were treated as connected levels rather than separate domains. Phase-vocoder analysis likewise makes local spectral evolution available to larger compositional planning. The connection is conceptual, and it should not be mistaken for a claim that one algorithm defines Vaggione’s output.

Kaija Saariaho’s electroacoustic and instrumental practice at IRCAM developed within a culture of spectral analysis, timbral interpolation, and computer-assisted composition. Works such as *Verblendungen*, *Lichtbogen*, and *Nymphaea* occupy a historical environment where spectra could inform harmony, orchestration, and electronics. Documentation for an individual work must determine which processes were actually used. The broader significance is the integration of analysis into a composer’s sustained language.

Tristan Murail and other composers associated with spectral music likewise transformed the cultural meaning of a spectrum. A spectrum could generate harmony, formal direction, orchestration, and processes of fusion or separation. Phase-vocoder technology is not synonymous with spectral composition, but both challenge the idea that timbre is a surface placed on independently conceived notes.

Curtis Roads explored granular and microsound methods that often provide alternatives to phase-vocoder stretching. His work matters here because it made the size and scheduling of sound particles audible as compositional choices. Comparing a granular prolongation with a phase-coherent spectral prolongation teaches more history than declaring one method universally superior.

Barry Truax developed real-time granular synthesis and composed extensively with environmental sound. His practice offers another neighboring history of duration change, source recognition, and texture. The contrast is instructive. Granular methods foreground event particles and density, while a phase vocoder foregrounds evolving spectral channels or peaks. Many modern processors borrow from both.

Natasha Barrett’s electroacoustic and spatial work demonstrates how transformation history extends into multichannel composition. Once processed sound moves through an immersive field, spectral coherence interacts with localization, distance, and room impression. The phase vocoder’s mono origins are no longer sufficient. Spatial behavior becomes part of whether a transformation preserves identity.

These neighboring practices prevent a narrow great-inventor narrative. The algorithm developed because engineers solved technical problems, but its musical meaning developed across studios, concerts, teaching, criticism, and repeated listening. Composers who selected, rejected, combined, or deliberately exposed its artifacts participated in that history even when they did not publish an equation.

1.3.35 Artifact history as listening history

Every period heard phase-vocoder quality through the expectations of its own media. Early speech researchers prioritized intelligibility and channel economy. A synthetic voice could sound unnatural yet count as a success if words survived transmission. Computer musicians often valued recognizability under transformations far beyond ordinary speech needs. Production users later expected stretched material to sit invisibly inside a polished mix.

The word *phasiness* is itself historical evidence. It gathers several impressions: diffuseness, loss of focus, a reverberant halo, weakened attacks, or chorus-like instability. The term became useful before every mechanism was agreed upon. Researchers then separated horizontal phase continuity from vertical phase relations and proposed peak-based locking. Listening vocabulary helped organize mathematical inquiry.

Transient smear followed a similar path. A stretched attack could sound doubled, softened, or preceded by a faint cloud. Waveform inspection showed why a long analysis window distributed an abrupt event. Detection and phase reset became explicit remedies. Later systems separated transient energy or used alternate resolutions. What began as a complaint became an architectural branch.

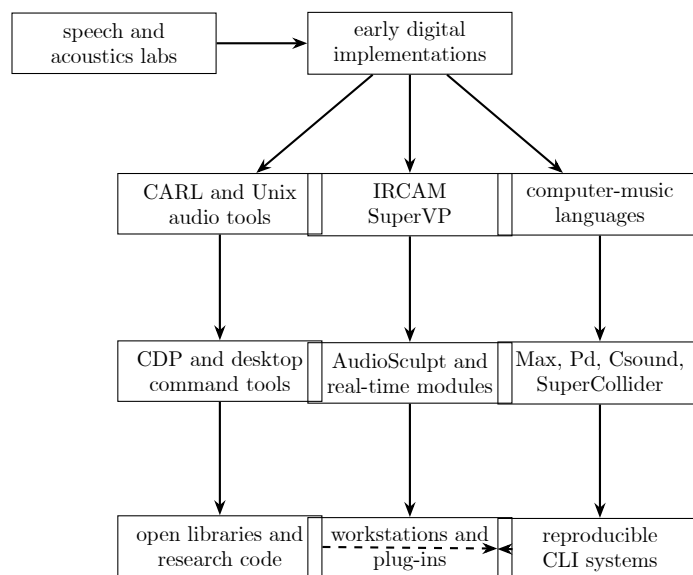


Figure 1.18: A conservative software genealogy. Solid lines indicate broad institutional or interface continuities; dashed lines indicate contemporary exchange rather than direct descent.

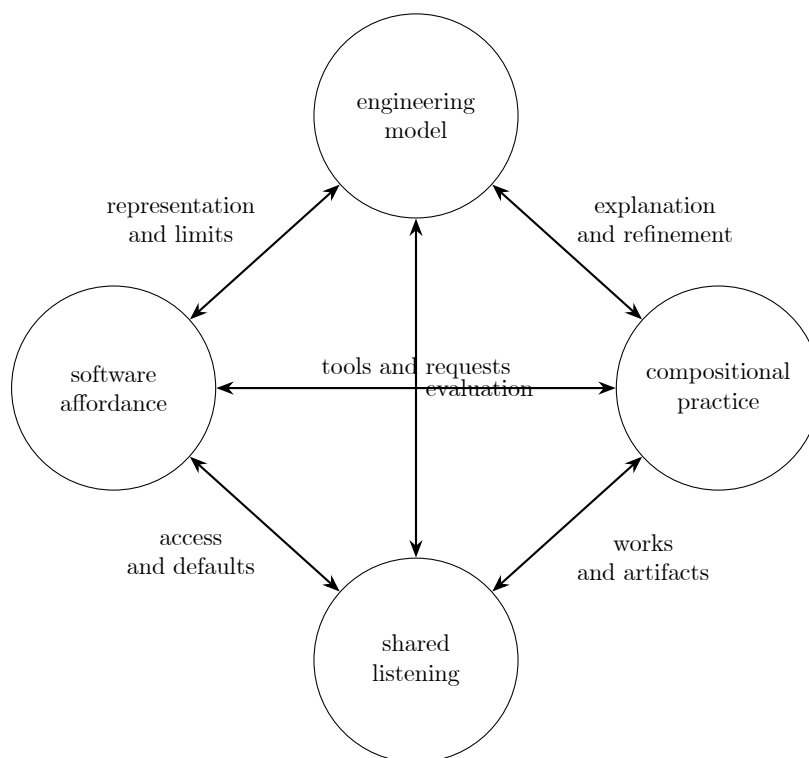


Figure 1.19: Musical technology develops through feedback among models, implementations, artistic practice, and learned perception.

Metallic noise revealed that a sinusoidal account had been extended too far. Random components acquired deterministic repetition or frozen phase relations. Hybrid sinusoidal and stochastic models responded by representing the residual differently. The artifact taught developers that coherence can be excessive as well as insufficient.

Stereo image motion exposed another hidden assumption. Two channels that sounded stable before processing could drift when analyzed independently. The historical response included shared peak decisions, reference phases, and constraints on interchannel relations. Once again, a perceptual failure revealed information that the original model had not represented.

Artifacts also became genres of sound. Long-window smearing, frozen spectra, phase randomization, and metallic extension were cultivated in ambient, electroacoustic, experimental, and popular production. A correction in one context could destroy the desired effect in another. Mature tools therefore preserve both transparent and exposed modes rather than assuming that historical progress means eliminating every trace of the process.

This double status complicates evaluation. A test for speech naturalness cannot rank an intentional spectral drone. A preference test may obscure whether listeners value fidelity, novelty, rhythm, source identity, or reduced roughness. Historical listening requires the criterion to be stated before the score is interpreted.

1.3.36 Teaching, diagrams, and the public life of the algorithm

The phase vocoder spread through diagrams as much as through source code. A bank of filters with envelope and phase paths made the communications model visible. A row of overlapping windows made short-time analysis visible. A spiral of accumulated phase made continuity visible. Peak-centered groups made phase locking visible. Each diagram selected what a reader should imagine the algorithm to be.

Textbooks often begin from Fourier analysis and proceed toward implementation. Computer-music tutorials frequently begin from an audible task, such as stretching a voice without changing its pitch. Patch-based lessons begin from signal objects and connections. Command-line manuals begin from files and options. These routes produce different intuitions even when they reach related mathematics.

The most successful explanations move among representations. A waveform shows timing, a spectrogram shows energy distribution, a phase plot shows continuity, and an audio example shows the perceptual result. No single view is sufficient. The history of pedagogy is therefore also a history of media for explanation.

Mailing lists, workshops, course handouts, and online examples carried practical knowledge omitted from papers. Users learned which windows behaved well, how much overlap a particular implementation expected, why boundaries clicked, and when a phase-locking mode harmed drums. Such advice is difficult to archive, yet it often determines whether an algorithm becomes usable.

The current abundance of tutorial videos and code repositories makes access easier while creating a verification problem. A demonstration may call any spectral effect a phase vocoder. A code sample may reconstruct audio while silently violating overlap normalization. Historical literacy gives the user a way to ask what representation, phase rule, and reconstruction policy are actually present.

For that reason, this book treats history as practical equipment. Knowing why a control exists helps predict what it will do. Knowing which problem a method was designed to solve helps identify when not to use it. Knowing that a famous composition combined several processes prevents a listener from mistaking one timbre for a universal algorithmic signature.

1.3.37 Bell Laboratories as a long technical precondition

Bell Laboratories appears repeatedly in this history because the phase vocoder depended on more than one isolated discovery. The laboratory joined telephone engineering, psychoacoustics, speech science, electronics, and later digital computation. Problems could move among departments and persist across decades. A device for compressing speech bandwidth, a study of vocal production, and a mathematical description of a filter bank belonged to a common institutional world.

The telephone system gave research an unusual scale. Speech was not an occasional laboratory signal. It was the content of a national and international infrastructure. Improvements in intelligibility, bandwidth, switching, coding, and noise performance could have immense practical value. This sustained investment in how speech survives technical mediation.

Homer Dudley's work illustrates the resulting breadth. The channel vocoder was at once a communication system and a model of speech. The Voder was at once a demonstration machine and a demanding human interface. Later secure-speech systems turned parametric coding into synchronized infrastructure. These projects established analysis-synthesis as a serious engineering practice long before musicians had routine digital access.

Flanagan and Golden's phase vocoder belongs to that accumulated practice. Their formulation assumes a reader comfortable with speech channels, phase, modulation, and reconstruction. The new representation was radical, but its questions were inherited: what information should pass through the system, how accurately can a source be reconstructed, and which changes remain intelligible?

Bell Labs also influenced computer music through figures whose work crossed research and art. Max Mathews developed foundational computer-music systems there. Jean-Claude Risset conducted important analysis and synthesis research in that environment. The institution did not simply hand telecommunications tools to composers. It helped create a setting in which digital sound itself became a research material.

The lesson is institutional rather than celebratory. Long-lived research environments can support ideas whose applications are not foreseeable at the start. The phase vocoder required mathematics, hardware, speech data, engineering practice, and enough continuity for these to meet. Its history warns against narratives that credit only the final paper while ignoring the infrastructure that made the paper thinkable.

1.3.38 Stanford, CCRMA, and model-based sound

Stanford's computer-music history provides another route from acoustic description to musical transformation. Work associated with the Center for Computer Research in Music and Acoustics developed synthesis, physical modeling, signal processing, and digital musical systems. The phase vocoder was one tool in a wider inquiry into how sound could be represented for both analysis and composition.

Sinusoidal modeling became especially important in this setting. Julius O. Smith, Xavier Serra, and collaborators developed analysis-synthesis methods that tracked deterministic partials and represented remaining energy as a stochastic residual. The resulting systems encouraged a source model richer than a uniform field of bins.

This model changed what a transformation could preserve. A tracked partial had a trajectory and could be continued, modified, or ended as an entity. A residual could retain noise character without pretending to be a collection of long-lived sinusoids. The distinction was particularly useful for musical sounds that mixed stable resonance with breath, scrape, or attack.

The relationship to the phase vocoder is reciprocal. Peak tracking and sinusoidal interpretation informed later phase-locking approaches. Phase-vocoder infrastructure supplied efficient short-time spectra from which peaks could be found. Hybrid systems increasingly treated the argument between dense bins and tracked sinusoids as a design choice within one processor.

Stanford's role also demonstrates the value of public technical writing. Reports, course materials, software descriptions, and later online books made advanced signal-processing ideas available well beyond one laboratory. An institution contributes to history not only by inventing a method but by maintaining a language through which others can implement and challenge it.

1.3.39 UCSD, CARL, and the command-line studio in detail

The CARL environment deserves close attention because its working model resembles contemporary reproducible audio tooling. Rather than hiding every stage in one application, Unix programs could analyze, transform, synthesize, inspect, and convert data. The shell became a compositional surface.

This modularity changed experimentation. A user could retain an expensive analysis, run several transformations from it, and compare outputs. A script could document a sequence more precisely than handwritten recollection. Programs could be chained in combinations their authors had not anticipated. The operating system supplied a general composition mechanism.

Modularity also made formats consequential. A tool had to know how frames, channels, and metadata were organized. If one program interpreted frequency as hertz and another expected phase radians, the pipeline failed semantically even if every file opened. Shared formats were therefore social contracts among developers and users.

Mark Dolson's work belongs to this culture of research through implementation. The tutorial is often encountered as a self-contained text, but its practical intelligence reflects experience with working systems and musical material. The account of transformations is persuasive because it connects representation to operations a composer might actually perform.

The command-line studio could be austere. Users needed to understand files, names, paths, parameter ranges, and processing order. Yet that effort produced a durable benefit: procedures were externalized. A session could be rerun after the sound had been forgotten, and a surprising result could be traced to its commands.

Modern interfaces often oppose usability to explicitness, but the CARL history suggests a more productive balance. A tool can provide safe defaults and still expose a complete invocation. It can explain a parameter in perceptual language while preserving its exact value. `pvx`'s manifests and stable commands pursue this balance rather than treating the shell as nostalgia.

1.3.40 IRCAM as a meeting of research and commission

IRCAM's importance comes partly from the proximity of long-term technical research to concrete artistic projects. Composers arrived with difficult requests, researchers developed representations and algorithms, and software teams turned those results into systems that could survive beyond one premiere. The phase-vocoder lineage became one strand within a larger ecology of analysis, synthesis, spatialization, and computer-assisted composition.

Commissioned works supplied demanding tests. A process that succeeded on a laboratory speech sample might fail on a bell, a flute multiphonic, or a whispered consonant. A composer might value an artifact that a speech engineer would reject. Artistic requirements widened the domain over which an algorithm had to be understood.

The progression from command-line or batch systems to AudioSculpt made spectral processing visually situated. A user could correlate a heard event with a region in a spectrogram, select it, and apply a treatment. This changed access while preserving the underlying analysis. The display became an interpretive layer between mathematical data and musical gesture.

SuperVP's continuing development shows that a phase-vocoder engine can become a platform for many related models. Time stretching and transposition coexist with source-filter processing, envelope control, transient handling, noise and sinusoidal treatment, cross-synthesis, and real-time operation. The name persists even as the internal system becomes more heterogeneous than the classic algorithm.

IRCAM Forum distribution and Max integrations widened the community around these tools. Workshops and documentation taught not only commands but listening strategies. Users carried techniques into universities, studios, installations, and commercial production. Institutional history thus extends through education and licensing as well as papers.

The archive of works and technical notes also permits unusually careful reconstruction. For Harvey's *Mortuos Plango, Vivos Voco* or Reynolds's *Transfigured Wind*, one can compare compositional statements, source descriptions, algorithm names, and audible results. This evidence helps resist the habit of attributing an entire work to one famous process.

1.3.41 Speech as the first difficult material

Speech drove much of the early research because it combines strict perceptual demands with complex acoustics. A listener notices disruptions in timing, pitch, consonant articulation, vowel identity, rhythm, and speaker character. The source changes quickly, yet it also contains stable harmonic and resonant structures.

Voiced speech invites a sinusoidal account. Vocal-fold pulses create a harmonic series, while the vocal tract shapes broad resonances. A time-scaling system should preserve local pitch while changing the schedule of syllables. A pitch shifter may need to move the harmonic source while retaining the vocal-tract envelope. These requirements made phase continuity and formant treatment practical necessities.

Unvoiced consonants resist the same model. Fricatives contain turbulent noise. Stops contain closures and abrupt releases. Aspirated sounds mix noise with voicing. A long stationary window can blur these events, while aggressive phase locking can make noise unnaturally coherent.

Prosody adds a larger time scale. Speech is not a line of independent frames. Pitch contours, stress, pauses, and coarticulation establish phrases. Expanding every local interval uniformly may be mathematically faithful yet rhetorically strange. Practical speech modification often requires event-aware timing as well as spectral reconstruction.

Early evaluation emphasized intelligibility, but naturalness and identity became increasingly important. A system could preserve every word while making the speaker sound distant, robotic, smaller, or emotionally altered. These outcomes encouraged perceptual tests and source-specific algorithms.

Musicians inherited all of these tensions. A voice may be stretched until words disappear while breath and resonance remain. A formant shift may deliberately change apparent body size. A consonant can become a percussive event. Speech research provided the problems, while composition expanded the acceptable answers.

1.3.42 Sustained instrumental sound and the discovery of coherence

Sustained pitched instruments initially appear ideal for a phase vocoder. Their partials persist across many frames, and large time changes can preserve a recognizable pitch. Yet these sounds made vertical incoherence painfully audible. A flute or string tone could become diffuse even when every bin followed a continuous phase path.

The reason lies in how a window represents one partial. Energy spreads across nearby bins according to the window's frequency response. Those bins are not independent physical oscillators. They are samples of one local spectral structure. If their phases evolve independently, the reconstructed waveform loses the organization associated with the original partial.

Phase locking responds by identifying a spectral peak and coordinating nearby bins. Identity locking preserves the relative relation within the peak region, while scaled approaches adjust those relations under transformation. The exact strategy differs among implementations, but the historical advance is shared: a peak neighborhood becomes a unit.

Vibrato complicates the picture. A partial moves in frequency, crossing the fixed grid. Peak tracking must decide whether adjacent observations belong to one trajectory. Too little continuity produces jitter; too much can force unrelated components together. Ensemble sounds make the assignment more difficult because several instruments contribute close or crossing partials.

This is why Dolson's work on tracking and ensemble analysis matters. It directs attention away from a static-bin ontology toward moving components. Later sinusoidal and hybrid models elaborate the same insight.

For composition, improved coherence expanded the range between transparency and abstraction. A sustained note could be prolonged while retaining presence, or its peak groups could be deliberately loosened into a cloud. The artifact became controllable because its cause was better understood.

1.3.43 Percussion and the historical challenge of the onset

Percussive sound reverses many assumptions that favor sustained tones. An attack is brief in time and broad in frequency. Its identity may depend more on the alignment of energy than on stable partial trajectories. A large Fourier window sees spectral detail but spreads the event across its own duration.

Ordinary time scaling moves synthesis frames to new positions. If several adjacent analysis frames contain pieces of one attack, spacing those frames farther apart distributes the attack energy. The result can be a soft lead-in, a doubled strike, or a reverberant tail that was not present in the source.

Transient detection attempts to mark the event before it is stretched. A processor may reset synthesis phase, preserve the local timing of transient frames, route the event through a time-domain method, or use a shorter window. Each remedy protects one property while risking another. A reset can cause discontinuity; a short window reduces frequency resolution; a detector can miss soft or gradual attacks.

Drums also contain resonant tails. The initial strike may need transient handling while the shell or cymbal decay benefits from spectral continuation. This mixed behavior encouraged within-event classification rather than one preset for the whole file.

Rhythmic material adds metrical expectations. A tiny onset shift can weaken groove even if the spectrum sounds clean. Evaluation therefore includes event timing, not just waveform similarity or spectral distance. Music information retrieval tools for onset detection became relevant to audio effects.

Historically, percussion kept the phase vocoder honest. It prevented improvements on speech and steady tones from being mistaken for universal quality. It also accelerated hybrid processing, one of the defining characteristics of modern stretching systems.

1.3.44 Noise, ambience, and environmental recordings

Environmental recordings combine events, textures, reverberation, and spatial cues. Wind may behave as colored noise, birds as moving partials, footsteps as transients, and a room as a decaying multichannel field. No single phase policy describes the whole recording.

Noise has statistical identity. Its exact waveform can change while its distribution remains perceptually similar. A phase vocoder that preserves the wrong details may produce a frozen or metallic texture. Noise-aware synthesis instead attempts to preserve energy, color, modulation, and decorrelation.

Reverberation poses a related problem. A room tail contains dense reflections whose phase relations support spaciousness but are not equivalent to one stable sinusoidal source. Independent channel processing can alter width and localization. Excessive phase locking can collapse diffuse energy toward a point.

Environmental composition often values source recognition. A stretched wave or insect may remain identifiable long after ordinary timing has disappeared. Alternatively, an apparently abstract texture may suddenly reveal its source through one unprocessed transient. The composer manages recognition as a formal parameter.

Field recordings also carry documentary and ethical context. Transformation can detach a sound from place, person, or event. Metadata and source notes become part of responsible practice, especially when a recording includes voices or culturally specific material. Technical provenance and cultural provenance meet in the archive.

For pvx, such material argues for automation that can vary through a file. A transient region, a stable call, and a diffuse background need not share one strategy. Time-varying control is not an ornamental feature. It is a response to the heterogeneous history of recorded sound.

1.3.45 Extreme duration and the change of listening scale

Moderate time scaling asks whether a performance can remain natural at a new duration. Extreme scaling asks a different question: what hidden structure appears when ordinary time ceases to govern perception? A short gesture can become a landscape. A modulation once heard as timbre can become rhythm.

This change of scale has precedents in tape slowing, granular synthesis, and studio feedback, but the phase vocoder offers distinctive control over pitch and spectral continuity. It can prolong a sound without the obligatory pitch descent of varispeed. The result may retain a spectral identity while abandoning the source's original gesture.

At large ratios, details ignored in ordinary playback become structural. Vibrato becomes a wide oscillation. A room reflection becomes a separate event. Quantization noise, background hum, and microphone motion become audible layers. The distinction between source and recording apparatus weakens.

Extreme stretching also magnifies algorithmic assumptions. Frame repetition becomes rhythm, phase errors become spatial haze, and a transient detector's decisions become formal cuts. A setting that is transparent at a ratio near one may produce strong periodicity at a ratio of hundreds or thousands.

Composers including JoAnn Kuchera-Morin and many later ambient and experimental artists treated this behavior as material rather than defect. The practice widened public awareness of phase-vocoder sound, though popular examples sometimes use other or hybrid stretch algorithms. Historical accuracy again requires listening and documentation rather than inference from duration alone.

A millionfold stretch reaches beyond ordinary rendering into installation, data, and archival questions. Output size and time become dominant. The process may require stages, checkpoints, sparse audition, and alternate representations. At that scale, software architecture is part of the composition.

1.3.46 Preservation, reconstruction, and obsolete systems

Historical phase-vocoder work is unusually vulnerable because the meaningful object may be distributed across several media. A source recording, an analysis file, transformation parameters, custom software, and a final tape can each preserve different parts of the process. Losing one may prevent exact reconstruction.

Obsolete analysis formats are a particular risk. Even when bytes survive, the reader may not know the window, hop, phase convention, scaling, or header semantics. Reimplementing an old synthesizer from a paper may produce a plausible sound without reproducing the original program's rounding, interpolation, or boundary behavior.

Hardware contributes another layer. Converter quality, sample rate, tape transfer, and analog studio stages influenced historical outputs. A mathematically exact modern rerender may sound cleaner while being less faithful to the work as realized. Preservation must distinguish the algorithmic plan from the historical artifact.

Composer archives can resolve some questions through sketches, job logs, source lists, tapes, and correspondence. Reynolds's preserved materials are valuable for precisely this reason. They show a work as a sequence of choices rather than an unexplained audio file.

Software preservation may use source archives, virtual machines, emulation, test fixtures, and rendered reference examples. No one method is sufficient. Source code without a compiler may be unreadable in practice; a binary without documentation may run but remain uninterpretable; a reference render without parameters preserves sound but not method.

Contemporary projects can learn from these losses. A manifest should record versions and units. Analysis artifacts should identify their schema. Tests should preserve expected behavior for representative sources. Documentation should say which public surface is stable. These habits are not administrative extras. They are the conditions under which future users can understand what present users heard.

1.3.47 A decade-by-decade change in the imagined user

In the 1960s, the implied phase-vocoder user was a communications researcher or engineer. The system was described through channels, modulators, phase derivatives, and speech examples. Access depended on a major laboratory. A successful output demonstrated analysis and resynthesis more than everyday creative convenience.

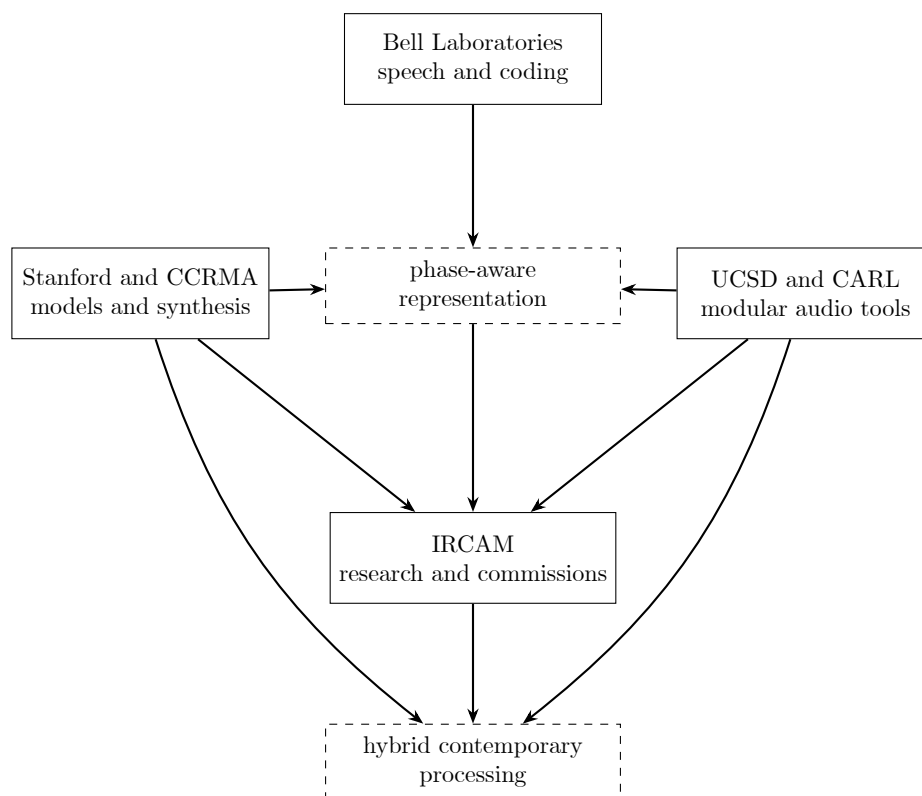


Figure 1.20: Institutions contributed different working emphases. The diagram records exchange and convergence, not exclusive ownership of ideas.

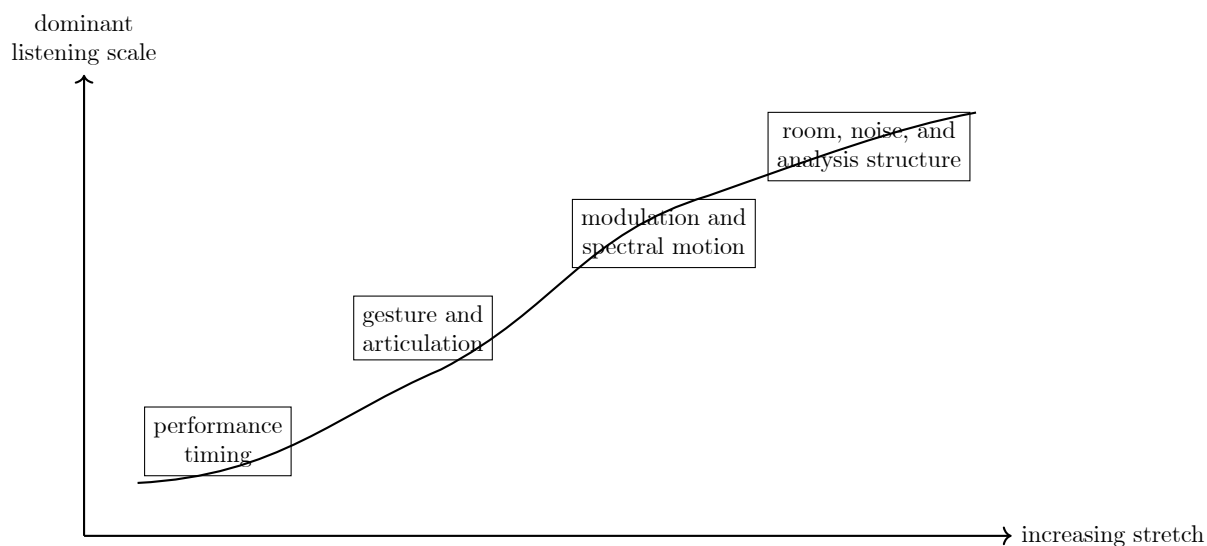


Figure 1.21: Extreme stretching changes which temporal layer behaves as musical form. The boundaries are material-dependent rather than fixed ratio thresholds.

In the 1970s, the implied user increasingly included a programmer working with digital signal-processing systems. FFT implementations and short-time analysis papers made the method more concrete. The user still needed specialized computing, but the algorithm could be described as repeatable operations on arrays.

In the 1980s, the computer musician became a central figure. Tutorials, institutional software, and landmark compositions made spectral transformation a recognizable studio practice. The user might wait for offline jobs, maintain analysis files, and collaborate closely with researchers. Time expansion became composition, not only signal correction.

In the 1990s, the implied user widened toward the interactive musician and desktop studio. Personal computers, graphical systems, Max, Pure Data, Csound, SoundHack, CDP, and improved workstations reduced institutional barriers. Research focused increasingly on audible artifacts, phase locking, pitch shifting, and practical musical quality.

In the 2000s, the production user expected real-time or near-real-time results, visual editing, source presets, and integration with larger sessions. Transient preservation, formants, and comparative evaluation became prominent. The algorithm often disappeared behind an application mode.

In the 2010s, phase-vocoder ideas circulated through open libraries, mobile processors, web demonstrations, DJ systems, games, and large research ecosystems. Hybrid methods became normal. Users selected outcomes while engines selected among multiple internal policies.

In the 2020s, learned models increasingly surround classical transforms. Separation, pitch estimation, transcription, classification, and neural synthesis can occur before or after a phase-vocoder stage. At the same time, reproducible command-line tools and open research code keep explicit algorithms valuable.

The imagined user is now plural. A dialogue editor wants transparent duration correction. A composer wants unstable spectral matter. A researcher wants inspectable intermediate data. A performer wants low latency. An archivist wants deterministic reconstruction. A durable tool must state which of these promises it supports rather than hiding incompatibilities behind the word *quality*.

1.3.48 Annotated chronology

The following chronology is selective. It emphasizes events that changed representation, implementation, dissemination, or musical use. Dates of compositions indicate completion or first realization where generally documented; software lineages often span several years.

Table 1.3: Selected chronology of phase-vocoder history and its neighboring lineages.

Date	Person, work, or system	Historical significance
1822	Joseph Fourier	Published the analytical theory of heat, helping establish harmonic decomposition as a general mathematical language.
1863	Hermann von Helmholtz	Connected partial structure, resonance, pitch, and tone sensation through acoustical experiments and theory.
1920s-1930s	Homer Dudley	Developed channel-vocoder analysis and synthesis at Bell Telephone Laboratories.
1939	Voder demonstration	Presented manually controlled speech synthesis to a mass public at the New York World's Fair.
1940s	SIGSALY	Demonstrated large-scale secure speech coding, synchronization, and reconstruction.
1950s-1960s	Tape rate changers	Used segmentation and rotating heads to alter speech duration with reduced pitch change.
1965	Cooley and Tukey	Published an influential fast Fourier transform algorithm suited to repeated digital spectral calculation.
1966	Flanagan and Golden	Published *Phase Vocoder*, presenting phase-aware analysis, transmission, resynthesis, and time modification.

Date	Person, work, or system	Historical significance
1976	Mark Portnoff	Published an FFT implementation of the digital phase vocoder.
1977	Allen and Rabiner	Unified short-time Fourier analysis and synthesis in a widely cited signal-processing account.
1978	James A. Moorer	Described phase-vocoder use in computer-music applications.
1980	Jonathan Harvey	Completed <i>*Mortuos Plango, Vivos Voco*</i> , linking source spectra, synthesis, and musical form at IRCAM.
1980	Mark Portnoff	Published short-time-Fourier-based time-scale modification of speech.
Early 1980s	Mark Dolson	Investigated tracking phase vocoders, ensemble analysis, software, and musical transformations.
1984	Griffin and Lim	Published iterative signal estimation from modified short-time Fourier magnitude.
1984-1985	Roger Reynolds	Developed the <i>*Transfigured Wind*</i> series with transformed flute materials and computer processes.
1986	Mark Dolson	Published <i>*The Phase Vocoder: A Tutorial*</i> , consolidating theory and musical applications.
1986	McAulay and Quatieri	Published influential sinusoidal speech analysis-synthesis based on tracked components.
1986	Trevor Wishart	Realized <i>*Vox 5*</i> , a canonical work of vocal spectral transformation and morphing.
1989	JoAnn Kuchera-Morin	Realized <i>*Dreampaths*</i> , noted for extensive time and spectral transformation.
1990	Serra and Smith	Presented spectral modeling synthesis using deterministic and stochastic decomposition.
1993	Verhelst and Roelands	Published WSOLA, an influential waveform-similarity method for time-scale modification.
1995	Miller Puckette	Presented a phase-locked vocoder approach for improved spectral coherence.
1997	Laroche and Dolson	Framed phasiness as a vertical-coherence problem in phase-vocoder output.
1999	Laroche and Dolson	Published improved time-scale modification using identity and scaled phase locking.
1999	Laroche and Dolson	Published new phase-vocoder techniques for pitch shifting, harmonizing, and related effects.
2002	Duxbury, Davies, Sandler	Developed transient-aware phase-locking approaches for musical time scaling.
2003	Axel Roebel	Presented new transient-processing approaches within the phase vocoder.
2005	Roebel and Rodet	Published efficient spectral-envelope estimation for pitch shifting and envelope preservation.
2006	Karrer, Lee, Borchers	Presented PhaVoRIT for real-time interactive time stretching and perceptual comparison.
2007	Bradford, Dobson, fitch	Presented a sliding phase-vocoder structure oriented toward continuous updates.
2010s	SuperVP extensions	Consolidated high-quality time, pitch, envelope, transient, noise, and cross-synthesis processing in research and professional tools.

Date	Person, work, or system	Historical significance
2016	Driedger and Muller	Published a broad review of music time-scale-modification methods and evaluation concerns.
2017	Ottosen and Dorfler	Developed a phase vocoder based on nonstationary Gabor frames.
2017	Juillerat and Hirsbrunner	Presented adaptive multiresolution time stretching with magnitude correction.
2023	Akaishi, Yatabe, Oikawa	Applied time-directional spectrogram squeezing to improve long phase-vocoder stretches of percussive material.

1.3.49 Reading the history through software design

The chronology can be translated into design principles. Filter-bank history explains why band behavior and spectral envelopes matter. Tape history explains why local units, joins, and attacks matter. FFT history explains why frame size and hop are exposed. Computer-music history explains why intermediate representations and automation matter. Coherence research explains phase-locking controls. Transient research explains hybrid modes.

A modern command therefore contains decades of argument. Choosing a Hann window invokes a history of spectral measurement and overlap. Choosing identity phase locking invokes the explanation of phasiness developed by Puckette, Laroche, and Dolson. Choosing transient preservation invokes the recognition that stationary sinusoidal assumptions fail at attacks. Choosing formant preservation invokes the source-filter tradition.

This perspective prevents parameter lists from becoming arbitrary. Options are not decorations around one timeless algorithm. They are answers to historical failures, extensions, and artistic demands. Some answers conflict, which is why a program needs policies rather than one universal maximum-quality switch.

The design of pvx continues that lineage through explicit commands, reproducible settings, checkpoints, and analysis artifacts. Its contribution is not to erase history behind a button. It is to make historical choices available in a coherent working environment.

1.3.50 Source notes for the expanded history

The principal technical lineage in this account is documented by Flanagan and Golden’s 1966 paper, Portnoff’s 1976 and 1980 papers, Moorer’s 1978 computer-music article, Dolson’s 1986 tutorial, Puckette’s 1995 phase-locked vocoder, Laroche and Dolson’s 1997 and 1999 work, Roebel’s transient and spectral-envelope research, and later multiresolution studies. Full bibliographic records appear in the Bibliography.

Musical and institutional accounts rely on composer writings and archives where possible. Trevor Wishart’s retrospective describes his long engagement with phase-vocoder data and morphing. The Library of Congress genealogy of Roger Reynolds’s *Transfigured Wind* documents source recordings, algorithms, plans, and versions. IRCAM’s research and software documentation describes SuperVP, AudioSculpt, and later real-time modules. IRCAM’s analytical account of *Mortuos Plango, Vivos Voco* distinguishes FFT analysis, Music V, synthesis, and transformations used in Harvey’s work.

Claims about JoAnn Kuchera-Morin’s *Dreampaths* are supported by composer records and secondary technical literature including Curtis Roads’s discussion of extensive phase-vocoder transformations. Software genealogy is stated conservatively because local versions, ports, and unpublished changes complicate simple lines of descent.

This chapter uses *phase vocoder* narrowly for phase-aware short-time analysis and resynthesis, while acknowledging neighboring channel-vocoder, sinusoidal-modeling, overlap-add, granular, and neural-vocoder histories. That terminological discipline is necessary because the same word *vocoder* now names substantially different systems.

1.3.51 What the historical work leaves us

The historical line from Dudley through Flanagan, Golden, Portnoff, Dolson, Laroche, and later researchers leaves three durable lessons. First, representation controls imagination: once amplitude and phase trajectories are explicit, time and pitch become editable structures. Second, efficiency changes art: the FFT turned a laboratory model into a repeatable process, and modern hardware turns it into a live or corpus-scale operation. Third, perceptual quality depends on relationships. A bin is not heard alone; it belongs to a peak, a frame, a transient, a channel, and a musical event.

pvx inherits all three lessons. It treats the phase vocoder as a practical engine, a controllable source of artifacts, and a framework around which automation, analysis, checkpointing, and reproducibility can be built. The rest of the book moves from this history into implementation detail, but the listening problem remains the same: decide which relationships must survive the transformation.

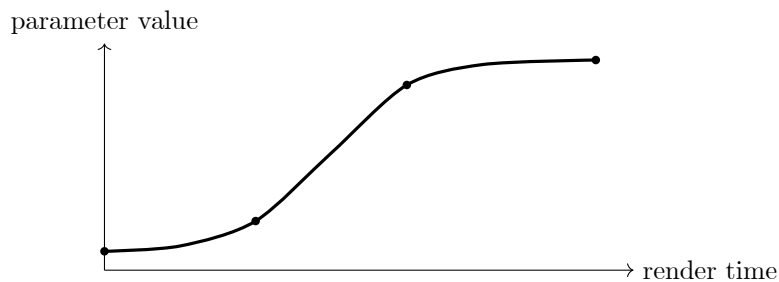


Figure 1.22: Automation treats a parameter as a continuous curve across the render, not as a sequence of unrelated settings.

1.4 What the transform sees

For every analysis frame, pvx creates a complex spectrum $X_t[k]$. Its magnitude describes the energy at bin k ; its phase describes that component's cycle position. A standard short-time Fourier transform is:

$$X_t[k] = \sum_{n=0}^{N-1} x[n + tH_a]w[n]e^{-j2\pi kn/N}$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

The window $w[n]$ softens frame boundaries. Overlap between windows lets the reconstructed signal remain smooth. A narrow window follows quick attacks well but provides less precise pitch information. A longer window gives a clearer view of closely spaced frequencies but can smear a drum hit or consonant across time. There is no universal perfect choice; musical material decides.

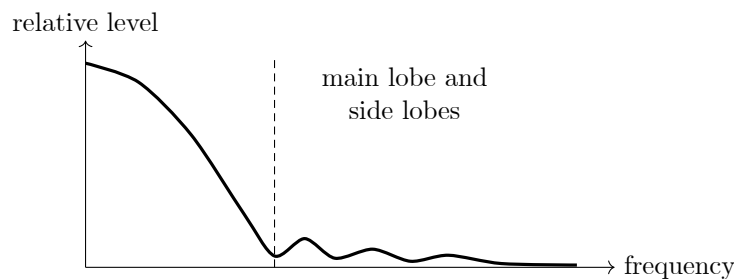


Figure 1.23: A conceptual window response. Window choice balances temporal detail against frequency selectivity.

1.5 A small comparison

Table 1.4: Three ways to think about changing recorded sound.

Method	What it changes directly	Characteristic limitation
Tape varispeed	Playback speed	Pitch and duration remain coupled.
Segmented tape systems	Short physical segments	Joins and repeats can be audible.
Phase vocoder	Spectral frames and phase trajectories	Transients and dense attacks need careful handling.

1.6 How pvx puts the idea to work

In pvx, the phase vocoder is the engine beneath time stretching, pitch shifting, freeze-like spectral holds, formant-aware workflows, and many automated transformations. The stable approach is usually modest: choose a sensible window and hop, preserve transients when the material needs it, use phase locking for harmonic sources, and audition a small section before committing to an extreme render.

```
pvx voc voice.wav --stretch 1.25 --transient-preserve --phase-locking identity --output voice_longer.wav
pvx voc piano.wav --pitch -3 --formant-preserve --output piano_down.wav
pvx freeze cymbal.wav --freeze-time 0.42 --duration 12 --output cymbal_cloud.wav
```

The commands look compact because the conceptual work is inside the analysis loop. The rest of this guide turns those choices into repeatable practice.

Part II

Orientation and Core Workflow

CHAPTER 2

Getting Started with pvx

This guide is for first-time users who want to understand what **pvx** does, why it exists, and how to get useful results without treating digital signal processing (DSP) as magic. It is practical first, mystical later.

2.1 Quick Setup (Install + PATH)

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -e .
pvx --help
```

Same setup with uv:

```
uv venv .venv
source .venv/bin/activate
uv pip install -e .
uv run pvx --help
```

If **pvx** is not found, add the project virtualenv to your path environment variable (**PATH**) (**zsh**):

```
printf 'export PATH="%s/.venv/bin:$PATH"\n' "$(pwd)" >> ~/.zshrc
source ~/.zshrc
pvx --help
```

No-PATH fallback:

```
.venv/bin/pvx voc input.wav --stretch 1.2 --output output.wav
# or
\index{or}
python3 -m pvx.cli.pvx voc input.wav --stretch 1.2 --output output.wav
```

No-PATH fallback with uv:

```
uv run pvx voc input.wav --stretch 1.2 --output output.wav
```

Man-page generation:

```
python3 scripts/scripts_install_man_pages.py
MANPATH="$(pwd)/man:$MANPATH" man pvx
```

2.2 Launch-Ready Helper Commands

Before announcing or demoing, run these:

```
pvx doctor
pvx quickstart input.wav --output output.wav
pvx safe input.wav --material mix --output output_safe.wav
pvx transforms
pvx smoke --output smoke_out.wav
pvx augment data/*.wav --output-dir aug_out --variants-per-input 4 --intent asr_robust --seed 1337
pvx augment-manifest validate aug_out/augment_manifest.jsonl --strict
```

What each does: - **pvx doctor**: checks environment, path environment variable (PATH), and optional dependencies. - **pvx quickstart**: prints the minimal launch command sequence. - **pvx safe**: runs pvx voc with conservative quality-first defaults. - **pvx transforms**: explains transform options and runtime availability. - **pvx smoke**: executes a fast synthetic end-to-end sanity render. - **pvx augment**: builds deterministic augmentation datasets and writes manifests for machine-learning pipelines. - **pvx augment-manifest**: validates, merges, and summarizes augmentation manifests.

2.3 Running Any Command with uv

uv can run every command in this guide without changing the DSP arguments.

- **pvx ... -> uv run pvx ...**
- **python3 some_script.py ... -> uv run python3 some_script.py ...**
- **python3 -m module.path ... -> uv run python3 -m module.path ...**

Example:

```
pvx voc input.wav --stretch 1.25 --output out.wav
uv run pvx voc input.wav --stretch 1.25 --output out.wav
```

If you are already muttering at your shell, that is normal.

2.4 First 3 Commands to Run

```
# Stretch without pitch change
\index{Stretch without pitch change}
pvx voc input.wav --stretch 1.25 --output input_stretch.wav

# Pitch change without duration change
\index{Pitch change without duration change}
pvx voc input.wav --stretch 1.0 --pitch -3 --output input_pitch.wav

# Same pitch shift, but with formant protection
\index{Same pitch shift but with formant protection}
pvx voc input.wav --stretch 1.0 --pitch -3 --pitch-mode formant-preserving --output input_pitch_formant.wav
```

What to listen for: - **input_stretch.wav**: longer timing, same note center - **input_pitch.wav**: lower note center, possible timbral darkening - **input_pitch_formant.wav**: lower note center with more stable vowel/timbre identity - if all three sound identical, double-check that you did not typo the output path at 2 a.m.

2.5 What Problem pvx Solves

Audio workflows often need one or more of the following, with controllable artifacts: - change duration without changing pitch - change pitch without changing duration - apply time-varying pitch/time trajectories from a map - preserve transients and formants where possible - process many files consistently from the command line

pvx gives you explicit control over those operations using phase-vocoder/short-time Fourier transform (STFT) methods plus specialized companion tools (freeze, harmonize, morph, retune, denoise, dereverb).

Design priority: - quality first: preserve musical intent and minimize artifacts - speed second: optimize runtime after quality targets are achieved - translation: we would rather be right than merely fast

2.6 Analog Tape Methods for Pitch/Time Shifting

Historically, reel-to-reel varispeed coupled time and pitch. Analog systems like the Eltro information rate changer and related Anton Springer regulator concepts were early attempts to separate them using segmented rotating-head playback methods instead of simple fixed-speed tape transport.

Why this matters for **pvx**: - artifact control is largely a continuity problem (then: segment stitching; now: phase/transient/stereo continuity), - high-quality time/pitch work usually benefits from conservative, staged processing.

Sources: - [Eltro information rate changer \(Wikipedia\)](#) - [Vintage Technologies: The Eltro and the Voice of HAL \(Wendy Carlos\)](#) - [Anton Springer and the Time and Pitch Regulator \(Sound and Science\)](#)

2.7 Basic DSP Terms (Plain Language)

This section begins with the following points, which establish the context for the details that follow.

- Sample rate: how many audio samples per second (e.g. 48,000 Hz).
- Frame: a short window of audio processed as one block.
- STFT: repeated short Fourier transforms over overlapping frames.
- Bin: one frequency slot in an STFT frame.
- Window: taper applied to each frame to reduce spectral leakage.
- Hop size: frame advance between adjacent STFT frames.
- Phase coherence: consistency of phase evolution across frames/bins.
- Transient: short attack event (drum hit, consonant burst).
- Formant: resonant spectral envelope that carries vowel/timbre identity.

2.8 Supported File Types

pvx audio I/O is provided by `soundfile/libsndfile`, so exact format support depends on your runtime build.

- Quick summary: `wav`, `flac`, `aiff`, `ogg`, `caf` are supported for stream output and are safe defaults.
- Full current format table: `FILE_TYPES.md`

2.9 Stretch vs Pitch Shift

This section begins with the following points, which establish the context for the details that follow.

- Stretch controls duration.
 - `stretch = 2.0` means output is twice as long.
 - `stretch = 0.5` means output is half as long.
- Pitch shift controls musical frequency position.
 - `+12 semitones` = one octave up.
 - `-12 semitones` = one octave down.

In **pvx voc**, duration and pitch can be controlled independently:

```
pvx voc input.wav --stretch 1.25 --pitch 0 --output stretched.wav
pvx voc input.wav --stretch 1.0 --pitch 3 --output pitched.wav
```

2.10 Denoising with Phase-Consistent STFT Processing

`pvx denoise` uses short-time Fourier transform (STFT) spectral subtraction while preserving phase continuity in overlap-add resynthesis.

Starter profiles:

```
# Speech-safe (conservative)
\index{Speech-safe (conservative)}
pvx denoise speech.wav --noise-seconds 0.4 --reduction-db 5 --floor 0.2 --smooth 9 --output speech_clean.wav

# Music-safe (preserve more ambience/harmonics)
\index{Music-safe (preserve more ambience/harmonics)}
pvx denoise mix.wav --noise-seconds 0.3 --reduction-db 4 --floor 0.25 --smooth 7 --output mix_clean.wav
```

If you hear chirpy/watery artifacts: - reduce `--reduction-db` - increase `--smooth` - raise `--floor` slightly
 - use `--noise-file roomtone.wav` when the beginning of the source is not a clean noise-only segment

2.11 Time-Varying Parameter Control (CSV/JSON)

You can pass a control file directly to many core phase-vocoder numeric flags.

Examples:

```
pvx voc input.wav --stretch controls/stretch.csv --interp linear --output out.wav
pvx voc input.wav --pitch-shift-ratio controls/pitch.json --interp polynomial --order 3 --output out.wav
pvx voc input.wav --n-fft controls/nfft.csv --hop-size controls/hop.csv --output out.wav
```

Common flags that accept scalar values or control files (`.csv` / `.json`): - time/pitch trajectory: `--stretch`, `--time-stretch`, `--pitch-shift-ratio`, `--pitch-shift-semitones`, `--pitch-shift-cents` - frame/spectral resolution: `--n-fft`, `--win-length`, `--hop-size`, `--kaiser-beta` - phase/transient/stereo shaping: `--ambient-phase-mix`, `--transient-threshold`, `--transient-sensitivity`, `--transient-protect-ms`, `--transient-crossfade-ms`, `--coherence-strength` - multistage/Fourier-sync tuning: `--extreme-stretch-threshold`, `--max-stage-stretch`, `--fourier-sync-min-fft`, `--fourier-sync-max-fft`, `--fourier-sync-smooth` - onset/formant shaping: `--onset-credit-pull`, `--onset-credit-max`, `--formant-lifter`, `--formant-strength`, `--formant-max-gain-db`

Control interpolation options: - `--interp none` (stairstep / no interpolation) - `--interp linear` (default) - `--interp nearest` - `--interp cubic` - `--interp polynomial --order N` (any integer $N \geq 1$, default $N=3$; effective degree is $\min(N, \text{control_points}-1)$)

Polynomial order examples: - `--interp polynomial --order 1` - `--interp polynomial --order 2` - `--interp polynomial --order 3` - `--interp polynomial --order 5`

Point-style CSV:

```
time_sec,value
0.0,1.0
0.5,1.2
1.0,1.8
```

Segment-style CSV:


```
start_sec,end_sec,value
0.0,0.4,1.0
0.4,0.8,1.2
0.8,1.2,1.6
```

JSON points:

```
{
  "interpolation": "linear",
  "points": [
    {"time_sec": 0.0, "value": 1.0},
    {"time_sec": 0.5, "value": 1.2},
    {"time_sec": 1.0, "value": 1.8}
  ]
}
```

JSON schema quick reference:

Key	Required	Type	Meaning
interpolation / interp	no	string	Interpolation override (none, linear, nearest, cubic, exponential, s_curve, smotherstep, polynomial)
order	no	integer	Polynomial order for polynomial interpolation
points	yes (point mode)	array	Time/value points
points[].time_sec	yes	number	Timestamp in seconds
points[].value	yes	number/string	Parameter value
segments	yes (segment mode)	array	Piecewise constant control regions
segments[].start_sec, segments[].end_sec	yes	number	Segment boundaries in seconds
segments[].value	yes	number/string	Segment value
parameters	no	object	Multi-parameter container keyed by parameter name

Notes: - per-parameter dynamic controls cannot be combined with legacy `--pitch-map` / `--pitch-map-stdin` in the same command - dynamic `--time-stretch` cannot be combined with `--target-duration`

Interpolation graph examples:

Mode	Example curve
none (stairstep)	
nearest	
linear	
cubic	
exponential	

Mode	Example curve
s_curve (smoothstep)	
smootherstep	
polynomial order 1	
polynomial order 2	
polynomial order 3	
polynomial order 5	

Core processing function charts:

Function family	Graph
Pitch ratio vs semitones	
Dynamics transfer curves	
Soft clip transfer functions	
Morph blend magnitude curves	
Mask exponent response	

2.12 Visual Mental Model of STFT

```
Time-domain signal
x[n] ----->

Frame + windowing
    [w[n] * x[n:n+N]]
      [w[n] * x[n+H:n+H+N]]
        [w[n] * x[n+2H:n+2H+N]]

Per-frame transform
    FFT -> X0[k]
    FFT -> X1[k]
    FFT -> X2[k]

Process in spectral domain
    modify magnitude/phase trajectories

Synthesis
    IFFT + overlap-add -> y[n]
```

Key tradeoff: - larger N (FFT) -> better frequency resolution, worse time localization - smaller N -> better time localization, rougher low-frequency precision - yes, this is the classic engineering compromise you were hoping to avoid

2.13 Example Audio Scenarios

This section begins with the following points, which establish the context for the details that follow.

- Dialogue cleanup + mild timing correction: `pvx voc + pvx denoise + pvx deverb`.
- Ambient texture from short sample: `pvx voc --preset ambient --target-duration ....`
- Vocal harmonies: `pvx harmonize` with interval and pan controls.
- Timeline-locked effects: `pvx conform / pvx warp` with CSV maps.

2.13.1 Use-Case Matrix (Where to start quickly)

Use case	First command to try
Speech slowdown for transcription	<code>pvx voc speech.wav --preset vocal_studio --stretch 1.25 --output speech_slow.wav</code>
Podcast timing cleanup	<code>pvx voc voice.wav --stretch 0.95 --output voice_tight.wav</code>
Vocal pitch correction	<code>pvx retune vocal.wav --scale major --root C --output-dir out --suffix _retune</code>
Harmonic backing voices	<code>pvx harmonize lead.wav --intervals 0,4,7 --gains 1,0.8,0.65 --output-dir out</code>
Unison widen synth	<code>pvx unison synth.wav --voices 7 --detune-cents 16 --width 1.1 --output-dir out</code>
Long ambient from short source	<code>pvx voc oneshot.wav --preset extreme_ambient --target-duration 600 --output ambient.wav</code>
Drum-safe stretch	<code>pvx voc drums.wav --preset drums_safe --stretch 1.3 --output drums_stretch.wav</code>
Stereo image stability	<code>pvx voc mix.wav --stretch 1.15 --stereo-mode mid_side_lock --coherence-strength 0.9 --output mix_lock.wav</code>
Noise reduction pre-pass	<code>pvx denoise field.wav --reduction-db 8 --output-dir out --suffix _den</code>
Room tail reduction	<code>pvx deverb room.wav --strength 0.5 --output-dir out --suffix _dry</code>
CSV time/pitch choreography	<code>pvx conform source.wav --map map_conform.csv --output-dir out</code>
Automated quality comparison	<code>uv run python3 benchmarks/run_bench.py --quick --out-dir benchmarks/out --gate --baseline benchmarks/baseline_small.json</code>

2.14 Step-by-Step Walkthrough (One Small WAV)

Assume `sample.wav` is about 2-5 seconds long.

2.14.1 Step A: Baseline stretch

```
pvx voc sample.wav --stretch 1.2 --output sample_stretch.wav
```

Expected: - 20% longer duration - same pitch - slight smearing possible on sharp transients

2.14.2 Step B: Add transient protection

```
pvx voc sample.wav --stretch 1.2 --transient-preserve --phase-locking identity --output  
sample_stretch_transient.wav
```

Expected: - attacks should feel tighter than Step A - fewer “swishy” transients

2.14.3 Step C: Pitch shift with formant preservation

```
pvx voc sample.wav --stretch 1.0 --pitch -4 --pitch-mode formant-preserving --output sample_pitch_formant.  
wav
```

Expected: - pitch lowered by 4 semitones - vowel/timbre identity more stable than plain shift

2.14.4 Step D: Auto profile planning

```
pvx voc sample.wav --auto-profile --auto-transform --explain-plan
```

Expected: - JSON plan with resolved profile, transform, and core processing settings - no audio output (plan only)

2.15 What Outputs Should Sound Like

This section begins with the following points, which establish the context for the details that follow.

- **sample_stretch.wav**: same pitch, longer phrase timing.
- **sample_stretch_transient.wav**: similar to above, but cleaner attack edges.
- **sample_pitch_formant.wav**: lower note center with less “character collapse.”

If artifacts are strong: - lower ratio magnitude - increase overlap (`--hop-size` smaller) - try preset `vocal` for speech/singing - test `--multires-fusion` for mixed content

2.16 New Quality Modes (Beginner Version)

2.16.1 Hybrid transient mode (recommended on speech/drums)

```
pvx voc sample.wav \  
--transient-mode hybrid \  
--transient-sensitivity 0.6 \  
--transient-protect-ms 30 \  
--transient-crossfade-ms 10 \  
--stretch 1.25 \  
--output sample_hybrid.wav
```

What to expect: - steadier harmonic regions from phase-vocoder processing - cleaner attacks from WSOLA handling around detected onsets

2.16.2 Stereo coherence lock (recommended for wide stereo sources)

```
pvx voc stereo_mix.wav \
  --stereo-mode mid_side_lock \
  --coherence-strength 0.9 \
  --stretch 1.2 \
  --output stereo_locked.wav
```

What to expect: - less left/right image wobble - more stable center image after heavy stretch

2.17 Intent Presets

Use presets when you do not want to tune many flags:

- `--preset vocal_studio`: formant-aware vocal defaults + transient hybrid handling
- `--preset drums_safe`: WSOLA-heavy transient safety for percussive content
- `--preset extreme_ambient`: extreme long-form ambient settings
- `--preset stereo_coherent`: stereo coupling defaults

Legacy presets remain available: `vocal`, `ambient`, `extreme`.

2.18 Simpler One-Line Pipelines

If you do not want long Unix pipe chains, use managed helpers:

```
pvx follow guide.wav target.wav --output target_follow.wav --emit pitch_to_stretch --pitch-conf-min 0.75
pvx chain sample.wav --pipeline "voc --stretch 1.2 | formant --mode preserve" --output sample_chain.wav
pvx stream sample.wav --output sample_stream.wav --chunk-seconds 0.2 --time-stretch 2.0
pvx stream sample.wav --mode wrapper --output sample_stream_wrapper.wav --chunk-seconds 0.2 --time-stretch 2.0
pvx stretch-budget sample.wav --disk-budget 20GB --bit-depth 16 --requested-stretch 1000000
```

- `pvx follow` replaces long sidechain pipes for pitch/control-map-driven workflows.
- `pvx chain` runs serial stages with managed intermediate files.
- `pvx stream` defaults to a stateful chunk processor for smoother long-form continuity.
- `pvx stream --mode wrapper` keeps legacy segmented-wrapper behavior.
- `pvx stretch-budget` estimates maximum practical stretch before you commit to a long render.

2.19 Estimate Stretch Budget Before Extreme Jobs

Use this helper before very large ratios (100x, 1000x, 1000000x) so disk limits are explicit.

```
pvx stretch-budget input.wav --disk-budget 20GB --bit-depth 16
pvx stretch-budget input.wav --disk-budget 20GB --requested-stretch 1000000 --fail-if-exceeds --json
```

What it uses: - input shape (frames/channels/sample rate) - output storage assumption (`--output-format`, `--bit-depth` / `--subtype`) - budget (`--disk-budget` or free space at `--budget-path`) - headroom (`--safety-margin`, default 0.90)

Recommendation: - for production, prefer `--target-duration` over arbitrary huge ratios. - if you run extreme jobs, combine `--stretch-mode multistage` with `--auto-segment-seconds`, `--checkpoint-dir`, and `--resume`.

2.20 Feature Tracking for Sidechain Control

`pvx pitch-track` now emits feature vectors (not just pitch map fields), including: - pitch/voicing (`f0_hz`, `pitch_ratio`, `confidence`, `voicing_prob`, `pitch_stability`) - loudness/dynamics (`rms`, `rms_db`,

short_lufs_db, crest_factor_db) - spectral features (spectral_centroid_hz, spectral_flatness, spectral_flux, rolloff_hz) - formants and cepstra (formant_f1_hz..formant_f3_hz, mfcc_01..mfcc_N) - rhythm markers (tempo_bpm, beat_phase, downbeat_phase, onset_strength, transient_mask) - MPEG-7-style descriptors (mpeg7_* columns, including audio spectrum envelope bands)

Use with control-bus routes:

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output - \
| pvx voc target.wav --control-stdin \
  --route pitch_ratio=affine(mfcc_01,0.002,1.0) \
  --route pitch_ratio=clip(pitch_ratio,0.5,2.0) \
  --route stretch=affine(spectral_flux,0.03,1.0) \
  --route stretch=clip(stretch,0.85,1.5) \
  --output target_feature_follow.wav
```

For a larger gallery (single-feature, multi-feature, MFCC/MPEG-7 vector, and multi-guide workflows), see: - docs/FEATURE_SIDECHAIN_EXAMPLES.md

You can also print built-in command snippets directly from the command-line interface (CLI):

```
pvx follow --example
pvx follow --example all
pvx follow --example formant_onset
```

2.21 Output Policy Controls

All audio-output tools now share deterministic output policy flags:

- --bit-depth {inherit,16,24,32f}
- --dither {none,tpdf} and --dither-seed
- --true-peak-max-dbtp
- --metadata-policy {none,sidecar,copy}
- --subtype for explicit low-level output subtype override

2.22 Next Steps

This section begins with the following points, which establish the context for the details that follow.

- Run practical recipes and advanced use cases (72+): docs/EXAMPLES.md
- Learn architecture and DSP diagrams (26+): docs/DIAGRAMS.md
- Study equation-level behavior (31 sections): docs/MATHEMATICAL_FOUNDATIONS.md
- Use Python API directly: docs/API_OVERVIEW.md
- Use benchmark runner: benchmarks/run_bench.py

2.23 Extra Beginner Recipes (Quick Wins)

```
# Speech slowdown
\index{Speech slowdown}
pvx voc speech.wav --preset vocal_studio --stretch 1.30 --output speech_slow.wav

# Drum-safe stretch
\index{Drum-safe stretch}
pvx voc drums.wav --preset drums_safe --stretch 1.20 --output drums_safe.wav

# Stereo coherence lock
\index{Stereo coherence lock}
pvx voc mix.wav --stretch 1.2 --stereo-mode mid_side_lock --coherence-strength 0.9 --output mix_lock.wav
```

```

# Freeze a transient into a pad
\index{Freeze a transient into a pad}
pvx freeze hit.wav --freeze-time 0.22 --duration 12 --output hit_pad.wav

# Morph two sources
\index{Morph two sources}
pvx morph a.wav b.wav --alpha 0.45 --blend-mode carrier_a_envelope_b --output a_b_morph.wav

# True A->B trajectory morph in one command
\index{True A- B trajectory morph in one command}
pvx morph A.wav B.wav --alpha controls/alpha_curve.csv --interp linear --blend-mode linear --output
    A_to_B_morph.wav

# Major-scale retune
\index{Major-scale retune}
pvx retune vocal.wav --root C --scale major --strength 0.8 --output vocal_c_major.wav

# Alternate concert pitch retune (A4 = 432 Hz)
\index{Alternate concert pitch retune (A4 432 Hz)}
pvx retune vocal.wav --root A --scale minor --a4-reference-hz 432 --output vocal_a432.wav

# Explicit root fundamental retune (C4 ~= 261.6256 Hz)
\index{Explicit root fundamental retune (C4 261.6256 Hz)}
pvx retune vocal.wav --root-hz 261.6256 --scale major --output vocal_c4_root.wav

# Auto-recommend root fundamental from the source
\index{Auto-recommend root fundamental from the source}
pvx retune vocal.wav --recommend-root --scale minor --output vocal_auto_root.wav

# Denoise then dereverb
\index{Denoise then dereverb}
pvx denoise noisy.wav --reduction-db 8 --stdout | pvx deverb - --strength 0.3 --output noisy_clean.wav

# Denoise then stretch
\index{Denoise then stretch}
pvx denoise noisy.wav --reduction-db 6 --stdout | pvx voc - --stretch 2.0 --output noisy_clean_stretch.wav

# Generate an LFO control map (triangle) and apply it as time-varying stretch
\index{Generate an LFO control map (triangle) and apply it as time-varying stretch}
pvx lfo --wave triangle --duration 12 --frequency-hz 0.4 --center 1.0 --amplitude 0.2 --key stretch --output
    controls/stretch_tri.csv
pvx voc input.wav --stretch controls/stretch_tri.csv --interp linear --output input_tri_stretch.wav

```

Run them in order, listen after each step, and resist changing ten parameters at once unless chaos is the objective.

CHAPTER 3

pvx Quality Guide

This guide maps audible artifacts to concrete `pvx` `voc` fixes.

3.1 Fast Starting Presets

Material	Recommended preset	Why
Vocal/speech	<code>--preset vocal_studio</code>	formant-aware pitch + hybrid transient handling
Drums/percussion	<code>--preset drums_safe</code>	WSOLA-focused transient path
Wide stereo mix	<code>--preset stereo_coherent</code>	channel-coupled coherence
Extreme ambient	<code>--preset extreme_ambient</code>	multistage long-stretch strategy

3.2 Artifact -> Fix Table

What you hear	Likely cause	Primary fixes
Attack smear / soft transients	transient regions processed purely in PV mode	<code>--transient-mode hybrid</code> or <code>--transient-mode wsola</code> ; raise <code>--transient-sensitivity</code> ; reduce <code>--transient-crossfade-ms</code>
Phasy tone / chorus blur	weak phase coherence	<code>--phase-locking identity</code> ; consider <code>--stereo-mode</code> <code>ref_channel_lock</code> with <code>--coherence-strength 0.7</code>
Stereo image wobble	per-channel phase drift	<code>--stereo-mode mid_side_lock</code> or <code>--stereo-mode</code> <code>ref_channel_lock</code> <code>--ref-channel 0</code>
Robotic vocal timbre after pitch shift	formant drift	<code>--pitch-mode</code> formant-preserving; tune <code>--formant-lifter</code> and <code>--formant-strength</code>

What you hear	Likely cause	Primary fixes
Pumping/flat loudness	aggressive mastering chain	reduce compressor/limiter settings; disable unused mastering stages
Grainy extreme stretch	ratio too high for single-stage path	<code>--stretch-mode multistage</code> ; lower <code>--max-stage-stretch</code> ; optionally <code>--multires-fusion</code>
Unexpected overs/clipped delivery files	output policy not constrained	set <code>--true-peak-max-dbtp</code> and target <code>--bit-depth</code> ; use <code>--dither tpdf</code> for PCM exports
Chirpy/watery denoise residuals	overly aggressive subtraction or weak profile estimate	lower <code>--reduction-db</code> ; raise <code>--smooth</code> ; raise <code>--floor</code> ; provide <code>--noise-file</code> room tone

3.3 Practical Recipes

3.3.1 Speech stretch with transient protection

```
pvx voc speech.wav \  
  --preset vocal_studio \  
  --transient-mode hybrid \  
  --stretch 1.3 \  
  --output speech_clean_stretch.wav
```

3.3.2 Drum-safe stretch

```
pvx voc drums.wav \  
  --preset drums_safe \  
  --time-stretch 1.4 \  
  --output drums_safe.wav
```

3.3.3 Stereo coherence lock

```
pvx voc mix.wav \  
  --stereo-mode ref_channel_lock \  
  --ref-channel 0 \  
  --coherence-strength 0.9 \  
  --stretch 1.2 \  
  --output mix_locked.wav
```

3.3.4 Denoise without musical-noise artifacts

```
pvx denoise speech.wav \  
  --noise-seconds 0.4 \  
  --reduction-db 5 \  
  --floor 0.2 \  
  --smooth 9 \  
  --output speech_clean.wav
```

3.4 Parameter Ranges That Usually Work

Parameter	Typical range	Notes
<code>--transient-sensitivity</code>	0.45 .. 0.75	higher = more transient regions
<code>--transient-protect-ms</code>	20 .. 45	increase for noisy or percussive content
<code>--transient-crossfade-ms</code>	6 .. 14	larger values smooth boundaries
<code>--coherence-strength</code>	0.5 .. 0.95	higher = stronger channel coupling
<code>--max-stage-stretch</code>	1.2 .. 1.8	lower values are safer for extreme ratios

3.5 Debug Workflow

This section begins with the following points, which establish the context for the details that follow.

1. Run `pvx voc --example all` to pick a close recipe.
2. Use `--dry-run --explain-plan` to inspect resolved settings.
3. Adjust one control at a time (`transient-mode`, then `coherence-strength`, then phase/window settings).
4. Compare A/B using short excerpts before full renders.
5. Only after quality is acceptable, optimize runtime (`--n-fft`, staging, segmentation/checkpoint options).

CHAPTER 4

Supported Surface

This page is the short version of what **pvx** expects alpha users to rely on.

If a longer generated reference lists more commands or flags, treat that as inventory, not a broader compatibility promise.

4.1 Stable in 0.1.x

These commands are the intended scripted/public alpha surface:

- **pvx**
- **pvxvoc**
- **pvxfreeze**
- **pvxwarp**
- **pvxformant**
- **pvxfilter**
- **pvxretune**
- **pvxanalysis**

4.2 Beta in 0.1.x

These are usable, but minor releases may still change flags or defaults:

- **pvxharmonize**
- **pvxconform**
- **pvxmorph**
- **pvxtransient**
- **pvxunison**
- **pvxdenoise**
- **pvxdeverb**
- **pvxlayer**
- **pvxresponse**
- **pvxenvelope**
- **pvxreshape**
- **pvxtvfilter**
- **pvxnoisefilter**
- **pvxbandamp**
- **pvxspeccompander**
- **pvxring**
- **pvxringfilter**
- **pvxringtvfilter**
- **pvxharmmmap**

4.3 Experimental in 0.1.x

These are available for exploration, not for stable automation contracts:

- `hps-pitch-track`
- `pvxchordmapper`
- `pvxinharmonator`
- `pvxtrajectoryreverb`
- `pvxnoise`
- `pvxrir`
- `pvxcodec`
- `pvxspecaugment`
- `pvxgain`

4.4 Compatibility Shims

The `pvxalgorithms*` import aliases remain available during 0.1.x only to ease migration.

- Canonical imports: `pvx.algorithms*`
- Deprecated aliases: `pvxalgorithms`, `pvxalgorithms.base`, `pvxalgorithms.registry`
- Removal target: first 0.2.0 release unless the release notes say otherwise

4.5 Docs Guide

If you only need the supported user-facing paths, start with:

- README
- Getting Started
- Homebrew Install
- Alpha Release Guide

Use the generated references only when you need exhaustive flag/file inventory:

- CLI Flags Reference
- Python File Help

CHAPTER 5

Supported File Types

pvx uses `python-soundfile` (`soundfile`) backed by `libsndfile` for audio input/output (I/O). This means some format support is runtime/platform dependent.

5.1 I/O Behavior by Category

Category	Used by	File types
Audio input (file path)	<code>pvxvoc</code> , <code>pvxfreeze</code> , <code>pvxharmonize</code> , <code>pvxmorph</code> , <code>pvxretune</code> , <code>pvxdenoise</code> , <code>pvxdeverb</code> , and other audio CLIs	Any audio container supported by the active <code>soundfile/libsndfile</code> build (see full table below)
Audio input (stdin, -)	CLIs that accept - as input	Any container <code>soundfile</code> can decode from byte stream
Audio output (file path)	Audio CLIs with <code>--output-format</code> or inferred output extension	Any format <code>soundfile</code> can write in the active build
Audio output (stdout, --stdout)	Stream/pipeline mode in <code>pvxvoc</code> and common CLI helpers	Explicitly supported: <code>wav</code> , <code>flac</code> , <code>aiff/aif</code> , <code>ogg/oga</code> , <code>caf</code>
Control maps	<code>pvxvoc --pitch-map</code> , <code>pvxwarp --map</code> , <code>pvxconform --map</code>	<code>csv</code>
Time-varying per-parameter control signals	<code>pvxvoc --stretch stretch.csv</code> , <code>pvxvoc --pitch-shift-ratio pitch.json</code> , <code>pvxvoc --n-fft nfft.csv</code> , <code>pvx stream ... --stretch stretch.csv</code>	<code>csv</code> , <code>json</code>
Pitch tracking map output	<code>pvx pitch-track / hps-pitch-track</code>	<code>csv</code>
Render manifest	<code>pvxvoc --manifest-json</code>	<code>json</code>
Documentation output	docs generators	<code>html</code> , <code>pdf</code>

5.2 Full Audio Container List (Current Build)

The following formats are reported by the current local `soundfile/libsndfile` runtime:

	Extension	Format
<code>aiff</code>		AIFF (Apple/SGI)

	Extension Format
au	AU (Sun/NeXT)
avr	AVR (Audio Visual Research)
caf	CAF (Apple Core Audio File)
flac	FLAC (Free Lossless Audio Codec)
htk	HTK (HMM Tool Kit)
ircam	SF (Berkeley/IRCAM/CARL)
mat4	MAT4 (GNU Octave 2.0 / Matlab 4.2)
mat5	MAT5 (GNU Octave 2.1 / Matlab 5.0)
mp3	MPEG-1/2 Audio
mpc2k	MPC (Akai MPC 2k)
nist	WAV (NIST Sphere)
ogg	OGG (OGG Container format)
paf	PAF (Ensoniq PARIS)
pvf	PVF (Portable Voice Format)
raw	RAW (header-less)
rf64	RF64 (RIFF 64)
sd2	SD2 (Sound Designer II)
sds	SDS (Midi Sample Dump Standard)
svx	IFF (Amiga IFF/SVX8/SV16)
voc	VOC (Creative Labs)
w64	W64 (SoundFoundry WAVE 64)
wav	WAV (Microsoft)
wavex	WAVEX (Microsoft)
wve	WVE (Psion Series 3)
xi	XI (FastTracker 2)

5.3 Verify on Your Machine

Use this command to print the exact formats available in your environment:

```
python3 - <<'PY'
import soundfile as sf
for ext, name in sorted(sf.available_formats().items()):
    print(f"{ext.lower():<8} {name}")
PY
```


Part III

Theory and Internal Model

CHAPTER 6

pvx Mathematical Foundations

Generated from commit `dff2a4f` (commit date: 2026-04-21T21:20:38-04:00).

This document explains the core signal-processing equations used by pvx, with plain-English interpretation. All equations are written in GitHub-renderable LaTeX and are intended to render directly in normal GitHub Markdown view.

6.1 Notation

This section begins with the following points, which establish the context for the details that follow.

- Analysis hop: H_a
- Synthesis hop: H_s
- FFT size: N
- Frame index: t
- Frequency-bin index: k
- Frame phase: $\phi_t[k]$
- Bin-center angular frequency: $\omega_k = 2\pi k/N$
- Selected transform backend: $m \in \{\text{fft}, \text{dft}, \text{czt}, \text{dct}, \text{dst}, \text{hartley}\}$

6.2 STFT Analysis and Synthesis

Analysis transform:

$$X_t[k] = \sum_{n=0}^{N-1} x[n + tH_a]w[n]e^{-j2\pi kn/N}$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Plain English: each frame $\mathbf{t}$ is windowed by $\mathbf{w}[\mathbf{n}]$ and transformed into complex frequency bins $\mathbf{k}$.

Weighted overlap-add synthesis:

$$\hat{x}[n] = \frac{\sum_t \hat{x}_t[n - tH_s]w[n - tH_s]}{\sum_t w^2[n - tH_s] + \varepsilon}$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Plain English: reconstructed frames are overlap-added, then normalized by accumulated window energy.

6.3 Transform Backend Families and Tradeoffs

pvx allows selecting the per-frame transform with `--transform` in STFT/ISTFT paths. The same overlap-add framework is used, but per-bin meaning and phase behavior differ by transform family.

Complex Fourier family (`fft`, `dft`):

$$X_t[k] = \sum_{n=0}^{N-1} x_t[n] e^{-j2\pi kn/N}, \quad x_t[n] = \frac{1}{N} \sum_{k=0}^{N-1} X_t[k] e^{j2\pi kn/N}$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Chirp-Z (**cz****t**, Bluestein-style contour):

$$X_t[k] = \sum_{n=0}^{N-1} x_t[n] A^{-n} W^{nk}$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

With pvx defaults $A = 1$ and $W = e^{-j2\pi/N}$, CZT matches DFT samples but uses a different numerical path.

DCT-II / IDCT-II (**d****ct**):

$$C_t[k] = \alpha_k \sum_{n=0}^{N-1} x_t[n] \cos\left(\frac{\pi}{N}\left(n + \frac{1}{2}\right)k\right)$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

DST-II / IDST-II (**d****st**):

$$S_t[k] = \beta_k \sum_{n=0}^{N-1} x_t[n] \sin\left(\frac{\pi}{N}\left(n + \frac{1}{2}\right)(k + 1)\right)$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Discrete Hartley (**h****artley**):

$$H_t[k] = \sum_{n=0}^{N-1} x_t[n] \operatorname{cas}\left(\frac{2\pi kn}{N}\right), \quad \operatorname{cas}(\theta) = \cos(\theta) + \sin(\theta)$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Transform	Strengths	Tradeoffs	Recommended use cases
fft	Fastest common path, stable, native one-sided complex bins, best CUDA coverage.	Needs careful window/hop tuning for extreme nonstationarity.	Default for most time-stretch, pitch-shift, and batch processing.
dft	Canonical Fourier definition, deterministic parity baseline versus FFT implementations.	Usually slower than fft , little practical quality advantage in normal runs.	Cross-checking, verification, or research comparisons.
cz t	Useful alternative numerical path for awkward frame sizes; can be robust for prime/non-power sizes.	Requires SciPy; typically CPU-only and slower than FFT.	Edge-case frame-size experiments and diagnostic reruns.

Transform	Strengths	Tradeoffs	Recommended use cases
dct	Real-valued compact basis with strong energy compaction for smooth envelopes.	Loses explicit complex phase; less transparent for strict phase-coherent resynthesis.	Spectral shaping, denoise-like creative processing, coefficient-domain experiments.
dst	Real-valued odd-symmetry basis; emphasizes different boundary behavior than DCT.	Same phase limitations as DCT; content-dependent coloration.	Creative timbre variants and odd-symmetry analysis experiments.
hartley	Real transform using cas , often helpful for CPU-friendly exploratory analysis.	Different bin semantics vs complex STFT; can alter artifact profile.	Alternative real-basis phase-vocoder experiments and pedagogical comparisons.

Plain English: choose **fft** first, then move to other transforms when you specifically need different numerical behavior, basis structure, or artifact character.

6.3.1 Sample Transform Use Cases

```

pvx voc dialog.wav --transform fft --time-stretch 1.08 --transient-preserve --output-dir out
pvx voc test_tones.wav --transform dft --time-stretch 1.00 --output-dir out
pvx voc archival_take.wav --transform czt --n-fft 1536 --win-length 1536 --hop-size 384 --output-dir out
pvx voc strings.wav --transform dct --pitch-shift-cents -17 --soft-clip-level 0.94 --output-dir out
pvx voc percussion.wav --transform dst --time-stretch 0.92 --output-dir out
pvx voc synth.wav --transform hartley --phase-locking off --time-stretch 1.30 --output-dir out

```

6.4 Phase-Vocoder Frequency and Phase Update

$$\Delta\phi_t[k] = \text{princarg}(\phi_t[k] - \phi_{t-1}[k] - \omega_k H_a)$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

$$\hat{\omega}_t[k] = \omega_k + \frac{\Delta\phi_t[k]}{H_a}$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

$$\hat{\phi}_t[k] = \hat{\phi}_{t-1}[k] + \hat{\omega}_t[k] H_s$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Plain English: pvx estimates true per-bin frequency from wrapped phase deviation, then re-accumulates phase at the synthesis hop.

6.5 Time Stretch and Pitch Ratio

Pitch shift ratio from semitones/cents:

$$r_{\text{pitch}} = 2^{\Delta s/12} = 2^{\Delta c/1200}$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Plain English: semitone and cent controls map to the same multiplicative ratio.

6.6 Dynamic Control Interpolation (CSV/JSON)

When a numeric flag receives a CSV/JSON control track, pvx samples a control function $u(t)$ over render time.

Polynomial mode fits a global polynomial of degree d :

$$u(t) = \sum_{i=0}^d a_i t^i, \quad d = \min(\text{order}, M - 1)$$

where M is the number of control points. In command-line interface (CLI) usage, `--order` accepts any integer ≥ 1 . The effective degree is automatically capped to avoid over-specification when there are too few points.

Nearest/linear/cubic/exponential/S-curve are local interpolation modes; `none` is sample-and-hold (stairstep). `exponential` uses piecewise exponential easing and `s_curve/smoothstep` use Hermite S-shaped easing between adjacent control points. Legend used in each plot: blue solid line = interpolated control curve $u(t)$, red dashed line = piecewise connection of control points, red circles labeled p_i = original control points.

Interpolation mode	CLI form	Example plot
none (stairstep)	<code>--interp none</code>	
nearest	<code>--interp nearest</code>	
linear	<code>--interp linear</code>	
cubic	<code>--interp cubic</code>	
exponential	<code>--interp exponential</code>	
S-curve (smoothstep)	<code>--interp s_curve</code>	
S-curve (smoothstep)	<code>--interp smoothstep</code>	
polynomial order 1	<code>--interp polynomial --order 1</code>	
polynomial order 2	<code>--interp polynomial --order 2</code>	
polynomial order 3	<code>--interp polynomial --order 3</code>	
polynomial order 5	<code>--interp polynomial --order 5</code>	

6.7 Microtonal Retune Mapping

For a detected frequency f , pvx scale-aware retuning chooses the nearest permitted scale target f_{scale} and applies:

$$\Delta s = 12 \log_2 \left(\frac{f_{\text{scale}}}{f} \right)$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Plain English: each frame is shifted only as much as needed to land on the selected microtonal scale degree.

6.8 Loudness and Dynamics

LUFS target gain:

$$g_{\text{LUFS}} = 10^{(L_{\text{target}} - L_{\text{in}})/20}, \quad y[n] = g_{\text{LUFS}} x[n]$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Compressor gain law (conceptual):

$$g(x) = \begin{cases} 1, & |x| \leq T \\ \left(\frac{T + (|x| - T)/R}{|x|} \right), & |x| > T \end{cases}$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Plain English: level is reduced above threshold T by ratio R , then makeup/loudness stages may be applied.

6.9 Spatial Coherence and Channel Alignment

Inter-channel phase difference:

$$\Delta\phi_{ij}(k, t) = \phi_i(k, t) - \phi_j(k, t)$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Phase-drift objective:

$$J = \sum_{k,t} |\Delta\phi_{ij}^{\text{out}}(k, t) - \Delta\phi_{ij}^{\text{in}}(k, t)|$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Channel delay estimate by phase-weighted cross-correlation:

$$\tau^* = \arg \max_{\tau} \mathcal{F}^{-1} \left\{ \frac{X_i(\omega) X_j^*(\omega)}{|X_i(\omega) X_j^*(\omega)| + \varepsilon} \right\}(\tau)$$

where the symbols retain the meanings established in this section and subscripts identify samples, frames, channels, or spectral bins as applicable.

Plain English: pvx spatial modules prioritize channel coherence, stable image cues, and robust delay alignment for multichannel processing chains.

6.10 Function Graph Gallery

The plots below visualize core transfer functions and parameter curves used across pvx processing modules.

Function family	Plot	Why this matters
Pitch ratio from semitones		<code>r = 2^(semitones/12)</code> conversion used by pitch-shift flags.
Pitch ratio from cents		<code>r = 2^(cents/1200)</code> conversion for microtonal cent controls.
Dynamics transfer functions		Compressor/expander/limiter conceptual transfer curves used in mastering stages.
Soft-clip transfer functions		Shows <code>tanh</code> , <code>arctan</code> , and <code>cubic</code> soft clipping behavior.
Morph blend magnitude curves		Compares <code>linear</code> , <code>geometric</code> , <code>product</code> , <code>max_mag</code> , and <code>min_mag</code> blend behaviors.
Mask exponent response		Shows how <code>--mask-exponent</code> shapes cross-synthesis mask gain.
Phase mix to output angle		Example of complex-vector phase blending used by morph phase mix.

CHAPTER 7

pvx Architecture

Generated from commit 35e9761 (commit date: 2026-05-25T08:14:42-04:00).

System architecture for runtime processing, algorithm dispatch, and documentation pipelines.

7.1 Runtime and CLI Flow

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

7.2 Algorithm Registry and Dispatch

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

7.3 Documentation Build Graph

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

7.4 CI + Pages

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

CHAPTER 8

pvx Window Reference

This chapter is a mathematical and visual atlas of every analysis window supported by pvx. A window is not a decorative preprocessing choice. It determines which samples contribute most strongly to each frame, how a sinusoid spreads across bins, how much neighbouring partials interfere, and how sharply a transient can be located in time.

8.1 Notation

The notation below is used throughout the atlas. Each symbol is restated in the where clause following every equation so an entry can be read independently.

The principal quantities are N , the window length; n , the sample index in $0, \dots, N-1$; $m = (N-1)/2$, the center index; $w[n]$, the window coefficient; and $W(e^{j\omega})$, the discrete-time Fourier transform of the window.

8.2 Formula Families

The 50 named windows map to 19 formula families. The constants and parameter values in each atlas entry identify the exact member used by pvx.

8.2.1 W0

$$w[n] = 1$$

where $w[n]$ is the window coefficient and n is any sample index from 0 through $N-1$.

8.2.2 W1

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

8.2.3 W2

$$w[n] = \sin\left(\frac{\pi n}{N-1}\right)$$

where n is the sample index and N is the window length.

8.2.4 W3

$$w[n] = 1 - \left|\frac{n-m}{m}\right|$$

where $m = (N-1)/2$ is the center sample, n is the sample index, and N is the window length.

8.2.5 W4

$$w[n] = \max\left(1 - \left|\frac{n-m}{(N+1)/2}\right|, 0\right)$$

where $m = (N-1)/2$ is the center sample, n is the sample index, and N is the window length.

8.2.6 W5

$$x = \frac{n}{N-1} - \frac{1}{2}, \quad w[n] = 0.62 - 0.48|x| + 0.38 \cos(2\pi x)$$

where x is the centered normalized position, n is the sample index, and N is the window length.

8.2.7 W6

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos\left(\pi \left(\frac{2x}{\alpha} - 1\right)\right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos\left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1\right)\right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N-1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

8.2.8 W7

$$w[n] = \begin{cases} 1 - 6u^2 + 6u^3, & 0 \leq u \leq 1/2 \\ 2(1-u)^3, & 1/2 < u \leq 1 \\ 0, & u > 1 \end{cases}$$

where $u = |2n/(N-1) - 1|$ is absolute centered position, n is the sample index, and N is the window length.

8.2.9 W8

$$w[n] = \text{sinc}\left(\frac{2n}{N-1} - 1\right)$$

where $\text{sinc}(x) = \sin(\pi x)/(\pi x)$, n is the sample index, and N is the window length.

8.2.10 W9

$$w[n] = \max(1 - x_n^2, 0)$$

where $x_n = (n-m)/m$ is centered normalized position, $m = (N-1)/2$, and N is the window length.

8.2.11 W10

$$w[n] = \exp\left[-\frac{1}{2} \left(\frac{n-m}{\sigma}\right)^2\right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

8.2.12 W11

$$w[n] = \exp\left[-\frac{1}{2} \left|\frac{n-m}{\sigma}\right|^{2p}\right]$$

where p controls shoulder shape, σ controls spread, $m = (N-1)/2$, and n is the sample index.

8.2.13 W12

$$w[n] = \exp\left(-\frac{|n-m|}{\tau}\right), \quad \tau = r_\tau m$$

where τ is the decay constant, r_τ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

8.2.14 W13

$$w[n] = \frac{1}{1 + \left(\frac{n-m}{\gamma}\right)^2}, \quad \gamma = r_\gamma m$$

where γ is the Lorentzian scale, r_γ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

8.2.15 W14

$$w[n] = \sin^p\left(\frac{\pi n}{N-1}\right)$$

where p is the cosine-power exponent, n is the sample index, and N is the window length.

8.2.16 W15

$$w[n] = w_{\text{Hann}}[n] \exp\left(-\alpha \frac{|n-m|}{m}\right)$$

where $w_{\text{Hann}}[n]$ is the Hann window, α controls exponential decay, and $m = (N-1)/2$.

8.2.17 W16

$$w[n] = \alpha - (1-\alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

8.2.18 W17

$$w[n] = (1-x) \cos(\pi x) + \frac{\sin(\pi x)}{\pi}$$

where $x = |2n/(N-1) - 1|$ is absolute centered position, n is the sample index, and N is the window length.

8.2.19 W18

$$w[n] = \frac{I_0\left(\beta \sqrt{1-r_n^2}\right)}{I_0(\beta)}, \quad r_n = \frac{n-m}{m}$$

where I_0 is the modified Bessel function, β controls taper strength, $m = (N-1)/2$, and n is the sample index.

8.3 Quantitative Metrics

The comparison uses coherent gain, equivalent noise bandwidth, scalloping loss, main-lobe width, and peak sidelobe level. Together these measurements describe amplitude calibration, noise integration, sensitivity to frequencies between FFT bins, frequency discrimination, and leakage rejection.

$$CG = \frac{1}{N} \sum_{n=0}^{N-1} w[n]$$

where CG is coherent gain, $w[n]$ is the window coefficient, n is the sample index, and N is the window length.

$$ENBW = N \frac{\sum_{n=0}^{N-1} w^2[n]}{\left(\sum_{n=0}^{N-1} w[n]\right)^2}$$

where $ENBW$ is equivalent noise bandwidth measured in FFT bins, $w[n]$ is the window coefficient, n is the sample index, and N is the window length.

$$SL = 20 \log_{10} \left| \frac{W(2\pi/(2N))}{W(0)} \right|$$

where SL is half-bin scalloping loss in decibels, $W(\omega)$ is the window spectrum, and N is the window length.

8.4 Comparative Window Summary

The following landscape table is deliberately limited to the quantities most useful for first selection. The atlas entries that follow carry the parameters, complete formula, strengths, limitations, advice, and full-size graphs.

Table 8.1: Comparison of all pvx analysis windows.

Window	Family	CG	ENBW	Main lobe	Sidelobe	Recommended use
hann	Cosine series	0.499756	1.500733	4.000	-31.468	Default choice for most pvx time-stretch and pitch-shift workflows.
hamming	Cosine series	0.539775	1.363305	4.000	-42.675	Default choice for most pvx time-stretch and pitch-shift workflows.
blackman	Cosine series	0.419795	1.727601	6.000	-58.109	Use for dense harmonic material when leakage artifacts dominate.
blackmanharris	Cosine series	0.358575	2.005332	8.000	-92.011	Use for dense harmonic material when leakage artifacts dominate.
nutall	Cosine series	0.355594	2.022220	8.000	-93.325	Use for dense harmonic material when leakage artifacts dominate.
flat	Cosine series	0.999514	3.772117	10.000	-68.311	Use for measurement-grade magnitude tracking, not fine pitch separation.
blackman_nuttall	Cosine series	0.363405	1.977073	8.000	-98.174	Use for dense harmonic material when leakage artifacts dominate.
exact_blackman	Cosine series	0.426386	1.694500	6.000	-68.236	Use for dense harmonic material when leakage artifacts dominate.
sine	Sinusoidal	0.636309	1.234304	3.000	-22.999	Use for stable, low-complexity alternatives to Hann.
bartlett	Triangular	0.499756	1.333985	4.000	-26.523	Use for low-cost processing or quick exploratory runs.
boxcar	Rectangular	1.000000	1.000000	2.000	-13.264	Use only for controlled test tones or when leakage is acceptable.
triangular	Triangular	0.500244	1.332683	4.000	-26.523	Use for low-cost processing or quick exploratory runs.
bartlett_hann	Hybrid taper	0.499756	1.456559	4.000	-35.874	Use as a middle-ground taper when Bartlett feels too sharp.
tukey	Tukey	0.749634	1.222819	2.656	-15.123	Use when you need a controllable flat center with soft edges.
tukey_0p1	Tukey	0.949536	1.039289	2.094	-13.309	Use when you need a controllable flat center with soft edges.
tukey_0p25	Tukey	0.874573	1.102579	2.281	-13.601	Use when you need a controllable flat center with soft edges.
tukey_0p75	Tukey	0.624695	1.360664	3.188	-19.395	Use when you need a controllable flat center with soft edges.
tukey_0p9	Tukey	0.549731	1.446988	3.625	-24.972	Use when you need a controllable flat center with soft edges.

Window	Family	CG	ENBW	Main lobe	Sidelobe	Recommended use
<code>parzen</code>	Polynomial	0.374817	1.918397	8.000	-53.046	Use for spectral denoising/analysis where sidelobe cleanup is critical.
<code>lanczos</code>	Sinc	0.589202	1.299668	3.281	-26.405	Use for interpolation-adjacent analysis experiments.
<code>welch</code>	Quadratic	0.666341	1.200587	2.875	-21.295	Use when center weighting is desired with minimal complexity.
<code>gaussian_0p25</code>	Gaussian	0.313156	2.258144	11.406	-87.677	Use sigma presets to tune transient locality versus spectral leakage.
<code>gaussian_0p35</code>	Gaussian	0.436580	1.626488	6.719	-52.077	Use sigma presets to tune transient locality versus spectral leakage.
<code>gaussian_0p45</code>	Gaussian	0.548949	1.320556	4.125	-35.366	Use sigma presets to tune transient locality versus spectral leakage.
<code>gaussian_0p55</code>	Gaussian	0.641515	1.171861	3.000	-28.906	Use sigma presets to tune transient locality versus spectral leakage.
<code>gaussian_0p65</code>	Gaussian	0.713490	1.097657	2.625	-22.575	Use sigma presets to tune transient locality versus spectral leakage.
<code>general_gaussian_1p5_0p35</code>	Generalized Gaussian	0.393587	2.016584	5.156	-24.895	Use for research/tuning tasks where shoulder steepness matters.
<code>general_gaussian_2p0_0p35</code>	Generalized Gaussian	0.377081	2.230016	5.281	-19.717	Use for research/tuning tasks where shoulder steepness matters.
<code>general_gaussian_3p0_0p35</code>	Generalized Gaussian	0.364287	2.445593	5.469	-16.299	Use for research/tuning tasks where shoulder steepness matters.
<code>general_gaussian_4p0_0p35</code>	Generalized Gaussian	0.359267	2.552433	5.562	-15.060	Use for research/tuning tasks where shoulder steepness matters.
<code>exponential_0p25</code>	Exponential	0.245310	2.075492	15.562	-31.885	Use for experiments emphasizing center-heavy weighting.
<code>exponential_0p5</code>	Exponential	0.432187	1.313323	3.625	-19.190	Use for experiments emphasizing center-heavy weighting.
<code>exponential_1p0</code>	Exponential	0.631991	1.082055	2.594	-20.141	Use for experiments emphasizing center-heavy weighting.
<code>cauchy_0p5</code>	Cauchy / Lorentzian	0.553402	1.229775	3.562	-22.733	Use when you want softer attenuation of far-window samples.
<code>cauchy_1p0</code>	Cauchy / Lorentzian	0.785259	1.041963	2.375	-19.034	Use when you want softer attenuation of far-window samples.
<code>cauchy_2p0</code>	Cauchy / Lorentzian	0.927233	1.004393	2.094	-14.689	Use when you want softer attenuation of far-window samples.
<code>cosine_power_2</code>	Cosine power	0.499756	1.500733	4.000	-31.468	Use to smoothly increase edge attenuation versus basic sine.

Window	Family	CG	ENBW	Main lobe	Sidelobe	Recommended use
<code>cosine_power_3</code>	Cosine power	0.424206	1.735739	5.000	-39.295	Use to smoothly increase edge attenuation versus basic sine.
<code>cosine_power_4</code>	Cosine power	0.374817	1.945394	6.000	-46.741	Use to smoothly increase edge attenuation versus basic sine.
<code>hann_poisson_0p5</code>	Hann-Poisson	0.432946	1.609944	5.188	-35.245	Use when you need stronger edge decay than Hann without full flattop cost.
<code>hann_poisson_1p0</code>	Hann-Poisson	0.378797	1.734176	103.750	-80.017	Use when you need stronger edge decay than Hann without full flattop cost.
<code>hann_poisson_2p0</code>	Hann-Poisson	0.297878	2.023174	164.719	-80.005	Use when you need stronger edge decay than Hann without full flattop cost.
<code>general_hamming_0p50</code>	General Hamming	0.499756	1.500733	4.000	-31.468	Use to sweep sidelobe-vs-width behavior near Hamming family defaults.
<code>general_hamming_0p60</code>	General Hamming	0.599805	1.222475	3.469	-31.600	Use to sweep sidelobe-vs-width behavior near Hamming family defaults.
<code>general_hamming_0p70</code>	General Hamming	0.699854	1.091920	2.656	-24.078	Use to sweep sidelobe-vs-width behavior near Hamming family defaults.
<code>general_hamming_0p80</code>	General Hamming	0.799902	1.031273	2.312	-18.649	Use to sweep sidelobe-vs-width behavior near Hamming family defaults.
<code>bohman</code>	Bohman	0.405087	1.786613	6.000	-45.997	Use for exploratory spectral work needing smooth derivatives.
<code>cosine</code>	Sinusoidal	0.636309	1.234304	3.000	-22.999	Use for stable, low-complexity alternatives to Hann.
<code>kaiser</code>	Kaiser-Bessel	0.331708	2.162181	9.125	-105.921	Use when you need explicit control over leakage versus resolution.
<code>rect</code>	Rectangular	1.000000	1.000000	2.000	-13.264	Use only for controlled test tones or when leakage is acceptable.

8.5 Complete Window Atlas

Each entry begins with the exact formula family and parameters, followed by calibrated metrics, a time-domain graph, a frequency-domain graph, and listening guidance. The repeated structure is intentional: it supports direct comparison without forcing the reader to shuttle between a catalog and a distant plate section.

8.5.1 hann

The **hann** window belongs to the cosine series family. Its configured parameters are `coeffs=(0.5, -0.5)`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.2: Measured properties of the hann window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.500733 bins
Scalloping loss	-1.422 dB
Main-lobe width	4.000 bins
Peak sidelobe	-31.468 dB

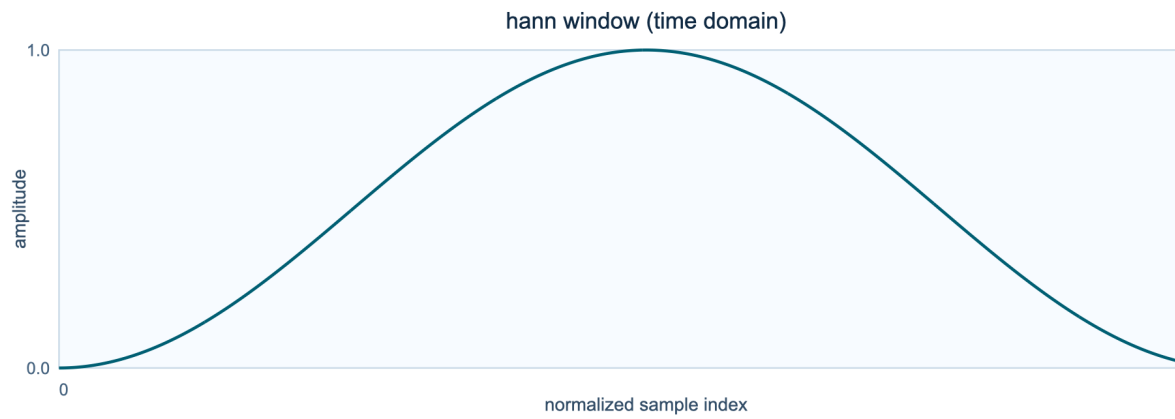


Figure 8.1: Time-domain shape of the hann window.

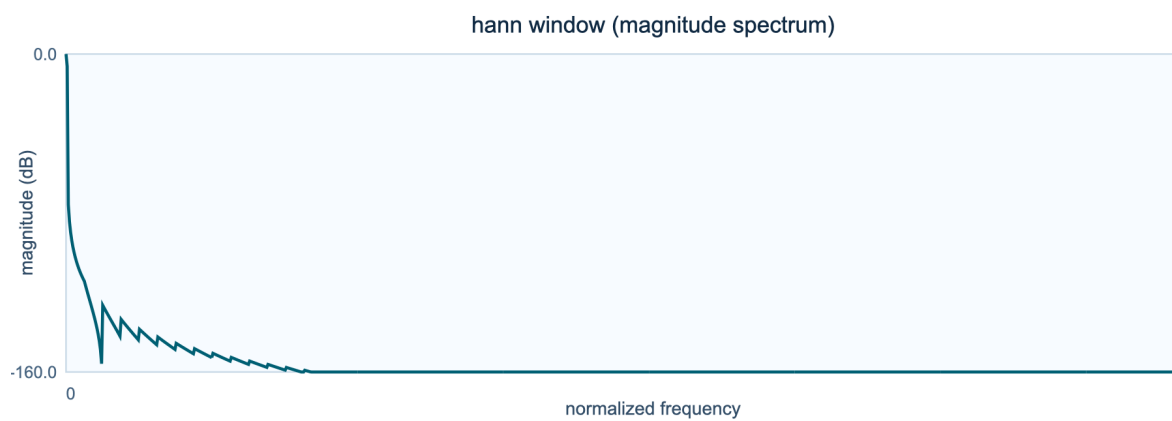


Figure 8.2: Magnitude spectrum of the hann window.

8.5.1.1 Interpretation and use

The principal strength of this window is straightforward: Balanced leakage suppression and frequency resolution. That advantage should be weighed against its main limitation: Not optimal for amplitude metering or extreme sidelobe rejection.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Default choice for most pvx time-stretch and pitch-shift workflows. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.2 hamming

The **hamming** window belongs to the cosine series family. Its configured parameters are **coeffs**=(0.54, -0.46). This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.3: Measured properties of the hamming window.

Metric	Value
Coherent gain	0.539775
Equivalent noise bandwidth	1.363305 bins
Scalloping loss	-1.750 dB
Main-lobe width	4.000 bins
Peak sidelobe	-42.675 dB

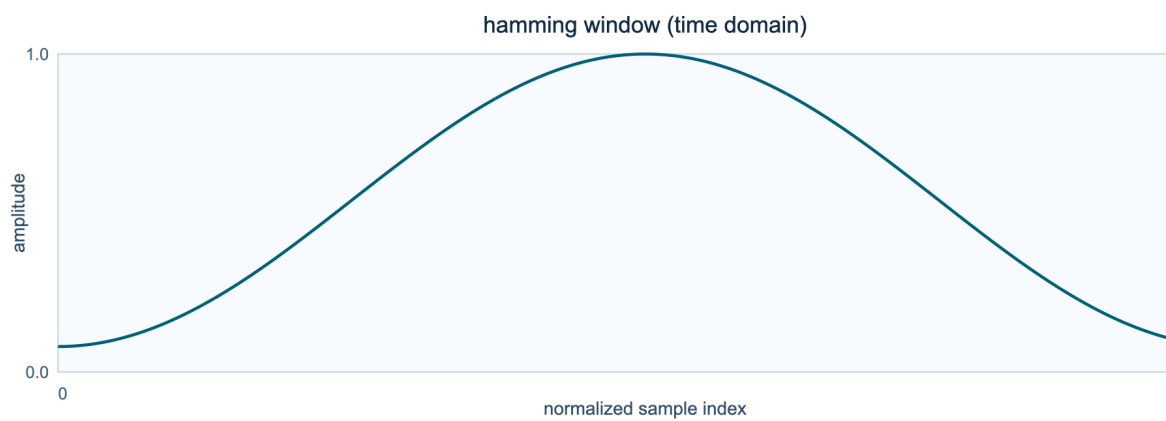


Figure 8.3: Time-domain shape of the hamming window.

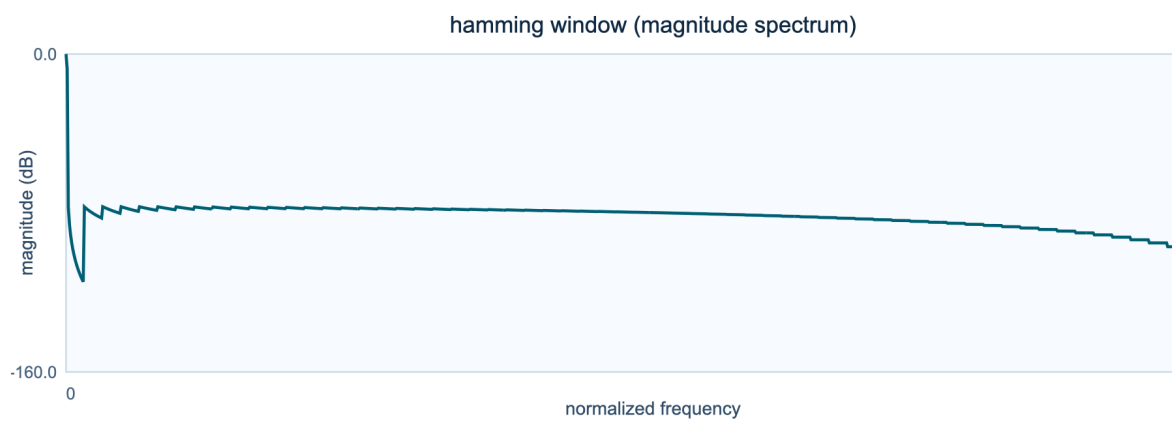


Figure 8.4: Magnitude spectrum of the hamming window.

8.5.2.1 Interpretation and use

The principal strength of this window is straightforward: Balanced leakage suppression and frequency resolution. That advantage should be weighed against its main limitation: Not optimal for amplitude metering or extreme sidelobe rejection.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Default choice for most pvx time-stretch and pitch-shift workflows. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.3 blackman

The **blackman** window belongs to the cosine series family. Its configured parameters are **coeffs**=(0.42, -0.5, 0.08). This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.4: Measured properties of the blackman window.

Metric	Value
Coherent gain	0.419795
Equivalent noise bandwidth	1.727601 bins
Scalloping loss	-1.098 dB
Main-lobe width	6.000 bins
Peak sidelobe	-58.109 dB

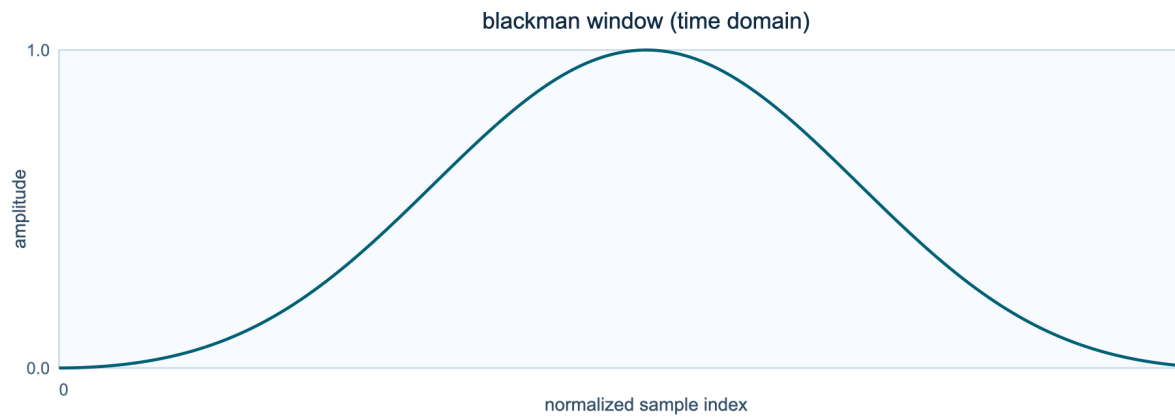


Figure 8.5: Time-domain shape of the blackman window.

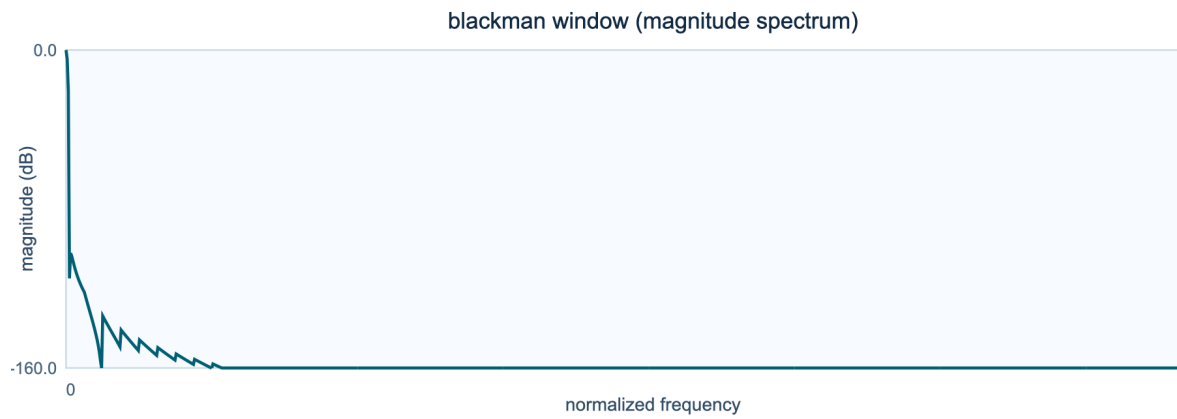


Figure 8.6: Magnitude spectrum of the blackman window.

8.5.3.1 Interpretation and use

The principal strength of this window is straightforward: Strong sidelobe suppression for cleaner spectral separation. That advantage should be weighed against its main limitation: Wider main lobe than Hann/Hamming.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for dense harmonic material when leakage artifacts dominate. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.4 blackmanharris

The `blackmanharris` window belongs to the cosine series family. Its configured parameters are `coeffs=(0.35875, -0.48829, 0.14128, -0.01168)`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.5: Measured properties of the blackmanharris window.

Metric	Value
Coherent gain	0.358575
Equivalent noise bandwidth	2.005332 bins
Scalloping loss	-0.825 dB
Main-lobe width	8.000 bins
Peak sidelobe	-92.011 dB

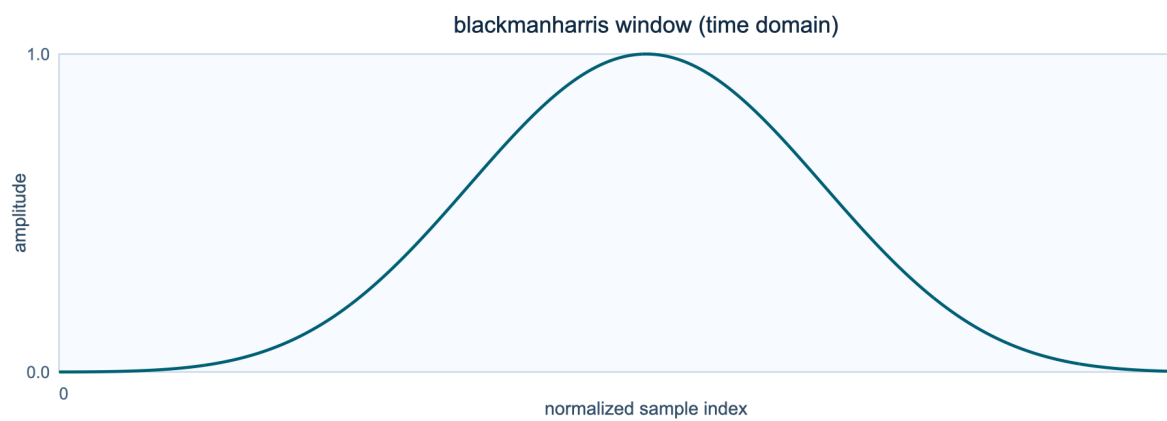


Figure 8.7: Time-domain shape of the blackmanharris window.

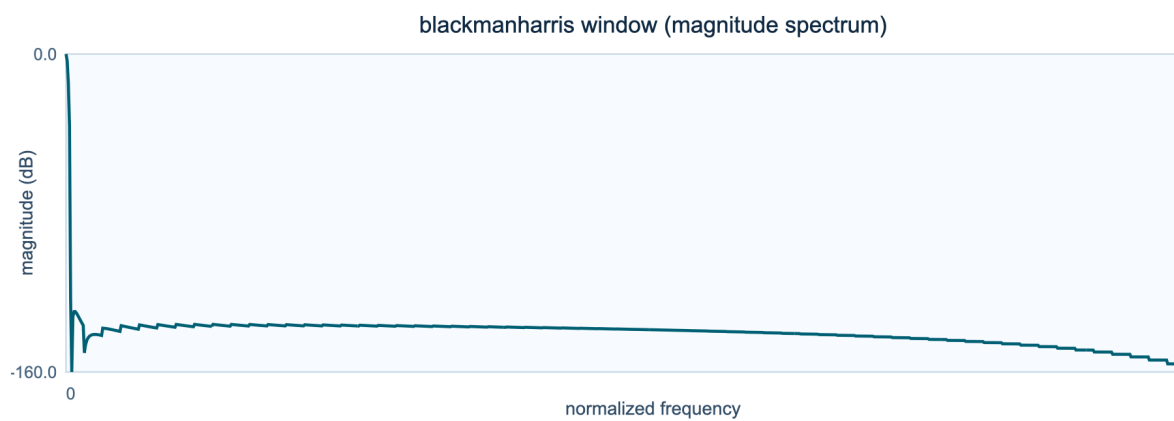


Figure 8.8: Magnitude spectrum of the blackmanharris window.

8.5.4.1 Interpretation and use

The principal strength of this window is straightforward: Strong sidelobe suppression for cleaner spectral separation. That advantage should be weighed against its main limitation: Wider main lobe than Hann/Hamming.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for dense harmonic material when leakage artifacts dominate. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.5 nuttall

The **nuttall** window belongs to the cosine series family. Its configured parameters are **coeffs**=(0.355768, -0.487396, 0.144232, -0.012604). This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.6: Measured properties of the nuttall window.

Metric	Value
Coherent gain	0.355594
Equivalent noise bandwidth	2.022220 bins
Scalloping loss	-0.811 dB
Main-lobe width	8.000 bins
Peak sidelobe	-93.325 dB

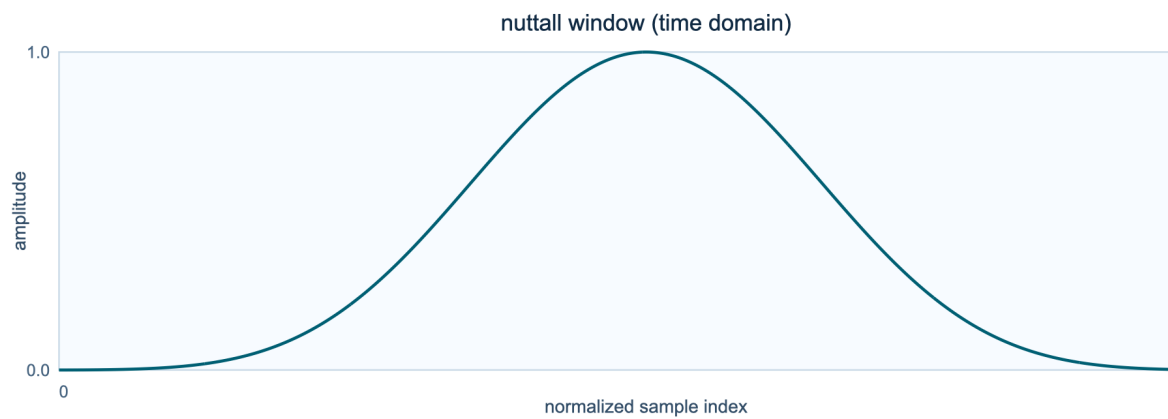


Figure 8.9: Time-domain shape of the nuttall window.

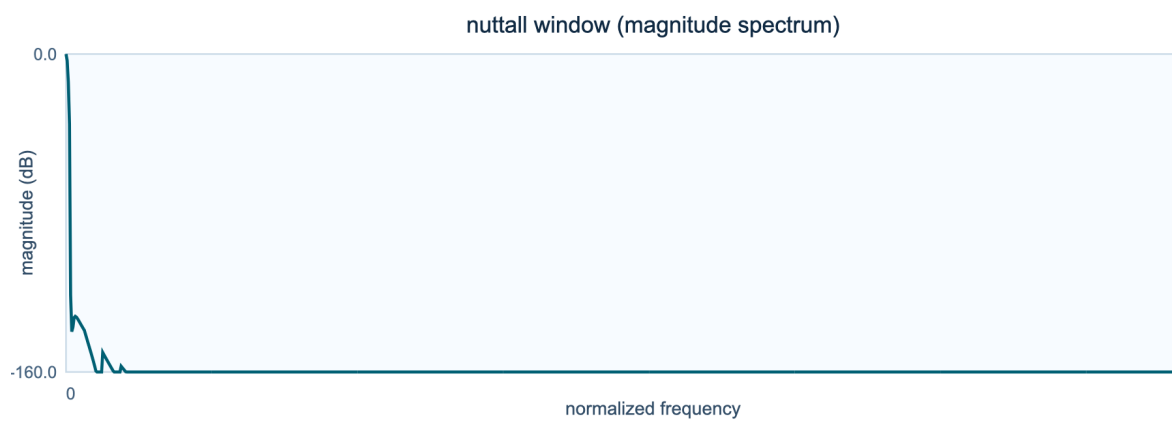


Figure 8.10: Magnitude spectrum of the nuttall window.

8.5.5.1 Interpretation and use

The principal strength of this window is straightforward: Strong sidelobe suppression for cleaner spectral separation. That advantage should be weighed against its main limitation: Wider main lobe than Hann/Hamming.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for dense harmonic material when leakage artifacts dominate. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.6 flattop

The **flattop** window belongs to the cosine series family. Its configured parameters are `coeffs=(1, -1.93, 1.29, -0.388, 0.0322)`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.7: Measured properties of the flattop window.

Metric	Value
Coherent gain	0.999514
Equivalent noise bandwidth	3.772117 bins
Scalloping loss	-0.016 dB
Main-lobe width	10.000 bins
Peak sidelobe	-68.311 dB

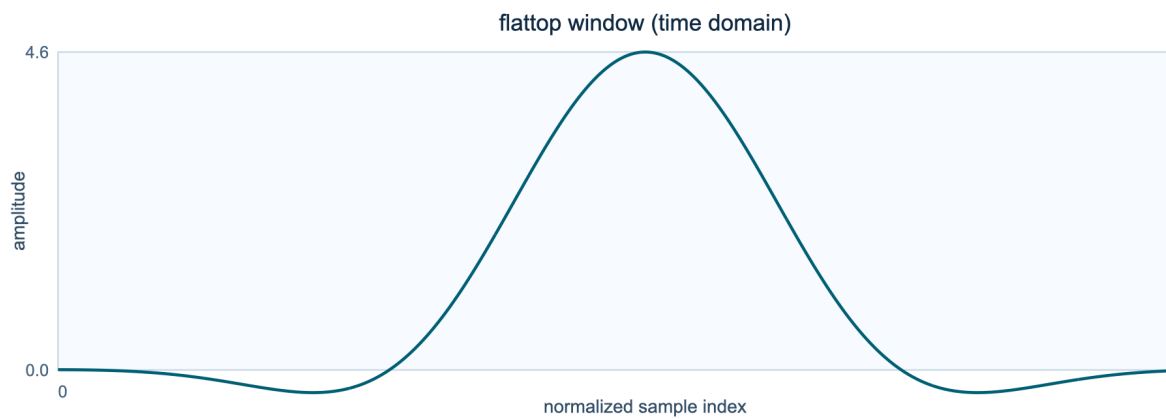


Figure 8.11: Time-domain shape of the flattop window.

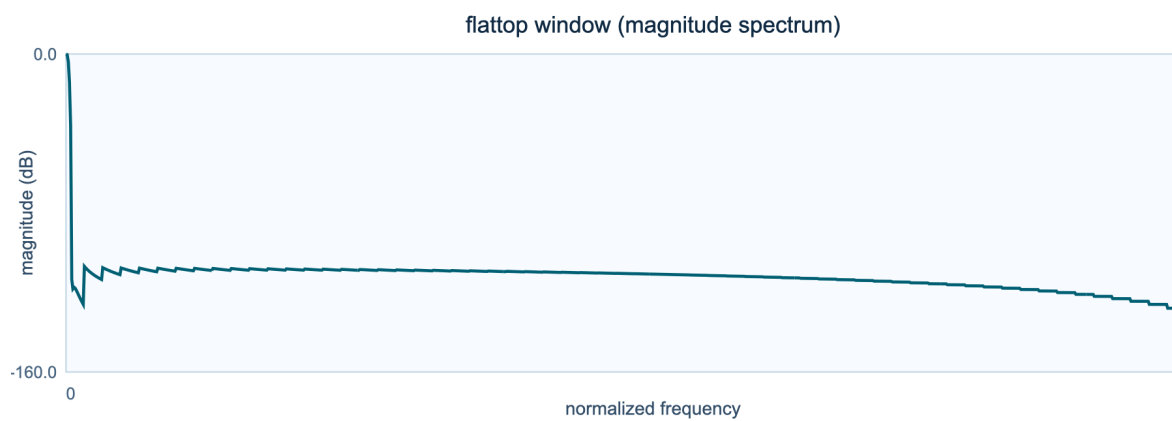


Figure 8.12: Magnitude spectrum of the flattop window.

8.5.6.1 Interpretation and use

The principal strength of this window is straightforward: Very accurate amplitude estimation in FFT bins. That advantage should be weighed against its main limitation: Very wide main lobe and reduced frequency discrimination.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for measurement-grade magnitude tracking, not fine pitch separation. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.7 blackman nuttall

The `blackman_nuttall` window belongs to the cosine series family. Its configured parameters are `coeffs=(0.363582, -0.489177, 0.1366, -0.0106411)`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.8: Measured properties of the blackman nuttall window.

Metric	Value
Coherent gain	0.363405
Equivalent noise bandwidth	1.977073 bins
Scalloping loss	-0.850 dB
Main-lobe width	8.000 bins
Peak sidelobe	-98.174 dB

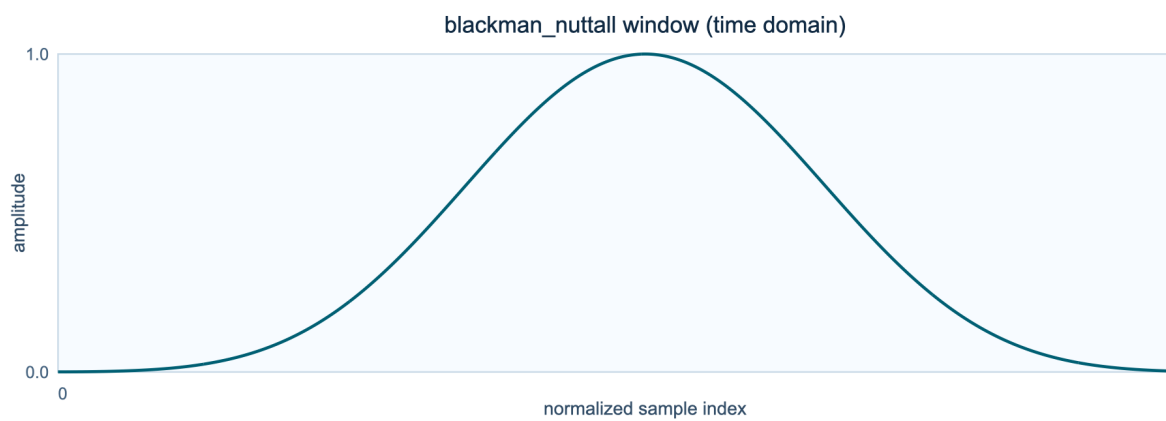


Figure 8.13: Time-domain shape of the blackman nuttall window.

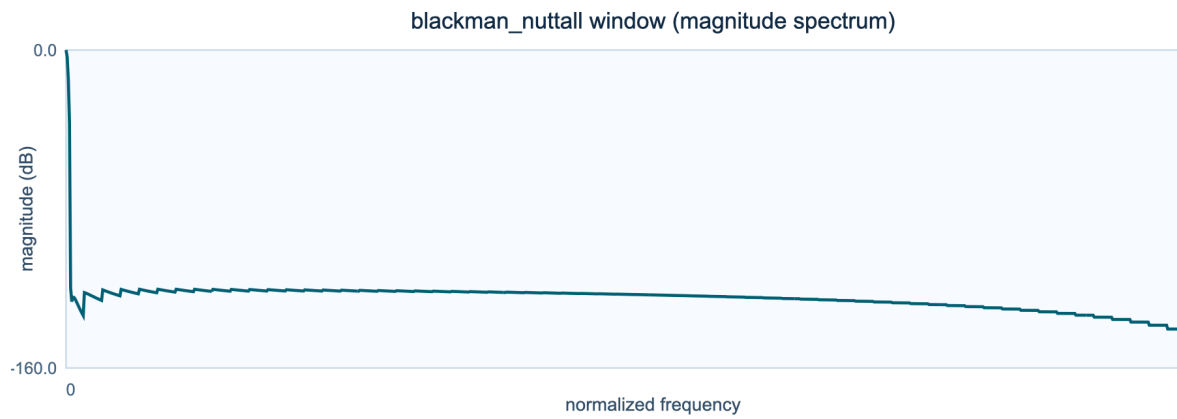


Figure 8.14: Magnitude spectrum of the blackman nuttall window.

8.5.7.1 Interpretation and use

The principal strength of this window is straightforward: Strong sidelobe suppression for cleaner spectral separation. That advantage should be weighed against its main limitation: Wider main lobe than Hann/Hamming.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for dense harmonic material when leakage artifacts dominate. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.8 exact blackman

The `exact_blackman` window belongs to the cosine series family. Its configured parameters are `coeffs=(0.426591, -0.496561, 0.0768487)`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.9: Measured properties of the exact blackman window.

Metric	Value
Coherent gain	0.426386
Equivalent noise bandwidth	1.694500 bins
Scalloping loss	-1.149 dB
Main-lobe width	6.000 bins
Peak sidelobe	-68.236 dB

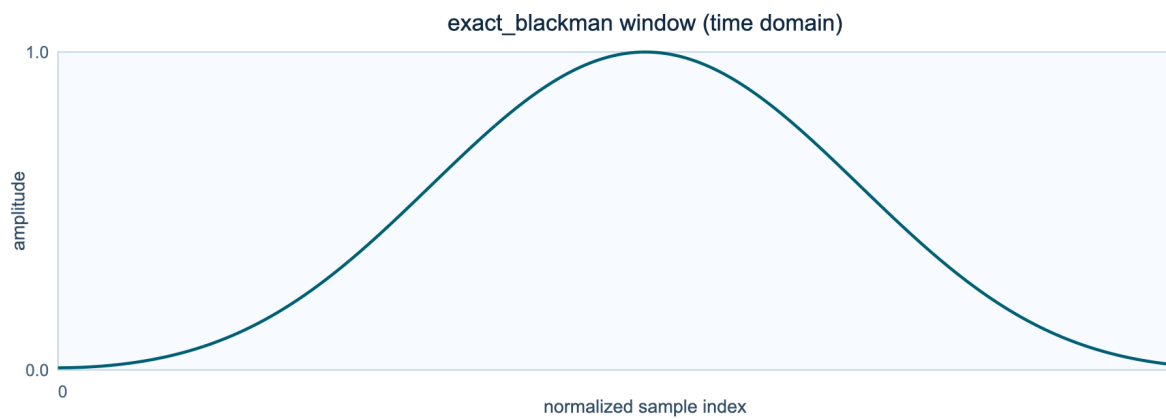


Figure 8.15: Time-domain shape of the exact blackman window.

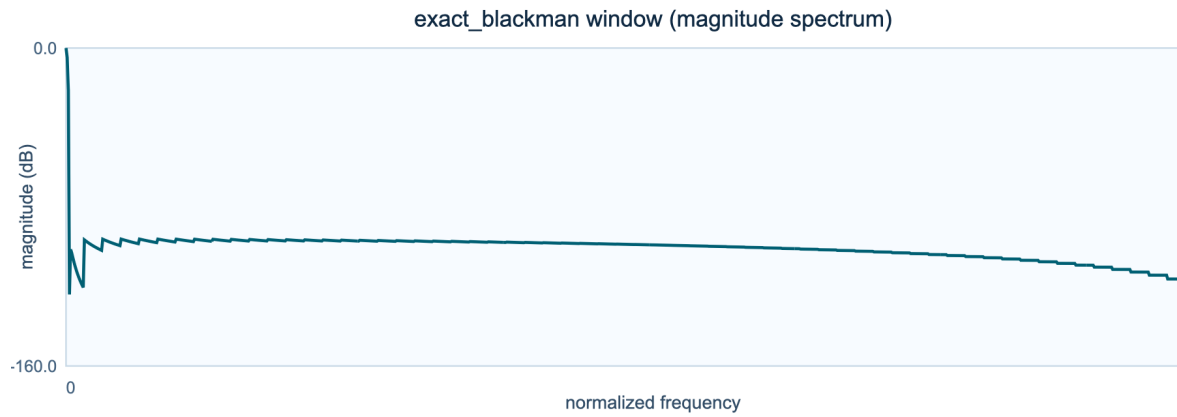


Figure 8.16: Magnitude spectrum of the exact blackman window.

8.5.8.1 Interpretation and use

The principal strength of this window is straightforward: Strong sidelobe suppression for cleaner spectral separation. That advantage should be weighed against its main limitation: Wider main lobe than Hann/Hamming.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for dense harmonic material when leakage artifacts dominate. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.9 sine

The **sine** window belongs to the sinusoidal family. Its configured parameters are **none**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sin\left(\frac{\pi n}{N-1}\right)$$

where n is the sample index and N is the window length.

Table 8.10: Measured properties of the sine window.

Metric	Value
Coherent gain	0.636309
Equivalent noise bandwidth	1.234304 bins
Scalloping loss	-2.096 dB
Main-lobe width	3.000 bins
Peak sidelobe	-22.999 dB

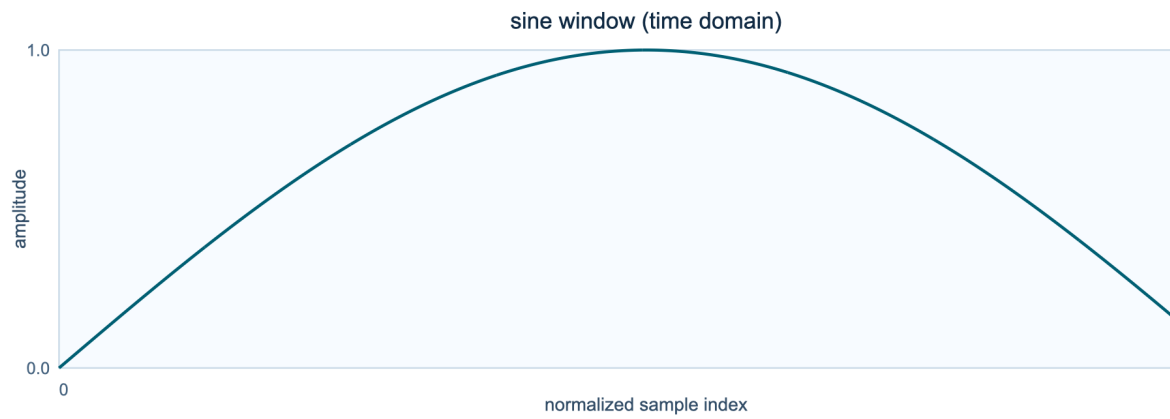


Figure 8.17: Time-domain shape of the sine window.

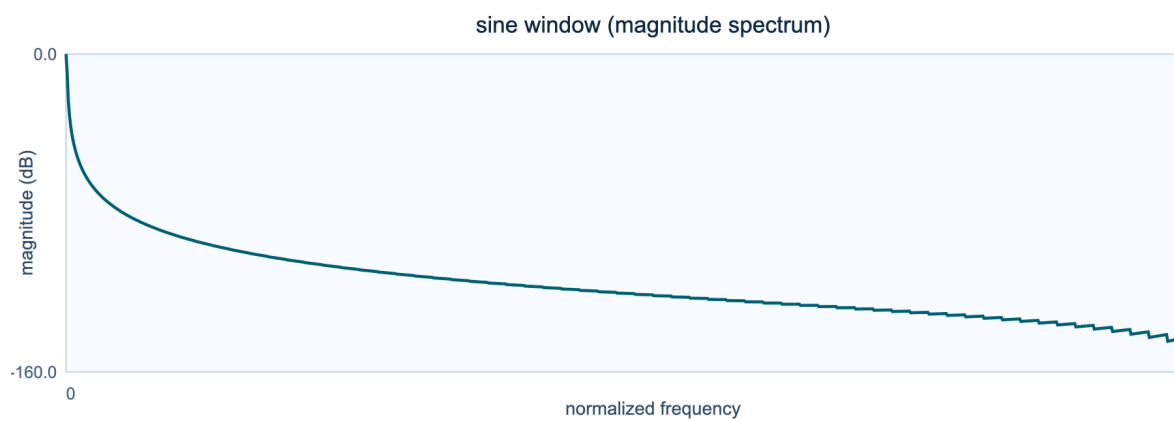


Figure 8.18: Magnitude spectrum of the sine window.

8.5.9.1 Interpretation and use

The principal strength of this window is straightforward: Smooth endpoint behavior and straightforward implementation. That advantage should be weighed against its main limitation: Less configurable than Kaiser/Tukey families.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for stable, low-complexity alternatives to Hann. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.10 bartlett

The **bartlett** window belongs to the triangular family. Its configured parameters are **none**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = 1 - \left| \frac{n-m}{m} \right|$$

where $m = (N - 1)/2$ is the center sample, n is the sample index, and N is the window length.

Table 8.11: Measured properties of the bartlett window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.333985 bins
Scalloping loss	-1.822 dB
Main-lobe width	4.000 bins
Peak sidelobe	-26.523 dB

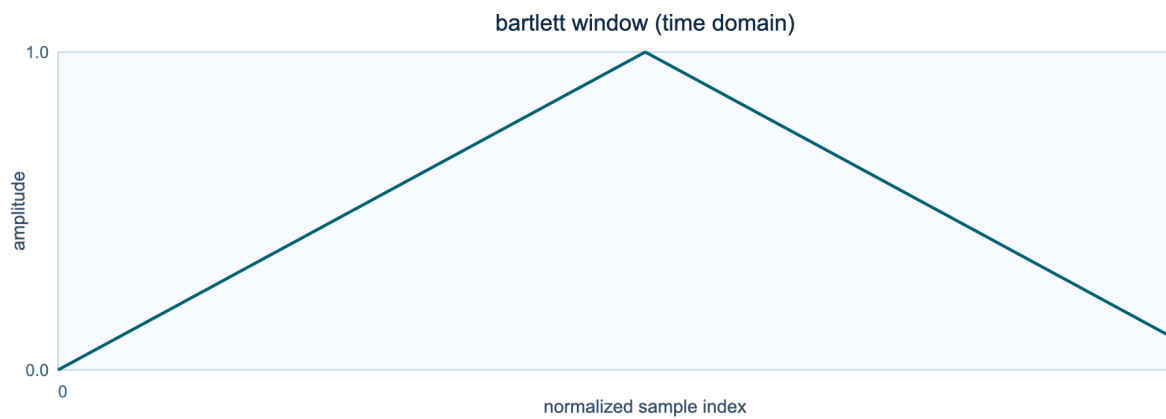


Figure 8.19: Time-domain shape of the bartlett window.

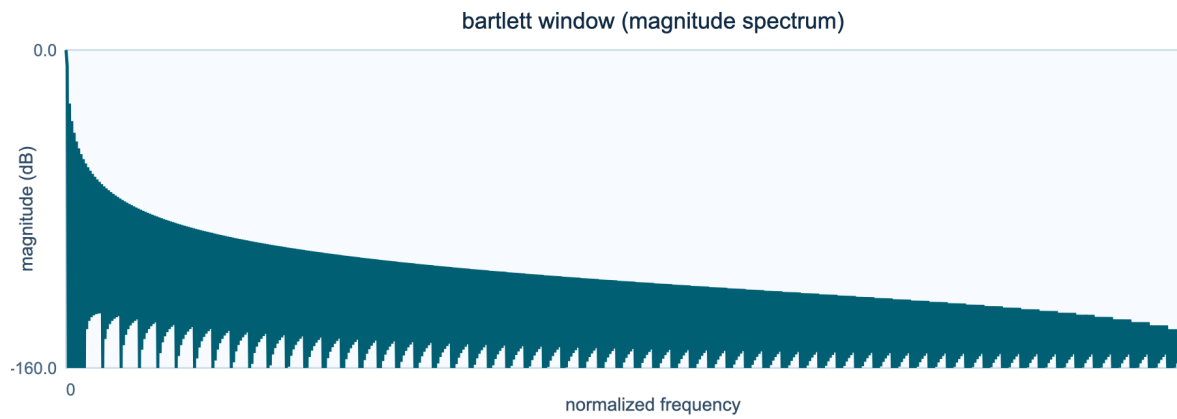


Figure 8.20: Magnitude spectrum of the bartlett window.

8.5.10.1 Interpretation and use

The principal strength of this window is straightforward: Cheap linear taper with intuitive behavior. That advantage should be weighed against its main limitation: Higher sidelobes than stronger cosine-sum windows.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for low-cost processing or quick exploratory runs. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.11 boxcar

The **boxcar** window belongs to the rectangular family. Its configured parameters are **none**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = 1$$

where $w[n]$ is the window coefficient and n is any sample index from 0 through $N - 1$.

Table 8.12: Measured properties of the boxcar window.

Metric	Value
Coherent gain	1.000000
Equivalent noise bandwidth	1.000000 bins
Scalloping loss	-3.922 dB
Main-lobe width	2.000 bins
Peak sidelobe	-13.264 dB

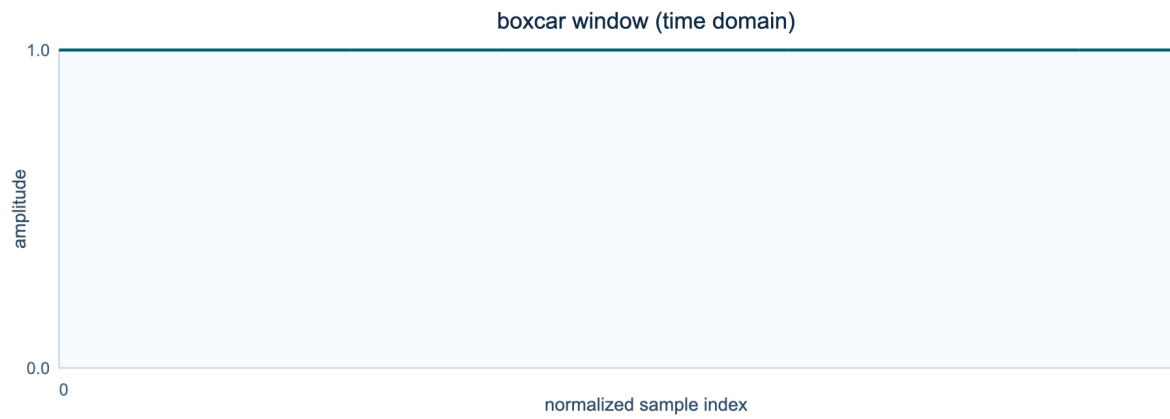


Figure 8.21: Time-domain shape of the boxcar window.

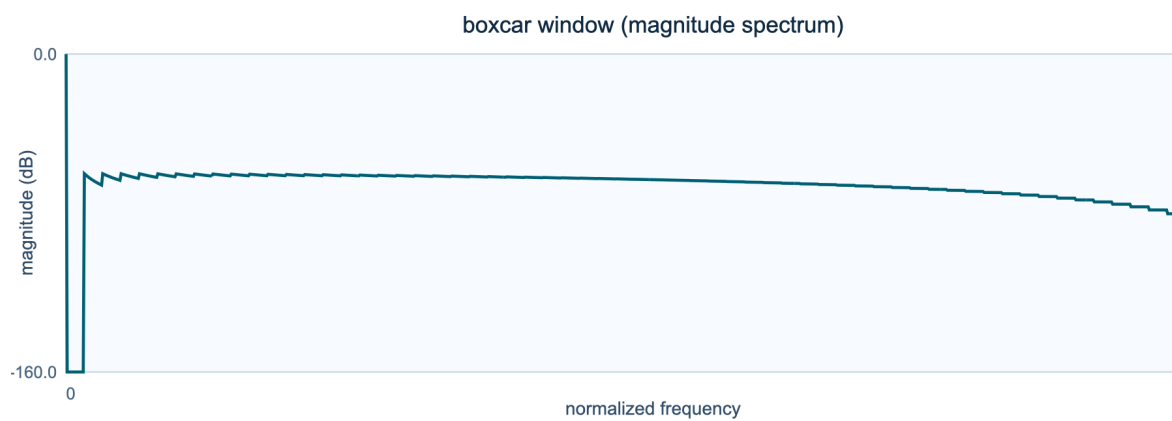


Figure 8.22: Magnitude spectrum of the boxcar window.

8.5.11.1 Interpretation and use

The principal strength of this window is straightforward: Narrowest main lobe and maximal bin sharpness. That advantage should be weighed against its main limitation: Highest sidelobes and strongest leakage/phasing on non-bin-centered content.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use only for controlled test tones or when leakage is acceptable. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.12 triangular

The **triangular** window belongs to the triangular family. Its configured parameters are **none**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \max\left(1 - \left|\frac{n - m}{(N + 1)/2}\right|, 0\right)$$

where $m = (N - 1)/2$ is the center sample, n is the sample index, and N is the window length.

Table 8.13: Measured properties of the triangular window.

Metric	Value
Coherent gain	0.500244
Equivalent noise bandwidth	1.332683 bins
Scalloping loss	-1.826 dB
Main-lobe width	4.000 bins
Peak sidelobe	-26.523 dB

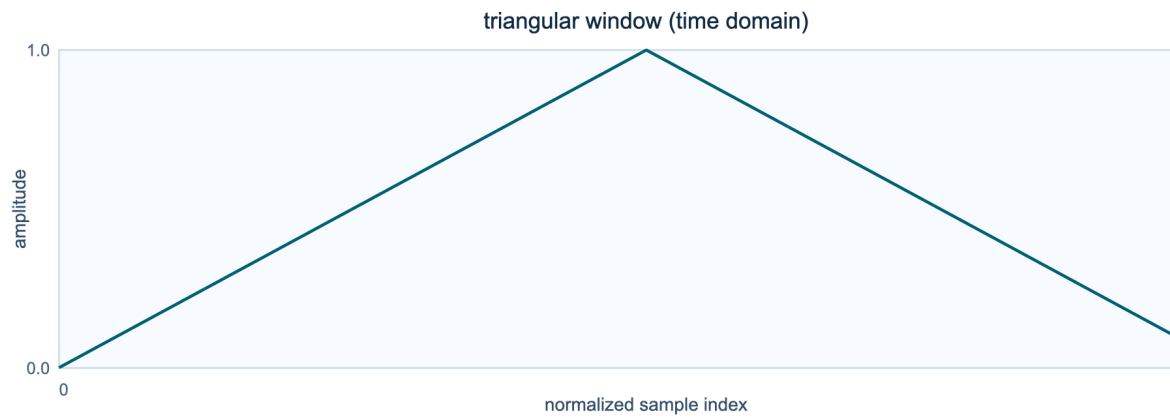


Figure 8.23: Time-domain shape of the triangular window.

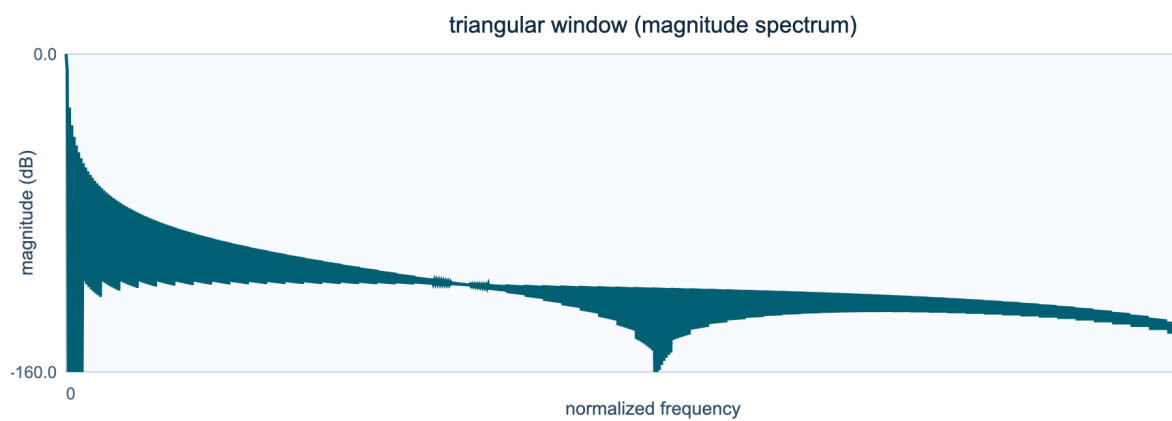


Figure 8.24: Magnitude spectrum of the triangular window.

8.5.12.1 Interpretation and use

The principal strength of this window is straightforward: Cheap linear taper with intuitive behavior. That advantage should be weighed against its main limitation: Higher sidelobes than stronger cosine-sum windows.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for low-cost processing or quick exploratory runs. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.13 bartlett hann

The `bartlett_hann` window belongs to the hybrid taper family. Its configured parameters are **fixed coefficients**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$x = \frac{n}{N-1} - \frac{1}{2}, \quad w[n] = 0.62 - 0.48|x| + 0.38 \cos(2\pi x)$$

where x is the centered normalized position, n is the sample index, and N is the window length.

Table 8.14: Measured properties of the bartlett hann window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.456559 bins
Scalloping loss	-1.517 dB
Main-lobe width	4.000 bins
Peak sidelobe	-35.874 dB

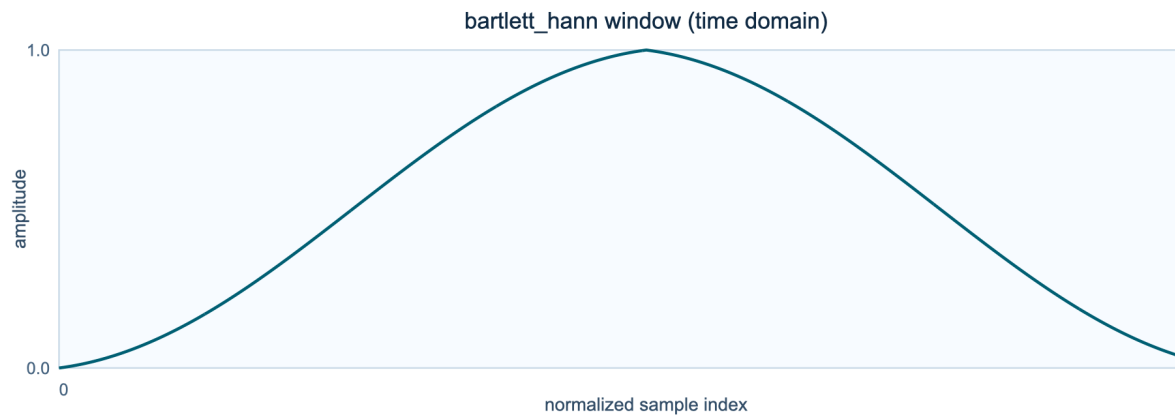


Figure 8.25: Time-domain shape of the bartlett hann window.

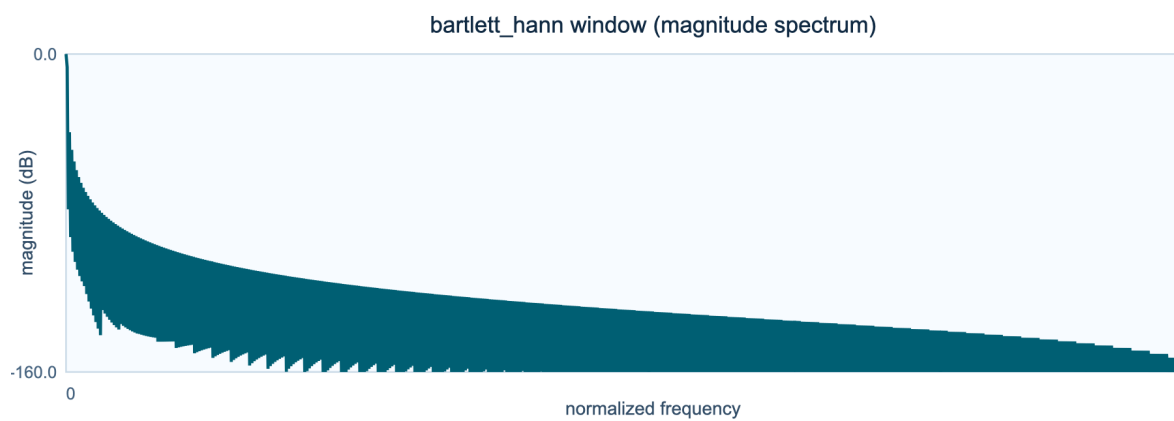


Figure 8.26: Magnitude spectrum of the bartlett hann window.

8.5.13.1 Interpretation and use

The principal strength of this window is straightforward: Moderate leakage suppression with lightweight computation. That advantage should be weighed against its main limitation: Less common and less interpretable than standard Hann/Hamming.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use as a middle-ground taper when Bartlett feels too sharp. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.14 tukey

The **tukey** window belongs to the tukey family. Its configured parameters are **alpha=0.5**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.15: Measured properties of the tukey window.

Metric	Value
Coherent gain	0.749634
Equivalent noise bandwidth	1.222819 bins
Scalloping loss	-2.236 dB
Main-lobe width	2.656 bins
Peak sidelobe	-15.123 dB

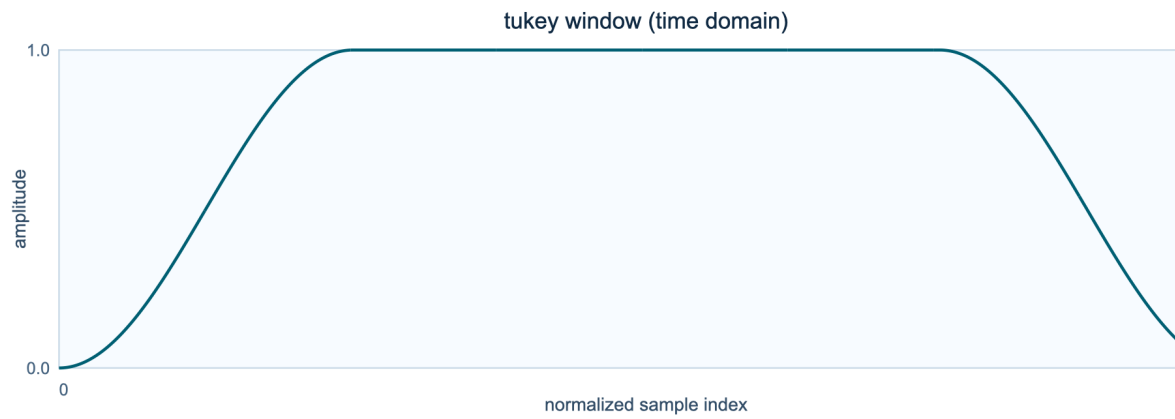


Figure 8.27: Time-domain shape of the tukey window.

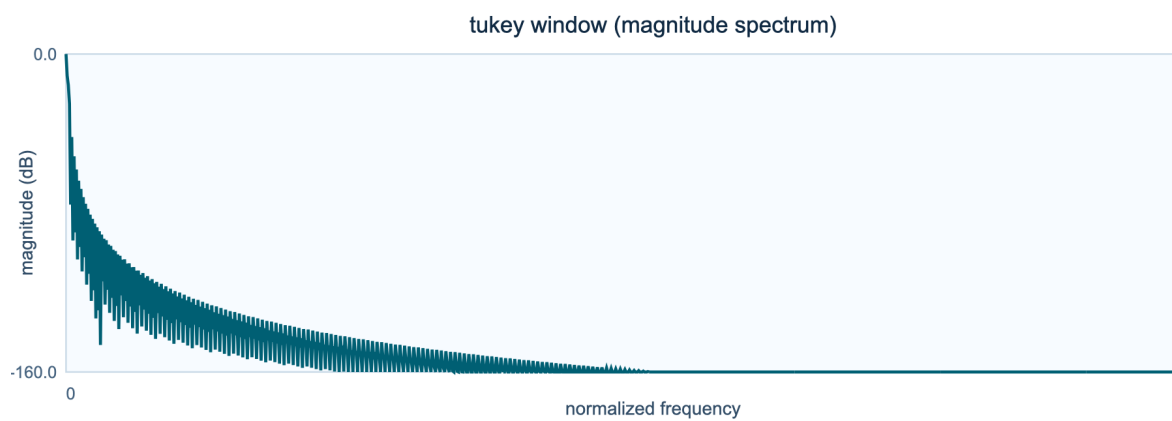


Figure 8.28: Magnitude spectrum of the tukey window.

8.5.14.1 Interpretation and use

The principal strength of this window is straightforward: Interpolates between rectangular and Hann behavior. That advantage should be weighed against its main limitation: Behavior changes strongly with α and can be inconsistent across presets.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you need a controllable flat center with soft edges. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.15 tukey 0p1

The `tukey_0p1` window belongs to the tukey family. Its configured parameters are `alpha=0.1`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.16: Measured properties of the tukey 0p1 window.

Metric	Value
Coherent gain	0.949536
Equivalent noise bandwidth	1.039289 bins
Scalloping loss	-3.505 dB
Main-lobe width	2.094 bins
Peak sidelobe	-13.309 dB

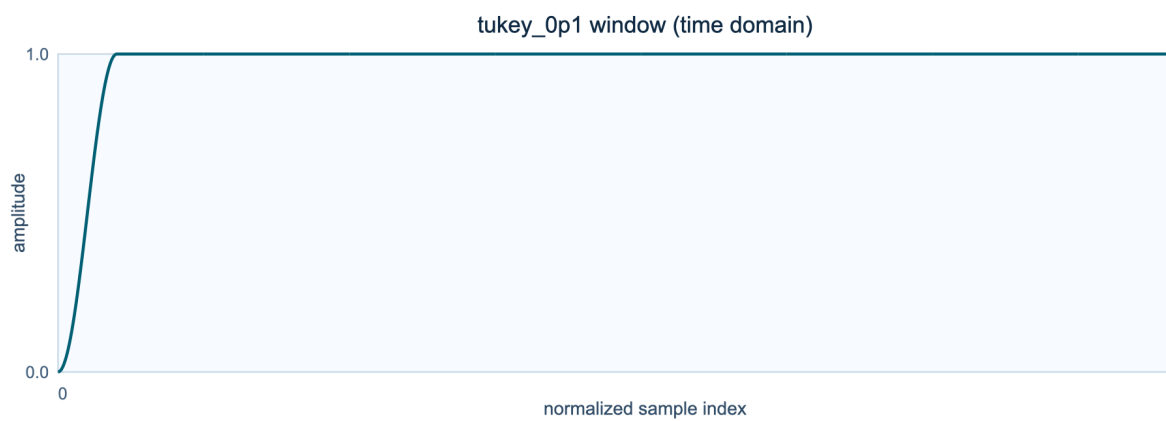


Figure 8.29: Time-domain shape of the tukey 0p1 window.

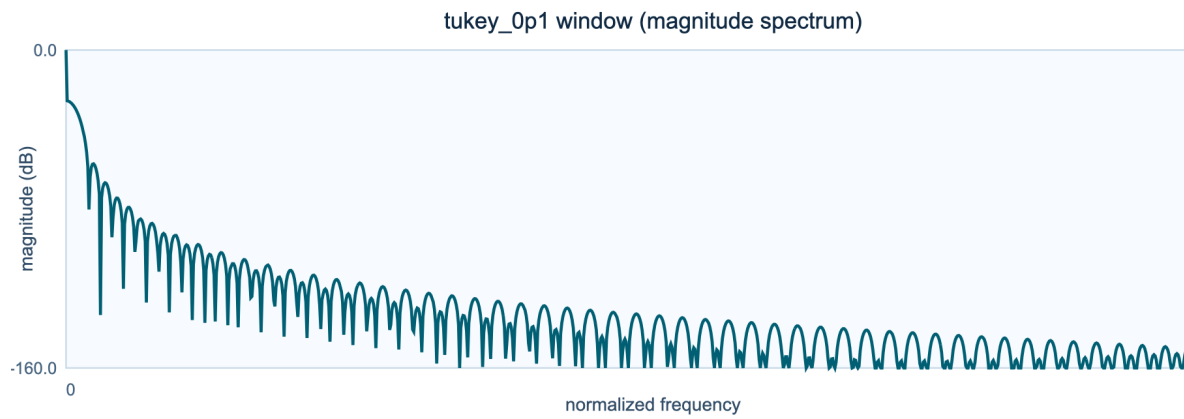


Figure 8.30: Magnitude spectrum of the tukey 0p1 window.

8.5.15.1 Interpretation and use

The principal strength of this window is straightforward: Interpolates between rectangular and Hann behavior. That advantage should be weighed against its main limitation: Behavior changes strongly with α and can be inconsistent across presets.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you need a controllable flat center with soft edges. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.16 tukey 0p25

The `tukey_0p25` window belongs to the tukey family. Its configured parameters are `alpha=0.25`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.17: Measured properties of the tukey 0p25 window.

Metric	Value
Coherent gain	0.874573
Equivalent noise bandwidth	1.102579 bins
Scalloping loss	-2.960 dB
Main-lobe width	2.281 bins
Peak sidelobe	-13.601 dB

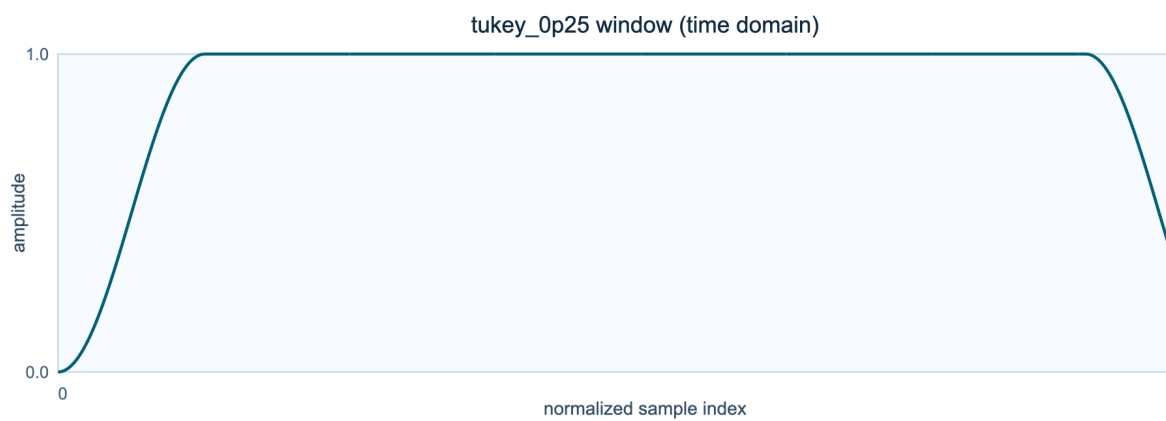


Figure 8.31: Time-domain shape of the tukey 0p25 window.

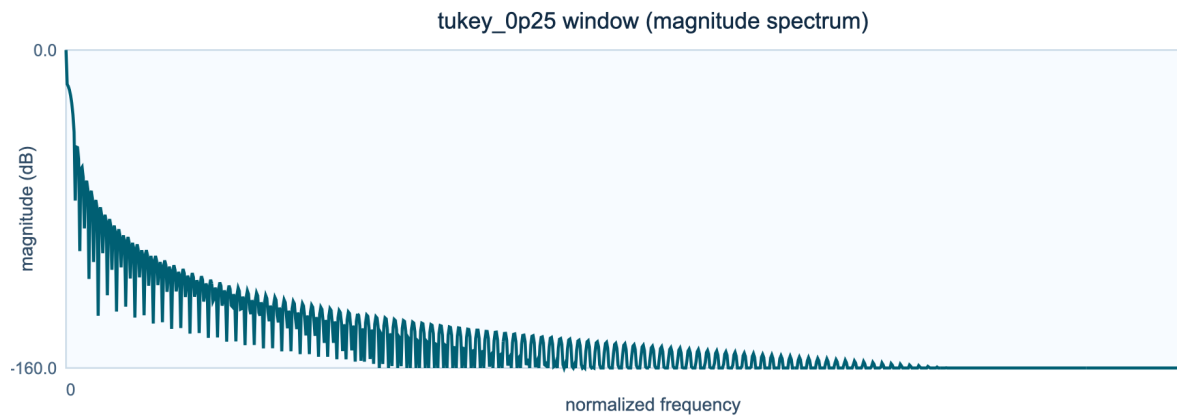


Figure 8.32: Magnitude spectrum of the tukey 0p25 window.

8.5.16.1 Interpretation and use

The principal strength of this window is straightforward: Interpolates between rectangular and Hann behavior. That advantage should be weighed against its main limitation: Behavior changes strongly with α and can be inconsistent across presets.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you need a controllable flat center with soft edges. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.17 tukey 0p75

The `tukey_0p75` window belongs to the tukey family. Its configured parameters are `alpha=0.75`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.18: Measured properties of the tukey 0p75 window.

Metric	Value
Coherent gain	0.624695
Equivalent noise bandwidth	1.360664 bins
Scalloping loss	-1.728 dB
Main-lobe width	3.188 bins
Peak sidelobe	-19.395 dB

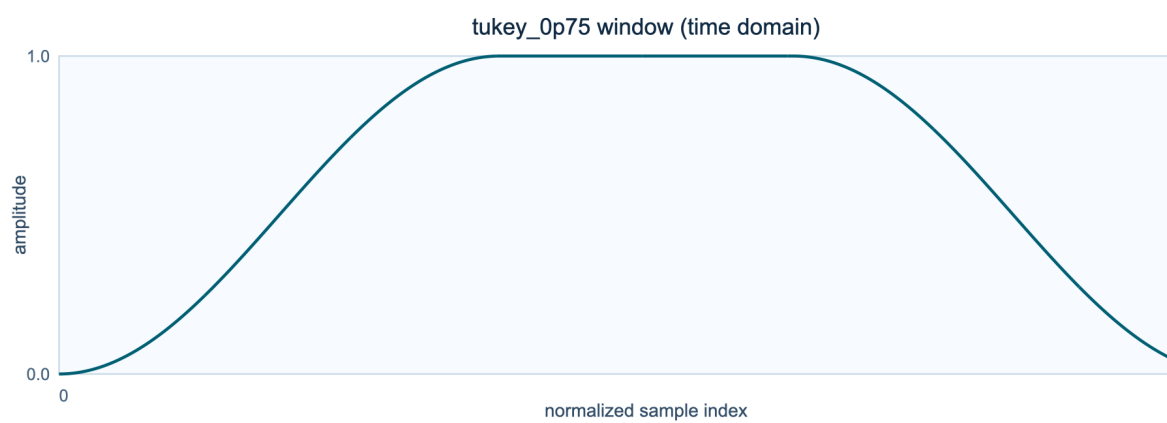


Figure 8.33: Time-domain shape of the tukey 0p75 window.

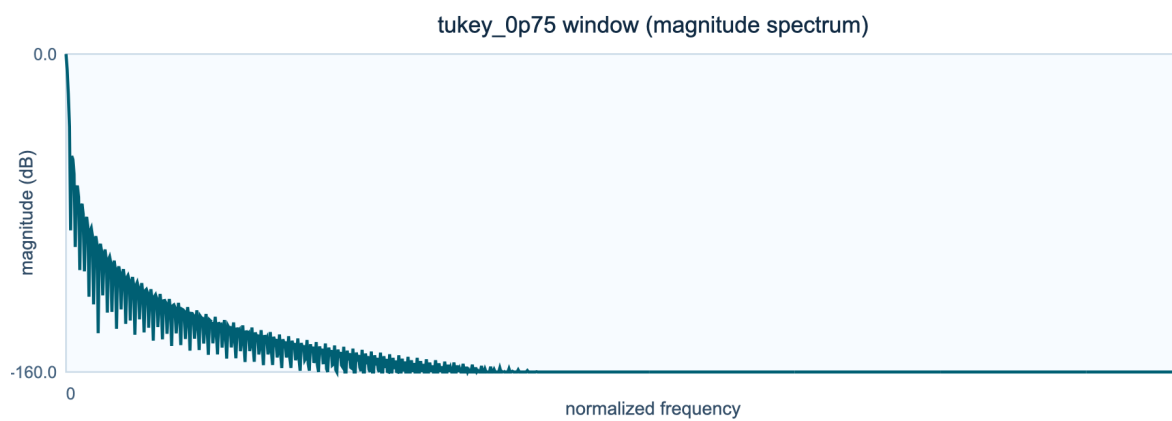


Figure 8.34: Magnitude spectrum of the tukey 0p75 window.

8.5.17.1 Interpretation and use

The principal strength of this window is straightforward: Interpolates between rectangular and Hann behavior. That advantage should be weighed against its main limitation: Behavior changes strongly with α and can be inconsistent across presets.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you need a controllable flat center with soft edges. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.18 tukey 0p9

The `tukey_0p9` window belongs to the tukey family. Its configured parameters are `alpha=0.9`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.19: Measured properties of the tukey 0p9 window.

Metric	Value
Coherent gain	0.549731
Equivalent noise bandwidth	1.446988 bins
Scalloping loss	-1.521 dB
Main-lobe width	3.625 bins
Peak sidelobe	-24.972 dB

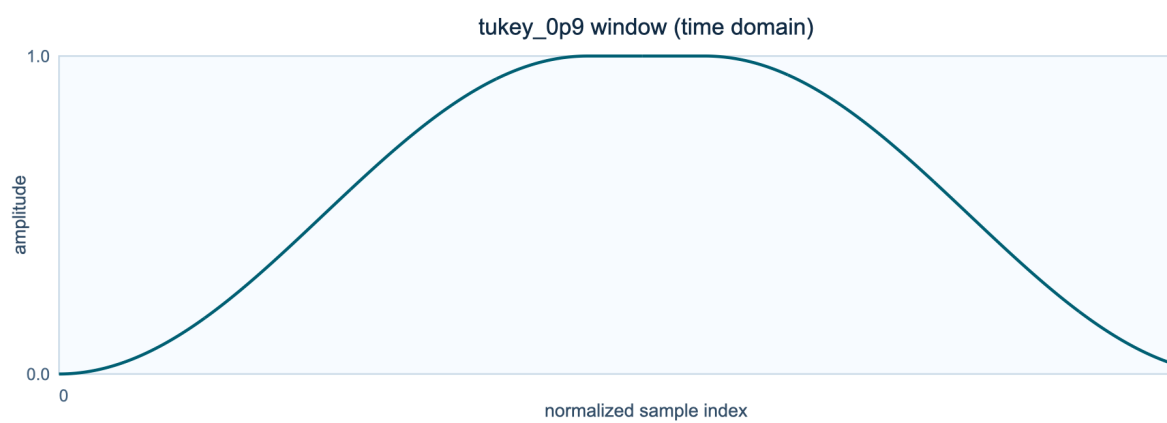


Figure 8.35: Time-domain shape of the tukey 0p9 window.

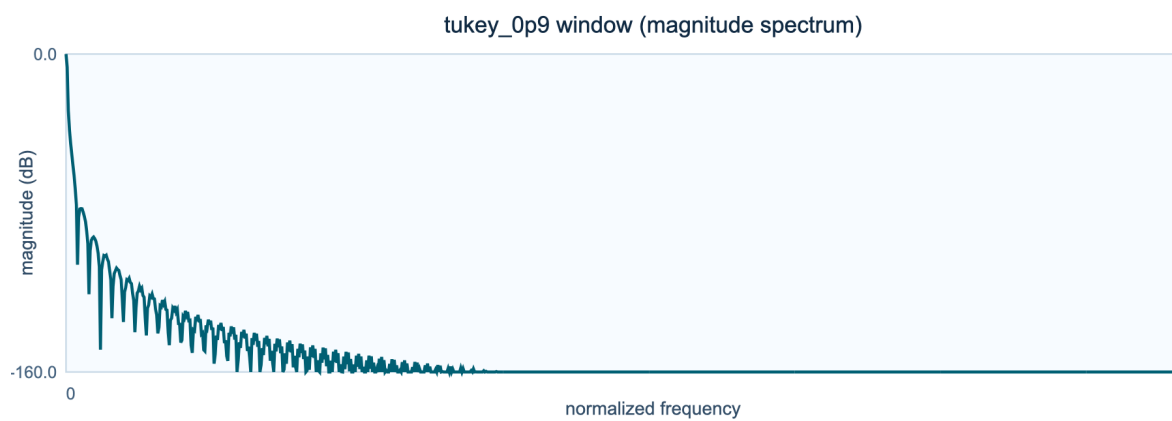


Figure 8.36: Magnitude spectrum of the tukey 0p9 window.

8.5.18.1 Interpretation and use

The principal strength of this window is straightforward: Interpolates between rectangular and Hann behavior. That advantage should be weighed against its main limitation: Behavior changes strongly with α and can be inconsistent across presets.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you need a controllable flat center with soft edges. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.19 parzen

The **parzen** window belongs to the polynomial family. Its configured parameters are **piecewise cubic**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \begin{cases} 1 - 6u^2 + 6u^3, & 0 \leq u \leq 1/2 \\ 2(1 - u)^3, & 1/2 < u \leq 1 \\ 0, & u > 1 \end{cases}$$

where $u = |2n/(N - 1) - 1|$ is absolute centered position, n is the sample index, and N is the window length.

Table 8.20: Measured properties of the parzen window.

Metric	Value
Coherent gain	0.374817
Equivalent noise bandwidth	1.918397 bins
Scalloping loss	-0.897 dB
Main-lobe width	8.000 bins
Peak sidelobe	-53.046 dB

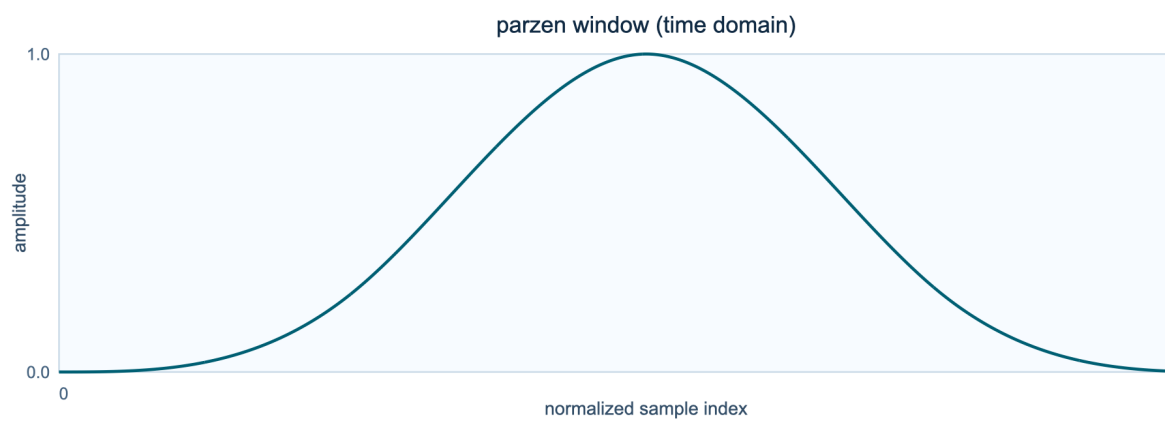


Figure 8.37: Time-domain shape of the parzen window.

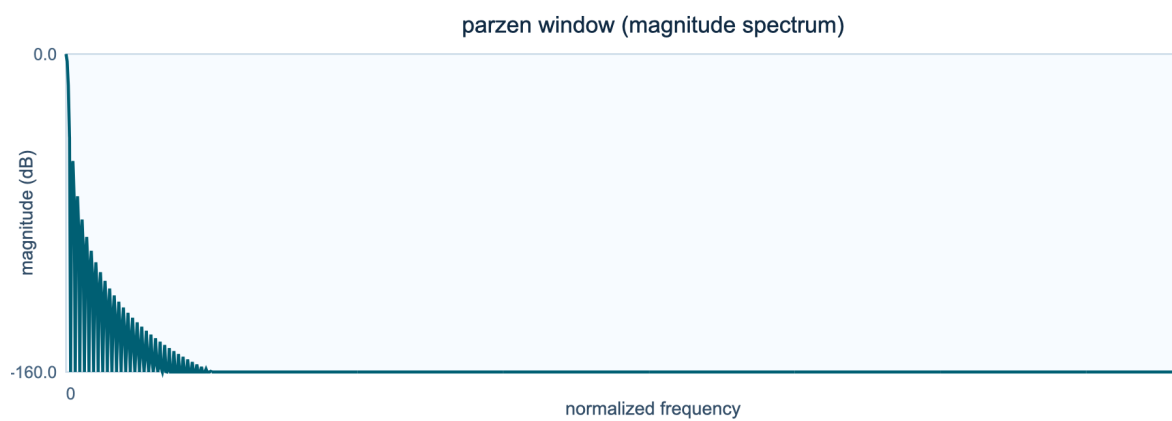


Figure 8.38: Magnitude spectrum of the parzen window.

8.5.19.1 Interpretation and use

The principal strength of this window is straightforward: Smooth high-order taper with good sidelobe control. That advantage should be weighed against its main limitation: Broader main lobe than lightweight windows.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for spectral denoising/analysis where sidelobe cleanup is critical. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.20 lanczos

The **lanczos** window belongs to the sinc family. Its configured parameters are **none**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \text{sinc}\left(\frac{2n}{N-1} - 1\right)$$

where $\text{sinc}(x) = \sin(\pi x)/(\pi x)$, n is the sample index, and N is the window length.

Table 8.21: Measured properties of the lanczos window.

Metric	Value
Coherent gain	0.589202
Equivalent noise bandwidth	1.299668 bins
Scalloping loss	-1.889 dB
Main-lobe width	3.281 bins
Peak sidelobe	-26.405 dB

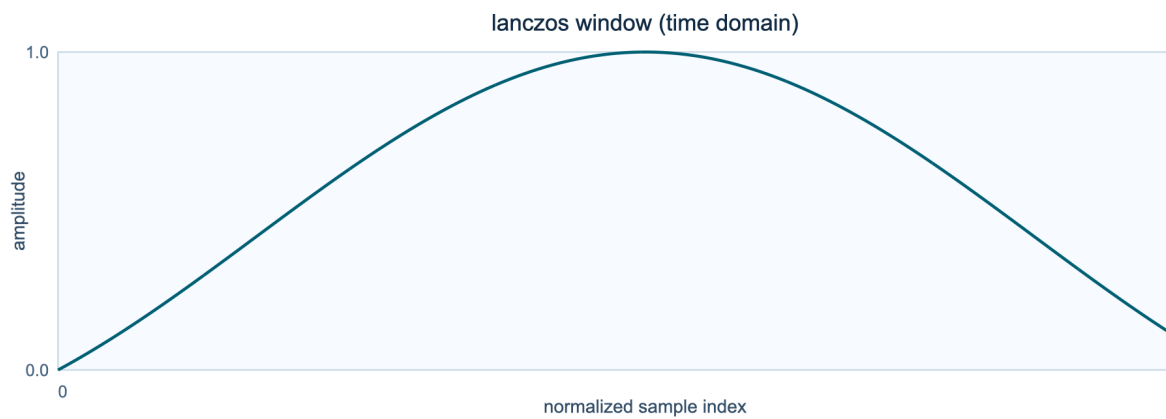


Figure 8.39: Time-domain shape of the lanczos window.

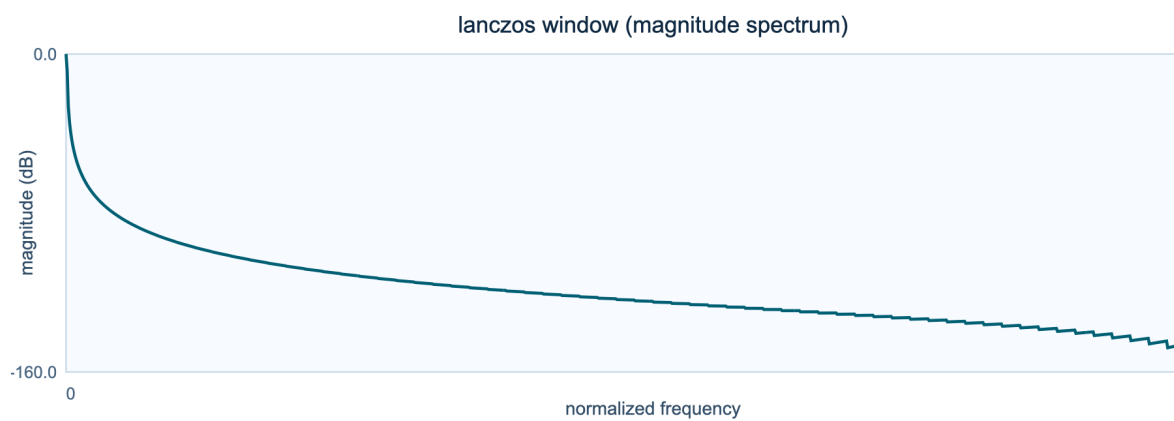


Figure 8.40: Magnitude spectrum of the lanczos window.

8.5.20.1 Interpretation and use

The principal strength of this window is straightforward: Sinc-derived shape with useful compromise behavior. That advantage should be weighed against its main limitation: Can exhibit oscillatory spectral behavior versus cosine-sum defaults.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for interpolation-adjacent analysis experiments. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.21 welch

The `welch` window belongs to the quadratic family. Its configured parameters are `none`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \max(1 - x_n^2, 0)$$

where $x_n = (n - m)/m$ is centered normalized position, $m = (N - 1)/2$, and N is the window length.

Table 8.22: Measured properties of the welch window.

Metric	Value
Coherent gain	0.666341
Equivalent noise bandwidth	1.200587 bins
Scalloping loss	-2.223 dB
Main-lobe width	2.875 bins
Peak sidelobe	-21.295 dB

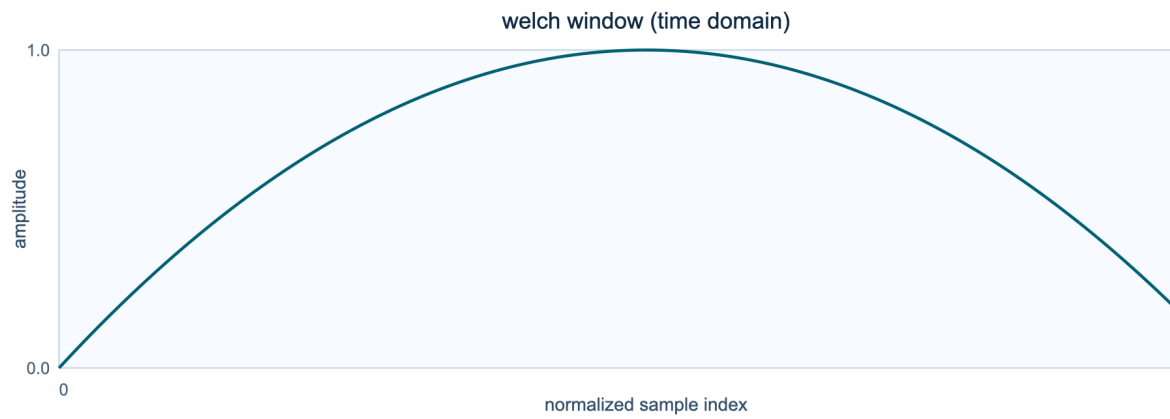


Figure 8.41: Time-domain shape of the welch window.

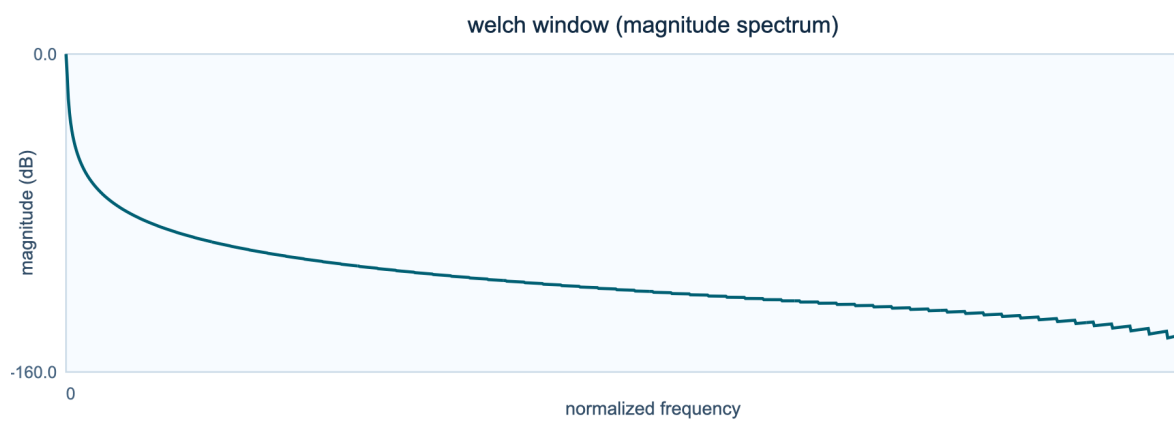


Figure 8.42: Magnitude spectrum of the welch window.

8.5.21.1 Interpretation and use

The principal strength of this window is straightforward: Center-emphasizing parabola with simple form. That advantage should be weighed against its main limitation: Not as strong at sidelobe suppression as Blackman-family windows.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when center weighting is desired with minimal complexity. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.22 gaussian 0p25

The `gaussian_0p25` window belongs to the gaussian family. Its configured parameters are `sigma_ratio=0.25`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp \left[-\frac{1}{2} \left(\frac{n-m}{\sigma} \right)^2 \right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.23: Measured properties of the gaussian 0p25 window.

Metric	Value
Coherent gain	0.313156
Equivalent noise bandwidth	2.258144 bins
Scalloping loss	-0.668 dB
Main-lobe width	11.406 bins
Peak sidelobe	-87.677 dB

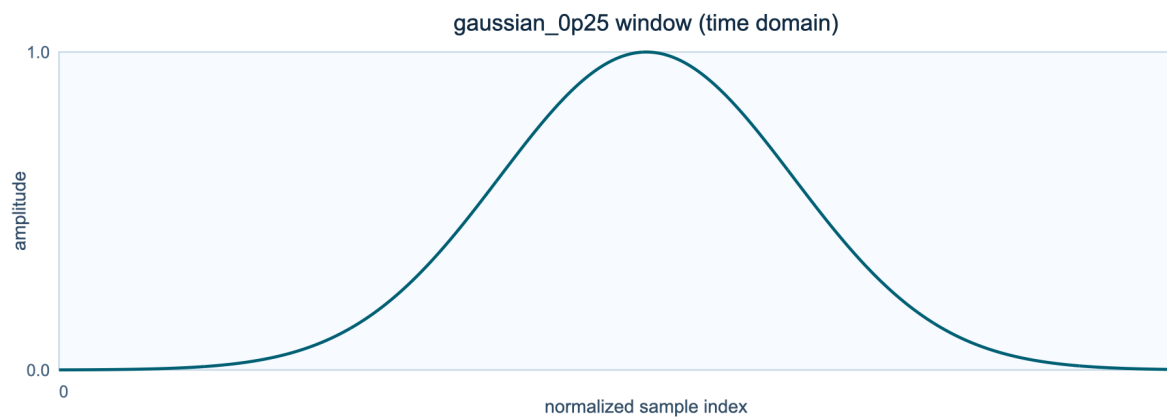


Figure 8.43: Time-domain shape of the gaussian 0p25 window.

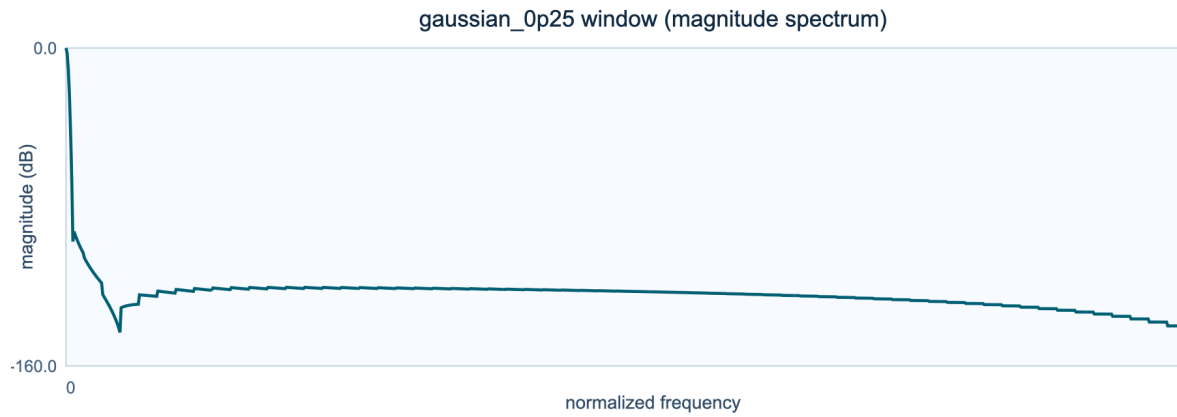


Figure 8.44: Magnitude spectrum of the gaussian 0p25 window.

8.5.22.1 Interpretation and use

The principal strength of this window is straightforward: Smooth bell taper with low ringing and predictable decay. That advantage should be weighed against its main limitation: Frequency resolution changes noticeably with sigma.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use sigma presets to tune transient locality versus spectral leakage. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.23 gaussian 0p35

The `gaussian_0p35` window belongs to the gaussian family. Its configured parameters are `sigma_ratio=0.35`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp \left[-\frac{1}{2} \left(\frac{n-m}{\sigma} \right)^2 \right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.24: Measured properties of the gaussian 0p35 window.

Metric	Value
Coherent gain	0.436580
Equivalent noise bandwidth	1.626488 bins
Scalloping loss	-1.268 dB
Main-lobe width	6.719 bins
Peak sidelobe	-52.077 dB

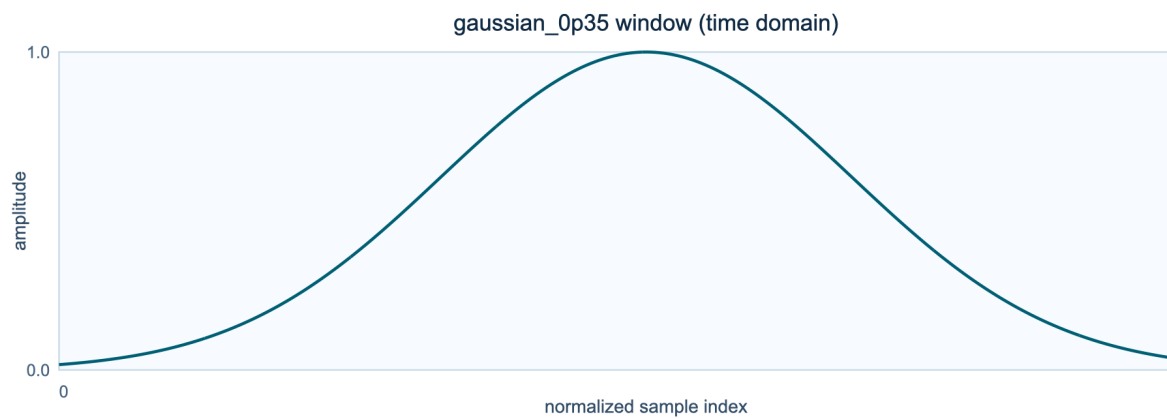


Figure 8.45: Time-domain shape of the gaussian 0p35 window.

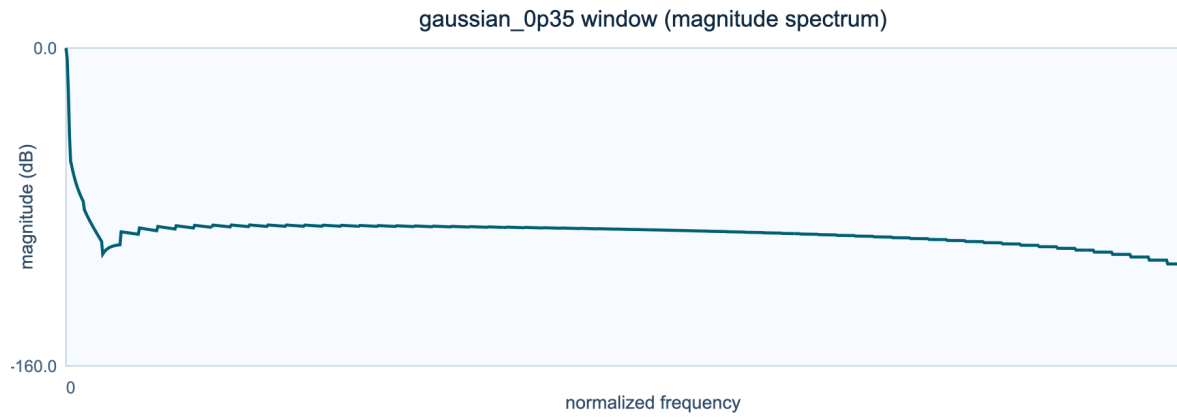


Figure 8.46: Magnitude spectrum of the gaussian 0p35 window.

8.5.23.1 Interpretation and use

The principal strength of this window is straightforward: Smooth bell taper with low ringing and predictable decay. That advantage should be weighed against its main limitation: Frequency resolution changes noticeably with sigma.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use sigma presets to tune transient locality versus spectral leakage. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.24 gaussian 0p45

The `gaussian_0p45` window belongs to the gaussian family. Its configured parameters are `sigma_ratio=0.45`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp \left[-\frac{1}{2} \left(\frac{n-m}{\sigma} \right)^2 \right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.25: Measured properties of the gaussian 0p45 window.

Metric	Value
Coherent gain	0.548949
Equivalent noise bandwidth	1.320556 bins
Scalloping loss	-1.869 dB
Main-lobe width	4.125 bins
Peak sidelobe	-35.366 dB

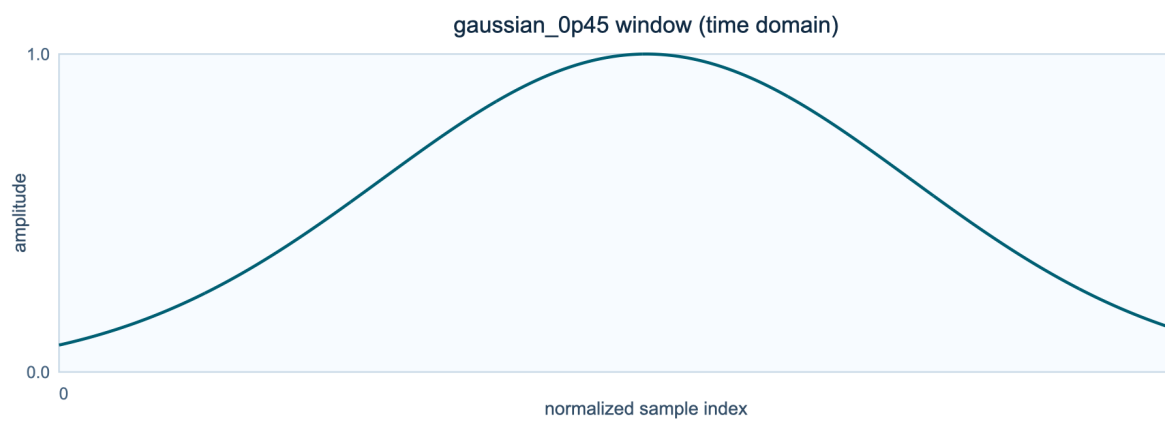


Figure 8.47: Time-domain shape of the gaussian 0p45 window.

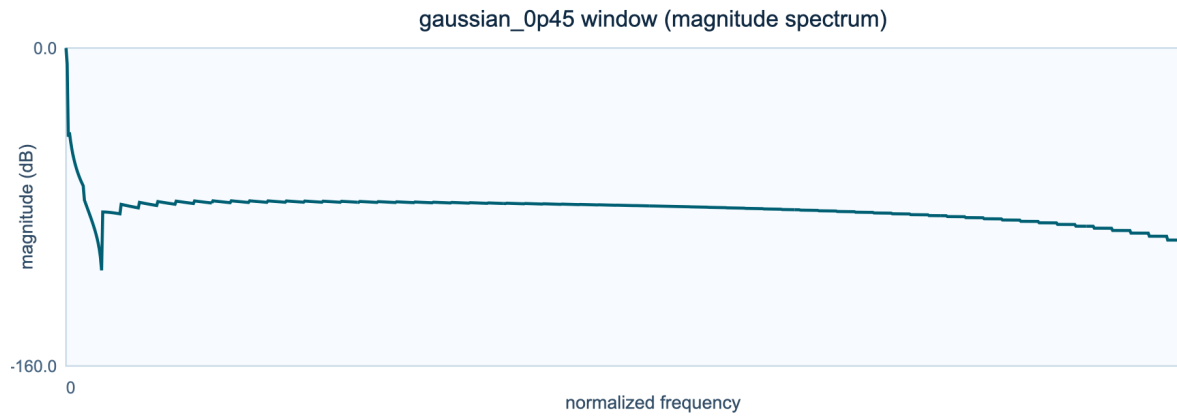


Figure 8.48: Magnitude spectrum of the gaussian 0p45 window.

8.5.24.1 Interpretation and use

The principal strength of this window is straightforward: Smooth bell taper with low ringing and predictable decay. That advantage should be weighed against its main limitation: Frequency resolution changes noticeably with sigma.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use sigma presets to tune transient locality versus spectral leakage. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.25 gaussian 0p55

The `gaussian_0p55` window belongs to the gaussian family. Its configured parameters are `sigma_ratio=0.55`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp \left[-\frac{1}{2} \left(\frac{n-m}{\sigma} \right)^2 \right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.26: Measured properties of the gaussian 0p55 window.

Metric	Value
Coherent gain	0.641515
Equivalent noise bandwidth	1.171861 bins
Scalloping loss	-2.350 dB
Main-lobe width	3.000 bins
Peak sidelobe	-28.906 dB

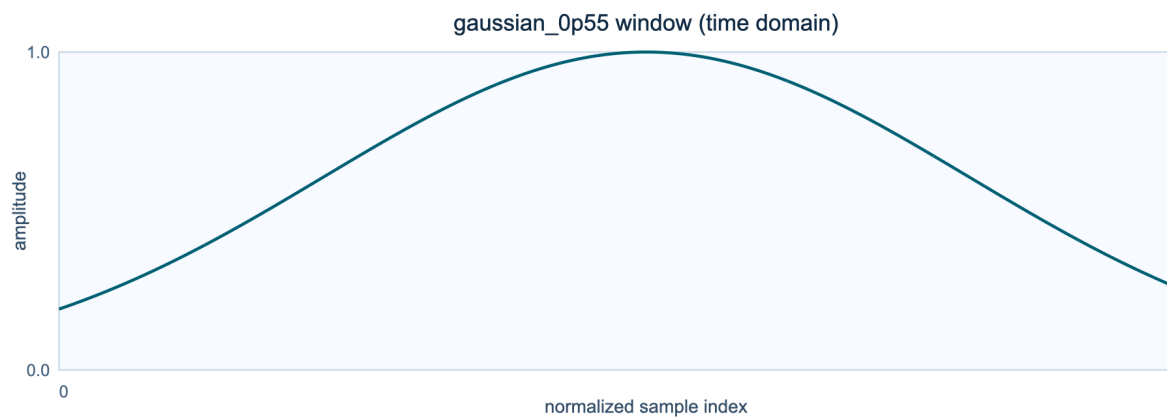


Figure 8.49: Time-domain shape of the gaussian 0p55 window.

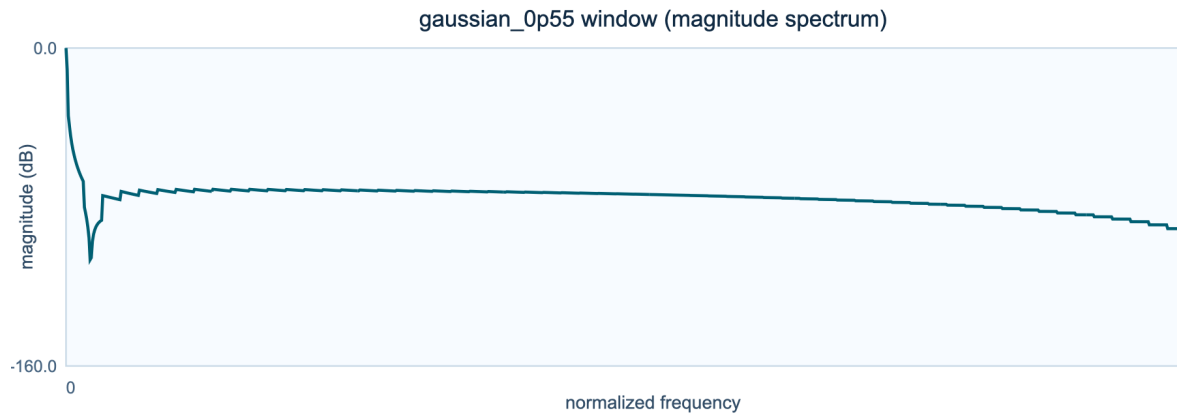


Figure 8.50: Magnitude spectrum of the gaussian 0p55 window.

8.5.25.1 Interpretation and use

The principal strength of this window is straightforward: Smooth bell taper with low ringing and predictable decay. That advantage should be weighed against its main limitation: Frequency resolution changes noticeably with sigma.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use sigma presets to tune transient locality versus spectral leakage. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.26 gaussian 0p65

The `gaussian_0p65` window belongs to the gaussian family. Its configured parameters are `sigma_ratio=0.65`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp \left[-\frac{1}{2} \left(\frac{n-m}{\sigma} \right)^2 \right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.27: Measured properties of the gaussian 0p65 window.

Metric	Value
Coherent gain	0.713490
Equivalent noise bandwidth	1.097657 bins
Scalloping loss	-2.705 dB
Main-lobe width	2.625 bins
Peak sidelobe	-22.575 dB

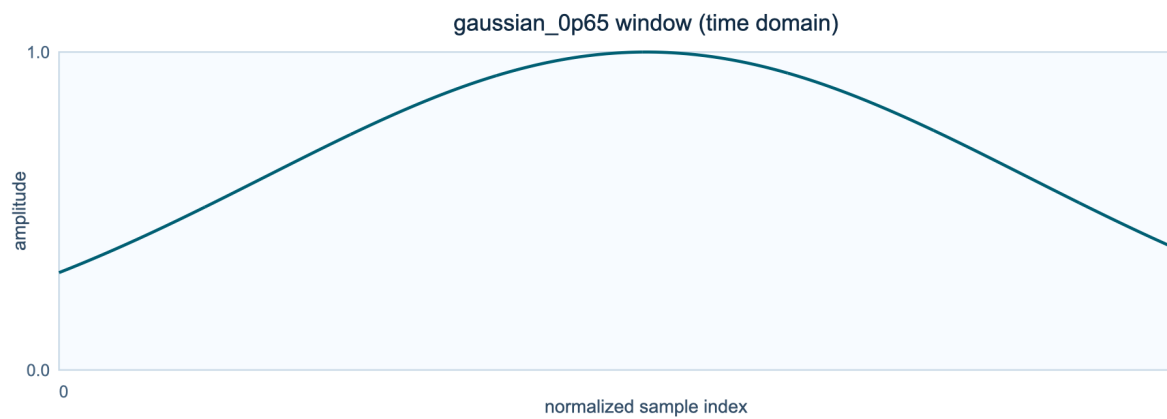


Figure 8.51: Time-domain shape of the gaussian 0p65 window.

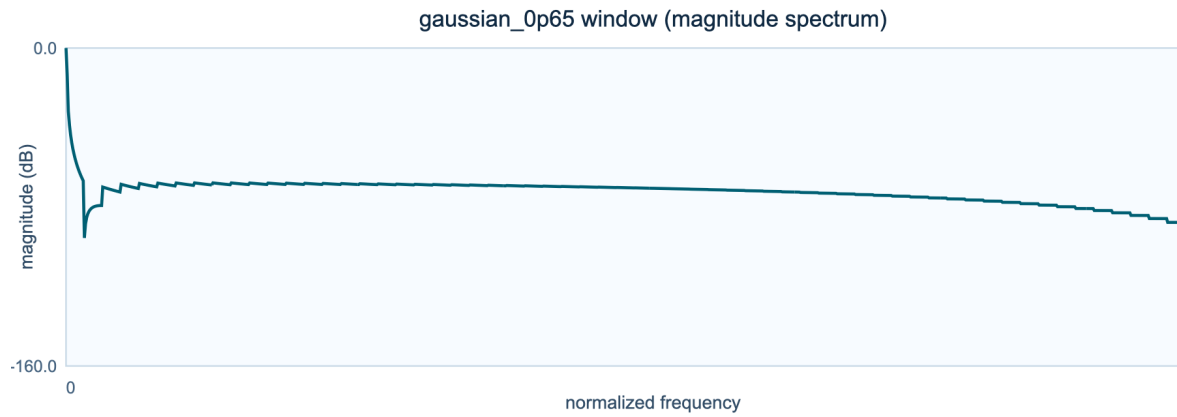


Figure 8.52: Magnitude spectrum of the gaussian 0p65 window.

8.5.26.1 Interpretation and use

The principal strength of this window is straightforward: Smooth bell taper with low ringing and predictable decay. That advantage should be weighed against its main limitation: Frequency resolution changes noticeably with sigma.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use sigma presets to tune transient locality versus spectral leakage. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.27 general gaussian 1p5 0p35

The `general_gaussian_1p5_0p35` window belongs to the generalized gaussian family. Its configured parameters are `power=1.5`, `sigma_ratio=0.35`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp \left[-\frac{1}{2} \left| \frac{n-m}{\sigma} \right|^{2p} \right]$$

where p controls shoulder shape, σ controls spread, $m = (N-1)/2$, and n is the sample index.

Table 8.28: Measured properties of the general gaussian 1p5 0p35 window.

Metric	Value
Coherent gain	0.393587
Equivalent noise bandwidth	2.016584 bins
Scalloping loss	-0.784 dB
Main-lobe width	5.156 bins
Peak sidelobe	-24.895 dB

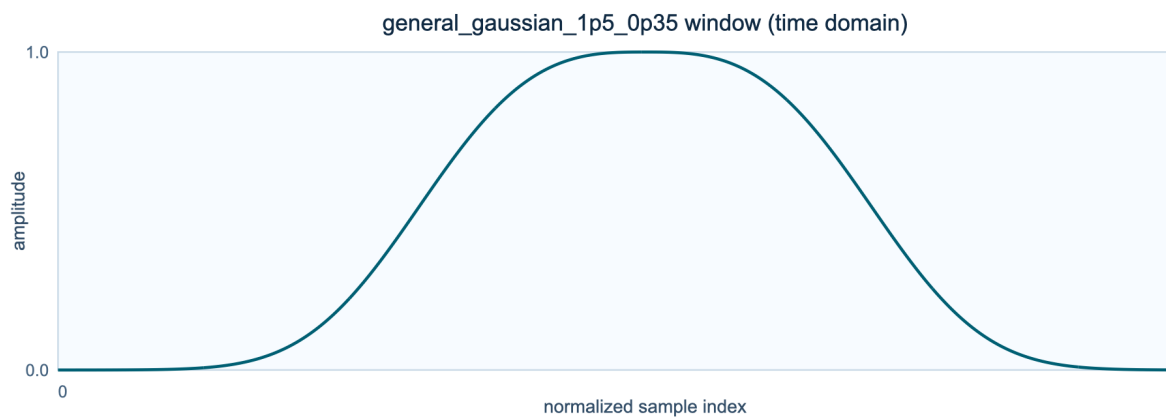


Figure 8.53: Time-domain shape of the general gaussian 1p5 0p35 window.

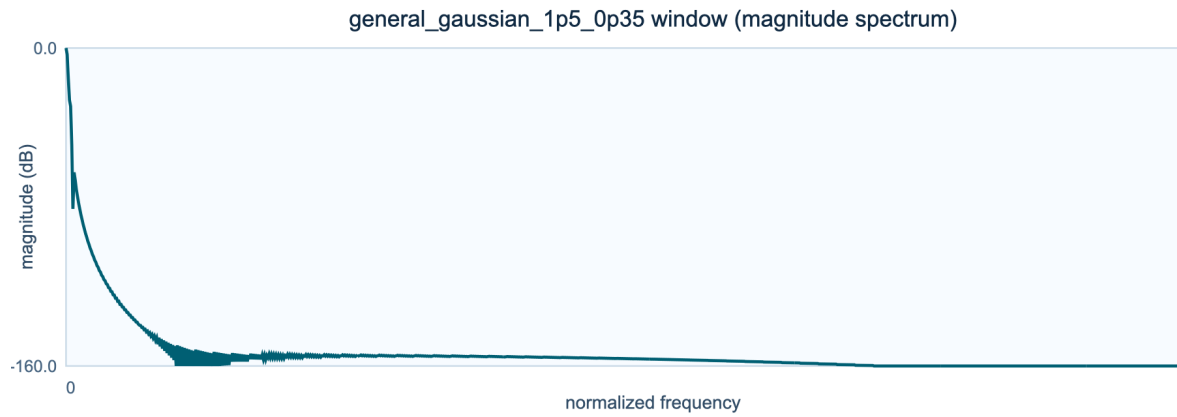


Figure 8.54: Magnitude spectrum of the general gaussian 1p5 0p35 window.

8.5.27.1 Interpretation and use

The principal strength of this window is straightforward: Additional shape control beyond standard Gaussian. That advantage should be weighed against its main limitation: Extra parameterization increases tuning complexity.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for research/tuning tasks where shoulder steepness matters. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.28 general gaussian 2p0 0p35

The `general_gaussian_2p0_0p35` window belongs to the generalized gaussian family. Its configured parameters are `power=2`, `sigma_ratio=0.35`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp \left[-\frac{1}{2} \left| \frac{n-m}{\sigma} \right|^{2p} \right]$$

where p controls shoulder shape, σ controls spread, $m = (N-1)/2$, and n is the sample index.

Table 8.29: Measured properties of the general gaussian 2p0 0p35 window.

Metric	Value
Coherent gain	0.377081
Equivalent noise bandwidth	2.230016 bins
Scalloping loss	-0.633 dB
Main-lobe width	5.281 bins
Peak sidelobe	-19.717 dB

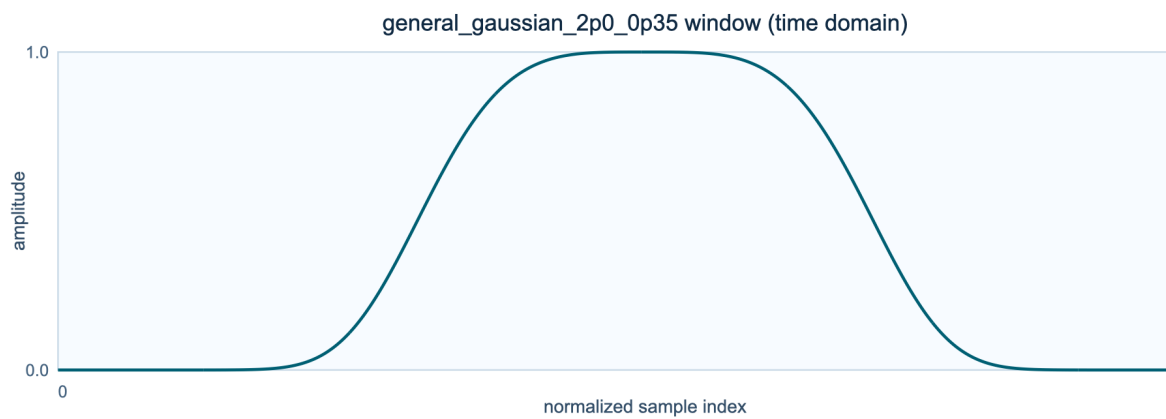


Figure 8.55: Time-domain shape of the general gaussian 2p0 0p35 window.

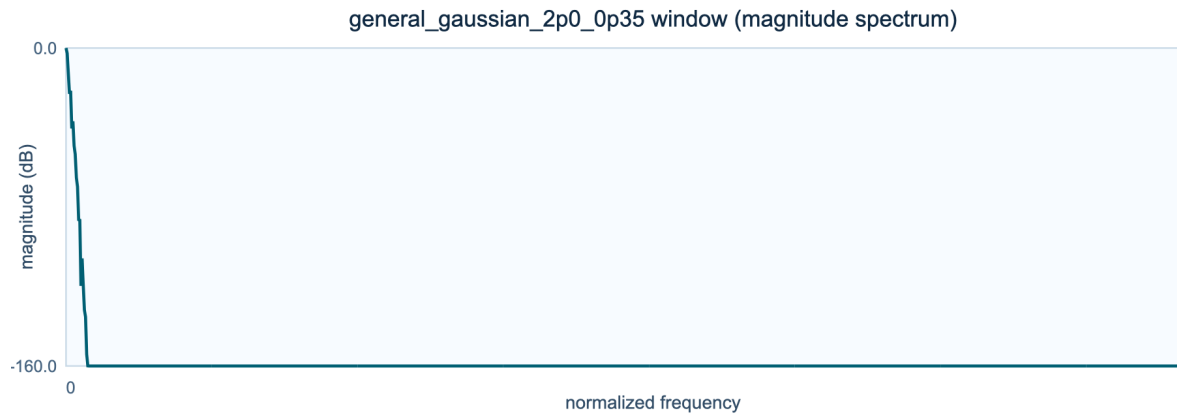


Figure 8.56: Magnitude spectrum of the general gaussian 2p0 0p35 window.

8.5.28.1 Interpretation and use

The principal strength of this window is straightforward: Additional shape control beyond standard Gaussian. That advantage should be weighed against its main limitation: Extra parameterization increases tuning complexity.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for research/tuning tasks where shoulder steepness matters. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.29 general gaussian 3p0 0p35

The `general_gaussian_3p0_0p35` window belongs to the generalized gaussian family. Its configured parameters are `power=3`, `sigma_ratio=0.35`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp \left[-\frac{1}{2} \left| \frac{n-m}{\sigma} \right|^{2p} \right]$$

where p controls shoulder shape, σ controls spread, $m = (N-1)/2$, and n is the sample index.

Table 8.30: Measured properties of the general gaussian 3p0 0p35 window.

Metric	Value
Coherent gain	0.364287
Equivalent noise bandwidth	2.445593 bins
Scalloping loss	-0.532 dB
Main-lobe width	5.469 bins
Peak sidelobe	-16.299 dB

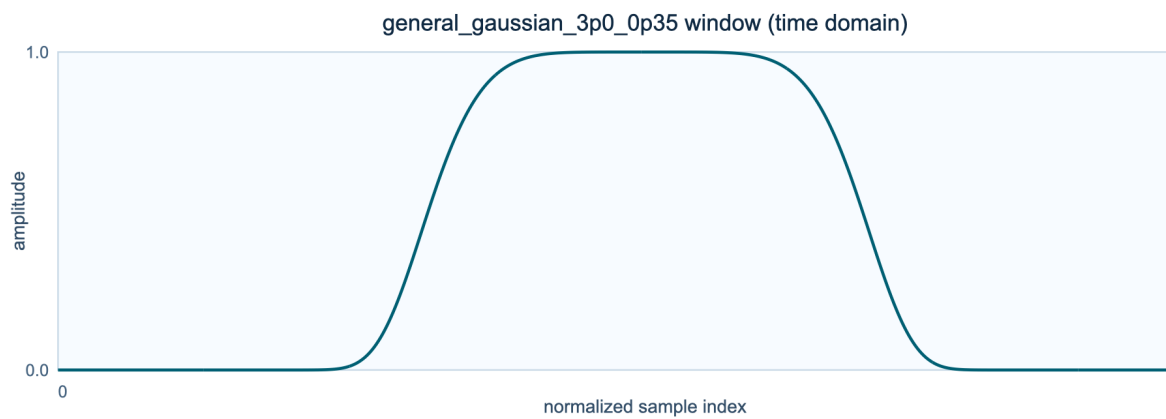


Figure 8.57: Time-domain shape of the general gaussian 3p0 0p35 window.

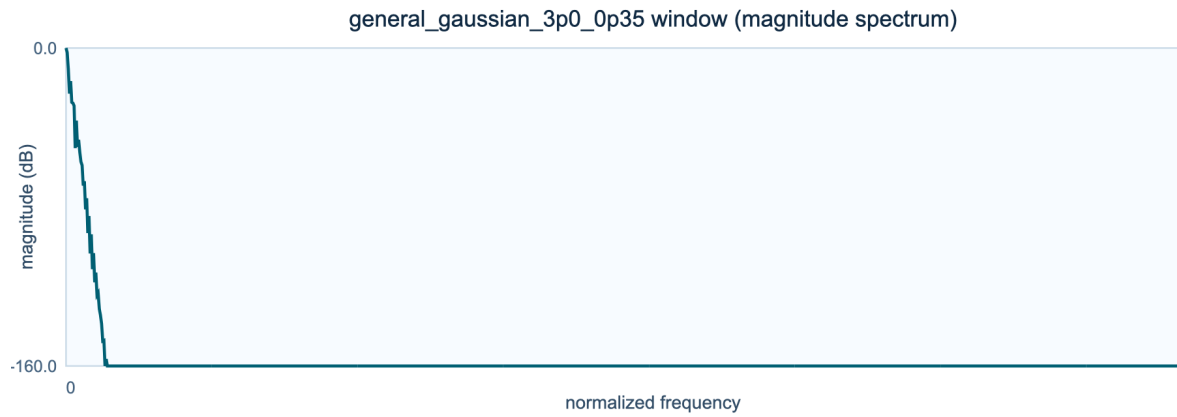


Figure 8.58: Magnitude spectrum of the general gaussian 3p0 0p35 window.

8.5.29.1 Interpretation and use

The principal strength of this window is straightforward: Additional shape control beyond standard Gaussian. That advantage should be weighed against its main limitation: Extra parameterization increases tuning complexity.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for research/tuning tasks where shoulder steepness matters. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.30 general gaussian 4p0 0p35

The `general_gaussian_4p0_0p35` window belongs to the generalized gaussian family. Its configured parameters are `power=4`, `sigma_ratio=0.35`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp \left[-\frac{1}{2} \left| \frac{n-m}{\sigma} \right|^{2p} \right]$$

where p controls shoulder shape, σ controls spread, $m = (N-1)/2$, and n is the sample index.

Table 8.31: Measured properties of the general gaussian 4p0 0p35 window.

Metric	Value
Coherent gain	0.359267
Equivalent noise bandwidth	2.552433 bins
Scalloping loss	-0.496 dB
Main-lobe width	5.562 bins
Peak sidelobe	-15.060 dB

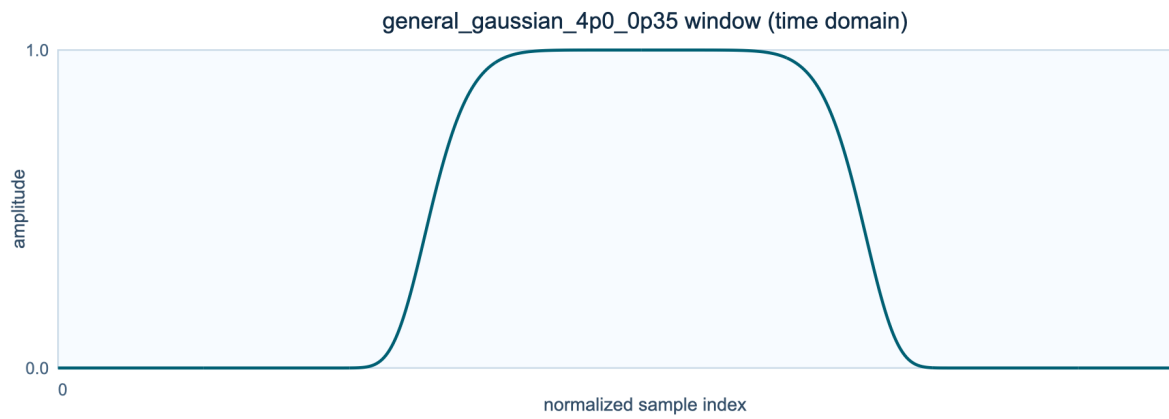


Figure 8.59: Time-domain shape of the general gaussian 4p0 0p35 window.

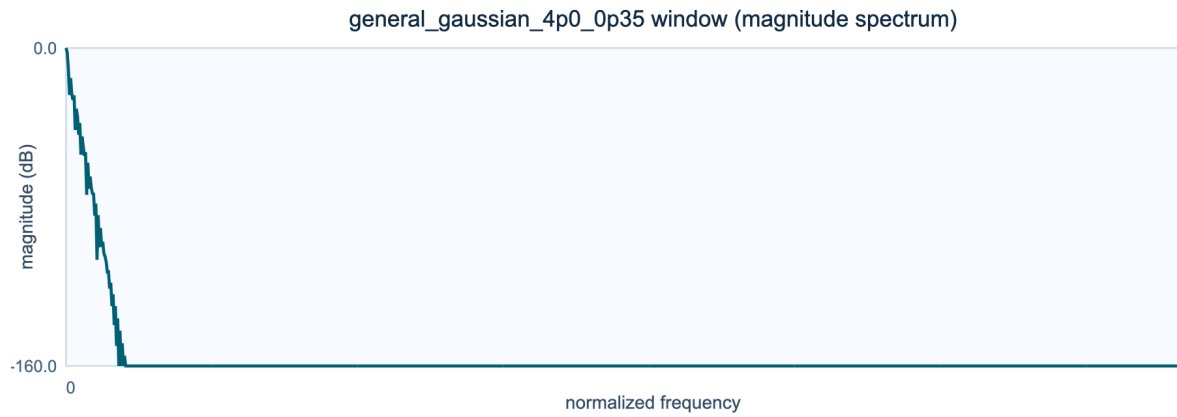


Figure 8.60: Magnitude spectrum of the general gaussian 4p0 0p35 window.

8.5.30.1 Interpretation and use

The principal strength of this window is straightforward: Additional shape control beyond standard Gaussian. That advantage should be weighed against its main limitation: Extra parameterization increases tuning complexity.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for research/tuning tasks where shoulder steepness matters. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.31 exponential 0p25

The `exponential_0p25` window belongs to the exponential family. Its configured parameters are `tau_ratio=0.25`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp\left(-\frac{|n-m|}{\tau}\right), \quad \tau = r_\tau m$$

where τ is the decay constant, r_τ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.32: Measured properties of the exponential 0p25 window.

Metric	Value
Coherent gain	0.245310
Equivalent noise bandwidth	2.075492 bins
Scalloping loss	-1.022 dB
Main-lobe width	15.562 bins
Peak sidelobe	-31.885 dB

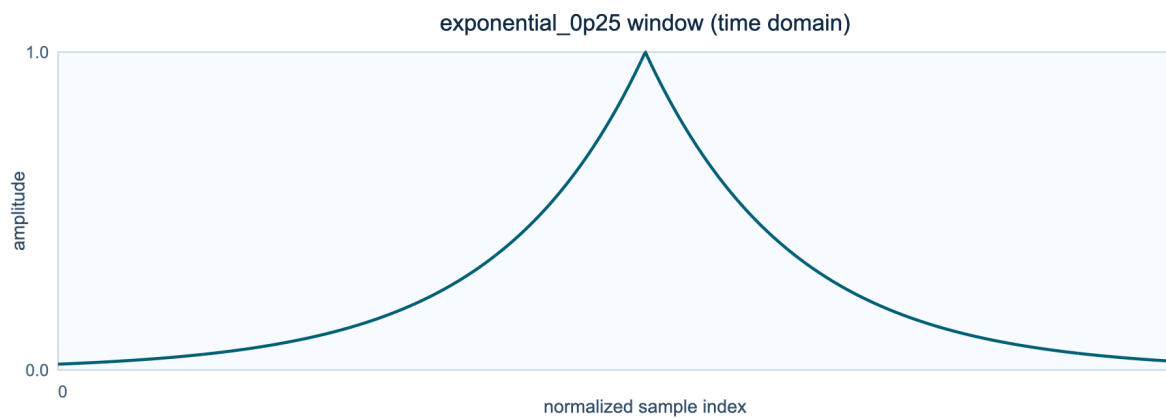


Figure 8.61: Time-domain shape of the exponential 0p25 window.

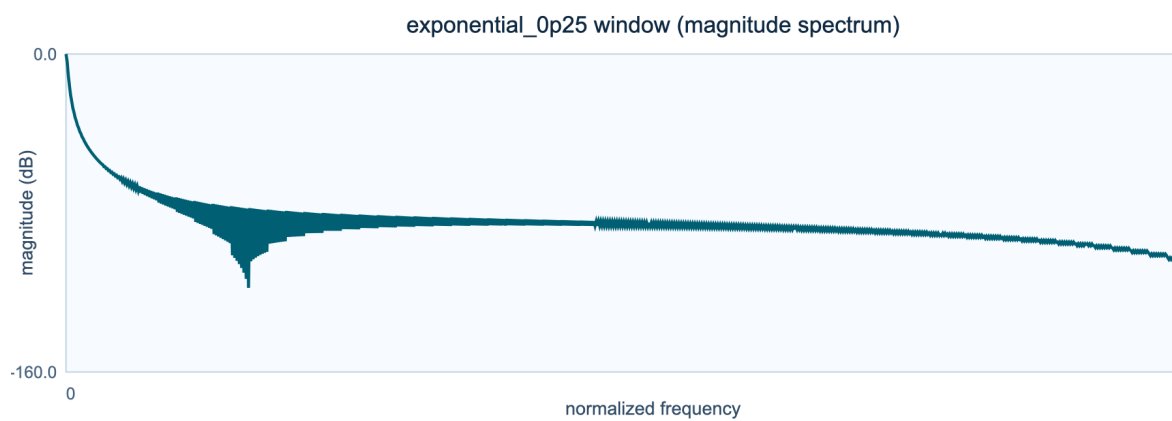


Figure 8.62: Magnitude spectrum of the exponential 0p25 window.

8.5.31.1 Interpretation and use

The principal strength of this window is straightforward: Simple edge decay with fast computation. That advantage should be weighed against its main limitation: Can produce less-uniform center weighting than cosine families.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for experiments emphasizing center-heavy weighting. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.32 exponential 0p5

The `exponential_0p5` window belongs to the exponential family. Its configured parameters are `tau_ratio=0.5`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp\left(-\frac{|n-m|}{\tau}\right), \quad \tau = r_\tau m$$

where τ is the decay constant, r_τ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.33: Measured properties of the exponential 0p5 window.

Metric	Value
Coherent gain	0.432187
Equivalent noise bandwidth	1.313323 bins
Scalloping loss	-2.032 dB
Main-lobe width	3.625 bins
Peak sidelobe	-19.190 dB

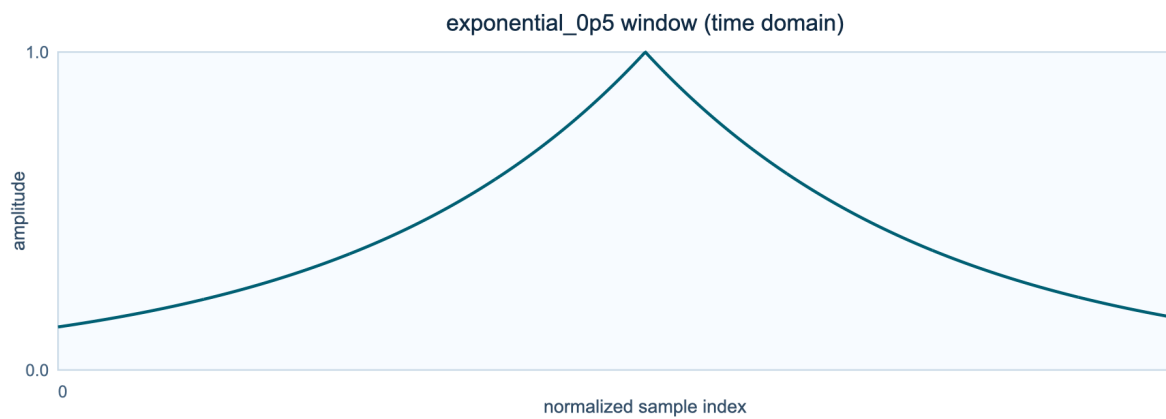


Figure 8.63: Time-domain shape of the exponential 0p5 window.

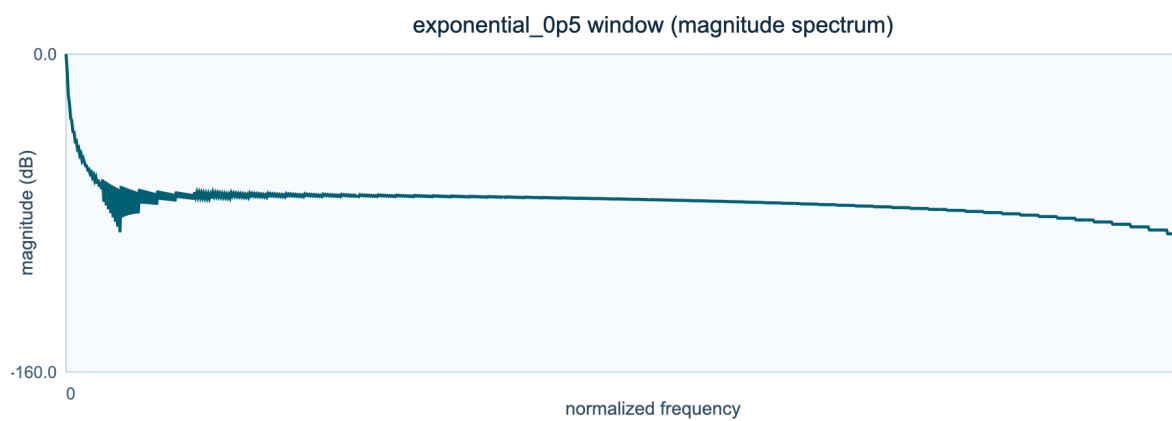


Figure 8.64: Magnitude spectrum of the exponential 0p5 window.

8.5.32.1 Interpretation and use

The principal strength of this window is straightforward: Simple edge decay with fast computation. That advantage should be weighed against its main limitation: Can produce less-uniform center weighting than cosine families.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for experiments emphasizing center-heavy weighting. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.33 exponential 1p0

The `exponential_1p0` window belongs to the exponential family. Its configured parameters are `tau_ratio=1`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \exp\left(-\frac{|n-m|}{\tau}\right), \quad \tau = r_\tau m$$

where τ is the decay constant, r_τ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.34: Measured properties of the exponential 1p0 window.

Metric	Value
Coherent gain	0.631991
Equivalent noise bandwidth	1.082055 bins
Scalloping loss	-2.854 dB
Main-lobe width	2.594 bins
Peak sidelobe	-20.141 dB

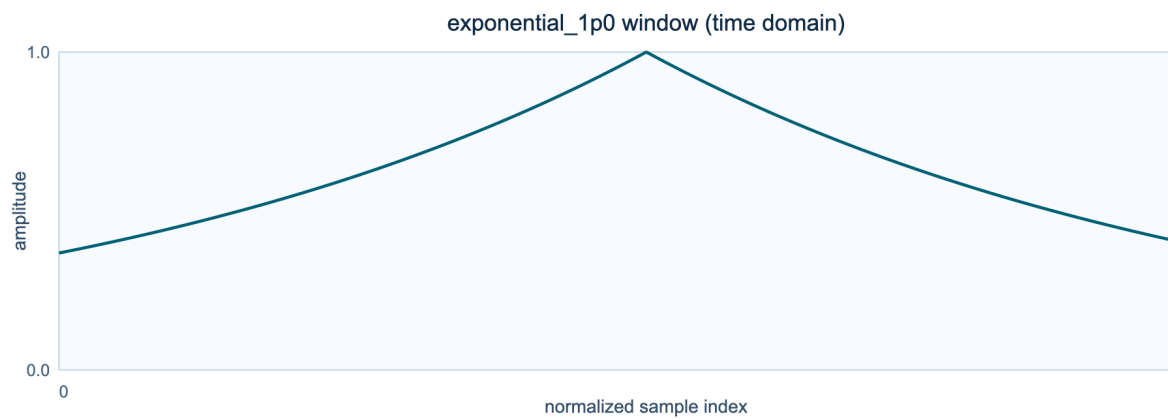


Figure 8.65: Time-domain shape of the exponential 1p0 window.

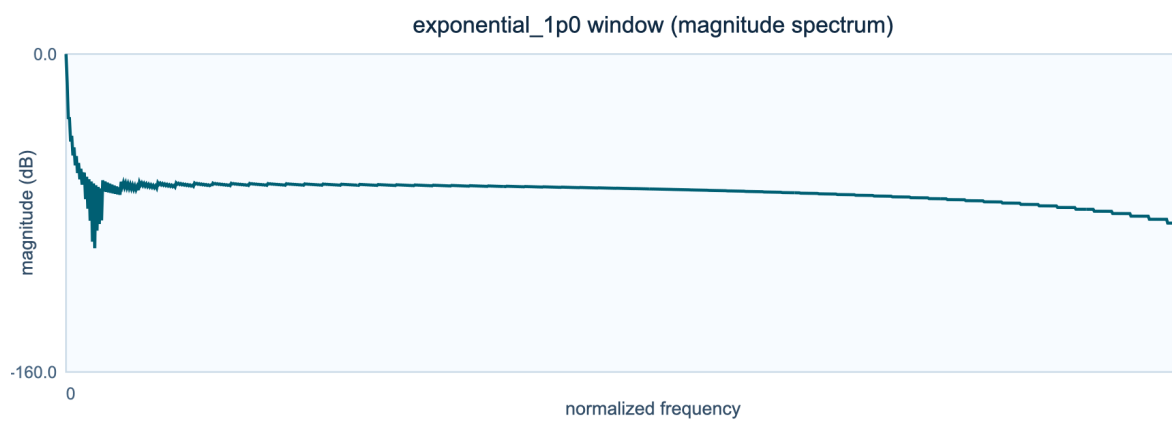


Figure 8.66: Magnitude spectrum of the exponential 1p0 window.

8.5.33.1 Interpretation and use

The principal strength of this window is straightforward: Simple edge decay with fast computation. That advantage should be weighed against its main limitation: Can produce less-uniform center weighting than cosine families.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for experiments emphasizing center-heavy weighting. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.34 cauchy 0p5

The `cauchy_0p5` window belongs to the `cauchy / lorentzian` family. Its configured parameters are `gamma_ratio=0.5`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \frac{1}{1 + \left(\frac{n-m}{\gamma}\right)^2}, \quad \gamma = r_\gamma m$$

where γ is the Lorentzian scale, r_γ is its configured ratio, $m = (N - 1)/2$, and n is the sample index.

Table 8.35: Measured properties of the `cauchy 0p5` window.

Metric	Value
Coherent gain	0.553402
Equivalent noise bandwidth	1.229775 bins
Scalloping loss	-2.209 dB
Main-lobe width	3.562 bins
Peak sidelobe	-22.733 dB

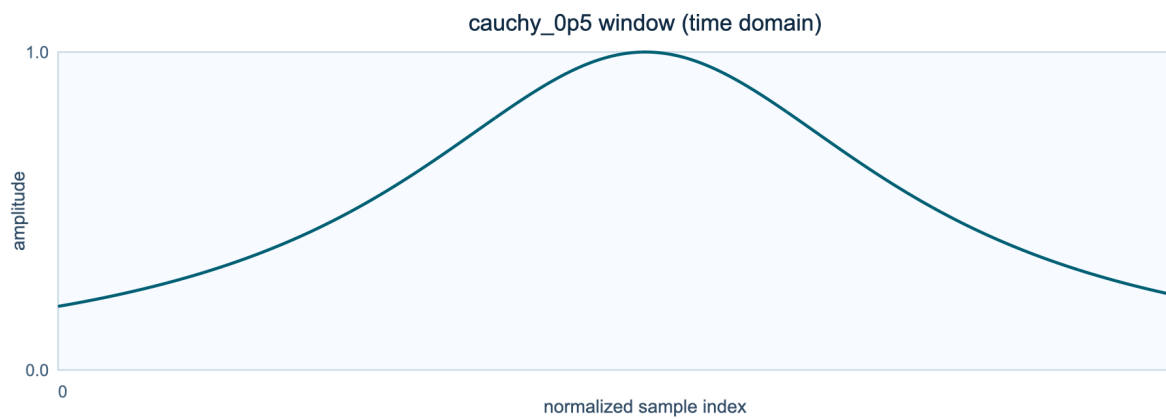


Figure 8.67: Time-domain shape of the cauchy 0p5 window.

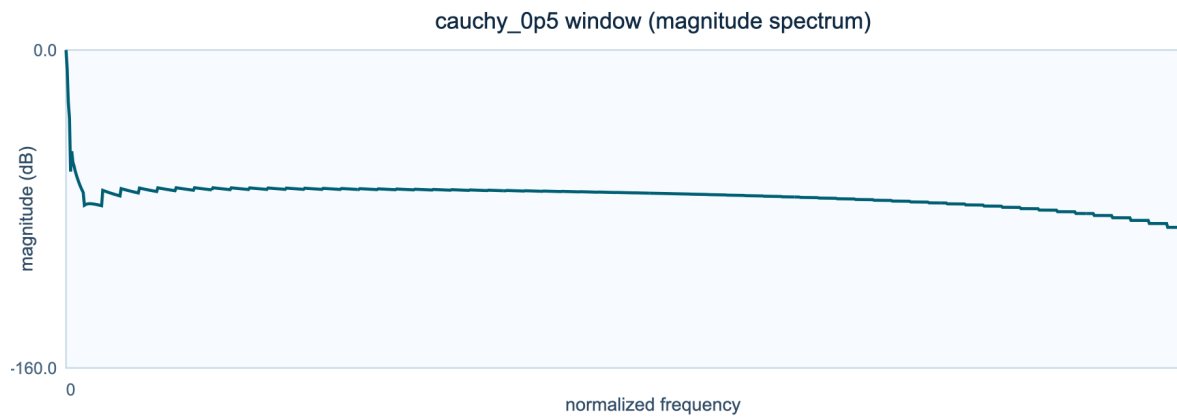


Figure 8.68: Magnitude spectrum of the cauchy 0p5 window.

8.5.34.1 Interpretation and use

The principal strength of this window is straightforward: Heavy tails preserve more peripheral samples than Gaussian. That advantage should be weighed against its main limitation: Tail energy can increase leakage relative to steeper tapers.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you want softer attenuation of far-window samples. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.35 cauchy 1p0

The `cauchy_1p0` window belongs to the `cauchy / lorentzian` family. Its configured parameters are `gamma_ratio=1`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \frac{1}{1 + \left(\frac{n-m}{\gamma}\right)^2}, \quad \gamma = r_\gamma m$$

where γ is the Lorentzian scale, r_γ is its configured ratio, $m = (N - 1)/2$, and n is the sample index.

Table 8.36: Measured properties of the `cauchy 1p0` window.

Metric	Value
Coherent gain	0.785259
Equivalent noise bandwidth	1.041963 bins
Scalloping loss	-3.108 dB
Main-lobe width	2.375 bins
Peak sidelobe	-19.034 dB

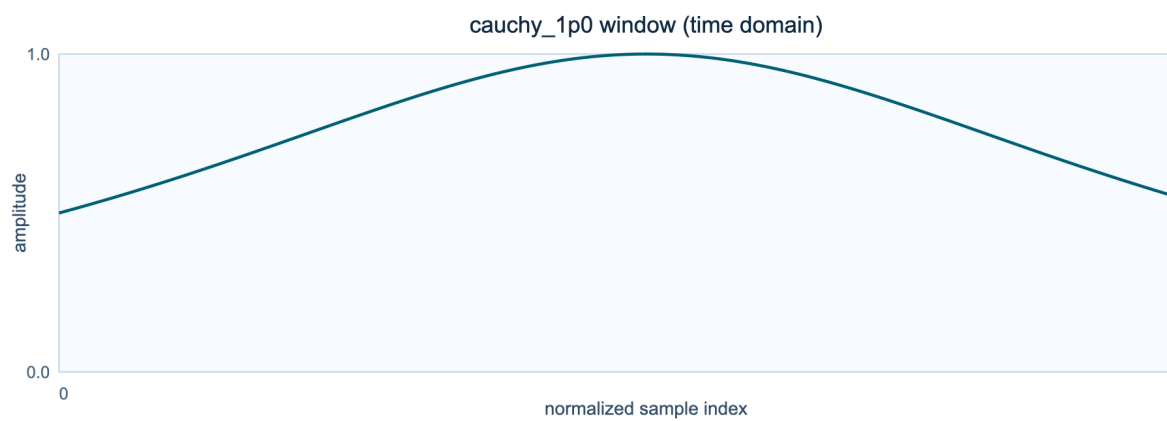


Figure 8.69: Time-domain shape of the cauchy 1p0 window.

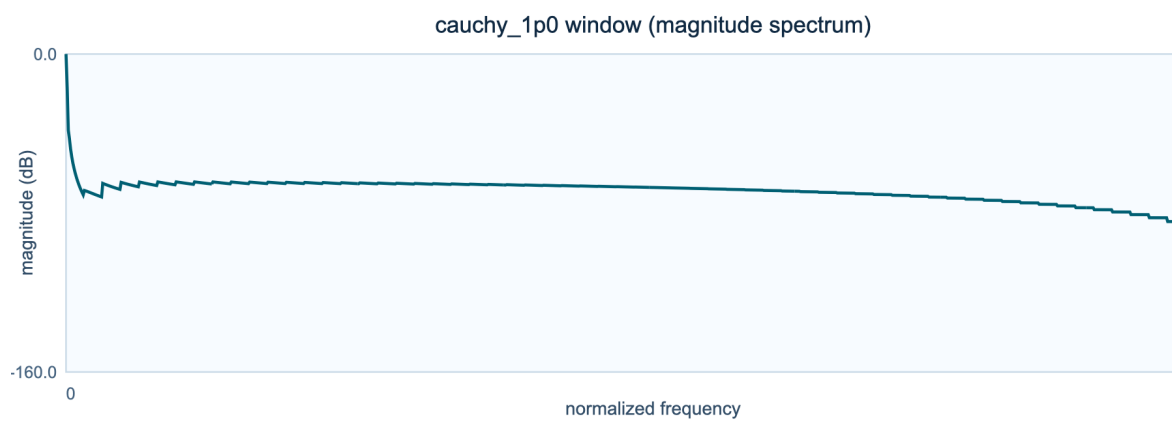


Figure 8.70: Magnitude spectrum of the cauchy 1p0 window.

8.5.35.1 Interpretation and use

The principal strength of this window is straightforward: Heavy tails preserve more peripheral samples than Gaussian. That advantage should be weighed against its main limitation: Tail energy can increase leakage relative to steeper tapers.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you want softer attenuation of far-window samples. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.36 cauchy 2p0

The `cauchy_2p0` window belongs to the `cauchy / lorentzian` family. Its configured parameters are `gamma_ratio=2`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \frac{1}{1 + \left(\frac{n-m}{\gamma}\right)^2}, \quad \gamma = r_\gamma m$$

where γ is the Lorentzian scale, r_γ is its configured ratio, $m = (N - 1)/2$, and n is the sample index.

Table 8.37: Measured properties of the `cauchy 2p0` window.

Metric	Value
Coherent gain	0.927233
Equivalent noise bandwidth	1.004393 bins
Scalloping loss	-3.648 dB
Main-lobe width	2.094 bins
Peak sidelobe	-14.689 dB

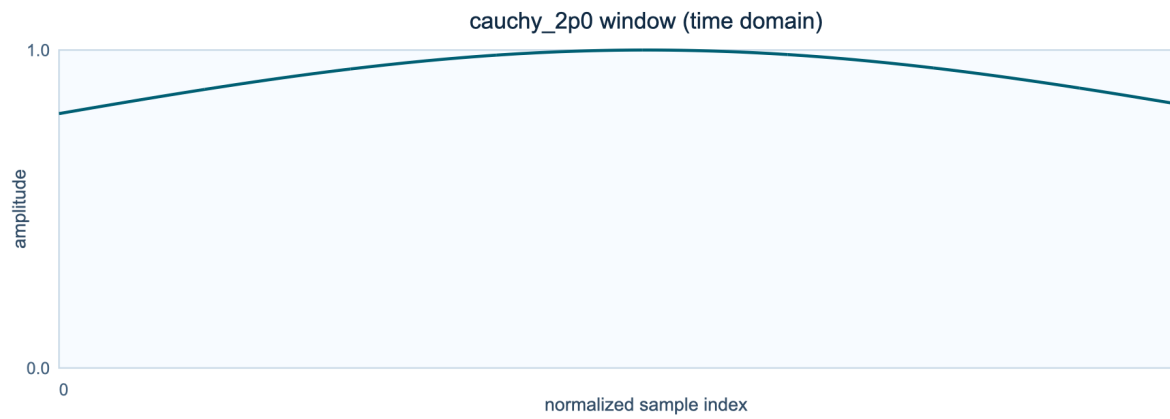


Figure 8.71: Time-domain shape of the cauchy 2p0 window.

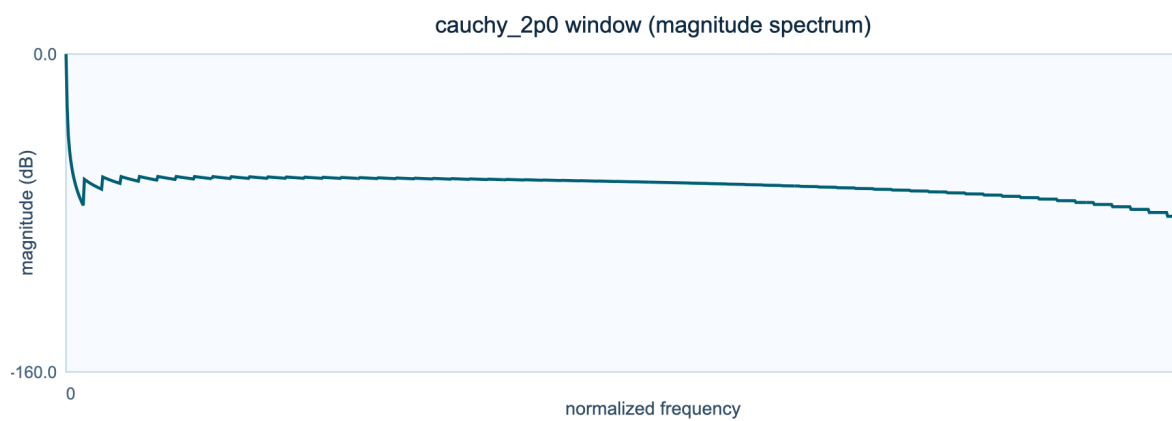


Figure 8.72: Magnitude spectrum of the cauchy 2p0 window.

8.5.36.1 Interpretation and use

The principal strength of this window is straightforward: Heavy tails preserve more peripheral samples than Gaussian. That advantage should be weighed against its main limitation: Tail energy can increase leakage relative to steeper tapers.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you want softer attenuation of far-window samples. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.37 cosine power 2

The `cosine_power_2` window belongs to the cosine power family. Its configured parameters are `power=2`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sin^p\left(\frac{\pi n}{N-1}\right)$$

where p is the cosine-power exponent, n is the sample index, and N is the window length.

Table 8.38: Measured properties of the cosine power 2 window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.500733 bins
Scalloping loss	-1.422 dB
Main-lobe width	4.000 bins
Peak sidelobe	-31.468 dB

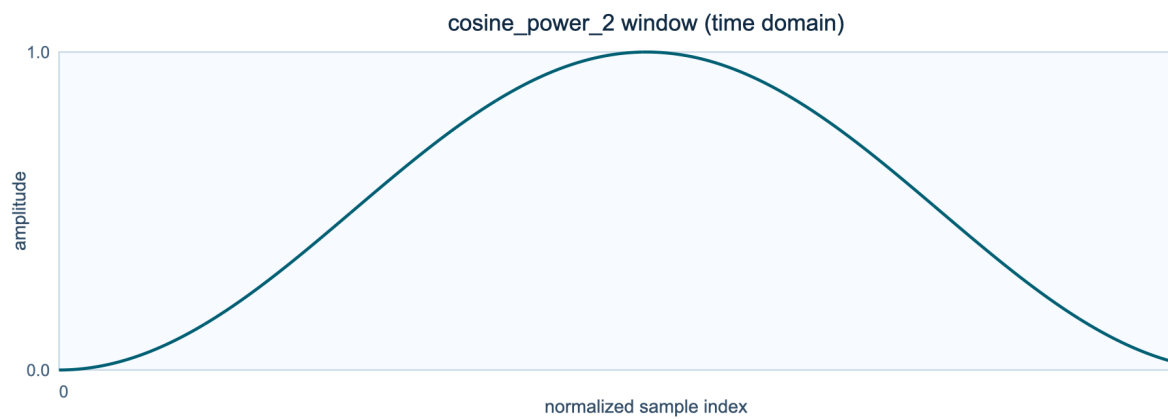


Figure 8.73: Time-domain shape of the cosine power 2 window.

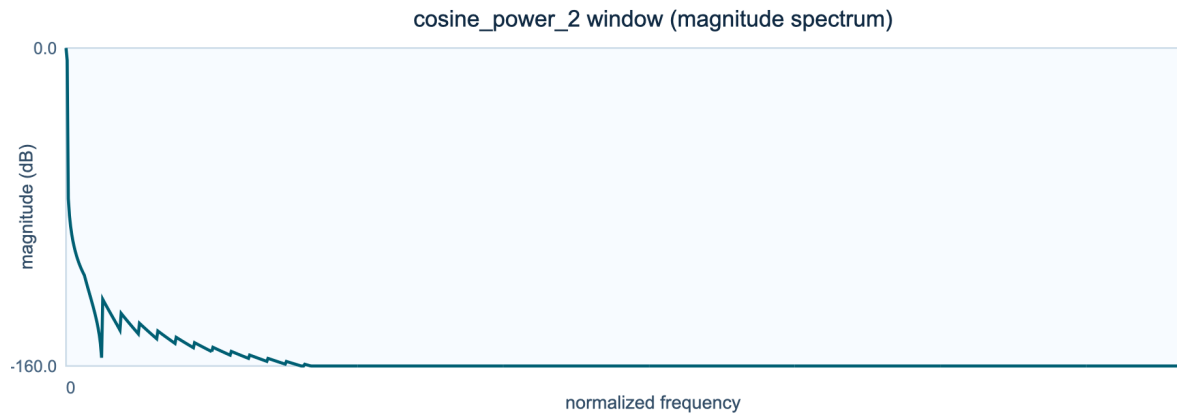


Figure 8.74: Magnitude spectrum of the cosine power 2 window.

8.5.37.1 Interpretation and use

The principal strength of this window is straightforward: Simple power parameter controls edge steepness. That advantage should be weighed against its main limitation: Higher powers can overly narrow effective support.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use to smoothly increase edge attenuation versus basic sine. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.38 cosine power 3

The `cosine_power_3` window belongs to the cosine power family. Its configured parameters are `power=3`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sin^p\left(\frac{\pi n}{N-1}\right)$$

where p is the cosine-power exponent, n is the sample index, and N is the window length.

Table 8.39: Measured properties of the cosine power 3 window.

Metric	Value
Coherent gain	0.424206
Equivalent noise bandwidth	1.735739 bins
Scalloping loss	-1.074 dB
Main-lobe width	5.000 bins
Peak sidelobe	-39.295 dB

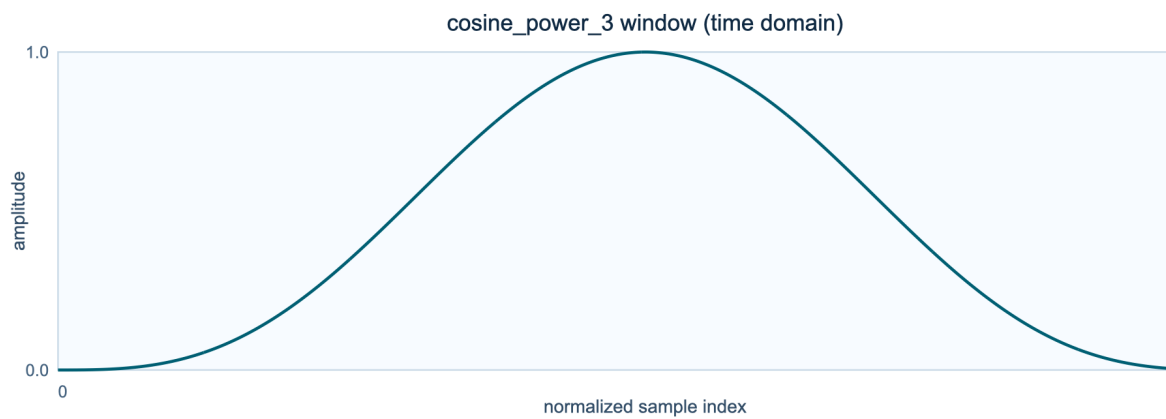


Figure 8.75: Time-domain shape of the cosine power 3 window.

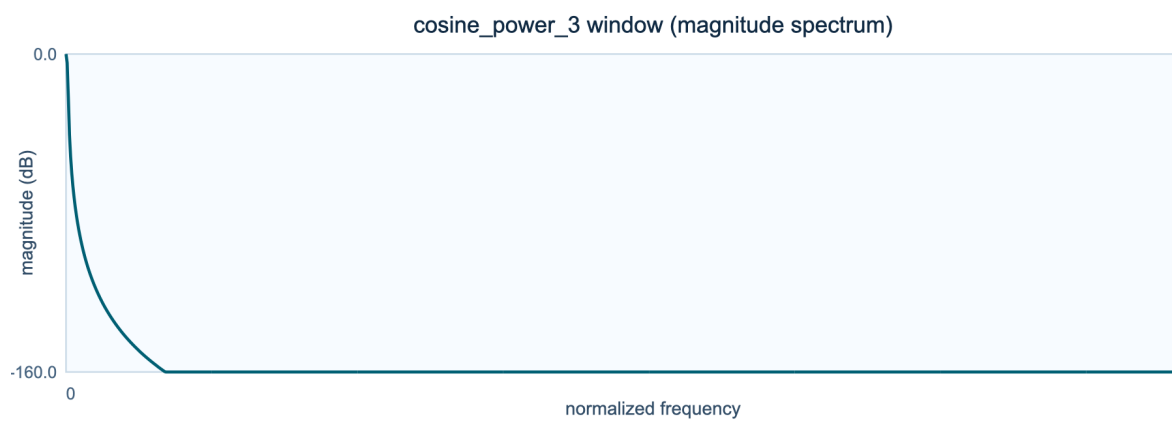


Figure 8.76: Magnitude spectrum of the cosine power 3 window.

8.5.38.1 Interpretation and use

The principal strength of this window is straightforward: Simple power parameter controls edge steepness. That advantage should be weighed against its main limitation: Higher powers can overly narrow effective support.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use to smoothly increase edge attenuation versus basic sine. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.39 cosine power 4

The `cosine_power_4` window belongs to the cosine power family. Its configured parameters are `power=4`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sin^p\left(\frac{\pi n}{N-1}\right)$$

where p is the cosine-power exponent, n is the sample index, and N is the window length.

Table 8.40: Measured properties of the cosine power 4 window.

Metric	Value
Coherent gain	0.374817
Equivalent noise bandwidth	1.945394 bins
Scalloping loss	-0.862 dB
Main-lobe width	6.000 bins
Peak sidelobe	-46.741 dB

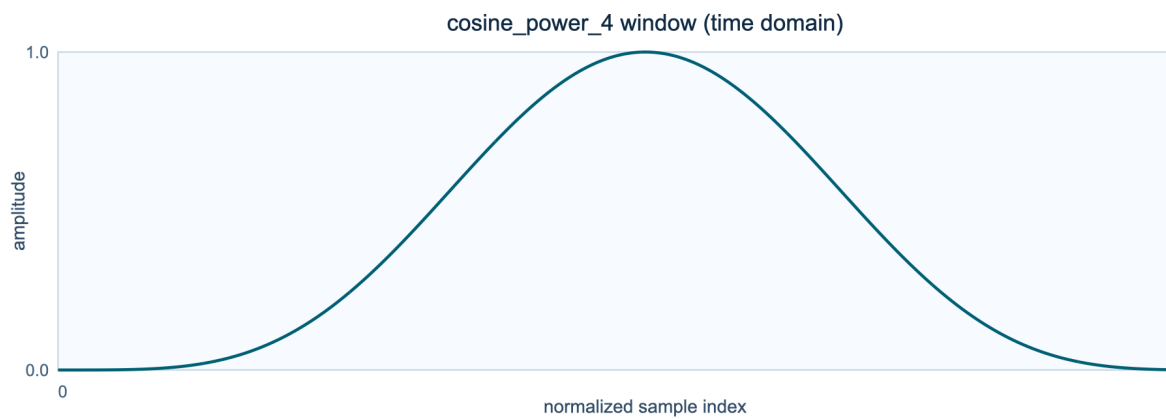


Figure 8.77: Time-domain shape of the cosine power 4 window.

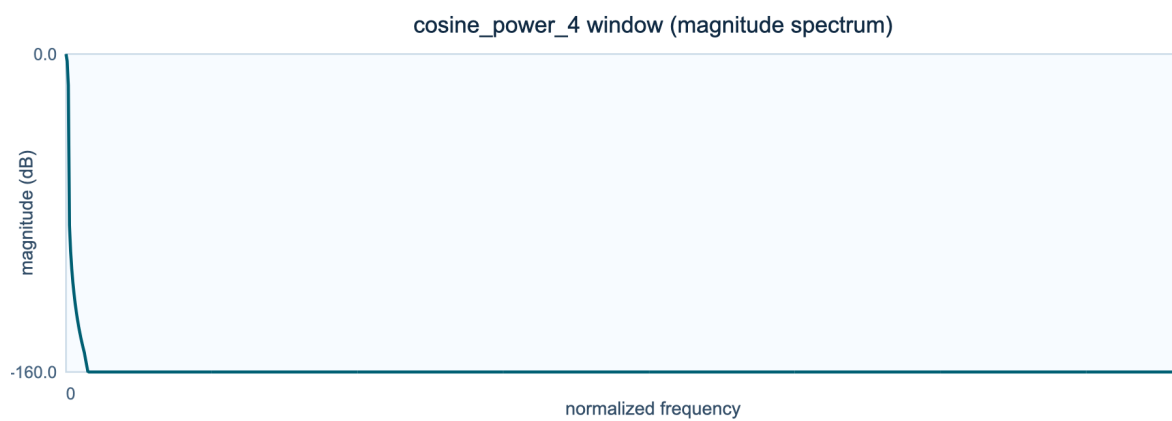


Figure 8.78: Magnitude spectrum of the cosine power 4 window.

8.5.39.1 Interpretation and use

The principal strength of this window is straightforward: Simple power parameter controls edge steepness. That advantage should be weighed against its main limitation: Higher powers can overly narrow effective support.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use to smoothly increase edge attenuation versus basic sine. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.40 hann poisson 0p5

The `hann_poisson_0p5` window belongs to the hann-poisson family. Its configured parameters are `alpha=0.5`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = w_{\text{Hann}}[n] \exp\left(-\alpha \frac{|n-m|}{m}\right)$$

where $w_{\text{Hann}}[n]$ is the Hann window, α controls exponential decay, and $m = (N-1)/2$.

Table 8.41: Measured properties of the hann poisson 0p5 window.

Metric	Value
Coherent gain	0.432946
Equivalent noise bandwidth	1.609944 bins
Scalloping loss	-1.258 dB
Main-lobe width	5.188 bins
Peak sidelobe	-35.245 dB

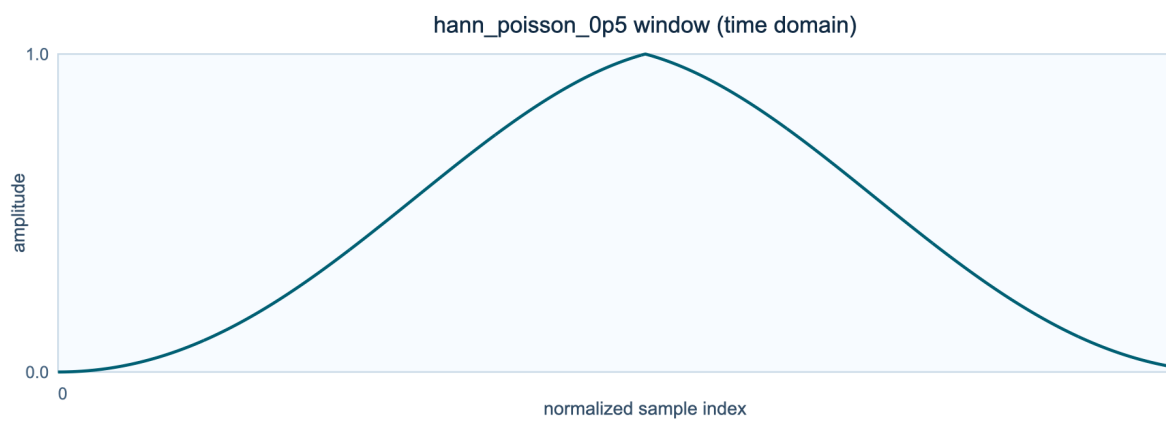


Figure 8.79: Time-domain shape of the hann poisson 0p5 window.

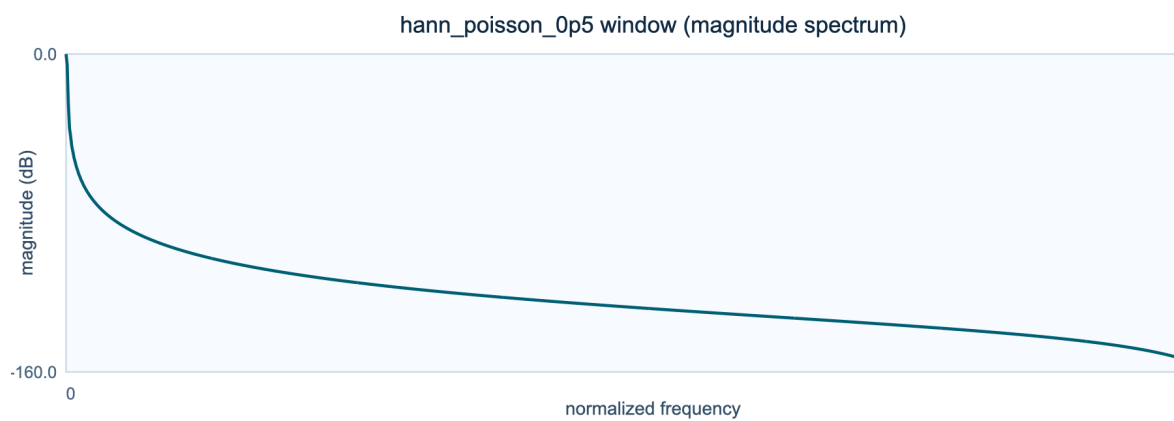


Figure 8.80: Magnitude spectrum of the hann poisson 0p5 window.

8.5.40.1 Interpretation and use

The principal strength of this window is straightforward: Combines smooth Hann center with exponential edge suppression. That advantage should be weighed against its main limitation: Can over-attenuate edges for high alpha values.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you need stronger edge decay than Hann without full flattop cost. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.41 hann poisson 1p0

The `hann_poisson_1p0` window belongs to the hann-poisson family. Its configured parameters are `alpha=1`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = w_{\text{Hann}}[n] \exp\left(-\alpha \frac{|n-m|}{m}\right)$$

where $w_{\text{Hann}}[n]$ is the Hann window, α controls exponential decay, and $m = (N-1)/2$.

Table 8.42: Measured properties of the hann poisson 1p0 window.

Metric	Value
Coherent gain	0.378797
Equivalent noise bandwidth	1.734176 bins
Scalloping loss	-1.112 dB
Main-lobe width	103.750 bins
Peak sidelobe	-80.017 dB

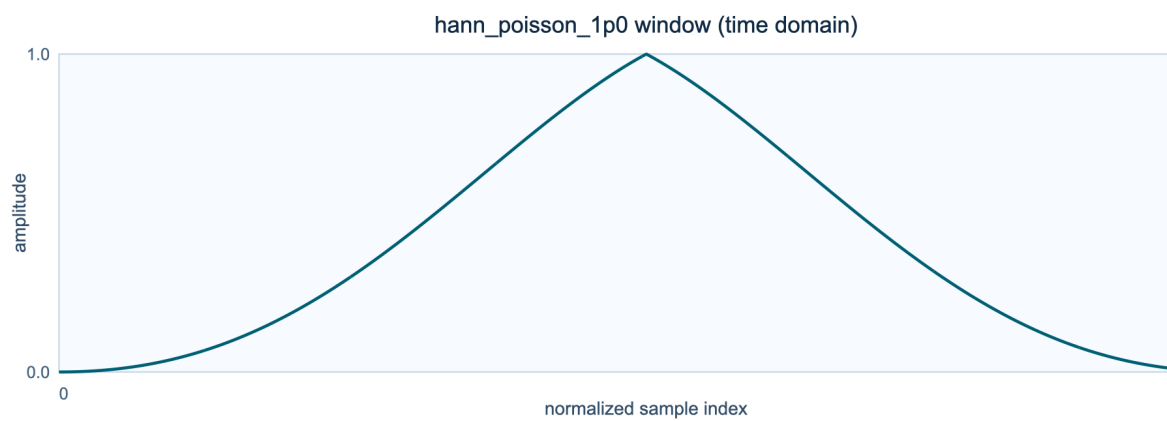


Figure 8.81: Time-domain shape of the hann poisson 1p0 window.

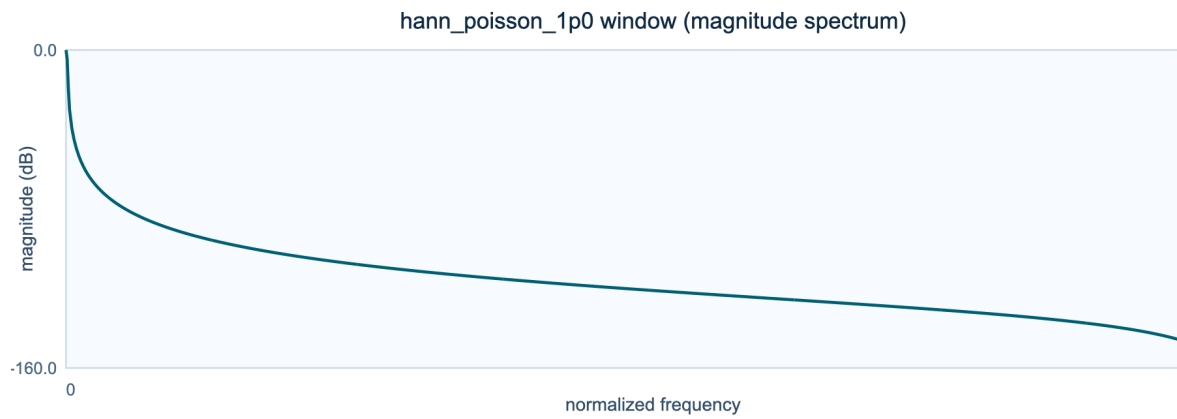


Figure 8.82: Magnitude spectrum of the hann poisson 1p0 window.

8.5.41.1 Interpretation and use

The principal strength of this window is straightforward: Combines smooth Hann center with exponential edge suppression. That advantage should be weighed against its main limitation: Can over-attenuate edges for high alpha values.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you need stronger edge decay than Hann without full flattop cost. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.42 hann poisson 2p0

The `hann_poisson_2p0` window belongs to the hann-poisson family. Its configured parameters are `alpha=2`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = w_{\text{Hann}}[n] \exp\left(-\alpha \frac{|n-m|}{m}\right)$$

where $w_{\text{Hann}}[n]$ is the Hann window, α controls exponential decay, and $m = (N-1)/2$.

Table 8.43: Measured properties of the hann poisson 2p0 window.

Metric	Value
Coherent gain	0.297878
Equivalent noise bandwidth	2.023174 bins
Scalloping loss	-0.870 dB
Main-lobe width	164.719 bins
Peak sidelobe	-80.005 dB

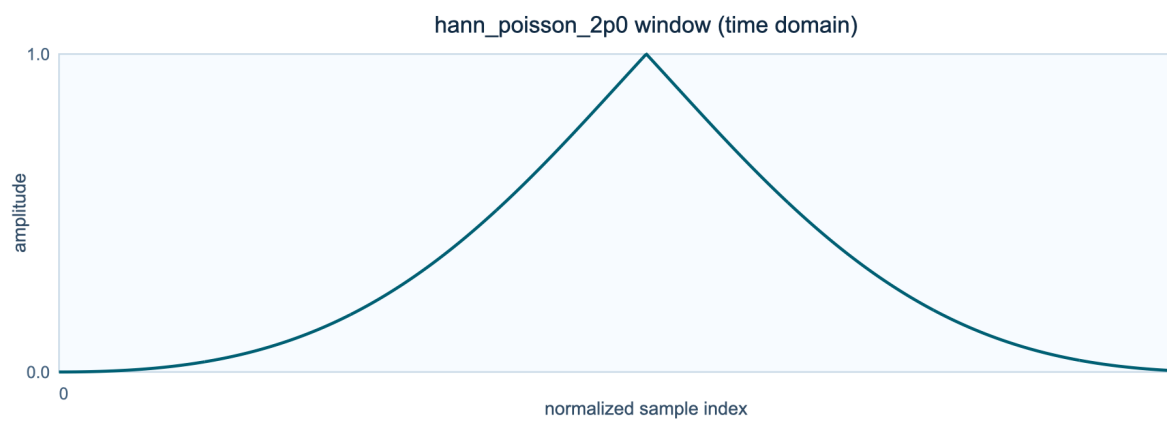


Figure 8.83: Time-domain shape of the hann poisson 2p0 window.

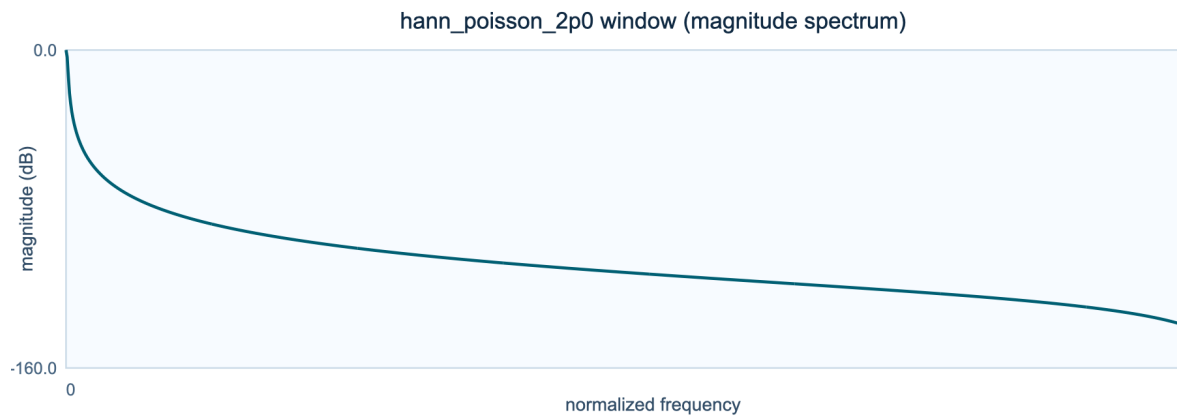


Figure 8.84: Magnitude spectrum of the hann poisson 2p0 window.

8.5.42.1 Interpretation and use

The principal strength of this window is straightforward: Combines smooth Hann center with exponential edge suppression. That advantage should be weighed against its main limitation: Can over-attenuate edges for high alpha values.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you need stronger edge decay than Hann without full flattop cost. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.43 general hamming 0p50

The `general_hamming_0p50` window belongs to the general hamming family. Its configured parameters are `alpha=0.50`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \alpha - (1 - \alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

Table 8.44: Measured properties of the general hamming 0p50 window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.500733 bins
Scalloping loss	-1.422 dB
Main-lobe width	4.000 bins
Peak sidelobe	-31.468 dB



Figure 8.85: Time-domain shape of the general hamming 0p50 window.

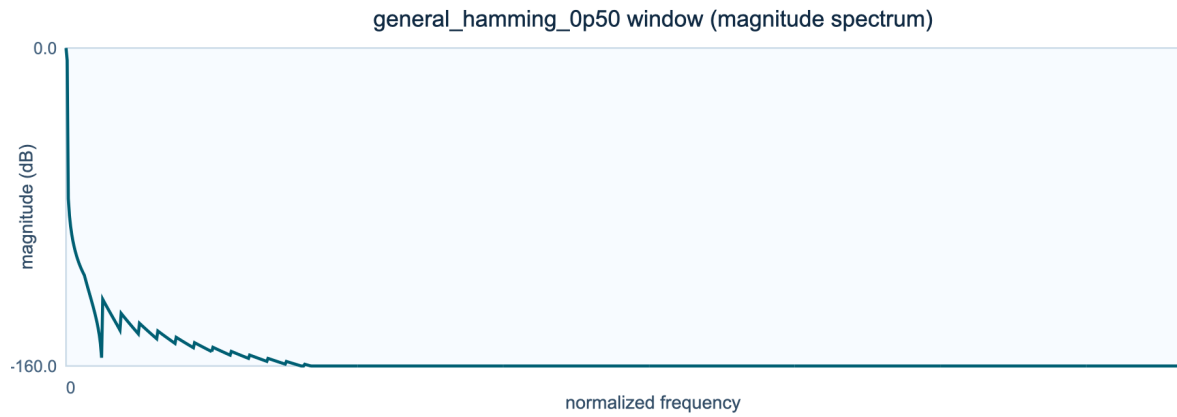


Figure 8.86: Magnitude spectrum of the general hamming 0p50 window.

8.5.43.1 Interpretation and use

The principal strength of this window is straightforward: Tunable Hamming-style cosine weighting. That advantage should be weighed against its main limitation: Less standardized than classic Hann/Hamming choices.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use to sweep sidelobe-vs-width behavior near Hamming family defaults. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.44 general hamming 0p60

The `general_hamming_0p60` window belongs to the general hamming family. Its configured parameters are `alpha=0.60`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \alpha - (1 - \alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

Table 8.45: Measured properties of the general hamming 0p60 window.

Metric	Value
Coherent gain	0.599805
Equivalent noise bandwidth	1.222475 bins
Scalloping loss	-2.179 dB
Main-lobe width	3.469 bins
Peak sidelobe	-31.600 dB



Figure 8.87: Time-domain shape of the general hamming 0p60 window.

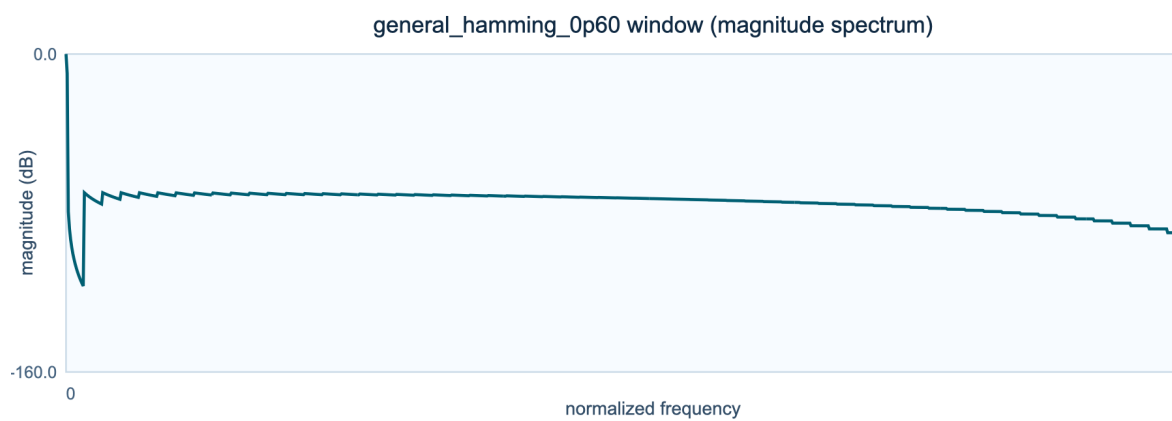


Figure 8.88: Magnitude spectrum of the general hamming 0p60 window.

8.5.44.1 Interpretation and use

The principal strength of this window is straightforward: Tunable Hamming-style cosine weighting. That advantage should be weighed against its main limitation: Less standardized than classic Hann/Hamming choices.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use to sweep sidelobe-vs-width behavior near Hamming family defaults. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.45 general hamming 0p70

The `general_hamming_0p70` window belongs to the general hamming family. Its configured parameters are `alpha=0.70`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \alpha - (1 - \alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

Table 8.46: Measured properties of the general hamming 0p70 window.

Metric	Value
Coherent gain	0.699854
Equivalent noise bandwidth	1.091920 bins
Scalloping loss	-2.762 dB
Main-lobe width	2.656 bins
Peak sidelobe	-24.078 dB

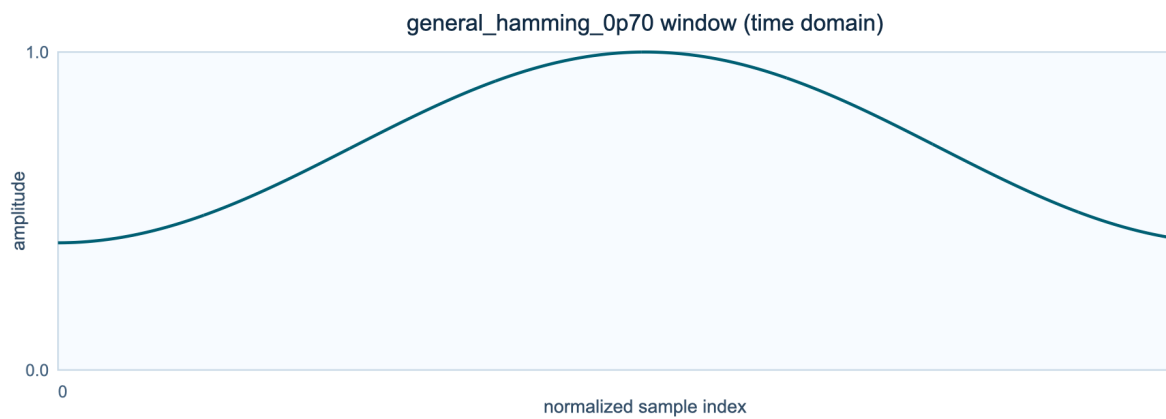


Figure 8.89: Time-domain shape of the general hamming 0p70 window.

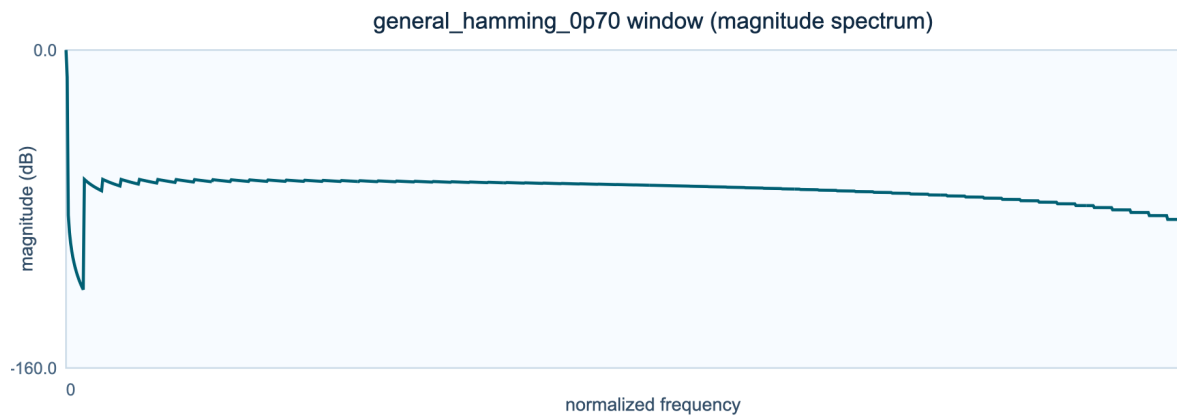


Figure 8.90: Magnitude spectrum of the general hamming 0p70 window.

8.5.45.1 Interpretation and use

The principal strength of this window is straightforward: Tunable Hamming-style cosine weighting. That advantage should be weighed against its main limitation: Less standardized than classic Hann/Hamming choices.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use to sweep sidelobe-vs-width behavior near Hamming family defaults. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.46 general hamming 0p80

The `general_hamming_0p80` window belongs to the general hamming family. Its configured parameters are `alpha=0.80`. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \alpha - (1 - \alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

Table 8.47: Measured properties of the general hamming 0p80 window.

Metric	Value
Coherent gain	0.799902
Equivalent noise bandwidth	1.031273 bins
Scalloping loss	-3.227 dB
Main-lobe width	2.312 bins
Peak sidelobe	-18.649 dB

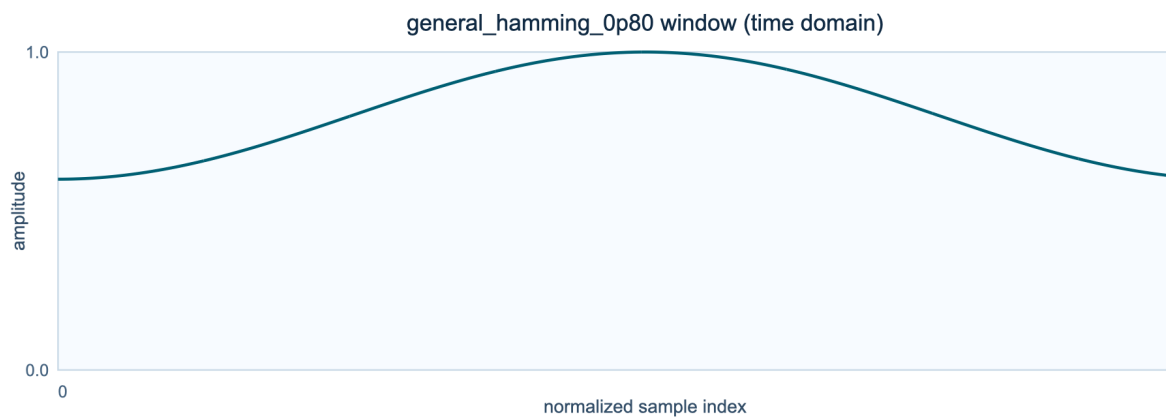


Figure 8.91: Time-domain shape of the general hamming 0p80 window.

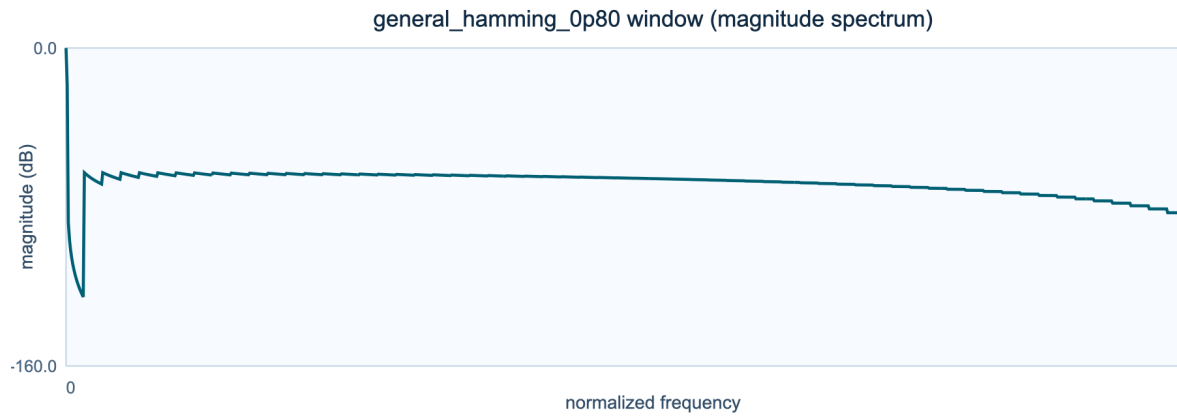


Figure 8.92: Magnitude spectrum of the general hamming 0p80 window.

8.5.46.1 Interpretation and use

The principal strength of this window is straightforward: Tunable Hamming-style cosine weighting. That advantage should be weighed against its main limitation: Less standardized than classic Hann/Hamming choices.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use to sweep sidelobe-vs-width behavior near Hamming family defaults. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.47 bohman

The **bohman** window belongs to the bohman family. Its configured parameters are **none**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = (1 - x) \cos(\pi x) + \frac{\sin(\pi x)}{\pi}$$

where $x = |2n/(N - 1) - 1|$ is absolute centered position, n is the sample index, and N is the window length.

Table 8.48: Measured properties of the bohman window.

Metric	Value
Coherent gain	0.405087
Equivalent noise bandwidth	1.786613 bins
Scalloping loss	-1.022 dB
Main-lobe width	6.000 bins
Peak sidelobe	-45.997 dB

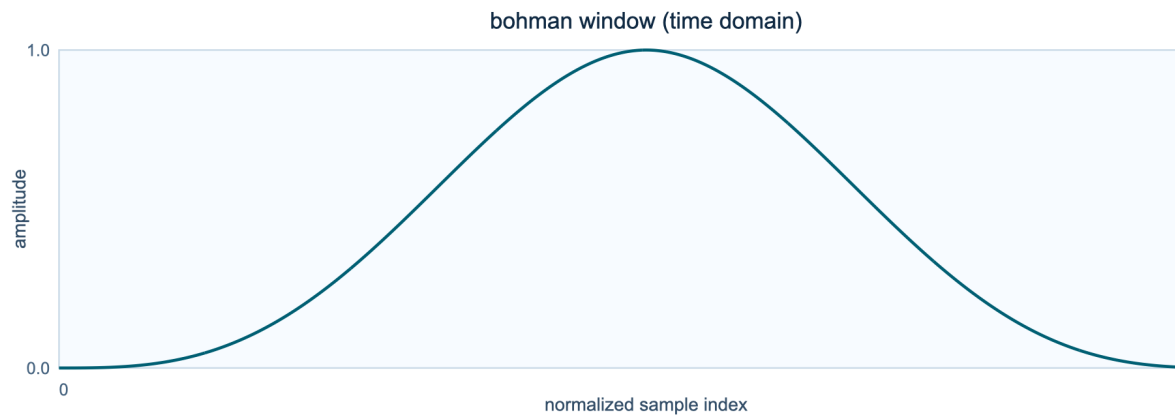


Figure 8.93: Time-domain shape of the bohman window.

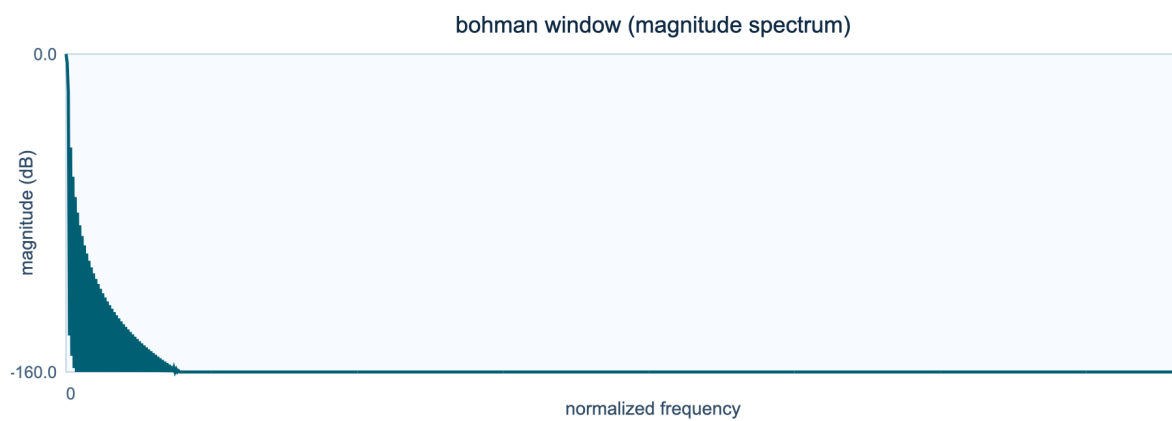


Figure 8.94: Magnitude spectrum of the bohman window.

8.5.47.1 Interpretation and use

The principal strength of this window is straightforward: Continuous-slope taper with good qualitative leakage control. That advantage should be weighed against its main limitation: Less common in audio tooling and harder to tune by intuition.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for exploratory spectral work needing smooth derivatives. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.48 cosine

The **cosine** window belongs to the sinusoidal family. Its configured parameters are **none**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \sin\left(\frac{\pi n}{N-1}\right)$$

where n is the sample index and N is the window length.

Table 8.49: Measured properties of the cosine window.

Metric	Value
Coherent gain	0.636309
Equivalent noise bandwidth	1.234304 bins
Scalloping loss	-2.096 dB
Main-lobe width	3.000 bins
Peak sidelobe	-22.999 dB

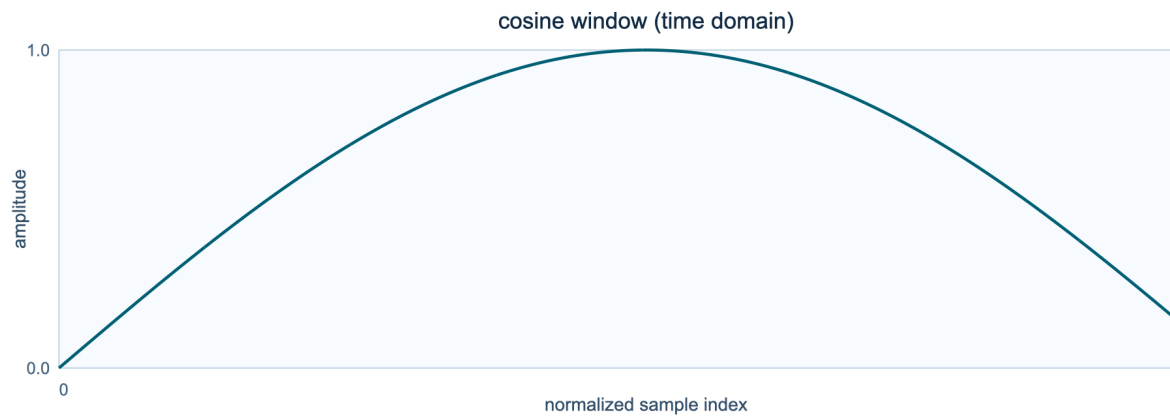


Figure 8.95: Time-domain shape of the cosine window.

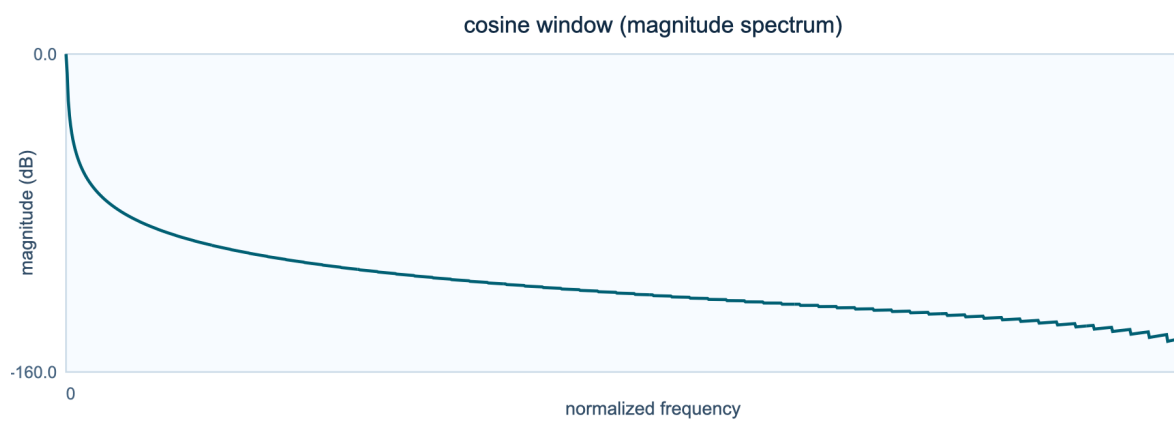


Figure 8.96: Magnitude spectrum of the cosine window.

8.5.48.1 Interpretation and use

The principal strength of this window is straightforward: Smooth endpoint behavior and straightforward implementation. That advantage should be weighed against its main limitation: Less configurable than Kaiser/Tukey families.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use for stable, low-complexity alternatives to Hann. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.49 kaiser

The **kaiser** window belongs to the kaiser-bessel family. Its configured parameters are **beta** from **--kaiser-beta**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = \frac{I_0(\beta\sqrt{1-r_n^2})}{I_0(\beta)}, \quad r_n = \frac{n-m}{m}$$

where I_0 is the modified Bessel function, β controls taper strength, $m = (N-1)/2$, and n is the sample index.

Table 8.50: Measured properties of the kaiser window.

Metric	Value
Coherent gain	0.331708
Equivalent noise bandwidth	2.162181 bins
Scalloping loss	-0.712 dB
Main-lobe width	9.125 bins
Peak sidelobe	-105.921 dB

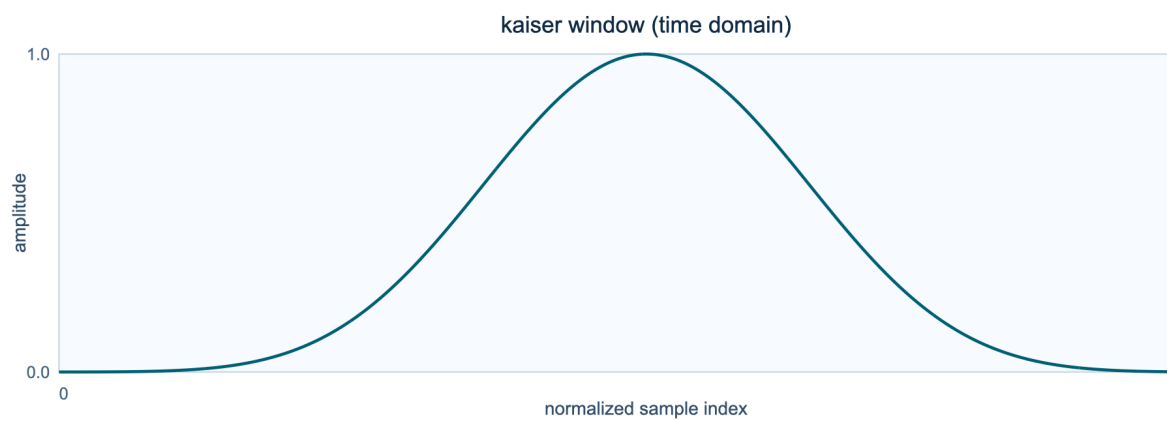


Figure 8.97: Time-domain shape of the kaiser window.

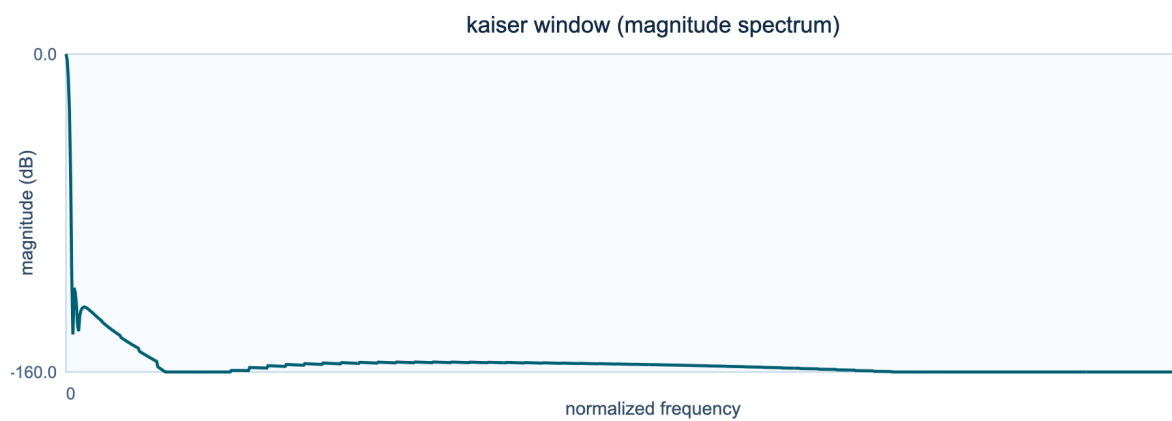


Figure 8.98: Magnitude spectrum of the kaiser window.

8.5.49.1 Interpretation and use

The principal strength of this window is straightforward: Continuously tunable width/sidelobe tradeoff via β . That advantage should be weighed against its main limitation: Needs β tuning; poor β choices can over-blur or under-suppress sidelobes.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use when you need explicit control over leakage versus resolution. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

8.5.50 rect

The **rect** window belongs to the rectangular family. Its configured parameters are **none**. This entry presents the mathematical definition before the graphs so the visible shape can be connected to the coefficients that produce it.

$$w[n] = 1$$

where $w[n]$ is the window coefficient and n is any sample index from 0 through $N - 1$.

Table 8.51: Measured properties of the rect window.

Metric	Value
Coherent gain	1.000000
Equivalent noise bandwidth	1.000000 bins
Scalloping loss	-3.922 dB
Main-lobe width	2.000 bins
Peak sidelobe	-13.264 dB

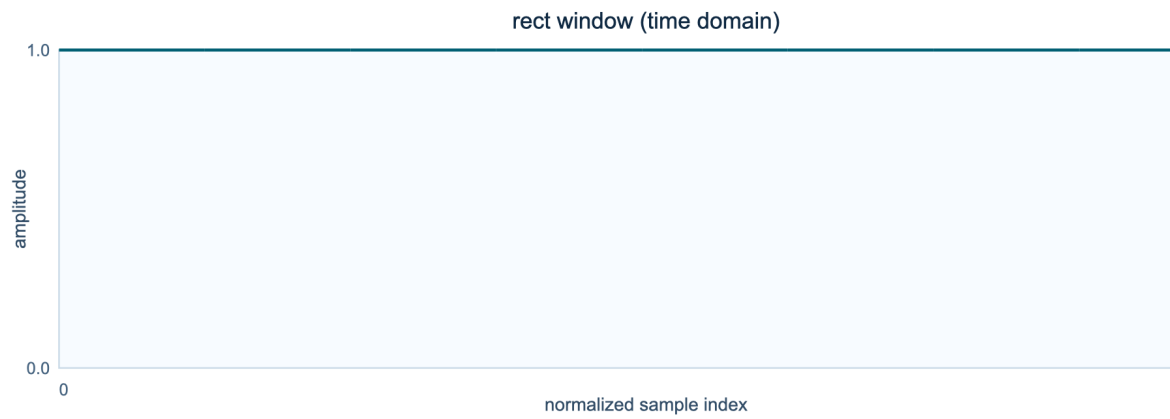


Figure 8.99: Time-domain shape of the rect window.

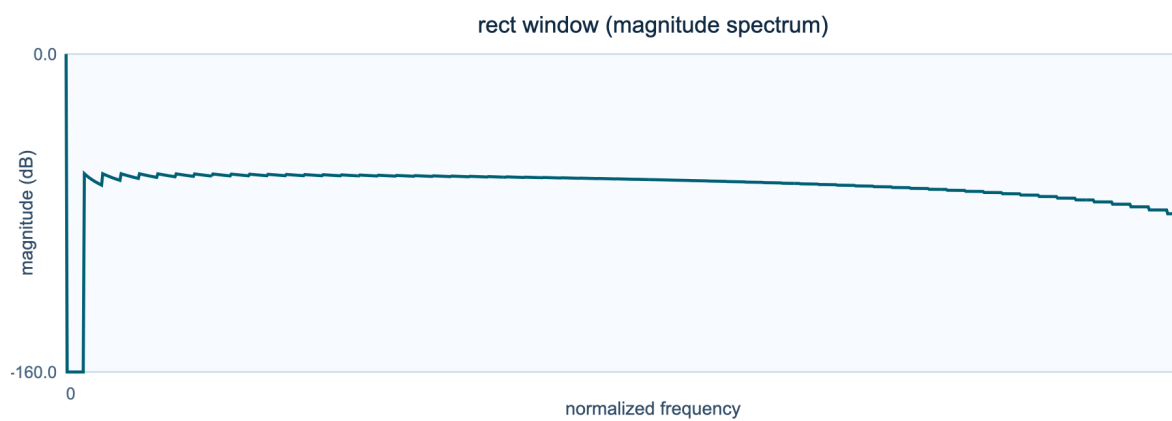


Figure 8.100: Magnitude spectrum of the rect window.

8.5.50.1 Interpretation and use

The principal strength of this window is straightforward: Narrowest main lobe and maximal bin sharpness. That advantage should be weighed against its main limitation: Highest sidelobes and strongest leakage/phasing on non-bin-centered content.

In a phase-vocoder render, the time-domain shape controls how strongly samples near the frame edges participate in the analysis. The frequency-domain graph shows the corresponding compromise between main-lobe width and sidelobe rejection. A narrower main lobe separates nearby stable partials more readily, while lower sidelobes reduce leakage from strong components into weaker neighbours.

The practical recommendation is: Use only for controlled test tones or when leakage is acceptable. Start with a short representative passage, compare against Hann at matched FFT and hop sizes, and listen separately for attack definition, tonal focus, high-frequency haze, and stereo stability. A window should be selected for the material and the requested transformation, not for a single attractive metric.

The reported measurements describe the window itself. They do not replace overlap-add verification. The synthesis hop, normalization policy, phase mode, and transient strategy can change the audible result even when the analysis window is held constant.

CHAPTER 9

Lessons from Paul Koonce's PVC for pvx

This note captures what remains valuable from the historical PVC package and how those ideas can continue to improve **pvx**.

Primary sources: - Princeton archive page (Paul Koonce): <https://www.cs.princeton.edu/courses/archive/spr99/cs325/koonce.html> - Linux Audio package index entry: <https://wiki.linuxaudio.org/apps/all/pvc>

9.1 What PVC Got Right

9.2 Composable command-line tools

PVC exposed many focused commands (**plainpv**, **twarp**, **harmonizer**, **ringfilter**, and others), each tuned for one processing goal.

Why it matters: - small tools are easier to test and reason about - shell pipelines naturally support complex workflows

pvx carry-forward: - subcommands such as **voc**, **freeze**, **harmonize**, **conform**, **retune**, **morph**, **denoise**, **deverb** - managed composition with **pvx chain** - reusable artifact tools: **analysis** (PVXAN) and **response** (PVXRF)

9.3 Dynamic control from external files

PVC supported dynamic parameter control by reading function files and interpolating through time.

Why it matters: - most real audio work needs parameter trajectories, not only constants

pvx carry-forward: - CSV/JSON control-rate signals on core numeric parameters - interpolation modes (**none**, **linear**, **nearest**, **cubic**, **exponential**, **s_curve**, **smootherstep**, **polynomial**)

9.4 Terminal-first discoverability

PVC documentation emphasized “run routine name to see flags/defaults.”

Why it matters: - users working in terminals should not need to read source code first

pvx carry-forward: - richer **--help** text with grouped argument sections - **--example** and **--guided** - script-level help epilogs with copy-paste commands

9.5 Explicit defaults and transparent controls

PVC exposed defaults directly in command help, including ranges and mode flags.

Why it matters: - predictable defaults reduce trial-and-error and improve reproducibility

pvx carry-forward: - explicit defaults in CLI help and docs - deterministic central processing unit (CPU) mode - reproducible benchmark scripts and quality gates

9.6 Script-based workflow reuse

PVC shipped shell scripts as reusable “poor-man’s graphical user interface (GUI)” presets.

Why it matters: - consistent processing recipes are as important as raw algorithms

pvx carry-forward: - cookbook recipes (docs/EXAMPLES.md) - `pvx examples <name>` - benchmark, regression, and documentation generation scripts

9.7 Where pvx Can Improve Further (PVC-Inspired)

Phase 3-5 implementations now cover: - response-driven operators (`filter`, `tvfilter`, `noisefilter`, `bandamp`, `spec-compander`) - ring/resonator family (`ring`, `ringfilter`, `ringtvfilter`) - harmonic mapping (`chordmapper`, `inharmonator`)

Remaining PVC-inspired opportunities: 1. Keep adding “single-purpose tools with shared conventions” over monolithic flags. 2. Preserve plain-text control formats and avoid opaque binary automation formats. 3. Keep command help readable and practical before adding new flags. 4. Ensure each new advanced feature ships with at least one runnable example. 5. Maintain a strong terminal-first workflow for research and production use.

9.8 Suggested Ongoing Checklist

This section begins with the following points, which establish the context for the details that follow.

- For every new algorithm:
 - add one direct one-liner example in `--help`
 - add one entry in docs/EXAMPLES.md
 - add one “when not to use” note
- For every new dynamic parameter:
 - document scalar usage
 - document CSV control usage
 - document JSON control usage
 - document interpolation implications
- For every quality feature:
 - show baseline vs improved command pair
 - state key artifacts to listen for

CHAPTER 10

PVC-to-pvx Parity Matrix

This document tracks functional parity work between Paul Koonce’s historical PVC package and modern pvx.

Primary references: - Princeton archive page (Paul Koonce): <https://www.cs.princeton.edu/courses/archive/spr99/cs325/koonce.html> - Linux Audio package index entry: <https://wiki.linuxaudio.org/apps/all/pvc>

10.1 Scope

This section begins with the following points, which establish the context for the details that follow.

- Goal: preserve useful PVC workflow primitives while keeping pvx quality-first and automation-friendly.
- Out of scope: byte-for-byte command-line compatibility with historical PVC binaries.

10.2 Capability Matrix

PVC capability	Why it matters	Current pvx status	Gap level
Persistent phase-vocoder analysis files (<code>pvanalysis</code>)	Reuse analysis across multiple processing passes	Implemented in phase 1/2 via PVXAN schema and <code>pvx analysis</code>	Closed (foundation)
Response-file workflows (<code>freqresponse</code> , <code>chordresponsemaker</code>)	Reusable filter/target curves	Implemented in phase 1/2 via PVXRF schema and <code>pvx response</code>	Closed (foundation)
Time-varying spectral filtering from response files (<code>tvfilter</code>)	Dynamic timbral trajectories	Implemented in phase 3 via <code>pvx tvfilter</code> + scalar map interpolation	Closed (phase 3)
Ring/resonator family (<code>ring</code> , <code>ringfilter</code> , <code>ringtvfilter</code>)	Controlled resonant coloration	Implemented in phase 4 via dedicated ring CLI family	Closed (phase 4)
Response-driven compander/noise/band emphasis (<code>compander</code> , <code>noisefilter</code> , <code>bandamp</code>)	Precision spectral dynamics	Implemented in phase 3 (<code>pvx spec-compander</code> , <code>pvx noisefilter</code> , <code>pvx bandamp</code>)	Closed (phase 3)
Harmonic/chord remapping (<code>harmonizer</code> , <code>chordmapper</code> , <code>inharmonator</code>)	Musically constrained spectral remap	Implemented in phase 5 via <code>pvx chordmapper</code> and <code>pvx inharmonator</code>	Closed (phase 5)
Spectral convolver with response semantics (<code>convolver</code>)	Controlled blend of source/target spectra	Partial overlap via morph/cross-synthesis	Medium
Generic function stream utilities (<code>envelope</code> , <code>reshape</code>)	Text-based control-signal authoring	Implemented in phase 6 via <code>pvx envelope</code> and <code>pvx reshape</code>	Closed (phase 6)

PVC capability	Why it matters	Current pvx status	Gap level
Parity benchmark scenarios and regression gates	Reproducible quality checks for PVC-inspired operators	Implemented in phase 7 via <code>benchmark-s/run_pvc_parity.py</code> + CI workflow gate	Closed (phase 7)
Explicit resynthesis families and thresholds	Operator transparency and reproducibility	Partially available but not as PVC-style dedicated modes	Medium

10.3 Implemented in Phase 1 and Phase 2

This section begins with the following points, which establish the context for the details that follow.

1. Data-model and schema foundations:
 - PVXAN analysis artifact schema in `/Users/cleider/dev/pvx/src/pvx/core/analysis_store.py`
 - PVXRF response artifact schema in `/Users/cleider/dev/pvx/src/pvx/core/response_store.py`
2. User-facing command-line interface (CLI) foundations:
 - `pvx analysis create|inspect` in `/Users/cleider/dev/pvx/src/pvx/cli/pvxanalysis.py`
 - `pvx response create|inspect` in `/Users/cleider/dev/pvx/src/pvx/cli/pvxresponse.py`
 - Unified CLI registration in `/Users/cleider/dev/pvx/src/pvx/cli/pvx.py`

10.4 Schema Notes

10.4.1 PVXAN

This section begins with the following points, which establish the context for the details that follow.

- Container: compressed NumPy `.npz`
- Required members:
 - `meta_json`
 - `spectrum_real (channels x frames x bins)`
 - `spectrum_imag (channels x frames x bins)`
- Includes deterministic `sha256` digest function for reproducibility checks.

10.4.2 PVXRF

This section begins with the following points, which establish the context for the details that follow.

- Container: compressed NumPy `.npz`
- Required members:
 - `meta_json`
 - `frequencies_hz (bins)`
 - `magnitude (channels x bins)`
 - `phase (channels x bins)`
- Includes deterministic `sha256` digest function for reproducibility checks.

10.5 Implemented in Phase 3, Phase 4, and Phase 5

This section begins with the following points, which establish the context for the details that follow.

1. Response-driven spectral operators (`filter`, `tvfilter`, `noisefilter`, `bandamp`, `spec-compander`):
 - core: `/Users/cleider/dev/pvx/src/pvx/core/pvc_ops.py`

- command-line interface (CLI): `/Users/cleider/dev/pvx/src/pvx/cli/pvxfilter.py` + wrappers
- 2. Ring/resonator family (`ring`, `ringfilter`, `ringtvfilter`):
 - core: `/Users/cleider/dev/pvx/src/pvx/core/pvc_resonators.py`
 - CLI: `/Users/cleider/dev/pvx/src/pvx/cli/pvxring.py` + wrappers
- 3. Harmonic/chord mapping family (`chordmapper`, `inharmonator`):
 - core: `/Users/cleider/dev/pvx/src/pvx/core/pvc_harmony.py`
 - CLI: `/Users/cleider/dev/pvx/src/pvx/cli/pvxharmpmap.py` + wrappers

10.6 Implemented in Phase 6 and Phase 7

This section begins with the following points, which establish the context for the details that follow.

1. Function-stream generators/transforms (`envelope`, `reshape`) interoperable with control-bus workflows:
 - core: `/Users/cleider/dev/pvx/src/pvx/core/pvc_functions.py`
 - CLI: `/Users/cleider/dev/pvx/src/pvx/cli/pvxenvelope.py`, `/Users/cleider/dev/pvx/src/pvx/cli/pvxreshape.py`
 - installed entry points: `pvx envelope`, `pvx reshape`
2. Parity benchmarks and regression gates:
 - benchmark runner: `/Users/cleider/dev/pvx/benchmarks/run_pvc_parity.py`
 - baseline: `/Users/cleider/dev/pvx/benchmarks/baseline_pvc_parity.json`
 - CI workflow: `/Users/cleider/dev/pvx/.github/workflows/pvc-parity-regression.yml`
 - expanded pipeline scenario: `analysis_response_function_chain` (analysis-derived response + function-stream modulation)

10.7 Immediate Next Steps

This section begins with the following points, which establish the context for the details that follow.

1. Add explicit PVC-style resynthesis family toggles and thresholds where they improve auditability without fragmenting the core `pvx voc` interface.
2. Add additional parity scenarios that stress the same pipeline family with persisted artifact round-trips (`pvx analysis create` + `pvx response create`) and stricter drift gates.
3. Add the sorted top-level command roadmap below to stage user-facing workflow expansion in quality-first order.

10.7.1 Sorted Top-Level Command Roadmap

Phase	Priority	Proposed commands	Parity and quality rationale
Phase 1	Highest	<code>doctor</code> , <code>inspect</code> , <code>validate</code> , <code>schema</code> , <code>preset</code> , <code>config</code>	Improves reproducibility and inspectability before introducing more operator complexity.
Phase 2	High	<code>render</code> , <code>graph</code> , <code>queue</code> , <code>watch</code> , <code>cache</code>	Supports longer research- and production-oriented runs with less shell friction.

10. PVC-to-pvx Parity Matrix

Phase	Priority	Proposed commands	Parity and quality rationale
Phase 3	Medium-High	<code>mod</code> , <code>derive</code> , <code>route</code> , <code>quantize</code> , <code>smooth</code>	Brings function-stream/control-signal workflows closer to classic PVC flexibility with modern parameter routing.
Phase 4	Medium	<code>bench</code> , <code>compare</code> , <code>regress</code> , <code>abx</code> , <code>report</code>	Establishes objective parity and regression tracking as first-class command surfaces.
Phase 5	Medium-Low	<code>align</code> , <code>match</code> , <code>stem</code> , <code>spatialize</code> , <code>live</code> , <code>serve</code>	Expands into advanced alignment, immersive, and deployment workflows after core parity and QA gates are stable.

Part IV

Workflow Cookbook

CHAPTER 11

pvx Example Cookbook

All commands are designed to be copy-paste runnable from the repository root. If one fails, the shell will usually inform you with all the warmth of a tax letter.

11.1 Starter Path (Run These First)

If you are new to pvx, run these in order:

```
pvx voc input.wav --stretch 1.2 --output step1_stretch.wav
pvx voc input.wav --stretch 1.0 --pitch -3 --output step2_pitch.wav
pvx voc input.wav --stretch 1.0 --pitch -3 --pitch-mode formant-preserving --output step3_pitch_formant.wav
pvx voc drums.wav --preset drums_safe --stretch 1.25 --output step4_drums.wav
pvx voc mix.wav --stretch 1.2 --stereo-mode mid_side_lock --coherence-strength 0.9 --output step5_stereo.wav
```

Expected audible progression: - **step1**: timing changes only - **step2**: pitch changes only - **step3**: pitch change with improved vocal timbre stability - **step4**: better attack retention on percussive material - **step5**: more stable stereo image during stretch - if all five sound identical, something is broken and tea alone will not save it

Preflight for extreme ratios:

```
pvx stretch-budget one_shot.wav --disk-budget 20GB --bit-depth 16 --requested-stretch 1000000
pvx stretch-budget one_shot.wav --disk-budget 20GB --requested-stretch 1000000 --fail-if-exceeds --json
```

Use this before launching very large renders so budget limits are explicit.

11.2 Use-Case Index (By Theme)

Theme	Start here	Typical tools
Speech intelligibility and timing	1, 2, 25, 61	pvx voc
Musical pitch and formants	3, 4, 5, 29, 37, 48	pvx voc, pvx retune
Extreme ambient and drones	6, 28, 45, 50, 65	pvx voc, pvx freeze
Transient-safe processing	26, 30, 32, 33, 52	pvx voc
Stereo/multichannel coherence	27, 34, 35, 57, 58	pvx voc
Morphing and hybrid sound design	9, 23, 49, 55	pvx morph, pvx layer, pvx harmonize
Cleanup/restoration	11, 12, 50, 53, 64	pvx denoise, pvx deverb

Theme	Start here	Typical tools
Automation and reproducibility	13, 14, 24, 31, 44, 47, 63	<code>pvx conform</code> , <code>pvx voc</code> , benchmarks
Transform and backend research	40, 41, 62	<code>pvx voc</code>
Live-style piping workflows	17, 46, 59	<code>pvx voc</code> + downstream tools

11.3 Slow Down Speech

Command

```
pvx voc speech.wav --preset vocal --stretch 1.35 --output speech_slow.wav
```

Explanation - Uses vocal preset and moderate stretch to increase intelligibility review time.

Before/After - Before: normal speech rate. - After: slower phrase timing, mostly same pitch.

Parameters that matter most - `--stretch` - `--preset vocal`

Artifacts to listen for - consonant smearing - subtle flutter in sustained vowels

11.4 Time-Compress Speech (Faster Playback)

Command

```
pvx voc speech.wav --preset vocal --stretch 0.85 --output speech_fast.wav
```

Explanation - Compresses duration while trying to preserve speech quality.

Before/After - Before: original pace. - After: shorter/faster delivery.

Parameters that matter most - `--stretch < 1` - `--phase-locking identity`

Artifacts to listen for - transient roughness on plosives - “grainy” tails on fricatives

11.5 Raise Pitch Without Changing Speed

Command

```
pvx voc vocal.wav --stretch 1.0 --pitch 3 --output vocal_up3.wav
```

Explanation - Raises pitch by 3 semitones with duration unchanged.

Before/After - Before: original key center. - After: same timing, higher pitch.

Parameters that matter most - `--pitch` - `--stretch 1.0`

Artifacts to listen for - timbral thinning - phasey upper harmonics at larger shifts

11.6 Lower Pitch With Formant Preservation

Command

```
pvx voc vocal.wav --stretch 1.0 --pitch -4 --pitch-mode formant-preserving --output vocal_down4_formant.wav
```

Explanation - Applies downward pitch shift while compensating formant envelope drift.

Before/After - Before: natural vocal size. - After: lower notes with less “giant voice” coloration.

Parameters that matter most - --pitch - --pitch-mode formant-preserving - --formant-lifter

Artifacts to listen for - residual “hollow” vowels - over-correction if formant settings are aggressive

11.7 Stretch + Preserve Formants

Command

```
pvx voc vocal.wav --stretch 1.2 --pitch -2 --pitch-mode formant-preserving --output vocal_stretch_formant.wav
```

Explanation - Combined timing and pitch modification tuned for voice.

Before/After - Before: original timing and key. - After: slower and lower while retaining more vowel identity.

Parameters that matter most - --stretch - --pitch - --pitch-mode

Artifacts to listen for - transient blur at phrase starts - comb-like coloration on loud sustained notes

11.8 Extreme Time Stretch Ambient Pad

Command

```
pvx voc one_shot.wav --preset ambient --target-duration 600 --output one_shot_10min.wav
```

Explanation - Uses ambient preset for very large stretch ratios.

Before/After - Before: short transient-rich source. - After: long evolving drone texture.

Parameters that matter most - --preset ambient - --target-duration - --n-fft, --hop-size

Artifacts to listen for - low-level flutter on bright components - transient residue if source is very percussive

11.9 Freeze a Harmonic Moment

Command

```
pvx freeze guitar_chord.wav --freeze-time 0.45 --duration 12 --output-dir out --suffix _freeze
```

Explanation - Captures one spectral slice and resynthesizes sustained texture.

Before/After - Before: normal decay envelope. - After: held spectral “snapshot.”

Parameters that matter most - `--freeze-time` - `--duration`

Artifacts to listen for - static texture if freeze point is too narrow-band - periodic shimmer if random phase is enabled heavily

11.10 Retune Vocal to A440

Command

```
pvx voc vocal_note.wav --target-f0 440 --pitch-mode formant-preserving --output vocal_A440.wav
```

Explanation - Estimates source F0 and applies a global ratio so the tracked center lands at A4 = 440 Hz.

Before/After - Before: note center may sit sharp/flat vs A440 reference. - After: center frequency aligns to 440 Hz.

Parameters that matter most - `--target-f0` - `--pitch-mode`

Artifacts to listen for - metallic edges on strong consonants - formant drift if `--pitch-mode standard` is used

11.11 Morph Two Sounds

Command

```
pvx morph source_a.wav source_b.wav --alpha 0.35 --blend-mode linear --output morph_35.wav --overwrite
```

Explanation - Interpolates spectral content between A and B (`linear` mode).

Before/After - Before: two distinct sources. - After: hybrid timbre weighted toward A at `alpha=0.35`.

Parameters that matter most - `--alpha` - `--blend-mode` - `--phase-mix` (optional phase control)

Artifacts to listen for - phasing if sources differ strongly in timing - hollow midrange from spectral mismatch

Cross-synthesis variants

```
# Envelope transfer: carrier A with modulator B spectral envelope
\index{Envelope transfer carrier A with modulator B spectral envelope}
pvx morph source_a.wav source_b.wav --blend-mode carrier_a_envelope_b --alpha 0.8 --envelope-lifter 36 --
  output morph_env_ab.wav --overwrite

# Spectral-mask transfer: carrier A masked by modulator B
\index{Spectral-mask transfer carrier A masked by modulator B}
pvx morph source_a.wav source_b.wav --blend-mode carrier_a_mask_b --alpha 0.7 --mask-exponent 1.2 --output
  morph_mask_ab.wav --overwrite

# Magnitude/phase exchange style
\index{Magnitude/phase exchange style}
pvx morph source_a.wav source_b.wav --blend-mode magnitude_b_phase_a --alpha 0.65 --phase-mix 0.1 --output
  morph_magB_phaseA.wav --overwrite
```

True A->B trajectory morph (time-varying alpha in one command)

```
pvx morph source_a.wav source_b.wav --alpha controls/alpha_curve.csv --interp linear --blend-mode linear --
output morph_a_to_b.wav --overwrite
```

Example controls/alpha_curve.csv:

```
time_sec,value
0.0,0.0
1.5,0.35
3.0,0.70
4.5,1.0
```

Notes: - --alpha accepts scalar or control file (.csv/.json). - --interp and --order control interpolation for trajectory files. - This produces a genuine frame-wise morph progression from A toward B over output time.

11.12 Create Chorus/Unison Thickness

Command

```
pvx unison synth.wav --voices 7 --detune-cents 18 --width 1.2 --dry-mix 0.15 --output-dir out --suffix _uni7
```

Explanation - Generates detuned multi-voice spread.

Before/After - Before: narrow mono/stereo lead. - After: wider, denser ensemble-like body.

Parameters that matter most - --voices - --detune-cents - --width

Artifacts to listen for - excessive beating at large detune - low-end smearing if source is bass-heavy

11.13 De-Noise Field Recording

Command

```
pvx denoise field.wav --noise-seconds 0.5 --reduction-db 10 --smooth 7 --output-dir out --suffix _den
```

Explanation - Builds a noise estimate from the opening region and subtracts adaptively.

Before/After - Before: broadband hiss/air/noise floor. - After: lower noise floor with possible detail loss.

Parameters that matter most - --noise-seconds - --reduction-db - --smooth

Artifacts to listen for - musical noise - dullness in high frequencies

Useful variants

```
# Speech-safe denoise (preserve intelligibility and avoid over-subtraction)
\index{Speech-safe denoise (preserve intelligibility and avoid over-subtraction)}
pvx denoise speech.wav --noise-seconds 0.4 --reduction-db 5 --floor 0.2 --smooth 9 --output speech_clean.wav

# Music-safe denoise (retain more ambience/harmonic detail)
\index{Music-safe denoise (retain more ambience/harmonic detail)}
pvx denoise mix.wav --noise-seconds 0.3 --reduction-db 4 --floor 0.25 --smooth 7 --output mix_clean.wav

# Denoise before stretching
\index{Denoise before stretching}
pvx denoise noisy.wav --reduction-db 6 --stdout | pvx voc - --stretch 2.0 --output noisy_clean_stretch.wav
```

Variant notes - Lower `--reduction-db` and higher `--smooth` generally reduce metallic residual artifacts.
- `--noise-file` is recommended when the file start is not representative noise.

11.14 De-Reverb a Room Recording

Command

```
pvx deverb room_voice.wav --strength 0.55 --decay 0.90 --floor 0.12 --output-dir out --suffix _dry
```

Explanation - Suppresses late reverberant tails in spectral domain.

Before/After - Before: long room decay and reduced articulation. - After: drier presentation and clearer direct sound.

Parameters that matter most - `--strength` - `--decay` - `--floor`

Artifacts to listen for - pumping on sustained vowels - “underwater” tails if too aggressive

11.15 Segment-Based Dynamic Stretch via CSV

Command

```
pvx voc phrase.wav --pitch-map map_warp.csv --output phrase_map.wav
```

Example `map_warp.csv`

```
start_sec,end_sec,stretch
0.0,0.8,1.00
0.8,1.6,1.25
1.6,2.2,0.90
2.2,3.0,1.10
```

Explanation - Applies piecewise timing control across segments.

Before/After - Before: fixed timing. - After: phrase-level temporal reshaping.

Parameters that matter most - CSV `stretch` - segment boundaries - `--pitch-map-crossfade-ms`

Artifacts to listen for - boundary clicks if crossfade is too short - unnatural tempo discontinuities

11.16 Sidechain Pitch Trajectory (A Controls B)

Command

```
pvx follow A.wav B.wav --emit pitch_to_stretch --pitch-conf-min 0.75 --output B_follow.wav
```

Explanation - Runs the full sidechain flow in one command: track A, build control map, apply to B.

Before/After - Before: B has its own pitch motion. - After: B follows A’s timing contour where confidence is sufficient.

Parameters that matter most - `--backend` and tracker frame settings - `--pitch-conf-min` - `--pitch-lowconf-mode`

Artifacts to listen for - warbling from noisy tracker segments - abrupt ratio jumps at low-confidence transitions

Advanced variant (feature vector sidechain with MFCC + MPEG-7-style flux)


```
pvx pitch-track A.wav --feature-set all --mfcc-count 13 --output - \
| pvx voc B.wav --control-stdin \
  --route pitch_ratio=affine(mfcc_01,0.002,1.0) \
  --route pitch_ratio=clip(pitch_ratio,0.5,2.0) \
  --route stretch=affine(mpeg7_spectral_flux,0.05,1.0) \
  --route stretch=clip(stretch,0.85,1.6) \
  --output B_feature_follow.wav
```

More feature-follow and feature-vector routing recipes: - docs/FEATURE_SIDECHAIN_EXAMPLES.md

Built-in pvx follow recipe printer:

```
pvx follow --example
pvx follow --example all
pvx follow --example noise_aware
```

11.17 GPU Acceleration

Command

```
pvx voc mix.wav --gpu --stretch 1.12 --output mix_gpu.wav
```

Explanation - Uses CUDA path if available (--gpu alias for --device cuda).

Before/After - Before: CPU render time baseline. - After: same algorithmic output path with potentially lower render time.

Parameters that matter most - --device / --gpu - FFT size and hop

Artifacts to listen for - normally identical class of artifacts to CPU; compare for parity in edge cases

11.18 Batch Processing Folder

Command

```
pvx voc stems/*.wav --preset vocal --stretch 1.05 --output-dir out/stems --overwrite
```

Explanation - Applies consistent processing across many files.

Before/After - Before: original folder material. - After: each file rendered with _pv suffix in target folder.

Parameters that matter most - wildcard expansion - --output-dir - --overwrite

Artifacts to listen for - per-file content mismatch under one global setting

11.19 Pipe Multiple Tools in One Line

Command

```
pvx voc input.wav --stretch 1.15 --stdout \  
| pvx denoise - --reduction-db 8 --stdout \  
| pvx deverb - --strength 0.4 --stdout > cleaned.wav
```

Explanation - Builds a linear chain without intermediate files.

Before/After - Before: raw source. - After: stretched + denoised + dereverberated result.

Parameters that matter most - each stage intensity - pipe ordering

Artifacts to listen for - cumulative over-processing - clipped peaks if mastering limits are omitted

11.20 Auto Profile + Auto Transform (Planning)

Command

```
pvx voc input.wav --auto-profile --auto-transform --explain-plan
```

Explanation - Prints selected processing plan JSON without rendering audio.

Before/After - Before: no run metadata. - After: explicit resolved settings.

Parameters that matter most - --auto-profile-lookahead-seconds

Artifacts to listen for - n/a (plan-only mode)

11.21 Multi-Resolution Fusion Render

Command

```
pvx voc input.wav --multires-fusion --multires-ffts 1024,2048,4096 --multires-weights 0.2,0.35,0.45 --  
stretch 1.25 --output input_multires.wav
```

Explanation - Blends multiple FFT scales to reduce single-resolution bias.

Before/After - Before: single-resolution render. - After: potentially smoother balance between transient sharpness and tonal stability.

Parameters that matter most - --multires-ffts - --multires-weights

Artifacts to listen for - slight chorus/phasing if fusion weights over-emphasize conflicting scales

11.22 Checkpoint + Resume Long Render

Command (first pass)

```
pvx voc long_source.wav --preset extreme --auto-segment-seconds 0.5 --checkpoint-dir checkpoints --manifest-  
json reports/run_manifest.json --output long_pass1.wav
```

Command (resume)

```
pvx voc long_source.wav --preset extreme --auto-segment-seconds 0.5 --checkpoint-dir checkpoints --resume --manifest-json reports/run_manifest.json --manifest-append --output long_pass2.wav
```

Explanation - Stores per-segment chunks and reuses completed ones.

Before/After - Before: interrupted run loses progress. - After: restart reuses cached segments.

Parameters that matter most - --auto-segment-seconds - --checkpoint-dir - --resume

Artifacts to listen for - segment boundary smoothness (tune crossfade)

11.23 Harmonize Triad Stack

Command

```
pvx harmonize lead.wav --intervals 0,4,7 --intervals-cents 0,4,-3 --gains 1.0,0.85,0.78 --pans -0.35,0.0,0.35 --force-stereo --output-dir out --suffix _triad
```

Explanation - Builds triad with subtle micro-detune offsets.

Before/After - Before: single voice. - After: harmonized stack with stereo spread.

Parameters that matter most - --intervals - --intervals-cents - --pans

Artifacts to listen for - beating and clutter if gains are too high

11.24 Timeline Warp with pvxwarp

Command

```
pvx warp drums.wav --map map_warp.csv --crossfade-ms 10 --output-dir out --suffix _warp
```

Explanation - Applies time-only map to drive phrase-level groove edits.

Before/After - Before: fixed groove. - After: section-dependent push/pull timing.

Parameters that matter most - map values - --crossfade-ms

Artifacts to listen for - timing discontinuities at map boundaries

11.25 Layer Harmonic and Percussive Paths

Command

```
pvx layer full_mix.wav \
  --harmonic-stretch 1.15 --harmonic-pitch-semitones 2 --harmonic-gain 0.95 \
  --percussive-stretch 1.00 --percussive-pitch-semitones 0 --percussive-gain 1.05 \
  --output-dir out --suffix _layered
```

Explanation - Separates harmonic/percussive content and processes each path differently.

Before/After - Before: one global process for all content. - After: tonal and transient content treated independently.

Parameters that matter most - harmonic/percussive stretch and gain - HPSS kernels

Artifacts to listen for - separation bleed - phase cancellation between paths if gains are extreme

11.26 JSON Manifest + Automation-Friendly Logging

Command

```
pvx voc input.wav --stretch 1.08 --pitch -1 \  
  --manifest-json reports/pvx_runs.json --manifest-append \  
  --output out/input_take1.wav
```

Explanation - Writes machine-readable run metadata for CI jobs, dataset pipelines, or batch monitoring.

Before/After - Before: render exists but settings/provenance are not captured. - After: output audio plus JSON metadata with resolved parameters and timing.

Parameters that matter most - --manifest-json - --manifest-append - stable output naming convention

Artifacts to listen for - not audio-specific; validate manifest integrity and path consistency

11.27 Speech Natural Stretch (Hybrid Transients)

Command

```
pvx voc speech.wav --preset vocal_studio --transient-mode hybrid --stretch 1.25 --output speech_natural.wav
```

Explanation - Uses speech-focused preset plus hybrid transient handling to reduce consonant smear.

Before/After - Before: original speech rate. - After: slower pacing with cleaner consonants vs pure PV.

Parameters that matter most - --preset vocal_studio - --transient-mode hybrid - --transient-sensitivity

Artifacts to listen for - residual metallic tone on strong fricatives

11.28 Drums Safe Stretch

Command

```
pvx voc drums.wav --preset drums_safe --time-stretch 1.4 --output drums_safe.wav
```

Explanation - Uses WSOLA-forward transient handling for percussive attacks.

Before/After - Before: tight attack transients. - After: longer groove with fewer softened hits than basic PV.

Parameters that matter most - --preset drums_safe - --transient-mode wsola

Artifacts to listen for - repetitive texture on sustained cymbal tails

11.29 Stereo Coherent Stretch

Command

```
pvx voc wide_mix.wav --preset stereo_coherent --stretch 1.2 --output wide_mix_coherent.wav
```

Explanation - Uses mid/side coherence coupling to reduce image wobble.

Before/After - Before: original stereo image. - After: stretched image with more stable center and side phase relation.

Parameters that matter most - `--stereo-mode` - `--coherence-strength`

Artifacts to listen for - slight narrowing if coherence is set too high

11.30 Extreme Ambient Stretch (10-minute target)

Command

```
pvx voc one_shot.wav --preset extreme_ambient --target-duration 600 --output one_shot_ambient_10min.wav
```

Explanation - Long-form multistage stretch preset with transient-aware blending.

Before/After - Before: short source event. - After: long ambient texture.

Parameters that matter most - `--preset extreme_ambient` - `--target-duration` - `--max-stage-stretch`

Artifacts to listen for - low-level swirl if source is very broadband

11.31 Pitch Shift with Formant Preservation

Command

```
pvx voc vocal.wav --stretch 1.0 --pitch -4 --pitch-mode formant-preserving --output vocal_down4_formant.wav
```

Explanation - Separates pitch movement from formant envelope correction.

Before/After - Before: original pitch center. - After: lower notes with better vowel identity retention.

Parameters that matter most - `--pitch` - `--pitch-mode formant-preserving` - `--formant-strength`

Artifacts to listen for - hollow timbre if over-corrected

11.32 Hybrid Transient Mode (Manual Tuning)

Command

```
pvx voc source.wav \
  --transient-mode hybrid \
  --transient-sensitivity 0.65 \
  --transient-protect-ms 26 \
  --transient-crossfade-ms 8 \
  --time-stretch 1.3 \
  --output source_hybrid_tuned.wav
```

Explanation - Explicitly tunes detector and stitch behavior instead of relying on preset defaults.

Before/After - Before: single global stretch strategy. - After: transient windows handled differently from steady-state regions.

Parameters that matter most - --transient-sensitivity - --transient-protect-ms - --transient-crossfade-ms

Artifacts to listen for - boundary roughness if crossfade is too short

11.33 Benchmark Command (pvx vs Rubber Band vs librosa)

Command

```
python3 benchmarks/run_bench.py --quick --out-dir benchmarks/out --baseline benchmarks/baseline_small.json --gate
```

Explanation - Runs cycle-consistency benchmark and fails if pvx regresses against baseline.

Before/After - Before: no objective quality snapshot. - After: JSON/Markdown benchmark report plus regression verdict.

Parameters that matter most - --quick - --baseline - --gate

Artifacts to listen for - n/a (objective benchmark command)

11.34 Transient Reset Mode for Cleaner Attacks

Command

```
pvx voc drums.wav --stretch 1.22 --transient-mode reset --transient-sensitivity 0.62 --output drums_reset.wav
```

Explanation - Applies phase resets around detected onsets while leaving steady regions in PV flow.

Before/After - Before: transient-rich grooves may soften at larger stretches. - After: snare/kick attack definition is usually sharper.

Parameters that matter most - --transient-mode reset - --transient-sensitivity

Artifacts to listen for - onset clicks if sensitivity is too high

11.35 WSOLA-Only Transient Handling

Command

```
pvx voc percussion.wav --stretch 1.35 --transient-mode wsola --transient-protect-ms 35 --output percussion_wsola.wav
```

Explanation - Prioritizes time-domain overlap/similarity handling for transient regions.

Before/After - Before: attack smearing in strictly spectral paths. - After: stronger transient solidity with different texture in sustains.

Parameters that matter most - --transient-mode wsola - --transient-protect-ms

Artifacts to listen for - grain boundaries if crossfade/protect windows are mismatched

11.36 Mid/Side Stereo Coherence Stretch

Command

```
pvx voc stereo_mix.wav --stretch 1.18 --stereo-mode mid_side_lock --coherence-strength 0.9 --output
stereo_ms_lock.wav
```

Explanation - Preserves stereo image geometry by constraining M/S phase behavior.

Before/After - Before: image may wobble on sustained sections. - After: center and width usually feel more stable.

Parameters that matter most - `--stereo-mode mid_side_lock` - `--coherence-strength`

Artifacts to listen for - over-constrained width if coherence is set too high

11.37 Reference-Channel Lock for Multichannel Content

Command

```
pvx voc multichannel.wav --stretch 1.12 --stereo-mode ref_channel_lock --ref-channel 0 --coherence-strength
0.85 --output multichannel_ref_lock.wav
```

Explanation - Uses one channel as phase anchor to reduce inter-channel decorrelation.

Before/After - Before: surround image drift on strong harmonics. - After: tighter channel relationship across processing.

Parameters that matter most - `--stereo-mode ref_channel_lock` - `--ref-channel` - `--coherence-strength`

Artifacts to listen for - channel dominance bias if reference channel is atypical

11.38 Irrational Pitch Ratio Input

Command

```
pvx voc tone.wav --stretch 1.0 --pitch-ratio "2^(1/12)" --output tone_up_1semitone_expr.wav
```

Explanation - Uses expression parsing for irrational ratio input directly on CLI.

Before/After - Before: original fundamental. - After: +1 equal-tempered semitone.

Parameters that matter most - `--pitch-ratio expression` - `--stretch 1.0`

Artifacts to listen for - none specific for small shifts; monitor formant drift on voice

11.39 Just-Ratio Pitch Input

Command

```
pvx voc tone.wav --stretch 1.0 --pitch-ratio 3/2 --output tone_perfect_fifth.wav
```

Explanation - Uses just-intonation ratio syntax for exact rational interval control.

Before/After - Before: root pitch. - After: perfect fifth above (3:2).

Parameters that matter most - `--pitch-ratio 3/2`

Artifacts to listen for - beating vs reference if mixed with equal-tempered material

11.40 N-TET CSV Map (19-TET) with pvxconform

Command

```
pvx conform lead.wav --map maps/map_19tet.csv --output-dir out --suffix _19tet
```

Explanation - Applies segmentwise pitch/time map from CSV using 19-tone equal temperament ratios.

Before/After - Before: 12-TET phrases. - After: 19-TET trajectory according to map.

Parameters that matter most - --map - CSV pitch_ratio values

Artifacts to listen for - boundary roughness if map segments are too short

11.41 Just-Intonation CSV Conform Map

Command

```
pvx conform choir.wav --map maps/map_just_intonation.csv --output-dir out --suffix _ji
```

Explanation - Retunes with segment ratios such as 5/4, 6/5, 7/4, 9/8.

Before/After - Before: equal-tempered intonation center. - After: just-intonation harmonic color.

Parameters that matter most - map segment durations - map pitch_ratio

Artifacts to listen for - abrupt color shifts between unrelated just ratios

11.42 Transform A/B: FFT vs DCT

Command

```
pvx voc source.wav --stretch 1.08 --transform fft --output fft_out.wav  
pvx voc source.wav --stretch 1.08 --transform dct --output dct_out.wav
```

Explanation - Compares complex-phase Fourier path against real cosine basis path.

Before/After - Before: one baseline source. - After: two different transform-character outputs.

Parameters that matter most - --transform - identical stretch/pitch settings for fair comparison

Artifacts to listen for - phase character and transient envelope differences

11.43 CZT for Non-Power-of-Two Frame Setup

Command

```
pvx voc source.wav --transform czt --n-fft 1536 --win-length 1536 --hop-size 384 --stretch 1.1 --output  
czt_1536.wav
```

Explanation - Uses chirp-z backend for a non-power-of-two frame strategy.

Before/After - Before: default FFT path. - After: alternate numerical behavior for same macro task.

Parameters that matter most - --transform czt - --n-fft, --win-length, --hop-size

Artifacts to listen for - subtle high-frequency texture differences vs FFT

11.44 Full Mastering Chain in One Render

Command

```
pvx voc mix.wav --stretch 1.03 --target-lufs -14 --compressor-threshold-db -18 --compressor-ratio 2.2 --limiter-threshold -0.9 --soft-clip-level 0.96 --output mix_mastered.wav
```

Explanation - Demonstrates integrated DSP + mastering path for one-step render.

Before/After - Before: un-leveled output with higher crest variance. - After: controlled loudness with safety limiting/clipping.

Parameters that matter most - --target-lufs - compressor and limiter thresholds - --soft-clip-level

Artifacts to listen for - pumping, flattened transients, clip harshness

11.45 Safety Limiter + Hard Clip Guard

Command

```
pvx voc loud_source.wav --stretch 1.0 --limiter-threshold -1.0 --hard-clip-level 0.98 --output loud_safe.wav
```

Explanation - Adds a limiter stage followed by a hard guard ceiling.

Before/After - Before: possible overs and intersample spikes. - After: bounded peaks with predictable ceiling.

Parameters that matter most - --limiter-threshold - --hard-clip-level

Artifacts to listen for - transient flattening if limiter is overworked

11.46 Plan-Only Diagnostic (No Render)

Command

```
pvx voc source.wav --auto-profile --auto-transform --manifest-json plan.json --dry-run --explain-plan
```

Explanation - Produces resolved strategy without writing output audio.

Before/After - Before: no visibility into automatic decision path. - After: JSON and console explanation of chosen settings.

Parameters that matter most - --auto-profile - --auto-transform - --dry-run

Artifacts to listen for - n/a (planning command)

11.47 Checkpointed Long Render Workflow

Command

```
pvx voc long_source.wav --target-duration 1200 --auto-segment-seconds 0.5 --checkpoint-dir .pvx_ckpt --output long_out.wav
pvx voc long_source.wav --target-duration 1200 --auto-segment-seconds 0.5 --checkpoint-dir .pvx_ckpt --resume --output long_out.wav
```

Explanation - First command starts segmented checkpointed render; second resumes if interrupted.

Before/After - Before: full rerender after interruption. - After: resumed render from saved chunk state.

Parameters that matter most - --checkpoint-dir - --auto-segment-seconds - --resume

Artifacts to listen for - chunk-boundary discontinuities if segmenting is too coarse

11.48 Multi-Tool Pipe with Streaming I/O

Command

```
pvx voc input.wav --stretch 1.1 --stdout \
| pvx denoise - --reduction-db 8 --stdout \
| pvx deverb - --strength 0.35 --output cleaned_chain.wav
```

Explanation - Streams PCM data through multiple tools in one line.

Before/After - Before: separate intermediate files. - After: one-pass chained processing.

Parameters that matter most - upstream --stdout - downstream - stdin input

Artifacts to listen for - cumulative artifact stacking from multiple stages

11.49 Batch Tree Processing with find + xargs

Command

```
find sessions -name "*.wav" -print0 | xargs -0 -I{} pvx voc "{}" --stretch 1.05 --output-dir renders --suffix _x105 --overwrite
```

Explanation - Applies a consistent transform to an entire tree of WAV files.

Before/After - Before: manual per-file invocation. - After: reproducible batch render set.

Parameters that matter most - stable suffix conventions - overwrite policy

Artifacts to listen for - content classes that need per-file presets instead of one global recipe

11.50 Vocal Retune by Scale/Root (pvxretune)

Command

```
pvx retune vocal.wav --scale minor --root A --a4-reference-hz 432 --f0-min 70 --f0-max 1000 --output-dir out --suffix _retune_amin

# Optional: anchor to an explicit root fundamental instead of note class
\index{Optional anchor to an explicit root fundamental instead of note class}
pvx retune vocal.wav --scale major --root-hz 261.6256 --output vocal_root_c4.wav

# Optional: ask pvx to estimate and use a root fundamental from the input
\index{Optional ask pvx to estimate and use a root fundamental from the input}
pvx retune vocal.wav --scale minor --recommend-root --output vocal_auto_root.wav
```

Explanation - Tracks monophonic F0 and snaps segments to requested tonal set.

Before/After - Before: natural pitch drift. - After: scale-constrained melody line.

Parameters that matter most - --scale, --root - --root-hz, --recommend-root - --a4-reference-hz - --f0-min, --f0-max

Artifacts to listen for - octave errors on noisy consonants

11.51 Harmonize Then Widen in One Pipeline

Command

```
pvx harmonize lead.wav --intervals 0,4,7 --gains 1,0.75,0.65 --stdout \  
| pvx unison - --voices 5 --detune-cents 10 --width 1.0 --output harmonized_wide.wav
```

Explanation - Builds harmony stack first, then adds unison detuned width.

Before/After - Before: single melodic line. - After: harmonized and spatially widened texture.

Parameters that matter most - harmonic intervals/gains - unison voice count and detune

Artifacts to listen for - dense masking in low-mid region

11.52 Ambient Macro-Chain (Stretch + Cleanup + Loudness)

Command

```
pvx voc seed.wav --preset extreme_ambient --target-duration 600 --stdout \  
| pvx denoise - --reduction-db 4 --smooth 9 --stdout \  
| pvx deverb - --strength 0.25 --stdout \  
| pvx voc - --stretch 1.0 --target-lufs -18 --output ambient_final.wav
```

Explanation - Demonstrates long-form ambient generation plus light restoration and final loudness targeting.

Before/After - Before: short seed sample. - After: long-form ambient render with controlled output level.

Parameters that matter most - ambient preset stretch controls - cleanup strength values - final --target-lufs

Artifacts to listen for - accumulated haze from over-processing

11.53 Broadcast-Style Speech Finishing (Stretch + Loudness + Safety)

Command

```
pvx voc narration.wav --preset vocal_studio --stretch 1.08 --target-lufs -16 --limiter-threshold -1.0 --soft  
-clip-level 0.98 --hard-clip-level 0.995 --output narration_finished.wav
```

Explanation - Applies subtle timing expansion and a conservative mastering finish in one pass.

Before/After - Before: untreated narration. - After: slightly slower pacing with controlled final loudness and peaks.

Parameters that matter most - `--stretch` - `--target-lufs` - limiter/clip safety thresholds

Artifacts to listen for - pumping from aggressive limiting - clipped consonant edges if clip levels are too low

11.54 Extreme Stretch with Checkpointed Recovery

Command

```
pvx voc short_source.wav --preset extreme_ambient --target-duration 900 --auto-segment-seconds 0.25 --  
checkpoint-dir checkpoints/short_source --output short_source_15min.wav
```

Explanation - Runs a very large render with small recoverable segments and persistent checkpoints.

Before/After - Before: short source gesture. - After: long ambient evolution with resumable rendering.

Parameters that matter most - `--target-duration` - `--auto-segment-seconds` - `--checkpoint-dir`

Artifacts to listen for - timbral drift from repeated stage boundaries

11.55 Room Recording Rehab (De-Reverb Then Stretch)

Command

```
pvx deverb room_take.wav --strength 0.45 --stdout \  
| pvx voc - --preset vocal_studio --stretch 1.15 --output room_take_rehab.wav
```

Explanation - Reduces reverberant smear first, then performs moderate timing stretch on cleaner material.

Before/After - Before: dense room reflections. - After: clearer direct signal with slower timing.

Parameters that matter most - `pvx deverb --strength` - `pvx voc --stretch`

Artifacts to listen for - metallic tails from over-dereverb - new blur if stretch is too large

11.56 Scale-Quantized Retune Followed by Unison Widening

Command

```
pvx retune lead.wav --scale major --root D --output-dir out --suffix _retuned \  
&& pvx unison out/lead_retuned.wav --voices 5 --detune-cents 9 --width 0.9 --output-dir out --suffix  
_retuned_unison
```

Explanation - First constrains melody to key, then adds controlled chorus/unison width.

Before/After - Before: dry monophonic lead. - After: tuned and widened lead texture.

Parameters that matter most - retune scale/root - unison voices and detune

Artifacts to listen for - tuning jumps on weak F0 regions - phasey image when width/detune are too high

11.57 Morph Then Freeze for Hybrid Pads

Command

```
pvx morph bells.wav choir.wav --alpha 0.5 --output bells_choir_morph.wav \  
&& pvx freeze bells_choir_morph.wav --freeze-time 0.8 --duration 20 --output-dir out --suffix _frozen
```

Explanation - Builds a hybrid timbre first, then captures a stable harmonic moment as a sustained pad.

Before/After - Before: two independent sources. - After: one blended frozen harmonic texture.

Parameters that matter most - morph --alpha - freeze capture time and duration

Artifacts to listen for - static texture if freeze time is poorly chosen

11.58 CUDA Attempt with Deterministic CPU Fallback Workflow

Command

```
pvx voc mix.wav --device cuda --stretch 1.12 --output mix_cuda.wav || pvx voc mix.wav --device cpu --stretch  
1.12 --output mix_cpu.wav
```

Explanation - Tries GPU first, then falls back to CPU using the same core settings.

Before/After - Before: original mix timing. - After: consistent stretched output regardless of device path.

Parameters that matter most - --device - stretch/quality controls shared between both paths

Artifacts to listen for - no expected quality change between CPU and CUDA modes

11.59 5.1/Multichannel Reference-Lock Stretch

Command

```
pvx voc surround.wav --stretch 1.1 --stereo-mode ref_channel_lock --ref-channel 2 --coherence-strength 0.95
--output surround_lock.wav
```

Explanation - Anchors phase evolution to a chosen reference channel to reduce inter-channel drift.

Before/After - Before: original multichannel phase relationships. - After: stretched output with better preserved channel coupling.

Parameters that matter most - `--stereo-mode ref_channel_lock` - `--ref-channel` - `--coherence-strength`

Artifacts to listen for - narrowed width if coherence is overly strong for decorrelated ambience

11.60 Mid/Side Lock with Formant-Preserving Pitch Shift

Command

```
pvx voc stereo_vocal.wav --stretch 1.0 --pitch 2 --pitch-mode formant-preserving --stereo-mode mid_side_lock
--coherence-strength 0.9 --output stereo_vocal_up2_lock.wav
```

Explanation - Combines vocal formant handling with stereo image preservation.

Before/After - Before: natural stereo vocal. - After: upshifted vocal with tighter image stability.

Parameters that matter most - `--pitch`, `--pitch-mode` - `--stereo-mode`, `--coherence-strength`

Artifacts to listen for - center blur if coherence strength is too low

11.61 Confidence-Gated External Pitch Control via Pipe

Command

```
pvx pitch-track A.wav --output - | pvx voc B.wav --control-stdin --pitch-conf-min 0.75 --pitch-lowconf-mode
hold --pitch-map-crossfade-ms 20 --output B_pitch_follow.wav
```

Explanation - Tracks pitch trajectory from A.wav, then applies only high-confidence pitch control to B.wav.

Before/After - Before: independent B.wav pitch contour. - After: B.wav follows trusted sections of A.wav pitch shape.

Parameters that matter most - `--pitch-conf-min` - `--pitch-lowconf-mode` - `--pitch-map-crossfade-ms`

Artifacts to listen for - contour lag if crossfade is too long - pitch jitter if confidence threshold is too low

11.62 Microtonal Just-Ratio Map Playback

Command

```
pvx voc mono_line.wav --pitch-map maps/just_ratios.csv --pitch-map-crossfade-ms 15 --output mono_line_ji.wav
```

Explanation - Applies time-varying just-ratio pitch targets from CSV.

Before/After - Before: fixed tuning. - After: trajectory-constrained microtonal tuning.

Parameters that matter most - --pitch-map - --pitch-map-crossfade-ms

Artifacts to listen for - step audibility if map density is too sparse

11.63 Speech Clarity Pass (Identity Lock + Conservative Window/Hop)

Command

```
pvx voc lecture.wav --stretch 1.18 --phase-locking identity --n-fft 2048 --hop-size 256 --output lecture_clear_stretch.wav
```

Explanation - Uses a quality-focused speech profile manually tuned for lower phase blur.

Before/After - Before: original lecture pace. - After: slower and clearer for note-taking.

Parameters that matter most - --phase-locking identity - --n-fft, --hop-size

Artifacts to listen for - mild transient smear if hop becomes too large

11.64 Transform Research A/B (FFT vs Hartley)

Command

```
pvx voc source.wav --stretch 1.1 --transform fft --output source_fft.wav \
&& pvx voc source.wav --stretch 1.1 --transform hartley --output source_hartley.wav
```

Explanation - Produces matched renders to compare transform-domain behavior.

Before/After - Before: one reference source. - After: two algorithmic variants for blind listening tests.

Parameters that matter most - --transform - all other parameters held constant

Artifacts to listen for - fine-grain differences in high-frequency texture

11.65 Multi-Resolution Fusion With Explicit Weights

Command

```
pvx voc texture.wav --stretch 1.4 --multires-fusion --multires-ffts 1024,2048,4096 --multires-weights 0.2,0.35,0.45 --output texture_multires_weighted.wav
```

Explanation - Fuses multiple FFT scales with explicit weighting rather than implicit defaults.

Before/After - Before: single-scale spectral processing. - After: blended resolution emphasizing low-frequency stability.

Parameters that matter most - --multires-ffts - --multires-weights

Artifacts to listen for - diffuse attacks if large-FFT weight is too dominant

11.66 Conservative Field-Restoration Chain

Command

```
pvx denoise field.wav --reduction-db 5 --smooth 7 --stdout \  
| pvx deverb - --strength 0.3 --stdout \  
| pvx voc - --stretch 1.0 --target-lufs -20 --limiter-threshold -1.5 --output field_restored.wav
```

Explanation - Applies light denoise + dereverb and ends with conservative loudness normalization.

Before/After - Before: noisy reverberant field capture. - After: cleaner and level-aligned output.

Parameters that matter most - denoise reduction amount - dereverb strength - final loudness/limiter settings

Artifacts to listen for - chirpy residuals from over-denoise

11.67 Reproducible Ambient Session With Manifest Logging

Command

```
pvx voc seed.wav --preset extreme_ambient --target-duration 600 --manifest-json sessions/ambient_manifest.  
json --manifest-append --output ambient_take.wav
```

Explanation - Writes run metadata into a JSON manifest for repeatable, auditable experiments.

Before/After - Before: short seed sample. - After: long ambient render plus machine-readable run record.

Parameters that matter most - --manifest-json - --manifest-append - chosen preset/stretch strategy

Artifacts to listen for - broad smearing from overly aggressive stretch factors

11.68 Managed One-Line Chain (No Manual Pipe Wiring)

Command

```
pvx chain source.wav --pipeline "voc --stretch 1.2 | formant --mode preserve --formant-shift-ratio 1.0" --  
output source_chain.wav
```

Explanation - Runs serial stages with managed intermediate files. - Useful when you want multi-stage processing without writing long shell pipelines.

Before/After - Before: single source file. - After: time-stretched + formant-preserved render in one managed command.

Parameters that matter most - --pipeline stage order and settings - per-stage quality controls (inside the pipeline string)

Artifacts to listen for - cumulative coloration when too many strong stages are chained

11.69 Chunked Stream Wrapper + Output Policy Controls

Command


```
pvx stream source.wav --output source_stream.wav --chunk-seconds 0.2 --time-stretch 2.0 --bit-depth 24 --
dither tpdf --dither-seed 7 --metadata-policy sidecar
```

Explanation - Uses `pvx stream` stateful mode (default) for chunked long renders with continuity context.
 - Adds deterministic output policy controls and metadata sidecar emission.

Before/After - Before: original source. - After: chunked long-form render with explicit output-depth/dither policy and sidecar metadata.

Parameters that matter most - `--chunk-seconds` - `--bit-depth` - `--dither`, `--dither-seed` - `--metadata-policy`

Artifacts to listen for - over-short chunks may increase boundary coloration

Legacy compatibility mode:

```
pvx stream source.wav --mode wrapper --output source_stream_wrapper.wav --chunk-seconds 0.2 --time-stretch
2.0
```

11.70 Dynamic Stretch via Direct CSV Flag Input

Command

```
pvx voc source.wav --stretch controls/stretch.csv --interp linear --output source_dyn_stretch.wav
```

Example `controls/stretch.csv`

```
time_sec,value
0.0,1.0
1.0,1.2
2.0,1.6
3.0,2.0
```

Explanation - Uses a time-varying control-rate signal directly on `--stretch` (no legacy `--pitch-map` wrapper needed). - Interpolates control points with `--interp linear` (default mode).

Before/After - Before: fixed global stretch ratio. - After: continuously changing stretch trajectory over time.

Parameters that matter most - `--stretch <csv/json>` - `--interp` - `--order` (if polynomial)

Artifacts to listen for - excessive local tempo curvature from over-aggressive control points - boundary coloration if your control points force rapid ratio changes

11.71 Dynamic Pitch Ratio via JSON + Polynomial Interpolation

Command

```
pvx voc vocal.wav --pitch-shift-ratio controls/pitch_curve.json --interp polynomial --order 3 --output
vocal_dyn_pitch.wav
```

Example controls/pitch_curve.json

```
{
  "points": [
    {"time_sec": 0.0, "value": 1.0},
    {"time_sec": 0.8, "value": 1.122462048},
    {"time_sec": 1.6, "value": 1.334839854},
    {"time_sec": 2.4, "value": 1.0}
  ]
}
```

Explanation - Drives pitch from a JSON control-rate signal attached directly to `--pitch-shift-ratio`.
`--interp polynomial --order 3` fits a smooth polynomial trajectory through points. `--order` supports any integer ≥ 1 ; the effective polynomial degree is capped to `min(order, control_points-1)`.

Before/After - Before: static transposition. - After: continuous pitch contour over time.

Parameters that matter most - `--pitch-shift-ratio <csv/json>` - `--interp` - `--order`

Artifacts to listen for - over/undershoot from high-order polynomial fitting - formant drift if large excursions are used without formant-preserving mode

Interpolation order quick visual:

	Mode/order	Graph
none		
linear		
cubic		
exponential		
s_curve		
smootherstep		
polynomial --order 1		
polynomial --order 2		
polynomial --order 3		
polynomial --order 5		

Related function charts for morphing and mastering:

Function family	Graph
Morph blend magnitude curves	
Mask exponent curves (<code>--mask-exponent</code>)	
Phase mix curve (<code>--phase-mix</code>)	
Dynamics transfer curves	
Soft clip transfer functions	

11.72 Stairstep (Sample-and-Hold) Control Mode

Command

```
pvx voc phrase.wav --stretch controls/step_stretch.csv --interp none --output phrase_step_control.wav
```

Example controls/step_stretch.csv

```
start_sec,end_sec,value
0.0,0.5,1.0
0.5,1.0,1.25
1.0,1.5,0.9
1.5,2.0,1.1
```

Explanation - Uses staircase control (`--interp none`) with piecewise constant parameter regions. - Best when you explicitly want quantized timeline regions.

Before/After - Before: smooth or fixed trajectory. - After: discrete region-based time changes.

Parameters that matter most - `--interp none` - segment boundaries - control value

Artifacts to listen for - audible discontinuities if regions are too short or jumps are too large

11.73 Dynamic Analysis Resolution (N-FFT + Hop Size Maps)

Command

```
pvx voc source.wav --n-fft controls/nfft.csv --hop-size controls/hop.csv --stretch 1.2 --output
source_dyn_resolution.wav
```

Example controls/nfft.csv

```
time_sec,value
0.0,1024
2.0,2048
4.0,4096
```

Example controls/hop.csv

```
time_sec,value
0.0,128
2.0,256
4.0,512
```

Explanation - Time-varying spectral resolution can trade transient precision and tonal stability across song sections. - Useful for research workflows where you want section-dependent analysis behavior.

Before/After - Before: one fixed analysis resolution for the whole render. - After: section-adaptive STFT resolution.

Parameters that matter most - `--n-fft <csv/json>` - `--hop-size <csv/json>` - `--win-length` (static or dynamic)

Artifacts to listen for - timbral shifts at parameter-transition boundaries - mismatched hop/window trajectories causing roughness

11.74 Stateful Stream Mode with Dynamic Stretch CSV

Command

```
pvx stream input.wav --output output_stream.wav --chunk-seconds 0.2 --stretch controls/stretch.csv --interp linear
```

Example controls/stretch.csv

```
time_sec,value
0.0,1.0
3.0,1.4
6.0,2.0
```

Explanation - Uses chunked stateful stream processing with the same control-file interface as `pvx voc`.
 - The stretch value is sampled from the interpolated trajectory for each processed chunk.

Before/After - Before: one global stretch factor. - After: evolving stretch profile over time, while staying in stream mode.

Parameters that matter most - `--chunk-seconds` - `--stretch <csv/json>` - `--interp`

Artifacts to listen for - chunk-boundary texture changes if `--chunk-seconds` is too short - abrupt pacing changes from steep control curves

11.75 Generate a Stretch Envelope (Function Stream)

Command

```
pvx envelope --mode adsr --duration 8 --rate 20 --attack-sec 0.2 --decay-sec 0.6 --sustain 1.1 --release-sec 1.0 --key stretch --output controls/stretch_env.csv
```

Periodic LFO variants (same tool, `pvx lfo` alias)

```
pvx lfo --wave sine --duration 12 --cycles 6 --center 1.0 --amplitude 0.25 --key stretch --output controls/stretch_sine.csv
pvx lfo --wave triangle --duration 8 --frequency-hz 0.5 --center 1.0 --amplitude 0.2 --key stretch --output controls/stretch_triangle.csv
pvx lfo --wave square --duration 8 --frequency-hz 2.0 --center 1.0 --amplitude 0.3 --duty-cycle 0.35 --key pitch_ratio --output controls/pitch_square.csv
pvx lfo --wave saw_up --duration 8 --frequency-hz 0.5 --center 1.0 --amplitude 0.2 --key stretch --output controls/stretch_saw_up.csv
```

Explanation - Generates a control-rate trajectory directly from the command line for later reuse in `pvx voc`, `pvx tvfilter`, and other map-driven tools. - Keeps control authoring reproducible and scriptable in shell pipelines.

Before/After - Before: manual spreadsheet editing for every map revision. - After: deterministic, one-command control-map generation.

Parameters that matter most - `--mode` - `--wave` (alias for `--mode`) - `--duration` - `--rate` - `--frequency-hz` or `--cycles` for periodic shapes - `--center` and `--amplitude` for periodic offsets/depth - `--key`

Artifacts to listen for - abrupt transitions if envelope segment times are too short - over-aggressive modulation ranges for stretch/pitch controls

11.76 Reshape and Densify a Control Map

Command

```
pvx reshape controls/stretch_env.csv --key stretch --operation resample --rate 50 --interp polynomial --
order 5 --output controls/stretch_env_dense.csv
```

Explanation - Resamples sparse control points into a denser trajectory for smoother frame-level sampling in downstream tools. - Supports additional transform operations (**scale**, **offset**, **clip**, **normalize**, **smooth**, **time-scale**, and others).

Before/After - Before: coarse control points with larger step size. - After: denser curve with smoother interpolation behavior.

Parameters that matter most - **--operation** - **--rate** - **--interp** - **--order**

Artifacts to listen for - overshoot if high-order polynomial interpolation is used on sparse/irregular points - excessive smoothing reducing intended modulation detail

11.77 Run PVC Parity Benchmark Gate

Command

```
python3 benchmarks/run_pvc_parity.py --quick --out-dir benchmarks/out_pvc_parity --baseline benchmarks/
baseline_pvc_parity.json --gate --gate-tolerance 0.20
```

Explanation - Runs deterministic parity scenarios for PVC-inspired phase 3-7 operators, then compares metrics to a committed baseline. - Useful for regression gating in local workflows and continuous integration (CI).

Before/After - Before: no focused parity regression coverage for the new operator family. - After: repeatable identity/effect scenario checks with a hard pass/fail gate.

Parameters that matter most - **--baseline** - **--gate** - **--gate-tolerance** - **--quick**

Artifacts to listen for - identity-case drift (unexpected coloration when effect depth should be near-neutral) - unstable runtime or metric jumps between revisions

11.78 Persist STFT Analysis Artifact (PVXAN)

Command

```
pvx analysis create input.wav --output input.pvxan.npz --n-fft 4096 --win-length 4096 --hop-size 256 --
window hann
pvx analysis inspect input.pvxan.npz
```

Explanation - Saves a reusable short-time Fourier transform (STFT) analysis artifact so you can inspect or re-use spectral analysis across multiple downstream workflows. - Useful when iterating on response design, parity checks, and research traces without re-running the same front-end analysis assumptions manually each time.

Before/After - Before: analysis lives only in-memory during a single command invocation. - After: analysis is persisted as a versionable artifact (**.pvxan.npz**) you can inspect and pass to other tools.

Parameters that matter most - **--n-fft** - **--win-length** - **--hop-size** - **--window**

Artifacts to listen for - not an audio render; inspect analysis metadata consistency (sample rate, frame count, transform/window choices) - mismatch between analysis resolution and intended downstream response usage

11.79 Derive and Inspect a Reusable Response Artifact (PVXRF)

Command

```
pvx response create input.pvxan.npz --output input.pvxrf.npz --method median --phase-mode mean --normalize
peak --smoothing-bins 3
pvx response inspect input.pvxrf.npz
```

Explanation - Builds a reusable frequency-response profile from a persisted analysis artifact. - This response can drive `pvx filter`, `pvx tvfilter`, `pvx noisefilter`, `pvx bandamp`, and `pvx spec-compander`.

Before/After - Before: no explicit reusable spectral profile for cross-session reuse. - After: deterministic response artifact (`.pvxrf.npz`) with inspectable metadata.

Parameters that matter most - `--method` - `--phase-mode` - `--normalize` - `--smoothing-bins`

Artifacts to listen for - not an audio render; watch for overly jagged responses if smoothing is too low - flattened response detail if smoothing is too high

11.80 Response-Profile Denoising with `pvx noisefilter`

Command

```
pvx analysis create roomtone.wav --output roomtone.pvxan.npz
pvx response create roomtone.pvxan.npz --output roomtone.pvxrf.npz --method median --normalize peak
pvx noisefilter dialog.wav --response roomtone.pvxrf.npz --noise-floor 1.2 --response-mix 1.0 --dry-mix 0.05
--output dialog_noisefilter.wav
```

Explanation - Uses external room-tone analysis to build a targeted noise profile, then applies response-referenced suppression to the noisy program material. - This is often more controlled than blind denoising when consistent environmental noise is present.

Before/After - Before: broadband hiss/hum/air conditioning texture over dialogue. - After: reduced steady-state noise with dialogue content better preserved.

Parameters that matter most - `--noise-floor` - `--response-mix` - `--dry-mix` - quality of `roomtone.wav`

Artifacts to listen for - speech dulling from over-suppression - tonal holes if the room-tone profile does not match the program noise

11.81 Time-Varying Spectral Filtering with `pvx tvfilter`

Command

```
pvx tvfilter pad.wav --response color_profile.pvxrf.npz --tv-map controls/response_mix.csv --tv-key
response_mix --tv-interp s_curve --tv-order 3 --output pad_tvfilter.wav
```

Example `controls/response_mix.csv`

```
time_sec,response_mix
0.0,0.10
4.0,0.65
8.0,1.00
12.0,0.35
```

Explanation - Morphs response influence over time so timbre evolves dynamically instead of staying static for the full render. - Useful for long-form pads, scenes, and texture design where one fixed filter shape is too rigid.

Before/After - Before: static timbre profile. - After: time-varying spectral color trajectory controlled by a map.

Parameters that matter most - `--tv-map` - `--tv-key` - `--tv-interp` - `--response-mix`

Artifacts to listen for - abrupt tonal jumps if control map points are sparse and interpolation is too sharp - over-filtered sections when response mix is pinned near 1.0 for too long

11.82 Spectral Peak Emphasis with `pvx bandamp`

Command

```
pvx bandamp source.wav --response profile.pvxrf.npz --band-gain-db 8 --peak-count 10 --band-width-bins 6 --output source_bandamp.wav
```

Explanation - Detects high-energy response regions and boosts corresponding bands to emphasize characteristic partial groups. - Helpful for presence enhancement and stylized spectral contour exaggeration.

Before/After - Before: flatter spectral emphasis. - After: more pronounced response-defined band structure.

Parameters that matter most - `--band-gain-db` - `--peak-count` - `--band-width-bins` - response profile content

Artifacts to listen for - harshness if gain and peak count are too high - ringing coloration when boosted bands are too narrow

11.83 Response-Shaped Spectral Dynamics with `pvx spec-compander`

Command

```
pvx spec-compander mix.wav --response profile.pvxrf.npz --comp-threshold-db -18 --comp-ratio 2.5 --expand-ratio 1.3 --output mix_speccomp.wav
```

Explanation - Applies response-informed compression and expansion behavior in the spectral domain. - Useful for reshaping dynamic contrast in specific timbral regions rather than broad full-band dynamics only.

Before/After - Before: less controlled spectral dynamic contour. - After: response-informed dynamic shaping that can clarify or stylize spectral balance.

Parameters that matter most - `--comp-threshold-db` - `--comp-ratio` - `--expand-ratio` - response

Artifacts to listen for - pumping/breathing if ratios are aggressive - unstable ambience tails if threshold is too low

11.84 Ring Modulation Fundamentals with pvx ring

Command

```
pvx ring synth.wav --frequency-hz 43 --depth 0.9 --mix 0.8 --feedback 0.15 --output synth_ring.wav
```

Explanation - Applies controllable ring modulation with optional feedback memory for sideband-rich timbre design. - Works well for metallic, bell-like, and animated tremolo-adjacent transformations.

Before/After - Before: original oscillator or instrument tone. - After: sideband-rich ring-modulated texture with controllable wet/dry blend.

Parameters that matter most - --frequency-hz - --depth - --mix - --feedback

Artifacts to listen for - harsh alias-like texture when carrier frequency/depth are extreme - runaway coloration if feedback is too high

11.85 Resonant Ring Filtering with pvx ringfilter

Command

```
pvx ringfilter input.wav --frequency-hz 60 --depth 0.8 --mix 0.9 --resonance-hz 1200 --resonance-q 9 --  
resonance-mix 0.4 --resonance-decay 0.2 --output input_ringfilter.wav
```

Explanation - Chains ring modulation with a resonant peak stage for tuned, animated spectral coloration. - Useful for vocal/synth character design where simple ring modulation alone is too blunt.

Before/After - Before: dry source or plain ring modulation. - After: ringed tone plus focused resonant emphasis and decay behavior.

Parameters that matter most - --resonance-hz - --resonance-q - --resonance-mix - --resonance-decay

Artifacts to listen for - whistling resonances if --resonance-q is too high - tonal masking if resonance center sits on dominant formants/partials unintentionally

11.86 Harmonic Chord Mapping with pvx chordmapper

Command

```
pvx chordmapper vocal.wav --root-hz 220 --chord min7 --strength 0.75 --tolerance-cents 35 --boost-db 6 --  
attenuation 0.45 --output vocal_chordmapped.wav
```

Explanation - Pulls spectral emphasis toward chord-constrained pitch-class regions relative to a chosen root. - Good for harmonic re-coloring and pseudo-harmonization textures without explicit multi-voice synthesis.

Before/After - Before: unconstrained harmonic emphasis. - After: chord-guided spectral emphasis with out-of-chord attenuation.

Parameters that matter most - --root-hz - --chord - --strength - --tolerance-cents

Artifacts to listen for - unnatural pitch-class bias if tolerance is too narrow - muffling when attenuation is too high for non-chord content

11.87 Inharmonic Spectral Warping with `pvx inharmonator`

Command

```
pvx inharmonator bell.wav --inharmonic-f0-hz 220 --inharmonicity 0.0002 --inharmonic-mix 1.0 --dry-mix 0.1 --output bell_inharm.wav
```

Explanation - Applies controlled inharmonic spectral remapping inspired by stiff-string/inharmonic partial spacing models. - Useful for bell, plate, metallic, and abstract hybrid timbres.

Before/After - Before: harmonic or near-harmonic partial spacing. - After: increasingly inharmonic overtone structure with controllable blend.

Parameters that matter most - `--inharmonic-f0-hz` - `--inharmonicity` - `--inharmonic-mix` - `--dry-mix`

Artifacts to listen for - excessive dissonance if inharmonicity is too high - loss of note center perception at high wet mix on dense sources

11.88 Diagnose Environment and Install Issues (`pvx doctor`)

Command

```
pvx doctor
```

Explanation - Runs launch-readiness diagnostics for virtual environment, path environment variable (PATH), and optional dependency visibility. - Prints concrete remediation commands when warnings are found.

Before/After - Before: trial-and-error debugging when `pvx` commands fail or optional features are missing. - After: one-command diagnosis with direct fix suggestions.

Parameters that matter most - `--strict` - `--json`

Artifacts to listen for - not an audio render; inspect warnings list for missing PATH entries and dependency gaps

11.89 Minimal Public-Demo Sequence (`pvx quickstart`)

Command

```
pvx quickstart input.wav --output output.wav
```

Explanation - Prints a concise launch script that chains diagnostics, safe first render, transform guidance, examples, and smoke test. - Useful when preparing demos and avoiding ad-hoc command selection.

Before/After - Before: inconsistent launch flow and missed preflight checks. - After: repeatable launch sequence suitable for public demos.

Parameters that matter most - positional input path - `--output` - `--material`

Artifacts to listen for - not an audio render; this is command-sequence generation

11.90 Quality-First First Pass (`pvx safe`)

Command

```
pvx safe input.wav --material mix --output input_safe.wav
```

Explanation - Wraps `pvx voc` with conservative defaults (`identity` phase locking, hybrid transient handling, stereo coherence defaults) to reduce first-pass artifact risk. - Intended for “first render should not embarrass us” scenarios.

Before/After - Before: first render quality depends heavily on manual flag selection. - After: consistent conservative baseline output with one command.

Parameters that matter most - `--material` - passthrough `pvx voc` arguments (for example `--stretch`, `--pitch`) - `--overwrite`

Artifacts to listen for - reduced, but not eliminated, smear/phase artifacts on extreme settings - material-specific tuning may still be required for final delivery

11.91 Transform Availability and Selection (`pvx transforms`)

Command

```
pvx transforms
```

Explanation - Prints transform backend options (`fft`, `dft`, `czt`, `dct`, `dst`, `hartley`) with runtime availability and practical recommendations. - Clarifies when optional SciPy-backed transforms are unavailable.

Before/After - Before: uncertainty about whether non-FFT transforms are supported in the active runtime. - After: explicit transform availability and usage guidance in one place.

Parameters that matter most - `--json`

Artifacts to listen for - not an audio render; review backend availability and recommendation text

11.92 Fast End-to-End Health Check (`pvx smoke`)

Command

```
pvx smoke --output smoke_out.wav
```

Explanation - Generates a short synthetic input, runs a conservative `pvx voc` render, verifies output creation, and reports output metadata. - Useful for pre-demo confidence checks and basic continuous integration (CI) sanity gates.

Before/After - Before: no quick integrated health check for “does this build actually render audio.” - After: one-command smoke render with pass/fail signal.

Parameters that matter most - `--output` - `--duration` - `--sample-rate` - `--stretch` - `--pitch`

Artifacts to listen for - simple synthetic tone render; severe distortion/silence indicates environment/runtime problems

11.93 Deterministic AI Dataset Augmentation (`pvx augment`)

Command

```
pvx augment data/*.wav --output-dir aug_out --variants-per-input 4 --intent asr_robust --split 0.8,0.1,0.1
--seed 1337
```

Explanation - Generates reproducible augmentation variants per source file and writes machine-readable manifests for training pipelines. - Intended for automatic speech recognition (ASR), music information retrieval (MIR), and self-supervised learning (SSL) experiments.

Before/After - Before: ad-hoc augmentation scripts with weak reproducibility and incomplete provenance. - After: deterministic rendered variants plus JSON Lines (JSONL) and comma-separated values (CSV) manifests including split assignment and parameters.

Parameters that matter most - `--variants-per-input` - `--intent` - `--seed` - `--split` - `--grouping` / `--group-separator` - `--dry-run`

Artifacts to listen for - augmentation policy mismatch to task (too mild for contrastive learning, too aggressive for label-sensitive training) - split leakage if downstream pipeline ignores source-family grouping

11.94 Speaker-Balanced Split Assignment for Augmentation

Command

```
pvx augment data/*.wav --output-dir aug_speaker --variants-per-input 3 --intent asr_robust --split-mode
speaker_balanced --labels-csv labels.csv --seed 1337
```

Explanation - Uses speaker metadata to assign train/validation/test splits with better speaker balance. - Reduces cross-speaker skew and helps avoid accidental split leakage.

Before/After - Before: random split may under-represent some speakers in validation/test. - After: split assignment uses speaker-aware balancing constraints.

Parameters that matter most - `--split-mode speaker_balanced` - `--labels-csv` - `--split` - `--grouping`

Artifacts to listen for - metadata mismatch (wrong speaker IDs) causing unhelpful balancing - tiny corpora where strict balancing is statistically limited

11.95 Contrastive Paired Views for Self-Supervised Learning

Command

```
pvx augment data/*.wav --output-dir aug_ssl --variants-per-input 2 --intent ssl_contrastive --pair-mode
contrastive2 --seed 2026
```

Explanation - Generates paired views (`view a` / `view b`) per augmentation variant for contrastive learning pipelines. - Pair relationships are tracked in manifest fields (`pair_id`, `view_id`).

Before/After - Before: only independent single-view augment outputs. - After: structured paired views suitable for positive-pair training setups.

Parameters that matter most - `--pair-mode contrastive2` - `--intent ssl_contrastive` - `--seed`

Artifacts to listen for - over-aggressive pair divergence reducing semantic overlap - too-similar pairs limiting contrastive learning value

11.96 Resume + Append for Long Augmentation Runs

Command

```
pvx augment data/*.wav --output-dir aug_resume --variants-per-input 6 --intent mir_music --resume --append-manifest --workers 4 --seed 9001
```

Explanation - Resumes from existing outputs/manifests and appends new records without losing previous lineage metadata. - Useful for interrupted jobs and incremental dataset growth.

Before/After - Before: restarts can duplicate work or overwrite provenance. - After: incremental, resumable augmentation workflow.

Parameters that matter most - `--resume` - `--append-manifest` - `--workers` - `--seed`

Artifacts to listen for - stale manifest rows if files are manually moved/deleted outside tooling - mixed policy runs when appending across dissimilar settings

11.97 Manifest Validation and Merge

Command

```
pvx augment-manifest validate aug_run_a/augment_manifest.jsonl --strict
pvx augment-manifest merge aug_run_a/augment_manifest.jsonl aug_run_b/augment_manifest.jsonl --output-jsonl
aug_merged/manifest.jsonl --output-csv aug_merged/manifest.csv
pvx augment-manifest stats aug_merged/manifest.jsonl
```

Explanation - Validates required manifest fields, merges runs with de-duplication, and prints quick stats for auditability. - Supports production data curation workflows where multiple augmentation passes are combined.

Before/After - Before: ad-hoc manual joins and uncertain manifest integrity. - After: deterministic validation + merge + reporting in one command family.

Parameters that matter most - `validate` - `--strict` - `merge` - `--dedupe-by` - `stats`

Artifacts to listen for - duplicate lineage entries if dedupe key does not match your data model - invalid rows from older manifests lacking required fields

11.98 Augmentation Benchmark Gate

Command

```
python benchmarks/run_augment_bench.py --quick --out-dir benchmarks/out_augment --baseline benchmarks/baseline_augment_small.json --gate --gate-tolerance 0.30
```

Explanation - Runs deterministic augmentation benchmark checks and compares summary metrics against a baseline. - Intended for release and continuous integration (CI) regression gating.

Before/After - Before: no objective gate for augmentation drift. - After: reproducible benchmark reports with pass/fail criteria.

Parameters that matter most - `--baseline` - `--gate` - `--gate-tolerance` - `--quick`

Artifacts to listen for - split-balance drift - reduced augmentation diversity (low stretch/pitch standard deviation) - clipping/level anomalies in audit metrics

CHAPTER 12

Feature-Driven Sidechain Examples

This guide shows copy-paste command-line interface (CLI) recipes where one file (the guide) controls another file (the target) using tracked audio features.

- Guide file: `guide.wav` (feature source).
- Target file: `target.wav` (sound being transformed).
- Output folder in examples: `out/`.
- Most recipes use `pvx follow` (shortest path) and `--route` formulas.

Create an output folder once:

```
mkdir -p out maps
```

Quick built-in recipe discovery:

```
pvx follow --example
pvx follow --example all
pvx follow --example mfcc_flux
```

12.1 Quick Start

Basic pitch-to-stretch sidechain:

```
pvx follow guide.wav target.wav --emit pitch_to_stretch --pitch-conf-min 0.75 --output out/follow_basic.wav
```

Direct pitch-follow (guide pitch contour drives target pitch contour):

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 --output out/follow_pitch.wav
```

Track first, render later (decoupled analysis/render):

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output maps/guide_features.csv
pvx voc target.wav --pitch-map maps/guide_features.csv \
  --route stretch=pitch_ratio \
  --route pitch_ratio=const(1.0) \
  --output out/track_then_render.wav
```

12.2 Inspect Feature Columns Before Routing

Emit all tracked columns:

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 20 --output maps/guide_all.csv
```

Print the CSV header as one column per line:

```
head -n 1 maps/guide_all.csv | tr ',' '\n'
```

Useful columns include: - pitch/voicing: f0_hz, pitch_ratio, confidence, voicing_prob, pitch_stability
- spectral: spectral_centroid_hz, spectral_flux, spectral_flatness, rolloff_hz - dynamics: rms, rms_db, short_lufs_db, transientness, crest_factor_db - timbre vectors: mfcc_01..mfcc_N, mpeg7_*, mpeg7_audio_spectrum_envelope_01..10

12.3 Single-Feature Recipes

12.3.1 f0_hz -> pitch ratio

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(f0_hz,0.002272727,0.0) \  
--route pitch_ratio=clip(pitch_ratio,0.5,2.0) \  
--output out/f0_to_pitch.wav
```

12.3.2 pitch_ratio -> stretch (inverse motion)

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route stretch=inv(pitch_ratio) \  
--route stretch=clip(stretch,0.7,1.8) \  
--route pitch_ratio=const(1.0) \  
--output out/pitch_to_stretch_inverse.wav
```

12.3.3 confidence -> subtle pitch bend depth

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(confidence,0.25,0.9) \  
--route pitch_ratio=clip(pitch_ratio,0.85,1.15) \  
--output out/confidence_to_pitch.wav
```

12.3.4 voicing_prob -> stretch

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route stretch=affine(voicing_prob,0.6,0.8) \  
--route stretch=clip(stretch,0.8,1.4) \  
--route pitch_ratio=const(1.0) \  
--output out/voicing_to_stretch.wav
```

12.3.5 rms_norm -> stretch

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route stretch=affine(rms_norm,1.2,0.6) \  
--route stretch=clip(stretch,0.8,1.8) \  
--route pitch_ratio=const(1.0) \  
--output out/rms_to_stretch.wav
```

12.3.6 spectral_flux -> stretch

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route stretch=affine(spectral_flux,0.04,1.0) \
--route stretch=clip(stretch,0.85,1.6) \
--route pitch_ratio=const(1.0) \
--output out/flux_to_stretch.wav
```

12.3.7 onset_strength -> attack-reactive stretch

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route stretch=affine(onset_strength,0.05,0.95) \
--route stretch=clip(stretch,0.75,1.7) \
--route pitch_ratio=const(1.0) \
--output out/onset_to_stretch.wav
```

12.3.8 transientness -> transient-safe stretch variation

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route stretch=affine(transientness,-0.4,1.2) \
--route stretch=clip(stretch,0.8,1.2) \
--route pitch_ratio=const(1.0) \
--output out/transientness_to_stretch.wav
```

12.3.9 spectral_centroid_hz -> brightness-linked pitch

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(spectral_centroid_hz,0.00025,0.7) \
--route pitch_ratio=clip(pitch_ratio,0.7,1.6) \
--output out/centroid_to_pitch.wav
```

12.3.10 spectral_flatness -> noisy/tonal pitch response

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(spectral_flatness,-0.5,1.25) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.25) \
--output out/flatness_to_pitch.wav
```

12.3.11 harmonic_ratio -> harmonicity-linked pitch

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(harmonic_ratio,0.3,0.85) \
--route pitch_ratio=clip(pitch_ratio,0.85,1.2) \
--output out/harmonic_ratio_to_pitch.wav
```

12.3.12 inharmonicity -> anti-harmonic pitch detune

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(inharmonicity,-0.8,1.2) \
--route pitch_ratio=clip(pitch_ratio,0.75,1.2) \
--output out/inharmonicity_to_pitch.wav
```

12.3.13 rolloff_norm -> pitch

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(rolloff_norm,0.45,0.8) \  
--route pitch_ratio=clip(pitch_ratio,0.8,1.3) \  
--output out/rolloff_to_pitch.wav
```

12.3.14 pitch_stability -> micro-variation amount

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(pitch_stability,0.2,0.9) \  
--route pitch_ratio=clip(pitch_ratio,0.9,1.12) \  
--output out/pitch_stability_to_pitch.wav
```

12.4 Multi-Feature Recipes (Different Features -> Different Targets)

12.4.1 MFCC + MPEG-7 spectral flux

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 13 --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(mfcc_01,0.002,1.0) \  
--route pitch_ratio=clip(pitch_ratio,0.5,2.0) \  
--route stretch=affine(mpeg7_spectral_flux,0.05,1.0) \  
--route stretch=clip(stretch,0.85,1.6) \  
--output out/mfcc_flux_dual.wav
```

12.4.2 Formant + onset

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(formant_f1_hz,0.0016,0.2) \  
--route pitch_ratio=clip(pitch_ratio,0.7,1.5) \  
--route stretch=affine(onset_norm,-0.35,1.2) \  
--route stretch=clip(stretch,0.8,1.3) \  
--output out/formant_onset_dual.wav
```

12.4.3 Beat phase + downbeat phase

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(beat_phase,0.12,0.94) \  
--route pitch_ratio=clip(pitch_ratio,0.88,1.12) \  
--route stretch=affine(downbeat_phase,0.3,0.85) \  
--route stretch=clip(stretch,0.85,1.25) \  
--output out/beat_downbeat_dual.wav
```

12.4.4 Tempo + centroid

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \  
--route stretch=affine(tempo_bpm,0.003,0.6) \  
--route stretch=clip(stretch,0.8,1.4) \  
--route pitch_ratio=affine(centroid_norm,0.4,0.8) \  
--route pitch_ratio=clip(pitch_ratio,0.8,1.3) \  
--output out/tempo_centroid_dual.wav
```

12.4.5 Stereo cues: interaural level difference + interaural time difference


```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(ild_db,0.02,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.9,1.1) \
--route stretch=affine(itd_ms,0.12,1.0) \
--route stretch=clip(stretch,0.9,1.15) \
--output out/ild_itd_dual.wav
```

12.4.6 Noise-aware: hiss + hum

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route stretch=affine(hiss_ratio,-0.6,1.2) \
--route stretch=clip(stretch,0.8,1.2) \
--route pitch_ratio=affine(hum_60_ratio,-0.4,1.15) \
--route pitch_ratio=clip(pitch_ratio,0.9,1.2) \
--output out/noise_aware_dual.wav
```

12.4.7 Speech vs music probabilities

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route stretch=affine(speech_prob,0.5,0.8) \
--route stretch=clip(stretch,0.8,1.35) \
--route pitch_ratio=affine(music_prob,0.2,0.9) \
--route pitch_ratio=clip(pitch_ratio,0.9,1.2) \
--output out/speech_music_dual.wav
```

12.4.8 Formant 2 + spectral spread

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(formant_f2_hz,0.0008,0.5) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.5) \
--route stretch=affine(spectral_spread_hz,0.00008,0.85) \
--route stretch=clip(stretch,0.8,1.4) \
--output out/formant2_spread_dual.wav
```

12.4.9 Loudness proxy + transient mask

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route stretch=affine(short_lufs_db,0.02,1.4) \
--route stretch=clip(stretch,0.75,1.4) \
--route pitch_ratio=affine(transient_mask,0.1,0.95) \
--route pitch_ratio=clip(pitch_ratio,0.95,1.08) \
--output out/lufs_transientmask_dual.wav
```

12.4.10 MPEG-7 centroid + MPEG-7 spread

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(mpeg7_spectral_centroid_hz,0.00022,0.75) \
--route pitch_ratio=clip(pitch_ratio,0.7,1.5) \
--route stretch=affine(mpeg7_spectral_spread_hz,0.00008,0.9) \
--route stretch=clip(stretch,0.8,1.5) \
--output out/mpeg7_centroid_spread.wav
```

12.4.11 MPEG-7 attack-time + MFCC driver

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 13 --emit pitch_map --stretch 1.0 \  
--route stretch=affine(mpeg7_log_attack_time_s,0.4,1.0) \  
--route stretch=clip(stretch,0.85,1.3) \  
--route pitch_ratio=affine(mfcc_03,0.0015,1.0) \  
--route pitch_ratio=clip(pitch_ratio,0.8,1.25) \  
--output out/mpeg7_attack_mfcc.wav
```

12.5 Route Operator Patterns (const, inv, pow, mul, add, affine, clip)

12.5.1 Start from a feature, then scale and bias

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=pitch_ratio \  
--route pitch_ratio=mul(pitch_ratio,0.85) \  
--route pitch_ratio=add(pitch_ratio,0.15) \  
--route pitch_ratio=clip(pitch_ratio,0.8,1.4) \  
--output out/route_mul_add.wav
```

12.5.2 Exaggerate movement with power law

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route stretch=affine(flux_norm,1.0,0.0) \  
--route stretch=pow(stretch,1.8) \  
--route stretch=affine(stretch,0.9,0.7) \  
--route stretch=clip(stretch,0.8,1.8) \  
--route pitch_ratio=const(1.0) \  
--output out/route_pow_flux.wav
```

12.5.3 Invert and clamp

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route stretch=inv(rms_norm) \  
--route stretch=clip(stretch,0.8,1.5) \  
--route pitch_ratio=const(1.0) \  
--output out/route_inv_rms.wav
```

12.6 Feature-Vector Recipes (MFCC + MPEG-7)

12.6.1 MFCC second coefficient driver

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 13 --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(mfcc_02,0.0018,1.0) \  
--route pitch_ratio=clip(pitch_ratio,0.7,1.4) \  
--output out/mfcc02_pitch.wav
```

12.6.2 MFCC + MPEG-7 audio spectrum envelope band

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 13 --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(mfcc_04,0.0015,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.75,1.35) \
--route stretch=affine(mpeg7_audio_spectrum_envelope_03,0.08,1.0) \
--route stretch=clip(stretch,0.85,1.4) \
--output out/mfcc_envband_dual.wav
```

12.6.3 MPEG-7 spectral crest and decrease

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(mpeg7_spectral_crest,0.03,0.9) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.25) \
--route stretch=affine(mpeg7_spectral_decrease,-3.0,1.0) \
--route stretch=clip(stretch,0.85,1.2) \
--output out/mpeg7_crest_decrease.wav
```

12.6.4 Increase MFCC dimensionality to 20

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 20 --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(mfcc_10,0.0012,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.25) \
--route stretch=affine(mfcc_12,0.03,1.0) \
--route stretch=clip(stretch,0.9,1.2) \
--output out/mfcc10_mfcc12_dual.wav
```

12.6.5 Build custom vector features, then route them

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 20 --output maps/guide_vec.csv
python3 - <<'PY'
import csv
from pathlib import Path
src = Path('maps/guide_vec.csv')
dst = Path('maps/guide_vec_aug.csv')
with src.open() as f_in, dst.open('w', newline='') as f_out:
    r = csv.DictReader(f_in)
    fn = list(r.fieldnames or []) + ['mfcc_energy_1_4', 'mfcc_tilt_1_8']
    w = csv.DictWriter(f_out, fieldnames=fn)
    w.writeheader()
    for row in r:
        m1 = float(row.get('mfcc_01', 0.0) or 0.0)
        m2 = float(row.get('mfcc_02', 0.0) or 0.0)
        m3 = float(row.get('mfcc_03', 0.0) or 0.0)
        m4 = float(row.get('mfcc_04', 0.0) or 0.0)
        m8 = float(row.get('mfcc_08', 0.0) or 0.0)
        row['mfcc_energy_1_4'] = 0.25 * (m1 + m2 + m3 + m4)
        row['mfcc_tilt_1_8'] = m1 - m8
    w.writerow(row)
PY
pvx voc target.wav --pitch-map maps/guide_vec_aug.csv \
--route pitch_ratio=affine(mfcc_tilt_1_8,0.0018,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.75,1.4) \
--route stretch=affine(mfcc_energy_1_4,0.05,1.0) \
--route stretch=clip(stretch,0.85,1.4) \
--output out/vector_augmented.wav
```

12.7 Multiple Guide Files (A and B both influence target)

12.7.1 Blend pitch behavior from two guides

```

pvx pitch-track guide_A.wav --feature-set all --output maps/guide_A.csv
pvx pitch-track guide_B.wav --feature-set all --output maps/guide_B.csv
python3 - <<'PY'
import csv
from pathlib import Path

a = Path('maps/guide_A.csv')
b = Path('maps/guide_B.csv')
out = Path('maps/guide_blend.csv')
with a.open() as fa, b.open() as fb, out.open('w', newline='') as fo:
    ra = csv.DictReader(fa)
    rb = csv.DictReader(fb)
    rows_b = list(rb)
    fields = list(ra.fieldnames or [])
    if 'pitch_ratio' not in fields:
        fields.append('pitch_ratio')
    w = csv.DictWriter(fo, fieldnames=fields)
    w.writeheader()
    rows_b_len = max(1, len(rows_b))
    for i, row_a in enumerate(ra):
        row_b = rows_b[min(i, rows_b_len - 1)]
        pa = float(row_a.get('pitch_ratio', 1.0) or 1.0)
        pb = float(row_b.get('pitch_ratio', 1.0) or 1.0)
        row_a['pitch_ratio'] = 0.6 * pa + 0.4 * pb
    w.writerow(row_a)
PY
pvx voc target.wav --pitch-map maps/guide_blend.csv \
--route pitch_ratio=clip(pitch_ratio,0.7,1.5) \
--route stretch=const(1.0) \
--output out/two_guide_pitch_blend.wav

```

12.7.2 Guide A controls pitch, guide B controls stretch

```

pvx pitch-track guide_A.wav --feature-set all --output maps/guide_A.csv
pvx pitch-track guide_B.wav --feature-set all --output maps/guide_B.csv
python3 - <<'PY'
import csv
from pathlib import Path

a = Path('maps/guide_A.csv')
b = Path('maps/guide_B.csv')
out = Path('maps/guide_Apitch_Bstretch.csv')
with a.open() as fa, b.open() as fb, out.open('w', newline='') as fo:
    ra = csv.DictReader(fa)
    rb = csv.DictReader(fb)
    rows_b = list(rb)
    fields = list(dict.fromkeys((ra.fieldnames or []) + ['stretch']))
    w = csv.DictWriter(fo, fieldnames=fields)
    w.writeheader()
    rows_b_len = max(1, len(rows_b))
    for i, row_a in enumerate(ra):
        row_b = rows_b[min(i, rows_b_len - 1)]
        flux_b = float(row_b.get('spectral_flux', 0.0) or 0.0)
        row_a['stretch'] = max(0.8, min(1.5, 1.0 + 0.03 * flux_b))
    w.writerow(row_a)
PY
pvx voc target.wav --pitch-map maps/guide_Apitch_Bstretch.csv \
--route pitch_ratio=clip(pitch_ratio,0.75,1.35) \
--route stretch=clip(stretch,0.8,1.5) \
--output out/two_guide_split_roles.wav

```

12.8 Manual Pipe Variants

Pitch-track to pvx voc in one pipe:

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output - \
| pvx voc target.wav --control-stdin \
  --route pitch_ratio=affine(mfcc_01,0.002,1.0) \
  --route pitch_ratio=clip(pitch_ratio,0.5,2.0) \
  --route stretch=affine(mpeg7_spectral_flux,0.05,1.0) \
  --route stretch=clip(stretch,0.85,1.6) \
  --output out/manual_pipe_feature_follow.wav
```

Time-varying control map followed by denoise/deverb in one chain:

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
  --route stretch=affine(spectral_flux,0.04,1.0) \
  --route stretch=clip(stretch,0.9,1.4) \
  --route pitch_ratio=affine(mfcc_01,0.0015,1.0) \
  --route pitch_ratio=clip(pitch_ratio,0.85,1.2) \
  --output out/follow_stage.wav
pvx denoise out/follow_stage.wav --reduction-db 8 --output out/follow_stage_dn.wav
pvx deverb out/follow_stage_dn.wav --strength 0.35 --output out/follow_stage_dn_dr.wav
```

12.9 Reuse a Saved Feature Map for Fast Iteration

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output maps/guide_full.csv
pvx voc target.wav --pitch-map maps/guide_full.csv \
  --route pitch_ratio=affine(mfcc_01,0.002,1.0) \
  --route pitch_ratio=clip(pitch_ratio,0.5,2.0) \
  --route stretch=affine(spectral_flux,0.04,1.0) \
  --route stretch=clip(stretch,0.85,1.6) \
  --output out/reused_map_follow.wav

pvx voc target.wav --pitch-map maps/guide_full.csv \
  --route pitch_ratio=affine(formant_f1_hz,0.0012,0.6) \
  --route pitch_ratio=clip(pitch_ratio,0.8,1.4) \
  --route stretch=affine(onset_norm,-0.3,1.2) \
  --route stretch=clip(stretch,0.8,1.3) \
  --output out/reused_map_alt_formula.wav
```

12.10 Compact Feature Set (Smaller CSV)

```
pvx pitch-track guide.wav --feature-set basic --output maps/guide_basic.csv
pvx voc target.wav --pitch-map maps/guide_basic.csv \
  --route pitch_ratio=affine(spectral_centroid_hz,0.0002,0.8) \
  --route pitch_ratio=clip(pitch_ratio,0.75,1.4) \
  --output out/basic_features_follow.wav
```

12.11 Safe Starting Ranges

When designing formulas, start conservative and widen only if needed:

- pitch_ratio: 0.85 .. 1.15
- stretch: 0.8 .. 1.4
- confidence floor: 0.65 .. 0.85
- map smoothing: --pitch-map-smooth-ms 10..40

Example conservative profile:

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \  
  --pitch-conf-min 0.8 --pitch-lowconf-mode hold --pitch-map-smooth-ms 20 \  
  --route pitch_ratio=affine(mfcc_01,0.001,1.0) \  
  --route pitch_ratio=clip(pitch_ratio,0.9,1.1) \  
  --route stretch=affine(spectral_flux,0.02,1.0) \  
  --route stretch=clip(stretch,0.9,1.2) \  
  --output out/conservative_sidechain.wav
```

CHAPTER 13

Follow Workflow Migration

This note describes how to migrate from long pitch-follow shell pipelines to the unified `pvx follow` helper.

13.1 Why This Change

This section begins with the following points, which establish the context for the details that follow.

- shorter commands for common sidechain workflows
- fewer shell-specific quoting and pipe errors
- same control-map architecture under the hood (`pitch-track + voc --control-stdin`)
- backward compatibility for existing pipelines

13.2 Old vs New

Old (manual pipe):

```
pvx pitch-track A.wav --emit pitch_to_stretch --output - \  
| pvx voc B.wav --control-stdin --pitch-conf-min 0.75 --output B_follow.wav
```

New (one command):

```
pvx follow A.wav B.wav --emit pitch_to_stretch --pitch-conf-min 0.75 --output B_follow.wav
```

13.3 When to Keep Manual Pipes

Use explicit pipes when you need:

- custom route rewrites (`--route stretch=pitch_ratio, --route pitch_ratio=const(1.0)`)
- external non-pvx processing between producer and consumer
- shell orchestration with additional non-audio transforms

Equivalent advanced manual path:

```
pvx pitch-track A.wav --output - \  
| pvx voc B.wav --control-stdin \  
  --route stretch=pitch_ratio \  
  --route pitch_ratio=const(1.0) \  
  --output B_manual.wav
```

13.4 Rollout Guidance

This section begins with the following points, which establish the context for the details that follow.

1. For new scripts, default to `pvx follow`.
2. Keep existing manual pipelines unchanged until they are touched for other reasons.
3. Add regression tests for representative follow jobs (already covered in `tests/test_cli_regression.py`).
4. Keep manual route examples in documentation for advanced control-bus users.

CHAPTER 14

100 Crazy Things To Try With pvx

These are intentionally bold, sometimes impractical ideas for sound-design exploration. Use `pvx stretch-budget` and sensible disk limits before launching extreme renders.

14.1 Extreme Time-Scale Madness

This section begins with the following points, which establish the context for the details that follow.

1. Stretch a one-shot to absurd length:

```
pvx stretch-budget one_shot.wav --disk-budget 50GB --requested-stretch 1000000 --fail-if-exceeds --json &&  
pvx voc one_shot.wav --stretch 1000000 --stretch-mode multistage --auto-segment-seconds 0.25 --  
checkpoint-dir ckpt --resume --output one_shot_blackhole.wav
```

2. Make a 10-minute ambient tail from a single hit:

```
pvx voc hit.wav --target-duration 600 --preset extreme_ambient --output hit_10min.wav
```

3. Stretch then shrink in two stages for texture hysteresis:

```
pvx chain input.wav --pipeline "voc --stretch 12 | voc --stretch 0.08" --output hysteresis.wav
```

4. Warp tempo with aggressive map swings:

```
pvx warp loop.wav --map maps/warp_chaos.csv --crossfade-ms 10 --output loop_chaos_warp.wav
```

5. Build a staircase of tempo regimes:

```
pvx conform input.wav --map maps/tempo_stairs.csv --output input_tempo_stairs.wav
```

6. Force tiny windows for intentionally crunchy time stretch:

```
pvx voc source.wav --stretch 8 --n-fft 512 --hop-size 64 --output source_gritstretch.wav
```

7. Force oversized windows for smeared macro motion:

```
pvx voc source.wav --stretch 8 --n-fft 8192 --hop-size 256 --output source_glassstretch.wav
```

8. Alternate compress/expand blocks in one chain:

```
pvx chain groove.wav --pipeline "voc --stretch 0.5 | voc --stretch 2.0 | voc --stretch 0.5 | voc --stretch  
2.0" --output groove_pumpfold.wav
```

9. Create a “time collapse” impulse sculpture:

```
pvx voc ambience.wav --stretch 0.03 --output ambience_collapse.wav
```

10. Use stateful stream for ultra-long continuity tests:

```
pvx stream long_take.wav --output long_take_stateful_20x.wav --chunk-seconds 0.2 --time-stretch 20
```

14.2 Freeze + Morph Experiments

This section begins with the following points, which establish the context for the details that follow.

11. Freeze the loudest drum transient into a pad:

```
pvx freeze drums.wav --freeze-time 0.42 --duration 90 --output drums_freeze_pad.wav
```

12. Freeze a vocal consonant into a synthetic whisper bed:

```
pvx freeze vocal.wav --freeze-time 1.03 --duration 40 --output vocal_consonant_pad.wav
```

13. Morph two unrelated worlds with static alpha:

```
pvx morph piano.wav subway.wav --alpha 0.5 --output piano_subway_50.wav
```

14. Morph by trajectory map from A to B to A:

```
pvx morph a.wav b.wav --alpha controls/alpha_aba.csv --interp linear --output a_b_a_morph.wav
```

15. Use envelope cross-synthesis mode:

```
pvx morph source_a.wav source_b.wav --blend-mode carrier_a_envelope_b --alpha 0.8 --envelope-lifter 36 --output env_cross.wav
```

16. Use mask-driven morph with hard spectral selection:

```
pvx morph source_a.wav source_b.wav --blend-mode carrier_a_mask_b --alpha 0.7 --mask-exponent 1.5 --output mask_cross.wav
```

17. Keep A magnitude but steal B phase attitude:

```
pvx morph source_a.wav source_b.wav --blend-mode magnitude_b_phase_a --alpha 0.65 --phase-mix 0.1 --output phase_cross.wav
```

18. Morph after a freeze for evolving drone color:

```
pvx freeze hit.wav --freeze-time 0.2 --duration 60 --output hit_frozen.wav && pvx morph hit_frozen.wav choir.wav --alpha 0.35 --output frozen_morph.wav
```

19. Morph speech into noise and retune it:

```
pvx morph speech.wav noise.wav --alpha 0.6 --output speech_noise_morph.wav && pvx retune speech_noise_morph.wav --root C --scale minor --strength 1.0 --output speech_noise_retune.wav
```

20. Build a morph ladder at fixed alpha steps for curation:

```
pvx morph a.wav b.wav --alpha controls/alpha_ladder.csv --interp none --output morph_ladder.wav
```

14.3 Pitch, Harmony, and Tuning Extremes

This section begins with the following points, which establish the context for the details that follow.

21. Build a choir from one voice in one command:

```
pvx harmonize lead.wav --intervals 0,4,7,12 --gains 1,0.8,0.7,0.5 --pans -0.4,-0.1,0.1,0.4 --force-stereo --output lead_choir.wav
```

22. Create detuned micro-harmony clusters:

```
pvx harmonize lead.wav --intervals 0,3,7 --intervals-cents 0,7,-11 --gains 1,0.8,0.8 --output lead_microcluster.wav
```

23. Hard retune spoken voice to a scale:

```
pvx retune speech.wav --root D --scale major --strength 1.0 --output speech_tuned.wav
```

24. Gentle retune with minimal intervention:

```
pvx retune vocal.wav --root A --scale minor --strength 0.25 --output vocal_subtle_retune.wav
```

25. Force ratio-based pitch with no stretch change:

```
pvx voc input.wav --stretch 1.0 --ratio 7/4 --output input_ratio_7_4.wav
```

26. Extreme semitone dive with formant-preserving mode:

```
pvx voc vocal.wav --stretch 1.0 --pitch -19 --pitch-mode formant-preserving --output vocal_deep_formant.wav
```

27. Conform to 19-TET map:

```
pvx conform lead.wav --map maps/map_19tet.csv --output lead_19tet.wav
```

28. Conform to just intonation map:

```
pvx conform choir.wav --map maps/map_just_intonation.csv --output choir_ji.wav
```

29. Layer two opposite pitch moves:

```
pvx chain vocal.wav --pipeline "voc --pitch 7 | voc --pitch -12" --output vocal_pitch_fold.wav
```

30. Pitch-shift then freeze at peak weirdness:

```
pvx chain vocal.wav --pipeline "voc --pitch 12 | freeze --freeze-time 0.5 --duration 45" --output vocal_octave_freeze.wav
```

14.4 Transients, Stereo, and Layer Split

This section begins with the following points, which establish the context for the details that follow.

31. Run hybrid transient preservation on speech at high stretch:

```
pvx voc speech.wav --transient-mode hybrid --transient-sensitivity 0.65 --stretch 2.5 --output  
speech_hybrid_2_5.wav
```

32. Stretch drums with transient-first preset:

```
pvx voc drums.wav --preset drums_safe --stretch 1.8 --output drums_safe_1_8.wav
```

33. Mid/side lock for stable wide mixes:

```
pvx voc mix.wav --stereo-mode mid_side_lock --coherence-strength 0.95 --stretch 1.4 --output mix_ms_lock.wav
```

34. Intentionally lower coherence for drifting sides:

```
pvx voc stereo.wav --stereo-mode mid_side_lock --coherence-strength 0.2 --stretch 2 --output  
stereo_side_drift.wav
```

35. Split HPSS layers and stretch only harmonics:

```
pvx layer full_mix.wav --harmonic-stretch 2.0 --percussive-stretch 1.0 --output full_mix_harm_stretch.wav
```

36. Split HPSS layers and tighten percussion only:

```
pvx layer beat.wav --harmonic-stretch 1.0 --percussive-stretch 0.7 --output beat_tight_perc.wav
```

37. Pitch harmonics up while lowering percussion gain:

```
pvx layer groove.wav --harmonic-pitch-semitones 5 --percussive-gain 0.7 --output groove_harm_up.wav
```

38. Push transient mode with tiny protect windows:

```
pvx voc drumloop.wav --transient-mode hybrid --transient-protect-ms 8 --transient-crossfade-ms 2 --stretch  
1.5 --output drumloop_snappy.wav
```

39. Push transient mode with huge protect windows:

```
pvx voc drumloop.wav --transient-mode hybrid --transient-protect-ms 80 --transient-crossfade-ms 20 --stretch  
1.5 --output drumloop_smeashield.wav
```

40. Compare wrapper vs stateful stream seam behavior:

```
pvx stream source.wav --mode wrapper --output source_wrapper.wav --chunk-seconds 0.2 --time-stretch 2.0 &&  
pvx stream source.wav --mode stateful --output source_stateful.wav --chunk-seconds 0.2 --time-stretch  
2.0
```

14.5 Sidechain Follow and Feature-Driven Control

This section begins with the following points, which establish the context for the details that follow.

41. Basic pitch-follow stretch transfer:

```
pvx follow guide.wav target.wav --emit pitch_to_stretch --pitch-conf-min 0.75 --output target_follow.wav
```

42. Build pitch map and route manually through voc:

```
pvx pitch-track guide.wav --emit pitch_map --output - | pvx voc target.wav --control-stdin --route
pitch_ratio=pitch_ratio --output target_manual_follow.wav
```

43. Modulate stretch from spectral flux:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=
affine(spectral_flux,0.04,1.0) --route stretch=clip(stretch,0.8,1.6) --output target_flux_stretch.wav
```

44. Modulate pitch from MFCC component:

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output - | pvx voc target.wav --control-stdin
--route pitch_ratio=affine(mfcc_01,0.002,1.0) --route pitch_ratio=clip(pitch_ratio,0.5,2.0) --output
target_mfcc_pitch.wav
```

45. Gate effect strength by voicing probability:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=
affine(voicing_prob,0.4,0.8) --output target_voicing_gate.wav
```

46. Route onset strength into dynamic stretch pull:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=
affine(onset_strength,0.2,0.9) --route stretch=clip(stretch,0.9,1.3) --output target_onset_push.wav
```

47. Use MPEG-7 spectral descriptors for wild macro motion:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=
affine(mpeg7_spectral_flux,0.05,1.0) --output target_mpeg_flux.wav
```

48. Build a “rhythm-following pad” from spoken guide:

```
pvx follow speech.wav pad.wav --feature-set all --emit pitch_map --route stretch=affine(transientness
,0.2,0.9) --output pad_rhythm_follow.wav
```

49. Use low-confidence pitch regions as chaos zones:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=
affine(confidence,-0.5,1.5) --route stretch=clip(stretch,0.6,2.0) --output target_conf_chaos.wav
```

50. Create sidechain-driven timing from pitch map with manual routing:

```
pvx pitch-track guide.wav --emit pitch_map --output - | pvx voc target.wav --control-stdin --route stretch=
pitch_ratio --route pitch_ratio=const(1.0) --output target_pitchtime_follow.wav
```

14.6 Control-Map Authoring With Envelope + Reshape

This section begins with the following points, which establish the context for the details that follow.

51. Generate ADSR stretch macro map:

```
pvx envelope --mode adsr --duration 12 --rate 20 --attack-sec 0.3 --decay-sec 1.0 --sustain 1.4 --release-
sec 2.0 --key stretch --output controls/stretch_adsr.csv
```

52. Generate sine pitch-ratio wobble map:

```
pvx envelope --mode sine --duration 10 --rate 40 --start 1.0 --peak 0.08 --sine-cycles 12 --key pitch_ratio
--output controls/pitch_wobble.csv
```

53. Generate exponential ramp map for acceleration:

```
pvx envelope --mode exp --duration 8 --rate 30 --start 0.7 --end 1.8 --exp-curve 6 --key stretch --output
controls/stretch_exp.csv
```

54. Normalize a control map to safe range:

```
pvx reshape controls/stretch_raw.csv --key stretch --operation normalize --target-min 0.8 --target-max 1.5
--output controls/stretch_norm.csv
```

55. Resample map to dense control rate:

```
pvx reshape controls/stretch_env.csv --key stretch --operation resample --rate 80 --interp polynomial --
order 5 --output controls/stretch_dense.csv
```

56. Smooth noisy map before applying:

```
pvx reshape controls/stretch_noisy.csv --key stretch --operation smooth --window 11 --output controls/
stretch_smooth.csv
```

57. Time-shift map late by half second:

```
pvx reshape controls/stretch_env.csv --key stretch --operation time-shift --offset 0.5 --output controls/
stretch_shifted.csv
```

58. Time-scale map to half speed modulation:

```
pvx reshape controls/stretch_env.csv --key stretch --operation time-scale --factor 2.0 --output controls/
stretch_slowmod.csv
```

59. Invert a modulation map for opposite behavior:

```
pvx reshape controls/mix.csv --key response_mix --operation invert --output controls/mix_inverted.csv
```

60. Build map then apply it in two deterministic steps:

```
pvx envelope --mode adsr --duration 8 --rate 20 --key stretch --output controls/stretch_env.csv && pvx
reshape controls/stretch_env.csv --key stretch --operation resample --rate 50 --interp polynomial --
order 5 --output controls/stretch_env_dense.csv && pvx voc input.wav --stretch controls/
stretch_env_dense.csv --interp linear --output input_envdriven.wav
```

14.7 Analysis/Response Artifact and Spectral Operator Ideas

This section begins with the following points, which establish the context for the details that follow.

61. Cache PV analysis for repeated experiments:

```
pvx analysis create source.wav --output source.pvxan.npz --n-fft 4096 --hop-size 256
```

62. Inspect artifact stats for pipeline sanity:

```
pvx analysis inspect source.pvxan.npz --json
```

63. Build median response profile:

```
pvx response create source.pvxan.npz --method median --normalize peak --output source.pvxrf.npz
```

64. Build RMS response profile for denser weighting:

```
pvx response create source.pvxan.npz --method rms --normalize rms --output source_rms.pvxrf.npz
```

65. Apply static spectral imprint:

```
pvx filter target.wav --response source.pvxrf.npz --response-mix 1.0 --output target_imprint.wav
```

66. Animate response imprint over time:

```
pvx tvfilter target.wav --response source.pvxrf.npz --tv-map controls/response_mix.csv --tv-interp linear --output target_tv_imprint.wav
```

67. Use response-referenced noise filtering:

```
pvx noisefilter noisy.wav --response noise_profile.pvxrf.npz --noise-floor 1.2 --output noisy_cleaner.wav
```

68. Exaggerate dominant response peaks:

```
pvx bandamp source.wav --response source.pvxrf.npz --band-gain-db 10 --peak-count 12 --output source_peakpump.wav
```

69. Compress/expand in response-relative spectral space:

```
pvx spec-compander source.wav --response source.pvxrf.npz --comp-threshold-db -22 --comp-ratio 3 --expand-ratio 1.4 --output source_specdyn.wav
```

70. Pitch-shift response curve instead of source pitch:

```
pvx filter source.wav --response source.pvxrf.npz --transpose-semitones 7 --shift-bins 24 --output source_resp_shifted.wav
```

14.8 Ring, Resonator, Chordmapper, Inharmonator

This section begins with the following points, which establish the context for the details that follow.

71. Slow ring tremolo with deep mix:

```
pvx ring input.wav --frequency-hz 3.5 --depth 1.0 --mix 1.0 --output input_ring_trem.wav
```

72. Audio-rate ring metallicizer:

```
pvx ring input.wav --frequency-hz 97 --depth 0.95 --mix 0.9 --output input_ring_metal.wav
```

73. Ring with near-chaotic feedback memory:

```
pvx ring input.wav --frequency-hz 43 --depth 1.0 --mix 1.0 --feedback 0.92 --output input_ring_fb.wav
```

74. Ring + resonant peak filter:

```
pvx ringfilter input.wav --frequency-hz 60 --resonance-hz 1600 --resonance-q 12 --resonance-mix 0.6 --output input_ringfilter.wav
```

75. Time-varying ring map performance:

```
pvx ringtvfilter input.wav --tv-map controls/ring_map.csv --tv-interp cubic --output input_ringtv.wav
```

76. Chord-lock noisy ambience to minor harmony:

```
pvx chordmapper ambience.wav --root-hz 110 --chord minor --strength 1.0 --boost-db 8 --attenuation 0.3 --output ambience_minorlock.wav
```

77. Tight chord attraction with low tolerance:

```
pvx chordmapper pad.wav --root-hz 220 --chord diminished --tolerance-cents 12 --output pad_tight_chord.wav
```

78. Soft chord attraction with high tolerance:

```
pvx chordmapper pad.wav --root-hz 220 --chord major7 --tolerance-cents 80 --output pad_soft_chord.wav
```

79. Inharmonic stiff-string emulation:

```
pvx inharmonator bells.wav --inharmonic-f0-hz 220 --inharmonicity 0.0002 --inharmonic-mix 1.0 --output bells_stiff.wav
```

80. Subtle inharmonic shimmer blend:

```
pvx inharmonator pad.wav --inharmonic-f0-hz 110 --inharmonicity 0.00005 --inharmonic-mix 0.25 --dry-mix 0.8 --output pad_shimmer.wav
```

14.9 Multi-Stage Pipelines, Streaming, and Batch Tricks

This section begins with the following points, which establish the context for the details that follow.

81. One-line cleanup + stretch + formant chain:

```
pvx chain vocal.wav --pipeline "denoise --reduction-db 6 | deverb --strength 0.35 | voc --stretch 1.3 | formant --mode preserve" --output vocal_full_chain.wav
```

82. Stream with explicit output policy controls:

```
pvx stream source.wav --output source_stream.wav --chunk-seconds 0.2 --time-stretch 2.0 --bit-depth 24 --dither tpdf --dither-seed 7 --metadata-policy sidecar
```

83. Stream wrapper compatibility baseline:

```
pvx stream source.wav --mode wrapper --output source_wrapper.wav --chunk-seconds 0.2 --time-stretch 2.0
```

84. Stateful stream driven by stretch map:


```
pvx stream input.wav --output output_stream_map.wav --chunk-seconds 0.2 --stretch controls/stretch.csv --
interp linear
```

85. Pipe denoise straight into deverb with no temp file:

```
pvx denoise field.wav --reduction-db 5 --smooth 7 --stdout | pvx deverb - --strength 0.35 --output
field_clean.wav
```

86. Pipe harmonize into voc for post-stack stretch:

```
pvx harmonize lead.wav --intervals 0,4,7 --gains 1,0.75,0.65 --stdout | pvx voc - --stretch 1.5 --output
lead_stack_stretched.wav
```

87. Chain with alternating transient modes for A/B blend:

```
pvx chain speech.wav --pipeline "voc --transient-mode hybrid --stretch 1.5 | voc --transient-mode off --
stretch 1.0" --output speech_transient_combo.wav
```

88. Run two-pass pitch: retune then ratio-jump:

```
pvx chain vocal.wav --pipeline "retune --root E --scale minor --strength 0.7 | voc --ratio 9/8" --output
vocal_two_pass_pitch.wav
```

89. Long-run resume-safe rendering script target:

```
pvx voc long.wav --target-duration 14400 --checkpoint-dir ckpt_long --auto-segment-seconds 0.25 --resume --
output long_4h.wav
```

90. Dynamic spectral map workflow with reusable artifacts:

```
pvx analysis create source.wav --output source.pvxn.npz && pvx response create source.pvxn.npz --output
source.pvxrf.npz && pvx tvfilter source.wav --response source.pvxrf.npz --tv-map controls/mix.csv --
output source_dynspec.wav
```

14.10 QA, Stress, and Reproducibility Challenges

This section begins with the following points, which establish the context for the details that follow.

91. Budget-gate every render before execution:

```
pvx stretch-budget input.wav --disk-budget 10GB --requested-stretch 5000 --fail-if-exceeds
```

92. Compare quality at multiple FFT sizes quickly:

```
pvx chain source.wav --pipeline "voc --stretch 2 --n-fft 1024 | voc --stretch 1 --n-fft 4096" --output
source_fft_compare.wav
```

93. Build deterministic dithered exports for null tests:

```
pvx voc input.wav --stretch 1.1 --bit-depth 24 --dither tpdf --dither-seed 123 --output out_seeded.wav
```

94. Generate manifest metadata for every run:

```
pvx voc input.wav --stretch 1.08 --manifest-json reports/pvx_runs.json --manifest-append --output out_manifest.wav
```

95. Stress dynamic controls with dense maps:

```
pvx voc input.wav --stretch controls/stretch_dense.csv --interp polynomial --order 5 --output out_densectrl.wav
```

96. Test phase behavior by toggling phase engines:

```
pvx chain source.wav --pipeline "voc --stretch 2 --phase-engine propagate | voc --stretch 1 --phase-engine random" --output source_phase_duel.wav
```

97. Validate sidechain routing with built-in examples:

```
pvx follow --example all
```

98. Run parity benchmark regression module directly:

```
python3 -m unittest tests.test_pvc_parity_benchmark
```

99. Run full regression tests after extreme experiments:

```
python3 -m unittest discover -s tests -p "test_*.py"
```

100. Build a curated “crazy pack” by rendering 100 variants and scoring them with metrics:

```
pvx voc input.wav --stretch 1.2 --output candidate.wav && python3 -m unittest tests.test_cli_regression
```

Tips: - Keep experimental commands in scripts with fixed seeds/options for repeatability. - For very long jobs, always use `--checkpoint-dir` and `--resume`. - Validate impossible requests with `pvx stretch-budget` before running expensive renders.

Part V

Python, ML, and Dataset Work

CHAPTER 15

pvx Application Programming Interface (API) Overview

This guide shows how to use `pvx` as a Python library instead of shell commands.

15.1 Import Paths

If installed with `pip install -e .`:

```
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch, resample_1d
```

From repository source without install:

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path.cwd() / "src"))
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch, resample_1d
```

15.2 Minimal Time-Stretch Example

```
import numpy as np
import soundfile as sf
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch

x, sr = sf.read("input.wav", always_2d=True)
mono = np.mean(x, axis=1)

cfg = VocoderConfig(
    n_fft=2048,
    win_length=2048,
    hop_size=512,
    window="hann",
    center=True,
    phase_locking="identity",
    transient_preserve=True,
    transient_threshold=2.0,
)

y = phase_vocoder_time_stretch(mono, 1.25, cfg)
sf.write("output_stretch.wav", y, sr)
```

15.3 Minimal Pitch-Shift (Duration-Preserving)

```
import numpy as np
import soundfile as sf
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch, resample_1d

x, sr = sf.read("input.wav", always_2d=True)
mono = np.mean(x, axis=1)

semitones = 3.0
ratio = 2 ** (semitones / 12.0)
internal_stretch = ratio

cfg = VocoderConfig(
    n_fft=2048,
    win_length=2048,
    hop_size=512,
    window="hann",
    center=True,
    phase_locking="identity",
    transient_preserve=True,
    transient_threshold=2.0,
)

stretched = phase_vocoder_time_stretch(mono, internal_stretch, cfg)
y = resample_1d(stretched, int(round(stretched.size / ratio)), mode="linear")
sf.write("output_pitch.wav", y, sr)
```

15.4 Jupyter Notebook Snippets

15.4.1 Spectrogram visualization snippet

```
import numpy as np
import matplotlib.pyplot as plt

def db_mag(sig, sr):
    n_fft = 2048
    hop = 512
    win = np.hanning(n_fft)
    frames = []
    for i in range(0, max(1, sig.size - n_fft), hop):
        chunk = sig[i:i+n_fft]
        if chunk.size < n_fft:
            break
        mag = np.abs(np.fft.rfft(chunk * win))
        frames.append(20 * np.log10(mag + 1e-9))
    return np.array(frames).T

S = db_mag(y, sr)
plt.figure(figsize=(10, 4))
plt.imshow(S, origin="lower", aspect="auto", cmap="magma")
plt.colorbar(label="dB")
plt.title("Output spectrogram")
plt.xlabel("Frame")
plt.ylabel("Bin")
plt.tight_layout()
```

15.4.2 Compare phase locking modes

```

from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch

cfg_free = VocoderConfig(
    n_fft=2048, win_length=2048, hop_size=512, window="hann",
    center=True, phase_locking="off", transient_preserve=True, transient_threshold=2.0
)
cfg_lock = VocoderConfig(
    n_fft=2048, win_length=2048, hop_size=512, window="hann",
    center=True, phase_locking="identity", transient_preserve=True, transient_threshold=2.0
)

y_free = phase_vocoder_time_stretch(mono, 1.25, cfg_free)
y_lock = phase_vocoder_time_stretch(mono, 1.25, cfg_lock)

```

15.5 Batch Processing Example (Python)

```

from pathlib import Path
import numpy as np
import soundfile as sf
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch

cfg = VocoderConfig(
    n_fft=2048, win_length=2048, hop_size=512,
    window="hann", center=True,
    phase_locking="identity", transient_preserve=True, transient_threshold=2.0
)

inp = Path("stems")
out = Path("out_stems")
out.mkdir(exist_ok=True)

for path in sorted(inp.glob("*.wav")):
    x, sr = sf.read(path, always_2d=True)
    y_channels = []
    for ch in range(x.shape[1]):
        y_channels.append(phase_vocoder_time_stretch(x[:, ch], 1.08, cfg))
    n = max(len(ch) for ch in y_channels)
    y = np.zeros((n, len(y_channels)), dtype=np.float64)
    for idx, ch in enumerate(y_channels):
        y[:len(ch), idx] = ch
    sf.write(out / f"{path.stem}_pv.wav", y, sr)

```

15.6 Segment Processing Example (Python)

```

from dataclasses import dataclass
import numpy as np
import soundfile as sf
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch, resample_id

@dataclass
class Segment:
    start_sec: float
    end_sec: float
    stretch: float
    pitch_ratio: float

segments = [
    Segment(0.0, 1.0, 1.00, 1.00),
    Segment(1.0, 2.0, 1.20, 1.00),
    Segment(2.0, 3.0, 0.95, 2 ** (2/12)),

```

```
]

x, sr = sf.read("input.wav", always_2d=True)
mono = np.mean(x, axis=1)
cfg = VocoderConfig(
    n_fft=2048, win_length=2048, hop_size=512,
    window="hann", center=True,
    phase_locking="identity", transient_preserve=True, transient_threshold=2.0
)

parts = []
for seg in segments:
    s = int(round(seg.start_sec * sr))
    e = int(round(seg.end_sec * sr))
    if e <= s:
        continue
    block = mono[s:e]
    internal = seg.stretch * seg.pitch_ratio
    z = phase_vocoder_time_stretch(block, internal, cfg)
    if abs(seg.pitch_ratio - 1.0) > 1e-9:
        z = resample_1d(z, int(round(z.size / seg.pitch_ratio)), mode="linear")
    parts.append(z)

y = np.concatenate(parts) if parts else np.zeros(0, dtype=np.float64)
sf.write("segment_output.wav", y, sr)
```

15.7 Related CLI Equivalents

This section begins with the following points, which establish the context for the details that follow.

- `pvx voc ...` for direct production workflows.
- `pvx voc --explain-plan` to inspect resolved processing settings.
- `pvx voc --manifest-json ...` to log run metadata.

CHAPTER 16

ML/DL Framework Integration Guide

Alpha release (0.1.0a1) , `pvx.augment` is under active development. Public interfaces may change between minor versions until 1.0. Pin your dependency to an exact version (`pvx==0.1.0a1`) if you need stability, and please report issues on GitHub.

`pvx` ships a first-class Python API for audio data augmentation (`pvx.augment`) and thin, zero-boilerplate adapters for the three most popular deep-learning ecosystems.

16.1 Contents

This section begins with the following points, which establish the context for the details that follow.

1. Installation
 2. Quick Start , Pure Python
 3. PyTorch Integration
 4. HuggingFace Datasets Integration
 5. TensorFlow / tf.data Integration
 6. Determinism and Reproducibility
 7. Performance Tips
 8. GPU-Accelerated Transforms (PyTorch)
 9. Available Transforms
-

16.2 Installation

```
# Core DSP + augmentation (no ML framework required)
\index{Core DSP + augmentation (no ML framework required)}
pip install pvx

# With PyTorch
\index{With PyTorch}
pip install "pvx[torch]"

# With HuggingFace
\index{With HuggingFace}
pip install "pvx[huggingface]"

# With TensorFlow
\index{With TensorFlow}
pip install "pvx[tensorflow]"

# All ML extras
\index{All ML extras}
pip install "pvx[ml]"
```

16.3 Quick Start , Pure Python

```
import soundfile as sf
from pvx.augment import (
    Pipeline,
    GainPerturber,
    RoomSimulator,
    AddNoise,
    SpecAugment,
    CodecDegradation,
)

# Load audio (any format supported by soundfile)
\index{Load audio (any format supported by soundfile)}
audio, sr = sf.read("speech.wav", dtype="float32", always_2d=False)

# Build an augmentation pipeline
\index{Build an augmentation pipeline}
pipeline = Pipeline([
    GainPerturber(gain_db=(-3, 3), p=0.8),
    RoomSimulator(rt60_range=(0.05, 0.8), wet_range=(0.1, 0.5), p=0.4),
    AddNoise(snr_db=(15, 35), noise_type="pink", p=0.5),
    CodecDegradation(codec="random", p=0.2),
    SpecAugment(freq_mask_param=27, time_mask_param=100, p=0.5),
], seed=42)

# Apply with a reproducible seed
\index{Apply with a reproducible seed}
audio_aug, sr_out = pipeline(audio, sr, seed=0)

# Save result
\index{Save result}
sf.write("speech_aug.wav", audio_aug, sr_out)
```

16.4 Intent Preset Pipelines

Ready-made pipelines for common research use cases:

```
from pvx.augment import asr_pipeline, music_pipeline, speech_enhancement_pipeline, contrastive_pipeline

# ASR robustness training
\index{ASR robustness training}
pipeline = asr_pipeline(seed=42)

# Music information retrieval
\index{Music information retrieval}
pipeline = music_pipeline(seed=42)

# Speech enhancement (noisy/reverberant input, clean target)
\index{Speech enhancement (noisy/reverberant input clean target)}
noisy_pipeline = speech_enhancement_pipeline(seed=42)

# Contrastive self-supervised learning (two correlated views)
\index{Contrastive self-supervised learning (two correlated views)}
pipeline_a, pipeline_b = contrastive_pipeline(seed=42)
clean_audio, sr = sf.read("speech.wav", dtype="float32")
view_a, _ = pipeline_a(clean_audio, sr, seed=idx)
view_b, _ = pipeline_b(clean_audio, sr, seed=idx)
```

16.5 PyTorch Integration

16.5.1 PvxAugmentDataset

```
import torch
from torch.utils.data import DataLoader
from pvx.augment import Pipeline, AddNoise, RoomSimulator, SpecAugment
from pvx.integrations.pytorch import PvxAugmentDataset

pipeline = Pipeline([
    AddNoise(snr_db=(10, 30), p=0.5),
    RoomSimulator(rt60_range=(0.1, 0.8), p=0.4),
    SpecAugment(freq_mask_param=30, time_mask_param=50, p=0.5),
], seed=42)

file_list = ["data/train/speech_001.wav", "data/train/speech_002.wav", ...]
labels = [0, 1, ...] # optional

dataset = PvxAugmentDataset(
    file_list,
    pipeline=pipeline,
    sample_rate=16000,
    labels=labels,
    duration_s=3.0,      # crop/pad all clips to 3 seconds
    mono=True,
)

loader = DataLoader(
    dataset.as_torch_dataset(),
    batch_size=32,
    num_workers=4,
    collate_fn=PvxAugmentDataset.collate_fn,
    pin_memory=True,
)

for batch in loader:
    audio = batch["audio"] # (B, T) or (B, C, T)
    label = batch["label"] # (B,)
    lengths = batch["lengths"] # (B,) , true clip lengths before padding
    # ... training step ...
    \index{training step}
```

16.5.2 PvxAugmentTransform (torchvision-style)

Use as a drop-in transform in any torchaudio-compatible dataset:

```
from pvx.integrations.pytorch import PvxAugmentTransform

transform = PvxAugmentTransform(pipeline, sample_rate=16000)

# Use with torchaudio
\index{Use with torchaudio}
import torchaudio
ds = torchaudio.datasets.LIBRISPEECH(
    root="data/", url="train-clean-100",
    download=True,
)

# Apply transform manually:
\index{Apply transform manually}
audio, sr, transcript, *_ = ds[0]
audio_aug = transform(audio)
```

16.5.3 AudioCollator for variable-length batches

```
from pvx.integrations.pytorch import AudioCollator

loader = DataLoader(
    dataset.as_torch_dataset(),
    batch_size=16,
    collate_fn=AudioCollator(return_lengths=True),
)

for audio_batch, lengths in loader:
    # audio_batch: (B, C, T_max) padded to longest clip
    \index{audio batch (B C T max) padded to longest clip}
    # lengths: (B,) true lengths
    \index{lengths (B ) true lengths}
    pass
```

16.6 HuggingFace Datasets Integration

16.6.1 Basic map() usage

```
from datasets import load_dataset
from pvx.augment import Pipeline, AddNoise, RoomSimulator
from pvx.integrations.huggingface import make_augment_map_fn

ds = load_dataset("mozilla-foundation/common_voice_13_0", "en", split="train")

pipeline = Pipeline([
    AddNoise(snr_db=(10, 30), p=0.5),
    RoomSimulator(rt60_range=(0.1, 0.8), p=0.4),
], seed=42)

# HuggingFace Audio feature rows have format: {"array": np.ndarray, "sampling_rate": int}
\index{HuggingFace Audio feature rows have format array np.ndarray sampling rate int}
augment_fn = make_augment_map_fn(
    pipeline,
    audio_column="audio",
    output_column="audio_aug", # write to new column (keeps original)
    base_seed=42,
)

ds_aug = ds.map(
    augment_fn,
    batched=False,
    with_indices=True, # required for per-row seed
    num_proc=8,
)
```

16.6.2 Contrastive view generation (SSL)

```

from pvx.integrations.huggingface import HFAugmentMapper
from pvx.augment import contrastive_pipeline

pipeline_a, pipeline_b = contrastive_pipeline(seed=42)

# Generate both views in a single map pass
\index{Generate both views in a single map pass}
mapper_a = HFAugmentMapper(pipeline_a, audio_column="audio", output_prefix="view_a")
mapper_b = HFAugmentMapper(pipeline_b, audio_column="audio", output_prefix="view_b")

# Or use built-in pair mode
\index{Or use built-in pair mode}
from pvx.augment import music_pipeline
mapper = HFAugmentMapper(
    music_pipeline(seed=42),
    audio_column="audio",
    generate_pair=True,
    output_prefix="view",
)
ds_ssl = ds.map(mapper, batched=False, with_indices=True)
# ds_ssl["view_a"]["array"], ds_ssl["view_b"]["array"]
\index{ds ssl view a array ds ssl view b array}

```

16.6.3 One-liner convenience

```

from pvx.integrations.huggingface import augment_dataset

ds_aug = augment_dataset(ds, pipeline, audio_column="audio", num_proc=8)

```

16.6.4 Saving augmented datasets

```

# Save to disk for fast reloading
\index{Save to disk for fast reloading}
ds_aug.save_to_disk("data/augmented_librispeech")

# Or push to HuggingFace Hub
\index{Or push to HuggingFace Hub}
ds_aug.push_to_hub("my-org/librispeech-augmented")

```

16.7 TensorFlow / tf.data Integration

16.7.1 Tensor map function

```

import tensorflow as tf
from pvx.augment import Pipeline, AddNoise, RoomSimulator
from pvx.integrations.tensorflow import make_tf_augment_fn, pvx_augment_tf_dataset

pipeline = Pipeline([
    AddNoise(snr_db=(10, 30), p=0.5),
    RoomSimulator(rt60_range=(0.1, 0.8), p=0.4),
], seed=42)

# From raw tensor dataset
\index{From raw tensor dataset}
ds = tf.data.Dataset.from_tensor_slices(audio_array) # shape: (N, T)
augment_fn = make_tf_augment_fn(pipeline, sample_rate=16000)
ds_aug = ds.map(augment_fn, num_parallel_calls=tf.data.AUTOTUNE)

```

```
# From dict dataset (e.g. TF-Datasets)
\index{From dict dataset (e.g. TF-Datasets)}
import tensorflow_datasets as tfds
tfds_raw = tfds.load("speech_commands", split="train")
ds_aug = pvx_augment_tf_dataset(tfds_raw, pipeline, audio_key="audio")
```

16.7.2 Dict-based map for keyed datasets

```
from pvx.integrations.tensorflow import make_tf_augment_map_fn

fn = make_tf_augment_map_fn(
    pipeline,
    audio_key="audio",
    output_key="audio_aug",
    default_sr=16000,
)
ds_aug = ds.map(fn, num_parallel_calls=tf.data.AUTOTUNE)
```

16.8 Determinism and Reproducibility

All pvx transforms accept a **seed** parameter that fully determines their output. The seed hierarchy is:

```
global_seed + item_index -> per-item seed
per-item seed + transform_index -> per-transform seed
```

This means: - Same **seed** + same input -> identical output (byte-for-byte reproducible) - Different item indices produce different augmentations even with the same global seed - Validation sets can be deterministically generated by fixing the seed

```
# Training: random seed per epoch for variety
\index{Training random seed per epoch for variety}
audio_aug, _ = pipeline(audio, sr, seed=epoch * n_items + item_idx)

# Validation: fixed seed for reproducible evaluation
\index{Validation fixed seed for reproducible evaluation}
audio_aug, _ = pipeline(audio, sr, seed=99999 + item_idx)
```

16.9 Performance Tips

16.9.1 Offline pre-computation (recommended for large datasets)

```
# Generate the augmented dataset once and save it
\index{Generate the augmented dataset once and save it}
pvx augment data/train/*.wav \
  --output-dir data/train_aug \
  --variants-per-input 4 \
  --intent asr_robust \
  --seed 1337 \
  --workers 16
```

Then load pre-augmented files directly , eliminates augmentation overhead at training time.

16.9.2 Online augmentation with worker processes

```
# PyTorch: set num_workers >= 4 , each worker gets a separate process
\index{PyTorch set num workers 4 each worker gets a separate process}
loader = DataLoader(dataset.as_torch_dataset(), num_workers=8, ...)

# HuggingFace: use num_proc
\index{HuggingFace use num proc}
ds_aug = ds.map(augment_fn, num_proc=8)

# TensorFlow: use AUTOTUNE
\index{TensorFlow use AUTOTUNE}
ds_aug = ds.map(fn, num_parallel_calls=tf.data.AUTOTUNE)
```

16.9.3 Caching expensive transforms

For transforms like RoomSimulator that are computationally intensive, cache intermediate results:

```
# HuggingFace , cache augmented dataset after first run
\index{HuggingFace cache augmented dataset after first run}
ds_aug = ds.map(augment_fn, num_proc=8, cache_file_name="data/cache/aug.arrow")
```

16.9.4 TimeStretch and PitchShift , Engine Selection

TimeStretch and PitchShift support five processing engines via the `engine` parameter:

Engine	Requires	Notes
"auto" (default)	,	Prefers torchaudio > pytorch > pvx-cli
"torchaudio"	<code>pip install "pvx[torch]"</code>	Optimized C++ phase-vocoder kernel via torchaudio
"pytorch"	<code>pip install "pvx[torch]"</code>	Vectorized phase-vocoder in pure PyTorch
"wavelet"	built-in Haar; optional <code>pip install PyWavelets</code>	Experimental multiband wavelet backend
"pvx-cli"	<code>pvx CLI on PATH</code>	Full pvxvoc DSP stack via subprocess

```
from pvx.augment import TimeStretch, PitchShift

# Auto-select best available engine (recommended)
\index{Auto-select best available engine (recommended)}
ts = TimeStretch(rate=(0.8, 1.2), engine="auto")

# Force torchaudio engine (fastest, requires torchaudio)
\index{Force torchaudio engine (fastest requires torchaudio)}
ts = TimeStretch(rate=(0.8, 1.2), engine="torchaudio")

# Force PyTorch engine (no torchaudio needed, just torch)
\index{Force PyTorch engine (no torchaudio needed just torch)}
ts = TimeStretch(rate=(0.8, 1.2), engine="pytorch")

# Force experimental wavelet engine.
\index{Force experimental wavelet engine}
# wavelet="auto" uses PyWavelets db4 when available, otherwise built-in Haar.
\index{wavelet auto uses PyWavelets db4 when available otherwise built-in Haar}
ts = TimeStretch(rate=(0.8, 1.2), engine="wavelet", wavelet="auto", wavelet_levels=3)
ps = PitchShift(semitones=(-2, 2), engine="wavelet", wavelet="haar", wavelet_levels=4)
```

```
# Force pvx CLI subprocess (full DSP stack: transients, formants, stereo)
\index{Force pvx CLI subprocess (full DSP stack transients formants stereo)}
ts = TimeStretch(rate=(0.8, 1.2), engine="pvx-cli")
```

The CLI `pvx augment` command also accepts `--engine`:

```
pvx augment data/train/*.wav \
  --output-dir data/train_aug \
  --engine torchaudio \
  --intent asr_robust
```

To compare backend speed and simple pitch/duration smoke metrics:

```
python benchmarks/run_backend_compare.py --engines pytorch,pvx-cli,wavelet --wavelet auto
```

If no engine is available, `TimeStretch` and `PitchShift` will raise a `RuntimeError` with a clear message, they will **not** silently return unmodified audio.

For a lightweight, pure-Python pitch shift that requires no external engine, use `PitchShiftSimple` (spectral bin interpolation, ± 1 -3 semitone approximation).

16.9.5 Streaming Augmentation for Long-Form Audio

For podcasts, audiobooks, and other long-form content, use the streaming API to process files in chunks with bounded memory:

```
from pvx.augment import stream_augment_file, Pipeline, AddNoise, GainPerturber

pipeline = Pipeline([
    GainPerturber(gain_db=(-3, 3), p=0.8),
    AddNoise(snr_db=(15, 35), noise_type="pink", p=0.5),
], seed=42)

# Process a 2-hour podcast in 30-second chunks
\index{Process a 2-hour podcast in 30-second chunks}
stream_augment_file(
    "podcast_2h.wav",
    "podcast_2h_aug.wav",
    pipeline=pipeline,
    chunk_duration_s=30.0,
    seed=42,
)
```

16.9.6 Impulse Response Database

For physics-based room acoustics, use real impulse responses instead of synthetic approximations:

```
from pvx.augment import IRDatabase, ImpulseResponseConvolver

db = IRDatabase()
db.download("echothief") # 115 real-world IRs, cached locally

# Use all IRs
\index{Use all IRs}
aug = ImpulseResponseConvolver(db.ir_dir("echothief"), wet_range=(0.4, 1.0))

# Or filter by category
\index{Or filter by category}
halls = db.filter("echothief", category="hall")
aug = ImpulseResponseConvolver(halls, wet_range=(0.5, 0.9))
```


16.9.7 Mix online and offline augmentation

Use the CLI for heavy transforms (time-stretch, pitch-shift) and the Python API for lightweight ones (gain, noise, SpecAugment) at runtime:

```
# Heavy transforms pre-computed offline
\index{Heavy transforms pre-computed offline}
ds_heavy = load_from_disk("data/train_heavy_aug")

# Lightweight transforms applied online
\index{Lightweight transforms applied online}
light_pipeline = Pipeline([
    GainPerturber(gain_db=(-3, 3), p=0.8),
    AddNoise(snr_db=(15, 35), noise_type="white", p=0.4),
    SpecAugment(freq_mask_param=27, time_mask_param=100, p=0.5),
], seed=42)

fn = make_augment_map_fn(light_pipeline, audio_column="audio")
ds_final = ds_heavy.map(fn, batched=False, with_indices=True, num_proc=4)
```

16.10 GPU-Accelerated Transforms (PyTorch)

For maximum throughput during GPU training, `pvx.augment.gpu` provides native PyTorch transforms that operate directly on `torch.Tensor` objects, no NumPy round-tripping required. These support batched operation and run on CUDA, MPS, or CPU.

```
import torch
from pvx.augment.gpu import (
    TorchPipeline,
    TorchGainPerturber,
    TorchAddNoise,
    TorchEQPerturber,
    TorchSpecAugment,
    TorchNormalizer,
    TorchClippingSimulator,
    TorchTimeStretch,
    TorchPitchShift,
    TorchRoomSimulator,
    TorchMixup,
    NumpyTransformAdapter, # wrap any NumPy transform for GPU use
)

# Build a GPU-native pipeline
\index{Build a GPU-native pipeline}
gpu_pipeline = TorchPipeline([
    TorchGainPerturber(gain_db=(-6, 6), p=0.8),
    TorchAddNoise(snr_db=(10, 30), noise_type="pink", p=0.5),
    TorchEQPerturber(n_bands=4, gain_db_range=(-6, 6), p=0.4),
    TorchSpecAugment(freq_mask_param=27, time_mask_param=100, p=0.5),
], seed=42)

# Apply to a batch of tensors (B, C, T)
\index{Apply to a batch of tensors (B C T)}
audio_batch = torch.randn(32, 1, 48000).cuda()
audio_aug = gpu_pipeline(audio_batch, sr=16000)
```

16.10.1 Mixing GPU and NumPy transforms

Use `NumpyTransformAdapter` to wrap any NumPy-based pvx transform for use in a `TorchPipeline`. The adapter moves data to CPU per-sample, so native `Torch*` transforms are preferred for the hot path:

```
from pvx.augment import CodecDegradation

gpu_pipeline = TorchPipeline([
    TorchGainPerturber(gain_db=(-6, 6)),
    NumpyTransformAdapter(CodecDegradation(codec="random", p=0.2)),
    TorchSpecAugment(freq_mask_param=27),
])
```

16.10.2 Available GPU transforms

Transform	Equivalent NumPy transform	Notes
<code>TorchGainPerturber</code>	<code>GainPerturber</code>	Batched random gain
<code>TorchAddNoise</code>	<code>AddNoise</code>	white/pink/brown via FFT on GPU
<code>TorchEQPerturber</code>	<code>EQPerturber</code>	STFT-domain parametric EQ
<code>TorchSpecAugment</code>	<code>SpecAugment</code>	Frequency + time masking
<code>TorchNormalizer</code>	<code>Normalizer</code>	Peak/RMS normalization
<code>TorchClippingSimulator</code>	<code>ClippingSimulator</code>	Hard/soft clipping
<code>TorchTimeStretch</code>	<code>TimeStretch</code>	Phase-vocoder time stretch on GPU
<code>TorchPitchShift</code>	<code>PitchShift</code>	Pitch shift via stretch + resample
<code>TorchRoomSimulator</code>	<code>RoomSimulator</code>	Synthetic reverb approximation
<code>TorchMixup</code>	,	Zhang et al. 2018 batch mixing
<code>NumpyTransformAdapter</code>	any Transform	CPU fallback wrapper

16.11 Available Transforms

Transform	Module	Key Parameters	Use Case
<code>AddNoise</code>	<code>noise</code>	<code>snr_db</code> , <code>noise_type</code>	ASR, VAD, denoising
<code>BackgroundMixer</code>	<code>noise</code>	<code>background_sources</code> , <code>snr_db</code>	Real-world noise robustness
<code>ImpulseNoise</code>	<code>noise</code>	<code>rate</code> , <code>amplitude_range</code>	Transmission artifact simulation

Transform	Module	Key Parameters	Use Case
RoomSimulator	room	rt60_range, wet_range	Synthetic reverb approximation for augmentation
ImpulseResponseConvolver	room	ir_sources, wet_range	Real IR convolution
CodecDegradation	codec	codec	Codec-robust ASR, telephony
BitCrusher	codec	bits	Lo-fi, retro effects
BandwidthLimiter	codec	cutoff_hz	Telephone/radio simulation
SpecAugment	spectral	freq_mask_param, time_mask_param	ASR, keyword spotting
EQPerturber	spectral	n_bands, gain_db_range	Microphone/room coloration
SpectralNoise	spectral	noise_std_range	Feature extractor robustness
GainPerturber	time_domain	gain_db	Loudness invariance
Normalizer	time_domain	mode, target_db	Pre-processing
ClippingSimulator	time_domain	percentile, mode	ADC artifact simulation
TimeShift	time_domain	shift	Temporal invariance
Reverse	time_domain	,	Data doubling
Fade	time_domain	fade_in, fade_out	Boundary artifact reduction
TrimSilence	time_domain	threshold_db	Pre-processing
FixedLengthCrop	time_domain	duration_s	Batching fixed-size inputs
TimeStretch	time_domain	rate, preset, engine, wavelet, wavelet_levels	Tempo invariance (auto/torchaudio/pytorch/wavelet/pvx-cli)
PitchShift	time_domain	semitones, formant_mode, engine, wavelet, wavelet_levels	Pitch invariance (auto/torchaudio/pytorch/wavelet/pvx-cli)
stream_augment_file	streaming	chunk_duration_s, overlap_s	Memory-bounded long-form processing
IRDatabase	ir_database	cache_dir	Real IR collection manager
PitchShiftSimple	spectral	semitones	Lightweight pitch approximation
Pipeline	core	transforms, seed	Sequential composition

Transform	Module	Key Parameters	Use Case
OneOf	core	transforms, weights	Random selection
SomeOf	core	transforms, k	Random subset
RandomApply	core	transform, p	Stochastic application

CHAPTER 17

AI Research and Data Augmentation with pvx

`pvx augment` provides deterministic, manifest-driven audio augmentation for machine-learning workflows.

Primary use cases: - automatic speech recognition (ASR) robustness experiments - music information retrieval (MIR) augmentation studies - self-supervised learning (SSL) contrastive view generation - ablation studies where exact augmentation parameters must be replayed

17.1 Core Command

```
pvx augment data/*.wav --output-dir aug_out --variants-per-input 4 --intent asr_robust --seed 1337
```

What this does: - resolves input files/globs/directories - renders N deterministic variants per input (`--variants-per-input`) - writes per-variant metadata to: - `augment_manifest.jsonl` - `augment_manifest.csv`

Manifest tool companion:

```
pvx augment-manifest validate aug_out/augment_manifest.jsonl --strict
```

17.2 Intent Profiles

Intent	Typical target	Behavior
<code>asr_robust</code>	Speech pipelines	Mild time/pitch perturbation with conservative vocal/formant defaults
<code>mir_music</code>	Music analytics	Moderate timing/pitch/spectral variation suited for musical content
<code>ssl_contrastive</code>	Contrastive representation learning	Wider perturbation envelope for view diversity while remaining reproducible

17.3 Reproducibility Controls

Option	Meaning
<code>--seed</code>	Global deterministic seed for all sampled parameters
<code>--split train,val,test</code>	Deterministic split assignment ratios written into manifest

Option	Meaning
<code>--split-mode</code>	Split strategy: <code>random</code> , <code>label_balanced</code> , <code>speaker_balanced</code>
<code>--labels-csv</code>	Metadata table with <code>path/label/speaker</code> fields for balanced split modes
<code>--grouping</code>	Split assignment grouping strategy (<code>stem-prefix</code> or <code>none</code>)
<code>--group-separator</code>	Prefix separator for grouping (default: <code>--</code>)
<code>--pair-mode</code>	Pair-view mode: <code>off</code> or <code>contrastive2</code>
<code>--label-policy</code>	Perturbation policy: <code>allow_alter</code> or <code>preserve</code>
<code>--policy</code>	JSON policy file with reproducible defaults and parameter bounds
<code>--workers</code>	Parallel render workers with deterministic seed behavior
<code>--resume</code>	Skip previously rendered outputs using existing manifests/output files
<code>--append-manifest</code>	Merge with existing manifests rather than replace
<code>--dry-run</code>	Plan outputs and write manifests without rendering audio
<code>--manifest-jsonl</code> / <code>--manifest-csv</code>	Explicit manifest output paths

17.4 Manifest Fields

Each JSON Lines (JSONL) row contains:

- `source_path`
- `output_path`
- `intent`
- `seed`
- `split` (`train`, `val`, `test`)
- `group_key` (split-group identifier)
- `status` (`planned`, `rendered`, or `error:<code>`)
- `source_sha256`, `output_sha256`
- `params` object, including:
 - `stretch`
 - `pitch`
 - `preset`
 - `window`
 - `transform`
 - `formant_strength`
 - `transient_sensitivity`
 - `target_lufs`
- optional audit object:
 - `duration_sec`
 - `peak_dbfs`
 - `rms_dbfs`
 - `clip_pct`

– **zcr**

CSV manifest includes the same essential fields in tabular form.

17.5 Example Workflows

17.5.1 Speech Robustness Set

```
pvx augment corpus/speech/*.wav \
  --output-dir data_aug/speech \
  --variants-per-input 6 \
  --intent asr_robust \
  --grouping stem-prefix \
  --group-separator "__" \
  --split 0.8,0.1,0.1 \
  --seed 1337
```

17.5.2 Music Information Retrieval Set

```
pvx augment corpus/music/**/*.wav \
  --output-dir data_aug/music \
  --variants-per-input 4 \
  --intent mir_music \
  --split-mode label_balanced \
  --labels-csv labels.csv \
  --split 0.7,0.2,0.1 \
  --seed 2026
```

17.5.3 Contrastive Planning-Only Pass

```
pvx augment corpus/*.wav \
  --output-dir data_aug/plan_only \
  --variants-per-input 3 \
  --intent ssl_contrastive \
  --pair-mode contrastive2 \
  --dry-run \
  --seed 42
```

17.5.4 With explicit manifest paths

```
pvx augment corpus/*.wav \
  --output-dir data_aug/run_a \
  --variants-per-input 5 \
  --intent asr_robust \
  --label-policy preserve \
  --policy policies/asr_policy.json \
  --manifest-jsonl reports/run_a_manifest.jsonl \
  --manifest-csv reports/run_a_manifest.csv \
  --seed 9001
```

17.5.5 Resume and append

```
pvx augment corpus/*.wav \  
  --output-dir data_aug/run_resume \  
  --variants-per-input 5 \  
  --intent mir_music \  
  --resume \  
  --append-manifest \  
  --seed 9001
```

17.5.6 Manifest merge + stats

```
pvx augment-manifest merge \  
  reports/run_a_manifest.jsonl reports/run_b_manifest.jsonl \  
  --output-jsonl reports/merged_manifest.jsonl \  
  --output-csv reports/merged_manifest.csv  
  
pvx augment-manifest stats reports/merged_manifest.jsonl
```

17.6 Policy File Template

```
{  
  "augment": {  
    "intent": "asr_robust",  
    "variants_per_input": 4,  
    "split": "0.8,0.1,0.1",  
    "split_mode": "speaker_balanced",  
    "grouping": "stem-prefix",  
    "group_separator": "__",  
    "pair_mode": "off",  
    "label_policy": "preserve",  
    "workers": 4,  
    "device": "cpu",  
    "output_format": "wav",  
    "bounds": {  
      "stretch": [0.94, 1.08],  
      "pitch": [-1.0, 1.0],  
      "formant_strength": [0.60, 0.92],  
      "transient_sensitivity": [0.50, 0.75],  
      "target_lufs": [-24.0, -16.0]  
    },  
    "choices": {  
      "window": ["hann", "hamming"],  
      "transform": ["fft", "dft"],  
      "preset": ["vocal_studio"]  
    }  
  }  
}
```

17.7 Benchmarking Augmentation Pipelines

Use the profile suite benchmark (speech/music/noisy/stereo):

```
python benchmarks/run_augment_profile_suite.py \  
  --quick \  
  --gate \  
  --out-dir benchmarks/out_augment_profiles
```

Refresh all per-profile baselines after intentional benchmark changes:


```
python benchmarks/run_augment_profile_suite.py \
  --quick \
  --refresh-baselines \
  --out-dir benchmarks/out_augment_profiles_refresh
```

Outputs: - Per-profile reports: - benchmarks/out_augment_profiles/<profile>/report.json - benchmarks/out_augment_profiles/<profile>/report.md - Suite report: - benchmarks/out_augment_profiles/suite_report.json - benchmarks/out_augment_profiles/suite_report.md

17.8 Running with uv

```
uv run pvx augment data/*.wav --output-dir aug_out --variants-per-input 4 --intent asr_robust --seed 1337
```

17.9 Research Notes

This section begins with the following points, which establish the context for the details that follow.

- Keep original clean file IDs in your training metadata so you can group augmented siblings.
- Avoid split leakage: use `--grouping stem-prefix` with a stable naming convention (for example `speaker42__take3.wav`).
- Prefer fixed seeds for published experiments; change seeds only for explicit variance studies.
- Use `--dry-run` before large jobs to validate output counts and manifest content.

CHAPTER 18

Audio Augmentation Cookbook

Alpha release (0.1.0a1) , These recipes use `pvx.augment`, which is under active development. APIs may change before 1.0.

Real-world augmentation recipes for common deep-learning audio tasks. Each recipe includes the rationale, a pipeline configuration, and CLI equivalents.

18.1 Contents

This section begins with the following points, which establish the context for the details that follow.

1. ASR , Automatic Speech Recognition
 2. Keyword Spotting / Wake-Word Detection
 3. Speaker Verification / Identification
 4. Speech Enhancement and Denoising
 5. Music Classification and Tagging
 6. Beat Tracking and Tempo Estimation
 7. Pitch Estimation and Melody Extraction
 8. Audio Event Detection / Sound Classification
 9. Self-Supervised / Contrastive Learning (SimCLR, MoCo, BYOL)
 10. Music Source Separation
-

18.2 ASR , Automatic Speech Recognition

Goal: Train models robust to microphone quality, background noise, room acoustics, and channel distortion.

Augmentation rationale: - Room simulation: phones and meetings happen in reverberant spaces - Background noise: cafes, offices, street noise are the real world - Codec degradation: telephone and VoIP channels are common deployment targets - Gain perturbation: microphone levels vary across recordings - SpecAugment: proven to reduce WER on LibriSpeech, Common Voice

```
from pvx.augment import Pipeline, GainPerturber, RoomSimulator, AddNoise, CodecDegradation, SpecAugment

asr_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=0.9),
    RoomSimulator(rt60_range=(0.05, 1.0), wet_range=(0.1, 0.7), p=0.5),
    AddNoise(snr_db=(5, 35), noise_type="pink", p=0.6),
    CodecDegradation(codec="random", p=0.2),
    SpecAugment(
        freq_mask_param=27,
        time_mask_param=100,
        num_freq_masks=2,
        num_time_masks=2,
        p=0.8,
    ),
], seed=42)
```

CLI:

```
# Step 1: Heavy offline augmentation (time-stretch + pitch)
\index{Step 1 Heavy offline augmentation (time-stretch + pitch)}
pvx augment data/train/*.wav \
  --output-dir data/train_aug \
  --variants-per-input 3 \
  --intent asr_robust \
  --engine auto \
  --seed 1337 \
  --workers 16

# Step 2: Add noise per file
\index{Step 2 Add noise per file}
for f in data/train_aug/*.wav; do
  pvxnoise "$f" --snr 5,30 --noise-type pink --output "$f"
done

# Step 3: Simulate codec degradation (optional)
\index{Step 3 Simulate codec degradation (optional)}
for f in data/train_aug/*.wav; do
  pvxcodec "$f" --codec random --seed "$RANDOM" --output "$f"
done
```

Note: SpecAugment has been shown to improve WER in the original [Park et al. 2019](#) paper. The impact of the full pipeline above will depend on your model, data, and evaluation conditions, we recommend benchmarking on your own held-out test set.

18.3 Keyword Spotting / Wake-Word Detection

Goal: Detect specific words under highly diverse acoustic conditions.

Key considerations: - Label-preserving augmentation only, semantic content must be unchanged - Aggressive time-stretch causes word timing drift; keep it mild - Heavy SpecAugment helps greatly since models often rely on a few spectral cues

```
from pvx.augment import Pipeline, GainPerturber, AddNoise, CodecDegradation, SpecAugment, TimeShift, Fade

kws_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=0.9),
    Fade(fade_in=(0.001, 0.02), fade_out=(0.001, 0.02), p=0.4),
    TimeShift(shift=(-0.05, 0.05), p=0.3),
    AddNoise(snr_db=(5, 30), noise_type="pink", p=0.7),
    CodecDegradation(codec="random", p=0.3),
    SpecAugment(
        freq_mask_param=30,
        time_mask_param=25, # smaller T, keyword clips are short
        num_freq_masks=2,
        num_time_masks=2,
        p=0.7,
    ),
], seed=42)
```

CLI:

```
pvx augment keywords/*.wav \
  --output-dir keywords_aug \
  --variants-per-input 5 \
  --intent asr_robust \
  --label-policy preserve \   # keeps pitch/time perturbation mild
  --seed 7
```

18.4 Speaker Verification / Identification

Goal: Train embeddings invariant to microphone, room, and channel effects but discriminative across speakers.

Important: Pitch perturbation should be minimal (large shifts can change perceived speaker identity and confuse the model during training).

```
from pvx.augment import Pipeline, GainPerturber, RoomSimulator, AddNoise, CodecDegradation, EQPerturber

sv_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=0.9),
    EQPerturber(n_bands=4, gain_db_range=(-8, 8), p=0.6),   # microphone colouration
    RoomSimulator(rt60_range=(0.05, 1.5), wet_range=(0.1, 0.8), p=0.6),
    AddNoise(snr_db=(10, 40), noise_type="pink", p=0.5),
    CodecDegradation(codec="random", p=0.25),
], seed=42)
```

Contrastive learning setup (ECAPA-TDNN / ResNet style):

```
from pvx.augment import contrastive_pipeline

pipeline_a, pipeline_b = contrastive_pipeline(seed=42)

for audio, speaker_id in dataloader:
    view_a, _ = pipeline_a(audio, sr, seed=item_idx)
    view_b, _ = pipeline_b(audio, sr, seed=item_idx)
    loss = nt_xent_loss(embed(view_a), embed(view_b), labels=speaker_id)
```

18.5 Speech Enhancement and Denoising

Goal: Train models to map noisy/reverberant speech -> clean speech. Augmentation creates the *noisy input*; the clean original is the *target*.

```
from pvx.augment import Pipeline, GainPerturber, RoomSimulator, AddNoise, ImpulseNoise, CodecDegradation,
    OneOf, Identity

# This pipeline degrades clean speech to create training pairs
\index{This pipeline degrades clean speech to create training pairs}
degradation_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=1.0),
    RoomSimulator(rt60_range=(0.1, 2.5), wet_range=(0.2, 0.95), p=0.7),
    AddNoise(snr_db=(0, 20), noise_type="pink", p=0.8),
    OneOf([
        CodecDegradation(codec="voip_narrow"),
        CodecDegradation(codec="telephone"),
        Identity(),
    ], p=0.4),
```

```
        ImpulseNoise(rate=1.0, amplitude_range=(0.01, 0.1), p=0.1),
    ], seed=42)

# During training:
\index{During training}
for clean_audio, sr in clean_dataset:
    noisy_audio, _ = degradation_pipeline(clean_audio, sr, seed=item_idx)
    predicted_clean = model(noisy_audio)
    loss = si_snr_loss(predicted_clean, clean_audio)
```

Data generation CLI:

```
# Parallel degradation of a clean corpus
\index{Parallel degradation of a clean corpus}
pvxrir clean/*.wav --rt60 0.1,2.0 --wet 0.3,0.9 --output-dir reverberant/
pvxnoise reverberant/*.wav --snr 0,20 --noise-type pink --output-dir noisy/
pvxcodec noisy/*.wav --codec random --output-dir degraded/
```

18.6 Music Classification and Tagging

Goal: Train genre, mood, instrument, and tag classifiers robust to diverse recording conditions and production styles.

```
from pvx.augment import Pipeline, GainPerturber, EQPerturber, RoomSimulator, AddNoise, SpecAugment, OneOf,
    Identity, ImpulseNoise

music_cls_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=0.9),
    EQPerturber(n_bands=5, gain_db_range=(-8, 8), p=0.6),
    RoomSimulator(rt60_range=(0.1, 1.0), wet_range=(0.05, 0.3), p=0.3),
    OneOf([
        AddNoise(snr_db=(25, 45), noise_type="white"),
        ImpulseNoise(rate=0.5),
        Identity(),
    ], p=0.3),
    SpecAugment(
        freq_mask_param=30,
        time_mask_param=80,
        num_freq_masks=2,
        num_time_masks=2,
        p=0.5,
    ),
], seed=42)
```

Offline augmentation with pvx augment:

```
pvx augment music/*.wav \
    --output-dir music_aug \
    --variants-per-input 4 \
    --intent mir_music \
    --seed 42 \
    --workers 12
```

18.7 Beat Tracking and Tempo Estimation

Goal: Train tempo-invariant beat trackers. Time-stretch is the most critical augmentation.

Important: Time-stretch must preserve beat structure. Use pvx's drum-safe preset which uses hybrid transient mode to keep kick/snare clicks intact.

```
from pvx.augment import Pipeline, GainPerturber, AddNoise, SpecAugment, TimeStretch

beat_pipeline = Pipeline([
    TimeStretch(
        rate=(0.8, 1.25),
        preserve_pitch=True,
        preset="drums_safe",
        engine="auto", # uses PyTorch when available, else pvx CLI
        p=0.8,
    ),
    GainPerturber(gain_db=(-4, 4), p=0.8),
    AddNoise(snr_db=(25, 45), noise_type="white", p=0.3),
], seed=42)
```

CLI , offline generation across a tempo range:

```
for rate in 0.80 0.90 1.00 1.10 1.20; do
    pvxvoc drums.wav \
        --stretch $rate \
        --preset drums_safe \
        --transient-mode wsola \
        --output "drums_${rate}.wav"
done
```

18.8 Pitch Estimation and Melody Extraction

Goal: Train f0 estimators robust to background noise, vibrato, and pitch transpositions.

```
from pvx.augment import Pipeline, GainPerturber, AddNoise, EQPerturber

pitch_pipeline = Pipeline([
    GainPerturber(gain_db=(-4, 4), p=0.8),
    EQPerturber(n_bands=4, gain_db_range=(-6, 6), p=0.5),
    AddNoise(snr_db=(15, 40), noise_type="pink", p=0.6),
], seed=42)

# For pitch shift augmentation use pvx voc (label must shift with pitch)
\index{For pitch shift augmentation use pvx voc (label must shift with pitch)}
# pvxvoc vocal.wav --stretch 1.0 --pitch 3 --output vocal_up3.wav
\index{pvxvoc vocal.wav --stretch 1.0 --pitch 3 --output vocal_up3.wav}
```

Offline pitch-shift augmentation with label tracking:

```
for semitones in -4 -2 0 2 4; do
    pvxvoc vocal.wav \
        --stretch 1.0 \
        --pitch $semitones \
        --preset vocal_studio \
        --output "vocal_pitch${semitones}.wav"
    echo "$semitones" > "vocal_pitch${semitones}_label.txt"
done
```

18.9 Audio Event Detection / Sound Classification

Goal: Classify acoustic events (dogs barking, glass breaking, gunshots, etc.) in diverse background conditions.

```
from pvx.augment import (
    Pipeline, GainPerturber, AddNoise, BackgroundMixer,
    RoomSimulator, SpecAugment, Fade, TimeShift
)

sed_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=0.9),
    Fade(fade_in=(0.0, 0.1), fade_out=(0.0, 0.1), p=0.3),
    TimeShift(shift=(-0.2, 0.2), p=0.4),
    BackgroundMixer("backgrounds/", snr_db=(5, 20), p=0.6),
    RoomSimulator(rt60_range=(0.1, 1.5), wet_range=(0.1, 0.6), p=0.4),
    SpecAugment(freq_mask_param=30, time_mask_param=60, p=0.5),
], seed=42)
```

18.10 Self-Supervised / Contrastive Learning

Goal: Learn audio representations without labels using contrastive objectives (SimCLR, MoCo, BYOL, VICReg, data2vec).

Two independently augmented views of the same clip form a positive pair.

```
from pvx.augment import contrastive_pipeline
from pvx.integrations.huggingface import HFAugmentMapper

# Two strongly-correlated but statistically independent views
\index{Two strongly-correlated but statistically independent views}
pipeline_a, pipeline_b = contrastive_pipeline(seed=42)

# HuggingFace
\index{HuggingFace}
mapper = HFAugmentMapper(
    pipeline_a, # pipeline_b handled internally via generate_pair=True
    generate_pair=True,
    output_prefix="view",
)

ds_ssl = ds.map(mapper, batched=False, with_indices=True, num_proc=8)
# ds_ssl columns: audio, view_a, view_b
\index{ds ssl columns audio view a view b}

# PyTorch training loop
\index{PyTorch training loop}
for batch in loader:
    view_a = batch["view_a"] # (B, T)
    view_b = batch["view_b"] # (B, T)
    z_a = encoder(view_a)
    z_b = encoder(view_b)
    loss = nt_xent(z_a, z_b)
```

Recommended SSL augmentation strength: - p values between 0.4-0.7 per transform - Include at least: gain, room simulation, noise, SpecAugment - Avoid transforms that destroy too much semantic content (extreme bit-crush, strong clipping) - Two views should be recognizably related but perceptually distinct

18.11 Music Source Separation

Goal: Separate music into stems (vocals, drums, bass, other). Augmentation creates harder mixtures and prevents over-fitting to recording conditions.

```
from pvx.augment import Pipeline, GainPerturber, EQPerturber, AddNoise

# Per-stem augmentation (applied before mixing)
\index{Per-stem augmentation (applied before mixing)}
stem_pipeline = Pipeline([
    GainPerturber(gain_db=(-3, 3), p=0.9),
    EQPerturber(n_bands=3, gain_db_range=(-4, 4), p=0.5),
], seed=42)

# During training , augment each stem independently then remix
\index{During training augment each stem independently then remix}
for vocal, drums, bass, other in stems:
    vocal_aug, _ = stem_pipeline(vocal, sr, seed=item_idx * 4 + 0)
    drums_aug, _ = stem_pipeline(drums, sr, seed=item_idx * 4 + 1)
    bass_aug, _ = stem_pipeline(bass, sr, seed=item_idx * 4 + 2)
    other_aug, _ = stem_pipeline(other, sr, seed=item_idx * 4 + 3)
    mixture = vocal_aug + drums_aug + bass_aug + other_aug
    predicted_stems = separator(mixture)
    loss = si_snr_loss(predicted_stems, [vocal_aug, drums_aug, bass_aug, other_aug])
```

18.12 Augmentation Intensity Guide

Strength	Stretch	Pitch	SNR	RT60	Notes
Very mild (preserve label)	0.97-1.03	± 0.5 st	> 30 dB	< 0.3 s	KWS preserve mode
Mild	0.92-1.12	± 1.5 st	20-35 dB	0.1-0.6 s	ASR, SV
Moderate	0.85-1.25	± 3.0 st	10-30 dB	0.2-1.5 s	Music tagging, classification
Strong	0.75-1.40	± 5.0 st	5-25 dB	0.3-2.5 s	SSL, enhancement
Extreme	0.50-2.00	± 7.0 st	0-15 dB	0.5-4.0 s	Robustness stress tests

CHAPTER 19

pvx Pipeline Cookbook

Generated from commit 35e9761 (commit date: 2026-05-25T08:14:42-04:00).

Curated one-line workflows for practical chaining, mastering, microtonal processing, and batch operation.

19.1 Analysis/QA

19.1.1 Quality metrics on processed speech

```
python3 -m pvx.algorithms.analysis_qa_and_automation.pesq_stoi_visqol_quality_metrics clean.wav noisy.wav --output out/qa.json
```

Why: Collects objective quality indicators for regression tracking.

19.2 Automation

19.2.1 A/B report generation

```
python3 scripts/scripts_ab_compare.py --input mix.wav --a-args "--time-stretch 1.1 --transform fft" --b-args "--time-stretch 1.1 --transform dct" --out-dir reports/ab --name fft_vs_dct
```

Why: Creates JSON/Markdown objective reports for fast algorithm and parameter comparisons.

19.2.2 Benchmark matrix sweep

```
python3 scripts/scripts_benchmark_matrix.py --input mix.wav --transforms fft,dft,dct --windows hann,kaiser --n-ffts 1024,2048 --devices cpu --out-dir reports/bench
```

Why: Produces reproducible CSV/JSON runtime matrices across transform/window/FFT combinations.

19.2.3 Quality regression check

```
python3 scripts/scripts_quality_regression.py --input mix.wav --output out/reg.wav --render-args "--time-stretch 1.2 --transform fft" --baseline-json reports/baseline.json --report-json reports/regression.json
```

Why: Compares current renders against baseline objective metrics with configurable tolerances.

19.3 Batch

19.3.1 Batch stretch over folder

```
pvx voc stems/*.wav --time-stretch 1.08 --output-dir out/stems --overwrite
```

Why: Applies consistent transform to many files with one command.

19.3.2 Dry-run output validation

```
pvx denoise takes/*.wav --reduction-db 8 --dry-run --output-dir out/preview
```

Why: Checks filename resolution and collisions without writing audio.

19.4 Mastering

19.4.1 Integrated loudness targeting with limiter

```
pvx voc mix.wav --time-stretch 1.0 --target-lufs -14 --compressor-threshold-db -20 --compressor-ratio 3 --  
limiter-threshold 0.98 --output-dir out --suffix _master
```

Why: Combines dynamics and loudness controls in shared mastering chain.

19.4.2 Soft clip and hard safety ceiling

```
pvx harmonize bus.wav --intervals 0,7,12 --mix 0.35 --soft-clip-level 0.92 --soft-clip-type tanh --hard-clip  
-level 0.99 --output-dir out
```

Why: Adds saturation while enforcing a strict final peak ceiling.

19.5 Microtonal

19.5.1 Custom cents map retune

```
pvx retune vox.wav --root 60 --scale-cents 0,90,204,294,408,498,612,702,816,906,1020,1110 --strength 0.8 --  
output-dir out
```

Why: Maps incoming notes to a custom 12-degree microtonal scale.

19.5.2 Conform CSV with per-segment ratios

```
pvx conform solo.wav map_conform.csv --pitch-mode ratio --output-dir out --suffix _conform
```

Why: Applies timeline-specific time and pitch trajectories from CSV.

19.6 Phase-vocoder core

19.6.1 Moderate vocal stretch with formant preservation

```
pvx voc vocal.wav --time-stretch 1.15 --pitch-mode formant-preserving --output-dir out --suffix _pv
```

Why: Retains speech-like vowel envelope while stretching timing.

19.6.2 Independent cents retune

```
pvx voc lead.wav --pitch-shift-cents -23 --time-stretch 1.0 --output-dir out --suffix _cents
```

Why: Applies precise microtonal offset without tempo change.

19.6.3 Extreme stretch with multistage strategy

```
pvx voc ambience.wav --target-duration 600 --ambient-preset --n-fft 16384 --win-length 16384 --hop-size 2048
--window kaiser --kaiser-beta 18 --output-dir out --suffix _ambient600x
```

Why: PaulStretch-style ambient profile for very large ratios using stochastic phase and onset time-credit controls.

19.6.4 Ultra-smooth speech stretch (600x)

```
pvx voc speech.wav --target-duration 600 --stretch-mode standard --phase-engine propagate --phase-locking
identity --n-fft 8192 --win-length 8192 --hop-size 256 --window hann --normalize peak --peak-dbfs -1 --
compressor-threshold-db -30 --compressor-ratio 2.0 --compressor-attack-ms 25 --compressor-release-ms
250 --compressor-makeup-db 4 --limiter-threshold 0.95 --output-dir out --suffix _speech600x
```

Why: Prefers continuity and intelligibility over texture animation; avoids choppy stochastic artifacts on speech sources.

19.6.5 Auto-profile plan preview

```
pvx voc input.wav --auto-profile --auto-transform --explain-plan
```

Why: Prints the resolved profile/config plan before long renders.

19.6.6 Multi-resolution fusion stretch

```
pvx voc input.wav --multires-fusion --multires-ffts 1024,2048,4096 --multires-weights 0.2,0.35,0.45 --time-
stretch 1.4 --output-dir out --suffix _multires
```

Why: Blends several FFT scales to reduce single-resolution bias on complex program material.

19.6.7 Checkpointed long render with manifest

```
pvx voc long.wav --time-stretch 12 --auto-segment-seconds 0.5 --checkpoint-dir checkpoints --manifest-json
reports/run_manifest.json --output-dir out --suffix _long
```

Why: Caches segment renders for resume workflows and writes run metadata for reproducibility.

19.7 Pipelines

19.7.1 Time-stretch -> denoise -> dereverb in one pipe

```
pvx voc input.wav --time-stretch 1.25 --stdout | pvx denoise - --reduction-db 10 --stdout | pvx deverb - --strength 0.45 --output-dir out --suffix _clean
```

Why: Single-pass CLI chain for serial DSP in Unix pipes.

19.7.2 Morph -> formant -> unison

```
pvx morph a.wav b.wav -o - | pvx formant - --mode preserve --stdout | pvx unison - --voices 5 --detune-cents 8 --output-dir out --suffix _morph_stack
```

Why: Builds a richer timbre chain with no intermediate files.

19.7.3 Pitch-follow sidechain map (A controls B)

```
pvx pitch-track A.wav | pvx voc B.wav --pitch-follow-stdin --pitch-conf-min 0.75 --pitch-lowconf-mode hold --time-stretch-factor 1.0 --output output.wav
```

Why: Tracks F0 contour from source A and applies it as a dynamic pitch-ratio control map on source B.

19.8 Spatial

19.8.1 VBAP adaptive panning via algorithm dispatcher

```
python3 -m pvx.algorithms.spatial_and_multichannel.imaging_and_panning.vbap_adaptive_panning input.wav --output-channels 6 --azimuth-deg 35 --width 0.8 --output out/vbap.wav
```

Why: Demonstrates algorithm-level spatial module invocation.

19.9 Transform selection

19.9.1 Default production backend (FFT + transient protection)

```
pvx voc mix.wav --transform fft --time-stretch 1.07 --transient-preserve --phase-locking identity --output-dir out --suffix _fft
```

Why: Use when you need the fastest and most stable general-purpose phase-vocoder path.

19.9.2 Reference Fourier baseline using explicit DFT mode

```
pvx voc tone_sweep.wav --transform dft --time-stretch 1.00 --pitch-shift-semitones 0 --output-dir out --suffix _dft_ref
```

Why: Useful for parity checks and controlled transform-comparison experiments.

19.9.3 Prime-size frame experiment with CZT backend

```
pvx voc archival_take.wav --transform czt --n-fft 1531 --win-length 1531 --hop-size 382 --time-stretch 1.03
--output-dir out --suffix _czt
```

Why: Alternative numerical path for awkward/prime frame sizes when validating edge cases.

19.9.4 DCT timbral compaction for smooth harmonic material

```
pvx voc strings.wav --transform dct --pitch-shift-cents -18 --soft-clip-level 0.95 --output-dir out --suffix
_dct
```

Why: Real-basis coefficients can emphasize envelope-like structure for creative reshaping.

19.9.5 DST odd-symmetry color pass

```
pvx voc snare_loop.wav --transform dst --time-stretch 0.92 --phase-locking off --output-dir out --suffix
_dst
```

Why: Provides an alternate real-basis artifact profile useful for creative percussive processing.

19.9.6 Hartley real-basis exploratory render

```
pvx voc synth_pad.wav --transform hartley --time-stretch 1.30 --pitch-shift-semitones 3 --output-dir out --
suffix _hartley
```

Why: Compares Hartley-domain behavior against complex FFT phase-vocoder output.

19.9.7 A/B sweep of transform backends from shell loop

```
for t in fft dft czt dct dst hartley; do pvx voc voice.wav --transform "$t" --time-stretch 1.1 --output-dir
out --suffix "_$t"; done
```

Why: Fast listening workflow for selecting the least-artifact transform on your source.

19.10 Notes

This section begins with the following points, which establish the context for the details that follow.

- Use `--stdout/-` to chain tools without intermediate files.
- Add `--quiet` for script-driven runs; use default verbosity for live progress bars.
- For production mastering, validate true peaks and loudness after all nonlinear stages.

Part VI

Operations and Delivery

CHAPTER 20

pvx Benchmarks

Generated from commit `35e9761` (commit date: `2026-05-25T08:14:42-04:00`).

Reproducible benchmark summary for core short-time Fourier transform/inverse short-time Fourier transform (STFT/ISTFT) path across central processing unit/Compute Unified Device Architecture/Apple-Silicon-native contexts.

20.1 Reproduce

```
python3 scripts/scripts_generate_docs_extras.py --run-benchmarks
```

20.2 Benchmark Spec

This section begins with the following points, which establish the context for the details that follow.

- Sample rate: 48000 Hz
- Duration: 4.0 s
- Signal suite size: 4
- Signal recipe: deterministic synthetic suite: tonal, speech-like, transient stereo, chirp/texture stereo
- STFT config: `n_fft=2048`, `win_length=2048`, `hop_size=512`, `window=hann`, `center=True`

Signal	Channels	Duration (s)	Description
tonal_ramp_mono	1	4.000	Deterministic harmonic blend with slow ramp.
speech_like_mono	1	4.000	Speech-like harmonic formants with sparse glottal-like impulses.
transient_drums_stereo	2	4.000	Percussive burst train for onset retention and stereo timing drift.
chirp_texture_stereo	2	4.000	Wideband chirp-plus-texture to stress spectral and phase stability.

20.3 Host

This section begins with the following points, which establish the context for the details that follow.

- Platform: `macOS-15.1.1-arm64-arm-64bit-Mach-0`
- Machine: `arm64`
- Python: `3.14.3`

20.4 Backend Summary

Backend	Status	Cases (ok/- to- StdDev)	Mean xRT (ms)	Peak	Mean	Mean dry score (/100)	Mean SNR (dB)	Mean SI- SDR (dB)	Mean Mean LSD (dB)	Mean Mod- Spec	Mean Smearv	Mean	Mean	Mean	Mean	Notes	
				host	qual-							Co-	Mu-	SNR	LSD		
				mem-	ity							in	SI-	Mean	Mean		her-
cpu	ok	4/4	146.33	1.02	17.70	97.50	100.98	153.81	0.0000	0.0000	0.0000	1.0000	0.0000	0.0511	0.0000	n/a	n/a
cuda	unavailable	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.
apple_silicon	ok	4/4	147.28	1.02	17.70	97.50	100.98	153.81	0.0000	0.0000	0.0000	1.0000	0.0000	0.0511	0.0000	100.98	0.0000

20.5 Per-Signal Results

Backend	Signal	Channels	Duration (s)	Status	xRT (ms)	Mean Quality (/100)	Mean SNR (dB)	Mean SI- SDR (dB)	Mean Mean LSD (dB)	Mean Mod- Spec	Mean Smearv	Mean EnvCorr	Mean Coherence Drift	Mean Phase ness	Mean Musical Noise (dB)	Mean CPU vs (dB)	Mean CPU LSD (dB)	Notes
apple_silicon	chirp_texture_stereo	2	4.000	ok	96.99	41.24	100.00	102.30	155.13	0.0000	0.0000	0.0000	1.0000	0.0000	0.0000	102.30	0.0000	
apple_silicon	speech_like_mono	1	4.000	ok	97.60	20.24	100.00	99.92	152.75	0.0000	0.0000	0.0000	1.0000	n/a	0.0000	99.92	0.0000	
apple_silicon	tonal_ramp_mono	1	4.000	ok	95.35	20.48	100.00	107.76	160.59	0.0000	0.0000	0.0000	1.0000	n/a	0.0000	107.76	0.0000	
apple_silicon	transient_drums_stereo	2	4.000	ok	99.12	40.36	90.00	93.93	146.76	0.0000	0.0000	0.0000	1.0000	0.0000	0.2042	93.93	0.0000	
cpu	chirp_texture_stereo	2	4.000	ok	93.13	42.95	100.00	102.30	155.13	0.0000	0.0000	0.0000	1.0000	0.0000	0.0000	n/a	n/a	
cpu	speech_like_mono	1	4.000	ok	98.94	20.11	100.00	99.92	152.75	0.0000	0.0000	0.0000	1.0000	n/a	0.0000	n/a	n/a	
cpu	tonal_ramp_mono	1	4.000	ok	194.70	20.54	100.00	107.76	160.59	0.0000	0.0000	0.0000	1.0000	n/a	0.0000	n/a	n/a	
cpu	transient_drums_stereo	2	4.000	ok	98.77	40.50	90.00	93.93	146.76	0.0000	0.0000	0.0000	1.0000	0.0000	0.2042	n/a	n/a	

Backend	Signal	Channels	Duration (s)	Status	Elapse RT (ms)	SNR				SI-				Coherence				SNR		Notes	
						Quality (/100)	(dB)	SDR	LSD	Mod	Spear	Env	CD	Drift	Phase	Noise	Music	CPU	vs		vs
cuda	chirp_texture_stereo	2	4.000	unavailable	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.		
cuda	speech_like_mono	1	4.000	unavailable	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.		
cuda	tonal_ramp_mono	1	4.000	unavailable	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.		

Backend	Signal	Channels	Duration (s)	Status	Elapsed RT (ms)	Quality (/100)				SNR SI-SDR LSD				Coherence Musical Noise				SNR LSD vs		Notes
						Quality	SI-SDR	LSD	ModSpec	Smear	EnvCorr	Coherence	Musical	Noise	CPU	CPU				
cuda	transient_drums_stereo	2	4.000	unavailable	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.		

Interpretation notes: - **xRT** (times real-time): higher is faster. - **Quality** (/100) is a composite from core artifact metrics; use it as a quick ranking aid, not as a single acceptance criterion. - Lower is better for LSD, ModSpec, Smear, Coherence Drift, Phasiness, and Musical Noise. - Higher is better for SNR, SI-SDR, and Envelope Correlation.

Raw machine-readable benchmark output: `docs/benchmarks/latest.json`.

CHAPTER 21

Installing and Managing pvx with Homebrew

Homebrew is the supported installation path for macOS users who want the **pvx** command suite without managing a project checkout or Python virtual environment. The formula is published through the **TheColby/pvx** tap because Homebrew requires third-party formulae to live in a tap. Installing **Formula/pvx.rb** directly from a local path is not supported by current Homebrew releases.

21.1 Stable Installation

The stable formula installs the tagged alpha release. Homebrew fetches the release archive, verifies its SHA-256 checksum, installs **libsndfile**, creates an isolated Python environment, and links the command-line programs into the Homebrew prefix.

```
brew tap TheColby/pvx
brew install TheColby/pvx/pvx
```

The tap command only needs to be run once. After installation, verify both Homebrew's record and the primary command:

```
brew list --versions pvx
pvx --help
pvxvoc --help
```

Homebrew 6 requires trust for third-party tap content. Homebrew can trust the selected formula during installation; when it instead reports that the tap is not trusted, grant trust only to the **pvx** formula and retry:

```
brew trust --formula TheColby/pvx/pvx
brew install TheColby/pvx/pvx
```

Formula-level trust is narrower than trusting the entire tap. Inspect the formula in the tap repository before granting trust, particularly on a shared or security-sensitive machine.

The formula installs every command entry point shipped by the Python package. The alpha support promise remains deliberately narrower: **pvx**, **pvxvoc**, **pvxfreeze**, **pvxwarp**, **pvxformant**, **pvxfilter**, **pvxre-tune**, and **pvxanalysis** are the stable command-line surface documented for production scripts.

21.2 Development Installation

The **HEAD** formula follows the latest commit on the repository's **main** branch. It is useful for testing an unreleased fix, but its behavior can move ahead of the guide and tagged release.

```
brew install --HEAD TheColby/pvx/pvx
```

Homebrew does not automatically replace a stable installation with HEAD. Reinstall explicitly when switching tracks:

```
brew uninstall pvx
brew install --HEAD TheColby/pvx/pvx
```

Return to the stable release in the same way:

```
brew uninstall pvx
brew install TheColby/pvx/pvx
```

21.3 Upgrading

Homebrew updates the tap metadata before upgrading the package. These commands show the available formula and install a newer tagged version when one has been published:

```
brew update
brew info TheColby/pvx/pvx
brew upgrade pvx
```

A HEAD installation is rebuilt from the latest repository state with:

```
brew reinstall --HEAD TheColby/pvx/pvx
```

Existing audio files, render outputs, presets, and manifests are not stored inside the Homebrew cellar and are not removed by an upgrade.

21.4 Uninstalling

Uninstalling removes the Homebrew-managed virtual environment and linked commands. It does not delete projects or rendered audio in user directories.

```
brew uninstall pvx
```

Remove the tap as well when no formula from it is needed:

```
brew untap TheColby/pvx
```

21.5 What the Formula Installs

The formula uses Homebrew's `Language::Python::Virtualenv` support. `pvx` and its Python runtime dependencies live under the formula's private `libexec` directory, while small launcher scripts are linked into `$(brew --prefix)/bin`. This prevents `pvx` packages from modifying the system Python or an activated project environment.

`libsndfile` is a Homebrew dependency because `pvx` uses it for audio file input and output through the Python `soundfile` package. The stable formula currently uses Homebrew's `python@3.12`. Optional machine-learning integrations such as PyTorch and TensorFlow are intentionally excluded from the base formula because of their size and platform-specific installation requirements.

The formula also installs available manual pages from `man/man1`. Homebrew exposes them through its normal manual-page path, so a supported command can be inspected with a command such as:


```
man pvx
```

21.6 Shell Path

Homebrew normally configures its binary directory during Homebrew installation. If the **brew** command works but **pvx** is not found after a successful install, inspect the active prefix and shell path:

```
brew --prefix
command -v brew
command -v pvx
printf '%s\n' "$PATH"
```

Apple Silicon Homebrew normally uses `/opt/homebrew`; Intel Homebrew normally uses `/usr/local`. Homebrew's recommended shell environment can be loaded on Apple Silicon with:

```
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Place the corresponding **brew shellenv** command in the shell startup file only when Homebrew itself is not already configuring the path.

21.7 Troubleshooting

A failed install is easiest to diagnose by separating tap state, formula state, dependencies, and command linking. Begin with the following inspection commands:

```
brew update
brew tap
brew info TheColby/pvx/pvx
brew doctor
```

If Homebrew reports that a local `Formula/pvx.rb` is rejected, install through the tap instead:

```
brew tap TheColby/pvx
brew install TheColby/pvx/pvx
```

If an interrupted build left partial state, remove and reinstall the formula:

```
brew uninstall --force pvx
brew cleanup pvx
brew install TheColby/pvx/pvx
```

If Homebrew reports that the Xcode Command Line Tools are outdated, update them through System Settings under General and Software Update. Homebrew may require a newer Command Line Tools release even when the `clang` compiler already works for ordinary local builds. Complete that system update before retrying the `pvx` formula.

If audio files fail to open after installation, confirm that Homebrew installed and linked `libsndfile`, then run the `pvx` smoke test:

```
brew list --versions libsndfile
pvx smoke
```

The full Homebrew build log is available through Homebrew’s usual diagnostic output. Include `brew config`, `brew info TheColby/pvx/pvx`, the failing command, and the relevant log tail in a bug report.

21.8 Maintainer Release Flow

The repository keeps the source formula at `Formula/pvx.rb`. A tagged release must have a stable archive URL, its exact SHA-256 checksum, and an explicit version before the formula is copied to the tap. The refresh helper is repeatable, so it can update both a previously stamped formula and a newly prepared release.

```
./scripts/refresh_homebrew_formula.sh v0.1.0a1
ruby -c Formula/pvx.rb
python3 scripts/scripts_sync_homebrew_tap_formula.py ../homebrew-pvx
```

The tap checkout should then be tested as Homebrew sees it:

```
brew style TheColby/pvx/pvx
brew audit --strict TheColby/pvx/pvx
brew install --build-from-source TheColby/pvx/pvx
brew test TheColby/pvx/pvx
```

After a successful test, commit and push `Formula/pvx.rb` in `TheColby/homebrew-pvx`. The tag-driven release workflow also validates Ruby syntax and uploads the stamped formula beside the Python distributions, providing one release artifact from which the tap can be synchronized.

21.9 Formula Version Policy

The stable formula follows tagged pvx releases and never points at a moving branch. `HEAD` is the only branch-following installation. This separation keeps normal `brew upgrade` behavior reproducible while retaining an explicit route for development testing.

The Homebrew formula is a distribution mechanism, not a wider stability promise. Alpha, beta, and experimental command tiers remain governed by the supported-surface chapter and release notes. A successful installation confirms packaging and executable discovery; it does not promote experimental commands into the stable interface.

CHAPTER 22

Alpha Release Guide

This page is the release-facing contract for `pvx` alpha builds.

22.1 Supported Surface

Stable entry points for 0.1.0a1: - `pvx` - `pvxvoc` - `pvxfreeze` - `pvxwarp` - `pvxformant` - `pvxfilter` - `pvxtune` - `pvxanalysis`

Beta entry points: - functional and documented, but flags may still move in minor releases

Experimental entry points: - useful for exploration - not promised stable for scripting

Compatibility shims: - `pvxalgorithms`, `pvxalgorithms.base`, and `pvxalgorithms.registry` remain available only as deprecated migration shims

22.2 Release Checklist

Before tagging an alpha build:

1. Run `uv run python scripts/scripts_alpha_check.py` Optional convenience wrapper: `make alpha-check` when local `make` is available
2. Run `uv run pytest -q`
3. Run `uv run python benchmarks/run_augment_profile_suite.py --quick --gate --out-dir benchmarks/out_augment_profiles_release`
4. Confirm `README.md`, `RELEASE.md`, and `pyproject.toml` agree on the stable/beta/experimental command surface
5. If publishing a tagged formula, run `./scripts/refresh_homebrew_formula.sh v0.1.0a1`, sync it with `python3 scripts/scripts_sync_homebrew_tap_formula.py ../homebrew-pvx`, publish the refreshed formula to `TheColby/homebrew-pvx`, and verify the tap instructions still match `docs/HOMEBREW.md`
6. Regenerate source docs with `make docs`
7. Regenerate HTML/man outputs only when user-facing rendered docs changed or when preparing the tag with `make docs-generated`
8. Make sure deprecated compatibility layers still warn cleanly and still route to the canonical modules

22.3 Alpha Principles

This section begins with the following points, which establish the context for the details that follow.

- prefer a smaller honest surface over a wider unstable one
- keep compatibility shims explicit and noisy rather than magical
- publish only the supported surface as the alpha contract; everything else is docs/reference, not a promise
- keep the supported-slice coverage gate at 100%
- treat docs source files as the review artifact; generated outputs are release artifacts
- add direct seam tests when extracting modules from `pvx.py` or `pvx.core.voc`

CHAPTER A

pvx Command-Line Interface (CLI) Flags Reference

Generated from commit 35e9761 (commit date: 2026-05-25T08:14:42-04:00).

This file enumerates long-form CLI flags discovered from argparse declarations in canonical pvx CLI sources.

Total tool+flag entries: **476** Total unique long flags: **338**

A.1 Unique Long Flags

--a4-reference-hz, --alpha, --ambient-phase-mix, --ambient-preset, --amplitude, --analysis-channel, --append-manifest, --attack-sec, --attenuation, --audit-metrics, --auto-profile, --auto-profile-lookahead-seconds, --auto-segment-seconds, --auto-transform, --backend, --background-dir, --band, --band-gain-db, --band-width-bins, --batch-size, --bit-depth, --bits, --blend-mode, --boost-db, --budget-path, --cache-dir, --category, --center, --cents, --checkpoint-dir, --checkpoint-id, --chord, --chunk-ms, --chunk-seconds, --clip, --codec, --coherence-strength, --comp-ratio, --comp-threshold-db, --compander-attack-ms, --compander-compress-ratio, --compander-expand-ratio, --compander-makeup-db, --compander-release-ms, --compander-threshold-db, --compressor-attack-ms, --compressor-makeup-db, --compressor-ratio, --compressor-release-ms, --compressor-threshold-db, --confidence-floor, --context-ms, --control-stdin, --coord-system, --cpu, --crossfade-ms, --cuda-device, --cycles, --decay, --decay-sec, --dedupe-by, --depth, --detune-cents, --device, --disk-budget, --distance-law, --dither, --dither-seed, --drr, --dry, --dry-mix, --dry-run, --duration, --duty-cycle, --emit, --end, --engine, --envelope-lifter, --example, --exp-curve, --expand-ratio, --expander-attack-ms, --expander-ratio, --expander-release-ms, --expander-threshold-db, --explain-plan, --exponent, --extreme-stretch-threshold, --extreme-time-stretch, --f0-max, --f0-min, --factor, --fail-if-exceeds, --feature-set, --feedback, --fill, --floor, --fmax, --fmin, --force, --force-stereo, --formant-lifter, --formant-max-gain-db, --formant-shift-ratio, --formant-strength, --format, --fourier-sync, --fourier-sync-max-fft, --fourier-sync-min-fft, --fourier-sync-smooth, --frame-length, --freeze-time, --freq, --freq-mask, --frequency-hz, --gain, --gains, --gpu, --group-separator, --grouping, --guided, --hard-clip-level, --harmonic-gain, --harmonic-kernel, --harmonic-pitch-cents, --harmonic-pitch-semitones, --harmonic-stretch, --hop-length, --hop-size, --inharmonic-f0-hz, --inharmonic-mix, --inharmonic-pitch, --input-format, --intent, --interp, --intervals, --intervals-cents, --ir, --ir-database, --ir-dir, --ir-file, --json, --kaiser-beta, --keep-intermediate, --key, --label-policy, --labels-csv, --limiter-threshold, --list-databases, --manifest-append, --manifest-csv, --manifest-json, --manifest-jsonl, --map, --mask-exponent, --material, --max, --max-length-s, --max-stage-stretch, --metadata-policy, --method, --mfcc-count, --min, --mix, --mode, --multires-ffts, --multires-fusion, --multires-weights, --n-fft, --no-audit-metrics, --no-center, --no-normalize-gains, --no-onset-realign, --no-progress, --no-trim, --noise-file, --noise-floor, --noise-seconds, --noise-type, --normalize, --normalize-energy, --normalize-gains, --num-freq-masks, --num-time-masks, --offset, --onset-credit-max, --onset-credit-pull, --onset-time-credit, --operation, --operator, --order, --out,

```
--output, --output-csv, --output-dir, --output-format, --output-jsonl, --output-key, --output-subtype, --output-suffix, --overlap-ms, --overwrite, --pair-mode, --pans, --peak, --peak-count, --peak-dbfs, --percussive-gain, --percussive-kernel, --percussive-pitch-cents, --percussive-pitch-semitones, --percussive-stretch, --phase, --phase-engine, --phase-locking, --phase-mix, --phase-mode, --phase-rad, --phase-random-seed, --pipeline, --pipeline-config, --pitch, --pitch-conf-min, --pitch-follow-stdin, --pitch-lowconf-mode, --pitch-map, --pitch-map-crossfade-ms, --pitch-map-smooth-ms, --pitch-map-stdin, --pitch-mode, --pitch-shift-cents, --pitch-shift-ratio, --pitch-shift-semitones, --policy, --pre-delay-ms, --preset, --quality-profile, --quiet, --random-phase, --rate, --ratio, --ratio-max, --ratio-min, --ratio-reference, --recommend-root, --reduction-db, --ref-channel, --reference-hz, --release-sec, --requested-stretch, --resample-mode, --resonance-decay, --resonance-hz, --resonance-mix, --resonance-q, --response, --response-gain-db, --response-mix, --resume, --rms-dbfs, --root, --root-hz, --route, --rt60, --safety-margin, --sample-rate, --scale, --scale-cents, --seed, --semitones, --shift-bins, --silent, --sine-cycles, --sine-phase-rad, --smooth, --smooth-frames, --smoothing-bins, --snr, --soft-clip-drive, --soft-clip-level, --soft-clip-type, --speaker-angles, --split, --split-mode, --sr, --start, --stdout, --stereo-mode, --strength, --stretch, --stretch-from, --stretch-max, --stretch-min, --stretch-mode, --stretch-scale, --strict, --strict-manifest, --subtype, --suffix, --summary-json, --sustain, --target, --target-duration, --target-f0, --target-lufs, --target-max, --target-min, --target-pitch-shift-semitones, --target-sample-rate, --time-mask, --time-stretch, --time-stretch-factor, --tolerance-cents, --trajectory-shape, --transform, --transient-crossfade-ms, --transient-mode, --transient-preserve, --transient-protect-ms, --transient-sensitivity, --transient-threshold, --transpose-semitones, --true-peak-max-dbt, --tv-interp, --tv-key, --tv-map, --tv-order, --variants-per-input, --verbose, --verbosity, --voices, --wave, --wet, --width, --win-length, --window, --work-dir, --workers
```

A.2 hps-pitch-track

Flag	Required	Default	Choices	Action	Description	Source
--backend	False	auto	auto, pyin, acf	Pitch backend (default: auto -> pyin if available, else acf)	Set confidence below this floor to 0.0 (default: 0.0).	Output mode: pitch_map (default), stretch_map, or pitch_to_stretch.
				`src/pvx/- cli/hps_pitch_track.py`	`src/pvx/- cli/hps_pitch_track.py`	
				`-- confidence- floor`		cli/hps_pitch_track.py`
				False	`--emit`	`-- feature-
				False	False	set`
				`0.0`	`pitch_map`	False
						False
						`all`
						`none,
						basic,
						advanced,
						all`

Flag	Required	Default	Choices	Action	Description	Source
<code>--fmax</code>	False	1200.0		Maximum F0 in Hz (default: 1200)	<code>src/pvx/cli/hps_pitch_track.py</code>	
<code>--fmin</code>	False	50.0		Minimum F0 in Hz (default: 50)	<code>src/pvx/cli/hps_pitch_track.py</code>	
<code>--frame-length</code>	False	2048		Frame length in samples (default: 2048)	<code>src/pvx/cli/hps_pitch_track.py</code>	
<code>--hop-size</code>	False	256		Hop size in samples (default: 256)	<code>src/pvx/cli/hps_pitch_track.py</code>	
<code>--mfcc-count</code>	False	13		Number of MFCC columns (mfcc_01..mfcc_N) when feature-set is advanced/all (default: 13).	<code>src/pvx/cli/hps_pitch_track.py</code>	
<code>--output</code>	False		Output CSV path (default: '-' for stdout) <code>`src/pvx/cli/hps_pitch_track.py`</code> <code>--ratio-max`</code>   False   <code>`4.0`</code>	Upper clamp for emitted pitch_ratio (default: 4.0). <code>--ratio-min`</code>   False   <code>`0.25`</code>	Lower clamp for emitted pitch_ratio (default: 0.25). <code>--ratio-reference`</code>   False   <code>`median`</code>   <code>`median, mean, first, hz`</code>	Reference for emitted pitch_ratio values (default: median voiced f0). <code>src/pvx/cli/hps_pitch_track.py`</code>

Flag	Required	Default	Choices	Action	Description	Source
<code>--reference-hz</code>	False		 Reference frequency in Hz when <code>--ratio-reference-hz</code> `src/pvx/-cli/hps_pitch_track.py` `--smooth-frames` False False `5`	 Smoothing window for <code>pitch_ratio</code> frames (default: 5). `src/pvx/-cli/hps_pitch_track.py` `--stretch` False `1.0`	Emit constant stretch column value for stretch-emit oriented emit modes (default: <code>pitch_ratio</code>). `src/pvx/-cli/hps_pitch_track.py` `--stretch-` False stretch- from`   False   `pitch_ratio`max` False `4.0`	Source signal used to derive stretch in stretch-oriented emit modes (default: <code>pitch_ratio</code>). `src/pvx/-cli/hps_pitch_track.py` `--stretch-` False stretch- from`   False   `pitch_ratio`max` False `4.0`

A.3 pvx

Flag	Required	Default	Choices	Action	Description	Source
<code>--append-manifest</code>	False		<code>store_true</code>	Append/merge with existing manifest instead of replacing it	<code>src/pvx/cli/pvx_augment.py</code>	
<code>--audit-metrics</code>	False	True	 `store_true` Compute per-output audit metrics (default: on) `src/pvx/-cli/pvx_augment.py` `--backend` False `auto` `auto`, pyin, acf` 	Pitch tracker backend	<code>src/pvx/cli/pvx_pipeline.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--batch-size</code>	False	16		Number of files processed simultaneously on GPU (default: 16)	<code>src/pvx/cli/pvx_augment.py</code>	
<code>--bit-depth</code>	False	inherit	inherit, 16, 24, 32f	 Bit-depth assumption when <code>--subtype</code> is not set (default: inherit from input subtype). <code>`src/pvx/-cli/pvx_helpers.py`</code> <code>`--budget-path`</code> False		Path used to query free space when <code>--disk-budget</code> is omitted (default: current directory).
<code>--chunk-seconds</code>	False	0.25		Chunk/segment duration for <code>--auto-segment-seconds</code> (default: 0.25)	<code>src/pvx/cli/pvx_pipeline.py</code>	
<code>--confidence-floor</code>	False	0.0		Minimum tracker confidence	<code>src/pvx/cli/pvx_pipeline.py</code>	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--context-ms</code>	False		Optional stateful context window in millisec- onds (default: auto from win- dow/hop) `src/pvx/- cli/pvx/- cli/pvx_pipeline.py` `-- crossfade- ms` False `0.0`	 Crossfade used for segment assembly in mil- liseconds (default: 0.0) `src/pvx/- cli/pvx_pipeline.py` dedupe-by`   False   `out- put_path` 	Dedupli- cation key (default: out- put_path) `src/pvx/- cli/pvx_augment.py` `--device` False `auto` `auto`, cuda, mps, cpu`	Compute device (default: auto , CUDA > MPS > CPU)
<code>--disk-budget</code>	False		Total budget size (e.g., 500MB, 20GB, 2TiB). If omitted, use free space at --budget- path. `src/pvx/- cli/pvx_helpers.py` `-- dry-run` False	 `store_true` Plan manifest and filenames without rendering audio `src/pvx/- cli/pvx_augment.py` `-- duration` False `0.3`	 Synthetic input duration seconds (default: 0.30) `src/pvx/- cli/pvx_helpers.py` False `pitch_to_stretch` `pitch_map`, stretch_map, pitch_to_stretch` 	Control map emit mode for the guide track (default: pitch_to_stretch)

405

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--feature-set</code>	False	all	none, basic, advanced, all	Feature columns emitted by pitch tracker (default: all) `src/pvx/- cli/pvx_pipeline.py` `--fmin` False `1200.0` Tracker hop size in samples	Maximum tracked f0 in Hz `src/pvx/- cli/pvx_pipeline.py` `--frame- length` False `2048`	Minimum tracked f0 in Hz `src/pvx/- cli/pvx_pipeline.py` `--frame- length` False `2048`
<code>--hop-size</code>	False	256			src/pvx/cli/pvx_pipeline.py	
<code>--intent</code>	False	asr_robust	asr_robust, mir_music, ssl_contrastive	Built-in augmentation intent profile when -- pipeline- config is not given (default: asr_robust) `src/pvx/- cli/pvx_augment.py` `--json` False	 `store_true` Emit JSON output `src/pvx/- cli/pvx_augment.py` `--keep- intermediate` (default: False	 `store_true` Keep in- termediate stage `cli/pvx_augment.py` after successful completion `src/pvx/- cli/pvx_pipeline.py` `--label- policy` False `al- low_alter` `al- low_alter, preserve`

Flag	Required	Default	Choices	Action	Description	Source
<code>--labels-csv</code>	False		Optional metadata CSV with path/label/speaker columns for balanced split modes `src/pvx/cli/pvx_augment.py` `--manifest-csv` False		Optional CSV manifest path (default: /augment_manifest.csv)	src/pvx/cli/pvx_augment.py
<code>--manifest-jsonl</code>	False		Optional JSONL manifest path (default: <output-dir>/augment_manifest.jsonl) `src/pvx/cli/pvx_augment.py` `--material` False `mix` `mix, speech, vocal, drums, ambient`	Material profile for pvx safe command generation		src/pvx/cli/pvx_helpers.py
<code>--max-length-s</code>	False	30.0		Maximum file duration in seconds (default: 30.0)		src/pvx/cli/pvx_augment.py
<code>--mfcc-count</code>	False	13		MFCC column count emitted by pitch tracker (default: 13)		src/pvx/cli/pvx_pipeline.py

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--mode</code>	False	<code>stateful</code>	<code>stateful</code> , <code>wrapper</code>	Stream engine: <code>stateful</code> <code>chunk</code> processor (default) or wrapper compati- bility mode	 <code>`store_false`</code> Disable audit metric computa- tion <code>`src/pvx/-`</code> <code>cli/pvx_augment.py`</code>     <code>`--out`</code> <code>cli/pvx_augment.py`</code> <code>`--no-</code> <code>progress`</code> <code>cli/pvx_pipeline.py`</code>     <code>`--no-</code> audit- metrics` False	 <code>`store_true`</code> Suppress progress messages <code>`src/pvx/-`</code> <code>cli/pvx_augment.py`</code>     <code>`--out`</code> <code>cli/pvx_augment.py`</code> <code>`--no-</code> <code>progress`</code> <code>cli/pvx_pipeline.py`</code>     <code>`--no-</code> audit- metrics` False
<code>--output</code>	False	<code>output.wav</code>		Output audio path (default: output.wav)	<code>src/pvx/cli/pvx_helpers.py</code>	
<code>--output-csv</code>	False		Optional merged CSV output path <code>`src/pvx/-`</code> <code>cli/pvx_augment.py`</code>     <code>`--output-</code> <code>dir`</code> True		Output directory for augmented files	<code>src/pvx/cli/pvx_augment.py`</code>
<code>--output-format</code>	False	<code>wav</code>	<code>wav</code> , <code>flac</code> , <code>aiff</code> , <code>ogg</code> , <code>caf</code>	Output format container (default: <code>wav</code>) <code>`src/pvx/-`</code> <code>cli/pvx_augment.py`</code>     <code>`--output-</code> <code>jsonl`</code> True		Merged JSONL output path

Flag	Required	Default	Choices	Action	Description	Source
<code>--output-subtype</code>	False	PCM_16	PCM_16, PCM_24, PCM_32, FLOAT	Output sample format (default: PCM_16) `src/pvx/- cli/pvx_augment.py` `--output- suffix` False `_aug`	Suffix appended before extension (default: _aug) `src/pvx/- cli/pvx_augment.py` `--     `-- overwrite`   False   pair-mode` False `off` `off, con- trastive2` 	`store_true` Overwrite existing outputs `src/pvx/- cli/pvx_augment.py` `--     `-- pair-mode`   False   `off`   `off, con- trastive2`
<code>--pipeline</code>	True		Pipeline string with stages separated by ' '. Example: "voc --stretch 1.2 formant --mode preserve" `src/pvx/- cli/pvx_pipeline.py` `-- pipeline- config` False		Optional YAML/J- SON pipeline manifest (see pvx.augment.config.load_pipeline)	src/pvx/cli/pvx_augment.py
<code>--pitch</code>	False	0.0		Smoke render pitch semitones (default: 0.0)	src/pvx/cli/pvx_helpers.py	
<code>--pitch-conf-min</code>	False	0.75		Minimum accepted map confidence for pvx voc (default: 0.75)	src/pvx/cli/pvx_pipeline.py	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--pitch-lowconf-mode</code>	False	hold	hold, unity, interp	Low-confidence handling mode in pvx voc (default: hold) `src/pvx/-cli/pvx_pipeline.py` `--pitch-map-crossfade-ms` False `20.0`	Map segment crossfade in pvx voc (millisec-onds, default: 20) `src/pvx/-cli/pvx_pipeline.py` `--pitch-map-crossfade-ms` False `0.0`	Additional map smoothing in pvx voc (millisec-onds) `src/pvx/-cli/pvx_pipeline.py` `--policy` False
<code>--quiet</code>	False		store_true	Reduce logs		src/pvx/cli/pvx_augment.py
<code>--ratio-max</code>	False	4.0		Maximum pitch_ratio clamp		src/pvx/cli/pvx_pipeline.py
<code>--ratio-min</code>	False	0.25		Minimum pitch_ratio clamp		src/pvx/cli/pvx_pipeline.py
<code>--ratio-reference</code>	False	median	median, mean, first, hz	 Reference mode for pitch_ratio derivation in tracking `src/pvx/-cli/pvx_pipeline.py` `--reference-hz` False		Reference Hz when -ratio-reference hz

Flag	Required	Default	Choices	Action	Description	Source
<code>--requested-stretch</code>	False		Optional stretch ratio to evaluate against the computed budget. <code>`src/pvx/-cli/pvx_helpers.py`</code> <code>`--resume`</code> False	<code>`store_true`</code> Skip already-rendered outputs found in existing manifest/output- <code>`src/pvx/-cli/pvx_helpers.py`</code> <code>`--route`</code> False <code>`[]`</code>	append	Optional pvx voc control route expression. Repeat to chain. Example: <code>-route stretch=pitch_ratio</code> <code>-route pitch_ratio=const(1.0)</code>
<code>--safety-margin</code>	False	0.9		Usable fraction of budget in (0,1]; default: 0.90 (10%% headroom).	src/pvx/cli/pvx_helpers.py	
<code>--sample-rate</code>	False	24000		Synthetic input sample rate (default: 24000)	src/pvx/cli/pvx_helpers.py	
<code>--seed</code>	False	1337		Deterministic base seed (default: 1337)	src/pvx/cli/pvx_augment.py	
<code>--silent</code>	False		<code>store_true</code>	Suppress logs	src/pvx/cli/pvx_augment.py	
<code>--smooth-frames</code>	False	5		Smoothing window in frames	src/pvx/cli/pvx_pipeline.py	
<code>--split</code>	False	0.8,0.1,0.1		train,val,test split ratios (default: 0.8,0.1,0.1)	src/pvx/cli/pvx_augment.py	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--split-mode</code>	False	random	random, la-bel_balanced, speaker_balanced	Split assignment mode (default: random) `src/pvx/cli/pvx_augment.py` `--sr` False `16000`	Target sample rate (default: 16000) `src/pvx/cli/pvx_augment.py` `--stretch` False `1.25`	Smoke render stretch factor (default: 1.25) `src/pvx/cli/pvx_helpers.py` `--stretch-from` False `pitch_ratio` `pitch_ratio, inv_pitch_ratio, f0_hz`
<code>--stretch-max</code>	False	4.0		Upper clamp for derived stretch	src/pvx/cli/pvx_pipeline.py	
<code>--stretch-min</code>	False	0.25		Lower clamp for derived stretch	src/pvx/cli/pvx_pipeline.py	
<code>--stretch-scale</code>	False	1.0		Scale factor for derived stretch track	src/pvx/cli/pvx_pipeline.py	
<code>--strict</code>	False		store_true	Return non-zero on validation errors	src/pvx/cli/pvx_augment.py	
<code>--strict-manifest</code>	False		store_true	Return non-zero if manifest validation errors are found	src/pvx/cli/pvx_augment.py	

Flag	Required	Default	Choices	Action	Description	Source
<code>--kaiser-beta</code>	False	14.0		Kaiser beta when <code>--window kaiser</code> (default: 14.0)	<code>src/pvx/cli/pvxanalysis.py</code>	
<code>--n-fft</code>	False	2048		STFT FFT size (default: 2048)	<code>src/pvx/cli/pvxanalysis.py</code>	
<code>--no-center</code>	False		<code>store_true</code>	Disable centered framing	<code>src/pvx/cli/pvxanalysis.py</code>	
<code>--output</code>	False		Output PVXAN path (default: <code><input>.pvxan.npz</code>) <code>`src/pvx/cli/pvxanalysis.py`</code> <code>`--summary-json`</code> False		Optional path to write summary JSON	<code>src/pvx/cli/pvxanalysis.py</code>
<code>--transform</code>	False	fft		Transform backend (default: <code>fft</code>)	<code>src/pvx/cli/pvxanalysis.py</code>	
<code>--win-length</code>	False	2048		Window length (default: 2048)	<code>src/pvx/cli/pvxanalysis.py</code>	
<code>--window</code>	False	hann		Window type	<code>src/pvx/cli/pvxanalysis.py</code>	

A.5 pvxcodec

Flag	Required	Default	Choices	Action	Description	Source
--bit-depth	False	24	16, 24, 32, float	Output file bit depth (default: 24) `src/pvx/cli/pvxcodec.py` `--bits` False		Bit depth for raw bit-crushing (4-16). Applied after codec if both specified.
--codec	False	`mp3_low, mp3_medium, telephone, voip_narrow, voip_wide, am_radio, lo-fi, random`	Codec preset to simulate (default: none , use -bits for raw bit-crush)	src/pvx/cli/pvxcodec.py		
--output	True		Output audio file `src/pvx/cli/pvxcodec.py` `--seed` False `0`	‘ Random seed (default: 0) src/pvx/cli/pvxcodec.py‘		

A.6 pvxconform

Flag	Required	Default	Choices	Action	Description	Source
<code>--crossfade-ms</code>	False	8.0		Segment crossfade in milliseconds	<code>src/pvx/cli/pvxconform.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--map</code>	True		CSV map path `src/pvx/-cli/pvx-conform.py` `--resample-mode` False `auto` `auto`, fft, linear`		src/pvx/cli/pvxconform.py	

A.7 pvxdenoise

Flag	Required	Default	Choices	Action	Description	Source
<code>--floor</code>	False	0.1		Noise floor multiplier	src/pvx/cli/pvxdenoise.py	
<code>--noise-file</code>	False		Optional external noise reference `src/pvx/-cli/pvxdenoise.py` `--noise-seconds` False `0.35`	Noise profile duration from start `src/pvx/-cli/pvxdenoise.py` `--reduction-db` False `12.0`	Reduction strength in dB `src/pvx/-cli/pvxdenoise.py` `--smooth` False `5`	‘ Temporal smoothing frames src/pvx/cli/pvxdenoise.py‘

A.8 pvxdeverb

Flag	Required	Default	Choices	Action	Description	Source
<code>--decay</code>	False	0.92		Tail memory decay 0..1	src/pvx/cli/pvxdeverb.py	
<code>--floor</code>	False	0.12		Per-bin floor multiplier	src/pvx/cli/pvxdeverb.py	

Flag	Required	Default	Choices	Action	Description	Source
<code>--strength</code>	False	0.45		Tail suppression strength 0..1	<code>src/pvx/cli/pvxdeverb.py</code>	

A.9 pvxenvelope

Flag	Required	Default	Choices	Action	Description	Source
<code>--amplitude</code>	False	1.0		Peak/depth/amplitude value (mode-dependent; periodic modes use this as waveform amplitude)	<code>src/pvx/cli/pvxenvelope.py</code>	
<code>--attack-sec</code>	False	0.1		ADSR attack in seconds	<code>src/pvx/cli/pvxenvelope.py</code>	
<code>--center</code>	False	0.0		Start/center value (mode-dependent; for periodic modes this is the center/DC offset)	<code>src/pvx/cli/pvxenvelope.py</code>	
<code>--cycles</code>	False		Waveform cycles across full duration (periodic modes) <code>`src/pvx/-cli/pvxenvelope.py`</code> <code>`--duration`</code> True	ADSR decay in seconds <code>`src/pvx/-cli/pvxenvelope.py`</code> <code>`--duration`</code> True	Envelope duration in seconds	
<code>--duty-cycle</code>	False	0.5		Duty cycle in (0,1) for square mode (default: 0.5)	<code>src/pvx/cli/pvxenvelope.py</code>	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--end</code>	False	0.0		End value for ADSR/ramp/exp modes	<code>src/pvx/cli/pvxenvelope.py</code>	
<code>--exp-curve</code>	False	4.0		Exponential curve steepness for exp mode	<code>src/pvx/cli/pvxenvelope.py</code>	
<code>--format</code>	False	auto	csv, json, auto	Output format (default: auto from extension or csv for stdout) `src/pvx/-cli/pvxenvelope.py` `--freq` False		Waveform frequency in Hz for periodic modes (converted to cycles = frequency * duration)
<code>--frequency-hz</code>	False		Waveform frequency in Hz for periodic modes (converted to cycles = frequency * duration) `src/pvx/-cli/pvxenvelope.py` `--key` False `value`	Output control column/key name (default: value) `src/pvx/-cli/pvxenvelope.py` `--max` False		Optional value clamp maximum

Flag	Required	Default	Choices	Action	Description	Source
<code>--min</code>	False		Optional value clamp minimum `src/pvx/- cli/pvxen- velope.py` `--mode` False `adsr`	En- velope/LFO waveform mode `src/pvx/- cli/pvxen- velope.py` `--out` False		Output path. Default: stdout.
<code>--output</code>	False		Output path. Default: stdout. `src/pvx/- cli/pvxen- velope.py` `--peak` False `1.0`	 Peak/depth/am plitude value (mode- dependent; periodic modes use this as waveform amplitude) `src/pvx/- cli/pvxen- velope.py` `--phase` False `0.0`	Initial waveform phase in radians for periodic modes `src/pvx/- cli/pvxen- velope.py` `-- phase-rad` False `0.0`	Initial waveform phase in radians for periodic modes `src/pvx/- cli/pvxen- velope.py` `--rate` False `20.0`
<code>--sine- phase-rad</code>	False	0.0		Initial waveform phase in radians for periodic modes	<code>src/pvx/cli/pvxenvelope.py</code>	
<code>--start</code>	False	0.0		Start/center value (mode- dependent; for periodic modes this is the center/DC offset)	<code>src/pvx/cli/pvxenvelope.py</code>	
<code>--stdout</code>	False		<code>store_true</code>	Write map to stdout	<code>src/pvx/cli/pvxenvelope.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--sustain</code>	False	0.7		Sustain value for ADSR mode	<code>src/pvx/cli/pvxenvelope.py</code>	
<code>--wave</code>	False	adsr		Envelope/LFO waveform mode	<code>src/pvx/cli/pvxenvelope.py</code>	

A.10 pvxfilter

Flag	Required	Default	Choices	Action	Description	Source
<code>--band-gain-db</code>	False	6.0		Band boost amount for bandamp (dB)	<code>src/pvx/cli/pvxfilter.py</code>	
<code>--band-width-bins</code>	False	6		Band width in bins for bandamp	<code>src/pvx/cli/pvxfilter.py</code>	
<code>--comp-ratio</code>	False	2.0		Compression ratio for spec-compander	<code>src/pvx/cli/pvxfilter.py</code>	
<code>--comp-threshold-db</code>	False		Relative threshold for spec-compander <code>`src/pvx/-cli/pvx-filter.py`</code> <code>`--dry-mix`</code> False <code>`0.0`</code>	Dry signal mix amount in [0,1] (default: 0.0) <code>`src/pvx/-cli/pvx-filter.py`</code> <code>`--expand-ratio`</code> False <code>`1.2`</code>	Expansion ratio for spec-compander <code>`src/pvx/-cli/pvx-filter.py`</code> <code>`--noise-floor`</code> False <code>`1.0`</code>	Threshold scale for noise-filter <code>`src/pvx/-cli/pvx-filter.py`</code> <code>`--operator`</code> False
<code>--peak-count</code>	False	8		Number of response peaks for bandamp	<code>src/pvx/cli/pvxfilter.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--response</code>	True		Input PVXRF response artifact `src/pvx/- cli/pvx- filter.py` `-- response- gain-db` False `0.0`	Global response gain in dB (default: 0) `src/pvx/- cli/pvx- filter.py` `-- response- mix` False `1.0`	Wet response mix amount (default: 1.0) `src/pvx/- cli/pvx- filter.py` `--shift- bins` False `0`	Shift response curve by FFT bins `src/pvx/- cli/pvx- filter.py` `-- transpose- semitones` False `0.0`
<code>--tv-order</code>	False	3		Polynomial order for -tv-interp polynomial	src/pvx/cli/pvxfilter.py	

A.11 pvxformant

Flag	Required	Default	Choices	Action	Description	Source
<code>--formant-lifter</code>	False	32			src/pvx/cli/pvxformant.py	
<code>--formant-max-gain-db</code>	False	12.0			src/pvx/cli/pvxformant.py	
<code>--formant-shift-ratio</code>	False	1.0		Formant ratio (>1 up, <1 down)	src/pvx/cli/pvxformant.py	

Flag	Required	Default	Choices	Action	Description	Source
<code>--mode</code>	False	<code>shift</code>	<code>shift</code> , <code>preserve</code>	<code> </code> <code>`src/pvx/-</code> <code>cli/pvx-</code> <code>for-</code> <code>mant.py` </code> <code> </code> <code>`--pitch-</code> <code>shift-</code> <code>cents` </code> <code>False </code> <code>`0.0` </code>	<code> </code> Additional microtonal pitch shift in cents before formant stage <code>`src/pvx/-</code> <code>cli/pvx-</code> <code>for-</code> <code>mant.py` </code> <code> `--</code> resample- mode` <code>False </code> <code>`auto` </code> <code>`auto,</code> fft,   <code>False  </code> <code>linear` </code> <code>`0.0` </code>	<code> Optional</code> pitch shift before formant stage <code>`src/pvx/-</code> <code>cli/pvx-</code> <code>for-</code> <code>mant.py` </code> <code> `--</code> resample- mode` <code>False </code> <code>`auto` </code> <code>`auto,</code> fft, linear` <code>`0.0` </code>

A.12 pvxfreeze

Flag	Required	Default	Choices	Action	Description	Source
<code>--duration</code>	False	<code>3.0</code>	<code> </code>	Output freeze duration in seconds	<code>src/pvx/cli/pvxfreeze.py</code>	
<code>--freeze-time</code>	False	<code>0.2</code>	<code> </code>	Freeze anchor time in seconds	<code>src/pvx/cli/pvxfreeze.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--phase-mode</code>	False	<code>instantaneous</code>	<code>instantaneous</code> <code>bin, hold</code>	Phase progression mode inside the frozen segment: <code>instantaneous</code> (default, <code>least flutter</code>), <code>bin</code> (<code>bin-center</code>), <code>hold</code> (no advance). `src/pvx/cli/pvxfreeze.py` `--random-phase` False	‘ store_true Add subtle phase randomization per frame src/pvx/cli/pvxfreeze.py’	

A.13 pvxgain

Flag	Required	Default	Choices	Action	Description	Source
<code>--bit-depth</code>	False	24	16, 24, 32, float	Output bit depth (default: 24) `src/pvx/cli/pvxgain.py` `--gain` False	 Gain in dB. Single value (fixed) or ‘min,max’ (random range). Mutually exclusive with <code>--normalize</code> .	
<code>--normalize</code>	False	`peak,rms`	Normalization mode. Mutually exclusive with <code>--gain</code> .	<code>src/pvx/cli/pvxgain.py</code>		

Flag	Required	Default	Choices	Action	Description	Source
<code>--output</code>	True		Output audio file `src/pvx/- cli/pvx- gain.py` `--seed` False `0`	Random seed (default: 0) `src/pvx/- cli/pvx- gain.py` `--target` False		Target level in dBFS for normaliza- tion (default: -1.0)

A.14 pvxharmmmap

Flag	Required	Default	Choices	Action	Description	Source
<code>-- attenuation</code>	False	0.45		Chordmapper out-of-chord attenuation in [0,1]	<code>src/pvx/cli/pvxharmmmap.py</code>	
<code>--boost-db</code>	False	6.0		Chordmapper in-chord boost in dB	<code>src/pvx/cli/pvxharmmmap.py</code>	
<code>--chord</code>	False	major		Chord quality for chordmap- per	<code>src/pvx/cli/pvxharmmmap.py</code>	
<code>--dry-mix</code>	False	0.0		Dry signal mix in [0,1]	<code>src/pvx/cli/pvxharmmmap.py</code>	
<code>-- inharmonic- f0-hz</code>	False	220.0		Reference f0 for inhar- monator	<code>src/pvx/cli/pvxharmmmap.py</code>	
<code>-- inharmonic- mix</code>	False	1.0		Inharmonator mix in [0,1]	<code>src/pvx/cli/pvxharmmmap.py</code>	
<code>-- inharmonicity</code>	False	0.0001		Inharmonicity coefficient B	<code>src/pvx/cli/pvxharmmmap.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--operator</code>	False		 `src/pvx/- cli/pvx- har- mmap.py` `--root- hz` False `220.0`	Chord root frequency in Hz `src/pvx/- cli/pvx- har- mmap.py` `-- strength` False `0.75`	 Chordmap- per per emphasis strength in [0,1] `src/pvx/- cli/pvx- har- mmap.py` `-- tolerance- cents` False `35.0`	‘ Chordmap- per tolerance in cents src/pvx/cli/pvxharmmap.py

A.15 pvxharmonize

Flag	Required	Default	Choices	Action	Description	Source
<code>--force-stereo</code>	False		<code>store_true</code>	Mix result as stereo with panning	<code>src/pvx/cli/pvxharmonize.py</code>	
<code>--gains</code>	False		Optional comma- separated linear gain per voice `src/pvx/- cli/pvx- harmoni- ze.py` `-- intervals` False `0,4,7`	Comma- separated semitone intervals per voice (supports fractional values) `src/pvx/- cli/pvx- harmoni- ze.py` `-- intervals- cents` False	 Optional cents offsets per voice, added to -intervals (e.g. 0,14,- 12)	

Flag	Required	Default	Choices	Action	Description	Source
<code>--pans</code>	False		Optional comma- separated pan per voice [-1..1] `src/pvx/- cli/pvx- harmoni- nize.py` `-- resample- mode` False `auto` `auto`, fft, linear`		<code>src/pvx/cli/pvxharmonize.py</code>	

A.16 pvxlayer

Flag	Required	Default	Choices	Action	Description	Source
<code>--harmonic-gain</code>	False	1.0			<code>src/pvx/cli/pvxlayer.py</code>	
<code>--harmonic-kernel</code>	False	31			<code>src/pvx/cli/pvxlayer.py</code>	
<code>--harmonic-pitch-cents</code>	False	0.0			<code>src/pvx/cli/pvxlayer.py</code>	
<code>--harmonic-pitch-semitones</code>	False	0.0			<code>src/pvx/cli/pvxlayer.py</code>	
<code>--harmonic-stretch</code>	False	1.0			<code>src/pvx/cli/pvxlayer.py</code>	
<code>--percussive-gain</code>	False	1.0			<code>src/pvx/cli/pvxlayer.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
-- percussive- kernel	False	31			src/pvx/cli/pvxlayer.py	
-- percussive- pitch- cents	False	0.0			src/pvx/cli/pvxlayer.py	
-- percussive- pitch- semitones	False	0.0			src/pvx/cli/pvxlayer.py	
-- percussive- stretch	False	1.0			src/pvx/cli/pvxlayer.py	
-- resample- mode	False	auto	auto, fft, linear	‘ src/pvx/cli/pvxlayer.py’		

A.17 pvxmorph

Flag	Required	Default	Choices	Action	Description	Source
--alpha	False	0.5		Morph amount 0..1 (0=A, 1=B). Accepts scalar or control file (.csv/.json) for time-varying A->B trajectory morphing.	src/pvx/cli/pvxmorph.py	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--blend-mode</code>	False	<code>linear</code>		Cross-synthesis blend style. linear/geo-metric are symmetric blends; carrier_* modes transfer envelope/mask from modulator to carrier.	<code>src/pvx/cli/pvxmorph.py</code>	
<code>--envelope-lifter</code>	False	32		Cepstral lifter cutoff for carrier_envelope modes (default: 32).	<code>src/pvx/cli/pvxmorph.py</code>	
<code>--interp</code>	False	<code>linear</code>		Interpolation mode for -alpha/-phase-mix control files (default: linear).	<code>src/pvx/cli/pvxmorph.py</code>	
<code>--mask-exponent</code>	False	1.0		Exponent used by carrier_mask modes (default: 1.0).	<code>src/pvx/cli/pvxmorph.py</code>	
<code>--normalize-energy</code>	False		<code>store_true</code>	Normalize each output channel RMS toward alpha-blended input RMS.	<code>src/pvx/cli/pvxmorph.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--order</code>	False	3		Polynomial order for <code>-interp</code> polynomial (default: 3). Accepts any integer ≥ 1 ; effective fit degree is <code>min(order, control_points-1)</code> .	<code>src/pvx/cli/pvxmorph.py</code>	
<code>--output</code>	False		Output file path <code>`src/pvx/-cli/pvx-morph.py`</code> <code>`--output-format`</code> False		Output extension/-format; for <code>-stdout</code> defaults to <code>wav</code>	<code>src/pvx/cli/pvxmorph.py</code>
<code>--overwrite</code>	False		<code>store_true</code>		<code>src/pvx/cli/pvxmorph.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
--phase-mix	False		Phase blend in [0,1]. If omitted, mode-specific defaults apply (A-phase for *_phase_a/carrier_a_*, B-phase for *_phase_b/carrier_b_*, alpha for symmetric modes). Accepts scalar or control file (.csv/.json). `src/pvx/cli/pvxmorph.py` `--stdout` False	‘ store_true Write processed audio to stdout stream (for piping); equivalent to -o - src/pvx/cli/pvxmorph.py‘		

A.18 pvxnoise

Flag	Required	Default	Choices	Action	Description	Source	
--background-dir	False		 Directory of background audio files to mix instead of synthetic noise `src/pvx/-cli/pvxnoise.py` `--band` False `300,4000`	Low,high frequency band in Hz for bandlimited noise (default: 300,4000) `src/pvx/-cli/pvxnoise.py` bit-depth`     False   `24`   `16, 24, 32, float`	Output bit depth (default: 24)	src/pvx/cli/pvxnoise.py	
--noise-type	False	white	white, pink, brown, gaussian, bandlimited	Noise type (default: white) `src/pvx/-cli/pvxnoise.py` `--output` True		Output audio file	
--seed	False	0		Random seed (default: 0)	src/pvx/cli/pvxnoise.py		
--snr	False	20		Signal-to-noise ratio in dB. Single value for fixed SNR or ‘min,max’ for uniform sampling. (default: 20)	src/pvx/cli/pvxnoise.py		

A.19 pvxreshape

Flag	Required	Default	Choices	Action	Description	Source
--exponent	False	1.0		Exponent for pow operation	src/pvx/cli/pvxreshape.py	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--factor</code>	False	1.0		Scale factor for scale/time-scale ops		<code>src/pvx/cli/pvxreshape.py</code>
<code>--format</code>	False	auto	auto, csv, json	Output format (default: auto from --output suffix or csv for stdout) <code>`src/pvx/cli/pvxreshape.py`</code> <code>`--input-format`</code> False <code>`auto`</code> <code>`auto, csv, json`</code>	Input format override (default: auto by suffix or csv for stdin)	<code>src/pvx/cli/pvxreshape.py</code>
<code>--interp</code>	False	linear		Interpolation mode		<code>src/pvx/cli/pvxreshape.py</code>
<code>--key</code>	False	value		Control column/key to read (default: value)		<code>src/pvx/cli/pvxreshape.py</code>
<code>--max</code>	False		Maximum clamp value <code>`src/pvx/cli/pvxreshape.py`</code> <code>`--min`</code> False		Minimum clamp value	<code>src/pvx/cli/pvxreshape.py</code>
<code>--offset</code>	False	0.0		Offset for offset/time-shift ops		<code>src/pvx/cli/pvxreshape.py</code>

Flag	Required	Default	Choices	Action	Description	Source
<code>--operation</code>	True		Reshape operation `src/pvx/-cli/pvxreshape.py` `--order` False `3`	Polynomial order for `--interp` polynomial `src/pvx/-cli/pvxreshape.py` `--out` False		Output file path. Default: stdout.
<code>--output</code>	False		Output file path. Default: stdout. `src/pvx/-cli/pvxreshape.py` `--output-key` False		Output control key. Default: same as <code>-key</code>	<code>src/pvx/cli/pvxreshape.py</code>
<code>--rate</code>	False	20.0		Resample control rate (Hz) for resample operation	<code>src/pvx/cli/pvxreshape.py</code>	
<code>--stdout</code>	False		<code>store_true</code>	Write reshaped map to stdout	<code>src/pvx/cli/pvxreshape.py</code>	
<code>--target-max</code>	False	1.0		Target maximum for normalize operation	<code>src/pvx/cli/pvxreshape.py</code>	
<code>--target-min</code>	False	0.0		Target minimum for normalize operation	<code>src/pvx/cli/pvxreshape.py</code>	
<code>--window</code>	False	5		Window size for smooth operation	<code>src/pvx/cli/pvxreshape.py</code>	

A.20 pvxresponse

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--json</code>	False		<code>store_true</code>	Print summary as JSON	<code>src/pvx/cli/pvxresponse.py</code>	
<code>--method</code>	False	<code>median</code>		Magnitude aggregation across frames (default: median)	<code>src/pvx/cli/pvxresponse.py</code>	
<code>--normalize</code>	False	<code>peak</code>		Magnitude normalization mode (default: peak)	<code>src/pvx/cli/pvxresponse.py</code>	
<code>--output</code>	False		Output PVXRF path (default: <code><analysis>.pvxrf.npy</code>) <code>`src/pvx/cli/pvxresponse.py`</code> <code>`--phase-mode`</code> <code>False</code> <code>`mean`</code>	Phase aggregation strategy (default: <code>mean</code>) <code>`src/pvx/cli/pvxresponse.py`</code> <code>`--smoothing-bins`</code> <code>False</code> <code>`1`</code>	Moving-average smoothing width in bins (default: 1) <code>`--cli/pvxresponse.py`</code> <code>`--summary-json`</code> <code>False</code>	

A.21 pvxretune

Flag	Required	Default	Choices	Action	Description	Source
<code>--a4-reference-hz</code>	False	<code>440.0</code>		Concert-pitch reference for A4 in Hz (default: 440.0)	<code>src/pvx/cli/pvxretune.py</code>	
<code>--chunk-ms</code>	False	<code>80.0</code>		Analysis/process chunk duration in ms	<code>src/pvx/cli/pvxretune.py</code>	
<code>--f0-max</code>	False	<code>1200.0</code>			<code>src/pvx/cli/pvxretune.py</code>	
<code>--f0-min</code>	False	<code>60.0</code>			<code>src/pvx/cli/pvxretune.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--overlap-ms</code>	False	20.0		Chunk overlap in ms		<code>src/pvx/cli/pvxretune.py</code>
<code>--recommend-root</code>	False		<code>store_true</code>	Estimate and use a per-file root fundamental from tracked F0.		<code>src/pvx/cli/pvxretune.py</code>
<code>--resample-mode</code>	False	auto	auto, fft, linear	 `src/pvx/-cli/pvxretune.py` `--root` False `C`	Scale root note (C,C#,D,...,B) `src/pvx/-cli/pvxretune.py` `--root-hz` False	
<code>--scale</code>	False	chromatic		Named scale for 12-TET quantization		<code>src/pvx/cli/pvxretune.py</code>
<code>--scale-cents</code>	False		Optional comma-separated microtonal scale degrees in cents within one octave, relative to --root (example: 0,90,204,294,408,498,612,702,816,906,1020,1110) `src/pvx/-cli/pvxretune.py` `--strength` False `0.85`	Correction strength 0..1 src/pvx/cli/pvxretune.py`		

A.22 pvxring

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--depth</code>	False	1.0		Ring modulation depth in [0,1]	<code>src/pvx/cli/pvxring.py</code>	
<code>--feedback</code>	False	0.0		Feedback amount in [0,1)	<code>src/pvx/cli/pvxring.py</code>	
<code>--frequency-hz</code>	False	40.0		Carrier frequency in Hz	<code>src/pvx/cli/pvxring.py</code>	
<code>--mix</code>	False	1.0		Ring wet mix in [0,1]	<code>src/pvx/cli/pvxring.py</code>	
<code>--operator</code>	False		 `src/pvx/-cli/pvxring.py` `--resonance-decay` False `0.2`	 Resonator feedback-decay in [0,1) `src/pvx/-cli/pvxring.py` `--resonance-hz` False `1200.0`	 Resonance center frequency (Hz) `src/pvx/-cli/pvxring.py` `--resonance-q` False `8.0`	 Resonator wet mix in [0,1] `src/pvx/-cli/pvxring.py` `--resonance-q` False `8.0`
<code>--tv-order</code>	False	3		Polynomial order for -tv-interp polynomial	<code>src/pvx/cli/pvxring.py</code>	

A.23 pvxrir

Flag	Required	Default	Choices	Action	Description	Source
<code>--bit-depth</code>	False	24	16, 24, 32, float	Output bit depth (default: 24) `src/pvx/-cli/pvxrir.py` `--cache-dir` False	 Cache directory for downloaded IR databases (default: <code>~/pvx/ir_cache</code>)	

Flag	Required	Default	Choices	Action	Description	Source
--category	False		Filter IRs by category when using --ir-database (e.g., hall, church, outdoor, room, large) `src/pvx/-cli/pvxrir.py` `--ir-database` cli/pvxrir.py `--dr` False `3,12`	Direct-to-reverb ratio in dB. Single value or 'min,max' range (default: 3,12) `--ir-database` False		Use IRs from a curated database (auto-downloads on first use). Available: echothief, mit_kemar
--ir-dir	False		Directory of impulse response files (one is chosen randomly) `src/pvx/-cli/pvxrir.py` `--ir-file` False		Path to an impulse response WAV file (overrides synthetic generation)	src/pvx/cli/pvxrir.py
--list-databases	False	False	`store_true` List available IR databases and exit `src/pvx/-cli/pvxrir.py` `--no-trim` False	`store_true` Do not trim output to input length (include reverb tail) `src/pvx/-cli/pvxrir.py` `--output` True		Output audio file

Flag	Required	Default	Choices	Action	Description	Source
<code>--pre-delay-ms</code>	False	5.0		Pre-delay in ms before the reverberant tail (default: 5.0)	<code>src/pvx/cli/pvxrir.py</code>	
<code>--rt60</code>	False	0.2,1.5		Reverberation time in seconds. Single value or 'min,max' range (default: 0.2,1.5)	<code>src/pvx/cli/pvxrir.py</code>	
<code>--seed</code>	False	0		Random seed (default: 0)	<code>src/pvx/cli/pvxrir.py</code>	
<code>--wet</code>	False	0.3,0.7		Wet/dry mix 0-1. Single value or 'min,max' range (default: 0.3,0.7)	<code>src/pvx/cli/pvxrir.py</code>	

A.24 pvxspecaugment

Flag	Required	Default	Choices	Action	Description	Source
<code>--bit-depth</code>	False	24	16, 24, 32, float	Output bit depth (default: 24) `src/pvx/-cli/pvxspecaugment.py` `--fill` False `0`	Fill value for masked regions: 0, mean, or a float (default: 0) `src/pvx/-cli/pvxspecaugment.py` `--hop-length` `--freq-mask` False `27`	Maximum frequency mask width F in bins (default: 27) `src/pvx/-cli/pvxspecaugment.py` `--hop-length` False `128`
<code>--seed</code>	False	0		Random seed (default: 0)	<code>src/pvx/cli/pvxspecaugment.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--time-mask</code>	False	100		Maximum time mask width T in frames (default: 100)	<code>src/pvx/cli/pvxspecaugment.py</code>	

A.25 pvxtrajectoryreverb

Flag	Required	Default	Choices	Action	Description	Source
<code>--coord-system</code>	False	cartesian	cartesian, spherical	 Coordinate format for <code>--start/--end</code> (default: cartesian) <code>`src/pvx/cli/pvx-trajectoryreverb.py`</code> <code>`--distance-law`</code> False <code>`inverse`</code> <code>`none, inverse, inverse-square`</code>	Distance gain law applied across trajectory (default: inverse)	<code>src/pvx/cli/pvxtrajectoryreverb.py</code>
<code>--dry</code>	False	0.0		Dry direct mix scalar (default: 0.0)	<code>src/pvx/cli/pvxtrajectoryreverb.py</code>	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--end</code>	True		End position. Cartesian: x,y,z. Spherical: az_deg,el_deg,r. `src/pvx/- cli/pvx- trajecto- ryre- verb.py` `--ir` True		Multichannel impulse response path (for example 4-channel room capture).	<code>src/pvx/cli/pvxtrajectory</code>
<code>--no-normalize-gains</code>	False		<code>store_false</code>	Disable per-sample gain normal- ization.		<code>src/pvx/cli/pvxtrajectoryreverb.py</code>
<code>--normalize-gains</code>	False	True	 `store_true` Normalize per-sample channel gains before distance weighting (default: on) `src/pvx/- cli/pvx- trajecto- ryre- verb.py` `-- speaker- angles` False		Optional speaker layout azimuth/elevation list in degrees as “az,el;az,el;...”. Count must match impulse- response channels.	<code>src/pvx/cli/pvxtrajectory</code>

Flag	Required	Default	Choices	Action	Description	Source
<code>--start</code>	True		Start position. Cartesian: x,y,z. Spherical: az_deg,el_deg,r. `src/pvx/-cli/pvx-trajectoryreverb.py` `--trajectory-shape` False `linear` `linear`, ease-in, ease-out, ease-in-out`	Interpolation shape from start to end (default: linear)	<code>src/pvx/cli/pvxtrajectoryreverb.py</code>	
<code>--wet</code>	False	1.0		Wet reverb mix scalar (default: 1.0)	<code>src/pvx/cli/pvxtrajectoryreverb.py</code>	

A.26 pvxtransient

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--pitch-shift-cents</code>	False		Optional microtonal pitch shift in cents (added to <code>--pitch-shift-semitones</code>) <code>`src/pvx/-cli/pvx-transient.py`</code> <code>`--pitch-shift-ratio`</code> False		Pitch ratio override. Accepts decimals (1.5), integer ratios (3/2), and expressions ($2^{(1/12)}$).	<code>src/pvx/cli/pvxtransient.py</code>
<code>--pitch-shift-semitones</code>	False	0.0				<code>src/pvx/cli/pvxtransient.py</code>
<code>--resample-mode</code>	False	auto	auto, fft, linear	 <code>`src/pvx/-cli/pvx-transient.py`</code> <code>`--target-duration`</code> False		Target duration in seconds
<code>--time-stretch</code>	False	1.0				<code>src/pvx/cli/pvxtransient.py</code>
<code>--transient-threshold</code>	False	1.6				<code>src/pvx/cli/pvxtransient.py</code>

A.27 pvxunison

Flag	Required	Default	Choices	Action	Description	Source
<code>--detune-cents</code>	False	14.0		Total detune span in cents		<code>src/pvx/cli/pvxunison.py</code>

Flag	Required	Default	Choices	Action	Description	Source
<code>--dry-mix</code>	False	0.2		Dry signal mix amount	<code>src/pvx/cli/pvxunison.py</code>	
<code>--resample-mode</code>	False	auto	auto, fft, linear	 `src/pvx/- cli/pvxu- nison.py` `--voices` False `5`	Number of unison voices `src/pvx/- cli/pvxu- nison.py` False `--width` False `1.0`	‘ Stereo width multiplier 0..2 src/pvx/cli/pvxunison.py’

A.28 pvxvoc

Flag	Required	Default	Choices	Action	Description	Source
<code>--ambient-phase-mix</code>	False	0.5		Random- phase blend when <code>-phase-engine</code> hybrid (0.0=propa- gated only, 1.0=random only; default: 0.5). Accepts scalar or control file (.csv/.json). Convenience	<code>src/pvx/core/voc_parser.py</code>	
<code>--ambient-preset</code>	False		store_true	preset for ambient extreme stretch (random phase engine, onset-time- credit, transient preserve, conservative staging).	<code>src/pvx/core/voc_parser.py</code>	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
--analysis-channel	False	mix	first, mix	Channel strategy for F0 estimation with --target-f0 (default: mix) `src/pvx/-core/voc_parser.py` `--auto-profile` False	Seconds of audio used when estimating --auto-profile (default: 6.0). `src/pvx/-core/voc_parser.py` `--auto-profile` segment-seconds` False	
--checkpoint-dir	False		Directory used to cache per-segment checkpoint chunks for resume workflows. `src/pvx/-core/voc_parser.py` `--checkpoint-id` False		Optional checkpoint run identifier (default: hash of input/settings).	src/pvx/core/voc_parser.py
--clip	False		store_true	Legacy alias: hard clip at +/-1.0 when set		src/pvx/core/voc.py

Flag	Required	Default	Choices	Action	Description	Source
-- coherence- strength	False	0.0		Coherence lock strength in [0,1] (0=off, 1=full lock). Accepts scalar or control file (.csv/.json).	src/pvx/core/voc_parser.py	
-- compander- attack-ms	False	8.0		Compander attack time in ms	src/pvx/core/voc.py	
-- compander- compress- ratio	False	3.0		Compander compression ratio (>=1)	src/pvx/core/voc.py	
-- compander- expand- ratio	False	1.8		Compander expansion ratio (>=1)	src/pvx/core/voc.py	
-- compander- makeup-db	False	0.0		Compander makeup gain in dB	src/pvx/core/voc.py	
-- compander- release-ms	False	120.0		Compander release time in ms	src/pvx/core/voc.py	
-- compander- threshold- db	False		Enable compander threshold in dBFS `src/pvx/- core/voc.py` `-- compressor- attack-ms` False `10.0`	Compressor attack time in ms `src/pvx/- core/voc.py` `-- compressor- makeup-db` False `0.0`	Compressor makeup gain in dB `src/pvx/- core/voc.py` `-- compressor- release- ms` False `120.0`	Compressor ratio (>=1) `src/pvx/- core/voc.py` `-- compressor- release- ms` False `120.0`
--control- stdin	False		store_true	Alias for -pitch-map- stdin (canonical control-bus CSV stdin path).	src/pvx/core/voc_parser.py	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--cpu</code>	False		<code>store_true</code>	Alias for <code>--device cpu</code> .		<code>src/pvx/core/voc_parser.py</code>
<code>--cuda-device</code>	False	0		CUDA device index used when <code>--device</code> is <code>auto/cuda</code> (default: 0)		<code>src/pvx/core/voc.py</code>
<code>--device</code>	False	auto	auto, cpu, cuda	Compute device: auto (prefer CUDA), cpu, or cuda `src/pvx/-core/voc.py` `--dither` False `none`	Dither policy before quantized writes (default: none) `src/pvx/-core/voc_parser.py` `--dither-seed` False	
<code>--dry-run</code>	False		<code>store_true</code>	Resolve settings without writing files		<code>src/pvx/core/voc_parser.py</code>
<code>--example</code>	False		Print copy-paste example command(s) and exit. `src/pvx/-core/voc_parser.py` `--expander-ratio` `--expander-attack-ms` False `5.0`	Expander attack time in ms `src/pvx/-core/voc.py` `--expander-release-ms` False `2.0`	Expander ratio (≥ 1) `src/pvx/-core/voc.py` `--expander-release-ms` False `120.0`	Expander release time in ms `src/pvx/-core/voc.py` `--expander-threshold-db` False
<code>--explain-plan</code>	False		<code>store_true</code>	Print resolved processing plan JSON and exit without rendering audio.		<code>src/pvx/core/voc_parser.py</code>

Flag	Required	Default	Choices	Action	Description	Source
<code>--extreme-stretch-threshold</code>	False	2.0		Auto-mode threshold for multistage activation (default: 2.0). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	
<code>--extreme-time-stretch</code>	False		<code>store_true</code>	Force multistage strategy even when ratio is moderate.	<code>src/pvx/core/voc_parser.py</code>	
<code>--f0-max</code>	False	1000.0		Maximum F0 search bound in Hz (default: 1000)	<code>src/pvx/core/voc_parser.py</code>	
<code>--f0-min</code>	False	50.0		Minimum F0 search bound in Hz (default: 50)	<code>src/pvx/core/voc_parser.py</code>	
<code>--force</code>	False		<code>store_true</code>	Overwrite existing outputs and other write targets	<code>src/pvx/core/voc_parser.py</code>	
<code>--formant-lifter</code>	False	32		Cepstral lifter cutoff for formant envelope extraction (default: 32). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	
<code>--formant-max-gain-db</code>	False	12.0		Max per-bin formant correction gain in dB (default: 12). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--formant-strength</code>	False	1.0		Formant correction blend 0..1 when pitch mode is formant-preserving (default: 1.0). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	
<code>--fourier-sync</code>	False		<code>store_true</code>	Enable fundamental frame locking. Uses generic short-time Fourier transforms with per-frame FFT sizes locked to detected F0.	<code>src/pvx/core/voc_parser.py</code>	
<code>--fourier-sync-max-fft</code>	False	8192		Maximum frame FFT size for <code>--fourier-sync</code> (default: 8192). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	
<code>--fourier-sync-min-fft</code>	False	256		Minimum frame FFT size for <code>--fourier-sync</code> (default: 256). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--fourier-sync-smooth</code>	False	5		Smoothing span (frames) for prescanned F0 track in <code>--fourier-sync</code> (default: 5). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	
<code>--gpu</code>	False		<code>store_true</code>	Alias for <code>--device cuda</code> .	<code>src/pvx/core/voc_parser.py</code>	
<code>--guided</code>	False		<code>store_true</code>	Interactive guided mode for first-time users.	<code>src/pvx/core/voc_parser.py</code>	
<code>--hard-clip-level</code>	False		Hard clip level in linear full-scale <code>`src/pvx/-core/voc.py`</code> <code>--hop-size`</code>     False     <code>`512`</code>	Hop size in samples (default: 512). Accepts scalar or control file (.csv/.json). <code>`src/pvx/-core/voc_parser.py`</code> <code>--interp`</code>     False     <code>`linear`</code>	Interpolation mode for time-varying control signals loaded from CSV/JSON (default: <code>`linear`</code>). <code>`src/pvx/-core/voc_parser.py`</code> <code>--kaiser-beta`</code>   False     <code>`14.0`</code>	Kaiser window beta parameter used when <code>--window kaiser</code> (default: 14.0). Accepts scalar or control file <code>`src/pvx/-core/voc_parser.py`</code> . <code>--limiter-threshold`</code> False
<code>--manifest-append</code>	False		<code>store_true</code>	Append entries to an existing <code>--manifest-json</code> file instead of replacing it.	<code>src/pvx/core/voc_parser.py</code>	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source	
--manifest-json	False		Write processing manifest JSON with per-file settings and outcomes. `src/pvx/-core/voc_parser.py` `--max-stage-stretch` False `1.8`	Maximum per-stage ratio used in multistage mode (default: 1.8). Accepts scalar or control file (.csv/.json).	Output metadata policy: none, sidecar, or copy (sidecar implementation) `--ffts` False	Comma-separated FFT sizes for --multires-fusion (default: 1024,2048,4096) `--multires-fusion` False	`src/pvx/-core/voc_parser.py`
--n-fft	False	2048		FFT size (default: 2048). Accepts scalar or control file (.csv/.json).	src/pvx/core/voc_parser.py		
--no-center	False		store_true	Disable center padding in STFT/ISTFT	src/pvx/core/voc_parser.py		
--no-onset-realign	False		store_true	Disable fractional read-position realignment on onsets when -onset-time-credit is enabled.	src/pvx/core/voc_parser.py		
--no-progress	False		store_true		src/pvx/core/voc_console.py		

Flag	Required	Default	Choices	Action	Description	Source
-- normalize	False	none	none, peak, rms	Output normaliza- tion mode `src/pvx/- core/voc.py` `--onset- credit- max` False `8.0`	Maximum accumu- lated onset time credit in analysis- frame units (default: 8.0). Accepts scalar or control file (.csv/.json). control `src/pvx/- core/voc_parser.py` `--onset- credit- pull` False `0.5`	Fraction of per-frame read advance removable while onset credit exists (0.0..1.0, default: 0.5). Accepts scalar or control file (.csv/.json). control file (.csv/.json). `src/pvx/- core/voc_parser.py` `--onset- time- credit` False
--output	False		Explicit output file path (single- input mode only). Alias: --out `src/pvx/- core/voc_parser.py` `--output- dir` False	 	Directory for output files (default: same directory as each input)	src/pvx/core/voc_parser.py

Flag	Required	Default	Choices	Action	Description	Source
<code>--output-format</code>	False		Output format/extension (e.g. wav, flac, aiff). Default: keep input extension. <code>`src/pvx/-core/voc_parser.py`</code> <code>`--overwrite`</code> False	<code>`store_true`</code> Overwrite existing outputs and other write targets <code>`src/pvx/-core/voc_parser.py`</code> <code>`peak-dbfs`</code> False	Target peak dBFS when <code>--normalize</code> peak	
<code>--phase-engine</code>	False	propagate		Phase synthesis engine: propagate (classic phase vocoder), hybrid (propagated + stochastic blend), random (ambient stochastic phase).	<code>src/pvx/core/voc_parser.py</code>	
<code>--phase-locking</code>	False	identity	off, identity	Inter-bin phase locking mode for transient fidelity (default: identity) <code>`src/pvx/-core/voc_parser.py`</code> <code>`--phase-random-seed`</code> False	Optional deterministic seed for random/hybrid phase generation.	

453

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--pitch-shift-cents</code>	False		Pitch shift in cents (+1200 is one octave up). Accepts scalar or control file (.csv/.json). <code>`src/pvx/-core/voc_parser.py`</code> <code>`--pitch-shift-ratio`</code> False		Pitch ratio (>1 up, <1 down). Accepts decimals (1.5), integer ratios (3/2), expressions ($2^{(1/12)}$), or a control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>
<code>--pitch-shift-semitones</code>	False		Pitch shift in semitones (+12 is one octave up). Accepts scalar or control file (.csv/.json). <code>`src/pvx/-core/voc_parser.py`</code> <code>`--preset`</code> False <code>`none`</code>	High-level intent preset. Legacy: none/vocal/ambient/extreme. New: default/vocal/studio/drums/quality/ambient/stereo_coherence. False <code>`src/pvx/-core/voc_parser.py`</code> <code>`--quality-profile`</code> False <code>`neutral`</code>	Named tuning profile for vocoder defaults (default: neutral) <code>`src/pvx/-core/voc_console.py`</code> <code>`--ratio`</code> <code>`src/pvx/-core/voc_parser.py`</code> <code>`--ratio`</code> <code>`src/pvx/-core/voc_parser.py`</code>	<code>`store_true`</code> Reduce output and hide status bars <code>`src/pvx/-core/voc_console.py`</code>
<code>--ref-channel</code>	False	0		Reference channel index used by -stereo-mode ref_channel_lock (default: 0).	<code>src/pvx/core/voc_parser.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
-- resample- mode	False	auto	auto, fft, linear	 Resampling engine (auto=fft if scipy available, else linear) `src/pvx/- core/voc_parser.py` `--resume` False	 `store_true` Reuse existing checkpoint chunks from -- checkpoint- dir when available. `src/pvx/- core/voc_parser.py` `-- rms-dbf` False	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--route</code>	False	[]	`append` Control- bus routing expression for map rows. Repeat flag to chain routes. Syntax: tar- get=source, tar- get=const(v), tar- get=inv(source), tar- get=pow(source,exp), tar- get=mul(source,factor), tar- get=add(source,offset), tar- get=affine(source, scale, bias), tar- get=clip(source, lo, hi). Targets: stretch, pitch_ratio. Sources: any numeric column present in the control- map CSV. `src/pvx/- core/voc_parser.py` `-- semitones` False		Pitch shift in semitones (+12 is one octave up). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>
<code>--silent</code>	False		store_true	Suppress all console output		<code>src/pvx/core/voc_console.py</code>
<code>--soft-clip-drive</code>	False	1.0		Soft clip drive amount (>0)		<code>src/pvx/core/voc.py</code>

Flag	Required	Default	Choices	Action	Description	Source
<code>--soft-clip-level</code>	False		Soft clip output ceiling in linear full-scale `src/pvx/-core/voc.py` `--soft-clip-type` False `tanh` `tanh`, arctan, cubic`	Soft clip transfer type		<code>src/pvx/core/voc.py</code>
<code>--stdout</code>	False		<code>store_true</code>	Write processed audio to stdout stream (for piping); requires exactly one input		<code>src/pvx/core/voc_parser.py</code>
<code>--stereo-mode</code>	False	<code>independent</code>	<code>independent</code> , Channel <code>mid_side_lock</code> coherence <code>ref_channel_lock</code> strategy: independent (legacy), <code>mid_side_lock</code> (M/S-coupled), <code>ref_channel_lock</code> (phase-lock to reference channel). `src/pvx/-core/voc_parser.py` `--stretch` False	Channel coherence strategy: independent (legacy), <code>mid_side_lock</code> (M/S-coupled), <code>ref_channel_lock</code> (phase-lock to reference channel). `src/pvx/-core/voc_parser.py` `--stretch` False	 Alias for –time-stretch. Accepts scalar or control file (.csv/.json).	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--stretch-mode</code>	False	<code>auto</code>	<code>auto, standard, multistage</code>	Stretch strategy: standard (single pass), multistage (chained moderate passes), or auto (multi-stage only for extreme ratios; default: auto). <code>`src/pvx/-core/voc_parser.py`</code> <code>--subtype`</code> False	Explicit libsndfile output subtype override (e.g., PCM_16, PCM_24, FLOAT)	
<code>--suffix</code>	False	<code>_pv</code>		Suffix appended to output filename stem (default: <code>_pv</code>)	<code>src/pvx/core/voc_parser.py</code>	
<code>--target-duration</code>	False		Absolute target duration in seconds (overrides <code>--time-stretch</code>) <code>`src/pvx/-core/voc_parser.py`</code> <code>--target-f0`</code> False	Target fundamental frequency in Hz. Auto-estimates source F0 per file.	<code>src/pvx/core/voc_parser.py</code>	

Flag	Required	Default	Choices	Action	Description	Source
<code>--target-lufs</code>	False		 Integrated loudness target in LUFS `src/pvx/- core/voc.py` `--target- pitch- shift- semitones` False		Pitch shift in semitones (+12 is one octave up). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>
<code>--target-sample-rate</code>	False		Output sample rate in Hz (default: keep input rate) `src/pvx/- core/voc_parser.py` `--time- stretch` False `1.0`	Final duration multiplier (1.0=un- changed, 2.0=2x longer). Accepts scalar or control file (.csv/.json). `src/pvx/- core/voc_parser.py` `--time- stretch- factor` False `1.0`	Final duration multiplier (1.0=un- changed, 2.0=2x longer). Accepts scalar or control file (.csv/.json). `src/pvx/- core/voc_parser.py` `-- transform` False `fft`	 Per-frame transform backend for STFT/ISTFT paths (default: fft; options: fft, dft, czt, dct, dst, hartley) `src/pvx/- core/voc_parser.py` `-- transient- crossfade- ms` False `10.0`
<code>--transient-preserve</code>	False		<code>store_true</code>	Enable transient phase resets based on spectral flux	<code>src/pvx/core/voc_parser.py</code>	
<code>--transient-protect-ms</code>	False	30.0		Transient protection width in milliseconds (default: 30). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	

A. pvx Command-Line Interface (CLI) Flags Reference

Flag	Required	Default	Choices	Action	Description	Source
<code>--transient-sensitivity</code>	False	0.5		Transient detector sensitivity in [0,1] (higher catches more onsets). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	
<code>--transient-threshold</code>	False	2.0		Spectral-flux multiplier for transient detection (default: 2.0). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	
<code>--true-peak-max-dbtp</code>	False		Apply output gain trim to enforce max true-peak in dBTP `src/pvx/-core/voc_parser.py` `--verbose` False `0`	count	Increase verbosity (repeat for extra detail)	<code>src/pvx/core/voc_console.py</code>
<code>--verbosity</code>	False	normal		Console verbosity level	<code>src/pvx/core/voc_console.py</code>	
<code>--win-length</code>	False	2048		Window length in samples (default: 2048). Accepts scalar or control file (.csv/.json).	<code>src/pvx/core/voc_parser.py</code>	
<code>--window</code>	False	hann		Window type (default: hann)	<code>src/pvx/core/voc_parser.py</code>	

A.29 pvxwarp

Flag	Required	Default	Choices	Action	Description	Source
-- crossfade- ms	False	8.0				src/pvx/cli/pvxwarp.py
--map	True		CSV map with start_sec,end_sec,stretch `src/pvx/- cli/pvxwarp.py` `-- resample- mode` False `auto` `auto, fft, linear`			src/pvx/cli/pvxwarp.py

CHAPTER B

pvx Algorithm Limitations and Applicability

Generated from commit **35e9761** (commit date: 2026-05-25T08:14:42-04:00).

This document summarizes assumptions, likely failure modes, and practical exclusion cases for each algorithm group and algorithm module.

B.1 Group-Level Summary

Group	Assumptions	Failure Modes	When Not To Use
<code>analysis_qa_and_automation</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>creative_spectral_effects</code>	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
<code>denoise_and_restoration</code>	Noise/artifacts are distinguishable from desired signal statistics.	Over-reduction can remove detail and create modulation artifacts.	Avoid high reduction settings on sparse acoustic sources without auditioning.
<code>dereverb_and_room_correction</code>	Late reverberation is separable from direct content under chosen model.	Speech/music clarity can drop if early reflections are over-suppressed.	Avoid strong dereverb when room character is part of artistic intent.
<code>dynamics_and_loudness</code>	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
<code>granular_and_modulation</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
<code>pitch_detection_and_tracking</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>retune_and_intonation</code>	Detected notes map cleanly to intended tonal center/scale.	Over-correction can flatten expressive vibrato or slides.	Avoid aggressive correction when preserving natural micro-intonation is required.

Group	Assumptions	Failure Modes	When Not To Use
<code>separation_and_decomposition</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>spatial_and_multichannel</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spectral_time_frequency_transforms</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
<code>time_scale_and_pitch_core</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.

B.2 analysis_qa_and_automation

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>analysis_qa_and_automation_auto_parameter_tuning_bayesian_optimization</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation_batch_preset_recommendation_based_on_source_features</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation_clip_hum_buzz_artifact_detection</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation_key_chord_detection</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation_onset_beat_downbeat_tracking</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation_pesq_stoi_visqol_quality_metrics</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
analysis_qa_and_automation_silence_speech_music_classifiers	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
analysis_qa_and_automation_structure_segmentation Verse, Chorus, Sections	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.

B.3 creative_spectral_effects

Algorithm ID	Assumptions	Failure Modes	When Not To Use
creative_spectral_effects_cross_synthesis_vocoder	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
creative_spectral_effects_formant_painting_warping	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
creative_spectral_effects_phase_randomization_textures	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
creative_spectral_effects_resonator_filterbank_morphing	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
creative_spectral_effects_spectral_blur_smear	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
creative_spectral_effects_spectral_contrast_exaggeration	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
creative_spectral_effects_spectral_convolution_effects	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
creative_spectral_effects_spectral_freeze_banks	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.

B.4 denoise_and_restoration

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>denoise_and_restoration_declip_decrackle_median_wavelet_interpolation</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration_declip_via_sparse_reconstruction</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration_diffusion_based_speech_audio_denoise</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration_log_mmse</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration_minimum_statistics_noise_tracking</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration_mmse_stea</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
denoise_and_restoration_rnnoise_style_denoiser	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
denoise_and_restoration_wiener_denoising	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.

B.5 dereverb_and_room_correction

Algorithm ID	Assumptions	Failure Modes	When Not To Use
dereverb_and_room_correction_blind_deconvolution_dereverb	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction_drr_guided_dereverb	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction_late_reverb_suppression_via_coherence	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction_multi_band_adaptive_dereverb	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
dereverb_and_room_correction_neural_dereverb_module	Late reverberation is separable from direct content under chosen model. Model priors assume training-like signal statistics. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Generalization gaps can produce unstable artifacts. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid fully unattended use on out-of-domain material. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction_room_impulse_inverse_filtering	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction_spectral_decay_subtraction	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction_wpe_dereverb	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.

B.6 dynamics_and_loudness

Algorithm ID	Assumptions	Failure Modes	When Not To Use
dynamics_and_loudness.ebu_r128_normalization	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
dynamics_and_loudness.itu_bs_1770_loudness_measurement_gating	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
dynamics_and_loudness.lufs_target_mastering_chain	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
dynamics_and_loudness.multi_band_compression	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
dynamics_and_loudness.spectral_dynamics_bin-wise_compression_expander	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
dynamics_and_loudness.transient_shaping	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
dynamics_and_loudness.true_peak_limiting	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
dynamics_and_loudness.upward_compression	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.

B.7 granular_and_modulation

Algorithm ID	Assumptions	Failure Modes	When Not To Use
granular_and_modulation.am_fm_ring_modulation_blocks	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
granular_and_modulation.envelope_followed_modulation_routing	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
granular_and_modulation.formant_lfo_modulation	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
granular_and_modulation.freeze_grain_morphing	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
granular_and_modulation.grain_cloud_pitch_textures	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>granular_and_modulation_granular_time_stretch_engine</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
<code>granular_and_modulation_rhythmic_gate_stutter_quantizer</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
<code>granular_and_modulation_spectral_tremolo</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.

B.8 pitch_detection_and_tracking

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>pitch_detection_and_tracking_crepe_style_neural_f0</code>	F0 evidence is strong in the selected analysis band and frame size. Model priors assume training-like signal statistics.	Octave errors and voicing flips under heavy noise/polyphony. Generalization gaps can produce unstable artifacts.	Avoid as the sole control signal for dense polyphonic mixtures. Avoid fully unattended use on out-of-domain material.
<code>pitch_detection_and_tracking_harmonic_product_spectrum_hps</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking_pvin</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking_rapt</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking_subharmonic_summation</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking_swipe</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking_viterbi_smoothed_pitch_contour_tracking</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking_vin</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.

B.9 retune_and_intonation

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>retune_and_intonation.adaptive_intonation_context_sensitive_intervals</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.chord_aware_retuning</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.just_intonation_mapping_per_key_center</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.key_aware_retuning_with_confidence_weighting</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.portamento_aware_retune_curves</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.scales_mts_scale_import_and_quantization</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.time_varying_cents_maps</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>retune_and_intonation.vibrato_preserving_correction</code>	Detected notes map clearly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.

B.10 separation_and_decomposition

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>separation_and_decomposition.demucs_style_stem_separation_backend</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.ica_bss_for_multichannel_stems</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.nmf_decomposition</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.probabilistic_latent_component_separation</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.rpca_hnss</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.sinusoidal_residual_transient_decomposition</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.tensor_decomposition_cp_tucker</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.u_net_vocal_accompaniment_split</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.

B.11 spatial_and_multichannel

Algorithm ID	Assumptions	Failure Modes	When Not To Use
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct. Reverberation tail is assumed more diffuse than direct content.	Spatial collapse, combing, or localization bias from misalignment. Over-suppression can thin tonal body and ambience.	Avoid blind spatial processing when channel order/calibration is unknown. Avoid for intentionally wet effects unless mix preservation is planned.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct. Noise model should be representative of observed noise floor.	Spatial collapse, combing, or localization bias from misalignment. Mismatched noise estimate leaves residue or damages detail.	Avoid blind spatial processing when channel order/calibration is unknown. Avoid static settings on rapidly varying nonstationary noise.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, spectral warp, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, pvx interaural coherence shaping, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, pvx interchannel phase locking, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, pvx multichannel time alignment, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, pvx spatial freeze and trajectory, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, pvx spatial transient preservation, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, rotating speaker doppler field, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, spatial freeze resynthesis, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, spectral spatial granulator, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, stereo width frequency dependent control, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, stochastic spatial diffusion cloud, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
spatial_and_multichannel	Channel geometry/order and timing metadata are correct.	Spatial collapse, transaural crosstalk cancellation, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>spatial_and_multichannel_vban_adaptive_panning</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.

B.12 spectral_time_frequency_transforms

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>spectral_time_frequency_transforms_chirplet_transform_analysis</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
<code>spectral_time_frequency_transforms_constant_q_transform_cqt_processing</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
<code>spectral_time_frequency_transforms_multi_window_stft_fusion</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
<code>spectral_time_frequency_transforms_nsqt_based_processing</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
<code>spectral_time_frequency_transforms_reassigned_spectrogram_methods</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
<code>spectral_time_frequency_transforms_synchrosqueezed_stft</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
<code>spectral_time_frequency_transforms_variable_q_transform_vqt</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>spectral_time_frequency_transforms_wavelet_packet_processing</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.

B.13 `time_scale_and_pitch_core`

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>time_scale_and_pitch_core_beat_synchronous_time_warping</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core_harmonic_percussive_split_tsm</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core_lp_psola</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core_multi_resolution_phase_vocoder</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth. Phase continuity assumptions hold best for moderate stretch ratios.	High-ratio stretch can introduce phasiness and blurred transients. Extreme settings increase phasiness/transient blur risk.	Avoid for extreme percussive-only material when attack realism is critical. Avoid very large stretch+pitch shifts without transient controls.
<code>time_scale_and_pitch_core_nonlinear_time_maps</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core_td_psola</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core_wsola_waveform_similarity_overlap_add</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.

CHAPTER C

pvx Diagram Atlas

This document provides expanded architecture and digital signal processing (DSP) flow diagrams for pvx. Diagrams are a mix of Mermaid (GitHub-rendered) and ASCII (terminal-friendly).

C.1 End-to-End pvxvoc Flow

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.2 STFT Analysis / Synthesis Timeline

```
signal: x[n] ----->

windows:
    |---N---|
      |---N---|
        |---N---|
hop:      <---Ha--->

analysis:
  frame t0 -> FFT -> X0[k]
  frame t1 -> FFT -> X1[k]
  frame t2 -> FFT -> X2[k]

processing:
  magnitude trajectory + phase trajectory updates

synthesis:
  IFFT(frame0), IFFT(frame1), IFFT(frame2) + overlap-add with Hs
```

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.3 Phase Propagation and Locking

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
Frame index:   t0      t1      t2      t3
Peak bin:      0-----0-----0-----0
Neighbor free:  o--o-----o--o-----o----- (drift can increase blur)
Neighbor lock:  o-----o-----o-----o----- (relative peak shape preserved)
```

C.4 Hybrid Transient Engine

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
time ----->
mask 00000011110000001111000000
    steady^^trans^^steady^^trans^^steady
```

C.5 Transient Feature Stack

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.6 WSOLA Similarity Search

```
reference analysis frame:      [=====R=====]

candidate search window:
                                [c0][c1][c2][c3][c4][c5][c6]
                                \  \  \  \  \  \
similarity score:              s0  s1  s2  s3  s4  s5

choose argmax(similarity) -> best offset -> overlap-add with crossfade
```

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.7 Stereo / Multichannel Coherence Modes

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
independent:      L and R evolve freely
mid_side_lock:    preserve center/side coherence envelope
ref_channel_lock: keep phase relation anchored to one channel
```

C.8 Microtonal Map Processing

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.9 Control-Rate Interpolation Graph Examples

The same control points produce different trajectories depending on `--interp` and `--order`.

Mode/order	Graph
none	
nearest	
linear	
cubic	
exponential	
s_curve	
smootherstep	
polynomial --order 1	
polynomial --order 2	
polynomial --order 3	
polynomial --order 5	

C.10 Function Graph Gallery

Function family	Graph
Pitch ratio vs semitones	
Pitch ratio vs cents	
Dynamics transfer curves	
Soft clip transfer functions	
Morph blend magnitude curves	
Mask exponent response	
Phase mix curve	

C.11 Extreme Stretch: Multistage Strategy

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
example total stretch = 600x
600 = 5 * 5 * 4.8 * 5
render sequentially with safer stage factors
```

C.12 Transform Backend Decision Path

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
preferred default: fft
research/verification: dft/czt/dct/dst/hartley
```

C.13 Mastering Chain Block Diagram

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.14 Metrics Printing Path

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.15 Benchmark Runner + CI Gate

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.16 Pipe Chaining Pattern

```
pvx voc input.wav --stdout \  
| pvx denoise - --stdout \  
| pvx deverb - --output final.wav  
  
rule:  
- producer uses --stdout  
- consumer reads '-' as stdin
```

C.17 Checkpoint / Resume State Machine

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.18 Troubleshooting Decision Tree

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.19 Data Product Map

```
markdown docs -> docs/*.md  
html docs -> docs/html/*.html  
pdf bundle -> docs/pvx_documentation.pdf  
benchmark json -> benchmarks/out/report.json  
benchmark md -> benchmarks/out/report.md
```

C.20 Window/Overlap Tradeoff Map

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
larger window + higher overlap -> smoother/cleaner sustained content  
smaller window + lower overlap -> sharper time localization, rougher bass resolution
```

C.21 Phase Reset vs Hybrid Boundary Behavior

```
time ----->  
transient mask: 000011100000001110000  
  
mode=reset:  
phase tracks ---->|reset|----->|reset|----->  
  
mode=hybrid:  
pv steady -----[PV]-----[PV]-----  
wsola transient [WSOLA] [WSOLA]  
stitch <xfade> <xfade>
```

C.22 Multi-Resolution Fusion Fan-In

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.23 Pitch-Map Confidence Gate

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.24 Microtonal CSV Interpolation Flow

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.25 Audio Metrics Table Generation

```
input.wav -> basic metrics (sr,ch,duration,peak,rms,crest,dc,zcr,bw95,clip%)
output.wav -> basic metrics (same fields)
paired cmp -> compare metrics (snr,si-sdr,lsd,mod-dist,-envelope corr,...)

console:
table A: input summary
table B: output summary
table C: input vs output (input / output / delta)
```

C.26 Quality-First Optimization Loop

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.27 Benchmark Metric Taxonomy

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.28 Documentation Build Pipeline

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

Use this atlas together with: - docs/MATHEMATICAL_FOUNDATIONS.md for equation-level details
- docs/EXAMPLES.md for copy-paste command recipes

CHAPTER D

pvx Phasiness Implementation Plan

D.1 Why this paper matters

About this **Phasiness Business** isolates why classic phase-vocoder output sounds “phasier” under stretch: weak cross-bin phase coherence, unstable peak-region phase behavior, and frame-to-frame phase inconsistency in dense harmonic material.

For **pvx**, this gives a quality-first roadmap:

- strengthen local phase locking around spectral peaks
- preserve vertical coherence (within a frame) and horizontal coherence (across frames)
- reduce phase random walk in low-energy bins
- make coherence strategy content-aware (speech/tonal/percussive)

D.2 Benefits we can extract for pvx

This section begins with the following points, which establish the context for the details that follow.

1. Better harmonic solidity at moderate and high stretch ratios.
2. Lower “swirl/smear” in sustained vocals and pads.
3. More stable stereo image when coherence is enforced consistently across channels.
4. Cleaner transient neighborhoods when hybrid transient handling and phase resets are coordinated.
5. Stronger objective benchmark results on modulation- and coherence-sensitive metrics.

D.3 Phased implementation

D.3.1 Phase 1: Coherence-aware phase propagation in core

Files: - `src/pvx/core/voc.py` - `src/pvx/core/stereo.py`

Work: - Add explicit “peak-region phase-locking radius” control. - Lock neighboring bins to anchor-bin phase increments near detected peaks. - Add low-energy-bin damping to avoid unstable phase diffusion. - Keep deterministic branch for central processing unit (CPU) mode.

CLI additions: - `--phase-lock-radius <bins>` - `--phase-lock-strength <0..1>` - `--phase-noise-floor-db <dB>`

D.3.2 Phase 2: Adaptive coherence policy

Files: - `src/pvx/core/presets.py` - `src/pvx/core/transients.py` - `src/pvx/core/voc.py`

Work: - Auto-select coherence mode by frame class: - tonal frames: stronger identity/peak lock - noisy/percussive frames: relaxed lock + transient protection - Fuse with transient-mode handling (`reset`, `hybrid`, `wsola`).

CLI additions: - `--phase-policy {static,adaptive}` - `--phase-tonal-lock <0..1>` - `--phase-noise-lock <0..1>`

D.3.3 Phase 3: Stereo/multichannel coherence extension

Files: - `src/pvx/core/stereo.py` - `src/pvx/metrics/coherence.py`

Work: - Apply lock decisions in mid/side (M/S) domain for stereo-preserving stretch. - Improve reference-channel lock to retain inter-channel phase deltas near peaks. - Add diagnostics for per-band coherence drift.

CLI additions: - `--coherence-report-json <path>`

D.3.4 Phase 4: Benchmark and regression gates

Files: - `benchmarks/metrics.py` - `benchmarks/run_bench.py` - `.github/workflows/bench-regression.yml`

Work: - Add phasiness-focused metrics: - inter-frame phase-deviation variance - peak-region phase-consistency score - modulation-spectrum stability score - Add strict pass/fail thresholds for tonal and speech subsets.

D.3.5 Phase 5: UX and docs

Files: - `README.md` - `docs/QUALITY_GUIDE.md` - `docs/EXAMPLES.md`

Work: - Add “phasiness reduction” intent preset. - Add examples: - vocal stretch quality profile - ambient pad high-ratio profile - stereo-coherent mix-bus profile - Add troubleshooting matrix: artifact -> parameter moves.

D.4 Validation targets

This section begins with the following points, which establish the context for the details that follow.

- Lower modulation-spectrum distance on sustained harmonic tests.
- Lower stereo coherence drift on stereo test corpus.
- Audible A/B improvement on speech and vocal sustained vowels.
- No regression in `stretch=1.0` identity reconstruction tolerance.

D.5 Source paper and cited references to track

This section begins with the following points, which establish the context for the details that follow.

- J. Laroche; M. Dolson (1997), *About this Phasiness Business*.
- M. Dolson (1986), *The phase vocoder: A tutorial*.
- D. W. Griffin; J. S. Lim (1984), *Signal estimation from modified short-time Fourier transform*.
- J. Laroche; M. Dolson (1997), *Phase-vocoder: About this phasiness business* (submitted manuscript citation in paper).
- E. Moulines; J. Laroche (1995), *Non-parametric techniques for pitch-scale and time-scale modification of speech*.
- M. S. Puckette (1995), *Phase-locked vocoder*.

CHAPTER E

pvx Citation Quality Report

Generated from commit **35e9761** (commit date: 2026-05-25T08:14:42-04:00).

This report classifies bibliography URLs by citation quality and highlights entries still using search-index links.

Total references analyzed: **248**

E.1 Link-Type Summary

	Link type	Count
	arxiv	8
	doi	90
	publisher_or_standard	10
	scholar	88
	web	52

E.2 Entries Still Using Scholar Links

These are prime targets for future DOI/publisher URL upgrades.

Year	Authors	Title	URL
2021	S. Wisdom et al.	Differentiable Consistency Constraints for Improved Deep Speech Enhancement	https://scholar.google.com/scholar?q=Differentiable+Consistency+Constraints+for+Improved+Deep+Speech+Enhancement
2020	J. Kong et al.	HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis	https://scholar.google.com/scholar?q=HiFi-GAN%3A+Generative+Adversarial+Networks+for+Efficient+and+High+Fidelity+Speech+Synthesis
2020	S. Stables et al.	A Study of Loudness Normalization in Streaming Services	https://scholar.google.com/scholar?q=A+Study+of+Loudness+Normalization+in+Streaming+Services
2019	R. Prenger et al.	WaveGlow: A Flow-based Generative Network for Speech Synthesis	https://scholar.google.com/scholar?q=WaveGlow%3A+A+Flow-based+Generative+Network+for+Speech+Synthesis
2019	M. H. C. de Gesmundo et al.	ViSQOL v3: An Open Source Production Ready Objective Speech and Audio Metric	https://scholar.google.com/scholar?q=ViSQOL+v3%3A+An+Open+Source+Production+Ready+Objective+Speech+and+Audio+Metric

Year	Authors	Title	URL
2019	J. Le Roux; S. Wisdom; H. Erdogan; J. R. Hershey	SDR half-baked or well done?	https://scholar.google.com/scholar?q=SDR+half-baked+or+well+done%3F
2019	M. Doring; M. M. Kokuer	Practical NSGT audio applications	https://scholar.google.com/scholar?q=Practical+NSGT+audio+applications
2019	F.-R. Stoter; S. Uhlich; A. Liutkus; Y. Mitsufuji	Open-Unmix	https://scholar.google.com/scholar?q=Open-Unmix
2019	A. Defossez et al.	Music Source Separation in the Waveform Domain	https://scholar.google.com/scholar?q=Music+Source+Separation+in+the+Waveform+Domain
2019	K. Kumar et al.	MelGAN: Generative Adversarial Networks for Conditional Waveform Synthesis	https://scholar.google.com/scholar?q=MelGAN+Generative+Adversarial+Networks+for+Conditional+Waveform+Synthesis
2019	A. Pandey; D. Wang	A New Framework for CNN-Based Speech Enhancement in the Time Domain	https://scholar.google.com/scholar?q=A+New+Framework+for+CNN-Based+Speech+Enhancement+in+the+Time+Domain
2018	S. Bock; M. Korzeniowski	piano_transcription - F0 and onset tracking	https://scholar.google.com/scholar?q=piano_transcription-F0+and+onset+tracking
2018	N. W. D. Evans et al.	The voicehome and CHiME challenges	https://scholar.google.com/scholar?q=The+voicehome+and+CHiME+challenges
2018	M. Fenton	Practical Dynamics Processing for Modern Music Production	https://scholar.google.com/scholar?q=Practical+Dynamics+Processing+for+Modern+Music+Production
2018	N. Takahashi et al.	MMDenseLSTM for source separation	https://scholar.google.com/scholar?q=MMDenseLSTM+for+source+separation
2018	N. Kalchbrenner et al.	Efficient Neural Audio Synthesis	https://scholar.google.com/scholar?q=Efficient+Neural+Audio+Synthesis
2018	C. Donahue; J. McAuley; M. Puckette	Adversarial Audio Synthesis	https://scholar.google.com/scholar?q=Adversarial+Audio+Synthesis
2018	J.-M. Valin	A Hybrid DSP/Deep Learning Approach to Real-Time Full-Band Speech Enhancement	https://scholar.google.com/scholar?q=A+Hybrid+DSP+Deep+Learning+Approach+to+Real-Time+Full-Band+Speech+Enhancement
2017	Y. Wang et al.	Tacotron: Towards End-to-End Speech Synthesis	https://scholar.google.com/scholar?q=Tacotron+Towards+End-to-End+Speech+Synthesis
2017	A. Gibiansky et al.	Deep Voice 2: Multi-Speaker Neural Text-to-Speech	https://scholar.google.com/scholar?q=Deep+Voice+2+Multi-Speaker+Neural+Text-to-Speech
2016	K. Kinoshita et al.	A summary of the REVERB challenge	https://scholar.google.com/scholar?q=A+summary+of+the+REVERB+challenge
2015	M. Riess; A. R. Chhetri	Transient Preservation in Time-Scale Modification	https://scholar.google.com/scholar?q=Transient+Preservation+in+Time-Scale+Modification
2015	B. W. Schuller et al.	Paralinguistics and robust pitch tracking methods	https://scholar.google.com/scholar?q=Paralinguistics+and+robust+pitch+tracking+methods

Year	Authors	Title	URL
2015	M. Ballou	Handbook for Sound Engineers (Loudness and Dynamics chapters)	https://scholar.google.com/scholar?q=Ha
2014	J. Driedger; T. Pratzlich; M. Muller	Let It Bee - Towards NMF-Inspired Audio Mosaicing	https://scholar.google.com/scholar?q=Le+Towards+NMF-Inspired+Audio+Mosaicing
2013	P. C. Loizou	Speech Enhancement: Theory and Practice	https://scholar.google.com/scholar?q=Sp
2013	M. Dolson	History of the Phase Vocoder	https://scholar.google.com/scholar?q=Hi
2012	A. Hines et al.	VISQOL: An Objective Speech Quality Model	https://scholar.google.com/scholar?q=VI
2012	B. Zavalishin	The Art of VA Filter Design (sections on phase and spectral processing)	https://scholar.google.com/scholar?q=Th
2012	N. D. Gaubitch; P. A. Naylor	Speech Dereverberation	https://scholar.google.com/scholar?q=Sp
2012	K. Dressler	Sinusoidal Extraction using Frequency-Domain Matching Pursuit	https://scholar.google.com/scholar?q=Sin+Domain+Matching+Pursuit
2012	S. Essid; G. Richard	Musical signal analysis with reassignment methods	https://scholar.google.com/scholar?q=M
2012	Y. Yoshioka; T. Nakatani	Generalization of Multi-Channel Linear Prediction Methods for Blind MIMO Impulse Response Shortening	https://scholar.google.com/scholar?q=Ge+Channel+Linear+Prediction+Methods+f
2012	T. Yoshioka; T. Nakatani	Generalization of Multi-Channel Linear Prediction Methods for Blind MIMO Impulse Response Shortening	https://scholar.google.com/scholar?q=Ge+Channel+Linear+Prediction+Methods+f
2012	D. Giannoulis; M. Massberg; J. Reiss	Digital Dynamic Range Compressor Design	https://scholar.google.com/scholar?q=Di
2011	J. Reiss	Under the Hood of a Dynamic Range Compressor	https://scholar.google.com/scholar?q=Un
2011	N. Ono	Stable and fast update rules for independent low-rank matrix analysis based on auxiliary function technique	https://scholar.google.com/scholar?q=Sta+rank+matrix+analysis+based+on+auxili
2011	E. J. Candes; X. Li; Y. Ma; J. Wright	Robust Principal Component Analysis?	https://scholar.google.com/scholar?q=Ro
2011	K. K. Paliwal	Importance of phase in speech enhancement	https://scholar.google.com/scholar?q=Im

Year	Authors	Title	URL
2011	M. Le Roux; E. Vincent	Consistent Wiener Filtering for Audio Source Separation and Time-Frequency Processing	https://scholar.google.com/scholar?q=Consistent+Wiener+Filtering+for+Audio+Source+Separation+and+Time-Frequency+Processing
2011	C. Taal et al.	An Algorithm for Intelligibility Prediction of Time-Frequency Weighted Noisy Speech	https://scholar.google.com/scholar?q=An+Algorithm+for+Intelligibility+Prediction+of+Time-Frequency+Weighted+Noisy+Speech
2010	D. S. Hamon; A. Lazarides	Improved WSOLA for Real-Time Time-Scale Modification	https://scholar.google.com/scholar?q=Improved+WSOLA+for+Real-Time+Time-Scale+Modification
2010	A. V. Oppenheim; R. W. Schafer	Discrete-Time Signal Processing (STFT and filter-bank chapters)	https://scholar.google.com/scholar?q=Discrete-Time+Signal+Processing+%28STFT+and+filter-bank+chapters%29
2010	C. Schörkhuber; A. Klapuri	Constant-Q Transform Toolbox for Music Processing	https://scholar.google.com/scholar?q=Constant-Q+Transform+Toolbox+for+Music+Processing
2010	A. Roebel	A Shape-Invariant Phase Vocoder for Speech Transformation	https://scholar.google.com/scholar?q=A+Shape-Invariant+Phase+Vocoder+for+Speech+Transformation
2009	N. Moreau	Toolbox for Time-Scale Modification and Pitch-Shifting of Audio Signals	https://scholar.google.com/scholar?q=Toolbox+for+Time-Scale+Modification+and+Pitch-Shifting+of+Audio+Signals
2009	C. Fevotte; N. Bertin; J.-L. Durrieu	Nonnegative Matrix Factorization with the Itakura-Saito Divergence	https://scholar.google.com/scholar?q=Nonnegative+Matrix+Factorization+with+the+Itakura-Saito+Divergence
2008	J. Bonada	Wide-Band Harmonic Sinusoidal Modeling for Time-Scale and Pitch-Scale Modification	https://scholar.google.com/scholar?q=Wide-Band+Harmonic+Sinusoidal+Modeling+for+Time-Scale+and+Pitch-Scale+Modification
2007	T. Virtanen	Monaural Sound Source Separation by Nonnegative Matrix Factorization with Temporal Continuity and Sparseness Criteria	https://scholar.google.com/scholar?q=Monaural+Sound+Source+Separation+by+Nonnegative+Matrix+Factorization+with+Temporal+Continuity+and+Sparseness+Criteria
2007	T. Lund	Loudness and True-Peak in Digital Audio	https://scholar.google.com/scholar?q=Loudness+and+True-Peak+in+Digital+Audio
2007	P. Smaragdis	Convolutional Speech Bases and Their Application to Supervised Speech Separation	https://scholar.google.com/scholar?q=Convolutional+Speech+Bases+and+Their+Application+to+Supervised+Speech+Separation
2006	A. Klapuri	Multiple Fundamental Frequency Estimation by Harmonicity and Spectral Smoothness	https://scholar.google.com/scholar?q=Multiple+Fundamental+Frequency+Estimation+by+Harmonicity+and+Spectral+Smoothness

Year	Authors	Title	URL
2006	A. C. Gilbert; M. J. Strauss	Approximation of signals in sparse Fourier dictionaries	https://scholar.google.com/scholar?q=Approximation+of+signals+in+sparse+Fourier+dictionaries
2005	M. McLeod; G. Wyvill	A Smarter Way to Find Pitch	https://scholar.google.com/scholar?q=A+Smarter+Way+to+Find+Pitch
2004	A. de Cheveigne	Pitch and Time Manipulation of Speech	https://scholar.google.com/scholar?q=Pitch+and+Time+Manipulation+of+Speech
2002	P. Flandrin; F. Auger; E. Chassande-Mottin	Time-Frequency Reassignment: From Principles to Algorithms	https://scholar.google.com/scholar?q=Time-Frequency+Reassignment%3A+From+Principles+to+Algorithms
2002	C. Duxbury; M. Davies; M. Sandler	Improved Time-Scaling of Musical Audio Using Phase Locking at Transients	https://scholar.google.com/scholar?q=Improved+Time-Scaling+of+Musical+Audio+Using+Phase+Locking+at+Transients
2002	S. Arfib; D. Keiler; U. Zölzer	DAFX: Digital Audio Effects (chapter references on pitch/time processing)	https://scholar.google.com/scholar?q=DAFX+Digital+Audio+Effects
2001	I. Cohen; B. Berdugo	Speech Enhancement for Non-Stationary Noise Environments	https://scholar.google.com/scholar?q=Speech+Enhancement+for+Non-Stationary+Noise+Environments
2001	H. Kawahara; A. de Cheveigne	STRAIGHT, Exploitation of the Other Aspects of Vocal Source Information	https://scholar.google.com/scholar?q=STRAIGHT+Exploitation+of+the+Other+Aspects+of+Vocal+Source+Information
2001	A. Rix et al.	Perceptual Evaluation of Speech Quality (PESQ)	https://scholar.google.com/scholar?q=Perceptual+Evaluation+of+Speech+Quality+(PESQ)
2001	K. Grochenig	Foundations of Time-Frequency Analysis	https://scholar.google.com/scholar?q=Foundations+of+Time-Frequency+Analysis
2000	S. Kum; C. L. Nikias	Robust F0 Estimation in Noise	https://scholar.google.com/scholar?q=Robust+F0+Estimation+in+Noise
2000	A. Hyvarinen; E. Oja	Independent Component Analysis: Algorithms and Applications	https://scholar.google.com/scholar?q=Independent+Component+Analysis+Algorithms+and+Applications
2000	J. Bonada	Automatic Technique in Frequency Domain for Near-Lossless Time-Scale Modification of Audio	https://scholar.google.com/scholar?q=Automatic+Technique+in+Frequency+Domain+for+Near-Lossless+Time-Scale+Modification+of+Audio
1999	S. Ahmadi; H. Spanias	Cepstrum-Based Pitch Detection Using FFT	https://scholar.google.com/scholar?q=Cepstrum-Based+Pitch+Detection+Using+FFT
1998	P. Flandrin	Time-Frequency/Time-Scale Analysis	https://scholar.google.com/scholar?q=Time-Frequency%2FTime-Scale+Analysis
1998	S. Disch	A New Phase Vocoder Technique for Time-Scale Modification of Audio Signals	https://scholar.google.com/scholar?q=A+New+Phase+Vocoder+Technique+for+Time-Scale+Modification+of+Audio+Signals

Year	Authors	Title	URL
1997	S. Pulkki	Virtual Sound Source Positioning Using Vector Base Amplitude Panning	https://scholar.google.com/scholar?q=Vi
1996	P. Scalart; J. V. Filho	Speech Enhancement Based on a Priori Signal to Noise Estimation	https://scholar.google.com/scholar?q=Sp
1995	M. Puckette	Phase-Locked Vocoder	https://scholar.google.com/scholar?q=Ph
1995	D. L. Donoho	De-Noising by Soft-Thresholding	https://scholar.google.com/scholar?q=De
1995	A. J. Bell; T. J. Sejnowski	An Information-Maximization Approach to Blind Separation and Blind Deconvolution	https://scholar.google.com/scholar?q=Ar
1995	D. Talkin	A Robust Algorithm for Pitch Tracking (RAPT)	https://scholar.google.com/scholar?q=A
1993	S. Mallat; Z. Zhang	Matching Pursuits with Time-Frequency Dictionaries	https://scholar.google.com/scholar?q=Ma
1993	W. Verhelst; M. Roelands	An Overlap-Add Technique Based on Waveform Similarity (WSOLA) for High Quality Time-Scale Modification of Speech	https://scholar.google.com/scholar?q=W
1993	P. Boersma	Accurate Short-Term Analysis of the Fundamental Frequency and the Harmonics-to-Noise Ratio of a Sampled Sound	https://scholar.google.com/scholar?q=Ac
1991	S. Mann; S. Haykin	The Chirplet Transform: Physical Considerations	https://scholar.google.com/scholar?q=Th
1989	L. Cohen	Time-Frequency Distributions - A Review	https://scholar.google.com/scholar?q=Ti
1989	R. Bristow-Johnson; M. Bogdanowicz	Phase Vocoder Done Right	https://scholar.google.com/scholar?q=Ph
1986	R. J. McAulay; T. F. Quatieri	Speech Analysis/Synthesis Based on a Sinusoidal Representation	https://scholar.google.com/scholar?q=Sp
1985	N. Roucos; A. Wilgus	High Quality Time-Scale Modification for Speech	https://scholar.google.com/scholar?q=Hi

Year	Authors	Title	URL
1980	M. R. Portnoff	Time-Scale Modification of Speech Based on Short-Time Fourier Analysis	https://scholar.google.com/scholar?q=Time+Scale+Modification+of+Speech+Based+on+Short-Time+Fourier+Analysis
1979	S. Boll	Suppression of Acoustic Noise in Speech Using Spectral Subtraction	https://scholar.google.com/scholar?q=Suppression+of+Acoustic+Noise+in+Speech+Using+Spectral+Subtraction
1976	M. R. Portnoff	Implementation of the Digital Phase Vocoder Using the Fast Fourier Transform	https://scholar.google.com/scholar?q=Implementation+of+the+Digital+Phase+Vocoder+Using+the+Fast+Fourier+Transform
1976	L. R. Rabiner; M. J. Cheng; A. E. Rosenberg; C. A. McGonegal	A Comparative Performance Study of Several Pitch Detection Algorithms	https://scholar.google.com/scholar?q=A+Comparative+Performance+Study+of+Several+Pitch+Detection+Algorithms
1967	A. M. Noll	Cepstrum Pitch Determination	https://scholar.google.com/scholar?q=Cepstrum+Pitch+Determination
1949	N. Wiener	Extrapolation, Interpolation, and Smoothing of Stationary Time Series	https://scholar.google.com/scholar?q=Extrapolation,+Interpolation,+and+Smoothing+of+Stationary+Time+Series

CHAPTER F

PVC Phase 3-5 Architecture

This document describes the architecture used to implement the next three PVC-inspired phases:

1. Phase 3: response-driven spectral operators
2. Phase 4: ring/resonator family
3. Phase 5: harmonic/chord mapping family

F.1 Module Layout

Core signal processing modules:

- `/Users/cleider/dev/pvx/src/pvx/core/pvc_ops.py`
- `/Users/cleider/dev/pvx/src/pvx/core/pvc_resonators.py`
- `/Users/cleider/dev/pvx/src/pvx/core/pvc_harmony.py`

Command-line interface (CLI) entry points:

- Response operators:
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxfilter.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxtvfilter.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxnoisefilter.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxbandamp.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxspeccompander.py`
- Ring/resonator operators:
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxring.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxringfilter.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxringtvfilter.py`
- Harmonic/chord operators:
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxharmmap.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxchordmapper.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxinharmonator.py`

Unified dispatcher registration:

- `/Users/cleider/dev/pvx/src/pvx/cli/pvx.py`

F.2 Processing Contracts

All new operators follow the same signal contract:

1. Read audio (mono or multichannel) as floating point.
2. Process each channel deterministically.
3. Preserve sample rate and target output length.
4. Pass output through shared mastering/output policy.
5. Emit non-silent console metrics tables and status bars.

F.3 Phase 3 Design

`pvc_ops.py` provides five operators:

- **filter**: static response-curve spectral shaping
- **tvfilter**: time-varying response mix from CSV/JSON
- **noisefilter**: response-floor spectral attenuation
- **bandamp**: response-peak selective amplification
- **spec-compander**: response-referenced spectral companding

Key design choices:

- Response artifacts are loaded from PVXRF.
- Response curves are resized/shifted to match active STFT bins.
- Time-varying controls are scalar maps with interpolation modes:
 - `none`, `stairstep`, `nearest`, `linear`, `cubic`, `exponential`, `s_curve`, `smootherstep`, `polynomial`

F.4 Phase 4 Design

`pvc_resonators.py` provides:

- **ring**: ring modulation
- **ringfilter**: ring modulation + resonant peak filter
- **ringtvfilter**: time-varying ring controls + resonant filter

Key design choices:

- Sample-rate-domain modulation for phase continuity.
- Bounded feedback and bounded resonance decay for stability.
- Optional CSV/JSON control maps for **frequency_hz**, **depth**, and **mix**.

F.5 Phase 5 Design

`pvc_harmony.py` provides:

- **chordmapper**: chord-class spectral weighting (root + chord quality)
- **inharmonator**: inharmonic spectral frequency warp

Key design choices:

- Chord mapping uses circular pitch-class distance in cents.
- Inharmonator uses an analytic inverse warp from output-bin frequency to input-bin sampling frequency.
- Both operators are implemented in the short-time Fourier transform (STFT) domain with overlap-add resynthesis.

F.6 Backward Compatibility and UX

This section begins with the following points, which establish the context for the details that follow.

- Existing tools are untouched.
- New tools are additive and available both:
 - via unified command-line interface (CLI): `pvx <tool>`
 - via root wrappers: `python3 pvx<tool>.py ...`
- Shared conventions are reused: progress/status, verbosity, metrics table, mastering chain.

CHAPTER G

A Phase-Vocoder Listening Library: 100 Musical Works

This appendix is a guided listening and processing curriculum. It combines a small set of historically grounded phase-vocoder precedents with a much larger laboratory of famous works chosen because their voices, attacks, resonances, textures, or large-scale timing make particular transform behaviors easy to hear. The label attached to each entry matters. **Documented** identifies direct phase-vocoder practice supported by the historical literature. **Historical context** identifies related spectral and computer-music practice without claiming that every sound in the named work was made by a phase vocoder. **Listening laboratory** means that the work is recommended as source material or as a perceptual reference; it is not a claim about the production of the original recording.

YouTube changes individual upload URLs, regional availability, editions, and rights status frequently. For that reason, every entry supplies catalog-search links rather than endorsing one upload. The first search favors complete performances or recordings. The second favors scores, analyses, composer interviews, and demonstrations. Readers should choose lawful sources, respect recording and composition rights, and use public-domain or properly licensed audio for exported experiments.

The historically grounded group is supported by Trevor Wishart’s retrospective on twenty-five years of phase-vocoder practice, the Library of Congress genealogy of Roger Reynolds’s *Transfigured Wind*, IRCAM’s analysis of Jonathan Harvey’s *Mortuos Plango, Vivos Voco*, and the broader phase-vocoder literature cited in Chapter 1. These sources can be consulted through [Wishart’s retrospective](#), the [Library of Congress essay](#), and the [IRCAM analysis collection](#).

G.1 How to use the library

Each entry proposes one focused experiment. Begin with a short, legally obtained excerpt and keep an unprocessed reference at matched loudness. Change one variable at a time, render enough duration to expose the behavior, and record the command, window, hop, phase mode, transient policy, and random seed. The listening notes are prompts rather than expected answers.

G.2 Documented phase-vocoder practice

The works in this group emphasize documented phase-vocoder practice. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

G.2.1 1. Trevor Wishart: Vox 5

Documented phase-vocoder practice. *Trevor Wishart: Vox 5* (1986). Listen for the voice crossing into animal, mechanical, and environmental identities without a clean boundary.

Stretch a spoken vowel eight times, preserve formants, then morph its magnitude envelope toward a noisy recording.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.2.2 2. Roger Reynolds: Transfigured Wind I

Documented phase-vocoder practice. *Roger Reynolds: Transfigured Wind I* (1984). Follow the flute as a performed gesture becomes prolonged spectral architecture.

Freeze a flute tone after its attack, then automate stretch and spectral focus while retaining the original onset.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.2.3 3. JoAnn Kuchera-Morin: Dreampaths

Documented phase-vocoder practice. *JoAnn Kuchera-Morin: Dreampaths* (1989). Attend to long transformations whose internal spectral motion remains perceptible at expanded scale.

Expand a thirty-second source to three minutes and compare independent, locked, and transient-aware phase modes.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.2.4 4. Jonathan Harvey: Mortuos Plango, Vivos Voco

Documented phase-vocoder practice. *Jonathan Harvey: Mortuos Plango, Vivos Voco* (1980). Compare the partial structures of a boy's voice and a cathedral bell as they approach and depart from one another.

Analyze voice and bell recordings, transpose each by measured partial ratios, and cross-synthesize their envelopes.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.2.5 5. Trevor Wishart: Supernova

Documented phase-vocoder practice. *Trevor Wishart: Supernova* (2012). Listen for spectral data treated as compositional material across a large spatial and temporal field.

Create several extreme stretches from one source, filter different spectral regions, and distribute the layers in space.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.2.6 6. Trevor Wishart: Vox Cycle

Documented phase-vocoder practice. *Trevor Wishart: Vox Cycle* (1979-1988). Hear vocal identity as a continuum extending from speech and song to noise and imagined creatures.

Build a chain of short vocal transformations in which each output becomes the source for the next operation.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.3 Spectral-composition context

The works in this group emphasize spectral-composition context. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

G.3.1 7. Trevor Wishart: Red Bird

Related historical and spectral context. *Trevor Wishart: Red Bird* (1977). Study the dramatic continuity between recognizable recordings and transformed sonic images.

Use spectral morphing to connect speech, birdsong, machinery, and crowd noise while preserving selected gestures.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.3.2 8. Trevor Wishart: Tongues of Fire

Related historical and spectral context. *Trevor Wishart: Tongues of Fire* (1994). Listen for the multiplication of a single vocal world into a dense ecology of related timbres.

Separate voiced and noisy regions of speech, stretch them by different amounts, and recombine them with automation.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.3.3 9. Tristan Murail: Desintegrations

Related historical and spectral context. *Tristan Murail: Desintegrations* (1982-1983). Trace how instrumental harmony and computer-generated spectra share one harmonic vocabulary.

Extract prominent partials from a resonant source and retune an instrumental passage toward those frequencies.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.3.4 10. Wendy Carlos: Digital Moonscapes

Related historical and spectral context. *Wendy Carlos: Digital Moonscapes* (1984). Compare sharply articulated synthetic events with broad, evolving digital sonorities.

Render one synthesizer phrase at several stretch factors and compare transient preservation against spectral smoothness.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.3.5 11. Jean-Claude Risset: Inharmonique

Related historical and spectral context. *Jean-Claude Risset: Inharmonique* (1977). Listen for the changing boundary between soprano resonance and computer-generated spectral space.

Freeze several sung vowels, shift their formants independently of pitch, and layer the results under the original voice.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.3.6 12. Jean-Claude Risset: Songes

Related historical and spectral context. *Jean-Claude Risset: Songes* (1979). Attend to illusions of source, scale, and motion in a continuously changing electroacoustic field.

Automate spectral transposition and filtering on resonant recordings so that familiar sources become ambiguous.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.3.7 13. Kaija Saariaho: Lichtbogen

Related historical and spectral context. *Kaija Saariaho: Lichtbogen* (1986). Follow the exchange of brightness, noise, and harmonic focus between ensemble and electronics.

Track spectral centroid from a bowed tone and use it to control filtering and stretch in a second layer.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.3.8 14. Kaija Saariaho: NoaNoa

Related historical and spectral context. *Kaija Saariaho: NoaNoa* (1992). Listen to how breath, flute tone, speech, and electronics form a single timbral sentence.

Split flute attacks, breath noise, and sustained tone into separate paths, then transform each with a different window.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.3.9 15. Paul Lansky: Six Fantasies on a Poem by Thomas Campion

Related historical and spectral context. *Paul Lansky: Six Fantasies on a Poem by Thomas Campion* (1978-1979). Study the musical organization of speech resonance, contour, and repetition.

Retune and time-stretch spoken phrases while comparing preserved and shifted formant envelopes.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4 Voice, text, and formants

The works in this group emphasize voice, text, and formants. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

G.4.1 16. Hildegard von Bingen: *O vis aeternitatis*

Comparative listening laboratory. *Hildegard von Bingen: O vis aeternitatis* (12th century). The unaccompanied line exposes pitch continuity, breath, room decay, and vowel formants.

Stretch a sustained phrase four times with formant preservation, then repeat without it and compare vowel identity.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.2 17. Anonymous: *Dies irae*

Comparative listening laboratory. *Anonymous: Dies irae* (13th century). Monophonic chant makes phase continuity and consonant treatment unusually easy to hear.

Use transient protection on consonants while applying a slowly changing pitch map to the sustained syllables.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.3 18. Josquin des Prez: *Ave Maria virgo serena*

Comparative listening laboratory. *Josquin des Prez: Ave Maria virgo serena* (c. 1485). Imitative entries reveal whether independent vocal transformations retain contrapuntal clarity.

Stretch each voice separately with shared timing but independent formant settings, then compare against a linked render.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.4 19. Thomas Tallis: *Spem in alium*

Comparative listening laboratory. *Thomas Tallis: Spem in alium* (c. 1570). Forty vocal parts provide an exacting test of phase density, localization, and spectral accumulation.

Apply a modest global stretch, then compare channel-independent and reference-locked processing for image stability.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.5 20. Claudio Monteverdi: Lamento d'Arianna

Comparative listening laboratory. *Claudio Monteverdi: Lamento d'Arianna* (1608). The expressive line combines long vowels with speech-like attacks and rapid ornaments.

Protect attacks, stretch vowels selectively, and automate formant shift only during the longest notes.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.6 21. Henry Purcell: Dido's Lament

Comparative listening laboratory. *Henry Purcell: Dido's Lament* (1689). A descending ground and sustained vocal line make pitch drift and vibrato handling conspicuous.

Stretch the voice independently of the ground bass and test identity phase locking on the sustained notes.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.7 22. Johann Sebastian Bach: Erbarme dich

Comparative listening laboratory. *Johann Sebastian Bach: Erbarme dich* (1727). Voice and obbligato violin offer two related but distinct sources of vibrato and legato continuity.

Match their stretch maps, preserve each source's attacks, and compare linked versus independent phase treatment.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.8 23. George Frideric Handel: Lascia ch'io pianga

Comparative listening laboratory. *George Frideric Handel: Lascia ch'io pianga* (1711). Regular phrases and exposed vowels make formant displacement and modulation artifacts easy to identify.

Create three pitch-shifted versions at minus five, zero, and plus seven semitones with a fixed formant envelope.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.9 24. Wolfgang Amadeus Mozart: Der Holle Rache

Comparative listening laboratory. *Wolfgang Amadeus Mozart: Der Holle Rache* (1791). Coloratura attacks stress transient detection, peak tracking, and rapid pitch movement.

Compare short and long windows on a fast passage, keeping output duration and pitch unchanged.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.10 25. Franz Schubert: Erlkönig

Comparative listening laboratory. *Franz Schubert: Erlkönig* (1815). Rapid text, register changes, and repeated piano figures reveal temporal blur immediately.

Stretch voice and piano separately, using stronger transient protection for the accompaniment.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.11 26. Richard Wagner: Mild und leise

Comparative listening laboratory. *Richard Wagner: Mild und leise* (1859). The long final ascent tests spectral accumulation, vibrato continuity, and orchestral masking.

Freeze a climactic vowel and slowly expand its spectral envelope while the orchestral source remains time-aligned.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.12 27. Gustav Mahler: Ich bin der Welt abhanden gekommen

Comparative listening laboratory. *Gustav Mahler: Ich bin der Welt abhanden gekommen* (1901). Quiet sustained singing against transparent orchestration exposes noise, breath, and low-level phase modulation.

Render a gentle 1.5-times stretch with three windows and compare breath texture and instrumental tails.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.13 28. Arnold Schoenberg: Mondestrunken

Comparative listening laboratory. *Arnold Schoenberg: Mondestrunken* (1912). Sprechstimme sits between speech and song, making pitch and formant policy an artistic decision.

Extract the pitch contour, smooth it at several rates, and use it to retune a duplicated vocal layer.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.14 29. Luciano Berio: Sequenza III

Comparative listening laboratory. *Luciano Berio: Sequenza III* (1965). Whispers, laughter, phonemes, and sung tones demand separate treatment of voiced and unvoiced material.

Route voiced and noisy frames to different transformations, then crossfade the outputs using the original voicing measure.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.4.15 30. Steve Reich: Different Trains

Comparative listening laboratory. *Steve Reich: Different Trains* (1988). Speech melody, instrumental doubling, and repeated pulse invite close comparison of contour extraction and resynthesis.

Track a spoken phrase, retune a string sample to the contour, and preserve consonants in a parallel dry path.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5 Sustained tone and resonance

The works in this group emphasize sustained tone and resonance. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

G.5.1 31. Antonio Vivaldi: Winter, Largo

Comparative listening laboratory. *Antonio Vivaldi: Winter, Largo* (1725). The violin melody and pizzicato accompaniment separate sustained tone from short attacks.

Stretch the solo line strongly while leaving the accompaniment nearly unchanged, then recombine them.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.2 32. Johann Sebastian Bach: Air on the G String

Comparative listening laboratory. *Johann Sebastian Bach: Air on the G String* (c. 1730). Long string lines and stepwise bass motion reveal modulation and ensemble coherence.

Compare a single stereo render with stem-based processing under a shared stretch trajectory.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.3 33. Ludwig van Beethoven: Symphony No. 7, Allegretto

Comparative listening laboratory. *Ludwig van Beethoven: Symphony No. 7, Allegretto* (1812). Repeated rhythm under expanding orchestral layers tests sustained resonance without losing pulse.

Freeze selected chord tails while preserving the dry rhythmic attacks in a parallel path.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.4 34. Franz Schubert: String Quintet in C, Adagio

Comparative listening laboratory. *Franz Schubert: String Quintet in C, Adagio* (1828). Wide string registers and very long phrases expose beating, detuning, and vertical phase coherence.

Apply a two-times stretch with peak locking, then compare it with independent-bin propagation.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.5 35. Frederic Chopin: Nocturne in E-flat major, Op. 9 No. 2

Comparative listening laboratory. *Frederic Chopin: Nocturne in E-flat major, Op. 9 No. 2* (1832). Piano resonance, ornament, and rubato provide a compact test of tail preservation.

Protect note onsets while extending only the decays, using a time-varying stretch map.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.6 36. Richard Wagner: Prelude to Das Rheingold

Comparative listening laboratory. *Richard Wagner: Prelude to Das Rheingold* (1869). A single harmonic field grows from quiet fundamentals into dense orchestral resonance.

Extend the opening four times and automate window length as the orchestration becomes denser.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.7 37. Anton Bruckner: Symphony No. 7, Adagio

Comparative listening laboratory. *Anton Bruckner: Symphony No. 7, Adagio* (1883). Brass chorales and string sustain test phase stability at high spectral density.

Freeze a brass chord, retune selected partial regions, and crossfade into a stretched continuation.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.8 38. Claude Debussy: Prelude a l'après-midi d'un faune

Comparative listening laboratory. *Claude Debussy: Prelude a l'après-midi d'un faune* (1894). Solo winds and blended orchestration make timbral identity more important than strict note attacks.

Morph the flute magnitude envelope toward a string harmonic while preserving the flute phase trajectory.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.9 39. Maurice Ravel: Daphnis et Chloe, Lever du jour

Comparative listening laboratory. *Maurice Ravel: Daphnis et Chloe, Lever du jour* (1912). The gradual orchestral sunrise tests long dynamic trajectories and accumulating spectral brightness.

Use centroid-following automation to open a spectral filter over an extended stretch.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.10 40. Ralph Vaughan Williams: Fantasia on a Theme by Thomas Tallis

Comparative listening laboratory. *Ralph Vaughan Williams: Fantasia on a Theme by Thomas Tallis* (1910). Divided strings and antiphonal groups expose spatial coherence and slowly evolving resonance.

Process ensemble groups separately with one reference phase track and compare the stereo image.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.11 41. Gustav Holst: Neptune, the Mystic

Comparative listening laboratory. *Gustav Holst: Neptune, the Mystic* (1915). Static harmony, celesta, and distant chorus make an ideal extreme-stretch texture.

Render an eight-times stretch with slow spectral filtering and a long wet-dry automation curve.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.12 42. Olivier Messiaen: Louange a l'Eternite de Jesus

Comparative listening laboratory. *Olivier Messiaen: Louange a l'Eternite de Jesus* (1941). Extremely slow cello melody over repeated piano chords separates continuity from attack.

Stretch the cello with phase locking while processing piano attacks and decays on separate paths.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.13 43. Gyorgy Ligeti: Lux Aeterna

Comparative listening laboratory. *Gyorgy Ligeti: Lux Aeterna* (1966). Micropolyphonic voices form a dense, slowly moving spectral cloud.

Apply a subtle spectral tilt and two-times stretch, then inspect whether internal beating remains alive.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.14 44. Arvo Part: Spiegel im Spiegel

Comparative listening laboratory. *Arvo Part: Spiegel im Spiegel* (1978). Sparse piano attacks and a sustained melodic line reveal every artifact.

Compare transient reset, identity locking, and independent phase propagation at identical settings.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.5.15 45. John Tavener: The Protecting Veil

Comparative listening laboratory. *John Tavener: The Protecting Veil* (1988). Extended cello phrases and luminous string harmony reward gradual formant and spectral-envelope motion.

Automate a modest downward formant shift across one phrase without changing its fundamental pitch.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6 Transient and rhythmic stress tests

The works in this group emphasize transient and rhythmic stress tests. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

G.6.1 46. Johann Sebastian Bach: Prelude in C major, BWV 846

Comparative listening laboratory. *Johann Sebastian Bach: Prelude in C major, BWV 846* (1722). Evenly spaced piano attacks make timing blur and pre-echo easy to hear.

Stretch two times using several transient thresholds and compare note-front sharpness.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.2 47. Ludwig van Beethoven: Piano Sonata No. 14, third movement

Comparative listening laboratory. *Ludwig van Beethoven: Piano Sonata No. 14, third movement* (1801). Fast repeated attacks stress onset detection and frame resetting.

Slow the passage to half tempo while preserving pitch, then audit flam, smearing, and high-frequency loss.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.3 48. Niccolò Paganini: Caprice No. 24

Comparative listening laboratory. *Niccolò Paganini: Caprice No. 24* (1817). Rapid articulation and register leaps reveal peak-tracking failures.

Compare short-window and multiresolution processing on the fastest variation.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.4 49. Frederic Chopin: Etude Op. 10 No. 4

Comparative listening laboratory. *Frederic Chopin: Etude Op. 10 No. 4* (1830). Continuous sixteenth notes expose accumulated timing and phase errors.

Render 0.75-times and 1.5-times versions with identical loudness and compare attack definition.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.5 50. Nikolai Rimsky-Korsakov: Flight of the Bumblebee

Comparative listening laboratory. *Nikolai Rimsky-Korsakov: Flight of the Bumblebee* (1900). Near-continuous fast notes are a severe test of local stationarity.

Sweep FFT size over the passage and graph transient flux against perceived clarity.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.6 51. Igor Stravinsky: The Rite of Spring, Augurs of Spring

Comparative listening laboratory. *Igor Stravinsky: The Rite of Spring, Augurs of Spring* (1913). Accented orchestral chords test synchronized transient handling across many channels.

Use a shared onset map for all stems and compare it with independent per-stem detection.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.7 52. Edgard Varese: Ionisation

Comparative listening laboratory. *Edgard Varese: Ionisation* (1931). Percussion timbres expose both attack smear and spectral-noise coloration.

Stretch metal, skin, and noise instruments separately using windows matched to their decay behavior.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.8 53. John Cage: First Construction in Metal

Comparative listening laboratory. *John Cage: First Construction in Metal* (1939). Metallic attacks and long resonances invite independent control of onset and tail.

Split at detected transients, preserve each attack, and stretch the following decay by a larger factor.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.9 54. Dave Brubeck Quartet: Take Five

Comparative listening laboratory. *Dave Brubeck Quartet: Take Five* (1959). The clear quintuple meter reveals timing drift while cymbals challenge phase coherence.

Slow the track while preserving pitch and compare cymbal texture under several transient modes.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.10 55. James Brown: Funky Drummer

Comparative listening laboratory. *James Brown: Funky Drummer* (1970). The famous drum break provides tightly related kick, snare, and hi-hat transients.

Create a half-time render, then compare broadband and band-dependent transient protection.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.11 56. Kraftwerk: Numbers

Comparative listening laboratory. *Kraftwerk: Numbers* (1981). Sparse electronic percussion and speech make small timing errors obvious.

Stretch the rhythm independently from the voice, then align both with checkpointed rendering.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.12 57. Public Enemy: Bring the Noise

Comparative listening laboratory. *Public Enemy: Bring the Noise* (1987). Dense samples, voice, and percussion test a phase vocoder under deliberate spectral congestion.

Compare full-mix stretching against source-separated processing at the same target duration.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.13 58. Aphex Twin: Vordhosbn

Comparative listening laboratory. *Aphex Twin: Vordhosbn* (2001). Rapid programmed attacks and elastic timing push conventional onset models hard.

Apply a gentle stretch, inspect the spectrogram, and tune transient sensitivity to avoid doubled attacks.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.14 59. Squarepusher: My Red Hot Car

Comparative listening laboratory. *Squarepusher: My Red Hot Car* (2001). Breakbeat detail and pitched bass expose different optimal frame scales.

Use multiband processing with shorter frames above the crossover and longer frames below it.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.6.15 60. Autechre: Gantz Graf

Comparative listening laboratory. *Autechre: Gantz Graf* (2002). Abrupt synthetic gestures reveal frame boundaries, ringing, and interpolation behavior.

Render a continuously varying stretch curve and compare linear, cubic, and smoothstep automation.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7 Polyphony and orchestral density

The works in this group emphasize polyphony and orchestral density. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

G.7.1 61. Giovanni Pierluigi da Palestrina: Missa Papae Marcelli, Kyrie

Comparative listening laboratory. *Giovanni Pierluigi da Palestrina: Missa Papae Marcelli, Kyrie* (1562). Closely spaced voices test vertical phase coherence without instrumental attacks.

Stretch a choral excerpt with identity locking and compare the intelligibility of inner voices.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.2 62. Johann Sebastian Bach: The Art of Fugue, Contrapunctus I

Comparative listening laboratory. *Johann Sebastian Bach: The Art of Fugue, Contrapunctus I* (1740s). Independent lines provide a controlled test of polyphonic clarity.

Process each contrapuntal voice as a stem under one shared timing map, then compare a full-mix render.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.3 63. Wolfgang Amadeus Mozart: Symphony No. 40, first movement

Comparative listening laboratory. *Wolfgang Amadeus Mozart: Symphony No. 40, first movement* (1788). Repeated figures and orchestral doublings reveal loss of articulation and image focus.

Apply a modest tempo change and compare peak locking with transient-reset synthesis.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.4 64. Ludwig van Beethoven: Symphony No. 9, finale

Comparative listening laboratory. *Ludwig van Beethoven: Symphony No. 9, finale* (1824). Soloists, chorus, and orchestra create extreme density across the full spectrum.

Render selected sections with stem-aware settings and test loudness normalization only after recombination.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.5 65. Hector Berlioz: *Symphonie fantastique*, *Marche au supplice*

Comparative listening laboratory. *Hector Berlioz: Symphonie fantastique, Marche au supplice* (1830). Orchestral attacks, rolls, and brass resonance demand mixed transient policies.

Drive transient sensitivity from spectral flux and protect low-frequency drum attacks separately.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.6 66. Johannes Brahms: *Symphony No. 4*, finale

Comparative listening laboratory. *Johannes Brahms: Symphony No. 4, finale* (1885). Variation form supplies repeated material under changing orchestration.

Use matched excerpts to compare how one parameter set behaves as texture density changes.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.7 67. Gustav Mahler: *Symphony No. 2*, finale

Comparative listening laboratory. *Gustav Mahler: Symphony No. 2, finale* (1894). The progression from near silence to full chorus and orchestra tests dynamic and spectral range.

Automate FFT size, wet-dry balance, and spectral smoothing according to measured density.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.8 68. Claude Debussy: *La mer*, *Dialogue du vent et de la mer*

Comparative listening laboratory. *Claude Debussy: La mer, Dialogue du vent et de la mer* (1905). Layered orchestral motion tests phase coherence without a single dominant pulse.

Stretch by 1.25 times and compare frequency-dependent phase locking across low, middle, and high bands.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.9 69. Charles Ives: *The Unanswered Question*

Comparative listening laboratory. *Charles Ives: The Unanswered Question* (1908). Three independent musical layers invite distinct temporal transformations.

Stretch the string background, leave the trumpet near original time, and compress the woodwind interruptions.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.10 70. Igor Stravinsky: The Firebird, Infernal Dance

Comparative listening laboratory. *Igor Stravinsky: The Firebird, Infernal Dance* (1910). Sharp attacks and bright orchestration reveal pre-echo and high-frequency smearing.

Compare overlap factors while holding window length fixed and log transient metrics.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.11 71. Bela Bartok: Music for Strings, Percussion and Celesta, third movement

Comparative listening laboratory. *Bela Bartok: Music for Strings, Percussion and Celesta, third movement* (1936). Glissandi, tremolo, percussion, and spatial string groups cover many phase-vocoder edge cases.

Assign different analysis presets to tremolo strings, pitched percussion, and noise-like gestures.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.12 72. Benjamin Britten: The Young Person's Guide to the Orchestra

Comparative listening laboratory. *Benjamin Britten: The Young Person's Guide to the Orchestra* (1945). Successive instrumental families make timbre-dependent artifacts easy to compare.

Run one preset across each variation and assemble a window-by-instrument evaluation table.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.13 73. Iannis Xenakis: Metastaseis

Comparative listening laboratory. *Iannis Xenakis: Metastaseis* (1954). Massed glissandi challenge peak identity and frequency-trajectory tracking.

Slow the central texture and compare independent bins with peak-tracked partial trajectories.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.14 74. Gyorgy Ligeti: Atmospheres

Comparative listening laboratory. *Gyorgy Ligeti: Atmospheres* (1961). Dense micropolyphony tests whether spectral processing preserves internal motion rather than flattening it.

Freeze successive frames and crossfade them to study the boundary between texture and stasis.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.7.15 75. John Adams: Short Ride in a Fast Machine

Comparative listening laboratory. *John Adams: Short Ride in a Fast Machine* (1986). A steady pulse under bright orchestration exposes timing drift immediately.

Apply a curved tempo map while using a shared transient clock for the entire ensemble.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8 Electronic, studio, and ambient listening

The works in this group emphasize electronic, studio, and ambient listening. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

G.8.1 76. Karlheinz Stockhausen: *Gesang der Junglinge*

Comparative listening laboratory. *Karlheinz Stockhausen: Gesang der Junglinge* (1956). Electronic tones and a boy's voice establish a foundational comparison between synthesis and recorded sound.

Morph magnitude envelopes between a sung vowel and a synthetic oscillator bank.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.2 77. Karlheinz Stockhausen: *Kontakte*

Comparative listening laboratory. *Karlheinz Stockhausen: Kontakte* (1958-1960). Continuities between pitch, pulse, and timbre suggest phase-vocoder transformations across temporal scale.

Stretch short impulses until their spectra become sustained tones, then reverse the process.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.3 78. Delia Derbyshire: *Doctor Who Theme*

Comparative listening laboratory. *Delia Derbyshire: Doctor Who Theme* (1963). Layered electronic lines and tape-shaped attacks offer clear material for pitch and duration experiments.

Shift one melodic layer by a fifth without changing duration and compare formant-preserving and unconstrained modes.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.4 79. The Beatles: *Tomorrow Never Knows*

Comparative listening laboratory. *The Beatles: Tomorrow Never Knows* (1966). Tape loops, drone, and processed voice provide contrasting repetitive sources.

Create phase-aligned loop layers at several stretch ratios and automate their spectral masks.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.5 80. Pink Floyd: On the Run

Comparative listening laboratory. *Pink Floyd: On the Run* (1973). Sequenced motion and noise effects reveal how stretching changes perceived speed and space.

Apply a nonlinear time map that slows the middle while preserving the beginning and ending durations.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.6 81. Brian Eno: An Ending (Ascent)

Comparative listening laboratory. *Brian Eno: An Ending (Ascent)* (1983). Slow harmonic motion and diffuse attacks make spectral breathing easy to study.

Extend the piece-like source four times and automate room size and wet-dry mix over the render.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.7 82. Laurie Anderson: O Superman

Comparative listening laboratory. *Laurie Anderson: O Superman* (1981). Processed voice and a repeated pulse separate formant character from rhythmic continuity.

Stretch the voice while keeping the pulse fixed, then retune the vocal layer with preserved formants.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.8 83. Kate Bush: Running Up That Hill

Comparative listening laboratory. *Kate Bush: Running Up That Hill* (1985). Layered drums, synthesizers, and expressive voice expose compromises in full-mix processing.

Compare a full-mix stretch with a stem-separated render and document vocal consonants and drum transients.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.9 84. Cocteau Twins: Lorelei

Comparative listening laboratory. *Cocteau Twins: Lorelei* (1984). Dense reverberant voice and guitar create a complex field of overlapping partials.

Freeze and retune a reverberant tail while preserving a parallel dry attack layer.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.10 85. My Bloody Valentine: Only Shallow

Comparative listening laboratory. *My Bloody Valentine: Only Shallow* (1991). Distorted guitars and diffuse vocals test processing on intentionally blurred spectra.

Use spectral filtering and small pitch shifts to separate layers without reducing their textural density.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.11 86. The Orb: Little Fluffy Clouds

Comparative listening laboratory. *The Orb: Little Fluffy Clouds* (1990). Speech, ambience, and repeating electronic material invite independent time-scale treatment.

Stretch the environmental bed, keep speech near original timing, and interpolate between the two clocks.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.12 87. Boards of Canada: Roygbiv

Comparative listening laboratory. *Boards of Canada: Roygbiv* (1998). Soft attacks and unstable analog pitch make excessive phase correction easy to hear.

Apply gentle time stretching while preserving low-frequency modulation and avoiding aggressive pitch stabilization.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.13 88. Bjork: All Is Full of Love

Comparative listening laboratory. *Bjork: All Is Full of Love* (1997). Layered voices and electronic resonance form a rich laboratory for formant-aware processing.

Create a three-voice spectral chorus from one line using independent pitch and formant trajectories.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.14 89. Radiohead: Everything in Its Right Place

Comparative listening laboratory. *Radiohead: Everything in Its Right Place* (2000). Looped keyboard harmony and fragmented voice demonstrate repetition at multiple scales.

Freeze the keyboard bed, granularly reshape vocal timing, and join both with a common automation curve.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.8.15 90. Oneohtrix Point Never: Replica

Comparative listening laboratory. *Oneohtrix Point Never: Replica* (2011). Short sampled fragments become harmonically and emotionally legible through repetition and transformation.

Stretch one fragment to a sustained pad, retain another as a transient marker, and morph between them.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9 Extreme duration and temporal form

The works in this group emphasize extreme duration and temporal form. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

G.9.1 91. Johann Pachelbel: Canon in D

Comparative listening laboratory. *Johann Pachelbel: Canon in D* (c. 1680). The repeating bass and accumulating canonic voices make long-form timing relationships transparent.

Assign progressively larger stretch factors to successive entries while preserving one common harmonic grid.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9.2 92. Ludwig van Beethoven: Symphony No. 5, opening

Comparative listening laboratory. *Ludwig van Beethoven: Symphony No. 5, opening* (1808). The universally recognizable motif makes even radical temporal transformation perceptually traceable.

Expand the four-note gesture one hundred times, protect the initial attacks, and study the resulting internal spectra.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9.3 93. Richard Wagner: Prelude to Tristan und Isolde

Comparative listening laboratory. *Richard Wagner: Prelude to Tristan und Isolde* (1859). Suspended harmonic resolution and long crescendos suit continuous nonlinear stretching.

Draw a stretch map that slows harmonic tension points and accelerates connective material.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9.4 94. Maurice Ravel: Bolero

Comparative listening laboratory. *Maurice Ravel: Bolero* (1928). A nearly fixed tempo and melody under continuous orchestral growth isolate changes in timbre and scale.

Hold duration constant while morphing spectral envelopes among successive instrumental statements.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9.5 95. Samuel Barber: Adagio for Strings

Comparative listening laboratory. *Samuel Barber: Adagio for Strings* (1936). Long arches and gradual registral expansion reveal drift over extended processing.

Apply a stretch factor that rises toward the climax and returns afterward, with checkpointed rendering.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9.6 96. John Cage: Organ2/ASLSP

Comparative listening laboratory. *John Cage: Organ2/ASLSP* (1987). The premise of performing as slowly as possible invites a direct investigation of computationally extreme duration.

Render a short organ chord at escalating factors, then compare memory, checkpoint, and streaming strategies.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9.7 97. Steve Reich: Piano Phase

Comparative listening laboratory. *Steve Reich: Piano Phase* (1967). Gradual process depends on tiny timing differences that phase-vocoder automation can imitate or destroy.

Create two copies with slightly different time maps and measure their evolving alignment.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9.8 98. Alvin Lucier: I Am Sitting in a Room

Comparative listening laboratory. *Alvin Lucier: I Am Sitting in a Room* (1969). Iterated room coloration demonstrates how a transfer function becomes musical form.

Estimate the response of one generation and apply it iteratively while controlling normalization and decay.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9.9 99. William Basinski: The Disintegration Loops 1.1

Comparative listening laboratory. *William Basinski: The Disintegration Loops 1.1* (2002). Slow material change across repetition suggests automation beyond a fixed stretch factor.

Build a loop whose bandwidth, noise floor, and stretch factor evolve slightly on every pass.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.9.10 100. Max Richter: Sleep

Comparative listening laboratory. *Max Richter: Sleep* (2015). An eight-hour form supplies a practical model for streaming, checkpoints, and imperceptibly slow parameter motion.

Render a long source in resumable chunks while automating room size, wet-dry mix, and spectral brightness.

Catalog searches provide two starting points: [YouTube performance search](#) and [YouTube score, analysis, and interview search](#).

G.10 Comparative listening worksheet

The catalog becomes more useful when impressions are recorded consistently. For each experiment, note the source and edition, time range, rights status, pvx command, window and hop, stretch or pitch trajectory, phase mode, transient policy, normalization method, and monitoring level. Then rate attack definition, tonal focus, formant credibility, ambience continuity, stereo stability, noise coloration, and overall musical usefulness. A failed transformation is still useful evidence when its settings and source are documented.

The most revealing comparisons use matched loudness and short randomized presentation. Keep the dry source, a conservative transform, and an intentionally extreme transform. Describe artifacts in operational terms such as doubled attack, pre-echo, diffuse partial, unstable center image, shifted vowel, or metallic tail. Those descriptions lead back to parameters more reliably than a single global quality score.

CHAPTER H

Transfer-Function and Automation Graph Atlas

This appendix restores the project graphs at a scale suitable for print. Each graph is introduced by a short statement of the mathematical or control relationship it illustrates. The figures are original pvx project assets.

H.1 Transfer functions

The transfer-function group covers pitch ratios, dynamics, soft clipping, spectral morphing, masking, and phase mixing. These graphs connect command-line values to the curves applied inside a render.

$$r_s = 2^{s/12}, \quad r_c = 2^{c/1200}$$

where r_s is the pitch ratio for s semitones and r_c is the pitch ratio for c cents.

H.1.1 pitch ratio from semitones

The following graph presents the pitch ratio from semitones relationship at full print scale. Read the horizontal axis as the controlling value and the vertical axis as the resulting ratio, gain, magnitude, or angle identified by the labels.

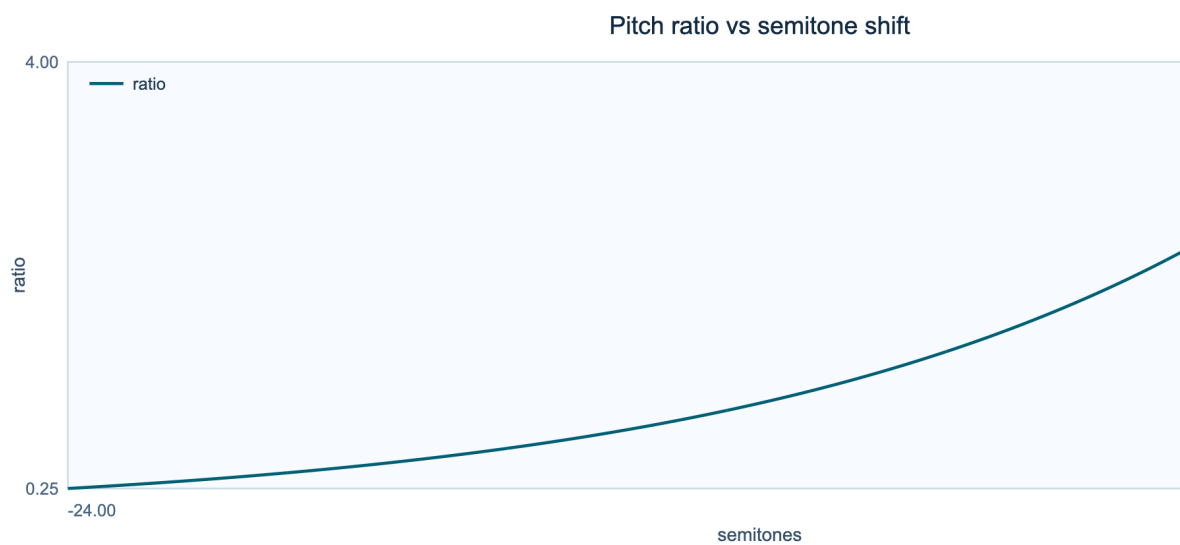


Figure H.1: Pitch ratio from semitones.

H.1.2 pitch ratio from cents

The following graph presents the pitch ratio from cents relationship at full print scale. Read the horizontal axis as the controlling value and the vertical axis as the resulting ratio, gain, magnitude, or angle identified by the labels.

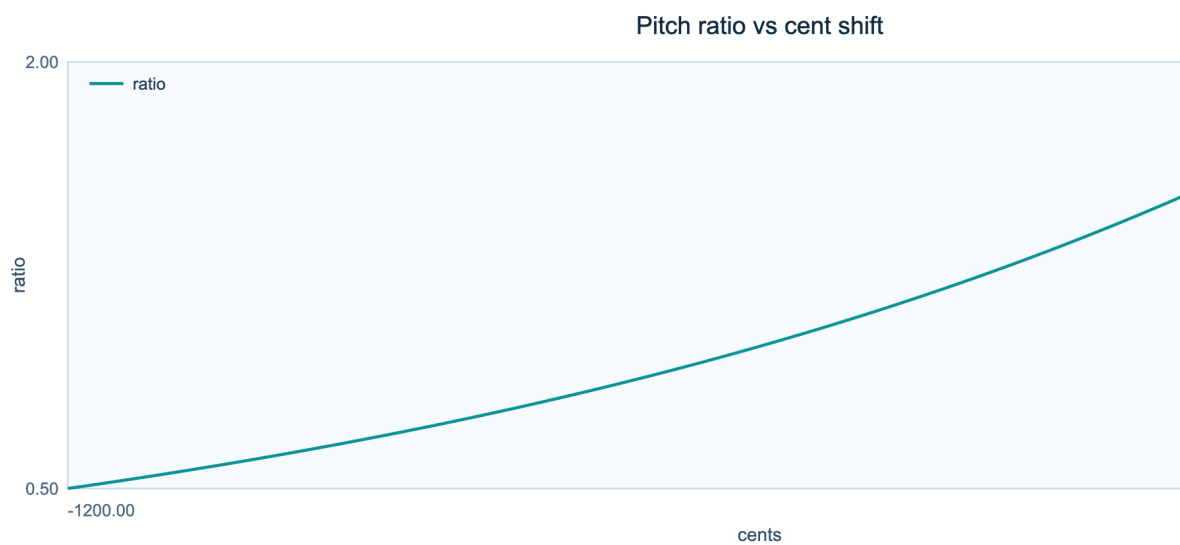


Figure H.2: Pitch ratio from cents.

H.1.3 dynamics transfer curves

The following graph presents the dynamics transfer curves relationship at full print scale. Read the horizontal axis as the controlling value and the vertical axis as the resulting ratio, gain, magnitude, or angle identified by the labels.

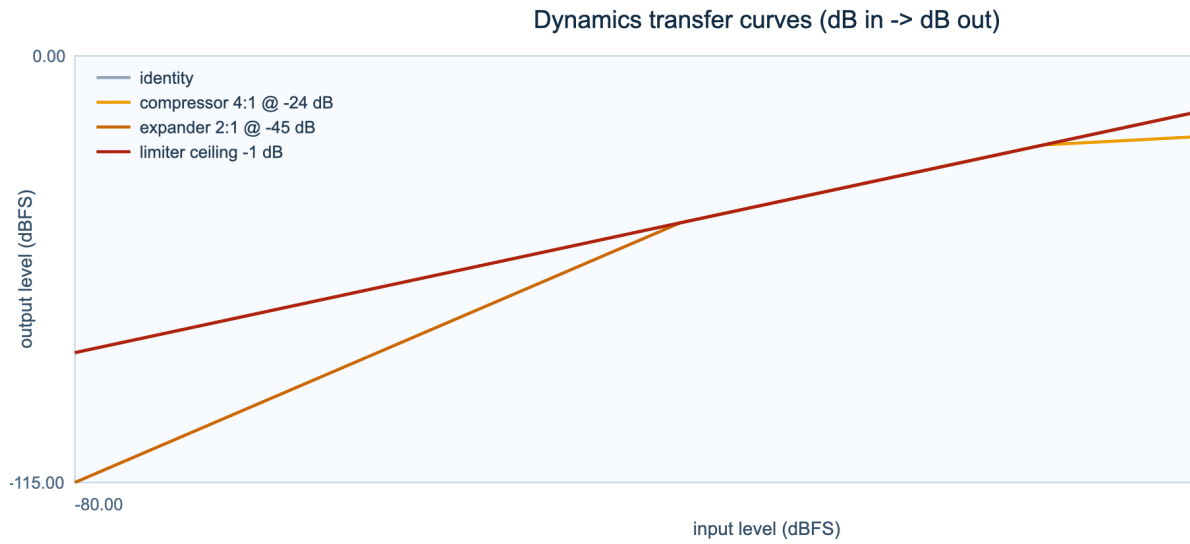


Figure H.3: Dynamics transfer curves.

H.1.4 soft-clip transfer functions

The following graph presents the soft-clip transfer functions relationship at full print scale. Read the horizontal axis as the controlling value and the vertical axis as the resulting ratio, gain, magnitude, or angle identified by the labels.

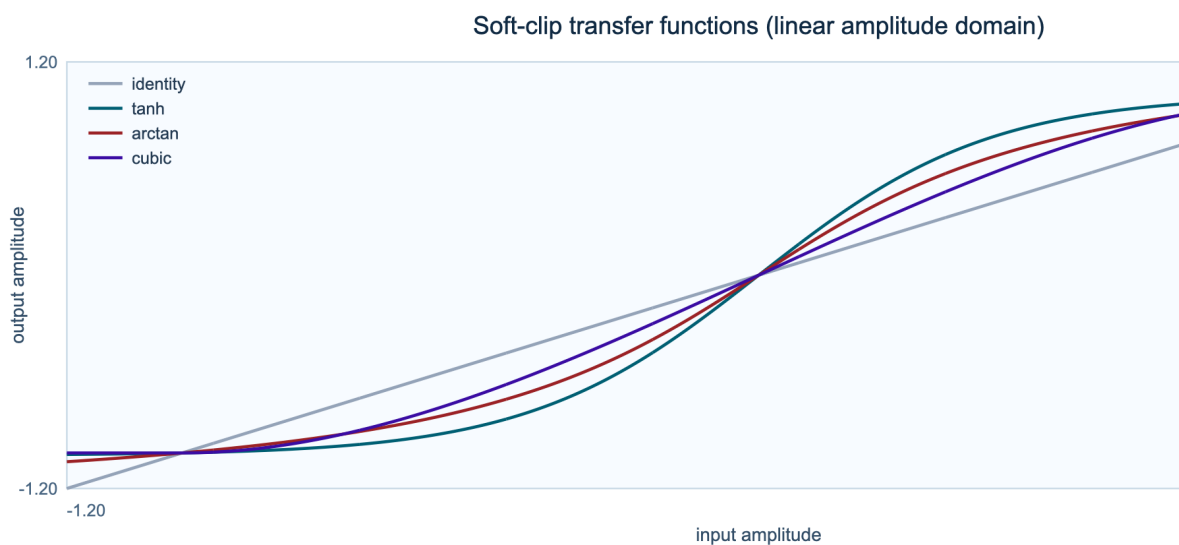


Figure H.4: Soft-clip transfer functions.

H.1.5 morph blend magnitude curves

The following graph presents the morph blend magnitude curves relationship at full print scale. Read the horizontal axis as the controlling value and the vertical axis as the resulting ratio, gain, magnitude, or angle identified by the labels.

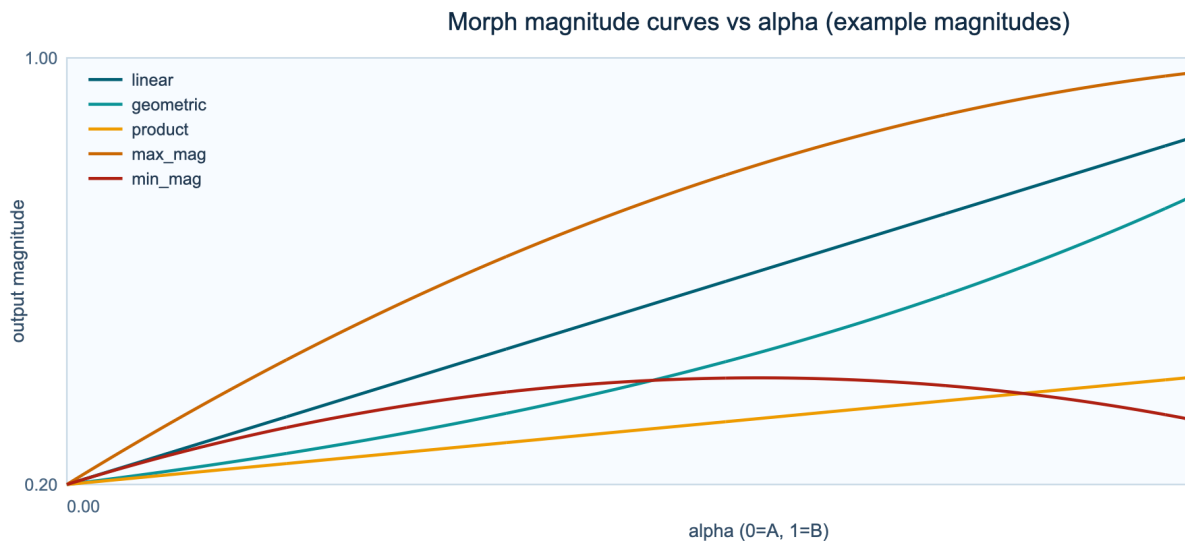


Figure H.5: Morph blend magnitude curves.

H.1.6 mask exponent response

The following graph presents the mask exponent response relationship at full print scale. Read the horizontal axis as the controlling value and the vertical axis as the resulting ratio, gain, magnitude, or angle identified by the labels.

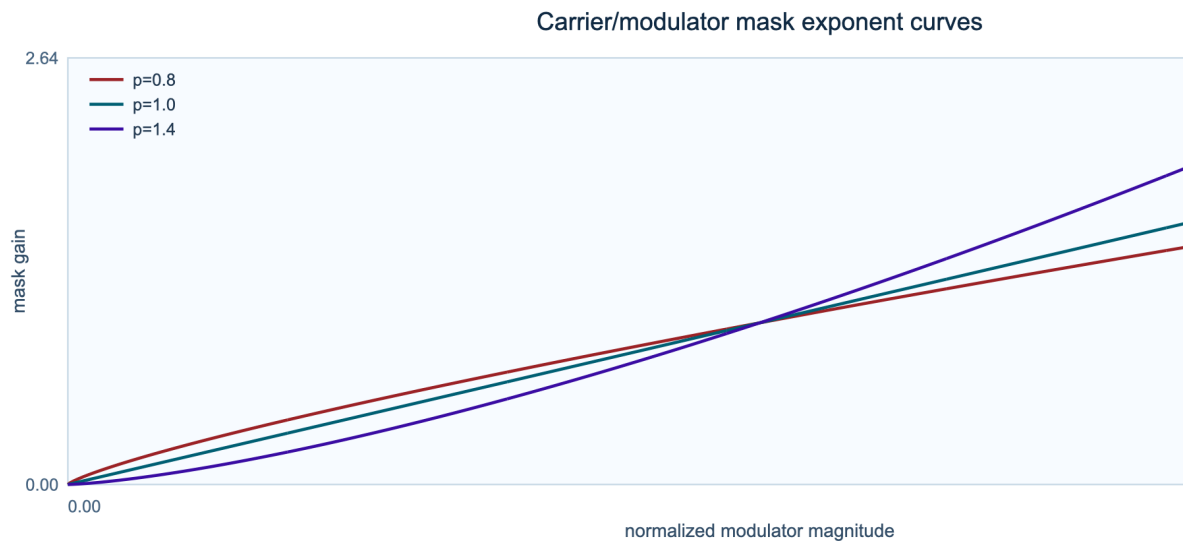


Figure H.6: Mask exponent response.

H.1.7 phase-mix angle response

The following graph presents the phase-mix angle response relationship at full print scale. Read the horizontal axis as the controlling value and the vertical axis as the resulting ratio, gain, magnitude, or angle identified by the labels.

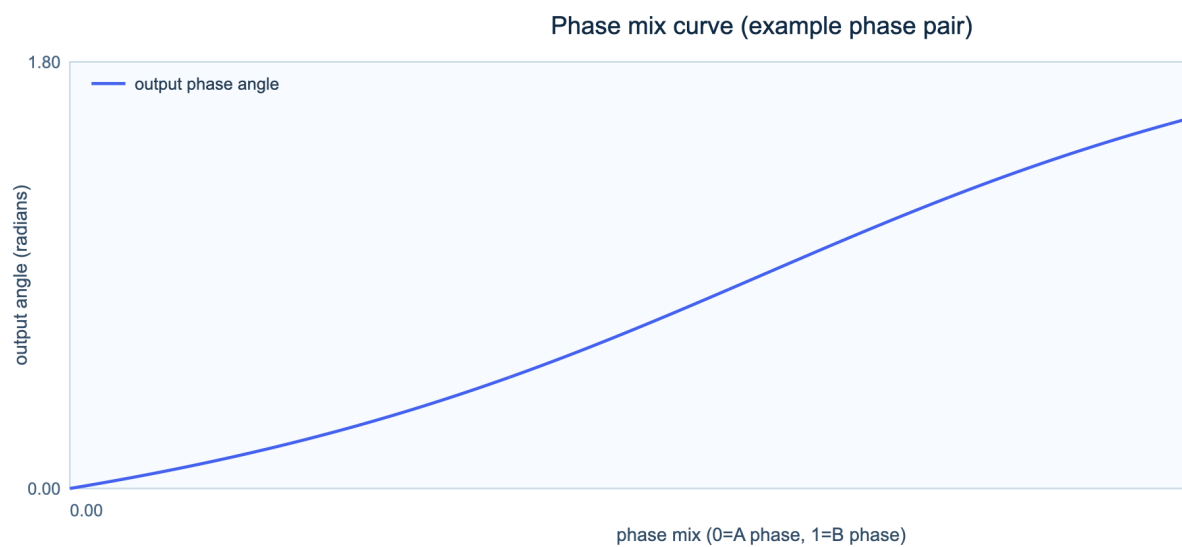


Figure H.7: Phase-mix angle response.

H.2 Automation and interpolation

The interpolation group shows how pvx connects control points across render time. An interpolation rule is not merely a visual convenience. Its slope and curvature determine how quickly an audible parameter moves and whether the motion has discontinuous derivatives.

$$u_{\text{linear}}(t) = (1 - \lambda)u_i + \lambda u_{i+1}$$

where u_i and u_{i+1} are adjacent control values, t lies between their timestamps, and λ is normalized segment position in $[0, 1]$.

$$u_{\text{smooth}}(t) = (1 - h(\lambda))u_i + h(\lambda)u_{i+1}, \quad h(\lambda) = 3\lambda^2 - 2\lambda^3$$

where $h(\lambda)$ is the smoothstep easing function, λ is normalized segment position, and u_i, u_{i+1} are adjacent control values.

H.2.1 sample and hold

The following graph shows sample and hold interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

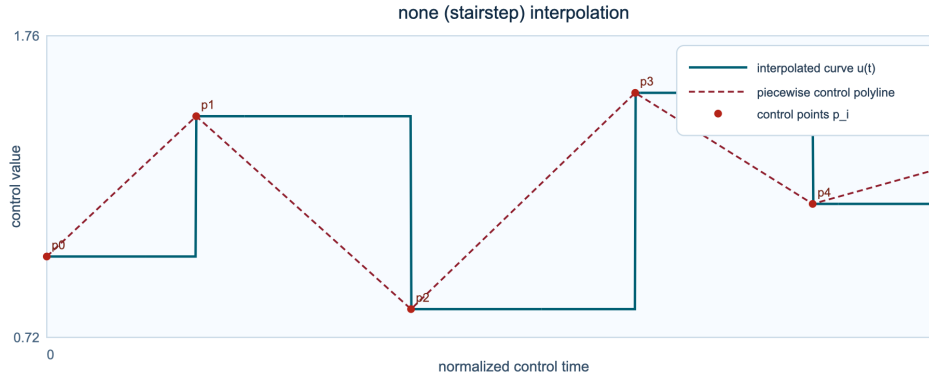


Figure H.8: Sample and hold interpolation.

H.2.2 nearest neighbour

The following graph shows nearest neighbour interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

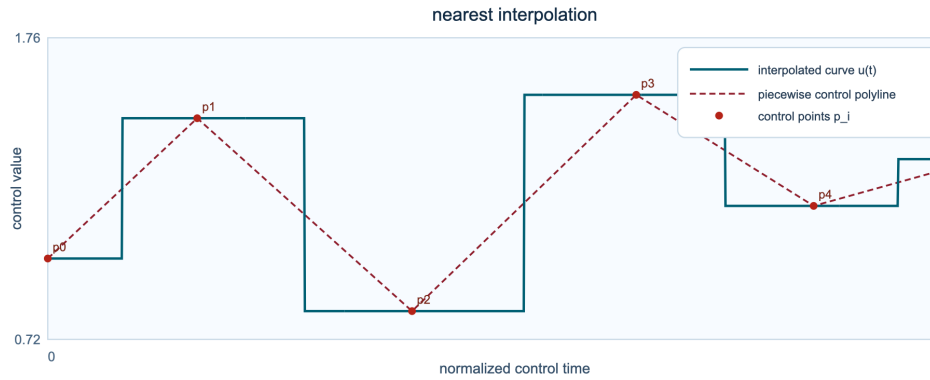


Figure H.9: Nearest neighbour interpolation.

H.2.3 linear

The following graph shows linear interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

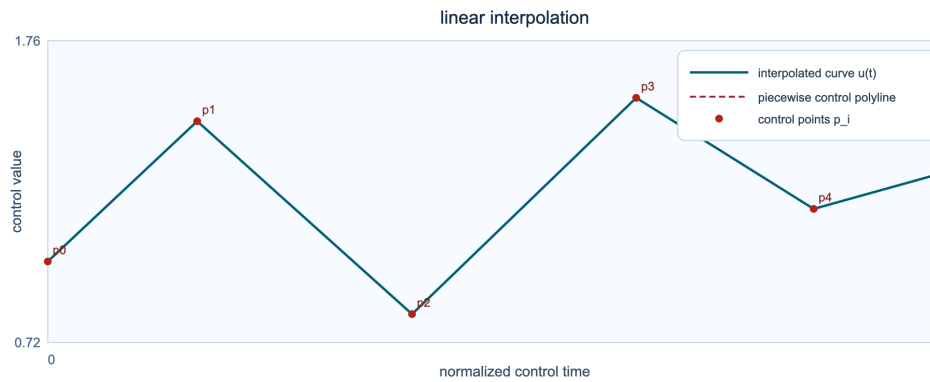


Figure H.10: Linear interpolation.

H.2.4 cubic

The following graph shows cubic interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

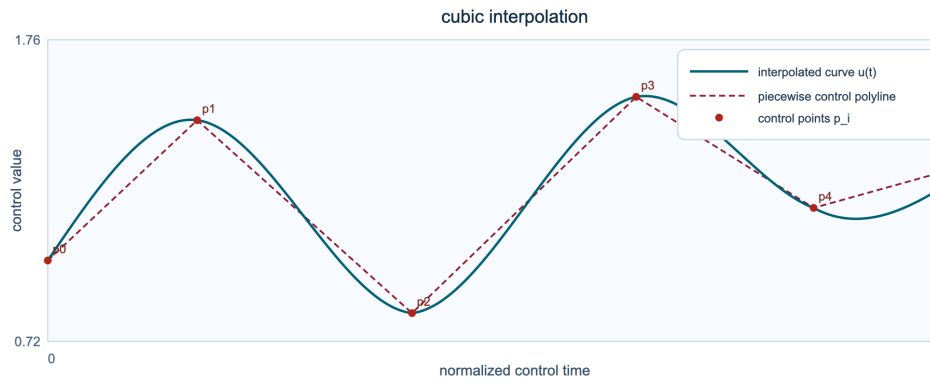


Figure H.11: Cubic interpolation.

H.2.5 exponential

The following graph shows exponential interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

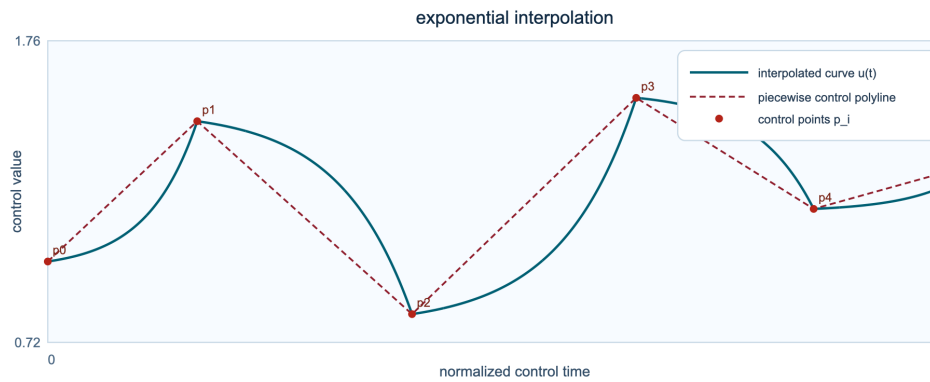


Figure H.12: Exponential interpolation.

H.2.6 smoothstep S-curve

The following graph shows smoothstep S-curve interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

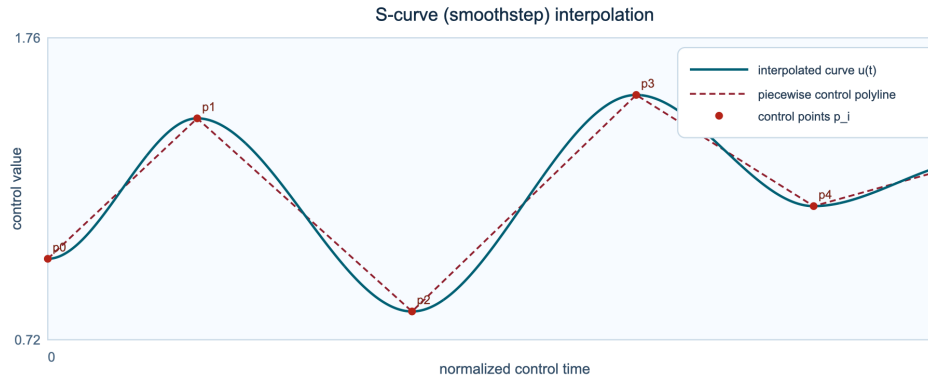


Figure H.13: Smoothstep s-curve interpolation.

H.2.7 smotherstep

The following graph shows smotherstep interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

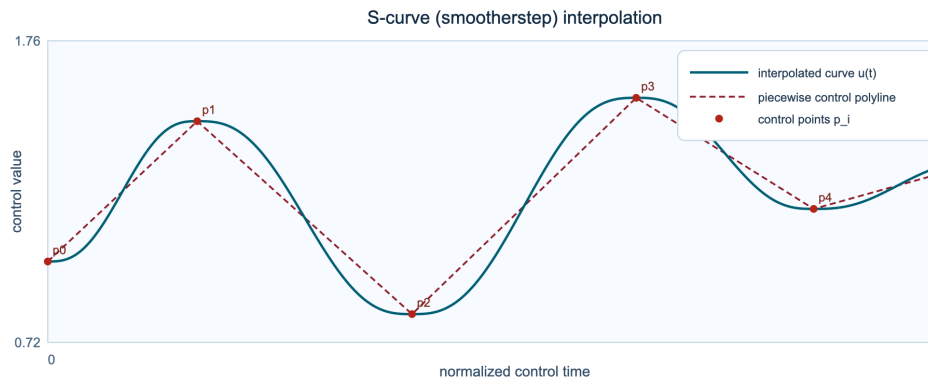


Figure H.14: Smotherstep interpolation.

H.2.8 polynomial order 1

The following graph shows polynomial order 1 interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

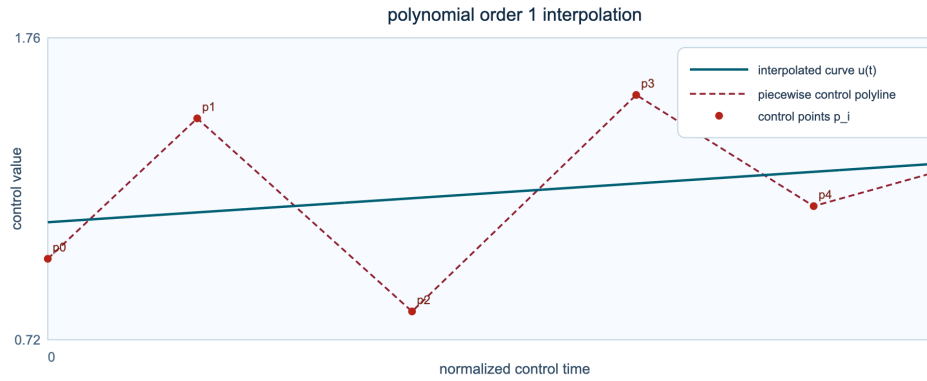


Figure H.15: Polynomial order 1 interpolation.

H.2.9 polynomial order 2

The following graph shows polynomial order 2 interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

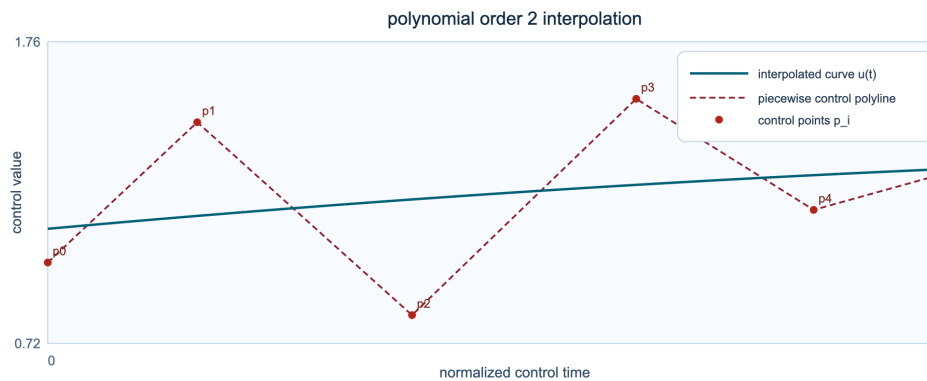


Figure H.16: Polynomial order 2 interpolation.

H.2.10 polynomial order 3

The following graph shows polynomial order 3 interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

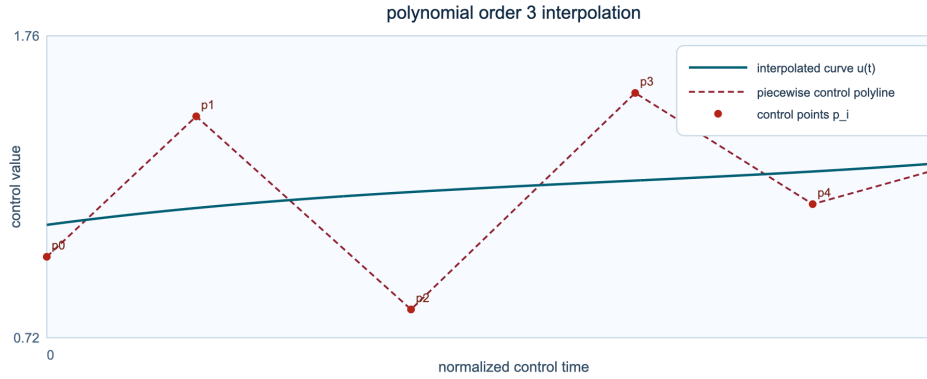


Figure H.17: Polynomial order 3 interpolation.

H.2.11 polynomial order 5

The following graph shows polynomial order 5 interpolation between a fixed set of control points. The comparison is most useful when the curve near each point and the continuity of its slope are examined together.

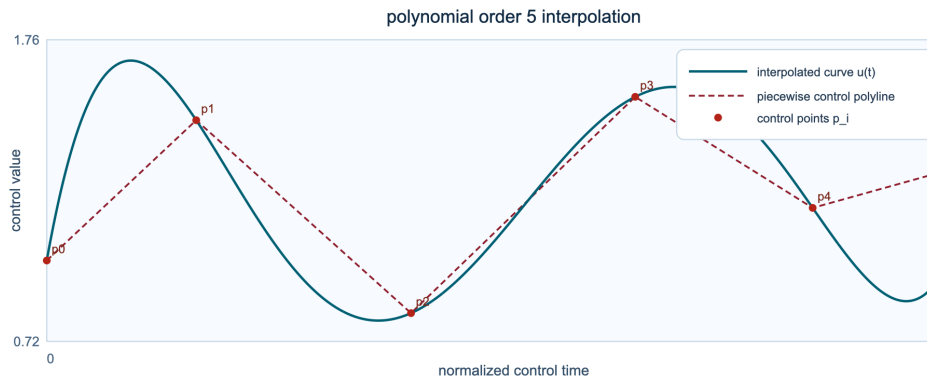


Figure H.18: Polynomial order 5 interpolation.

Index Registration Notes

The following invisible index registrations connect the alphabetical index to the generated CLI inventory.

Glossary

Aliasing. The appearance of false frequencies when a signal contains energy above the Nyquist frequency or when spectral modification moves energy beyond the representable band. Anti-alias filtering and an adequate sample rate reduce it.

Amplitude. A measure of signal strength. In a waveform it describes sample displacement; in a spectrum it describes the strength of a bin or partial. Amplitude may be expressed linearly or in decibels.

Analysis frame. A short, usually overlapping segment selected from the input signal for windowing and transformation. Its length limits the balance between temporal detail and frequency discrimination.

Analysis hop. The number of input samples between the starts of consecutive analysis frames. A smaller hop produces greater overlap and finer control sampling at the cost of more computation.

Analysis-synthesis system. A process that decomposes sound into a representation, modifies or stores that representation, and reconstructs audio. A phase vocoder uses short-time complex spectra as its central representation.

Artifact. An audible feature introduced by processing rather than present in the source. Typical phase-vocoder artifacts include attack smear, pre-echo, phasiness, metallic decay, unstable pitch, and stereo-image drift.

Attack. The beginning of a sound event, often marked by a rapid rise in energy or spectral flux. Attacks require special treatment because the stationary-sinusoid assumption is weakest there.

Automation curve. A time-indexed sequence controlling a parameter such as stretch, pitch, mix, filter gain, or room size. Interpolation determines the values between authored control points.

Bandlimit. To restrict a signal to frequencies below a chosen upper boundary. Bandlimiting before down-sampling or large upward spectral shifts helps prevent aliasing.

Bandwidth. The frequency range occupied or passed by a signal, filter, spectral peak, or analysis band. Bandwidth can be stated in hertz, bins, octaves, or perceptual units.

Bin. One discrete frequency sample of a Fourier spectrum. Bin index is not always identical to sinusoidal frequency because a partial may lie between bin centers and spread into neighboring bins.

Bin-center frequency. The nominal frequency represented by a DFT bin. For bin index k , sample rate F_s , and transform length N , the center frequency is k times F_s divided by N .

Bit depth. The number of bits used to represent each encoded sample in a fixed-point audio format. Greater bit depth lowers the quantization-noise floor but does not repair clipping or processing artifacts.

Block processing. Audio computation performed on groups of samples rather than one sample at a time. Spectral algorithms are naturally block based because every transform consumes a complete frame.

Broadband. Containing energy across a wide frequency range. Impulses, consonants, cymbals, and noise are broadband sources and often require shorter windows or transient-aware processing.

Buffer. A region of memory holding samples, frames, or encoded bytes while they are read, processed, or written. Buffer size affects throughput, latency, and streaming behavior.

Cepstrum. A representation obtained by transforming a log-magnitude spectrum. Slow cepstral components describe the spectral envelope, while faster components can describe harmonic spacing and pitch-related structure.

Channel. One component of a multichannel audio signal, such as left or right in stereo. Independent processing can disturb interchannel phase relationships, so reference locking may be preferable.

Checkpoint. A saved rendering state that permits a long process to resume without starting from the beginning. A useful checkpoint records timing, phase state, automation position, configuration, and output progress.

Chroma. A twelve-class pitch representation that folds pitches separated by octaves into the same class. Chroma is useful for harmony analysis but discards register and much timbral detail.

Clipping. Nonlinear truncation when signal magnitude exceeds an allowed range. Hard clipping creates strong high-frequency products; soft clipping rounds the transition but still changes timbre.

Coherent gain. The average value of a window, interpreted as the gain applied to a sinusoid exactly centered on a transform bin. It is used when comparing or calibrating windowed spectral amplitudes.

COLA. Constant overlap-add. A window and hop combination satisfies COLA when shifted window copies sum to a constant, permitting level-stable reconstruction under the corresponding overlap-add scheme.

Complex number. A value with real and imaginary components. Fourier bins are complex because they jointly encode magnitude and phase, or equivalently the cosine and sine contributions at a frequency.

Complex spectrum. The set of complex Fourier coefficients for a frame. Retaining both magnitude and phase is necessary for direct inverse transformation and coherent time evolution.

Conjugate symmetry. The mirrored relationship in the DFT of a real signal. Positive- and negative-frequency bins are complex conjugates, allowing a real-input transform to store only the nonnegative half.

Convolution. An operation that combines a signal with an impulse response. In audio it models filtering and reverberation; in the frequency domain it corresponds to multiplication of spectra.

Cross-synthesis. A transformation that combines properties of two sounds, commonly the magnitude or envelope of one with the phase, excitation, or timing of another. The result can interpolate between recognizable identities.

Crossfade. A transition formed by decreasing one signal while increasing another. Equal-power shapes are often used when the sources are weakly correlated; complementary linear fades can suit matched signals.

dBFS. Decibels relative to digital full scale. Zero dBFS is the maximum representable peak in a normalized digital system; ordinary signals use negative values below that ceiling.

Decay. The reduction of energy after an attack or excitation. Preserving the texture and modulation of a decay is a different problem from preserving its initial transient.

Deterministic render. A process that produces identical output for identical input, options, software version, and seed. Determinism is essential for regression tests, resumable jobs, and reproducible datasets.

DFT. Discrete Fourier transform. It maps a finite sequence of N samples to N complex frequency coefficients. The FFT is an efficient family of algorithms for computing it.

Dry signal. The unprocessed source in a wet-dry mixture. A dry path can restore attacks, intelligibility, or localization that would otherwise be softened by an extreme spectral effect.

Dynamic range. The span between low and high signal levels. It may describe a recording, a processing stage, or a format, and should not be confused with average loudness.

Energy normalization. Gain compensation intended to keep a chosen energy measure consistent after transformation. Peak, RMS, and perceptual-loudness normalization solve different problems and can produce different balances.

Envelope. A slowly varying contour describing amplitude, spectral shape, or another feature. Envelopes suppress fine oscillation so larger gestures can be measured or transferred.

Envelope follower. A detector that tracks changing signal level using attack and release smoothing. Its time constants determine how strongly it reacts to transients versus sustained energy.

Equalization. Frequency-dependent gain adjustment. In a spectral processor, equalization may be fixed, automated, measured from a reference, or applied separately to time-frequency regions.

ERB. Equivalent rectangular bandwidth. A psychoacoustic scale approximating the frequency selectivity of human hearing. ERB-spaced bands are often more perceptually meaningful than equal-width hertz bands.

Exponential interpolation. Interpolation whose values change by a constant ratio rather than a constant difference. It suits positive quantities such as frequency, pitch ratio, decay time, and some gain controls.

FFT. Fast Fourier transform. A computational method that evaluates the DFT efficiently. FFT size affects frequency sampling, frame duration, memory use, and computational cost.

FFT size. The transform length N . It may equal the window length or exceed it through zero padding. A larger size samples the spectrum more densely but does not automatically add true resolution.

File subtype. The encoded sample representation inside an audio container, such as PCM 16-bit, PCM 24-bit, or floating point. Container extension alone may not identify the subtype.

Filter. A process that changes a signal according to frequency. Filters may be described by magnitude, phase, impulse response, poles and zeros, or a direct time-frequency mask.

Floor. A lower limit imposed on a value. Magnitude floors prevent logarithms and divisions from becoming singular, but a floor set too high can raise noise or flatten weak detail.

Formant. A broad resonance region that shapes the identity of a voice or instrument. Formants are related to the spectral envelope rather than to the fundamental pitch alone.

Formant preservation. A pitch-shifting strategy that holds or separately transforms the spectral envelope while moving harmonic content. It helps a shifted voice retain vowel identity and apparent vocal size.

Frame. A finite slice of samples or spectral data processed as one unit. Frames overlap so that local analysis can be joined into a continuous output.

Frame rate. The number of analysis or control frames per second. It is the sample rate divided by hop size for a uniformly sampled frame sequence.

Frequency resolution. The ability to distinguish nearby frequencies. It depends primarily on effective window duration and shape, not merely on the number of zero-padded FFT samples.

Fundamental frequency. The repetition frequency associated with perceived pitch in a periodic or nearly periodic sound. It is often written f_0 and may be absent as a literal spectral peak.

Gain. A multiplier applied to amplitude. Linear gain 1 leaves level unchanged, while decibel gain expresses the same scaling logarithmically.

Griffin-Lim algorithm. An iterative method for finding a signal whose STFT magnitude approximates a target magnitude when phase is unavailable or inconsistent. It is reconstruction, not ordinary phase propagation.

Group delay. The frequency-dependent delay derived from phase slope. Irregular group delay can smear transient structure even when a magnitude response appears unchanged.

Harmonic. A sinusoidal component near an integer multiple of a fundamental frequency. Real instruments depart from exact harmonicity through stiffness, modulation, noise, and resonant filtering.

Harmonic-percussive separation. A decomposition that distinguishes spectrally persistent components from temporally impulsive components. Separate paths can use long windows for harmonic material and short windows for attacks.

Hertz. Cycles per second, the physical unit of frequency. Hertz spacing is linear, while musical pitch perception is approximately logarithmic.

Hop ratio. A hop expressed relative to window or FFT length. It provides a resolution-independent way to describe overlap, such as one quarter of a window.

Hop size. The sample distance between adjacent frame starts. Analysis and synthesis hops may differ; their ratio is central to basic phase-vocoder time scaling.

Identity phase locking. A method associated with Laroche and Dolson that propagates the phase of prominent peaks and preserves nearby bins' relative phase relationships. It improves vertical coherence for tonal material.

Instantaneous frequency. A local frequency estimate derived from phase change over time after removing the phase advance expected at a bin center. It refines partial trajectories beyond discrete bin locations.

Interpolation. The estimation of values between known control points or samples. Linear, cubic, exponential, and eased methods differ in continuity, overshoot, and perceived motion.

Inverse FFT. The transform that converts complex frequency coefficients back into a time-domain frame. A complete synthesis also requires windowing, overlap, and normalization.

Kaiser beta. The shape parameter of a Kaiser window. Increasing beta generally lowers sidelobes while widening the main lobe, allowing a continuous tradeoff between leakage and frequency separation.

Keyframe. An authored control point assigning a parameter value at a specified time. The interpolation rule and time coordinate system determine the path between keyframes.

Latency. The delay between input and corresponding output. Frame accumulation, lookahead, buffering, transforms, and device I/O all contribute to end-to-end latency.

Leakage. The spreading of a component's energy into neighboring spectral bins because a finite frame does not contain an integer number of cycles or uses a nonideal window.

LFO. Low-frequency oscillator. A slowly varying periodic control source used to modulate pitch, gain, filtering, position, or other parameters.

Linear phase. A phase response proportional to frequency, corresponding to a constant delay. Symmetric finite impulse response filters can provide linear phase but may introduce pre-ringing.

Loudness. The perceptual strength of sound, influenced by level, spectrum, duration, and context. Loudness normalization is more perceptually oriented than peak matching.

Magnitude. The nonnegative length of a complex spectral coefficient. Magnitude describes component strength but omits the phase information needed for exact waveform reconstruction.

Magnitude spectrum. The set of magnitudes across Fourier bins. It reveals spectral distribution while discarding the time alignment encoded by phase.

Main lobe. The central lobe of a window's frequency response. Its width controls how closely spaced sinusoids can be distinguished in a windowed analysis.

Mask. A time-frequency gain field applied to spectral data. Masks can isolate, attenuate, blend, denoise, or route components and may be binary or continuously valued.

Masking. A perceptual interaction in which one sound reduces the audibility of another nearby in frequency or time. It also informally describes multiplication by a spectral mask.

Metadata. Information associated with audio but not itself waveform samples, such as sample rate, channel layout, markers, tags, provenance, processing settings, and checksums.

Mid-side. A stereo representation using sum and difference channels. Mid carries shared center information; side carries channel difference and controls apparent width.

Mono. Audio containing one channel. Mono processing avoids interchannel coherence problems but does not remove phase-coherence issues within the spectrum.

Multichannel. Audio containing more than one channel, including stereo, surround, and microphone arrays. Transform consistency across channels is important for stable localization.

Multiresolution analysis. An approach using different time-frequency resolutions for different bands or signal conditions. Low frequencies often benefit from longer windows while transients benefit from shorter ones.

Noise floor. The background level below which desired details become masked or numerically unreliable. Processing can raise, color, modulate, or expose the noise floor.

Nonstationary. Changing in statistical or spectral character over the observation interval. Speech, music, and transients are nonstationary, which is why frame duration must remain locally appropriate.

Normalized frequency. Frequency expressed relative to sample rate, Nyquist, bin count, or another reference. The convention must be stated because the same numerical value can otherwise be ambiguous.

Nyquist frequency. Half the sample rate and the highest frequency representable without aliasing in a conventionally sampled real signal. Energy crossing it must be filtered or otherwise managed.

Onset. The perceptual beginning of a note or event. Onset detectors often combine energy change, spectral flux, phase behavior, and frequency-band evidence.

Overlap. The shared samples between adjacent frames. Greater overlap increases frame rate and can improve reconstruction or automation smoothness, while increasing computation.

Overlap-add. A synthesis procedure that places reconstructed frames at successive hop positions and sums their overlaps. Window and normalization choices determine whether level remains stable.

Oversampling. Processing at a sample rate higher than the final delivery rate. Oversampling creates transition space for nonlinear or frequency-shifting operations before filtered downsampling.

Partial. One locally coherent sinusoidal component of a sound. Partial can be harmonic, inharmonic, stable, gliding, or amplitude modulated.

Peak bin. A bin whose magnitude is locally greater than its neighbors. Spectral-peak interpolation estimates a more accurate frequency and amplitude between discrete bins.

Peak locking. A family of methods that groups bins around spectral peaks and coordinates their phase evolution. It reduces diffuse, chorus-like coloration in tonal signals.

Phase. The angular position of a periodic component. In a complex spectrum, phase determines the alignment of each sinusoidal contribution within and across frames.

Phase coherence. Consistency among phase relationships. Horizontal coherence concerns evolution across time; vertical coherence concerns relationships among bins within a frame; spatial coherence concerns channels.

Phase increment. The phase advance between frames. Comparing the measured increment with the expected bin-center advance yields an instantaneous-frequency estimate.

Phase propagation. The process of advancing synthesis phase from one frame to the next according to estimated frequencies and the synthesis hop.

Phase reset. The replacement or reinitialization of propagated phase, often near an onset. It can sharpen attacks but may cause discontinuity if applied too broadly.

Phase unwrapping. The selection of equivalent phase values differing by integer multiples of two π so that a phase trajectory changes continuously enough to interpret.

Phase vocoder. An STFT analysis-synthesis method that estimates and propagates spectral phase so duration, pitch, and spectral structure can be modified with greater continuity than frame repetition alone.

Phasiness. A diffuse, hollow, chorus-like artifact commonly associated with weak vertical phase coherence. It is especially noticeable on voices, solo instruments, and reverberant tonal material.

Pitch ratio. The multiplicative relationship between output and input pitch. One octave upward is ratio 2, one octave downward is ratio one half, and twelve-tone semitones use powers of two.

Pitch shifting. Changing perceived pitch while attempting to preserve duration. Phase-vocoder methods often combine spectral modification or time scaling with resampling and optional formant correction.

Pre-echo. Energy appearing before a transient because a frame or zero-phase process spreads future event energy backward in time. Long symmetric windows can make it conspicuous.

PVXRF. The pvx reusable response artifact format. It stores a measured or authored response together with schema, provenance, and interpretation needed for later spectral processing.

Quantization. Mapping a continuous or high-precision value to one of finitely many levels. Audio quantization produces an error whose audibility depends on bit depth, signal level, and dither.

Real-time factor. Processing time divided by audio duration. A factor below one is faster than real time; a factor above one takes longer than the rendered program.

Reconstruction. The conversion of an analysis representation back into audio. Perfect reconstruction is a mathematical property under stated conditions, not a guarantee that a modified representation sounds natural.

Reference locking. Using one channel, stem, or feature trajectory as the timing or phase reference for related signals. It can preserve stereo location and multichannel coherence.

Resampling. Changing sample timing through interpolation and filtering. Resampling changes duration and pitch together unless paired with a separate time-scale operation.

Resonance. A frequency region that responds more strongly or decays more slowly because of an acoustic, mechanical, or filter mode. Resonances shape formants, rooms, instruments, and effects.

RMS. Root mean square, an energy-related level measure computed from squared samples. It is more representative of sustained power than a peak measurement but less perceptual than integrated loudness.

Room size. An acoustic or model parameter influencing reflection density, modal spacing, and decay behavior. Automating room size may require smoothing to avoid pitch-like discontinuities in the reverberant field.

Sample. One discrete amplitude measurement in a digital waveform. A sample has no duration by itself; its time spacing is set by the sample rate.

Sample rate. The number of samples per second. It determines the Nyquist frequency and converts sample counts, frame lengths, and hops into physical time.

Scalloping loss. The reduction in measured magnitude when a sinusoid falls between bin centers rather than on one. Window shape determines the worst-case loss.

Side lobe. A secondary lobe in a window's frequency response. High sidelobes allow strong components to leak into distant bins; suppressing them generally widens the main lobe.

Sinusoid. A periodic waveform defined by amplitude, frequency, and phase. Phase-vocoder analysis treats many locally stable sounds as sums of slowly changing sinusoids plus residual components.

Smoothing. Reduction of rapid variation across time, frequency, or both. Smoothing can stabilize controls and masks but can also erase attacks, modulation, or narrow spectral detail.

Spectral centroid. The magnitude-weighted center of a spectrum. It often correlates with brightness but can move because of filtering, noise, orchestration, or pitch.

Spectral envelope. A smooth curve following broad spectral shape while ignoring individual harmonic peaks. It describes resonant coloration and is central to formant-aware transformation.

Spectral flux. A frame-to-frame measure of spectral change, often computed from positive magnitude differences. It is widely used for onset and transient detection.

Spectral freeze. A process that holds or evolves spectral data from a selected time while synthesis continues. Static magnitudes with coherent moving phase can create sustained, pitched textures.

Spectral leakage. Energy from one component appearing in adjacent or distant bins because of finite framing and window response. Leakage is controlled rather than completely eliminated.

Spectrogram. A visual time-frequency representation built from successive spectra. Color or intensity usually shows magnitude, while phase is commonly omitted.

Stereo. Two-channel audio intended to create a spatial image. Independent spectral modification can disturb center stability, width, and localization cues.

STFT. Short-time Fourier transform. It applies a windowed Fourier transform at successive times, producing a complex matrix indexed by frame and frequency bin.

Streaming. Processing incrementally without loading the entire source or output into memory. Streaming requires explicit state at chunk boundaries so results match uninterrupted processing.

Synthesis frame. A time-domain frame reconstructed from modified spectral data before windowing and overlap-add. Its placement is controlled by the synthesis hop.

Synthesis hop. The sample distance between successive output frames. Relative to the analysis hop, it controls the basic duration change in a phase-vocoder synthesis.

Temporal resolution. The ability to locate changes in time. Shorter windows improve temporal localization but provide less separation between nearby low frequencies.

Time map. A function relating output time to input time or the reverse. Nonlinear maps permit *accelerando*, *ritardando*, holds, jumps, and local alignment.

Time-frequency tradeoff. The constraint that analysis cannot make time and frequency localization arbitrarily precise at once. Window length and shape choose a compromise appropriate to the material.

Time-scale modification. Changing duration while attempting to preserve pitch. Success depends on phase continuity, transient treatment, source complexity, and the requested scale factor.

Transient. A short, rapidly changing event such as a drum hit, consonant, bow change, or pick attack. Transients violate assumptions of slow frame-to-frame spectral evolution.

Transient preservation. A strategy that detects attacks and protects their timing or waveform while transforming surrounding sustained material. Methods include phase reset, dry-path mixing, segmentation, and multiresolution processing.

True envelope. A high-resolution spectral-envelope estimate developed for robust separation of broad resonant shape from harmonic sampling. It is useful in high-quality formant-preserving transformations.

Unvoiced. Describing speech or sound without stable periodic vocal-fold excitation, such as many fricatives and breaths. Unvoiced regions often benefit from noise-aware rather than harmonic phase treatment.

Unwrap. To remove artificial two-pi discontinuities from wrapped phase values. Unwrapping selects a locally consistent trajectory but does not by itself guarantee correct component tracking.

Vibrato. A periodic or near-periodic modulation of pitch, often accompanied by amplitude and timbral modulation. Excessive smoothing can flatten it, while poor tracking can exaggerate or destabilize it.

Vocoder. A broad term for systems that encode or transform voice through an intermediate representation. Channel vocoders, phase vocoders, LPC vocoders, and neural vocoders use substantially different models.

Voiced. Describing sound with a stable or approximately periodic excitation and measurable fundamental frequency. Voiced frames support harmonic tracking and formant-preserving pitch operations.

Wet signal. The processed component of a wet-dry mixture. Its level should be interpreted relative to the dry path, correlation, latency, and any normalization applied inside the effect.

Wet-dry mix. A control blending processed and unprocessed signals. Time-varying mix can introduce comb filtering if paths are misaligned or strongly correlated with different phase responses.

Window. A weighting curve applied to each frame before transformation. It controls endpoint behavior, leakage, main-lobe width, sidelobes, coherent gain, and overlap-add behavior.

Window length. The number of samples receiving nonzero or defined window weights. It determines frame duration and is a principal control over time-frequency resolution.

WOLA. Weighted overlap-add. An analysis-synthesis framework in which analysis and synthesis windows jointly determine reconstruction. Their product and hop must satisfy the relevant overlap condition.

Zero padding. Appending zeros before a transform to sample the spectrum more densely. Zero padding improves visual and interpolation granularity but does not provide the resolving power of a longer observed signal.

Bibliography

This bibliography gathers one hundred twenty-five books and papers that support the theory, engineering practice, listening methods, and historical discussion in this guide. The papers emphasize phase-vocoder development, short-time analysis and reconstruction, pitch and formant processing, transient handling, time-scale modification, source separation, and objective evaluation. The books supply broader foundations in digital signal processing, speech, psychoacoustics, computer music, and sound engineering.

Papers and articles

The one hundred entries in this section are alphabetized by the first named author. DOI links are preferred when the repository record supplies one; otherwise, a stable publisher or catalog record is used where available.

- S. Ahmadi; H. Spanias. “Cepstrum-Based Pitch Detection Using FFT.” *Conference* (1999).
- Natsuki Akaishi; Kohei Yatabe; Yasuhiro Oikawa. “Improving Phase-Vocoder-Based Time Stretching by Time-Directional Spectrogram Squeezing.” *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing* (2023). [DOI](#).
- J. Engel et al.. “DDSP: Differentiable Digital Signal Processing.” *ICLR* (2020). [catalog record](#).
- S. Wisdom et al.. “Differentiable Consistency Constraints for Improved Deep Speech Enhancement.” *ICASSP* (2021).
- J. B. Allen; L. R. Rabiner. “A Unified Approach to Short-Time Fourier Analysis and Synthesis.” *Proceedings of the IEEE* (1977). [DOI](#).
- F. Auger; P. Flandrin. “Improving the Readability of Time-Frequency and Time-Scale Representations by the Reassignment Method.” *IEEE Trans. SP* (1995). [DOI](#).
- A. J. Bell; T. J. Sejnowski. “An Information-Maximization Approach to Blind Separation and Blind Deconvolution.” *Neural Computation* (1995).
- Michaël Betser; Patrice Collen; Gaël Richard; Bertrand David. “Estimation of Frequency for AM/FM Models Using the Phase Vocoder Framework.” *IEEE Transactions on Signal Processing* (2008). [DOI](#).
- P. Boersma. “Accurate Short-Term Analysis of the Fundamental Frequency and the Harmonics-to-Noise Ratio of a Sampled Sound.” *IFA Proceedings* (1993).
- S. F. Boll. “Suppression of Acoustic Noise in Speech Using Spectral Subtraction.” *IEEE Trans. ASSP* (1979). [DOI](#).
- Ian Bowler; Alan Purvis; Peter Manning; Nick Bailey. “New Techniques for a Real-time Phase Vocoder.” *The Journal of the Abraham Lincoln Association* (1990). [catalog record](#).
- R. Bristow-Johnson; M. Bogdanowicz. “Phase Vocoder Done Right.” *AES Preprint* (1989).
- Robert Bristow-Johnson; K. Bogdanowicz. “Intraframe time-scaling of nonstationary sinusoids within the phase vocoder.” *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics* (2002). [DOI](#).
- J. C. Brown. “Calculation of a Constant Q Spectral Transform.” *JASA* (1991). [DOI](#).
- J. C. Brown; M. S. Puckette. “An Efficient Algorithm for the Calculation of a Constant Q Transform.” *JASA* (1992). [DOI](#).
- A. Camacho; J. G. Harris. “A Sawtooth Waveform Inspired Pitch Estimator for Speech and Music (SWIPE).” *JASA* (2008). [DOI](#).

- E. J. Candes; X. Li; Y. Ma; J. Wright. “Robust Principal Component Analysis?” *Journal of ACM* (2011).
- J. Capon. “High-Resolution Frequency-Wavenumber Spectrum Analysis.” *Proceedings of the IEEE* (1969). DOI.
- A. de Cheveigné; H. Kawahara. “YIN, a Fundamental Frequency Estimator for Speech and Music.” *JASA* (2002). DOI.
- I. Cohen; B. Berdugo. “Speech Enhancement for Non-Stationary Noise Environments.” *Signal Processing* (2001).
- L. Cohen. “Time-Frequency Distributions - A Review.” *Proceedings of the IEEE* (1989).
- R. R. Coifman; M. V. Wickerhauser. “Entropy-Based Algorithms for Best Basis Selection.” *IEEE Trans. IT* (1992). DOI.
- P. Comon. “Independent Component Analysis, A New Concept?” *Signal Processing* (1994). DOI.
- I. Daubechies; J. Lu; H.-T. Wu. “Synchrosqueezed Wavelet Transforms.” *Applied and Computational Harmonic Analysis* (2011). DOI.
- S. Disch. “A New Phase Vocoder Technique for Time-Scale Modification of Audio Signals.” *DAFx* (1998).
- M. Dolson. “The Phase Vocoder: A Tutorial.” *Computer Music Journal* (1986). [catalog record](#).
- Mark Dolson. “A Tracking Phase Vocoder and its Use in the Analysis of Ensemble Sounds.” *Dissertation* (1983). DOI.
- D. L. Donoho. “De-Noising by Soft-Thresholding.” *IEEE Trans. Information Theory* (1995).
- K. Dressler. “Sinusoidal Extraction using Frequency-Domain Matching Pursuit.” *DAFx* (2012).
- J. Driedger; M. Müller. “A Review of Time-Scale Modification of Music Signals.” *Applied Sciences* (2016). DOI.
- C. Duxbury; M. Davies; M. Sandler. “Improved Time-Scaling of Musical Audio Using Phase Locking at Transients.” *AES Convention* (2002).
- Daniel P. W. Ellis; Ron J. Weiss. “Model-Based Monaural Source Separation Using a Vector-Quantized Phase-Vocoder Representation.” *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing* (2006). DOI.
- Y. Ephraim; D. Malah. “Speech Enhancement Using a Minimum Mean-Square Error Short-Time Spectral Amplitude Estimator.” *IEEE Trans. ASSP* (1984). DOI.
- Y. Ephraim; D. Malah. “Speech Enhancement Using a Minimum Mean-Square Error Log-Spectral Amplitude Estimator.” *IEEE Trans. ASSP* (1985). DOI.
- S. Essid; G. Richard. “Musical signal analysis with reassignment methods.” *Book chapter* (2012).
- Gianpaolo Evangelista. “Modified phase vocoder scheme for dynamic frequency warping.” *Proceedings of the IEEE International Symposium on Communications, Control and Signal Processing* (2008). DOI.
- Gianpaolo Evangelista; Monika Dörfler; Ewa Matusiak. “Phase Vocoder With Arbitrary Frequency Band Selection.” *Zenodo (CERN European Organization for Nuclear Research)* (2012). DOI.
- Gianpaolo Evangelista; Monika Dörfler; Ewa Matusiak. “Arbitrary Phase Vocoder by means of Warping.” *SHILAP Revista de lepidopterología* (2013). DOI.
- C. Fevotte; N. Bertin; J.-L. Durrieu. “Nonnegative Matrix Factorization with the Itakura-Saito Divergence.” *Neural Computation* (2009).
- D. Fitzgerald. “Harmonic/Percussive Separation Using Median Filtering.” *DAFx* (2010). [catalog record](#).
- J. L. Flanagan; R. M. Golden. “Phase Vocoder.” *Bell System Technical Journal* (1966). DOI.
- P. Flandrin; F. Auger; E. Chassande-Mottin. “Time-Frequency Reassignment: From Principles to Algorithms.” *Applications in Time-Frequency Signal Processing* (2002).
- John Garas; P.C.W. Sommen. “Time/pitch scaling using the constant-Q phase vocoder.” *Proceedings of ProRISC/IEEE CSSP98* (1998). [catalog record](#).

- D. Giannoulis; M. Massberg; J. Reiss. “Digital Dynamic Range Compressor Design.” *JAES* (2012).
- D. W. Griffin; J. S. Lim. “Signal Estimation from Modified Short-Time Fourier Transform.” *IEEE Trans. ASSP* (1984). [DOI](#).
- Amalia de Götzen; Nicola Bernardini; Daniel Arfib. “TRADITIONAL (?) IMPLEMENTATIONS OF A PHASE-VOCODER: THE TRICKS OF THE TRADE.” *VBN Forskningsportal (Aalborg Universitet)* (2000). [catalog record](#).
- A. Hyvarinen; E. Oja. “Independent Component Analysis: Algorithms and Applications.” *Neural Networks* (2000).
- Nicolas Juillerat; Béat Hirsbrunner. “Audio time stretching with an adaptive multiresolution phase vocoder.” *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing* (2017). [DOI](#).
- Thorsten Karrer; Eric Lee; Jan Borchers. “PhaVoRiT: A Phase Vocoder for Real-Time Interactive Time-Stretching.” *RWTH Publications (RWTH Aachen)* (2006). [catalog record](#).
- H. Kawahara; A. de Cheveigne. “STRAIGHT, Exploitation of the Other Aspects of Vocal Source Information.” *ICASSP* (2001).
- J. W. Kim; J. Salamon; P. Li; J. P. Bello. “CREPE: A Convolutional Representation for Pitch Estimation.” *ICASSP* (2018). [catalog record](#).
- A. Klapuri. “Multiple Fundamental Frequency Estimation by Harmonicity and Spectral Smoothness.” *IEEE TASP* (2006).
- C. Knapp; G. Carter. “The Generalized Correlation Method for Estimation of Time Delay.” *IEEE Trans. ASSP* (1976). [DOI](#).
- J. Laroche; M. Dolson. “About this Phasiness Business.” *IEEE ASSP Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)* (1997). [catalog record](#).
- J. Laroche; M. Dolson. “Improved Phase Vocoder Time-Scale Modification of Audio.” *IEEE Trans. Speech and Audio Processing* (1999). [DOI](#).
- J. Laroche; M. Dolson. “New Phase-Vocoder Techniques for Pitch-Shifting, Harmonizing and Other Exotic Effects.” *WASPAA* (1999). [DOI](#).
- D. D. Lee; H. S. Seung. “Learning the Parts of Objects by Non-negative Matrix Factorization.” *Nature* (1999). [DOI](#).
- D. D. Lee; H. S. Seung. “Algorithms for Non-negative Matrix Factorization.” *NIPS* (2001). [DOI](#).
- Eric Lee; Thorsten Karrer; Jan Borchers. “An Analysis of Startup and Dynamic Latency in phase vocoder-Based Time-stretching Algorithms.” *RWTH Publications (RWTH Aachen)* (2007). [catalog record](#).
- S. Mallat. “A Theory for Multiresolution Signal Decomposition: The Wavelet Representation.” *IEEE Trans. PAMI* (1989). [DOI](#).
- S. Mallat; Z. Zhang. “Matching Pursuits with Time-Frequency Dictionaries.” *IEEE Trans. Signal Processing* (1993).
- Sylvain Marchand. “Improving Spectral Analysis Precision With an Enhanced Phase Vocoder Using Signal Derivatives.” *Proceedings of the International Computer Music Conference* (1998). [catalog record](#).
- R. Martin. “Noise Power Spectral Density Estimation Based on Optimal Smoothing and Minimum Statistics.” *IEEE Trans. Speech and Audio Processing* (2001). [DOI](#).
- M. Mauch; S. Dixon. “pYIN: A Fundamental Frequency Estimator Using Probabilistic Threshold Distributions.” *ICASSP* (2014). [DOI](#).
- R. J. McAulay; T. F. Quatieri. “Speech Analysis/Synthesis Based on a Sinusoidal Representation.” *IEEE Trans. ASSP* (1986).
- M. McLeod; G. Wyvill. “A Smarter Way to Find Pitch.” *ICMC* (2005).

- James A. Moorer. “The Use of the Phase Vocoder in Computer Music Applications.” *Journal of the Audio Engineering Society* (1978). [catalog record](#).
- E. Moulines; F. Charpentier. “Pitch-Synchronous Waveform Processing Techniques for Text-to-Speech Synthesis Using Diphones.” *Speech Communication* (1990). [DOI](#).
- E. Moulines; J. Laroche. “Non-parametric Techniques for Pitch-scale and Time-scale Modification of Speech.” *Speech Communication* (1995). [DOI](#).
- Frederik Nagel; Sascha Disch; Nikolaus Rettelbach. “A Phase Vocoder Driven Bandwidth Extension Method with Novel Transient Handling for Audio Codecs.” *Publikationsdatenbank der Fraunhofer-Gesellschaft (Fraunhofer-Gesellschaft)* (2009). [catalog record](#).
- T. Nakatani; M. Miyoshi; K. Kinoshita. “Speech Dereverberation Based on Variance-Normalized Delayed Linear Prediction.” *IEEE Trans. Audio, Speech, and Language Processing* (2010). [DOI](#).
- A. M. Noll. “Cepstrum Pitch Determination.” *JASA* (1967).
- Emil Solsbaek Ottosen; Monika Dorfler. “A Phase Vocoder Based on Nonstationary Gabor Frames.” *IEEE/ACM Transactions on Audio Speech and Language Processing* (2017). [DOI](#).
- K. K. Paliwal. “Importance of phase in speech enhancement.” *Speech Communication* (2011).
- M. R. Portnoff. “Implementation of the Digital Phase Vocoder Using the Fast Fourier Transform.” *IEEE Trans. ASSP* (1976).
- M. R. Portnoff. “Time-Scale Modification of Speech Based on Short-Time Fourier Analysis.” *IEEE Trans. ASSP* (1980).
- M. Puckette. “Phase-Locked Vocoder.” *ICMC* (1995).
- Miller Puckette; Judith C. Brown. “Accuracy of frequency estimates using the phase vocoder.” *IEEE Transactions on Speech and Audio Processing* (1998). [DOI](#).
- S. Pulkki. “Virtual Sound Source Positioning Using Vector Base Amplitude Panning.” *JAES* (1997).
- L. R. Rabiner; M. J. Cheng; A. E. Rosenberg; C. A. McGonegal. “A Comparative Performance Study of Several Pitch Detection Algorithms.” *IEEE Trans. ASSP* (1976).
- J. Reiss. “Under the Hood of a Dynamic Range Compressor.” *AES* (2011).
- A. W. Rix; J. G. Beerends; M. P. Hollier; A. P. Hekstra. “Perceptual Evaluation of Speech Quality (PESQ).” *ICASSP* (2001). [DOI](#).
- A. Roebel. “A Shape-Invariant Phase Vocoder for Speech Transformation.” *Interspeech* (2010).
- N. Roucos; A. Wilgus. “High Quality Time-Scale Modification for Speech.” *ICASSP* (1985).
- J. Le Roux; S. Wisdom; H. Erdogan; J. R. Hershey. “SDR half-baked or well done?” *ICASSP* (2019).
- M. Le Roux; E. Vincent. “Consistent Wiener Filtering for Audio Source Separation and Time-Frequency Processing.” *IEEE Signal Processing Letters* (2011).
- A. Röbel. “A New Approach to Transient Processing in the Phase Vocoder.” *DAFx* (2003). [catalog record](#).
- A. Röbel; X. Rodet. “Efficient Spectral Envelope Estimation and Its Application to Pitch Shifting and Envelope Preservation.” *DAFx* (2005). [catalog record](#).
- Axel Röbel; Xavier Rodet. “REAL TIME SIGNAL TRANSPOSITION WITH ENVELOPE PRESERVATION IN THE PHASE VOCODER.” *HAL (Le Centre pour la Communication Scientifique Directe)* (2005). [catalog record](#).
- P. Scalart; J. V. Filho. “Speech Enhancement Based on a Priori Signal to Noise Estimation.” *ICASSP* (1996).
- X. Serra; J. O. Smith. “Spectral Modeling Synthesis: A Sound Analysis/Synthesis System Based on a Deterministic Plus Stochastic Decomposition.” *Computer Music Journal* (1990). [catalog record](#).
- P. Smaragdis. “Convolutional Speech Bases and Their Application to Supervised Speech Separation.” *IEEE TASLP* (2007).

- F.-R. Stöter; S. Uhlich; A. Liutkus; Y. Mitsufuji. “Open-Unmix - A Reference Implementation for Music Source Separation.”’ *Journal of Open Source Software* (2019). [DOI](#).
- C. H. Taal; R. C. Hendriks; R. Heusdens; J. Jensen. “An Algorithm for Intelligibility Prediction of Time-Frequency Weighted Noisy Speech (STOI).”’ *IEEE TASLP* (2011). [DOI](#).
- D. Talkin. “A Robust Algorithm for Pitch Tracking (RAPT).”’ *In Speech Coding and Synthesis* (1995).
- J.-M. Valin. “A Hybrid DSP/Deep Learning Approach to Real-Time Full-Band Speech Enhancement (RN-Noise).”’ *MMSP / arXiv* (2018). [catalog record](#).
- G. Velasco; N. Holighaus; M. Dörfler; T. Grill. “Constructing an Invertible Constant-Q Transform with Nonstationary Gabor Frames.”’ *DAFx* (2011). [catalog record](#).
- W. Verhelst; M. Roelands. “An Overlap-Add Technique Based on Waveform Similarity (WSOLA) for High Quality Time-Scale Modification of Speech.”’ *ICASSP* (1993).
- T. Virtanen. “Monaural Sound Source Separation by Nonnegative Matrix Factorization with Temporal Continuity and Sparseness Criteria.”’ *IEEE Trans. Audio, Speech, and Language Processing* (2007).
- Trevor Wishart. “From Sound Morphing to the Synthesis of Starlight. Musical experiences with the Phase Vocoder over 25 years.”’ *SHILAP Revista de lepidopterología* (2013). [DOI](#).

Books

The twenty-five books in this section provide durable background and extended treatments. Editions are stated when the catalog distinguishes them.

Ballou, Glen M., editor. *Handbook for Sound Engineers*. Focal Press, 2015. 5th edition.

Benesty, Jacob; Sondhi, M. Mohan; Huang, Yiteng, editors. *Springer Handbook of Speech Processing*. Springer, 2008.

Bregman, Albert S.. *Auditory Scene Analysis: The Perceptual Organization of Sound*. MIT Press, 1990.

Cohen, Leon. *Time-Frequency Analysis*. Prentice Hall, 1995.

Cook, Perry R.. *Real Sound Synthesis for Interactive Applications*. A K Peters, 2002.

Crochiere, Ronald E.; Rabiner, Lawrence R.. *Multirate Digital Signal Processing*. Prentice Hall, 1983.

Dodge, Charles; Jerse, Thomas A.. *Computer Music: Synthesis, Composition, and Performance*. Schirmer Books, 1997. 2nd edition.

Flanagan, James L.. *Speech Analysis, Synthesis and Perception*. Springer, 1972. 2nd edition.

Flandrin, Patrick. *Time-Frequency/Time-Scale Analysis*. Academic Press, 1999.

Grochenig, Karlheinz. *Foundations of Time-Frequency Analysis*. Birkhauser, 2001.

Klapuri, Anssi; Davy, Manuel, editors. *Signal Processing Methods for Music Transcription*. Springer, 2006.

Loizou, Philipos C.. *Speech Enhancement: Theory and Practice*. CRC Press, 2013. 2nd edition.

Muller, Meinard. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.

Oppenheim, Alan V.; Schafer, Ronald W.. *Discrete-Time Signal Processing*. Prentice Hall, 2010. 3rd edition.

Pohlmann, Ken C.. *Principles of Digital Audio*. McGraw-Hill, 2010. 6th edition.

Rabiner, Lawrence R.; Schafer, Ronald W.. *Digital Processing of Speech Signals*. Prentice Hall, 1978.

Roads, Curtis. *The Computer Music Tutorial*. MIT Press, 1996.

Roads, Curtis. *Microsound*. MIT Press, 2001.

Smith, Julius O.. *Mathematics of the Discrete Fourier Transform with Audio Applications*. W3K Publishing, 2007. 2nd edition.

Smith, Julius O.. *Spectral Audio Signal Processing*. W3K Publishing, 2011.

Wiener, Norbert. *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. MIT Press, 1949.

Wishart, Trevor. *Audible Design: A Plain and Easy Introduction to Practical Sound Composition*. Orpheus the Pantomime, 1994.

Wishart, Trevor. *On Sonic Art*. Harwood Academic Publishers, 1996. new and revised edition.

Zolzer, Udo, editor. *DAFX: Digital Audio Effects*. Wiley, 2011. 2nd edition.

Zwicker, Eberhard; Fastl, Hugo. *Psychoacoustics: Facts and Models*. Springer, 1999. 2nd edition.

Open Media Credits

The historical illustrations in Chapter 1 are reproduced from Homer Dudley's 1940 paper, *The Carrier Nature of Speech*, via Internet Archive Book Images and Wikimedia Commons. Wikimedia Commons records them as public-domain or no-known-copyright-restrictions scans. Source records: Voder demonstration, voice-mechanism block diagram, and vocoder schematic: <https://commons.wikimedia.org/wiki/Category:Vocoder>. The analytical plots and window diagrams are original pvx project assets and are included with the project documentation.

Index

1. Trevor Wishart Vox 5, 496
10. Wendy Carlos Digital Moonscapes, 498
100. Max Richter Sleep, 522
11. Jean-Claude Risset Inharmonique, 499
12. Jean-Claude Risset Songes, 499
13. Kaija Saariaho Lichtbogen, 499
14. Kaija Saariaho NoaNoa, 499
15. Paul Lansky Six Fantasies on a Poem by Thomas Campion, 499
16. Hildegard von Bingen O vis aeternitatis, 500
17. Anonymous Dies irae, 500
18. Josquin des Prez Ave Maria virgo serena, 500
19. Thomas Tallis Spem in alium, 500
2. Roger Reynolds Transfigured Wind I, 496
20. Claudio Monteverdi Lamento d Arianna, 501
21. Henry Purcell Dido s Lament, 501
22. Johann Sebastian Bach Erbarne dich, 501
23. George Frideric Handel Lascia ch io pianga, 501
24. Wolfgang Amadeus Mozart Der Holle Rache, 501
25. Franz Schubert Erlkonig, 502
26. Richard Wagner Mild und leise, 502
27. Gustav Mahler Ich bin der Welt abhanden gekommen, 502
28. Arnold Schoenberg Mondestrunken, 502
29. Luciano Berio Sequenza III, 502
3. JoAnn Kuchera-Morin Dreampaths, 496
30. Steve Reich Different Trains, 503
31. Antonio Vivaldi Winter Largo, 504
32. Johann Sebastian Bach Air on the G String, 504
33. Ludwig van Beethoven Symphony No. 7 Allegretto, 504
34. Franz Schubert String Quintet in C Adagio, 504
35. Frederic Chopin Nocturne in E-flat major Op. 9 No. 2, 505
36. Richard Wagner Prelude to Das Rheingold, 505
37. Anton Bruckner Symphony No. 7 Adagio, 505
38. Claude Debussy Prelude a l apres-midi d un faune, 505
39. Maurice Ravel Daphnis et Chloe Lever du jour, 505
4. Jonathan Harvey Mortuos Plango Vivos Voco, 496
40. Ralph Vaughan Williams Fantasia on a Theme by Thomas Tallis, 506
41. Gustav Holst Neptune the Mystic, 506
42. Olivier Messiaen Louange a l Eternite de Jesus, 506
43. Gyorgy Ligeti Lux Aeterna, 506
44. Arvo Part Spiegel im Spiegel, 506
45. John Tavener The Protecting Veil, 507
46. Johann Sebastian Bach Prelude in C major BWV 846, 508
47. Ludwig van Beethoven Piano Sonata No. 14 third movement, 508
48. Niccolo Paganini Caprice No. 24, 508
49. Frederic Chopin Etude Op. 10 No. 4, 508
5. Trevor Wishart Supernova, 497
- 5.1/Multichannel Reference-Lock Stretch, 311
50. Nikolai Rimsky-Korsakov Flight of the Bumblebee, 509
51. Igor Stravinsky The Rite of Spring Augurs of Spring, 509
52. Edgard Varese Ionisation, 509
53. John Cage First Construction in Metal, 509
54. Dave Brubeck Quartet Take Five, 509
55. James Brown Funky Drummer, 510
56. Kraftwerk Numbers, 510
57. Public Enemy Bring the Noise, 510
58. Aphex Twin Vordhosbn, 510
59. Squarepusher My Red Hot Car, 510
6. Trevor Wishart Vox Cycle, 497
60. Autechre Gantz Graf, 511
61. Giovanni Pierluigi da Palestrina Missa Papae Marcelli Kyrie, 512
62. Johann Sebastian Bach The Art of Fugue Contrapunctus I, 512
63. Wolfgang Amadeus Mozart Symphony No. 40 first movement, 512
64. Ludwig van Beethoven Symphony No. 9 finale, 512
65. Hector Berlioz Symphonie fantastique Marche au supplice, 513
66. Johannes Brahms Symphony No. 4 finale, 513
67. Gustav Mahler Symphony No. 2 finale, 513
68. Claude Debussy La mer Dialogue du vent et de la mer, 513
69. Charles Ives The Unanswered Question, 513
7. Trevor Wishart Red Bird, 498

- 70. Igor Stravinsky The Firebird Infernal Dance, 514
- 71. Bela Bartok Music for Strings Percussion and Celesta third movement, 514
- 72. Benjamin Britten The Young Person s Guide to the Orchestra, 514
- 73. Iannis Xenakis Metastaseis, 514
- 74. Gyorgy Ligeti Atmospheres, 514
- 75. John Adams Short Ride in a Fast Machine, 515
- 76. Karlheinz Stockhausen Gesang der Junglinge, 516
- 77. Karlheinz Stockhausen Kontakte, 516
- 78. Delia Derbyshire Doctor Who Theme, 516
- 79. The Beatles Tomorrow Never Knows, 516
- 8. Trevor Wishart Tongues of Fire, 498
- 80. Pink Floyd On the Run, 517
- 81. Brian Eno An Ending (Ascent), 517
- 82. Laurie Anderson O Superman, 517
- 83. Kate Bush Running Up That Hill, 517
- 84. Cocteau Twins Lorelei, 517
- 85. My Bloody Valentine Only Shallow, 518
- 86. The Orb Little Fluffy Clouds, 518
- 87. Boards of Canada Roygbiv, 518
- 88. Bjork All Is Full of Love, 518
- 89. Radiohead Everything in Its Right Place, 518
- 9. Tristan Murail Desintegrations, 498
- 90. Oneohtrix Point Never Replica, 519
- 91. Johann Pachelbel Canon in D, 520
- 92. Ludwig van Beethoven Symphony No. 5 opening, 520
- 93. Richard Wagner Prelude to Tristan und Isolde, 520
- 94. Maurice Ravel Bolero, 520
- 95. Samuel Barber Adagio for Strings, 521
- 96. John Cage Organ2/ASLSP, 521
- 97. Steve Reich Piano Phase, 521
- 98. Alvin Lucier I Am Sitting in a Room, 521
- 99. William Basinski The Disintegration Loops 1.1, 521
- A chronology of ideas not merely machines, 7
- A decade-by-decade change in the imagined user, 37
- A deeper history of spectral time, 13
- A fuller genealogy of phase-vocoder software, 27
- A historiography of claims and cautions, 25
- A Phase-Vocoder Listening Library 100 Musical Works, 495
- A quick mental model, 3
- A small comparison, 43
- A/B report generation, 381
- A/B sweep of transform backends from shell loop, 385
- Adaptive and nonstationary time-frequency analysis, 24
- Algorithm Registry and Dispatch, 75
- Aliasing, 549
- Alpha Principles, 397
- Ambient Macro-Chain (Stretch + Cleanup + Loudness), 309
- Amplitude, 549
- Analog Tape Methods for Pitch/Time Shifting, 49
- Analysis frame, 549
- Analysis hop, 549
- analysis hop, 5
- analysis qa and automation, 464
- Analysis-synthesis system, 549
- Analysis/QA, 381
- Analysis/Response Artifact and Spectral Operator Ideas, 344
- Annotated chronology, 39
- Artifact, 549
- Artifact - Fix Table, 59
- Artifact history as listening history, 30
- ASR Automatic Speech Recognition, 373
- Attack, 549
- Audio Event Detection / Sound Classification, 378
- Audio Metrics Table Generation, 481
- AudioCollator for variable-length batches, 358
- Augmentation Benchmark Gate, 326
- Augmentation Intensity Guide, 379
- Auto Profile + Auto Transform (Planning), 300
- Auto-profile plan preview, 383
- Automation, 381
- Automation and interpolation, 540
- Automation curve, 549
- Available GPU transforms, 364
- Available Transforms, 364
- Backend Summary, 390
- Backward Compatibility and UX, 494
- Bandlimit, 549
- Bandwidth, 549
- bartlett, 121
- bartlett hann, 133
- Basic DSP Terms (Plain Language), 49
- Basic map() usage, 358
- Batch, 381
- Batch Processing Example (Python), 353
- Batch Processing Folder, 299
- Batch stretch over folder, 381
- Batch Tree Processing with find + xargs, 308
- Beat phase + downbeat phase, 330

-
- Beat Tracking and Tempo Estimation, 376
 - Bell Laboratories as a long technical precondition, 32
 - Benchmark Command (pvx vs Rubber Band vs librosa), 304
 - Benchmark matrix sweep, 381
 - Benchmark Metric Taxonomy, 481
 - Benchmark Runner + CI Gate, 479
 - Benchmark Spec, 389
 - Benchmarking Augmentation Pipelines, 370
 - Benefits we can extract for pvx, 483
 - Beta in 0.1.x, 63
 - bibliography, 557
 - books, 562
 - papers and articles, 557
 - Bin, 549
 - Bin-center frequency, 549
 - Bit depth, 549
 - blackman, 93
 - blackman nuttall, 109
 - blackmanharris, 97
 - Blend pitch behavior from two guides, 333
 - Block processing, 549
 - bohman, 269
 - boxcar, 125
 - Broadband, 549
 - Broadcast-Style Speech Finishing (Stretch + Loudness + Safety), 309
 - Buffer, 549
 - Build custom vector features then route them, 333
 - Caching expensive transforms, 361
 - Capability Matrix, 287
 - cauchy 0p5, 217
 - cauchy 1p0, 221
 - cauchy 2p0, 225
 - Cepstrum, 549
 - Channel, 549
 - Checkpoint, 550
 - Checkpoint + Resume Long Render, 300
 - Checkpoint / Resume State Machine, 480
 - Checkpointed long render with manifest, 383
 - Checkpointed Long Render Workflow, 307
 - Chroma, 550
 - Chunked Stream Wrapper + Output Policy Controls, 314
 - CI + Pages, 75
 - CLI options
 - 1950s, 43
 - 1970s, 43
 - 1980s, 43
 - a-args, 385
 - a4-reference-hz, 57, 326, 461
 - pvxretune, 547
 - alpha, 57, 326, 348, 461
 - pvxmorph, 547
 - ambient-phase-mix, 57, 461
 - pvxvoc, 547
 - ambient-preset, 385, 461
 - pvxvoc, 547
 - amplitude, 57, 326, 461
 - pvxenvelope, 547
 - analysis-channel, 461
 - pvxanalysis, 547
 - pvxvoc, 547
 - append-manifest, 326, 371, 461
 - pvx, 547
 - attack-sec, 326, 348, 461
 - pvxenvelope, 547
 - attenuation, 326, 348, 461
 - pvxharmmap, 547
 - audit-metrics, 461
 - pvx, 547
 - auto-profile, 57, 326, 385, 461
 - pvxvoc, 547
 - auto-profile-lookahead-seconds, 326, 461
 - pvxvoc, 547
 - auto-segment-seconds, 57, 326, 348, 385, 461
 - pvxvoc, 547
 - auto-transform, 57, 326, 385, 461
 - pvxvoc, 547
 - azimuth-deg, 385
 - b-args, 385
 - backend, 326, 461
 - hps-pitch-track, 547
 - pvx, 547
 - background-dir, 461
 - pvxnoise, 547
 - band, 461
 - pvxnoise, 547
 - band-gain-db, 326, 348, 461
 - pvxfilter, 547
 - band-width-bins, 326, 461
 - pvxfilter, 547
 - baseline, 57, 326
 - baseline-json, 385
 - batch-size, 461
 - pvx, 547
 - bit-depth, 57, 61, 326, 348, 461
 - pvx, 547
 - pvxcodec, 547
 - pvxgain, 547
 - pvxnoise, 547

- pvxrir, 547
- pvxspecaugment, 547
- pvxvoc, 547
- bits, 461
 - pvxcodec, 547
- blend-mode, 57, 326, 348, 461
 - pvxmorph, 547
- boost-db, 326, 348, 461
 - pvxharmmap, 547
- budget-path, 57, 461
 - pvx, 547
- build-from-source, 396
- cache-dir, 461
 - pvxrir, 547
- category, 461
 - pvxrir, 547
- center, 57, 326, 461
 - pvxenvelope, 547
- cents, 461
 - pvxvoc, 547
- checkpoint-dir, 57, 326, 348, 385, 461
 - pvxvoc, 547
- checkpoint-id, 461
 - pvxvoc, 547
- chord, 326, 348, 461
 - pvxharmmap, 547
- chunk-ms, 461
 - pvxretune, 547
- chunk-seconds, 57, 326, 348, 461
 - pvx, 547
- clip, 461
 - pvxvoc, 547
- codec, 379, 461
 - pvxcodec, 547
- coherence-report-json, 484
- coherence-strength, 57, 61, 326, 348, 461
 - pvxvoc, 547
- comp-ratio, 326, 348, 461
 - pvxfilter, 547
- comp-threshold-db, 326, 348, 461
 - pvxfilter, 547
- compander-attack-ms, 461
 - pvxvoc, 547
- compander-compress-ratio, 461
 - pvxvoc, 547
- compander-expand-ratio, 461
 - pvxvoc, 547
- compander-makeup-db, 461
 - pvxvoc, 547
- compander-release-ms, 461
 - pvxvoc, 547
- compander-threshold-db, 461
 - pvxvoc, 547
- compressor-attack-ms, 385, 461
 - pvxvoc, 547
- compressor-makeup-db, 385, 461
 - pvxvoc, 547
- compressor-ratio, 326, 385, 461
 - pvxvoc, 547
- compressor-release-ms, 385, 461
 - pvxvoc, 547
- compressor-threshold-db, 326, 385, 461
 - pvxvoc, 547
- confidence-floor, 461
 - hps-pitch-track, 547
 - pvx, 547
- context-ms, 461
 - pvx, 547
- control-stdin, 57, 326, 336, 337, 348, 461
 - pvxvoc, 547
- coord-system, 461
 - pvxtrajectoryreverb, 547
- cpu, 461
 - pvxvoc, 547
- crossfade-ms, 326, 348, 461
 - pvx, 547
 - pvxconform, 547
 - pvxwarp, 547
- cuda-device, 461
 - pvxanalysis, 547
 - pvxvoc, 547
- cycles, 326, 461
 - pvxenvelope, 547
- decay, 326, 461
 - pvxdeverb, 547
- decay-sec, 326, 348, 461
 - pvxenvelope, 547
- dedupe-by, 326, 461
 - pvx, 547
- depth, 326, 348, 461
 - pvxring, 547
- detune-cents, 57, 326, 385, 461
 - pvxunison, 547
- device, 326, 461
 - pvx, 547
 - pvxanalysis, 547
 - pvxvoc, 547
- devices, 385
- disk-budget, 57, 326, 348, 461
 - pvx, 547
- distance-law, 461
 - pvxtrajectoryreverb, 547

-
- dither, 57, 61, 326, 348, 461
 - pvxvoc, 547
 - dither-seed, 57, 326, 348, 461
 - pvxvoc, 547
 - drr, 461
 - pvxrir, 547
 - dry, 461
 - pvxtrajectoryreverb, 547
 - dry-mix, 326, 348, 461
 - pvxfilter, 547
 - pvxharmmap, 547
 - pvxunison, 547
 - dry-run, 61, 326, 371, 385, 461
 - pvx, 547
 - pvxvoc, 547
 - duration, 43, 57, 326, 348, 461
 - pvx, 547
 - pvxenvelope, 547
 - pvxfreeze, 547
 - duty-cycle, 326, 461
 - pvxenvelope, 547
 - emit, 57, 326, 336, 337, 348, 461
 - hps-pitch-track, 547
 - pvx, 547
 - end, 348, 461
 - pvxenvelope, 547
 - pvxtrajectoryreverb, 547
 - engine, 366, 379, 461
 - pvx, 547
 - engines, 366
 - envelope-lifter, 326, 348, 461
 - pvxmorph, 547
 - example, 57, 61, 286, 326, 336, 348, 461
 - pvx, 547
 - pvxvoc, 547
 - exp-curve, 348, 461
 - pvxenvelope, 547
 - expand-ratio, 326, 348, 461
 - pvxfilter, 547
 - expander-attack-ms, 461
 - pvxvoc, 547
 - expander-ratio, 461
 - pvxvoc, 547
 - expander-release-ms, 461
 - pvxvoc, 547
 - expander-threshold-db, 461
 - pvxvoc, 547
 - explain-plan, 57, 61, 326, 354, 385, 461
 - pvxvoc, 547
 - exponent, 461
 - pvxreshape, 547
 - extreme-stretch-threshold, 57, 461
 - pvxvoc, 547
 - extreme-time-stretch, 461
 - pvxvoc, 547
 - f0-max, 326, 461
 - pvxretune, 547
 - pvxvoc, 547
 - f0-min, 326, 461
 - pvxretune, 547
 - pvxvoc, 547
 - factor, 348, 461
 - pvxreshape, 547
 - fail-if-exceeds, 57, 326, 348, 461
 - pvx, 547
 - feature-set, 57, 326, 336, 348, 461
 - hps-pitch-track, 547
 - pvx, 547
 - feedback, 326, 348, 461
 - pvxring, 547
 - fill, 461
 - pvxspecaugment, 547
 - floor, 57, 61, 326, 461
 - pvxdenoise, 547
 - pvxdeverb, 547
 - fmax, 461
 - hps-pitch-track, 547
 - pvx, 547
 - fmin, 461
 - hps-pitch-track, 547
 - pvx, 547
 - force, 396, 461
 - pvxvoc, 547
 - force-stereo, 326, 348, 461
 - pvxharmonize, 547
 - formant-lifter, 57, 61, 326, 461
 - pvxformant, 547
 - pvxvoc, 547
 - formant-max-gain-db, 57, 461
 - pvxformant, 547
 - pvxvoc, 547
 - formant-preserve, 43
 - formant-shift-ratio, 326, 461
 - pvxformant, 547
 - formant-strength, 57, 61, 326, 461
 - pvxvoc, 547
 - format, 461
 - pvxenvelope, 547
 - pvxreshape, 547
 - formula, 396
 - fourier-sync, 461
 - pvxvoc, 547

- fourier-sync-max-fft, 57, 461
 - pvxvoc, 547
- fourier-sync-min-fft, 57, 461
 - pvxvoc, 547
- fourier-sync-smooth, 57, 461
 - pvxvoc, 547
- frame-length, 461
 - hps-pitch-track, 547
 - pvx, 547
- freeze-time, 43, 57, 326, 348, 461
 - pvxfreeze, 547
- freq, 461
 - pvxenvelope, 547
- freq-mask, 461
 - pvxspecaugment, 547
- frequency-hz, 57, 326, 348, 461
 - pvxenvelope, 547
 - pvxring, 547
- gain, 461
 - pvxgain, 547
- gains, 57, 326, 348, 461
 - pvxharmonize, 547
- gate, 57, 326, 371, 397
- gate-tolerance, 326
- gpu, 326, 461
 - pvxvoc, 547
- group-separator, 326, 371, 461
 - pvx, 547
- grouping, 326, 371, 461
 - pvx, 547
- guided, 286, 461
 - pvxvoc, 547
- hard-clip-level, 326, 385, 461
 - pvxvoc, 547
- harmonic-gain, 326, 461
 - pvxlayer, 547
- harmonic-kernel, 461
 - pvxlayer, 547
- harmonic-pitch-cents, 461
 - pvxlayer, 547
- harmonic-pitch-semitones, 326, 348, 461
 - pvxlayer, 547
- harmonic-stretch, 326, 348, 461
 - pvxlayer, 547
- help, 57, 286, 396
- hop-length, 461
 - pvxspecaugment, 547
- hop-size, 57, 74, 326, 348, 385, 461
 - hps-pitch-track, 547
 - pvx, 547
 - pvxanalysis, 547
 - pvxvoc, 547
- inharmonic-f0-hz, 326, 348, 461
 - pvxharmmmap, 547
- inharmonic-mix, 326, 348, 461
 - pvxharmmmap, 547
- inharmonicity, 326, 348, 461
 - pvxharmmmap, 547
- input, 385
- input-format, 461
 - pvxreshape, 547
- intent, 57, 326, 366, 371, 379, 461
 - pvx, 547
- interp, 57, 74, 326, 348, 461, 481
 - pvxmorph, 547
 - pvxreshape, 547
 - pvxvoc, 547
- intervals, 57, 326, 348, 385, 461
 - pvxharmonize, 547
- intervals-cents, 326, 348, 461
 - pvxharmonize, 547
- ir, 461
 - pvxtrajectoryreverb, 547
- ir-database, 461
 - pvxrir, 547
- ir-dir, 461
 - pvxrir, 547
- ir-file, 461
 - pvxrir, 547
- json, 57, 326, 348, 461
 - pvx, 547
 - pvxanalysis, 547
 - pvxresponse, 547
- kaiser-beta, 57, 284, 385, 461
 - pvxanalysis, 547
 - pvxvoc, 547
- keep-intermediate, 461
 - pvx, 547
- key, 57, 326, 348, 461
 - pvxenvelope, 547
 - pvxreshape, 547
- label-policy, 371, 379, 461
 - pvx, 547
- labels-csv, 326, 371, 461
 - pvx, 547
- limiter-threshold, 326, 385, 461
 - pvxvoc, 547
- list-databases, 461
 - pvxrir, 547
- manifest-append, 326, 348, 461
 - pvxvoc, 547
- manifest-csv, 371, 461

- pvx, 547
- manifest-json, 66, 326, 348, 354, 385, 461
 - pvxvoc, 547
- manifest-jsonl, 371, 461
 - pvx, 547
- map, 57, 66, 326, 348, 461
 - pvxconform, 547
 - pvxwarp, 547
- mask-exponent, 74, 326, 348, 461
 - pvxmorph, 547
- material, 57, 326, 461
 - pvx, 547
- max, 461
 - pvxenvelope, 547
 - pvxreshape, 547
- max-length-s, 461
 - pvx, 547
- max-stage-stretch, 57, 61, 326, 461
 - pvxvoc, 547
- metadata-policy, 57, 326, 348, 461
 - pvxvoc, 547
- method, 326, 348, 461
 - pvxresponse, 547
- mfcc-count, 57, 326, 336, 348, 461
 - hps-pitch-track, 547
 - pvx, 547
- min, 461
 - pvxenvelope, 547
 - pvxreshape, 547
- mix, 326, 348, 385, 461
 - pvxring, 547
- mode, 57, 326, 348, 385, 461
 - pvx, 547
 - pvxenvelope, 547
 - pvxformant, 547
- multires-ffts, 326, 385, 461
 - pvxvoc, 547
- multires-fusion, 57, 61, 326, 385, 461
 - pvxvoc, 547
- multires-weights, 326, 385, 461
 - pvxvoc, 547
- n-fft, 57, 61, 66, 74, 326, 348, 385, 461
 - pvxanalysis, 547
 - pvxspecaugment, 547
 - pvxvoc, 547
- n-ffts, 385
- name, 385
- no-audit-metrics, 461
 - pvx, 547
- no-center, 461
 - pvxanalysis, 547
- pvxvoc, 547
- no-normalize-gains, 461
 - pvxtrajectoryreverb, 547
- no-onset-realign, 461
 - pvxvoc, 547
- no-progress, 461
 - pvx, 547
 - pvxvoc, 547
- no-trim, 461
 - pvxrir, 547
- noise-file, 57, 61, 326, 461
 - pvxdenoise, 547
- noise-floor, 326, 348, 461
 - pvxfilter, 547
- noise-seconds, 57, 61, 326, 461
 - pvxdenoise, 547
- noise-type, 379, 461
 - pvxnoise, 547
- normalize, 326, 348, 385, 461
 - pvxgain, 547
 - pvxresponse, 547
 - pvxvoc, 547
- normalize-energy, 461
 - pvxmorph, 547
- normalize-gains, 461
 - pvxtrajectoryreverb, 547
- num-freq-masks, 461
 - pvxspecaugment, 547
- num-time-masks, 461
 - pvxspecaugment, 547
- o——-o——-o, 481
- o——-o—o—o——, 481
- offset, 348, 461
 - pvxreshape, 547
- onset-credit-max, 57, 461
 - pvxvoc, 547
- onset-credit-pull, 57, 461
 - pvxvoc, 547
- onset-time-credit, 461
 - pvxvoc, 547
- operation, 326, 348, 461
 - pvxreshape, 547
- operator, 461
 - pvxfilter, 547
 - pvxharmmap, 547
 - pvxring, 547
- order, 57, 74, 326, 348, 461, 481
 - pvxmorph, 547
 - pvxreshape, 547
 - pvxvoc, 547
- out, 461

- pvx, 547
- pvxenvelope, 547
- pvxreshape, 547
- pvxvoc, 547
- out-dir, 57, 326, 371, 385, 397
- output, 43, 57, 61, 326, 336, 337, 348, 379, 385, 461, 481
 - hps-pitch-track, 547
 - pvx, 547
 - pvxanalysis, 547
 - pvxcodec, 547
 - pvxenvelope, 547
 - pvxgain, 547
 - pvxmorph, 547
 - pvxnoise, 547
 - pvxreshape, 547
 - pvxresponse, 547
 - pvxrir, 547
 - pvxspecaugment, 547
 - pvxvoc, 547
- output-channels, 385
- output-csv, 326, 371, 461
 - pvx, 547
- output-dir, 57, 74, 326, 366, 371, 379, 385, 461
 - pvx, 547
 - pvxvoc, 547
- output-format, 57, 66, 461
 - pvx, 547
 - pvxmorph, 547
 - pvxvoc, 547
- output-jsonl, 326, 371, 461
 - pvx, 547
- output-key, 461
 - pvxreshape, 547
- output-subtype, 461
 - pvx, 547
- output-suffix, 461
 - pvx, 547
- overlap-ms, 461
 - pvxretune, 547
- overwrite, 326, 385, 461
 - pvx, 547
 - pvxmorph, 547
 - pvxvoc, 547
- pair-mode, 326, 371, 461
 - pvx, 547
- pans, 326, 348, 461
 - pvxharmonize, 547
- peak, 348, 461
 - pvxenvelope, 547
- peak-count, 326, 348, 461
 - pvxfilter, 547
- peak-dbf, 385, 461
 - pvxvoc, 547
- percussive-gain, 326, 348, 461
 - pvxlayer, 547
- percussive-kernel, 461
 - pvxlayer, 547
- percussive-pitch-cents, 461
 - pvxlayer, 547
- percussive-pitch-semitones, 326, 461
 - pvxlayer, 547
- percussive-stretch, 326, 348, 461
 - pvxlayer, 547
- phase, 461
 - pvxenvelope, 547
- phase-engine, 348, 385, 461
 - pvxvoc, 547
- phase-lock-radius, 484
- phase-lock-strength, 484
- phase-locking, 43, 57, 61, 74, 326, 385, 461
 - pvxvoc, 547
- phase-mix, 326, 348, 461
 - pvxmorph, 547
- phase-mode, 326, 461
 - pvxfreeze, 547
 - pvxresponse, 547
- phase-noise-floor-db, 484
- phase-noise-lock, 484
- phase-policy, 484
- phase-rad, 461
 - pvxenvelope, 547
- phase-random-seed, 461
 - pvxvoc, 547
- phase-tonal-lock, 484
- pipeline, 57, 326, 348, 461
 - pvx, 547
- pipeline-config, 461
 - pvx, 547
- pitch, 43, 57, 326, 348, 379, 461
 - pvx, 547
 - pvxvoc, 547
- pitch-conf-min, 57, 326, 336, 337, 348, 385, 461
 - pvx, 547
 - pvxvoc, 547
- pitch-follow-stdin, 385, 461
 - pvxvoc, 547
- pitch-lowconf-mode, 326, 336, 385, 461
 - pvx, 547
 - pvxvoc, 547

-
- pitch-map, 57, 66, 326, 336, 461
 - pvxvoc, 547
 - pitch-map-crossfade-ms, 326, 461
 - pvx, 547
 - pvxvoc, 547
 - pitch-map-smooth-ms, 336, 461
 - pvx, 547
 - pvxvoc, 547
 - pitch-map-stdin, 57, 461
 - pvxvoc, 547
 - pitch-mode, 57, 61, 326, 348, 385, 461
 - pvxvoc, 547
 - pitch-ratio, 326
 - pitch-shift-cents, 57, 74, 385, 461
 - pvxformant, 547
 - pvxtransient, 547
 - pvxvoc, 547
 - pitch-shift-ratio, 57, 66, 326, 461
 - pvxtransient, 547
 - pvxvoc, 547
 - pitch-shift-semitones, 57, 385, 461
 - pvxformant, 547
 - pvxtransient, 547
 - pvxvoc, 547
 - policy, 371, 461
 - pvx, 547
 - pre-delay-ms, 461
 - pvxrir, 547
 - prefix, 396
 - preset, 57, 61, 326, 348, 379, 461
 - pvxvoc, 547
 - quality-profile, 461
 - pvxvoc, 547
 - quick, 57, 326, 371, 397
 - quiet, 385, 461
 - pvx, 547
 - pvxvoc, 547
 - random-phase, 461
 - pvxfreeze, 547
 - rate, 326, 348, 461
 - pvxenvelope, 547
 - pvxreshape, 547
 - ratio, 348, 461
 - pvxvoc, 547
 - ratio-max, 461
 - hps-pitch-track, 547
 - pvx, 547
 - ratio-min, 461
 - hps-pitch-track, 547
 - pvx, 547
 - ratio-reference, 461
 - hps-pitch-track, 547
 - pvx, 547
 - recommend-root, 57, 326, 461
 - pvxretune, 547
 - reduction-db, 57, 61, 326, 336, 348, 385, 461
 - pvxdenoise, 547
 - ref-channel, 61, 326, 461
 - pvxvoc, 547
 - reference-hz, 461
 - hps-pitch-track, 547
 - pvx, 547
 - refresh-baselines, 371
 - release-sec, 326, 348, 461
 - pvxenvelope, 547
 - render-args, 385
 - report-json, 385
 - requested-stretch, 57, 326, 348, 461
 - pvx, 547
 - resample-mode, 461
 - pvxconform, 547
 - pvxformant, 547
 - pvxharmonize, 547
 - pvxlayer, 547
 - pvxretune, 547
 - pvxtransient, 547
 - pvxunison, 547
 - pvxvoc, 547
 - pvxwarp, 547
 - resonance-decay, 326, 461
 - pvxring, 547
 - resonance-hz, 326, 348, 461
 - pvxring, 547
 - resonance-mix, 326, 348, 461
 - pvxring, 547
 - resonance-q, 326, 348, 461
 - pvxring, 547
 - response, 326, 348, 461
 - pvxfilter, 547
 - response-gain-db, 461
 - pvxfilter, 547
 - response-mix, 326, 348, 461
 - pvxfilter, 547
 - resume, 57, 326, 348, 371, 461
 - pvx, 547
 - pvxvoc, 547
 - rms-dbfs, 461
 - pvxvoc, 547
 - root, 57, 326, 348, 385, 461
 - pvxretune, 547
 - root-hz, 57, 326, 348, 461
 - pvxharmmap, 547

- pvxretune, 547
- route, 57, 326, 336, 337, 348, 461
 - pvx, 547
 - pvxvoc, 547
- rt60, 379, 461
 - pvxrir, 547
- run-benchmarks, 392
- safety-margin, 57, 461
 - pvx, 547
- sample-rate, 326, 461
 - pvx, 547
- scale, 57, 326, 348, 461
 - pvxretune, 547
- scale-cents, 385, 461
 - pvxretune, 547
- seed, 57, 326, 366, 371, 379, 461
 - pvx, 547
 - pvxcodec, 547
 - pvxgain, 547
 - pvxnoise, 547
 - pvxrir, 547
 - pvxspecaugment, 547
- semitones, 461
 - pvxvoc, 547
- shift-bins, 348, 461
 - pvxfilter, 547
- silent, 461
 - pvx, 547
 - pvxvoc, 547
- sine-cycles, 348, 461
 - pvxenvelope, 547
- sine-phase-rad, 461
 - pvxenvelope, 547
- smooth, 57, 61, 326, 348, 461
 - pvxdenoise, 547
- smooth-frames, 461
 - hps-pitch-track, 547
 - pvx, 547
- smoothing-bins, 326, 461
 - pvxresponse, 547
- snr, 379, 461
 - pvxnoise, 547
- soft-clip-drive, 461
 - pvxvoc, 547
- soft-clip-level, 74, 326, 385, 461
 - pvxvoc, 547
- soft-clip-type, 385, 461
 - pvxvoc, 547
- speaker-angles, 461
 - pvxtrajectoryreverb, 547
- split, 326, 371, 461
 - pvx, 547
- split-mode, 326, 371, 461
 - pvx, 547
- sr, 461
 - pvx, 547
- start, 348, 461
 - pvxenvelope, 547
 - pvxtrajectoryreverb, 547
- stdout, 57, 66, 326, 348, 385, 461, 481
 - pvxenvelope, 547
 - pvxmorph, 547
 - pvxreshape, 547
 - pvxvoc, 547
- stereo-mode, 57, 61, 326, 348, 461
 - pvxvoc, 547
- strength, 57, 326, 336, 348, 385, 461
 - pvxdeverb, 547
 - pvxharmmmap, 547
 - pvxretune, 547
- stretch, 43, 57, 61, 66, 326, 336, 348, 379, 461
 - hps-pitch-track, 547
 - pvx, 547
 - pvxvoc, 547
- stretch-from, 461
 - hps-pitch-track, 547
 - pvx, 547
- stretch-max, 461
 - hps-pitch-track, 547
 - pvx, 547
- stretch-min, 461
 - hps-pitch-track, 547
 - pvx, 547
- stretch-mode, 57, 61, 348, 385, 461
 - pvxvoc, 547
- stretch-scale, 461
 - hps-pitch-track, 547
 - pvx, 547
- strict, 57, 326, 371, 396, 461
 - pvx, 547
- strict-manifest, 461
 - pvx, 547
- subtype, 57, 461
 - pvx, 547
 - pvxvoc, 547
- suffix, 57, 326, 385, 461
 - pvxvoc, 547
- summary-json, 461
 - pvxanalysis, 547
 - pvxresponse, 547
- sustain, 326, 348, 461

- pvxenvelope, 547
- target, 461
 - pvxgain, 547
- target-duration, 57, 326, 348, 385, 461
 - pvxtransient, 547
 - pvxvoc, 547
- target-f0, 326, 461
 - pvxvoc, 547
- target-lufs, 326, 385, 461
 - pvxvoc, 547
- target-max, 348, 461
 - pvxreshape, 547
- target-min, 348, 461
 - pvxreshape, 547
- target-pitch-shift-semitones, 461
 - pvxvoc, 547
- target-sample-rate, 461
 - pvxvoc, 547
- time-mask, 461
 - pvxspecaugment, 547
- time-stretch, 57, 61, 74, 326, 348, 385, 461
 - pvxtransient, 547
 - pvxvoc, 547
- time-stretch-factor, 385, 461
 - pvxvoc, 547
- tolerance-cents, 326, 348, 461
 - pvxharmmap, 547
- trajectory-shape, 461
 - pvxtrajectoryreverb, 547
- transform, 74, 326, 385, 461
 - pvxanalysis, 547
 - pvxvoc, 547
- transforms, 385
- transient-crossfade-ms, 57, 61, 326, 348, 461
 - pvxvoc, 547
- transient-mode, 57, 61, 326, 348, 379, 461
 - pvxvoc, 547
- transient-preserve, 43, 57, 74, 385, 461
 - pvxvoc, 547
- transient-protect-ms, 57, 61, 326, 348, 461
 - pvxvoc, 547
- transient-sensitivity, 57, 61, 326, 348, 461
 - pvxvoc, 547
- transient-threshold, 57, 461
 - pvxtransient, 547
 - pvxvoc, 547
- transpose-semitones, 348, 461
 - pvxfilter, 547
- true-peak-max-dbtp, 57, 61, 461
 - pvxvoc, 547
- tv-interp, 326, 348, 461
 - pvxfilter, 547
 - pvxring, 547
- tv-key, 326, 461
 - pvxfilter, 547
- tv-map, 326, 348, 461
 - pvxfilter, 547
 - pvxring, 547
- tv-order, 326, 461
 - pvxfilter, 547
 - pvxring, 547
- variants-per-input, 57, 326, 366, 371, 379, 461
 - pvx, 547
- verbose, 461
 - pvxvoc, 547
- verbosity, 461
 - pvxvoc, 547
- versions, 396
- voices, 57, 326, 385, 461
 - pvxunison, 547
- wave, 57, 326, 461
 - pvxenvelope, 547
- wavelet, 366
- wet, 379, 461
 - pvxrir, 547
 - pvxtrajectoryreverb, 547
- width, 57, 326, 385, 461
 - pvxunison, 547
- win-length, 57, 74, 326, 385, 461
 - pvxanalysis, 547
 - pvxvoc, 547
- window, 326, 348, 385, 461
 - pvxanalysis, 547
 - pvxreshape, 547
 - pvxvoc, 547
- windows, 385
- work-dir, 461
 - pvx, 547
- workers, 326, 366, 371, 379, 461
 - pvx, 547
- Clipping, 550
- Coherent gain, 550
- COLA, 550
- commands
 - hps-pitch-track
 - backend, 547
 - confidence-floor, 547
 - emit, 547
 - feature-set, 547
 - fmax, 547
 - fmin, 547

- frame-length, 547
- hop-size, 547
- mfcc-count, 547
- output, 547
- ratio-max, 547
- ratio-min, 547
- ratio-reference, 547
- reference-hz, 547
- smooth-frames, 547
- stretch, 547
- stretch-from, 547
- stretch-max, 547
- stretch-min, 547
- stretch-scale, 547
- pvx, 43, 57, 61, 64, 66, 74, 284, 286, 290, 326, 336, 337, 348, 354, 366, 371, 379, 385, 396, 397, 461, 481, 484, 494, 523, 545
- append-manifest, 547
- audit-metrics, 547
- backend, 547
- batch-size, 547
- bit-depth, 547
- budget-path, 547
- chunk-seconds, 547
- confidence-floor, 547
- context-ms, 547
- crossfade-ms, 547
- dedupe-by, 547
- device, 547
- disk-budget, 547
- dry-run, 547
- duration, 547
- emit, 547
- engine, 547
- example, 547
- fail-if-exceeds, 547
- feature-set, 547
- fmax, 547
- fmin, 547
- frame-length, 547
- group-separator, 547
- grouping, 547
- hop-size, 547
- intent, 547
- json, 547
- keep-intermediate, 547
- label-policy, 547
- labels-csv, 547
- manifest-csv, 547
- manifest-jsonl, 547
- material, 547
- max-length-s, 547
- mfcc-count, 547
- mode, 547
- no-audit-metrics, 547
- no-progress, 547
- out, 547
- output, 547
- output-csv, 547
- output-dir, 547
- output-format, 547
- output-jsonl, 547
- output-subtype, 547
- output-suffix, 547
- overwrite, 547
- pair-mode, 547
- pipeline, 547
- pipeline-config, 547
- pitch, 547
- pitch-conf-min, 547
- pitch-lowconf-mode, 547
- pitch-map-crossfade-ms, 547
- pitch-map-smooth-ms, 547
- policy, 547
- quiet, 547
- ratio-max, 547
- ratio-min, 547
- ratio-reference, 547
- reference-hz, 547
- requested-stretch, 547
- resume, 547
- route, 547
- safety-margin, 547
- sample-rate, 547
- seed, 547
- silent, 547
- smooth-frames, 547
- split, 547
- split-mode, 547
- sr, 547
- stretch, 547
- stretch-from, 547
- stretch-max, 547
- stretch-min, 547
- stretch-scale, 547
- strict, 547
- strict-manifest, 547
- subtype, 547
- variants-per-input, 547
- work-dir, 547
- workers, 547
- pvx-cli, 366, 461

-
- pvxalgorithms, 64, 397
 - pvxan, 326, 348, 461
 - pvxanalysis, 64, 290, 396, 397, 461
 - analysis-channel, 547
 - cuda-device, 547
 - device, 547
 - hop-size, 547
 - json, 547
 - kaiser-beta, 547
 - n-fft, 547
 - no-center, 547
 - output, 547
 - summary-json, 547
 - transform, 547
 - win-length, 547
 - window, 547
 - pvxbandamp, 64, 494
 - pvxchordmapper, 64, 494
 - pvxcodec, 64, 379, 461
 - bit-depth, 547
 - bits, 547
 - codec, 547
 - output, 547
 - seed, 547
 - pvxconform, 64, 66, 326, 461
 - crossfade-ms, 547
 - map, 547
 - resample-mode, 547
 - pvxdenoise, 64, 66, 461
 - floor, 547
 - noise-file, 547
 - noise-seconds, 547
 - reduction-db, 547
 - smooth, 547
 - pvxdeverb, 64, 66, 461
 - decay, 547
 - floor, 547
 - strength, 547
 - pvxenvelope, 64, 290, 461
 - amplitude, 547
 - attack-sec, 547
 - center, 547
 - cycles, 547
 - decay-sec, 547
 - duration, 547
 - duty-cycle, 547
 - end, 547
 - exp-curve, 547
 - format, 547
 - freq, 547
 - frequency-hz, 547
 - key, 547
 - max, 547
 - min, 547
 - mode, 547
 - out, 547
 - output, 547
 - peak, 547
 - phase, 547
 - phase-rad, 547
 - rate, 547
 - release-sec, 547
 - sine-cycles, 547
 - sine-phase-rad, 547
 - start, 547
 - stdout, 547
 - sustain, 547
 - wave, 547
 - pvxfilter, 64, 290, 396, 397, 461, 494
 - band-gain-db, 547
 - band-width-bins, 547
 - comp-ratio, 547
 - comp-threshold-db, 547
 - dry-mix, 547
 - expand-ratio, 547
 - noise-floor, 547
 - operator, 547
 - peak-count, 547
 - response, 547
 - response-gain-db, 547
 - response-mix, 547
 - shift-bins, 547
 - transpose-semitones, 547
 - tv-interp, 547
 - tv-key, 547
 - tv-map, 547
 - tv-order, 547
 - pvxformant, 64, 396, 397, 461
 - formant-lifter, 547
 - formant-max-gain-db, 547
 - formant-shift-ratio, 547
 - mode, 547
 - pitch-shift-cents, 547
 - pitch-shift-semitones, 547
 - resample-mode, 547
 - pvxfreeze, 64, 66, 396, 397, 461
 - duration, 547
 - freeze-time, 547
 - phase-mode, 547
 - random-phase, 547
 - pvxgain, 64, 461
 - bit-depth, 547

- gain, 547
 - normalize, 547
 - output, 547
 - seed, 547
 - target, 547
- pvxharmmap, 64, 290, 461, 494
 - attenuation, 547
 - boost-db, 547
 - chord, 547
 - dry-mix, 547
 - inharmonic-f0-hz, 547
 - inharmonic-mix, 547
 - inharmonic-ity, 547
 - operator, 547
 - root-hz, 547
 - strength, 547
 - tolerance-cents, 547
- pvxharmonize, 64, 66, 461
 - force-stereo, 547
 - gains, 547
 - intervals, 547
 - intervals-cents, 547
 - pans, 547
 - resample-mode, 547
- pvxinharmonator, 64, 494
- pvxlayer, 64, 461
 - harmonic-gain, 547
 - harmonic-kernel, 547
 - harmonic-pitch-cents, 547
 - harmonic-pitch-semitones, 547
 - harmonic-stretch, 547
 - percussive-gain, 547
 - percussive-kernel, 547
 - percussive-pitch-cents, 547
 - percussive-pitch-semitones, 547
 - percussive-stretch, 547
 - resample-mode, 547
- pvxmorph, 64, 66, 461
 - alpha, 547
 - blend-mode, 547
 - envelope-lifter, 547
 - interp, 547
 - mask-exponent, 547
 - normalize-energy, 547
 - order, 547
 - output, 547
 - output-format, 547
 - overwrite, 547
 - phase-mix, 547
 - stdout, 547
- pvxnoise, 64, 379, 461
 - background-dir, 547
 - band, 547
 - bit-depth, 547
 - noise-type, 547
 - output, 547
 - seed, 547
 - snr, 547
- pvxnoisefilter, 64, 494
- pvxreshape, 64, 290, 461
 - exponent, 547
 - factor, 547
 - format, 547
 - input-format, 547
 - interp, 547
 - key, 547
 - max, 547
 - min, 547
 - offset, 547
 - operation, 547
 - order, 547
 - out, 547
 - output, 547
 - output-key, 547
 - rate, 547
 - stdout, 547
 - target-max, 547
 - target-min, 547
 - window, 547
- pvxresponse, 64, 290, 461
 - json, 547
 - method, 547
 - normalize, 547
 - output, 547
 - phase-mode, 547
 - smoothing-bins, 547
 - summary-json, 547
- pvxretune, 64, 66, 326, 396, 397, 461
 - a4-reference-hz, 547
 - chunk-ms, 547
 - f0-max, 547
 - f0-min, 547
 - overlap-ms, 547
 - recommend-root, 547
 - resample-mode, 547
 - root, 547
 - root-hz, 547
 - scale, 547
 - scale-cents, 547
 - strength, 547
- pvxrf, 326, 348, 461
- pvxring, 64, 290, 461, 494

- depth, 547
- feedback, 547
- frequency-hz, 547
- mix, 547
- operator, 547
- resonance-decay, 547
- resonance-hz, 547
- resonance-mix, 547
- resonance-q, 547
- tv-interp, 547
- tv-map, 547
- tv-order, 547
- pvxringfilter, 64, 494
- pvxringtvfilter, 64, 494
- pvxrir, 64, 379, 461
 - bit-depth, 547
 - cache-dir, 547
 - category, 547
 - drr, 547
 - ir-database, 547
 - ir-dir, 547
 - ir-file, 547
 - list-databases, 547
 - no-trim, 547
 - output, 547
 - pre-delay-ms, 547
 - rt60, 547
 - seed, 547
 - wet, 547
- pvxspecaugment, 64, 461
 - bit-depth, 547
 - fill, 547
 - freq-mask, 547
 - hop-length, 547
 - n-fft, 547
 - num-freq-masks, 547
 - num-time-masks, 547
 - output, 547
 - seed, 547
 - time-mask, 547
- pvxspeccompander, 64, 494
- pvxtrajectoryreverb, 64, 461
 - coord-system, 547
 - distance-law, 547
 - dry, 547
 - end, 547
 - ir, 547
 - no-normalize-gains, 547
 - normalize-gains, 547
 - speaker-angles, 547
 - start, 547
 - trajectory-shape, 547
 - wet, 547
- pvxtransient, 64, 461
 - pitch-shift-cents, 547
 - pitch-shift-ratio, 547
 - pitch-shift-semitones, 547
 - resample-mode, 547
 - target-duration, 547
 - time-stretch, 547
 - transient-threshold, 547
- pvxtvfilter, 64, 494
- pvxunison, 64, 461
 - detune-cents, 547
 - dry-mix, 547
 - resample-mode, 547
 - voices, 547
 - width, 547
- pvxvoc, 64, 66, 366, 379, 396, 397, 461, 481
 - ambient-phase-mix, 547
 - ambient-preset, 547
 - analysis-channel, 547
 - auto-profile, 547
 - auto-profile-lookahead-seconds, 547
 - auto-segment-seconds, 547
 - auto-transform, 547
 - bit-depth, 547
 - cents, 547
 - checkpoint-dir, 547
 - checkpoint-id, 547
 - clip, 547
 - coherence-strength, 547
 - compander-attack-ms, 547
 - compander-compress-ratio, 547
 - compander-expand-ratio, 547
 - compander-makeup-db, 547
 - compander-release-ms, 547
 - compander-threshold-db, 547
 - compressor-attack-ms, 547
 - compressor-makeup-db, 547
 - compressor-ratio, 547
 - compressor-release-ms, 547
 - compressor-threshold-db, 547
 - control-stdin, 547
 - cpu, 547
 - cuda-device, 547
 - device, 547
 - dither, 547
 - dither-seed, 547
 - dry-run, 547
 - example, 547
 - expander-attack-ms, 547

- expander-ratio, 547
- expander-release-ms, 547
- expander-threshold-db, 547
- explain-plan, 547
- extreme-stretch-threshold, 547
- extreme-time-stretch, 547
- f0-max, 547
- f0-min, 547
- force, 547
- formant-lifter, 547
- formant-max-gain-db, 547
- formant-strength, 547
- fourier-sync, 547
- fourier-sync-max-fft, 547
- fourier-sync-min-fft, 547
- fourier-sync-smooth, 547
- gpu, 547
- guided, 547
- hard-clip-level, 547
- hop-size, 547
- interp, 547
- kaiser-beta, 547
- limiter-threshold, 547
- manifest-append, 547
- manifest-json, 547
- max-stage-stretch, 547
- metadata-policy, 547
- multires-ffts, 547
- multires-fusion, 547
- multires-weights, 547
- n-fft, 547
- no-center, 547
- no-onset-realign, 547
- no-progress, 547
- normalize, 547
- onset-credit-max, 547
- onset-credit-pull, 547
- onset-time-credit, 547
- order, 547
- out, 547
- output, 547
- output-dir, 547
- output-format, 547
- overwrite, 547
- peak-dbfs, 547
- phase-engine, 547
- phase-locking, 547
- phase-random-seed, 547
- pitch, 547
- pitch-conf-min, 547
- pitch-follow-stdin, 547
- pitch-lowconf-mode, 547
- pitch-map, 547
- pitch-map-crossfade-ms, 547
- pitch-map-smooth-ms, 547
- pitch-map-stdin, 547
- pitch-mode, 547
- pitch-shift-cents, 547
- pitch-shift-ratio, 547
- pitch-shift-semitones, 547
- preset, 547
- quality-profile, 547
- quiet, 547
- ratio, 547
- ref-channel, 547
- resample-mode, 547
- resume, 547
- rms-dbfs, 547
- route, 547
- semitones, 547
- silent, 547
- soft-clip-drive, 547
- soft-clip-level, 547
- soft-clip-type, 547
- stdout, 547
- stereo-mode, 547
- stretch, 547
- stretch-mode, 547
- subtype, 547
- suffix, 547
- target-duration, 547
- target-f0, 547
- target-lufs, 547
- target-pitch-shift-semitones, 547
- target-sample-rate, 547
- time-stretch, 547
- time-stretch-factor, 547
- transform, 547
- transient-crossfade-ms, 547
- transient-mode, 547
- transient-preserve, 547
- transient-protect-ms, 547
- transient-sensitivity, 547
- transient-threshold, 547
- true-peak-max-dbtp, 547
- verbose, 547
- verbosity, 547
- win-length, 547
- window, 547
- pvxwarp, 64, 66, 326, 396, 397, 461
 - crossfade-ms, 547
 - map, 547

- resample-mode, 547
- Compact Feature Set (Smaller CSV), 335
- Comparative listening worksheet, 523
- Comparative Window Summary, 80
- Compare phase locking modes, 352
- Compatibility Shims, 64
- Complete Window Atlas, 84
- Complex number, 550
- Complex spectrum, 550
- Composable command-line tools, 285
- composers
 - Alvin Lucier, 521
 - Anonymous, 500
 - Anton Bruckner, 505
 - Antonio Vivaldi, 504
 - Aphex Twin, 510
 - Arnold Schoenberg, 502
 - Arvo Part, 506
 - Autechre, 511
 - Bela Bartok, 514
 - Benjamin Britten, 514
 - Bjork, 518
 - Boards of Canada, 518
 - Brian Eno, 517
 - Charles Ives, 513
 - Claude Debussy, 505, 513
 - Claudio Monteverdi, 501
 - Cocteau Twins, 517
 - Dave Brubeck Quartet, 509
 - Delia Derbyshire, 516
 - Edgard Varese, 509
 - Franz Schubert, 502, 504
 - Frederic Chopin, 505, 508
 - George Frideric Handel, 501
 - Giovanni Pierluigi da Palestrina, 512
 - Gustav Holst, 506
 - Gustav Mahler, 502, 513
 - Gyorgy Ligeti, 506, 514
 - Hector Berlioz, 513
 - Henry Purcell, 501
 - Hildegard von Bingen, 500
 - Iannis Xenakis, 514
 - Igor Stravinsky, 509, 514
 - James Brown, 510
 - Jean-Claude Risset, 499
 - JoAnn Kuchera-Morin, 496
 - Johann Pachelbel, 520
 - Johann Sebastian Bach, 501, 504, 508, 512
 - Johannes Brahms, 513
 - John Adams, 515
 - John Cage, 509, 521
 - John Tavener, 507
 - Jonathan Harvey, 496
 - Josquin des Prez, 500
 - Kaija Saariaho, 499
 - Karlheinz Stockhausen, 516
 - Kate Bush, 517
 - Kraftwerk, 510
 - Laurie Anderson, 517
 - Luciano Berio, 502
 - Ludwig van Beethoven, 504, 508, 512, 520
 - Maurice Ravel, 505, 520
 - Max Richter, 522
 - My Bloody Valentine, 518
 - Niccolo Paganini, 508
 - Nikolai Rimsky-Korsakov, 509
 - Olivier Messiaen, 506
 - Oneohtrix Point Never, 519
 - Paul Lansky, 499
 - Pink Floyd, 517
 - Public Enemy, 510
 - Radiohead, 518
 - Ralph Vaughan Williams, 506
 - Richard Wagner, 502, 505, 520
 - Roger Reynolds, 496
 - Samuel Barber, 521
 - Squarepusher, 510
 - Steve Reich, 503, 521
 - The Beatles, 516
 - The Orb, 518
 - Thomas Tallis, 500
 - Trevor Wishart, 496–498
 - Tristan Murail, 498
 - Wendy Carlos, 498
 - William Basinski, 521
 - Wolfgang Amadeus Mozart, 501, 512
- Compositional histories beyond the canonical examples, 29
- Computer music and the widening of purpose, 8
- confidence - subtle pitch bend depth, 328
- Confidence-Gated External Pitch Control via Pipe, 312
- Conform CSV with per-segment ratios, 382
- Conjugate symmetry, 550
- Conservative Field-Restoration Chain, 314
- Contents, 355, 373
- Contrastive Paired Views for Self-Supervised Learning, 325
- Contrastive Planning-Only Pass, 369
- Contrastive view generation (SSL), 358
- Control-Map Authoring With Envelope + Reshape, 343

- Control-Rate Interpolation Graph Examples, 478
- Convolution, 550
- Core Command, 367
- cosine, 273
- cosine power 2, 229
- cosine power 3, 233
- cosine power 4, 237
- Create Chorus/Unison Thickness, 297
- creative spectral effects, 465
- Cross-synthesis, 550
- Crossfade, 550
- cubic, 542
- CUDA Attempt with Deterministic CPU Fallback Workflow, 311
- Custom cents map retune, 382
- CZT for Non-Power-of-Two Frame Setup, 306

- Data Product Map, 480
- dBFS, 550
- DCT timbral compaction for smooth harmonic material, 385
- De-Noise Field Recording, 297
- De-Reverb a Room Recording, 298
- Debug Workflow, 61
- Decay, 550
- Default production backend (FFT + transient protection), 384
- denoise and restoration, 466
- Denoise without musical-noise artifacts, 60
- Denoising with Phase-Consistent STFT Processing, 50
- dereverb and room correction, 467
- Derive and Inspect a Reusable Response Artifact (PVXRF), 320
- Determinism and Reproducibility, 360
- Deterministic AI Dataset Augmentation (pvx augment), 324
- Deterministic render, 550
- Development Installation, 393
- DFT, 550
- Diagnose Environment and Install Issues (pvx doctor), 323
- Dict-based map for keyed datasets, 360
- Docs Guide, 64
- Documentation Build Graph, 75
- Documentation Build Pipeline, 481
- Documented phase-vocoder practice, 496
- Drum-safe stretch, 60
- Drums Safe Stretch, 302
- Dry signal, 550
- Dry-run output validation, 382
- DST odd-symmetry color pass, 385

- Dudley, Homer, 4
- Dynamic Analysis Resolution (N-FFT + Hop Size Maps), 317
- Dynamic control from external files, 285
- Dynamic Control Interpolation (CSV/JSON), 72
- Dynamic Pitch Ratio via JSON + Polynomial Interpolation, 315
- Dynamic range, 550
- Dynamic Stretch via Direct CSV Flag Input, 315
- dynamics and loudness, 468
- dynamics transfer curves, 530

- Electronic studio and ambient listening, 516
- End-to-End pvxvoc Flow, 477
- Energy normalization, 550
- Entries Still Using Scholar Links, 485
- Envelope, 550
- Envelope follower, 550
- Equalization, 550
- ERB, 551
- Estimate Stretch Budget Before Extreme Jobs, 55
- exact blackman, 113
- Exaggerate movement with power law, 332
- Example Audio Scenarios, 52
- Example Workflows, 369
- Experimental in 0.1.x, 64
- Explicit defaults and transparent controls, 285
- exponential, 542
- exponential 0p25, 205
- exponential 0p5, 209
- exponential 1p0, 213
- Exponential interpolation, 551
- Extra Beginner Recipes (Quick Wins), 56
- Extreme Ambient Stretch (10-minute target), 303
- Extreme duration and temporal form, 520
- Extreme duration and the change of listening scale, 36
- Extreme Stretch Multistage Strategy, 479
- Extreme Stretch with Checkpointed Recovery, 310
- Extreme stretch with multistage strategy, 383
- Extreme Time Stretch Ambient Pad, 295
- Extreme Time-Scale Madness, 339

- f0 hz - pitch ratio, 328
- Fast End-to-End Health Check (pvx smoke), 324
- Fast Starting Presets, 59
- Feature Tracking for Sidechain Control, 55
- Feature-Vector Recipes (MFCC + MPEG-7), 332
- FFT, 551
- FFT size, 5, 551
- File subtype, 551
- Filter, 551

-
- First 3 Commands to Run, 48
 - Flanagan and Golden in the Bell Labs setting, 16
 - Flanagan and Golden phase as transmitted information, 7
 - Flanagan's digital formulation, 5
 - Flanagan, James L., 5
 - flattop, 105
 - Floor, 551
 - Formant, 551
 - Formant + onset, 330
 - Formant 2 + spectral spread, 331
 - Formant preservation, 551
 - Formula Families, 77
 - Formula Version Policy, 396
 - Frame, 551
 - Frame rate, 551
 - Freeze + Morph Experiments, 340
 - Freeze a Harmonic Moment, 295
 - Frequency resolution, 551
 - From offline render to interactive instrument, 24
 - From research technique to musical tool, 7
 - Full Audio Container List (Current Build), 65
 - Full Mastering Chain in One Render, 307
 - Function Graph Gallery, 73, 479
 - Fundamental frequency, 551

 - Gain, 551
 - gaussian 0p25, 169
 - gaussian 0p35, 173
 - gaussian 0p45, 177
 - gaussian 0p55, 181
 - gaussian 0p65, 185
 - general gaussian 1p5 0p35, 189
 - general gaussian 2p0 0p35, 193
 - general gaussian 3p0 0p35, 197
 - general gaussian 4p0 0p35, 201
 - general hamming 0p50, 253
 - general hamming 0p60, 257
 - general hamming 0p70, 261
 - general hamming 0p80, 265
 - Generate a Stretch Envelope (Function Stream), 318
 - glossary, 549
 - GPU Acceleration, 299
 - GPU-Accelerated Transforms (PyTorch), 363
 - granular and modulation, 469
 - graphs
 - dynamics transfer curves, 530
 - mask exponent response, 536
 - morph blend magnitude curves, 534
 - phase-mix angle response, 538
 - pitch ratio from cents, 528
 - pitch ratio from semitones, 526
 - soft-clip transfer functions, 532
 - Griffin-Lim algorithm, 551
 - Group delay, 551
 - Group-Level Summary, 463
 - Guide A controls pitch guide B controls stretch, 334

 - hamming, 89
 - hann, 85
 - hann poisson 0p5, 241
 - hann poisson 1p0, 245
 - hann poisson 2p0, 249
 - Harmonic, 551
 - Harmonic analysis before electronic sound, 13
 - Harmonic Chord Mapping with pvx chordmapper, 322
 - harmonic ratio - harmonicity-linked pitch, 329
 - Harmonic-percussive separation, 551
 - Harmonize Then Widen in One Pipeline, 309
 - Harmonize Triad Stack, 301
 - Hartley real-basis exploratory render, 385
 - Hertz, 551
 - Homer Dudley the vocoder and the Voder, 14
 - Hop ratio, 551
 - Hop size, 551
 - Horizontal coherence and vertical coherence, 9
 - Host, 389
 - How pvx puts the idea to work, 43
 - How to use the library, 495
 - hps-pitch-track, 400
 - HuggingFace Datasets Integration, 358
 - Hybrid Transient Engine, 477
 - Hybrid Transient Mode (Manual Tuning), 303
 - Hybrid transient mode (recommended on speech/-drums), 54

 - I/O Behavior by Category, 65
 - Identity phase locking, 552
 - Immediate Next Steps, 289
 - Implemented in Phase 1 and Phase 2, 288
 - Implemented in Phase 3 Phase 4 and Phase 5, 288
 - Implemented in Phase 6 and Phase 7, 289
 - Import Paths, 351
 - Impulse Response Database, 362
 - Increase MFCC dimensionality to 20, 333
 - Independent cents retune, 383
 - Inharmonic Spectral Warping with pvx inharmonator, 323
 - inharmonicity - anti-harmonic pitch detune, 329
 - Inspect Feature Columns Before Routing, 327
 - Installation, 355

- Instantaneous frequency, 552
- Integrated loudness targeting with limiter, 382
- Intent Preset Pipelines, 356
- Intent Presets, 55
- Intent Profiles, 367
- Interpolation, 552
- interpolation
 - cubic, 542
 - exponential, 542
 - linear, 541
 - nearest neighbour, 541
 - polynomial order 1, 544
 - polynomial order 2, 544
 - polynomial order 3, 545
 - polynomial order 5, 545
 - sample and hold, 540
 - smotherstep, 543
 - smoothstep S-curve, 543
- Interpretation and use, 88, 92, 96, 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144, 148, 152, 156, 160, 164, 168, 172, 176, 180, 184, 188, 192, 196, 200, 204, 208, 212, 216, 220, 224, 228, 232, 236, 240, 244, 248, 252, 256, 260, 264, 268, 272, 276, 280, 284
- Inverse FFT, 552
- Invert and clamp, 332
- IRCAM and the institutionalization of spectral transformation, 19
- IRCAM as a meeting of research and commission, 34
- Irrational Pitch Ratio Input, 305
- James A. Moorer and the computer-music turn, 17
- Jean Laroche and Mark Dolson explaining phasiness, 10
- JoAnn Kuchera-Morin and expanded duration, 20
- Jonathan Harvey and spectral ambiguity, 19
- JSON Manifest + Automation-Friendly Logging, 302
- Jupyter Notebook Snippets, 352
- Just-Intonation CSV Conform Map, 306
- Just-Ratio Pitch Input, 305
- kaiser, 277
- Kaiser beta, 552
- Keyframe, 552
- Keyword Spotting / Wake-Word Detection, 374
- lanczos, 161
- Laroche and Dolson from artifact to explanatory model, 21
- Latency, 552
- Launch-Ready Helper Commands, 47
- Layer Harmonic and Percussive Paths, 301
- Leakage, 552
- LFO, 552
- linear, 541
- Linear phase, 552
- Link-Type Summary, 485
- listening guide, 495
 - Alvin Lucier
 - I Am Sitting in a Room, 521
 - Anonymous
 - Dies irae, 500
 - Anton Bruckner
 - Symphony No. 7 Adagio, 505
 - Antonio Vivaldi
 - Winter Largo, 504
 - Aphex Twin
 - Vordhosbn, 510
 - Arnold Schoenberg
 - Mondestrunken, 502
 - Arvo Part
 - Spiegel im Spiegel, 506
 - Autechre
 - Gantz Graf, 511
 - Bela Bartok
 - Music for Strings Percussion and Celesta third movement, 514
 - Benjamin Britten
 - The Young Person s Guide to the Orchestra, 514
 - Bjork
 - All Is Full of Love, 518
 - Boards of Canada
 - Roygbiv, 518
 - Brian Eno
 - An Ending (Ascent), 517
 - Charles Ives
 - The Unanswered Question, 513
 - Claude Debussy
 - La mer Dialogue du vent et de la mer, 513
 - Prelude a l apres-midi d un faune, 505
 - Claudio Monteverdi
 - Lamento d Arianna, 501
 - Cocteau Twins
 - Lorelei, 517
 - Dave Brubeck Quartet
 - Take Five, 509
 - Delia Derbyshire
 - Doctor Who Theme, 516
 - Edgard Varese

- Ionisation, 509
- Franz Schubert
 - Erlkonig, 502
 - String Quintet in C Adagio, 504
- Frederic Chopin
 - Etude Op. 10 No. 4, 508
 - Nocturne in E-flat major Op. 9 No. 2, 505
- George Frideric Handel
 - Lascia ch io pianga, 501
- Giovanni Pierluigi da Palestrina
 - Missa Papae Marcelli Kyrie, 512
- Gustav Holst
 - Neptune the Mystic, 506
- Gustav Mahler
 - Ich bin der Welt abhanden gekommen, 502
 - Symphony No. 2 finale, 513
- Gyorgy Ligeti
 - Atmospheres, 514
 - Lux Aeterna, 506
- Hector Berlioz
 - Symphonie fantastique Marche au supplice, 513
- Henry Purcell
 - Dido's Lament, 501
- Hildegard von Bingen
 - O vis aeternitatis, 500
- Iannis Xenakis
 - Metastaseis, 514
- Igor Stravinsky
 - The Firebird Infernal Dance, 514
 - The Rite of Spring Augurs of Spring, 509
- James Brown
 - Funky Drummer, 510
- Jean-Claude Risset
 - Inharmonique, 499
 - Songes, 499
- JoAnn Kuchera-Morin
 - Dreampaths, 496
- Johann Pachelbel
 - Canon in D, 520
- Johann Sebastian Bach
 - Air on the G String, 504
 - Erbarme dich, 501
 - Prelude in C major BWV 846, 508
 - The Art of Fugue Contrapunctus I, 512
- Johannes Brahms
 - Symphony No. 4 finale, 513
- John Adams
 - Short Ride in a Fast Machine, 515
- John Cage
 - First Construction in Metal, 509
- Organ2/ASLSP, 521
- John Tavener
 - The Protecting Veil, 507
- Jonathan Harvey
 - Mortuos Plango Vivos Voco, 496
- Josquin des Prez
 - Ave Maria virgo serena, 500
- Kaija Saariaho
 - Lichtbogen, 499
 - NoaNoa, 499
- Karlheinz Stockhausen
 - Gesang der Junglinge, 516
 - Kontakte, 516
- Kate Bush
 - Running Up That Hill, 517
- Kraftwerk
 - Numbers, 510
- Laurie Anderson
 - O Superman, 517
- Luciano Berio
 - Sequenza III, 502
- Ludwig van Beethoven
 - Piano Sonata No. 14 third movement, 508
 - Symphony No. 5 opening, 520
 - Symphony No. 7 Allegretto, 504
 - Symphony No. 9 finale, 512
- Maurice Ravel
 - Bolero, 520
 - Daphnis et Chloe Lever du jour, 505
- Max Richter
 - Sleep, 522
- My Bloody Valentine
 - Only Shallow, 518
- Niccolo Paganini
 - Caprice No. 24, 508
- Nikolai Rimsky-Korsakov
 - Flight of the Bumblebee, 509
- Olivier Messiaen
 - Louange à l'Eternite de Jesus, 506
- Oneohtrix Point Never
 - Replica, 519
- Paul Lansky
 - Six Fantasies on a Poem by Thomas Campion, 499
- Pink Floyd
 - On the Run, 517
- Public Enemy
 - Bring the Noise, 510
- Radiohead
 - Everything in Its Right Place, 518
- Ralph Vaughan Williams

- Fantasia on a Theme by Thomas Tallis, 506
- Richard Wagner
 - Mild und leise, 502
 - Prelude to Das Rheingold, 505
 - Prelude to Tristan und Isolde, 520
- Roger Reynolds
 - Transfigured Wind I, 496
- Samuel Barber
 - Adagio for Strings, 521
- Squarepusher
 - My Red Hot Car, 510
- Steve Reich
 - Different Trains, 503
 - Piano Phase, 521
- The Beatles
 - Tomorrow Never Knows, 516
- The Orb
 - Little Fluffy Clouds, 518
- Thomas Tallis
 - Spem in alium, 500
- Trevor Wishart
 - Red Bird, 498
 - Supernova, 497
 - Tongues of Fire, 498
 - Vox 5, 496
 - Vox Cycle, 497
- Tristan Murail
 - Desintegrations, 498
- Wendy Carlos
 - Digital Moonscapes, 498
- William Basinski
 - The Disintegration Loops 1.1, 521
- Wolfgang Amadeus Mozart
 - Der Holle Rache, 501
 - Symphony No. 40 first movement, 512
- Loudness, 552
- Loudness and Dynamics, 73
- Loudness proxy + transient mask, 331
- Lower Pitch With Formant Preservation, 295
- Machine learning enters the surrounding workflow, 24
- Magnitude, 552
- Magnitude spectrum, 552
- Main lobe, 552
- Maintainer Release Flow, 396
- Managed One-Line Chain (No Manual Pipe Wiring), 314
- Manifest Fields, 368
- Manifest merge + stats, 370
- Manifest Validation and Merge, 326
- Manual Pipe Variants, 335
- Mark Dolson and the tutorial synthesis, 9
- Mark Dolson research software and pedagogy, 18
- Mask, 552
- mask exponent response, 536
- Masking, 552
- Mastering, 382
- Mastering Chain Block Diagram, 479
- Metadata, 552
- Metrics Printing Path, 479
- MFCC + MPEG-7 audio spectrum envelope band, 332
- MFCC + MPEG-7 spectral flux, 330
- MFCC second coefficient driver, 332
- Microtonal, 382
- Microtonal CSV Interpolation Flow, 481
- Microtonal Just-Ratio Map Playback, 312
- Microtonal Map Processing, 478
- Microtonal Retune Mapping, 72
- Mid-side, 552
- Mid/Side Lock with Formant-Preserving Pitch Shift, 312
- Mid/Side Stereo Coherence Stretch, 304
- Minimal Pitch-Shift (Duration-Preserving), 351
- Minimal Public-Demo Sequence (pvx quickstart), 323
- Minimal Time-Stretch Example, 351
- Mix online and offline augmentation, 363
- Mixing GPU and NumPy transforms, 364
- Moderate vocal stretch with formant preservation, 382
- Module Layout, 493
- Mono, 552
- Morph - formant - unison, 384
- morph blend magnitude curves, 534
- Morph Then Freeze for Hybrid Pads, 311
- Morph Two Sounds, 296
- MPEG-7 attack-time + MFCC driver, 331
- MPEG-7 centroid + MPEG-7 spread, 331
- MPEG-7 spectral crest and decrease, 333
- Multi-Feature Recipes (Different Features - Different Targets), 330
- multi-resolution analysis, 7
- Multi-Resolution Fusion Fan-In, 480
- Multi-Resolution Fusion Render, 300
- Multi-resolution fusion stretch, 383
- Multi-Resolution Fusion With Explicit Weights, 313
- Multi-Stage Pipelines Streaming and Batch Tricks, 346
- Multi-Tool Pipe with Streaming I/O, 308
- Multichannel, 552

-
- Multiple Guide Files (A and B both influence target), 333
 - Multiresolution analysis, 552
 - Music Classification and Tagging, 376
 - Music Information Retrieval Set, 369
 - Music Source Separation, 379
 - N-TET CSV Map (19-TET) with pvxconform, 306
 - nearest neighbour, 541
 - New Quality Modes (Beginner Version), 54
 - Next Steps, 56
 - Noise ambience and environmental recordings, 36
 - Noise floor, 553
 - Noise residuals and the limits of sinusoidal phase, 23
 - Noise-aware hiss + hum, 331
 - Nonstationary, 553
 - Normalized frequency, 553
 - Notation, 69, 77
 - Notes, 385
 - nuttall, 101
 - Nyquist frequency, 553
 - Offline pre-computation (recommended for large datasets), 360
 - Old vs New, 337
 - One-liner convenience, 359
 - Online augmentation with worker processes, 360
 - Onset, 553
 - onset strength - attack-reactive stretch, 329
 - Open tools standards and software migration, 23
 - option values
 - 32f, 547
 - 16, 547
 - 24, 547
 - 32, 547
 - acf, 547
 - advanced, 547
 - aiff, 547
 - all, 547
 - allow alter, 547
 - am radio, 547
 - ambient, 547
 - arctan, 547
 - asr robust, 547
 - auto, 547
 - bandlimited, 547
 - basic, 547
 - bin, 547
 - brown, 547
 - caf, 547
 - cartesian, 547
 - contrastive2, 547
 - cpu, 547
 - csv, 547
 - cubic, 547
 - cuda, 547
 - drums, 547
 - ease-in, 547
 - ease-in-out, 547
 - ease-out, 547
 - f0 hz, 547
 - fft, 547
 - first, 547
 - flac, 547
 - FLOAT, 547
 - float, 547
 - formant-preserving, 547
 - gaussian, 547
 - hold, 547
 - hybrid, 547
 - hz, 547
 - identity, 547
 - independent, 547
 - inherit, 547
 - instantaneous, 547
 - interp, 547
 - inv pitch ratio, 547
 - inverse, 547
 - inverse-square, 547
 - json, 547
 - label balanced, 547
 - linear, 547
 - lo fi, 547
 - mean, 547
 - median, 547
 - mid side lock, 547
 - mir music, 547
 - mix, 547
 - mp3 low, 547
 - mp3 medium, 547
 - mps, 547
 - multistage, 547
 - none, 547
 - off, 547
 - ogg, 547
 - PCM 16, 547
 - PCM 24, 547
 - PCM 32, 547
 - peak, 547
 - pink, 547
 - pitch map, 547
 - pitch ratio, 547

- pitch to stretch, 547
- preserve, 547
- pvx-cli, 547
- pyin, 547
- pytorch, 547
- random, 547
- ref channel lock, 547
- reset, 547
- rms, 547
- shift, 547
- speaker balanced, 547
- speech, 547
- spherical, 547
- ssl contrastive, 547
- standard, 547
- stateful, 547
- stem-prefix, 547
- stretch map, 547
- tanh, 547
- telephone, 547
- torchaudio, 547
- unity, 547
- vocal, 547
- voip narrow, 547
- voip wide, 547
- wav, 547
- wavelet, 547
- white, 547
- wrapper, 547
- wsola, 547
- Output Policy Controls, 56
- Overlap, 553
- Overlap-add, 553
- Oversampling, 553

- Parameter Ranges That Usually Work, 60
- Partial, 553
- parzen, 157
- Peak bin, 553
- Peak locking, 553
- Per-Signal Results, 390
- Percussion and the historical challenge of the onset, 36
- Performance Tips, 360
- Persist STFT Analysis Artifact (PVXAN), 319
- Phase, 553
- Phase 1 Coherence-aware phase propagation in core, 483
- Phase 2 Adaptive coherence policy, 483
- Phase 3 Design, 493
- Phase 3 Stereo/multichannel coherence extension, 484
- Phase 4 Benchmark and regression gates, 484
- Phase 4 Design, 494
- Phase 5 Design, 494
- Phase 5 UX and docs, 484
- Phase coherence, 553
- Phase increment, 553
- phase locking, 7
- Phase propagation, 553
- Phase Propagation and Locking, 477
- Phase reset, 553
- Phase Reset vs Hybrid Boundary Behavior, 480
- Phase unwrapping, 553
- Phase vocoder, 553
- phase vocoder, 3–43
- phase-mix angle response, 538
- Phase-vocoder core, 382
- Phase-Vocoder Frequency and Phase Update, 71
- Phased implementation, 483
- Phasiness, 553
- phasiness, 3
- Pipe Chaining Pattern, 479
- Pipe Multiple Tools in One Line, 299
- Pipelines, 383
- pitch detection and tracking, 470
- Pitch Estimation and Melody Extraction, 377
- Pitch Harmony and Tuning Extremes, 341
- Pitch ratio, 553
- pitch ratio - stretch (inverse motion), 328
- pitch ratio from cents, 528
- pitch ratio from semitones, 526
- Pitch Shift with Formant Preservation, 303
- Pitch shifting, 553
- pitch shifting, 3
- Pitch shifting harmonising and spectral translation, 11
- pitch stability - micro-variation amount, 330
- Pitch-follow sidechain map (A controls B), 384
- Pitch-Map Confidence Gate, 480
- Plan-Only Diagnostic (No Render), 307
- Policy File Template, 370
- polynomial order 1, 544
- polynomial order 2, 544
- polynomial order 3, 545
- polynomial order 5, 545
- Polyphony and orchestral density, 512
- Portnoff and the practical digital formulation, 17
- Portnoff the FFT makes the method practical, 8
- Portnoff, Mark, 5
- Practical Recipes, 60
- Pre-echo, 553

-
- Preservation reconstruction and obsolete systems, 37
 - Prime-size frame experiment with CZT backend, 384
 - Processing Contracts, 493
 - Puckette phase locking and real-time environments, 21
 - pvx, 402
 - pvx Window Reference, 77
 - PVXAN, 288
 - pvxanalysis, 413
 - PvxAugmentDataset, 357
 - PvxAugmentTransform (torchvision-style), 357
 - pvxcodec, 414
 - pvxconform, 415
 - pvxdenoise, 416
 - pvxdeverb, 416
 - pvxenvelope, 417
 - pvxfilter, 420
 - pvxformant, 421
 - pvxfreeze, 422
 - pvxgain, 423
 - pvxharmmmap, 424
 - pvxharmonize, 425
 - pvxlayer, 426
 - pvxmorph, 427
 - pvxnoise, 430
 - pvxreshape, 431
 - pvxresponse, 433
 - pvxretune, 434
 - PVXRF, 288, 554
 - pvxring, 435
 - pvxrir, 436
 - pvxspecaugment, 438
 - pvxtrajectoryreverb, 439
 - pvxtransient, 441
 - pvxunison, 442
 - pvxvoc, 443
 - pvxwarp, 461
 - PyTorch Integration, 357
 - QA Stress and Reproducibility Challenges, 347
 - Quality metrics on processed speech, 381
 - Quality regression check, 381
 - Quality-First First Pass (pvx safe), 323
 - Quality-First Optimization Loop, 481
 - Quantitative Metrics, 79
 - Quantization, 554
 - Quick Setup (Install + PATH), 47
 - Quick Start, 327
 - Quick Start Pure Python, 356
 - Raise Pitch Without Changing Speed, 294
 - Reading the foundational papers as historical documents, 25
 - Reading the history through software design, 41
 - Real-time factor, 554
 - Reconstruction, 554
 - rect, 281
 - Reference Fourier baseline using explicit DFT mode, 384
 - Reference locking, 554
 - Reference-Channel Lock for Multichannel Content, 305
 - Related CLI Equivalents, 354
 - Release Checklist, 397
 - Reproduce, 389
 - Reproducibility Controls, 367
 - Reproducible Ambient Session With Manifest Logging, 314
 - Resampling, 554
 - Research Notes, 371
 - Reshape and Densify a Control Map, 318
 - Resonance, 554
 - Resonant Ring Filtering with pvx ringfilter, 322
 - Response-Profile Denoising with pvx noisefilter, 320
 - Response-Shaped Spectral Dynamics with pvx spec-compander, 321
 - Resume + Append for Long Augmentation Runs, 326
 - Resume and append, 369
 - retune and intonation, 470
 - Retune Vocal to A440, 296
 - Reuse a Saved Feature Map for Fast Iteration, 335
 - Ring Modulation Fundamentals with pvx ring, 322
 - Ring Resonator Chordmapper Inharmonator, 345
 - RMS, 554
 - rms norm - stretch, 328
 - Roger Reynolds and the genealogy of Transfigured Wind, 20
 - rolloff norm - pitch, 329
 - Rollout Guidance, 337
 - Room Recording Rehab (De-Reverb Then Stretch), 310
 - Room size, 554
 - Route Operator Patterns (const inv pow mul add affine clip), 332
 - Run PVC Parity Benchmark Gate, 319
 - Running Any Command with uv, 48
 - Running with uv, 371
 - Runtime and CLI Flow, 75
 - Safe Starting Ranges, 335

- Safety Limiter + Hard Clip Guard, 307
- Sample, 554
- sample and hold, 540
- Sample rate, 554
- Sample Transform Use Cases, 71
- Sampling digital audio and the FFT threshold, 16
- Saving augmented datasets, 359
- Scale-Quantized Retune Followed by Unison Widening, 310
- Scalloping loss, 554
- Schema Notes, 288
- Scope, 287
- Script-based workflow reuse, 286
- Secure speech and large-scale vocoder engineering, 15
- Segment Processing Example (Python), 353
- Segment-Based Dynamic Stretch via CSV, 298
- Self-Supervised / Contrastive Learning, 378
- separation and decomposition, 472
- Shell Path, 395
- short-time Fourier transform, *see* STFT
- Side lobe, 554
- Sidechain Follow and Feature-Driven Control, 342
- Sidechain Pitch Trajectory (A Controls B), 298
- Simpler One-Line Pipelines, 55
- sine, 117
- Single-Feature Recipes, 328
- Sinusoid, 554
- Sinusoidal modeling and a neighboring lineage, 18
- Slow Down Speech, 294
- smootherstep, 543
- Smoothing, 554
- smoothstep S-curve, 543
- Soft clip and hard safety ceiling, 382
- soft-clip transfer functions, 532
- Sorted Top-Level Command Roadmap, 289
- source modules
 - hps pitch track.py, 547
 - pvx augment.py, 547
 - pvx helpers.py, 547
 - pvx pipeline.py, 547
 - pvxanalysis.py, 547
 - pvxcodec.py, 547
 - pvxconform.py, 547
 - pvxdenoise.py, 547
 - pvxdeverb.py, 547
 - pvxenvelope.py, 547
 - pvxfilter.py, 547
 - pvxformant.py, 547
 - pvxfreeze.py, 547
 - pvxgain.py, 547
 - pvxharmmap.py, 547
 - pvxharmonize.py, 547
 - pvxlayer.py, 547
 - pvxmorph.py, 547
 - pvxnoise.py, 547
 - pvxreshape.py, 547
 - pvxresponse.py, 547
 - pvxretune.py, 547
 - pvxring.py, 547
 - pvxrir.py, 547
 - pvxspecaugment.py, 547
 - pvxtrajectoryreverb.py, 547
 - pvxtransient.py, 547
 - pvxunison.py, 547
 - pvxwarp.py, 547
 - voc console.py, 547
 - voc parser.py, 547
 - voc.py, 547
- Source notes for the expanded history, 41
- Source paper and cited references to track, 484
- Spatial, 384
- spatial and multichannel, 472
- Spatial Coherence and Channel Alignment, 73
- Speaker Verification / Identification, 375
- Speaker-Balanced Split Assignment for Augmentation, 325
- Spectral centroid, 554
- spectral centroid hz - brightness-linked pitch, 329
- Spectral envelope, 554
- Spectral envelopes formants and the source-filter return, 22
- spectral flatness - noisy/tonal pitch response, 329
- Spectral flux, 554
- spectral flux - stretch, 328
- Spectral freeze, 554
- Spectral leakage, 554
- Spectral Peak Emphasis with pvx bandamp, 321
- spectral time frequency transforms, 475
- Spectral-composition context, 498
- Spectrogram, 554
- Spectrogram visualization snippet, 352
- Speech analysis before digital audio, 4
- Speech as the first difficult material, 35
- Speech Clarity Pass (Identity Lock + Conservative Window/Hop), 313
- Speech Enhancement and Denoising, 375
- Speech Natural Stretch (Hybrid Transients), 302
- Speech Robustness Set, 369
- Speech stretch with transient protection, 60
- Speech vs music probabilities, 331
- Stable in 0.1.x, 63

-
- Stable Installation, 393
 - Stairstep (Sample-and-Hold) Control Mode, 316
 - Stanford CCRMA and model-based sound, 33
 - Start from a feature then scale and bias, 332
 - Starter Path (Run These First), 293
 - Stateful Stream Mode with Dynamic Stretch CSV, 317
 - Step A Baseline stretch, 53
 - Step B Add transient protection, 54
 - Step C Pitch shift with formant preservation, 54
 - Step D Auto profile planning, 54
 - Step-by-Step Walkthrough (One Small WAV), 53
 - Stereo, 555
 - Stereo / Multichannel Coherence Modes, 478
 - Stereo and multichannel coherence, 12
 - Stereo coherence lock, 60
 - Stereo coherence lock (recommended for wide stereo sources), 54
 - Stereo Coherent Stretch, 302
 - Stereo cues interaural level difference + interaural time difference, 330
 - Stereo spatial hearing and multichannel history, 23
 - STFT, 42, 555
 - STFT Analysis / Synthesis Timeline, 477
 - STFT Analysis and Synthesis, 69
 - Streaming, 555
 - Streaming Augmentation for Long-Form Audio, 362
 - Stretch + Preserve Formants, 295
 - Stretch vs Pitch Shift, 49
 - Suggested Ongoing Checklist, 286
 - Supported File Types, 49
 - Supported Surface, 397
 - Sustained instrumental sound and the discovery of coherence, 35
 - Sustained tone and resonance, 504
 - Synthesis frame, 555
 - Synthesis hop, 555
 - synthesis hop, 5
 - Tape music and independent control by segmentation, 15
 - Tape television and the wish to uncouple time from pitch, 4
 - Teaching diagrams and the public life of the algorithm, 32
 - Telegraphy telephony and the filter-bank imagination, 14
 - Tempo + centroid, 330
 - Temporal resolution, 555
 - Tensor map function, 359
 - TensorFlow / tf.data Integration, 359
 - Terminal-first discoverability, 285
 - The analysis file as a new kind of score, 29
 - The hidden history of computation time, 26
 - The historical thread, 4
 - Time map, 555
 - time scale and pitch core, 476
 - Time Stretch and Pitch Ratio, 71
 - time stretching, 3
 - Time-Compress Speech (Faster Playback), 294
 - Time-frequency tradeoff, 555
 - Time-scale modification, 555
 - Time-stretch - denoise - dereverb in one pipe, 383
 - Time-Varying Parameter Control (CSV/JSON), 50
 - Time-Varying Spectral Filtering with pvx tvfilter, 320
 - Timeline Warp with pvxwarp, 301
 - TimeStretch and PitchShift Engine Selection, 361
 - Transfer functions, 525
 - Transfer-Function and Automation Graph Atlas, 525
 - Transform A/B FFT vs DCT, 306
 - Transform Availability and Selection (pvx transforms), 324
 - Transform Backend Decision Path, 479
 - Transform Backend Families and Tradeoffs, 69
 - Transform Research A/B (FFT vs Hartley), 313
 - Transform selection, 384
 - Transient, 555
 - Transient and rhythmic stress tests, 508
 - Transient Feature Stack, 478
 - Transient preservation, 555
 - transient preservation, 7
 - Transient Reset Mode for Cleaner Attacks, 304
 - transientness - transient-safe stretch variation, 329
 - Transients become a separate research object, 21
 - Transients expose the limits of stationarity, 12
 - Transients Stereo and Layer Split, 341
 - Trevor Wishart and sound morphing, 20
 - triangular, 129
 - Troubleshooting, 395
 - Troubleshooting Decision Tree, 480
 - True envelope, 555
 - tukey, 137
 - tukey 0p1, 141
 - tukey 0p25, 145
 - tukey 0p75, 149
 - tukey 0p9, 153
 - UC San Diego CARL and a software culture, 17
 - UCSD CARL and the command-line studio in detail, 33

- Ultra-smooth speech stretch (600x), 383
- Uninstalling, 394
- Unique Long Flags, 399
- Unvoiced, 555
- Unwrap, 555
- Upgrading, 394
- Use-Case Index (By Theme), 293
- Use-Case Matrix (Where to start quickly), 52

- Validation targets, 484
- VBAP adaptive panning via algorithm dispatcher, 384
- Verify on Your Machine, 66
- Vibrato, 555
- Visual Mental Model of STFT, 52
- Vocal Retune by Scale/Root (pvxretune), 308
- Vocoder, 555
- vocoder, 4
- Voice text and formants, 500
- Voiced, 555
- voicing prob - stretch, 328

- W0, 77
- W1, 77
- W10, 78
- W11, 78
- W12, 79
- W13, 79
- W14, 79
- W15, 79
- W16, 79
- W17, 79
- W18, 79
- W2, 77
- W3, 77
- W4, 78
- W5, 78
- W6, 78
- W7, 78
- W8, 78
- W9, 78
- welch, 165
- Wet signal, 555
- Wet-dry mix, 555
- What Is a Phase Vocoder, 3
- What Outputs Should Sound Like, 54
- What Problem pvx Solves, 48
- What PVC Got Right, 285
- What the Formula Installs, 394
- What the historical work leaves us, 42
- What the transform sees, 42
- When to Keep Manual Pipes, 337

- Where pvx Can Improve Further (PVC-Inspired), 286
- Why This Change, 337
- Why this paper matters, 483
- Window, 555
- window function, 42
- Window length, 555
- Window/Overlap Tradeoff Map, 480
- windows
 - bartlett, 121
 - bartlett hann, 133
 - blackman, 93
 - blackman nuttall, 109
 - blackmanharris, 97
 - bohman, 269
 - boxcar, 125
 - cauchy 0p5, 217
 - cauchy 1p0, 221
 - cauchy 2p0, 225
 - cosine, 273
 - cosine power 2, 229
 - cosine power 3, 233
 - cosine power 4, 237
 - exact blackman, 113
 - exponential 0p25, 205
 - exponential 0p5, 209
 - exponential 1p0, 213
 - flattop, 105
 - gaussian 0p25, 169
 - gaussian 0p35, 173
 - gaussian 0p45, 177
 - gaussian 0p55, 181
 - gaussian 0p65, 185
 - general gaussian 1p5 0p35, 189
 - general gaussian 2p0 0p35, 193
 - general gaussian 3p0 0p35, 197
 - general gaussian 4p0 0p35, 201
 - general hamming 0p50, 253
 - general hamming 0p60, 257
 - general hamming 0p70, 261
 - general hamming 0p80, 265
 - hamming, 89
 - hann, 85
 - hann poisson 0p5, 241
 - hann poisson 1p0, 245
 - hann poisson 2p0, 249
 - kaiser, 277
 - lanczos, 161
 - nuttall, 101
 - parzen, 157
 - rect, 281

- sine, 117
- triangular, 129
- tukey, 137
- tukey 0p1, 141
- tukey 0p25, 145
- tukey 0p75, 149
- tukey 0p9, 153
- welch, 165
- With explicit manifest paths, 369
- WOLA, 555
- WSOLA PSOLA granular methods and productive rivalry, 22
- WSOLA Similarity Search, 478
- WSOLA-Only Transient Handling, 304
- YouTube catalog searches, 495
- Zero padding, 556

